

Oracle® XML Gateway

User's Guide

Release 12.2

Part No. E20929-21

December 2024

Oracle XML Gateway User's Guide, Release 12.2

Part No. E20929-21

Copyright © 2001, 2024, Oracle and/or its affiliates.

Primary Author: Leslie Studdard, Melody Yang

Contributing Author: Srinivasa Rao Atla, Rekha Ayothi, Satya Bandla, Arif Iqbal, Jagadeesh Maddila, Krishnakumar Nair, Claudio Peixoto, Vijay Shanmugam, Veshal Singh, Vebbhav Singh, Shivdas Tomar, Abhishek Verma, Jason Xie

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at

<http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc>.

Access to Oracle Support

Oracle customer access to and use of Oracle support services will be pursuant to the terms and conditions specified in their Oracle order for the applicable services.

Contents

Send Us Your Comments

Preface

1 Oracle XML Gateway Overview

Overview of XML Gateway	1-1
XML Standards.....	1-1
XML Gateway Introduction.....	1-1
XML Gateway Features.....	1-2
XML Gateway Architecture.....	1-3
XML Messages and EDI Transactions.....	1-10
Conclusion.....	1-10

2 Message Designer

Message Designer Overview.....	2-1
Message Designer Menus.....	2-2
Message Designer Toolbar.....	2-4
Message Designer Buttons.....	2-4
File > Properties Menu.....	2-5
Message Designer Wizards.....	2-16
Data Definition Creation Wizard Process Flow.....	2-18
Map Creation Wizard Process Flow.....	2-19
Using the Data Definition Creation Wizard.....	2-20
Data Definition Creation Wizard Steps.....	2-21
Message Designer Data Definition Window.....	2-24
Using the Map Creation Wizard.....	2-25

Transaction Map Window	2-43
Source Definition.....	2-43
Transaction Map - Target Definition.....	2-50
Transaction Map - Level Mapping Tab.....	2-57
Transaction Map - Element Mapping.....	2-62
Transaction Map - Actions.....	2-65
Map Action Editor	2-67
Overview.....	2-68
Map Action Editor - Assignments: Assign Variable Value.....	2-74
Map Action Editor - Assignments: Create Global Variable.....	2-74
Map Action Editor - Database Functions: Assign Next Sequence Value.....	2-76
Map Action Editor - Database Functions: Append Where Clause.....	2-76
Map Action Editor - Database Functions: Insert into Database Table.....	2-79
Map Action Editor - Derivations: Derive Address ID from Location Code.....	2-80
Map Action Editor - Derivations: Derive Parent ID from Location Code.....	2-81
Map Action Editor - Function Call: Execute Function Call.....	2-82
Map Action Editor - Math Functions.....	2-83
OAG Conversions.....	2-84
Map Action Editor - Convert to OAG DATETIME.....	2-85
Map Action Editor - Convert to OAG OPERAMT.....	2-86
Map Action Editor - Convert to OAG QUANTITY.....	2-88
Map Action Editor - Convert to OAG AMOUNT.....	2-89
Map Action Editor - Convert from OAG DATETIME.....	2-91
Map Action Editor - Convert from OAG OPERAMT.....	2-92
Map Action Editor - Convert from OAG QUANTITY.....	2-93
Map Action Editor - Convert from OAG AMOUNT.....	2-94
Map Action Editor - Other: Exit Program.....	2-96
Map Action Editor - Get Predefined Variable Value.....	2-96
Map Action Editor - Procedure Call: Execute Procedure.....	2-100
Map Action Editor - Return Error Message: Send Error Message.....	2-104
Map Action Editor - String Functions: Perform Concatenation.....	2-105
Map Action Editor - String Functions: Perform Substring.....	2-106
Map Action Editor - XSLT Transformation.....	2-107
How to Extend DTDs	2-108
How to Map a Pass-Through Message	2-110
How to Map to an API	2-110
Loading and Deleting Message Maps and DTDs	2-111
Loading/Deleting a Map.....	2-111
Loading and Deleting a DTD.....	2-112
Downloading a Map	2-113
Loading and Deleting an XSLT Style Sheet	2-114

How to Implement Attachments in XML Messages.....	2-115
Attachments and Oracle E-Business Suite.....	2-115
Attachments and OAG Standard.....	2-116
Attachments and Oracle Transport Agent (OTA).....	2-117
Attachments and Outbound Documents.....	2-117
Enable Attachments for Unit Test for Outbound Documents.....	2-118
Attachments and Inbound Documents.....	2-118
Enable Attachments for Unit Test for Inbound Documents.....	2-119

3 XML Gateway Setup

Setup Overview.....	3-1
Define System Profile Options	3-3
Assign XML Gateway Responsibility	3-8
Define Database Directories.....	3-8
Hub Definitions Form	3-8
Define XML Standards Form	3-10
Define Transactions Form	3-11
Define Transactions Form Fields.....	3-12
Transaction Type and Transaction Subtype Naming Conventions.....	3-14
Setting VERB and NOUN in OAG Standards.....	3-17
Define Lookup Values	3-17
Trading Partner Setup	3-19
Required Communications Data	3-30
Static and Dynamic Routing.....	3-31
Trading Partner User Security.....	3-36
Code Conversion.....	3-38
Code Categories.....	3-42
Accessing the Code Conversion Values.....	3-42
Standard Code Conversion Form	3-47
Trading Partner Code Conversion Form.....	3-48

4 Execution Engine

Execution Engine Overview.....	4-1
Protocol Type.....	4-5
XML Gateway Envelope.....	4-8
Trading Partner Validation for Inbound Messages.....	4-12
Validation Against Data in the Trading Partner Setup Form.....	4-14
Trading Partner Validation for Outbound Messages.....	4-16
Dynamic Embedding of XML Fragments.....	4-16
How to Implement the OAG Confirmation Business Object Document.....	4-17

Purpose of the Confirmation Message.....	4-17
Structure of the Confirmation Message.....	4-17
XML Gateway Seeded Confirmation Message Maps.....	4-17
E-Business Suite Seeded Events and Event Subscriptions.....	4-18
How to Implement or Disable a Seeded Confirmation Message.....	4-19

5 Message Queues

Queues	5-1
Outbound Queues.....	5-2
Inbound Queues.....	5-3
XML Gateway Message Format.....	5-4

6 Integrating Oracle XML Gateway with Oracle Workflow Business Event System

Integrating Oracle XML Gateway with Oracle Workflow Business Event System	6-1
Overview.....	6-1
Oracle Workflow Builder - Item Types	6-4
Components of an Item Type.....	6-4
XML Gateway Standard Item Type.....	6-5
XML Gateway Error Processing Item Type.....	6-5
E-Business Suite Application Module-Specific Item Type.....	6-6
XML Gateway Standard Item Type	6-7
Attributes.....	6-7
Processes.....	6-7
Functions.....	6-8
Events.....	6-22
Lookup Types.....	6-24
XML Gateway Error Processing Item Type	6-27
Attributes.....	6-28
Processes.....	6-28
Notifications.....	6-39
Functions.....	6-41
Events.....	6-45
Messages.....	6-46
Lookup Types.....	6-50
Configure Oracle Prebuilt Inbound Messages	6-52
Configure Oracle Prebuilt Outbound Messages	6-54
Application to Application Integration	6-57
Inbound Option.....	6-57
Outbound Option.....	6-58
Manage Workflow Processes	6-59

Register New Business Events and Event Subscription.....	6-59
Identify Seeded Item Types.....	6-59
Identify Seeded Business Events and Associated Event Subscriptions.....	6-59
Configure or Delete Seeded Event Subscriptions.....	6-59
View and Respond to Error Notifications.....	6-59
Purge XML Gateway Transactions.....	6-60
Monitor Workflow Processes.....	6-62
Transaction-Level Trace.....	6-62
Monitor Transaction Status.....	6-62
Review XML Message Returned by Generate Functions.....	6-62
Start Agent Listeners	6-62
Development Guidelines for Custom Messages for B2B Integration.....	6-64
Development Guidelines for Outbound Messages.....	6-64
Development Guidelines for Inbound Messages.....	6-65
Common Questions.....	6-65
Message Delivery Status.....	6-66
How Other Messaging Systems Use XML Message Delivery Callback.....	6-68

7 Oracle Transport Agent

Oracle Transport Agent Overview.....	7-1
OTA Message Propagation Flow.....	7-2
Oracle Transport Agent Post Message.....	7-3
Oracle Transport Agent Response Message.....	7-9
OTA and Attachments.....	7-10
Certificate-Based Authentication Methods.....	7-10
Configuring XML Gateway for TLS Authentication.....	7-12
Configuring TLS Authentication for Inbound Transactions.....	7-13
Configuring TLS Authentication for Outbound Transactions.....	7-14
Setup Parameters.....	7-19
Parameters Set Through AutoConfig.....	7-19
Parameters Not Set Through AutoConfig.....	7-21
Oracle Transport Agent Inbound Test Page.....	7-23
Connecting to Non-OTA Servers.....	7-26
Code Connection Samples.....	7-26
Troubleshooting.....	7-35
HTTP Status Codes.....	7-35

8 Web Services

Overview.....	8-1
Web Services Components and Features.....	8-3

Web Service Security.....	8-4
Understanding XML Gateway Web Services.....	8-5
Enabling XML Gateway Services through Oracle E-Business Suite Integrated SOA Gateway	8-5
Web Service Security.....	8-7
Strongly-typed XML Gateway Services.....	8-11
Generic XML Gateway Services.....	8-15
Process Flows.....	8-18
Inbound Process Flow.....	8-18
Outbound Process Flow.....	8-18
Setup Steps for Invoking XML Gateway Web Services.....	8-19
Implementation Steps.....	8-24
Examples of Using XML Gateway Services.....	8-29
Using XML Gateway Inbound Services.....	8-30
Using XML Gateway Inbound Services at Design Time.....	8-30
Creating a New SOA Composite Application with BPEL Process.....	8-35
Creating a Partner Link.....	8-38
Adding a Partner Link for File Adapter.....	8-39
Adding Invoke Activities.....	8-43
Adding Assign Activities.....	8-44
Deploying and Testing the SOA Composite with BPEL Process at Run Time.....	8-48
Deploying the SOA Composite Application with BPEL Process.....	8-49
Testing the SOA Composite Application with BPEL Process.....	8-52
Prerequisite Files.....	8-59
Using XML Gateway Outbound through Subscription Model.....	8-66
Using XML Gateway Outbound Services at Design Time.....	8-66
Creating a New SOA Composite Application with BPEL Process.....	8-68
Creating a Partner Link for AQ Adapter.....	8-69
Adding a Receive Activity.....	8-76
Adding a Partner Link for File Adapter.....	8-78
Adding an Invoke Activity.....	8-82
Adding an Assign Activity.....	8-83
Deploying and Testing the SOA Composite Application with BPEL Process at Run Time	8-84
Deploying the SOA Composite with BPEL Process.....	8-85
Manually Testing the SOA Composite Application with BPEL Process.....	8-87

9 XML Gateway B2B Transactions Using JMS Queues

Java Messaging Service (JMS) Overview.....	9-1
Oracle XML Gateway and B2B Transactions Integration Points.....	9-2
Oracle XML Gateway and JMS Integration.....	9-3

Integration Features.....	9-3
JMS Inbound and Outbound Messages.....	9-4
Impacts on the Trading Partner Setup and Seed Data.....	9-5
Creating Custom JMS Queues.....	9-6
Steps to Create Custom JMS Queues for Outbound B2B Transactions	9-7
Steps to Create Custom JMS Queues for Inbound B2B Transactions	9-14

10 Collaboration History

Understanding Collaboration History.....	10-2
Using the Collaboration History.....	10-6
Using the Collaboration History Window.....	10-7
Using Collaboration History Simple Search.....	10-7
Using Collaboration History Advanced Search.....	10-8
Viewing the Collaboration History Search Results.....	10-9
Using the Collaboration Events Window.....	10-10
Viewing the Collaboration Event Messages.....	10-11
Viewing the Collaboration Event XML Payload.....	10-11
Using the Collaboration Event Definitions Window.....	10-12
Creating a Collaboration Event Definition.....	10-13
Updating a Collaboration Event Definition.....	10-14
Using the Collaboration Final Event Definitions Window.....	10-15
Creating a Collaboration Final Event Definition.....	10-16
Updating a Collaboration Final Event Definition.....	10-17
Using Collaboration Properties.....	10-18
Creating a Collaboration Property.....	10-19
Updating a Collaboration Property.....	10-19
Using the Collaboration Display Setup.....	10-20
Creating a Collaboration Display Label Header.....	10-21
Updating the Collaboration Display Header and Details.....	10-22
Using the Notification Definition Window.....	10-23
Creating a Notification Definition.....	10-25
Updating a Notification Definition.....	10-25
Using the Notification Action Definitions Window.....	10-26
Creating a Notification Action Header.....	10-28
Updating a Notification Action Header.....	10-29
Creating and Updating Notification Action Details.....	10-29
Viewing Notification Action Definition Details.....	10-31
Viewing Action Definition Details.....	10-32

11 Setting Up Collaborations

Setting Up New Collaborations	11-1
Designing the Collaboration.....	11-2
Creating Message Maps.....	11-3
Setting Up FND Lookup Codes and Messages.....	11-3
Setting Up Collaboration Events and Collaboration Final Events.....	11-5
Adding the Workflow Events.....	11-5
Implementing Notification Processing for New Collaborations.....	11-17
Setting Up Trading Partners and Confirmation Messages.....	11-18
Setting Profile Options.....	11-18
Troubleshooting and Debugging Collaboration History.....	11-18
Setting Up Notification Processing for New Collaborations	11-20
Setting Up Customized Collaborations	11-21
Notify Administrator.....	11-21
Notify Trading Partner.....	11-21
Raise Business Event.....	11-22
Start Workflow.....	11-23
Call API.....	11-24

A Map Analysis Guidelines

Map Analysis Overview	A-1
Map Analysis Guidelines for Outbound Messages	A-1
Map Analysis Guidelines for Outbound Messages Checklist.....	A-1
Compare Database Views (Source) to DTD (Target).....	A-3
Map Analysis Guidelines for Inbound Messages	A-9
Map Analysis Guidelines for Inbound Messages Checklist.....	A-9
Compare Database Views (Source) to DTD (Target).....	A-11
Identifying Source and Target Document Levels	A-17
Collapsing Levels.....	A-17
Expanding Levels.....	A-18
Recommending DTD Additions or Changes to OAG	A-20
Special Considerations for Custom Messages	A-21

B Seeded Code Categories

XML Gateway Seeded Code Categories.....	B-1
---	-----

C Supported Actions

XML Gateway Supported Actions.....	C-1
------------------------------------	-----

D Naming Conventions

XML Gateway Naming Conventions Summary.....	D-1
---	-----

E Timezone Values

XML Gateway Valid Time Zone Values.....	E-1
---	-----

F APIs

Oracle XML Gateway APIs.....	F-1
Execution Engine APIs.....	F-1
APIs Defined in ECX_STANDARD.....	F-1
APIs Defined in ECX_ERRORLOG.....	F-3
Message Designer APIs.....	F-4
APIs Defined in ECX_STANDARD.....	F-6
APIs Defined in ECX_DOCUMENT.....	F-9
APIs Defined in ECX_CONDITIONS.....	F-12
APIs Defined in ECX_TRADING_PARTNER_PVT.....	F-14
APIs Defined in ECX_ERRORLOG.....	F-18
APIs Defined in ECX_ACTIONS.....	F-21
APIs Defined in ECX_ATTACHMENT.....	F-22
APIs Defined in ECX_ENG_UTILS.....	F-27
Trading Partner Setup API.....	F-29

G Configuring OTA Client for TLS Authentication Using Oracle Wallet

Overview	G-1
----------------	-----

H Sizing and Performance Guidelines

Overview.....	H-1
Recommendations for Inbound Messages and Transactions.....	H-1
Recommendations for Outbound Messages and Transactions.....	H-4

I Troubleshooting

Troubleshooting Your XML Gateway Installation.....	I-1
Automated Troubleshooting Script.....	I-1
Transaction Monitor.....	I-8
Transaction Monitor Search Page	I-8
Search Results Page	I-11
Transaction Monitor Details Page.....	I-13
Manual Troubleshooting Steps.....	I-15

XML Gateway Engine-Level Messages.....	I-16
XML Gateway API-Level Messages.....	I-36
XML Gateway Version Validation.....	I-44
Common TLS Issues.....	I-47
Oracle Diagnostic Tests.....	I-47
XML Gateway Tests.....	I-48

J Collaboration Setup Worksheet

Collaboration Setup Worksheet.....	J-1
------------------------------------	-----

Index

Send Us Your Comments

Oracle XML Gateway User's Guide, Release 12.2

Part No. E20929-21

Oracle welcomes customers' comments and suggestions on the quality and usefulness of this document. Your feedback is important, and helps us to best meet your needs as a user of our products. For example:

- Are the implementation steps correct and complete?
- Did you understand the context of the procedures?
- Did you find any errors in the information?
- Does the structure of the information help you with your tasks?
- Do you need different information or graphics? If so, where, and in what format?
- Are the examples correct? Do you need more examples?

If you find any errors or have any other suggestions for improvement, then please tell us your name, the name of the company who has licensed our products, the title and part number of the documentation and the chapter, section, and page number (if available).

Note: Before sending us your comments, you might like to check that you have the latest version of the document and if any concerns are already addressed. To do this, access the new Oracle E-Business Suite Release Online Documentation CD available on My Oracle Support and www.oracle.com. It contains the most current Documentation Library plus all documents revised or released recently.

Send your comments to us using the electronic mail address: appsdoc_us@oracle.com

Please give your name, address, electronic mail address, and telephone number (optional).

If you need assistance with Oracle software, then please contact your support representative or Oracle Support Services.

If you require training or instruction in using Oracle software, then please contact your Oracle local office and inquire about our Oracle University offerings. A list of Oracle offices is available on our Web site at www.oracle.com.

Preface

Intended Audience

Welcome to Release 12.2 of the *Oracle XML Gateway User's Guide*.

This guide assumes you have a working knowledge of the following:

- The principles and customary practices of your business area.
- Computer desktop application usage and terminology.

If you have never used Oracle E-Business Suite, we suggest you attend one or more of the Oracle E-Business Suite training classes available through Oracle University.

See Related Information Sources on page xviii for more Oracle E-Business Suite product information.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc>.

Access to Oracle Support

Oracle customer access to and use of Oracle support services will be pursuant to the terms and conditions specified in their Oracle order for the applicable services.

Structure

- 1 Oracle XML Gateway Overview
- 2 Message Designer
- 3 XML Gateway Setup
- 4 Execution Engine

5	Message Queues
6	Integrating Oracle XML Gateway with Oracle Workflow Business Event System
7	Oracle Transport Agent
8	Web Services
9	XML Gateway B2B Transactions Using JMS Queues
10	Collaboration History
11	Setting Up Collaborations
A	Map Analysis Guidelines
B	Seeded Code Categories
C	Supported Actions
D	Naming Conventions
E	Timezone Values
F	APIs
G	Configuring OTA Client for TLS Authentication Using Oracle Wallet
H	Sizing and Performance Guidelines
I	Troubleshooting
J	Collaboration Setup Worksheet

Related Information Sources

This book is included in the Oracle E-Business Suite Documentation Library. If this guide refers you to other Oracle E-Business Suite documentation, use only the latest Release 12.2 versions of those guides.

Integration Repository

The Oracle Integration Repository is a compilation of information about the service endpoints exposed by the Oracle E-Business Suite of applications. It provides a complete catalog of Oracle E-Business Suite's business service interfaces. The tool lets users easily discover and deploy the appropriate business service interface for integration with any system, application, or business partner.

The Oracle Integration Repository is shipped as part of the Oracle E-Business Suite. As your instance is patched, the repository is automatically updated with content appropriate for the precise revisions of interfaces in your environment.

Online Documentation

All Oracle E-Business Suite documentation is available online (HTML or PDF).

- **Online Help** - Online help patches (HTML) are available on My Oracle Support.
- **Oracle E-Business Suite Documentation Library** - This library, which is included in the Oracle E-Business Suite software distribution, provides PDF documentation as of the time of each release.
- **Oracle E-Business Suite Documentation Web Library** - This library, available on the Oracle Help Center (https://docs.oracle.com/cd/E26401_01/index.htm), provides

the latest updates to Oracle E-Business Suite Release 12.2 documentation. Most documents are available in PDF and HTML formats.

- **Release Notes** - For information about changes in this release, including new features, known issues, and other details, see the release notes for the relevant product, available on My Oracle Support.
- **Oracle Electronic Technical Reference Manual** -The Oracle Electronic Technical Reference Manual (eTRM) contains database diagrams and a detailed description of database tables, forms, reports, and programs for each Oracle E-Business Suite product. This information helps you convert data from your existing applications and integrate Oracle E-Business Suite data with non-Oracle applications, and write custom reports for Oracle E-Business Suite products. The Oracle eTRM is available as an application in Oracle E-Business Suite.

You should have the following related books on hand. Depending on the requirements of your particular installation, you may also need additional manuals or guides.

Related Guides

You may want to refer to other Oracle E-Business Suite implementation documentation when you set up and use Oracle XML Gateway. Additionally, Oracle XML Gateway is used by other Oracle E-Business Suite products. Therefore, if you are using the version of Oracle XML Gateway embedded in Oracle E-Business Suite, you may want to refer to other products' guides to learn more about product specific information.

Oracle Cloud Using the Oracle E-Business Suite Adapter with Oracle Integration 3

This guide describes how to set up and use Oracle E-Business Suite Adapter connections in Oracle Integration to access supported Oracle E-Business Suite interfaces and REST services as inbound or outbound integrations from Oracle E-Business Suite.

Note that this book is the latest generation of Oracle Integration, Oracle Integration 3. Its prior generation, Oracle Integration Generation 2 is called Oracle Cloud Using the Oracle E-Business Suite Adapter with Oracle Integration. Both books are part of the integration documentation in Oracle Cloud Platform as a Service (PaaS) and are available in the Oracle Cloud Library on the Oracle Help Center.

Oracle E-Business Suite Concepts

This book is intended for those planning to deploy Oracle E-Business Suite Release 12.2, or make significant changes to a configuration. After describing the Oracle E-Business Suite architecture and technology stack, it moves on to give an outline of the actions needed to achieve a particular goal, plus any installation and configuration choices.

Oracle E-Business Suite Developer's Guide

This guide contains the coding standards followed by Oracle E-Business Suite Development. It describes the Oracle Application Object Library components needed to implement the Oracle E-Business Suite user interface described in the *Oracle E-Business*

Suite User Interface Standards for Forms-Based Products. It provides information to help you build your custom Oracle Forms Developer forms so that they integrate with Oracle E-Business Suite. In addition, this guide has information for customizations in features such as concurrent programs, flexfields, messages, and logging.

Oracle E-Business Suite Installation Guide: Using Rapid Install

This book describes how to run Rapid Install to perform a fresh installation of Oracle EBusiness Suite Release 12.2 or to replace selected technology stack executables in an existing instance.

Oracle E-Business Suite Maintenance Guide

This book explains how to patch an Oracle E-Business Suite system, describing the adop patching utility and providing guidelines and tips for performing typical patching operations. It also describes maintenance strategies and tools designed to help keep a system running smoothly.

Oracle E-Business Suite Setup Guide

This guide contains information on system configuration tasks that are carried out either after installation or whenever there is a significant change to the system. The activities described include defining concurrent programs and managers, enabling Oracle Applications Manager features, and setting up printers and online help.

Oracle E-Business Suite User's Guide

This guide explains how to navigate, enter and query data, and run concurrent requests using the user interface (UI) of Oracle E-Business Suite. It includes information on setting preferences and customizing the UI. In addition, this guide describes accessibility features and keyboard shortcuts for Oracle E-Business Suite.

Oracle E-Business Suite User Interface Standards for Forms-Based Products

This guide contains the user interface (UI) standards followed by Oracle E-Business Suite Development. It describes the UI for Oracle E-Business Suite products based on Oracle Forms, and how to apply this UI to the design of such applications.

Oracle E-Business Suite Security Guide

This guide contains information on a comprehensive range of security-related topics, including access control, user management, function security, data security, secure configuration, and auditing. It also describes how Oracle E-Business Suite can be integrated into a single sign-on environment.

Oracle E-Business Suite Integrated SOA Gateway User's Guide

This guide describes the high level service enablement process, explaining how users can browse and view the integration interface definitions and services residing in Oracle Integration Repository.

Oracle E-Business Suite Integrated SOA Gateway Implementation Guide

This guide explains how integration administrators can manage and administer the web service activities for integration interfaces including native packaged integration

interfaces, composite services (BPEL type), and custom integration interfaces. It also describes how to set up and implement Service Invocation Framework to invoke SOAP and REST services from Oracle E-Business Suite, and how to manage web service security, configure logs, and monitor both inbound service invocations using Service Monitor and outbound service invocations through Service Invocation Framework using Service Invocation Monitor.

Oracle E-Business Suite Integrated SOA Gateway Developer's Guide

This guide describes how integration developers can perform end-to-end service integration activities. These include orchestrating discrete web services into meaningful end-to-end business processes using business process execution language (BPEL), and deploying BPEL processes at runtime.

This guide also explains how to invoke web services using the Service Invocation Framework. This includes defining web service invocation metadata, invoking web services, and testing web service invocation.

Oracle E-Business Suite Multiple Organizations Implementation Guide

This guide describes the multiple organizations concepts in Oracle E-Business Suite. It describes in detail on setting up and working effectively with multiple organizations in Oracle E-Business Suite.

Oracle e-Commerce Gateway User's Guide

This guide describes the functionality of Oracle e-Commerce Gateway and the necessary setup steps in order for Oracle E-Business Suite to conduct business with trading partners through Electronic Data Interchange (EDI). It also describes how to run extract programs for outbound transactions, import programs for inbound transactions, and the relevant reports.

Oracle e-Commerce Gateway Implementation Guide

This guide describes implementation details, highlighting additional setup steps needed for trading partners, code conversion, and Oracle E-Business Suite. It also provides architecture guidelines for transaction interface files, troubleshooting information, and a description of how to customize EDI transactions.

Oracle Workflow Administrator's Guide

This guide explains how to complete the setup steps necessary for any product that includes workflow-enabled processes. It also describes how to manage workflow processes and business events using Oracle Applications Manager, how to monitor the progress of runtime workflow processes, and how to administer notifications sent to workflow users.

Oracle Workflow Developer's Guide

This guide explains how to define new workflow business processes and customize existing Oracle E-Business Suite-embedded workflow processes. It also describes how to configure message metadata for Oracle Mobile Approvals for Oracle E-Business Suite and how to define and customize business events and event subscriptions.

Oracle Workflow User's Guide

This guide describes how users can view and respond to workflow notifications and monitor the progress of their workflow processes.

Oracle Workflow API Reference

This guide describes the APIs provided for developers and administrators to access Oracle Workflow.

Oracle Workflow Client Installation Guide

This guide describes how to install the Oracle Workflow Builder and Oracle XML Gateway Message Designer client components for Oracle E-Business Suite.

Do Not Use Database Tools to Modify Oracle E-Business Suite Data

Oracle STRONGLY RECOMMENDS that you never use SQL*Plus, Oracle Data Browser, database triggers, or any other tool to modify Oracle E-Business Suite data unless otherwise instructed.

Oracle provides powerful tools you can use to create, store, change, retrieve, and maintain information in an Oracle database. But if you use Oracle tools such as SQL*Plus to modify Oracle E-Business Suite data, you risk destroying the integrity of your data and you lose the ability to audit changes to your data.

Because Oracle E-Business Suite tables are interrelated, any change you make using an Oracle E-Business Suite form can update many tables at once. But when you modify Oracle E-Business Suite data using anything other than Oracle E-Business Suite, you may change a row in one table without making corresponding changes in related tables. If your tables get out of synchronization with each other, you risk retrieving erroneous information and you risk unpredictable results throughout Oracle E-Business Suite.

When you use Oracle E-Business Suite to modify your data, Oracle E-Business Suite automatically checks that your changes are valid. Oracle E-Business Suite also keeps track of who changes information. If you enter information into database tables using database tools, you may store invalid information. You also lose the ability to track who has changed your information because SQL*Plus and other database tools do not keep a record of changes.

Oracle XML Gateway Overview

This chapter covers the following topics:

- Overview of XML Gateway

Overview of XML Gateway

XML Standards

Many standards bodies (for example, OAG, Rosettanet, SOAP, and iFX) exist with published Document Type Definitions (DTDs). Some standards are strong at managing the message content while others excel at managing both the message content and its related processes.

As a provider of broad-based business application solutions to support all industries, Oracle XML Gateway supports all DTD based XML standards. The majority of messages delivered with the Oracle E-Business Suite are mapped using the Open Application Group (OAG) standard. Any prebuilt message can be remapped to any standard of choice using the XML Gateway Message Designer.

XML Gateway Introduction

Oracle XML Gateway is a set of services that allows easy integration with the Oracle E-Business Suite to support XML messaging.

The Oracle E-Business Suite utilizes the Oracle Workflow Business Event System to support event-based XML message creation and consumption.

Oracle XML Gateway consumes events raised by the Oracle E-Business Suite and subscribes to inbound events for processing. Oracle XML Gateway uses the message propagation feature of Oracle Advanced Queuing to integrate with the Oracle Transport Agent to deliver messages to and receive messages from business partners.

Oracle XML Gateway supports both Business-to-Business (B2B) and Application-to-Application (A2A) initiatives. B2B initiatives include communicating business

documents and participating in industry exchanges. An example of an A2A initiative is data integration with legacy and disparate systems.

With Oracle XML Gateway services, you are assured consistent XML message implementation when integrating with the Oracle E-Business Suite, thereby lowering integration costs and expediting message implementation while supporting corporate e-business initiatives.

XML Gateway Features

The Oracle XML Gateway offers the following features and integrations:

- **Message Designer**
Use this wizard-guided tool to define message maps containing the RDBMS-to-XML or XML-to-XML data transformations.
- **Robust Execution Engine**
The execution engine integrates with the Oracle Workflow Business Event System for event-based message creation and consumption.
- **Flexible Trading Partner Directory Service**
Use this service to define a hub, all trading partners exchanging on a hub, or a specific business partner. The definition includes information on enabled messages, confirmation requests, and message maps and message transport protocols.
- **Flexible message setup**
Flexible conversion of Oracle codes to recipient or standards-based codes, including a cross-reference between the Oracle internal and external transaction names.
- **Integration with Oracle Workflow Error Handling**
Oracle Workflow error handling process provides active error notification to the XML Gateway system administrator or Trading Partner contact with support for "retry" and "reprocess" for failed processes.
- **Integration with Oracle Advanced Queuing**
Oracle AQ provides persistent storage and message propagation.
- **Utilizes Servlet-Based Transport Agent**
The servlet-based Transport Agent delivers and receives XML messages using SMTP, HTTP, or HTTPS protocols.
- **Support for Web Services**
Any transactions supported by Oracle XML Gateway can be sent or received as a document style Web service using the Simple Object Access Protocol (SOAP).

- **Support for Java Messaging Service (JMS)**

To provide complete support for Business-to-Business transactions, Oracle XML Gateway leverages Oracle Workflow Business Event System to enable the exchange of JMS messages between the Oracle E-Business Suite and Trading Partners.

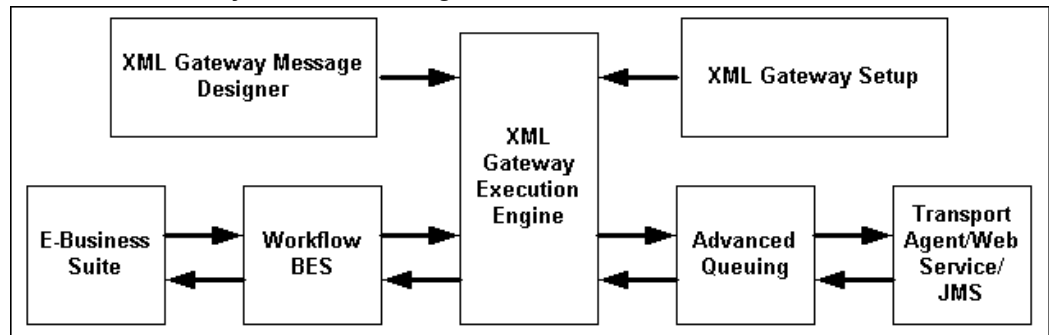
- **Collaboration History**

Use the Collaboration History feature to track and monitor all collaboration events and messages exchanged within Oracle E-Business Suite and with trading partners. Additionally, predefined notification actions can be taken in response to the needs of business transactions. For example, notification messages or business events can be raised and routed to appropriate people if an order is cancelled or created.

XML Gateway Architecture

The following diagram shows an overview of the Oracle XML Gateway architecture:

Oracle XML Gateway Architecture Diagram

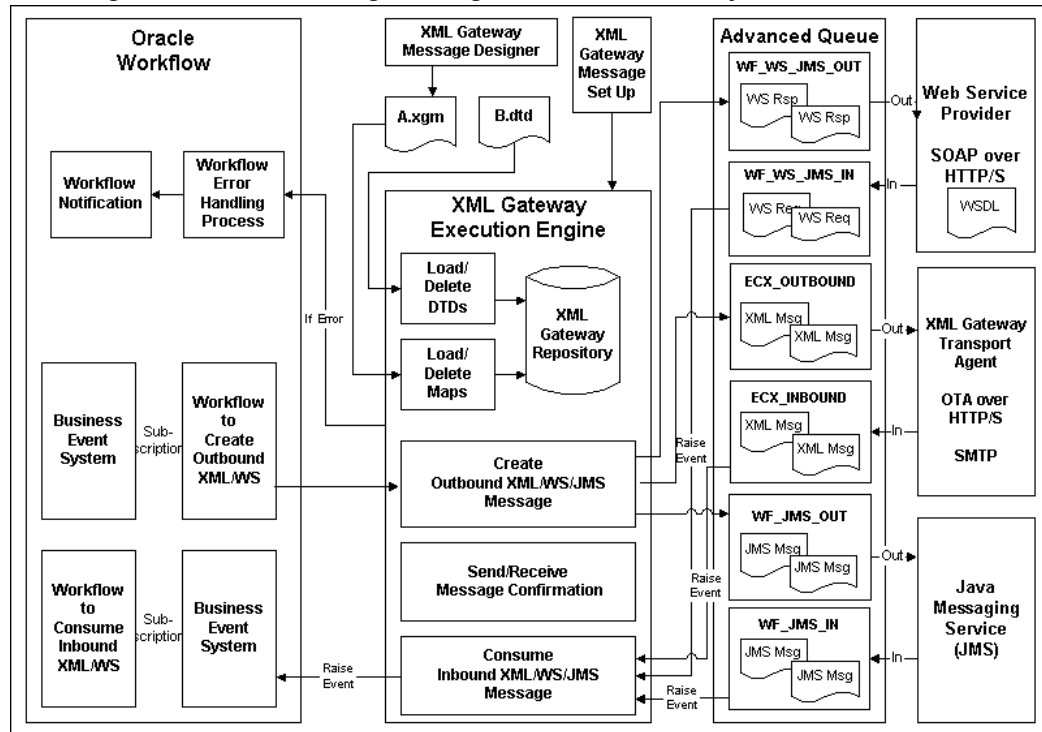


The services supported by Oracle XML Gateway are grouped into four functional areas as follows:

- Message Designer
- XML Gateway Setup
- Execution Engine
- Transport Agent

The following figure shows the detailed flow of an XML message through Oracle XML Gateway:

Flow Diagram of an XML Message Through Oracle XML Gateway



Message Designer

The XML Gateway Message Designer is a wizard-guided, repository-based tool used to define message maps. A message map represents the relationship between the source and target data elements.

Use the XML Gateway Message Designer to perform the following functions:

Define Data Source and Data Target

Each message map consists of a data source and data target representing where the data is coming from and where it is mapped to. The XML Gateway Message Designer supports the following combinations of source and target data definitions:

- Source RDBMS to Target XML
- Source XML to Target RDBMS
- Source XML to Target XML

RDBMS-based data definitions can be based on database tables, database views, Application Open Interface tables, or Application APIs.

XML-based data definitions can be based on an XML Document Type Definition or a production XML message. Using a production XML message as a data source or data

target is the recommended choice if you are migrating from an existing implementation or legacy system (where XML messaging is supported) to the Oracle E-Business Suite.

Map Source Data Structure to Target Data Structure

Once the data source and data target are defined, use the Message Designer to relate the source data structure to the target data structure.

This process is especially important if the data structure of the business document is different from the data structure of the application data model. For example, if an inbound business document is represented in three levels but the application data model is represented in two levels, the data in the business document must be collapsed to accommodate the application data model. The incoming data is expanded if the opposite case occurs in which the inbound business document is represented in two levels but the application data model is represented in three levels.

The XML Gateway Message Designer supports both expanding and collapsing hierarchies to ensure that data can be retrieved from or populated into the Oracle E-Business Suite data model.

Map Source Data Element to Target Data Element

Once the data source, data target, and source-to-target hierarchy are defined, use the Message Designer to map the source data elements to the target data elements.

The Message Designer user interface displays the data source and the data target in adjacent windows. A simple drag and drop between the source and target data elements creates a map relationship. The source data element name is noted next to the target data element name to identify the map relationship.

Identify Data Transformation and Process Control Functions

As part of the element mapping process, use the Message Designer to identify data transformation and process control functions. These functions can be defined at the source or target as follows:

- To be applied at the data element, document, or root level
- To be applied before, during, or after the message is created or consumed

The function may be qualified by a condition. If no condition is defined, the function will always be applied. See XML Gateway Supported Actions, page C-1 for a list of the functions supported by the XML Gateway.

The common data transformation functions involve math functions and string manipulation in addition to conversions between the Oracle and OAG formats representing date, operating amount, quantity, and amount values.

The common process control functions involve calling procedures or database functions to extend the integration with the Oracle E-Business Suite. Other common process control functions allow you to inquire on the status of a transaction and manage the process flow based on the status. For serious errors, the process may be aborted with

error notifications sent to the XML Gateway system administrator or Trading Partner contact.

Once the message map is created, it is loaded along with its associated DTDs into the XML Gateway repository for use by the Execution Engine to create outbound or to consume inbound XML messages.

XML Gateway Setup

To implement a message with a trading partner, use the XML Gateway setup features to define the Trading Partner or hub, code conversion values, and internal-to-external transaction name cross-references. In addition, you can identify the XML Gateway system administrator to notify for system or process errors.

Define Trading Partner or Hub

E-Business may be conducted directly with a business partner, commonly known as a trading partner, or via a hub, such as Oracle Exchange, where many buyers and sellers converge to conduct electronic commerce.

With Oracle XML Gateway, you can define the hub or the individual business partner as a trading partner. If you define the hub as the trading partner, you can identify all the buyers and sellers who are conducting business on the hub as Trading Partners to the hub.

To support multiple organizations, XML Gateway allows you to set up Trading Partners or query data for more than one operating unit associated with your responsibility. During the Trading Partner setup, the selections of Trading Partner types, Trading Partners, and Trading Partner sites are all restricted to the associated operating unit which is controlled by the security profile linked to your logon responsibility.

Included in the Trading Partner or hub definition are the following:

- Define Trading Partner or Hub name
- Enable XML messages for the partner
- Enable request for message confirmation
- Define the message map to use for message creation or consumption
- Define the e-mail address of the trading partner contact to notify for data errors
- Define trading partner-specific code conversion values
- Define transport protocol: SMTP, HTTP, HTTPS, JMS with credential and username and password as necessary

Define Code Conversion

With Oracle XML Gateway, you can cross-reference Oracle codes to codes that are meaningful to your recipient. Conversely, you can cross-reference codes you receive from your partner to codes that are meaningful to your application. Common examples of Oracle E-Business Suite codes requiring code conversion are units of measure and currency code.

Code conversion values can be defined to be applied universally for all Trading Partners and all messages. Additionally, code conversion values can be defined for a specific XML standard or specific to a Trading Partner.

Define Transactions

Use Oracle XML Gateway to define a cross-reference between the Oracle transaction name and the external transaction name. The external transaction name will be based on what is meaningful per the XML standard used by the recipient. The external transaction name will appear on the message envelope to support message transport.

Execution Engine

The XML Gateway Execution Engine is responsible for interacting with several Oracle technologies to process and transport XML messages to and from Trading Partners for B2B integration, or other information systems both within and outside the enterprise for A2A integration.

The Oracle technologies involved include the following:

- Oracle Workflow Business Event System
- Oracle E-Business Suite
- Oracle Advanced Queuing
- Oracle Workflow

Oracle Workflow Business Event System

Oracle XML Gateway leverages the Oracle Workflow Business Event System to publish and subscribe to application business events of interest to automatically trigger message creation or consumption.

Business events and event subscriptions to send outbound messages or to consume inbound messages are delivered for all Oracle prebuilt messages. The seeded event subscriptions can be configured during implementation to perform activities to address specific business requirements.

Oracle E-Business Suite

The XML Gateway Execution Engine interfaces with the Oracle E-Business Suite via business events and event subscriptions to retrieve data from or populate data into the

Oracle E-Business Suite tables.

Oracle Advanced Queuing

The XML Gateway Execution Engine interfaces with Oracle Advanced Queuing to stage outbound XML messages or receive inbound XML messages for processing.

Oracle Workflow

The XML Gateway Execution Engine interfaces with Oracle Workflow to actively notify the XML Gateway system administrator regarding system or process errors, or the Trading Partner contact for data errors.

The XML Gateway system administrator has the option to "retry" failed outbound processes, or "reprocess" failed inbound processes.

XML Gateway Execution Engine Internal Functions:

XML Gateway Execution Engine has the following internal functions:

Maintain Maps Cache if Processing in Java

When the execution engine receives payload for processing inbound messages in Java, the engine will check if the map code associated with the trading partner and transaction was loaded.

- If the map code was already loaded, the engine checks if map was already loaded in cache and then uses already loaded metadata for further processing of the XML messages.
- If the map code was not loaded, then load the map metadata from database and store into memory.

Validate Trading Partner or Hub

Verify that the Trading Partner is defined and the required document is enabled.

Retrieve Message Map from Repository

Retrieve the message map associated with the Trading Partner and required document.

Retrieve and Populate Data to and from the Oracle E-Business Suite

For outbound messages, gather the application data from the Oracle E-Business Suite using the information on the message map.

For inbound messages, process the data into the Oracle E-Business Suite using the information on the message map.

Apply Data Transformations or Process Control Functions

Apply code conversions, data transformations, and process control functions defined on the message map.

Validate Message Using the XML Parser

For outbound message, the XML Parser is used to validate the newly created message to ensure that it is well-formed and valid. Poorly formed or invalid messages will not be enqueued for delivery. An error notification is sent to the XML Gateway system

administrator.

Inbound messages dequeued from the inbound queue are validated using the XML Parser to ensure that it is well-formed and valid. Poorly formed or invalid messages will not be processed by the XML Gateway Execution Engine. An error notification is sent to the Trading Partner contact.

Enqueue and Dequeue XML Messages

Outbound messages are enqueued for delivery by the transport agent. Inbound messages are dequeued for processing by the XML Gateway Execution Engine.

Send and Receive Message Confirmations

An outbound confirmation is sent in response to an inbound request if the outbound confirmation is enabled for the Trading Partner.

An inbound confirmation is received in response to an outbound request if the inbound confirmation is enabled for the Trading Partner.

Direct Interaction with Execution Engine for A2A Integration

Seeded Workflow functions are provided for use in Workflow processes to interact directly with the Execution Engine to generate outbound or to consume inbound messages.

The outbound message generated by the XML Gateway Execution Engine is made available to the downstream Workflow activity for processing. The Execution Engine consumes the inbound message passed to it by a Workflow process.

The XML Gateway Execution Engine begins processing once it detects the Workflow process to send or generate an XML message or that an inbound message has arrived on the inbound queue.

Oracle Transport Agent

The Oracle Transport Agent interfaces with Oracle Advanced Queuing to deliver outbound messages and to receive inbound messages.

The Transport Agent server is a Java-based servlet that uses the Transport Agent Messaging Protocol to support the following:

- Guaranteed delivery, exactly once
- Message encryption
- Server-to-server certificate authentication

Web Services

Oracle XML Gateway uses Web Services Description Language (WSDL) to inform trading partners how to communicate with the Oracle E-Business Suite. The Suite also publishes the WSDL to a URL for customers to access. Partners can use any third party Web service tools to call for Web services.

All inbound Web service messages are received through the Simple Object Accessed Protocol (SOAP) servlet running under the service provider (Web Service Provider and SOA Provider). These messages are prepared as designed events before being enqueued for further processing.

JMS Queues

Java Messaging Service (JMS) is a message standard. To send JMS messages between the Oracle E-Business Suite and Trading Partners, the appropriate Protocol Type (JMS) and Protocol Address registered with the Business Event System must be identified first in the Trading Partner Setup form.

JMS providers can integrate with XML Gateway for B2B transactions using WF_JMS_IN, WF_JMS_OUT, or their own JMS queues.

For inbound transactions, this integration provides complete validation and authorization support so that XML Gateway processes only those messages that are valid and have the appropriate authorization.

For outbound transactions, this integration provides a mechanism to store the JMS queues as part of the Trading Partner setup so that the generated messages can be sent to the desired JMS queue.

XML Messages and EDI Transactions

EDI transactions and XML messages are two forms of electronic messaging based on their respective standards. EDI transactions are more batch-oriented while their XML counterpart is event-based, real-time, and tend to be based on a single transaction.

Oracle e-Commerce Gateway integrates with the Oracle E-Business Suite to extract or to import traditional EDI transactions using a flat ASCII file. A third party EDI Translator is required to map the data between the Oracle E-Business Suite and the EDI standard of choice such as ASC X12 or EDIFACT.

Oracle XML Gateway integrates with the Oracle E-Business Suite to create or to consume XML messages based on application business events. Oracle XML Gateway creates and consumes standards-compliant XML messages without the use of a translator.

Conclusion

Oracle XML Gateway is the XML message enabler for the Oracle E-Business Suite. It provides consistent XML message implementation within and outside the enterprise in support of A2A and B2B messaging initiatives.

Oracle XML Gateway leverages the publish and subscribe features of the Oracle Workflow Business Event System to automate message creation and consumption in addition to using Oracle Workflow to link key business processes. This combination enables seamless collaboration, coordination, and communication of business-critical data throughout the supply chain.

Message Designer

This chapter covers the following topics:

- Message Designer Overview
- Message Designer Wizards
- Using the Data Definition Creation Wizard
- Using the Map Creation Wizard
- Transaction Map Window
- Map Action Editor
- How to Extend DTDs
- How to Map a Pass-Through Message
- How to Map to an API
- Loading and Deleting Message Maps and DTDs
- Downloading a Map
- Loading and Deleting an XSLT Style Sheet
- How to Implement Attachments in XML Messages

Message Designer Overview

The Message Designer is a wizard-guided, repository-based tool used to create XML message maps. Map creation consists of defining the data source and data target, defining hierarchy and data maps between the source and target data, and defining actions for data transformation and process control.

The Message Designer is independent of XML standards. It complies with version 1.0 of the W3C XML specifications.

Note: Information regarding the W3C XML Standards can be found at

<http://www.w3.org/XML/Activity>.

The Message Designer can support map creation for any business document as long as the document conforms to a Document Type Definition (DTD).

The Message Designer can be used to develop custom message maps through the following ways:

- Modify the Oracle prebuilt message maps
- Create new message maps

If you are using the Message Designer to create new message maps, you must complete a map analysis before attempting to use the Message Designer. Refer to Map Analysis Guidelines, page A-1 for the details. The thoroughness of your map analysis will impact the success of your map creation.

Load the completed message maps and associated DTDs into the XML Gateway repository. The XML Gateway Execution Engine and the XML Parser use the maps to create outbound messages and to consume inbound XML messages.

For more information on how to develop and deploy Oracle XML Gateway customizations, refer to the 'Component-Specific Steps for Application Tier Objects' section, *Deploying Customizations in Oracle E-Business Suite Release 12.2*, My Oracle Support Knowledge Document 1577661.1.

Message Designer Menus

The Message Designer has the following menus: File, View, and Help. Some of the menu functions can also be accessed by toolbar, page 2-4 icons.

See: Message Designer Toolbar, page 2-4.

File Menu

Use the Message Designer to create new data definitions and transaction maps or to modify existing data definitions and transaction maps.

The Message Designer supports the following file types:

- XGD for source or target data definitions
- XGM for transaction maps

The .XGD and .XGM files are XML files that can be opened and read using a browser or any XML editor. The XGD and XGM files are used to describe the structure and content of a message. The XML message produced or consumed using the transaction map (XGM) contains the actual business data.

The File menu options are listed in the following table:

Menu Option	Description
New	Create a new data definition file or a new transaction map file.
Open	Open an existing file. Files can be either data definition files with a XGD extension, or transaction map files with a XGM extension.
Close	Close the open transaction map or data definition file.
Save	Save an open file. Save the data definition files as XGD files. Save the transaction map files as XGM files.
Properties	Provides access to key property values entered using the Data Definition Creation Wizard or Map Creation Wizard. Use this menu option to change any of the original key values presented. See File > Properties Menu Option, page 2-5 for more details regarding this menu option.
Exit	Exit Message Designer.

View Menu

Use the View Menu to view the source or target definitions in tree format only, table format only, or both tree and table formats.

The View menu options are listed in the following table:

Menu Option	Description
Tree	View source or target definition in tree format only.
Table	View source or target definition in table format only.
View Both	View source or target definition in both tree and table formats. This is the default viewing option.

Help Menu

The Help menu options are listed in the following table:

Menu Option	Description
Help Topics	Displays Message Designer help.
About	Displays the Message Designer version.

Message Designer Toolbar

The Message Designer toolbar uses the following icons to duplicate the noted menu options:

Message Designer Toolbar Icons



1. **Create New Map** icon. Invokes the Map Creation Wizard.
2. **Open Map** icon. Opens an existing map.
3. **Save Map** Icon. Saves the data definition as a XGD file. Saves the transaction map as a XGM file.
4. **View Tree** icon. Displays the tree format only.
5. **View Table** icon. Displays the table format only.
6. **View Both Tree and Table icon.** Displays both the tree and table formats.
7. **Help** icon. Invokes Message Designer Help.

Message Designer Buttons



The **Add Sibling Button** adds a new element at the same hierarchy level as the selected item on the map.

For additional information on this function, see the following:

For Source Data Definitions see (Source Definition Tab) Add Sibling, page 2-48.

For Target Data Definitions see (Target Definition Tab) Add Sibling, page 2-55.

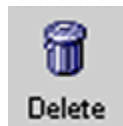


The **Add Child** button adds a new element at a lower hierarchy level than the selected item on the map.

For additional information on this function, see the following:

For Source Data Definitions see (Source Definition Tab) Add Child, page 2-48.

For Target Data Definitions see (Target Definition Tab) Add Child, page 2-56.



The **Delete** button deletes any element that has not been mapped. Be careful not to delete required data elements. Deleting required elements will cause a parser violation.

If an item has child elements associated with it, a warning is displayed before the delete occurs.

Important: If you are deleting any DTD extensions, be sure to remove the DTD extensions from the corresponding extension file created for the application or user. The extra information will not cause a parser violation, but it is a good practice to ensure the extension files match the message maps. Refer to How to Extend DTDs, page 2-108 for the details.

File > Properties Menu

The File > Properties menu option provides access to key property values entered using the Data Definition Creation Wizard or the Map Creation Wizard. Use the Properties window to change any of the key property values presented.

The Property Tabs and fields associated with each Tab vary depending on where the File > Properties menu option is invoked. The options available from each window are:

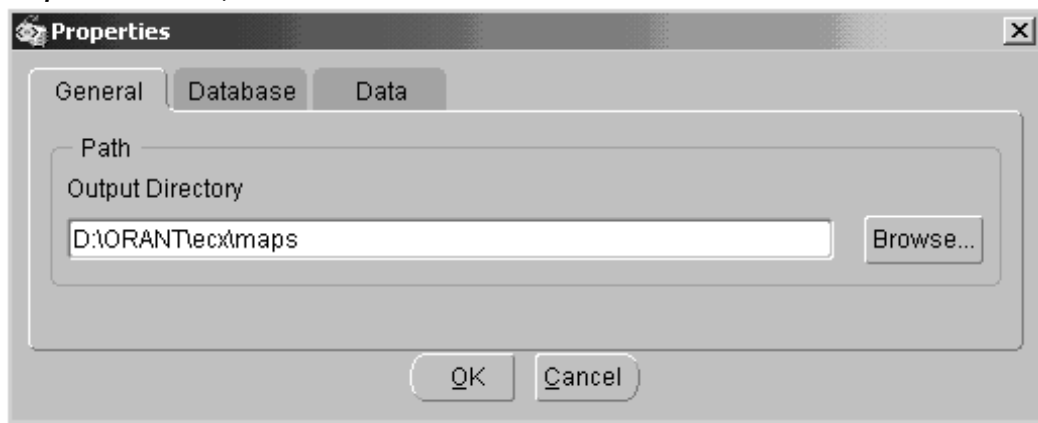
- Main Message Designer - General Tab, page 2-6, Database Tab, page 2-6
- Data Definition Window - General Tab, page 2-6, Database Tab, page 2-6, Data Tab, page 2-9

- Transaction Map Window- General Tab, page 2-6, Database Tab, page 2-6, Map Tab, page 2-11, Source Tab, page 2-13, Target Tab, page 2-15

General Tab

The General Tab allows you to update the Output Directory. This is the default directory used to store the data definition and message map files created using the Message Designer.

Properties Window, General Tab



Output Directory

Use the Browse button to select a default directory or enter a valid directory name.

Database Tab

Use the Database Tab to provide the default database connection information. The default values will be provided to the Data Definition and Map Creation Wizards as well as to the Procedure Call action.

The database connection fields prompted for are:

- Username
- Connect to Database
- Host
- Port
- Schema Name

Properties Window, Database Tab

The screenshot shows a 'Properties' dialog box with three tabs: 'General', 'Database', and 'Data'. The 'Database' tab is active. Inside this tab, there is a 'Database' section containing five text input fields: 'Username' with the value 'apps', 'Connect to Database' with 'dom1234', 'Host' with 'ap111dbs', 'Port' with '1521', and 'Schema Name' with 'APPS'. At the bottom of the dialog are 'OK' and 'Cancel' buttons.

If your Oracle E-Business Suite is on Oracle Database 23ai or Oracle Database 19c, perform the following steps to connect to Oracle Database 23ai or Oracle Database 19c:

1. Ensure that Java 1.5 or later is available in the local machine where Message Designer is installed.
2. Open Message Designer. Navigate to File > Properties. Copy the Output Directory value from the General tab.
3. Navigate to the Output Directory location.
4. Download `ojdbc8.jar` from https://download.oracle.com/otn-pub/otn_software/jdbc/1917/ojdbc8.jar. Copy it to the `.\jar` folder.
5. Take a backup of `MessageDesigner.bat` located in the Output Directory.
6. Edit `MessageDesigner.bat` as explained in the following steps. A sample file shown in the table below depicts the edit details.
 1. Replace the first line `:start java -noverify -cp ""` with `'start "title" javaw -mx50m -noverify -cp "<absolute path of ojdbc5.jar>;'`.
 2. Comment out the second line by adding `:'` before `'start'`.
 3. Save the changes.

The following table represents the before and after edits for a sample file

MessageDesigner.bat:

Sample File MessageDesigner.bat

Before the Edits

```
@echo off

:start java -noverify -cp "C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jre\lib\rt.jar;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jre\lib\i18n.jar;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\classes12.zip;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\classes111.zip;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\ewt-opt-3_3_12.zip;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\ewt-swingaccess-3_3_12.jar;C:\Oracle\emagent\core\12.
1.0.5.0\ecx\jar\share-opt-1_0_8.zip;C:\Oracle\emagent\core\12.
1.0.5.0\ecx\jar\xmlparserv2.jar;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\ecx_md.jar" oracle.apps.ecx.mapper.ui.MessageDesigner

start "title" "C:\Oracle\emagent\core\12.1.0.5.0\ecx\jre\bin\jrew"
-mx50m -noverify -cp "C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jre\lib\rt.jar;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jre\lib\i18n.jar;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\classes12.zip;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\classes111.zip;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\ewt-opt-3_3_12.zip;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\ewt-swingaccess-3_3_12.jar;C:\Oracle\emagent\core\12.
1.0.5.0\ecx\jar\share-opt-1_0_8.zip;C:\Oracle\emagent\core\12.
1.0.5.0\ecx\jar\xmlparserv2.jar;C:\Oracle\emagent\core\12.1.0.5.0
\ecx\jar\ecx_md.jar" oracle.apps.ecx.mapper.ui.MessageDesigner

@echo off
set JAR_ROOT=
```

Sample File MessageDesigner.bat

After the Edits

```
@echo off
```

```
start "title" javaw -mx50m -noverify -cp "C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\ojdbc5.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jre\lib\rt.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jre\lib\i18n.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\ewt-opt-3_3_12.zip;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\ewt-swingaccess-3_3_12.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\share-opt-1_0_8.zip;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\xmlparserv2.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\ecx_md.jar" oracle.apps.ecx.mapper.ui.MessageDesigner
```

```
:start "title" "C:\Oracle\emagent\core\12.1.0.5.0\ecx\jre\bin\jre" -mx50m -noverify -cp "C:\Oracle\emagent\core\12.1.0.5.0\ecx\jre\lib\rt.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jre\lib\i18n.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\ojdbc8.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\ewt-opt-3_3_12.zip;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\ewt-swingaccess-3_3_12.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\share-opt-1_0_8.zip;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\xmlparserv2.jar;C:\Oracle\emagent\core\12.1.0.5.0\ecx\jar\ecx_md.jar" oracle.apps.ecx.mapper.ui.MessageDesigner
```

```
@echo off
```

```
set JAR_ROOT=
```

7. Restart the Message Designer to connect to Oracle Database 23ai or Oracle Database 19c.

Data Tab

The Data Tab allows you to update data values originally entered using the Data Definition Creation Wizard. The fields on the tab vary depending on the type of data definition.

Properties Window, Data Tab

The screenshot shows a 'Properties' dialog box with three tabs: 'General', 'Database', and 'Data'. The 'Data' tab is active. Within this tab, there is a 'Data' section containing two text input fields. The first field, labeled 'Data Definition Name', contains the text 'AR_INVO_OAG70_XML'. The second field, labeled 'Data Definition Description', contains the text 'Outbound Invoice, OAG 7.0 standard, XML definition type'. At the bottom of the dialog are 'OK' and 'Cancel' buttons.

The fields in the following table display for all data definitions:

Field	Description
Data Definition Name	Update the name entered. Observe the naming conventions recommended in the Data Definition and Map Creation Wizards. See Select/Create a Source/Target Data Definition Name and Type, page 2-28 for the naming convention details.
Data Definition Description	Update the description.

Important: If the data definition changes affect any existing maps, the maps must be changed and reloaded. If the DTD reference is changed, the new DTD must be reloaded. Refer to How to Load/Delete Message Maps and DTDs, page 2-111.

The fields in the following table display when the data definition type is XML:

Field	Description
Root Element	Update the Root Element entered. The root element entered must match the root element of the DTD entered below.
Runtime DTD Location	Update the Runtime DTD Location with the new subdirectory name. Observe the naming conventions recommended in the Data Definition and Map Creation Wizards. See Identify the Runtime Location of a DTD, page 2-40 for naming convention details.
DTD File Name	Use the Browse button to select a DTD or enter a valid DTD file name. This will replace the file name originally entered. The root element defined in the DTD must match the root element value entered above.

Important: Changes to any of these property values require the message map to be reloaded. Changing the DTD associated with the message map requires the DTD to be reloaded. Refer to How to Load/Delete Message Maps and DTDs, page 2-111 for details.

Map Tab

This tab displays only if you are viewing a map transaction file (.XGM).

Properties Window, Map Tab

The screenshot shows a 'Properties' dialog box with a 'Map' tab selected. The 'Map Name' field contains 'AR_PROCESS_INVOICE_OAG_OUT' and the 'Map Description' field also contains 'AR_PROCESS_INVOICE_OAG_OUT'. The dialog has 'OK' and 'Cancel' buttons at the bottom.

Field	Description
Map Name	Update the name entered. Observe the naming conventions recommended in the Map Creation Wizard. See Specify a Map Name, page 2-26 for details.
Map Description	Update the description.

Important: If the map name change affects any existing Trading Partner definitions, update the Trading Partner definition with the new map name.

Important: The map containing the new name must be reloaded into the XML Gateway repository. Refer to How to Load/Delete Message Maps and DTDs, page 2-111 for the details.

Source Tab

Properties Window, Source Tab

The screenshot shows a 'Properties' dialog box with five tabs: General, Database, Map, Source, and Target. The 'Source' tab is selected. Inside the 'Source' tab, there is a 'Source' section containing two text input fields. The first field, labeled 'Data Definition Name', contains the text 'AR_INVO_OAG70_XML'. The second field, labeled 'Data Definition Description', contains the text 'Outbound Invoice, OAG 7.0 standard, XML definition type'. At the bottom of the dialog box are two buttons: 'OK' and 'Cancel'.

The fields in the following table display for all data definitions:

Field	Description
Data Definition Name	Update the name entered. Observe the naming conventions recommended in the Data Definition and Map Creation Wizards. See Specify Source/Target Data Definition Name and Type, page 2-29 for naming convention details.
Data Definition Description	Update the description.

Important: If the data definition changes affect any existing maps, the maps must be changed and reloaded. If the DTD reference is changed, the new DTD must be reloaded. Refer to How to Load/Delete Message Maps and DTDs, page 2-111.

The fields in the following table display when the data definition type is XML:

Field	Description
Root Element	Update the Root Element entered. The root element entered must match the root element of the DTD entered below.
Runtime DTD Location	Update the Runtime DTD Location with the new subdirectory name. Observe the naming conventions recommended in the Data Definition and Map Creation Wizards. See Identify the Runtime Location of a DTD, page 2-40 for naming convention details.
DTD File Name	Use the Browse button to select a DTD or enter a valid DTD file name. This will replace the file name originally entered. The root element defined in the DTD must match the root element value entered above.

Important: Changes to any of these property values require the message map to be reloaded. Changing the DTD associated with the message map requires the DTD to be reloaded. Refer to How to Load/Delete Message Maps and DTDs, page 2-111 for details.

Target Tab

Properties Window, Target Tab

The screenshot shows the 'Properties' dialog box with the 'Target' tab selected. The 'Target' section contains the following fields and values:

- Data Definition Name:** AR_INVO_OAG70_XML
- Data Definition Description:** Outbound Invoice, OAG 7.0 standard, XML definition type
- Root Element:** PROCESS_INVOICE_001
- Runtime DTD Location:** AR
- DTD File Name:** 901_process_invoice_001.dtd (with a 'Browse...' button)

At the bottom of the dialog are 'OK' and 'Cancel' buttons.

The fields in the following table display for all data definitions:

Field	Description
Data Definition Name	Update the name entered. Observe the naming conventions recommended in the Data Definition and Map Creation Wizards. See Specify Source/Target Data Definition Name and Type, page 2-29 for naming convention details.
Data Definition Description	Update the description.

Important: If the data definition changes affect any existing maps, the maps must be changed and reloaded. If the DTD reference is changed, the new DTD must be reloaded. Refer to How to Load/Delete Message Maps and DTDs, page 2-111.

The fields in the following table display when the data definition type is XML:

Field	Description
Root Element	Update the Root Element entered. The root element entered must match the root element of the DTD entered below.
Runtime DTD Location	Update the Runtime DTD Location with the new subdirectory name. Observe the naming conventions recommended in the Data Definition and Map Creation Wizards. See Identify the Runtime Location of a DTD, page 2-40 for naming convention details.
DTD File Name	Use the Browse button to select a DTD or enter a valid DTD file name. This will replace the file name originally entered. The root element defined in the DTD must match the root element value entered above.

Important: Changes to any of these property values require the message map to be reloaded. Changing the DTD associated with the message map requires the DTD to be reloaded. Refer to How to Load/Delete Message Maps and DTDs, page 2-111 for details.

Message Designer Wizards

The Message Designer contains two wizards to guide you through the map creation process. The two wizards are:

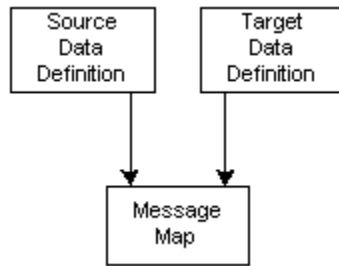
- Data Definition Creation Wizard
- Map Creation Wizard

The Data Definition Creation Wizard is used to define the data definitions.

A data definition is a collection of data used to describe a business object such as a customer profile or an invoice document. A data definition may be based on database views, database tables, application open interface tables, an application API, an XML Document Type Definition (DTD), or a production XML message.

A message map consists of a source data definition and a target data definition graphically represented as follows:

Graphical Representation for a Message Map



The data definitions created using the Data Definition Creation Wizard are used to create a preliminary message map. The Data Definition Creation Wizard is ideal for creating a primary definition to use as the basis for creating trading partner-specific message maps.

Three combinations of source and target data definitions are supported by the Message Designer as described in the following table:

Source	Target	Purpose
Database	XML	Outbound XML message
XML	Database	Inbound XML message
XML	XML	Transform one version of a DTD to the next version of the same DTD. Transform from DTD in one standard to DTD of another standard as long as the DTDs are for the same business function. Pass-through XML message (Refer to How to Map a Pass-Through Message, page 2-110) Mapping to an Application API (Refer to How to Map to an API, page 2-110)

Database-based data definitions represent a description of the Oracle data model required to support the XML message.

XML-based data definitions are based on an XML Document Type Definition (DTD) or a production XML message. XML to XML transformations from one DTD to another DTD where the DTDs are for different business purposes is done in two steps:

1. Database to XML
2. XML to database

The Map Creation Wizard is used to define the following:

- Source Definition
- Target Definition
- Preliminary Message Map

The source and target data definitions may be new or based on definitions previously created using the Data Definition Wizard. The source and target data definitions form the basis of a preliminary message map.

The Message Designer is used to define the following to complete the message map:

- Level Mapping
- Element Mapping
- Actions

Level mapping is the process of relating the source data structure to the target data structure. Element mapping is the process of relating a source data element to a target data element. Actions are data transformation or process control functions that can be applied at the data element, document, or root level.

Data Definition Creation Wizard Process Flow

Creating a Data Definition consists of two general steps:

Step 1: Name the data definition and select the type

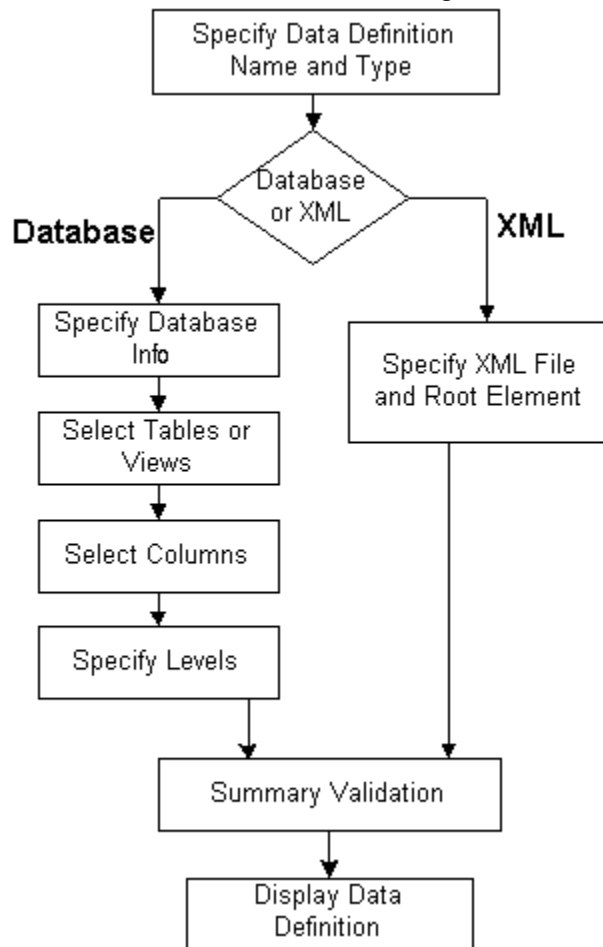
Step 2: Identify data definition details

The tasks required to complete Step 2 vary depending on your selections in Step 1.

The illustration below displays the flow of the screens presented to you in the Data Definition Creation Wizard. After you specify the data definition name and type (Step 1), you are prompted to provide the details based on whether you chose a type of database or XML.

When you have completed Step 2, you are presented with a summary screen where you can either finish or return to previous steps to make edits. Clicking Finish will close the Wizard and open the Message Designer Data Definition window.

Data Definition Creation Wizard Process Flow Diagram



Map Creation Wizard Process Flow

There are three general steps to creating a map. Each step consists of a variable number of tasks depending on the selections you make in the Wizard:

Step 1: Specify a Map Name

Step 2: Select/Create a Source Data Definition

Step 3: Select/Create a Target Data Definition

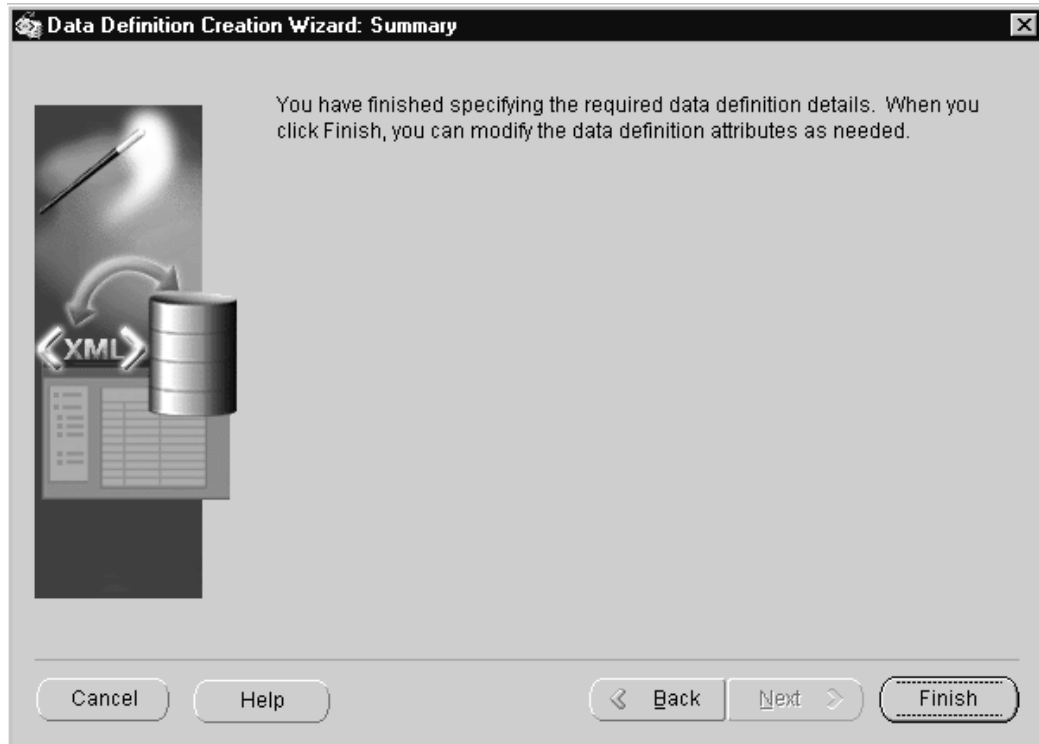
The illustration below displays the flow of the screens presented to you in the Map Creation Wizard. After you specify a map name (Step 1), you are prompted to select or create a source data definition (Step 2). If you choose to **create** the data definition, you are guided through the data definition creation steps (identical to the Data Definition Creation Wizard steps).

After you create your source data definition, or if you chose to **select** an existing data

definition, you are prompted to select or create a target data definition (Step 3). The target data definition steps are identical to the source data definition steps.

When you have completed Step 3, you are taken to a summary screen where you can either finish or return to previous steps to make edits.

Data Definition Creation Wizard: Summary



Using the Data Definition Creation Wizard

The Data Definition Creation Wizard guides you through the data definition creation process. The data definition is saved as a .xgd file and can be selected as a target or a source when you create a map.

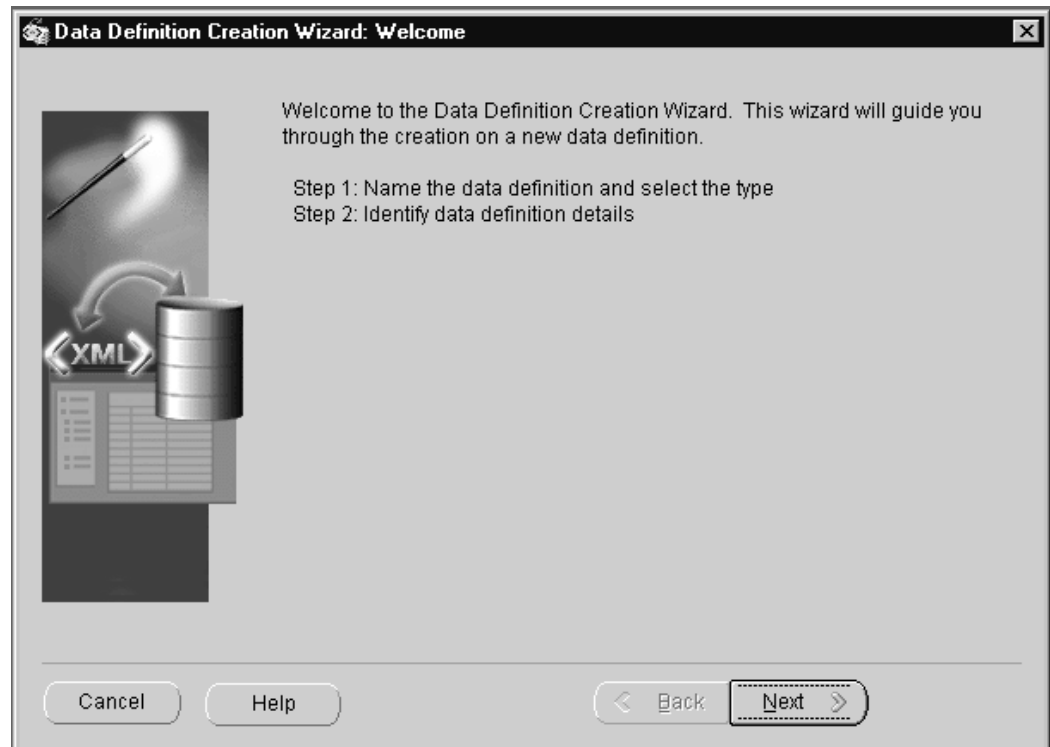
Important: It is not necessary to create your data definitions before you begin the map creation process. The Map Creation Wizard gives you the option of choosing an existing data definition or creating a new one. If you choose to create a new one, you will be guided through the data definition creation steps as part of your map creation process. See Map Creation Wizard, page 2-25.

Launch the Data Definition Creation Wizard by selecting

(M) File > New > Data Definition

The first window welcomes you to the wizard.

Data Definition Creation Wizard: Welcome



The following buttons are available for all Wizard steps:

- **Cancel**
Exit the Wizard and cancel all completed Wizard steps.
- **Back**
Return to the previous Wizard step.
- **Next**
Advance to the next Wizard step. The Next button is enabled only after the required information for the current Wizard step has been entered.

Data Definition Creation Wizard Steps

Follow the instructions presented for the Map Creation Wizard to complete each of the Data Definition Creation Wizard steps:

Specify Data Definition Name and Type

See Specify Source/Target Data Definition Name and Type, page 2-29.

If you selected Database...

Complete the following steps:

Specify Database Information

See Specify Source/Target Definition Database Information, page 2-31.

Select Tables or Views

See Select Source/Target Tables or Views, page 2-33

Select Columns

See Select Source/Target Columns, page 2-36

Specify Source/Target Levels

See Specify Source/Target Levels, page 2-37.

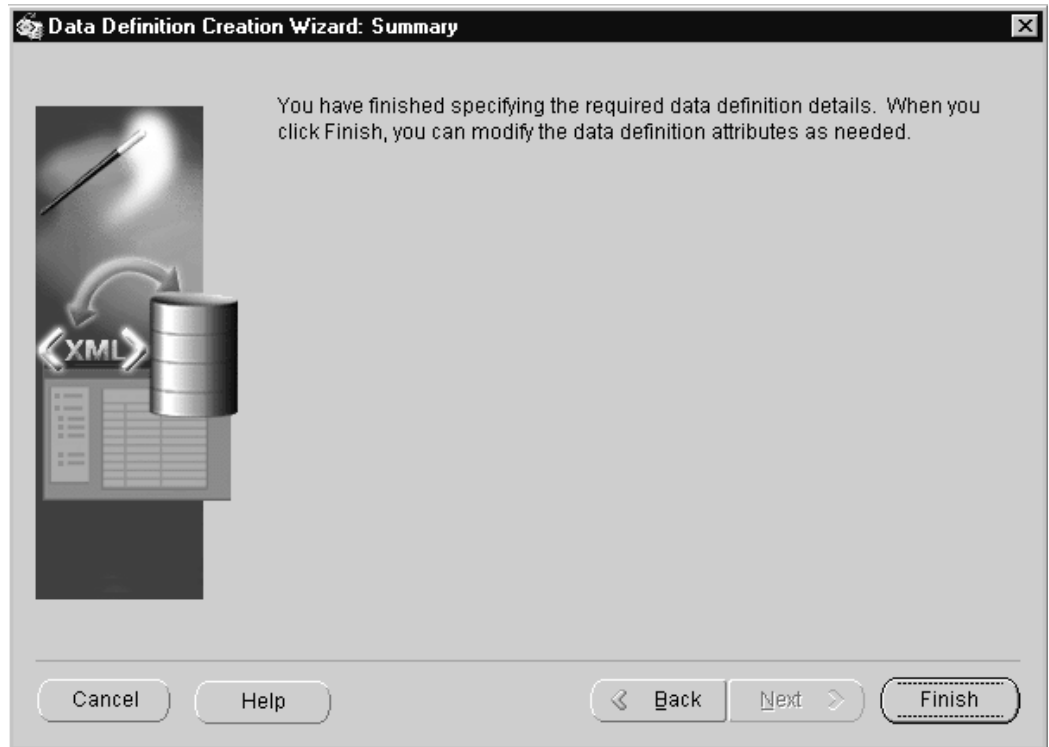
If you selected XML...

If you selected XML as the data definition type, you will be prompted for the XML information.

See Specify Source/Target XML File and Root Element, page 2-39.

Data Definition Creation Wizard: Summary

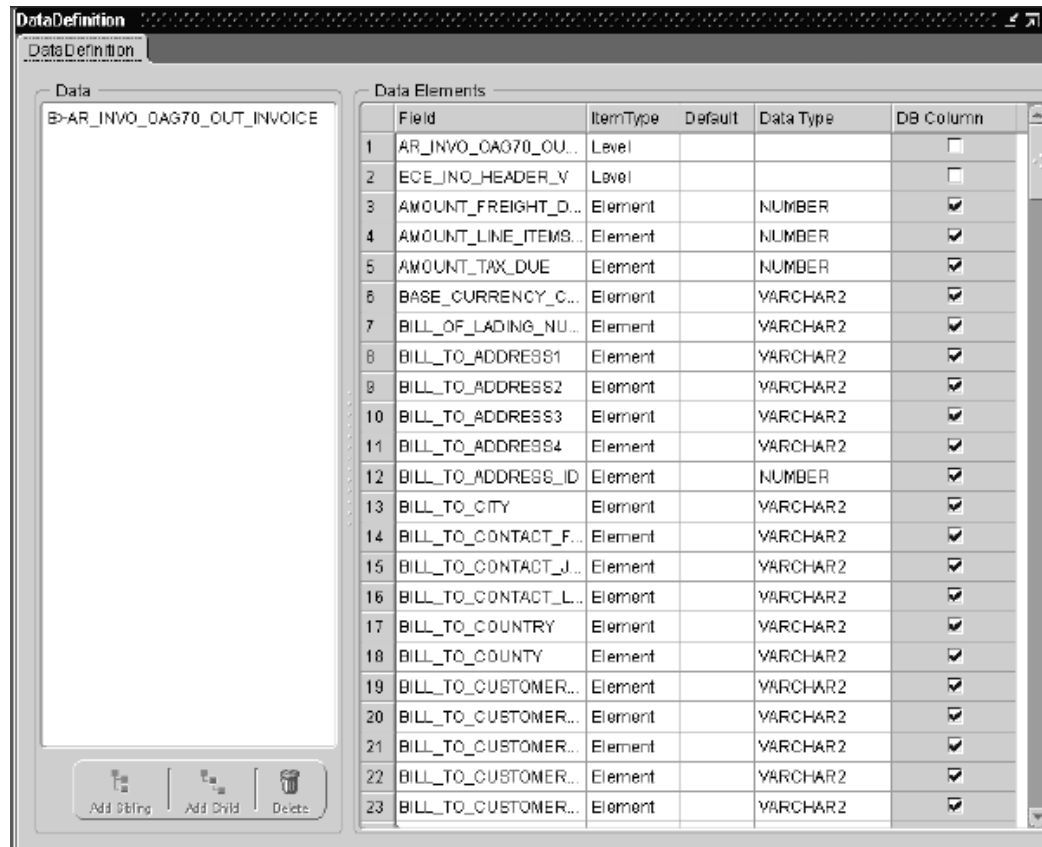
Data Definition Creation Wizard: Summary



Click Finish to complete the Wizard steps. If you wish to change any selections you have made in defining your data definition, you can use the Back button to return to the appropriate Wizard window.

Message Designer Data Definition Window

Data Definition Window



The data definition you just created is displayed in the Data Definition tab. You can extend the definition using the Data Definition window to perform the following:

- Provide default values
- Create additional nodes and elements using the Add Sibling button
- Add fields using the Add Child button
- Enable code conversion
- Define conditional node mapping rules for additional nodes or element (if source is DTD)

Once the data definition is complete, use the File > Save (Data Definition) menu option or the Toolbar equivalent. Use the File > Properties menu to change the default directory if necessary.

Using the Map Creation Wizard

The Map Creation Wizard guides you through the map creation process. A series of windows are presented to create a map based on the source and target definitions.

Launch the Map Creation Wizard by selecting

(M) File > New > Transaction Map

or by clicking the **Create New Map** icon on the toolbar. The first window welcomes you to the wizard.

Map Creation Wizard: Welcome



The following buttons are available for all Wizard steps:

- Cancel

Exit the Wizard and cancel all completed Wizard steps.

- Back

Return to the previous Wizard step.

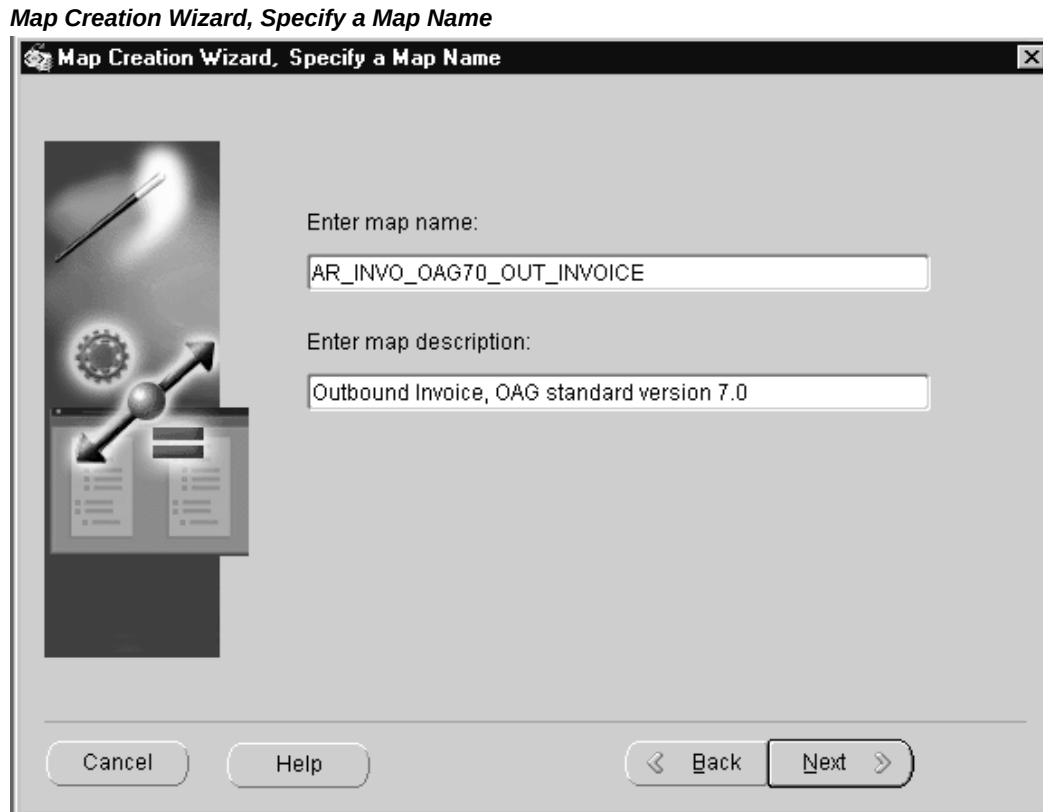
- Next

Advance to the next Wizard step. The Next button is disabled until you supply all

required data for a step.

Click Next to continue. The Wizard will display the Specify a Map Name window.

Specify a Map Name



Specify a unique map name. This is the name assigned to the message map you are creating. It is also the name to be used when associating a message map to a Trading Partner.

Enter Map Name

Enter a unique map name. This name is stored as the map code in the map definition file.

In addition to being unique, the name should describe the intended use of the map for easy identification. The recommended naming convention is as follows:

- Product mnemonic or user ID
- Transaction subtype as entered in the Define Transactions form

- XML standard and version used (for example, OAG, Rosettanet, iFX)
- Inbound or outbound message

Examples of map names using the recommended naming convention are:

- AR_INVO_OAG70_OUT_INVOICE

This map name is used for an Oracle Receivables outbound invoice message that uses the OAG standard, version 7.0 DTD.

- USER_ACK_OAG70_IN

This name represents a user-developed map for an inbound acknowledgment message that uses the OAG standard, version 7.0 DTD.

Note: You can use the File > Properties menu option to change the Map Name and Map Description as necessary.

Enter Map Description

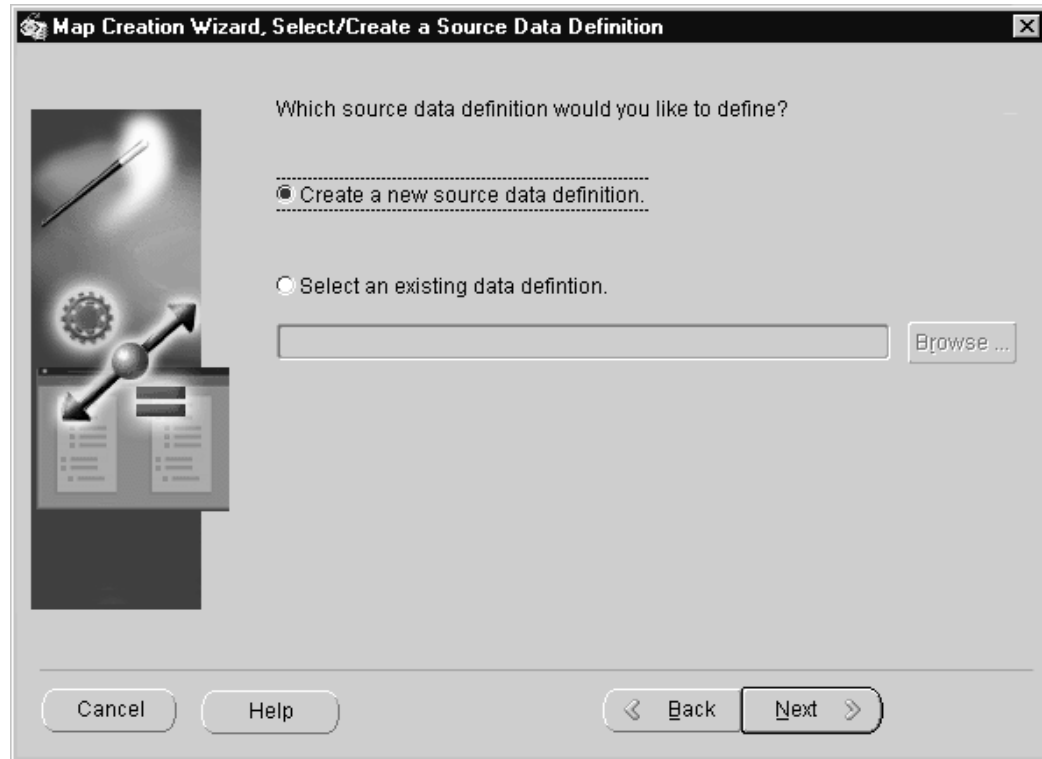
Enter a description for the map.

Note: You can use the File > Properties menu option to change the Map Name and Map Description as necessary.

Click Next to continue. The Wizard will display the Select/Create a Source Data Definition window.

Select/Create a Source/Target Data Definition

Map Creation Wizard, Select/Create a Source/Target Data Definition



Select an existing data definition file or create a new data definition file.

Create a new source/target data definition

Select this option to create a new data definition. The Wizard will guide you through creating your new .XGD file.

Select an existing data definition

Existing definition files are stored on the file system as *<filename>.XGD*. Use the Browse button to view the available data definition files.

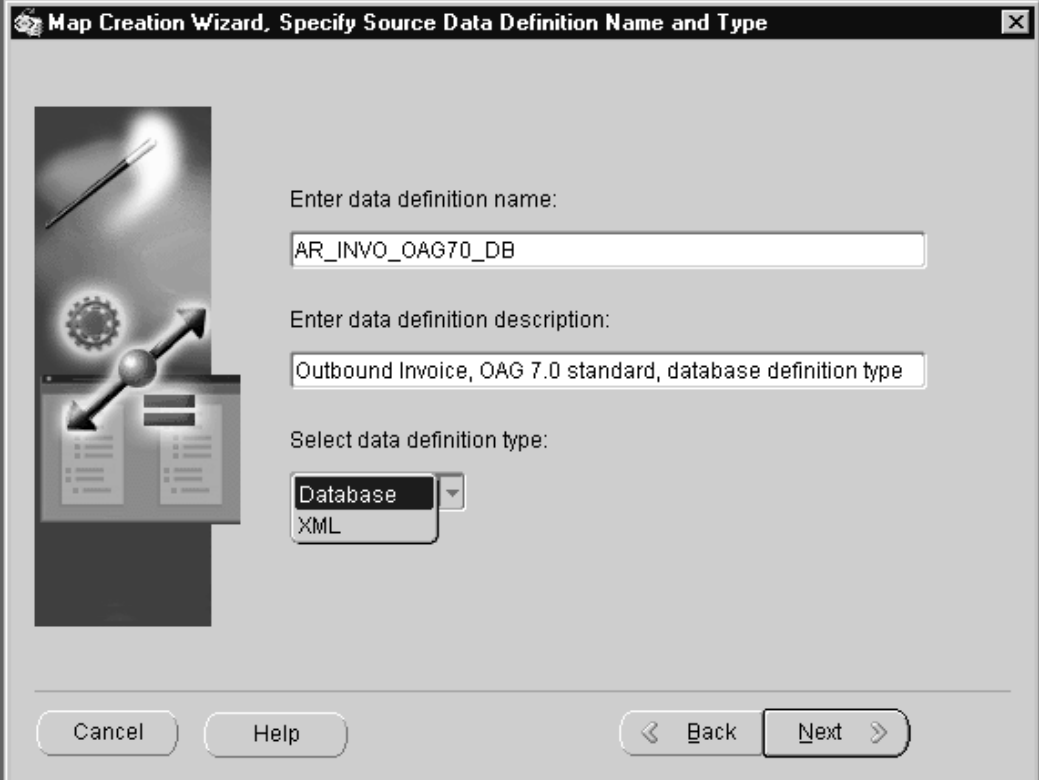
Click Next to continue

- If you are defining your **Source data definition** and you selected...
 - **Create** a new source data definition, proceed to Specify Source/Target Data Definition Name and Type, page 2-29.

- **Select** an existing Source data definition, repeat this step for your Target Data Definition.
- If you are defining your **Target data definition** and you selected...
 - **Create** a new target data definition, proceed to Specify Source/Target Data Definition Name and Type, page 2-29.
 - **Select** an existing Target data definition, proceed to Map Creation Wizard Summary, page 2-42.

Specify Source/Target Data Definition Name and Type

Map Creation Wizard, Specify Source Data Definition Name and Type



The screenshot shows a Windows-style dialog box titled "Map Creation Wizard, Specify Source Data Definition Name and Type". On the left is a vertical sidebar with a graphic of a pencil, a gear, and a document. The main area contains three input fields and a dropdown menu. The first field is labeled "Enter data definition name:" and contains the text "AR_INVO_OAG70_DB". The second field is labeled "Enter data definition description:" and contains the text "Outbound Invoice, OAG 7.0 standard, database definition type". The third field is labeled "Select data definition type:" and is a dropdown menu with "Database" selected and "XML" visible below it. At the bottom are four buttons: "Cancel", "Help", "< Back", and "Next >".

For a new data definition, enter a data definition name, description, and type.

Enter Data Definition Name

Enter a name that describes the contents of the data definition to allow easy identification.

This name is displayed in the Transaction Map window as the root node, if you select

database as the data definition type. If you select XML as the data definition type, the DTD root element is displayed in the Transaction Map window as the root node instead of the data definition name entered here.

The recommended naming convention is as follows:

- Product mnemonic or User ID
- Transaction subtype as entered in the Define Transactions form
- XML standard and version used (for example, OAG, Rosettanet, iFX)
- Database or XML data definition type

Examples of data definition names using the recommended naming convention are:

- AR_INVO_OAG70_DB

This name describes an Oracle Receivables-developed outbound invoice message that uses the OAG standard, version 7.0 DTD. This data definition is for the Database data definition type intended for use as the source data definition for an outbound message.

- USER_ACK_OAG70_XML

This name describes a user-developed inbound acknowledgement message that uses the OAG standard, version 7.0 DTD. This data definition is for the XML data definition type intended for use as the source data definition for an inbound message.

Use the File > Properties menu option to change the Data Definition Name as necessary.

Enter Data Definition Description

Enter a description for the data definition.

Use the File > Properties menu option to change the Data Definition Description as necessary.

Select Data Definition Type

Select a data definition type from the following list of values:

- Database
- XML

Click Next to continue

The wizard steps to follow will vary based on the data definition type selected.

- If you selected **Database**, proceed to Specify Source/Target Definition Database Information, page 2-31.
- If you selected **XML**, proceed to Specify Source/Target XML File and Root Element, page 2-39.

Specify Source/Target Definition Database Information

Map Creation Wizard, Specify Source Database Information

Map Creation Wizard, Specify Source Database Information



User Name:

Password:

Connect String:

Host:

Port:

The default database access information is displayed from the Database tab, page 2-6 of the File > Properties menu. Enter the password and make changes to the database access information if necessary.

Note: Changes made on this screen are not copied back to the Properties file. Entries on this screen are used for the current session only.

This step does not apply if the source/target is a DTD because database access is not required.

User Name

Enter the user name for the database schema to be accessed.

Password

Enter the password for your User Name.

Connect String

Enter the connect string for the database.

Host

Enter the host name for the database.

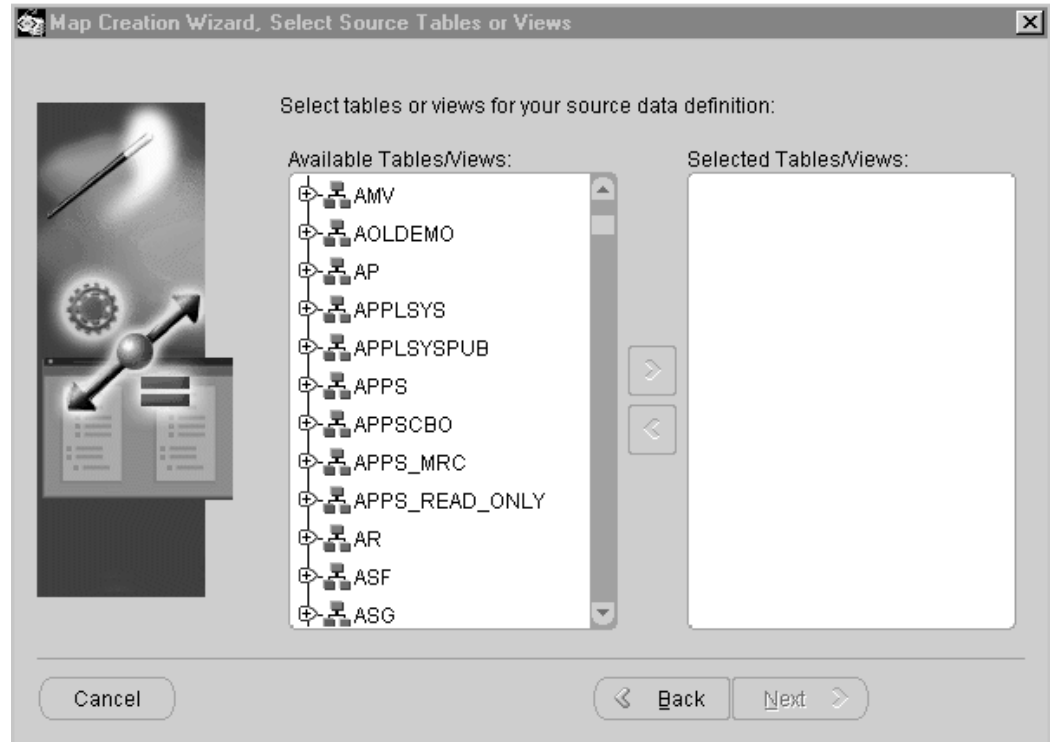
Port

The default value of "1521" is displayed. Enter another valid port value (if necessary) for the database to be accessed.

Click Next to continue. The Wizard will display the Select Source/Target Tables or Views window.

Select Source/Target Tables or Views

Map Creation Wizard, Select Source Tables or Views

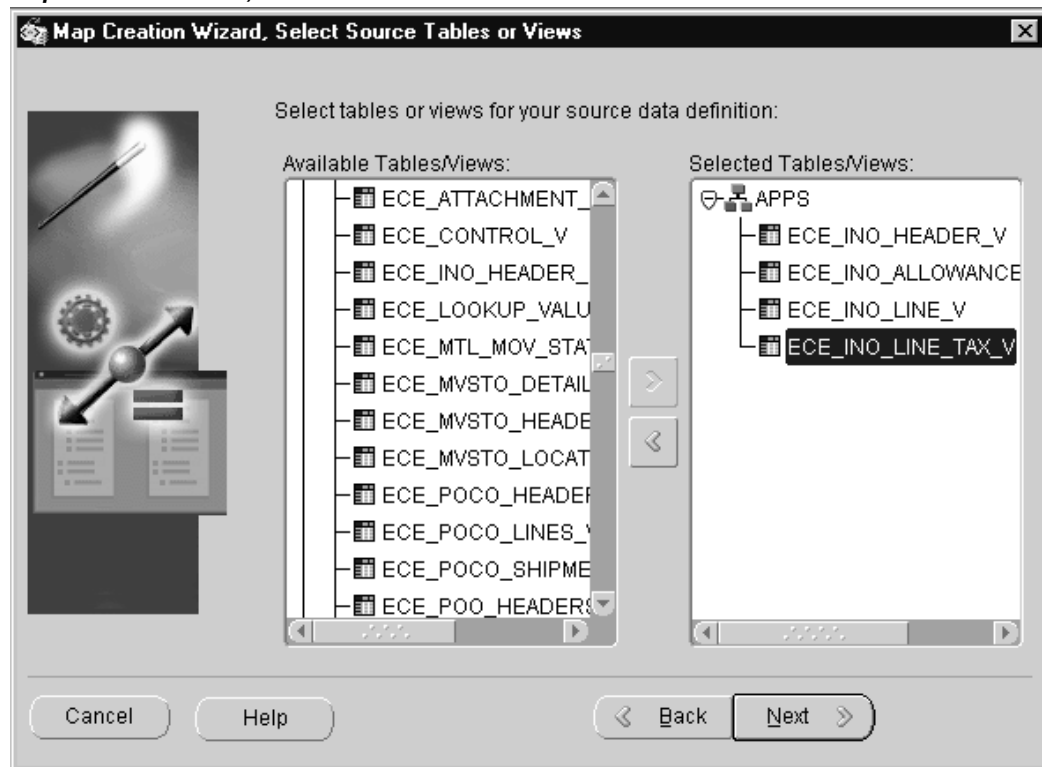


If you selected a Data Definition Type of Database, you will be prompted for the database schema, database views, and tables required by the message.

All application database views are defined in the APPS schema. The associated database tables are defined in the application-specific database schema. Access to application-specific database tables must be granted by defining a synonym in the APPS schema.

For applications with database views or tables shared across multiple hierarchical levels of the data model representing the business document, a database view or table alias is required for the second and each subsequent use of the view or table. This is necessary because this wizard step deletes a database view or table from the Available Tables/Views list once it has been selected.

Map Creation Wizard, Select Source Tables or Views



This step does not apply if the source/target is a DTD, because database information is not necessary.

Available Tables/Views

Expand the APPS database schema tree to view all the available database tables and views.

Selected Tables/Views

Select the desired database views and tables from the left window, click on the right shuttle to move the selected database view or table from Available Tables/Views to Selected Tables/Views. Continue this process until you have selected all the required database views and tables for each schema selected.

Once you have selected all the database views and tables required by the message, select any special XML Gateway database views you may have defined to support the specific requirements of the XML standard you are using.

For example, select the ECX_OAG_CONTROLAREA_TP_V (formerly ECX_OAG_CONTROLAREA_V) database view to map to the OAG CNTROLAREA data type.

Note: The ECX_OAG_CONTROLAREA_TP_V view is an upgraded version of the ECX_OAG_CONTROLAREA_V view. Oracle XML Gateway supports both versions of the database view.

The upgraded view includes new fields for USERNAME, SOURCE_TP_LOCATION_CODE, PARTY_ID, PARTY_SITE_ID, and PARTY_TYPE as well as changes to the following existing fields:

REFERENCE_ID is based on the system name, event name, and event key defined by the application business event. REFERENCE_ID was previously defaulted to "1" with recommendations to use the ECX_REFERENCE_ID sequence to get a unique number. The use of this field varies by message map.

You must add the ECX_EVENT_MESSAGE item attribute to your Workflow item type to have access to the event details.

CONFIRMATION is based on the setting defined for the Trading Partner and business document entered using the Define Trading Partners window. CONFIRMATION was previously defaulted to "0".

COMPONENT is based on the internal transaction type entered on the Define Transactions window for the business document. COMPONENT was previously based on the external transaction type.

TASK is based on the internal transaction subtype entered on the Define Transactions window for the business document. TASK was previously based on the external subtype.

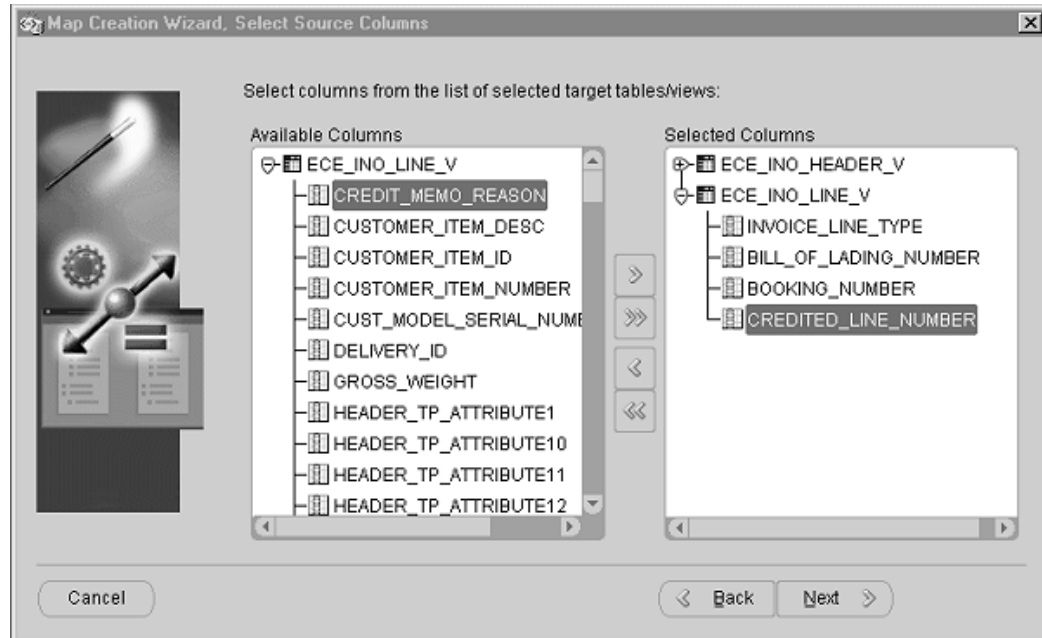
TRANSACTION_SUBTYPE is based on the internal transaction subtype entered on the Define Transactions window. TRANSACTION_SUBTYPE was previously defaulted to the TRANSACTION_TYPE if no value was found in the database.

To deselect a selected database view or table, select the desired database view/table from the right window, click on the left shuttle button to move the selection from the Selected Tables/Views back to Available Tables/Views.

Click Next to continue. The Wizard will display the Select Source/Target Columns window.

Select Source/Target Columns

Map Creation Wizard, Select Source Columns



This window prompts you for the columns required from each of the database views or tables selected in the previous step.

This step does not apply if the source/target is a DTD because database information is not necessary.

Available Columns

Expand each of the database views and tables to view all the available database view and table columns.

Selected Columns

Select the desired database view or table column from the left window, click on the right shuttle button to move the selected database view or table column from Available Columns to Selected Columns. To move all the columns from a view or table, select the table or view and click the double right shuttle button to move all its columns from Available Columns to Selected Columns. Continue this process until all the required database view and table columns are selected.

You cannot proceed to the next wizard step until you have selected at least one column from each of the selected database views and tables. If necessary, return to the previous wizard step to deselect a database view or table and then resume the column selection

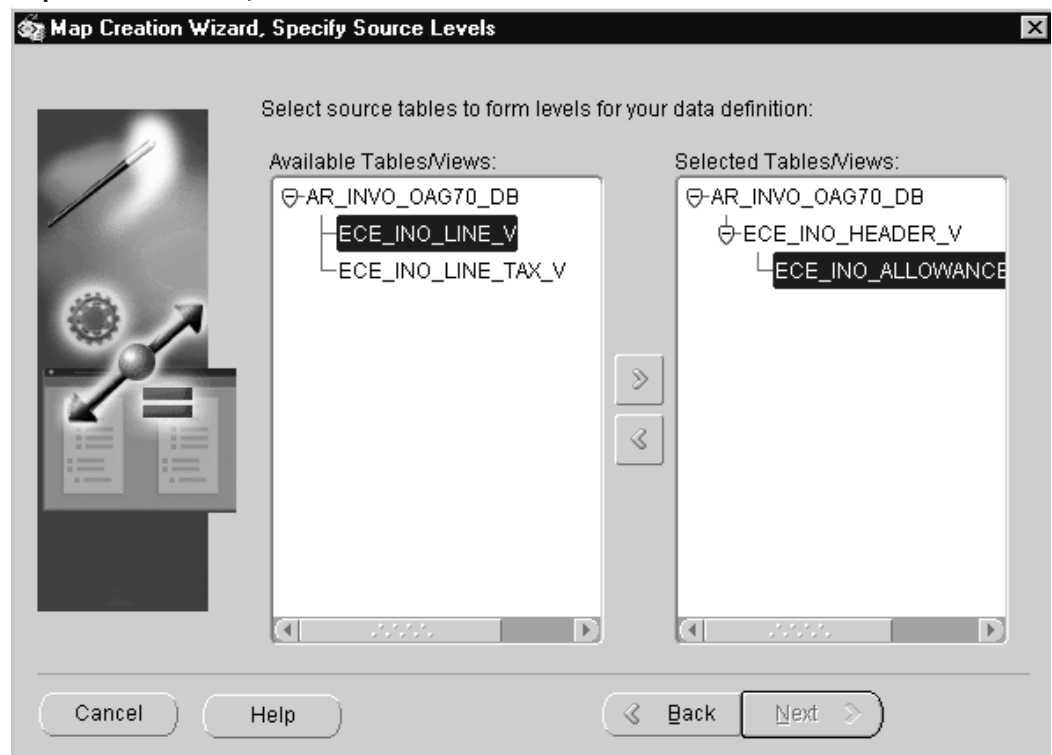
process.

To deselect a selected database view or table column, highlight the desired column from the right window and click the left shuttle button to move the highlighted column from Selected Columns back to Available Columns. To deselect all the columns from a table or view, highlight the table or view name under Selected Columns and click the double left shuttle button to move all the columns back to the Available Columns area.

Click Next to continue. The Wizard will display the Specify Source/Target Levels window.

Specify Source/Target Levels

Map Creation Wizard, Select Source Levels



If you selected a Data Definition Type of Database, you will be prompted to identify the hierarchy of the source or target data definition. This step is necessary only if your source/target data definition contains more than one level.

A level represents a collection of data that repeats. For example, Purchase Order lines represent a level within a Purchase Order because there are multiple PO lines to a single PO.

The purpose of this step is to identify the parent and child relationships of each database view. The source hierarchy will be used to relate to the target hierarchy as part of the map creation process.

This step does not apply if the source/target is a DTD because database information is not necessary.

Available Tables/Views

The available database views and tables are displayed for your selection.

Selected Tables/Views

This window allows you to define the parent and child relationships of the selected database views and tables along with the special database views necessary to relate the database data model to the DTD data model.

Start by identifying the parent node. For OAG, the parent node is the ECX_OAG_CONTROLAREA_TP_V view (formerly ECX_OAG_CONTROLAREA_V). Related to the parent node are sibling or child nodes.

Note: The ECX_OAG_CONTROLAREA_TP_V view is an upgraded version of the ECX_OAG_CONTROLAREA_V view. Oracle XML Gateway supports both versions of the database view. For more information see the Note, page 2-34.

Sibling and child relationships are defined by first identifying the parent node in the Selected Tables/Views window (right). The database view or table you move from the Available window to the Selected window will always be added as the last child of the parent node selected.

Therefore, to specify a sibling relationship to a specific node, select its parent node in the Selected Tables/Views column (right window). Next select the database view or table in the Available Tables/Views (left window) that you want to define as its sibling. Click on the right shuttle. The selected database view or table will be displayed as the last child of the parent node selected, or as a sibling to the desired node.

To specify a child relationship to a specific node, select the node in the Selected Tables/Views column (right window). Next select the database view or table in Available Tables/Views (left window) that you want to define as its child, and click on the right shuttle. The selected database view or table will be displayed as the last child of the selected parent node.

Continue this process until all available database views and tables have been moved to Selected Tables/Views.

This completes the data definition process.

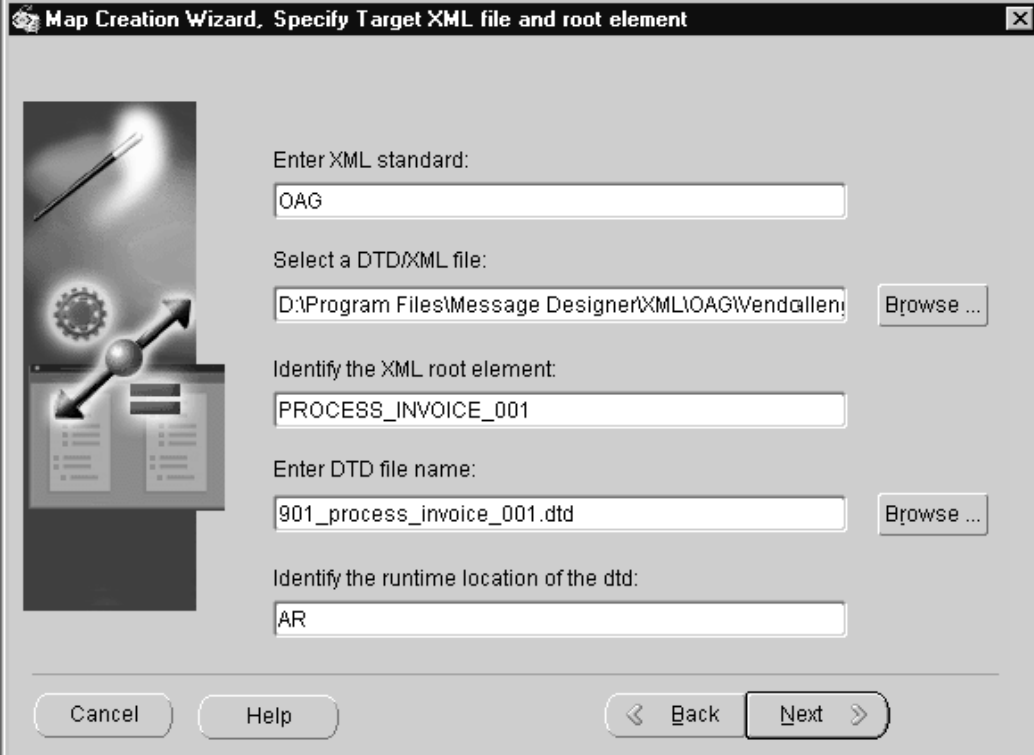
Click Next to continue

If you have just finished your Source Data Definition, return to Select/Create a Source/Target Data Definition, page 2-28 to define your Target Data Definition.

If you have just finished your Target Data Definition, proceed to Map Creation Wizard Summary, page 2-42.

Specify Source/Target XML File and Root Element

Map Creation Wizard, Specify Target XML File and Root Element



The screenshot shows a Windows-style dialog box titled "Map Creation Wizard, Specify Target XML file and root element". On the left is a vertical sidebar with a graphic of a target and arrows. The main area contains five input fields with labels and "Browse ..." buttons. The fields are: "Enter XML standard:" with "OAG", "Select a DTD/XML file:" with "D:\Program Files\Message Designer\XML\OAG\Vendor...", "Identify the XML root element:" with "PROCESS_INVOICE_001", "Enter DTD file name:" with "901_process_invoice_001.dtd", and "Identify the runtime location of the dtd:" with "AR". At the bottom are "Cancel", "Help", "Back", and "Next" buttons.

Map Creation Wizard, Specify Target XML file and root element

Enter XML standard:
OAG

Select a DTD/XML file:
D:\Program Files\Message Designer\XML\OAG\Vendor... Browse ...

Identify the XML root element:
PROCESS_INVOICE_001

Enter DTD file name:
901_process_invoice_001.dtd Browse ...

Identify the runtime location of the dtd:
AR

Cancel Help Back Next

If you selected XML as the data definition type, you will be prompted for the XML information. You can use a DTD or a production XML message as the data definition.

XML Standard

Enter the XML standard used.

"OAG" (Open Application Group XML Standard) is the default.

Select a DTD/XML File

Select the DTD from the list of available DTDs presented when you click on the Browse button, or enter a specific DTD including the file path. The selected DTD and its file path are displayed.

To base the data definition on a production XML message, enter the file path and file name of the XML message.

The DTD or production XML message is used to create the definition tree for the Message Designer.

This step assumes that the required DTD and the associated external reference DTD files are available on the file system and are accessible by the Message Designer. For example, the OAG definition files are

- oagis_domains.dtd
- oagis_resources.dtd
- oagis_fields.dtd
- oagis_segments.dtd

Identify the XML Root Element

Enter the XML root element. Open the DTD or production XML message using a browser or XML editor to determine the root element name if you do not know it.

Note: Because the representation of root element varies by XML standard, this step is required to inform the Execution Engine where to begin reading the DTD.

Use the File > Properties menu option to change the Root Element as necessary.

Enter DTD File Name

If you entered a DTD above, the DTD file name will display automatically.

If you entered a production XML message above, click the Browse button to select the corresponding DTD from the list of available DTDs, or enter a specific DTD if available.

The DTD file name entered is used to validate a message to ensure that it is well-formed and valid before placing an outbound message on the outbound queue or processing a message dequeued from the inbound queue.

In addition, the primary DTD and its associated definition files must be loaded into the XML Gateway repository for access by the XML Gateway Execution Engine.

Note: Refer to How to Load/Delete Message Maps and DTDs, page 2-111 for more details.

Use the File > Properties menu option to change the DTD File Name as necessary.

Identify the Runtime Location of a DTD

Enter the subdirectory name using the following naming convention:

`<application code>/xml/<standard><standards version>`

Examples:

`ar/xml/oag62`

`ap/xml/oag70`

Note: Do not use a period (.) when referencing a standards version.

The combination of the runtime DTD location and the DTD file name provides a unique identifier for the DTD required.

Use the File > Properties menu option to change the Runtime Location of a DTD as necessary.

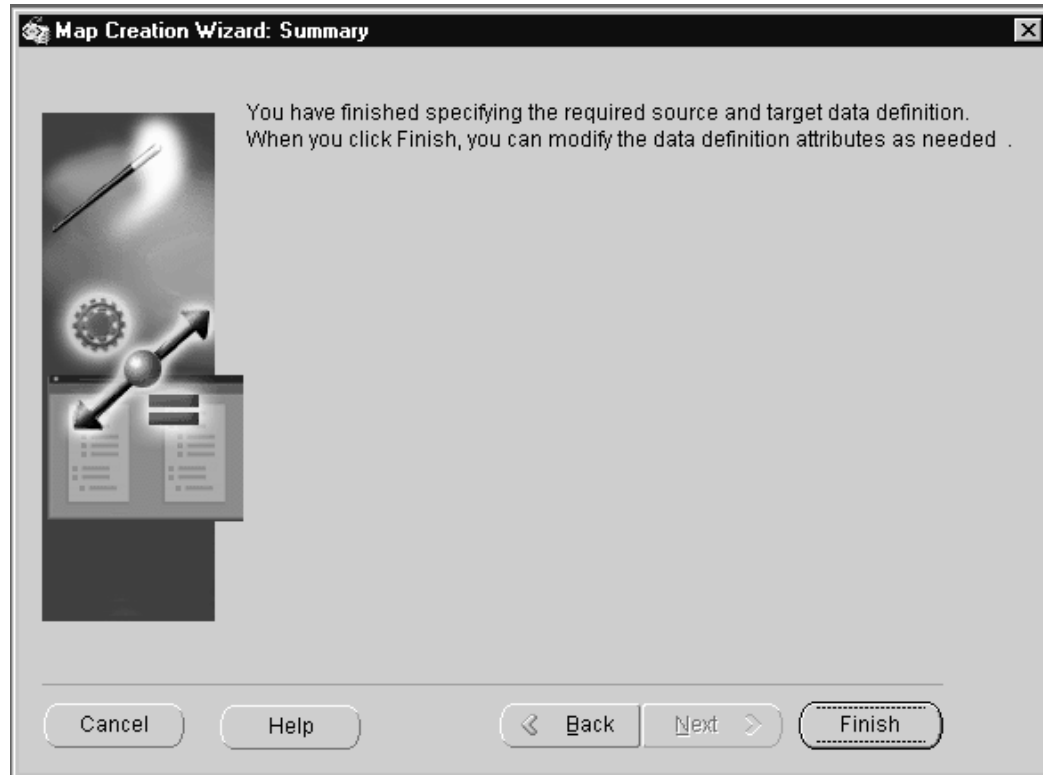
Click Next to continue

If you have just finished your Source Data Definition, return to Select/Create a Source/Target Data Definition, page 2-28 to define your Target Data Definition.

If you have just finished your Target Data Definition, proceed to Map Creation Wizard Summary, page 2-42.

Map Creation Wizard Summary

Map Creation Wizard: Summary



Click Finish to exit the Map Creation Wizard. If you wish to change any selections you have made in defining your map, use the Back button to return to the appropriate Wizard window.

Before proceeding to the level and element mapping process, the XML Gateway, in conjunction with the XML Parser, performs the following validations:

- When the DOCTYPE tag is present in an XML message, verify that the DTD referenced is accurate.
- Verify that the XML root element matches the DTD identified. When a production XML message is identified, verify that the root element and DTD identified match the production XML message.
- Verify that external DTDs referenced by the primary DTD are available.
- Check for circular DTD references. Process the first occurrence, truncate the remainder and warn user to manually add the necessary repeating occurrences.

Transaction Map Window

Upon exiting the Map Creation Wizard, the source and target definitions are presented in the Transaction Map window.

The Transaction Map window will also be presented if you select an existing map with a version number compatible with the version number of Message Designer.

The version number of the map (.xgm file) is stored in the <ECX_MAJOR_VERSION> and <ECX_MINOR_VERSION> tags. The Message Designer version number is available in the Help > About menu. Maps are compatible with Message Designer if the major version is the same and the minor version is the same or lower.

The Transaction Map window is divided into four tabs as follows:

- Source Definition
- Target Definition
- Level Mapping
- Element Mapping (this tab appears only after a Level has been mapped)

Source Definition

The source definition selected or created using the Map Creation Wizard is displayed in the Source Definition tab. You can extend the source definition to perform the following:

- Provide default values
- Enable code conversion
- Create duplicate nodes or elements using the Add Sibling button
- Add fields using the Add Child button
- Define conditional node mapping rules for duplicate nodes or elements (if the source is a DTD)

Once the source data definition is complete, it can be saved for reuse in other message maps. Use the File > Save (Data Definition) menu option or the Toolbar equivalent. Use the File > Properties menu to change the default directory or data definition property values if necessary.

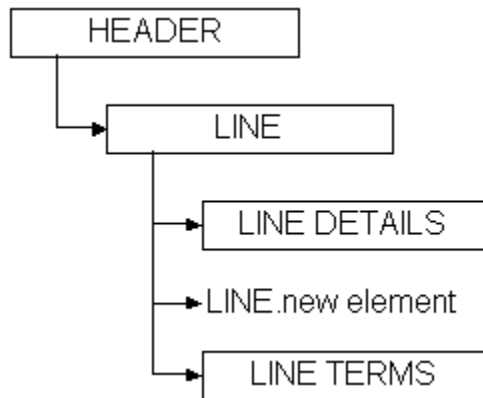
If you do not wish to save the source data definition as an entity independent of the message map, then continue with the mapping process and save the transaction map at the end of the Element Mapping process.

Source Definition Considerations

1. Refer to How to Map a Pass-Through Message, page 2-110, for guidelines related to developing a pass-through transaction.
2. Discontinuous nodes:

A discontinuous node is a non-level node that follows a level and is a sibling of that level. It can be represented graphically as follows:

Graphical Representation of a Discontinuous Node



In this example, the levels LINE DETAILS and LINE TERMS are children of the level LINE, and are siblings of each other. Inserted between LINE DETAILS and LINE TERMS is a new element of the LINE node. The new element is a continuation of the LINE node that creates a break between LINE DETAILS and LINE TERMS.

The ideal placement of the new element is as the last element of the LINE node before the LINE DETAILS node. However, some standards bodies do not have the flexibility to restructure an existing DTD, or may not wish to for backwards compatibility reasons. Regardless of the reason, the condition exists, and XML Gateway supports it.

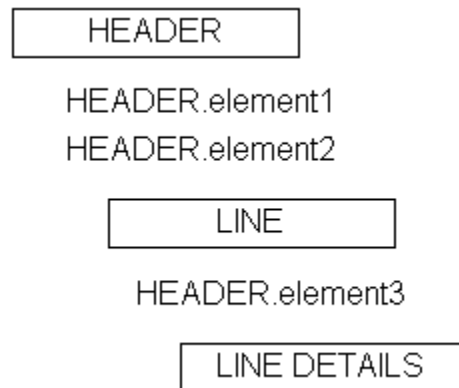
For data definitions based on the Oracle data model, use the Add Child or Add Sibling button in the Message Designer, Transaction Map window to define a discontinuous node. Any new node introduced on the source and distributed across multiple target levels (expanded) or consolidated into a single target level (collapsed) will be grouped with the parent node and mapped according to the target definition. The rules against level cross-over still apply.

For data definitions based on a DTD, use the Transaction Map window, Item Type column to explicitly identify the data levels. This exercise may create a discontinuous node, which is not a problem unless you define the invalid scenario described below. The only user extensions supported for OAG DTDs are in the USERAREA. If you modify the DTD (using the Add Child or Add Sibling buttons) outside of the USERAREA, a parsing error will be triggered.

Message Designer will allow you to define a discontinuous node anywhere in the source or target definition. You will not know until runtime if the definition is valid or not. Therefore you should avoid introducing a discontinuous node for a node which also contains a child node.

The following graphic illustrates an invalid definition:

Graphical Representation of an Invalid Definition



In the example above, HEADER.element3 is a continuation of the HEADER node. HEADER.element3 is also defined as a child of the LINE node. HEADER.element3 has a child node called LINE DETAILS. This is an invalid definition.

1. Refer to How to Implement the OAG Confirmation Business Object Document, page 4-17, for details on how to implement the optional confirmation message.
2. Refer to How to Implement Attachments in XML Messages, page 2-115 for details on how to include attachments with your XML documents.

Source Definition Tab

Field

Field identifies the name of the element, document, or root. The names are based on the database column names or DTD element names.

The field name may be changed if necessary, however, consider what this change implies. Because the field is based on the database column name or a DTD element name, corresponding changes may be necessary in the Oracle E-Business Suite or the DTD. The only changes allowed to a DTD are to the USERAREA. Refer to How to Extend DTDs, page 2-108 for details.

The first row is reserved for the Data Definition name (if the source is based on database views or tables) or root element name (if the source is a DTD).

Additional field names are displayed when sibling or child elements are added using the Add Sibling or Add Child buttons.

Item Type

Item type identifies the field as either a Level or an Element. Level represents the parent in a parent-child relationship. Element represents the child in a parent-child relationship.

The Item Type of the first row is defaulted to "Level". You cannot change this value.

If the source is based on database views or tables, the Item Type for the database view or table is defaulted to "Level". The Item Type for each database view or table column is defaulted to "Element".

If the source is based on a DTD or a production XML message, the default Item Type is "Element". DTDs do not support levels, therefore any levels must be explicitly defined by setting the Item Type to "Level".

Any node that represents a collection of data that repeats (for example, Purchase Order lines or shipment lines) represents a level. The Item Type for the node must be set to Level.

For the OAG standard, change the Item Type of the following to Level to support Level Mapping:

- Root
- CNTROLAREA
- DATAAREA

Make the same change for all other DTD data types identified as data levels during the map analysis process.

The Item Type for a new sibling or child element is defaulted to "Element". Change the Item Type setting when appropriate.

Default

Enter a default value for the field as appropriate. This value is used in the outbound message or an inbound message if the incoming value is null.

If the default value is based on a condition, set the default using the Assign Variable Value action. See Assignments: Assign Variable Value, page 2-74.

If Item Type is "Level", this column is disabled.

Category

Enable the field for code conversion by entering a valid code category. Validation of code category is performed at runtime because the database used to define the map may not be the database where the transaction is executed. Refer to Seeded Code Categories, page B-1 for a list of seeded code categories.

Universal or standards-specific code conversion values are defined using the Define Code Conversion Values form. Trading Partner-specific code conversion values are defined using the Trading Partner Code Conversion form.

The execution engine will search for code conversion values in the following sequence:

- Trading Partner
- Standard-specific
- Universal list

Use the Get Predefined Variable Value action to determine the status of the code conversion process for the source column enabled for code conversion.

See Get Predefined Variable Value, page 2-96 for details regarding the code conversion return status and possible actions to take if the code conversion cross-reference value is not found.

Code conversion is applied before any Actions are applied. Consider the code conversion return status when you define Actions for the source column enabled for code conversion. Actions are applied to the code-converted value only when the code conversion process is successful.

If Item Type is Level, this column is disabled.

Data Type

Each field is defined with a data type. The data types supported by the XML Gateway Execution Engine are VARCHAR2, DATE, NUMBER, CHAR, and CLOB.

The CLOB data type is used to support large objects up to 4 GB in size. The CLOB is displayed between CDATA tags to escape parsing.

If the source is based on database views or tables, the data types are defaulted to the data types defined for the view or table columns.

If the source is based on a DTD or a production XML message, the data type is defaulted to VARCHAR2.

Important: It is not necessary to change the DTD element's data type until XML schemas are supported by the XML standards bodies which will make data types more meaningful.

The Data Type for a new sibling or child element is defaulted to VARCHAR2. Change the data type if necessary.

If the Item Type is Level, the Data Type column is disabled.

DB Column

The DB Column is available only when the source is based on database views or tables.

A check mark indicates the column is defined in the Oracle E-Business Suite data model. This setting informs the XML Gateway Execution Engine whether to validate the element against the Oracle E-Business Suite data model or not.

If Item Type is Level, the DB Column is disabled.

Node Type

Node Type is available only when the source is a DTD. The default is the DTD setting.

The valid values are Element or Attribute. Elements contain the business data.

Attributes contain qualifiers for the business data to indicate its intended meaning.

The Node Type for a new sibling or child element is defaulted to Element. Change the Node Type setting where appropriate.

If Item Type is Level, this column is disabled.

Add Sibling (button)

The Add Sibling button allows you to add new fields required to complete the map. This feature is also used to create duplicate DTD nodes such as PARTNER.

When the source is based on a DTD or production XML message, and you are adding a sibling between two attributes, set the Node Type for the new field to "Attribute". The default Node Type of "Element" will cause a parser violation.

Important: You cannot add a sibling to the root node.

Refer to How to Extend DTDs, page 2-108 for details on how to extend a DTD. Included are the naming conventions that must be followed for the XML Parser to recognize the DTD extensions.

Add Child (button)

The Add Child button adds child elements to an existing sibling or child. In addition, Add Child can be used to define an attribute for the root element if the source or target is a DTD.

This function may be used if the source is database views or tables or a DTD.

If Node Type is "Attribute", the Add Child button is disabled. You cannot define an attribute for an attribute.

Note: Refer to How to Extend DTDs, page 2-108 for details on how to extend a DTD. Included are the naming conventions that must be followed for the XML Parser to recognize the DTD extensions.

Delete (button)

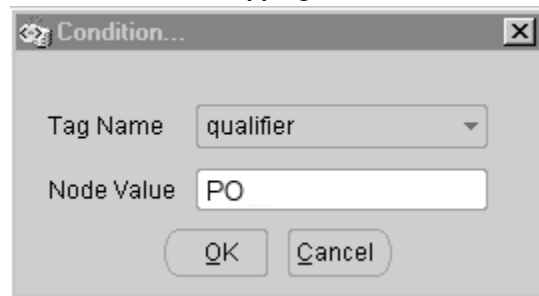
The Delete button allows you to delete any sibling or child element that has not been mapped.

If a sibling or child has child elements associated with it, a warning is displayed before the delete occurs.

Important: If you are deleting any DTD extensions, be sure to remove the DTD extensions from the corresponding extension file created for the application or user. The extra information will not cause a parser violation, but it is a good practice to ensure the extension files match the message maps. Refer to How to Extend DTDs, page 2-108 for the details.

Conditional Node Mapping Window

Conditional Node Mapping Window



Condition.../Delete Condition

If the source definition is based on a DTD with duplicate nodes or elements of the same name, use Conditional Node Mapping to ensure that the source-to-target mappings are performed correctly. This is required because the sequence of duplicate nodes and elements is not fixed.

Examples of duplicate nodes that occur frequently in a DTD are DATETIME, AMOUNT, OPERAMT, and QUANTITY. Using a purchase order as an example, the first DATETIME occurrence is for the Order Date and the second occurrence is for the Promise Date.

Conditional node mapping allows you to define the relationship of the source node to the target node based on the value of key elements. The key elements are identified by the Tag Name and Node Value combination. For the purchase order example above, the Tag Name is "qualifier" and the Node Value is "PO" for the first DATETIME occurrence representing the order date. The Tag Name for the second DATETIME occurrence is also "qualifier" with a Node Value of "PROMDELV" for promise date.

To define the key values, select the source DTD node and click the right mouse button to invoke the Conditional Node Mapping window. You will be prompted for the following:

Tag Name	Select Tag Name from the list of values. Using the above example, this is the "qualifier" attribute associated with the DATETIME segment that uniquely identifies the node and its intended meaning.
Node Value	Enter the node value. Using the above example, this is either "PO" or "PROMDELV".

Repeat this process for each of the duplicate nodes and elements. Message Designer supports one condition per node or element.

To delete the Conditional Node Mapping instruction for a node, select the node and click the right mouse button to invoke the Delete Condition function.

Conditional node mapping is applicable to source data definitions that are based on a DTD or a production XML message. It does not apply when the source data definitions are based on database views or tables.

Transaction Map - Target Definition

The target definition selected or created using the Map Creation Wizard is displayed in the Target Definition Tab. The target definition can be extended to perform the following:

- Provide default values
- Create duplicate nodes and elements using the Add Sibling button
- Add fields using the Add Child button
- Delete unused and unmapped DTD elements to prevent parser violations

Once the target data definition is complete, it can be saved for reuse in other message maps. Use the File > Save (Data Definition) menu option or the toolbar equivalent. Use the File > Properties menu option to change the default directory or data definition property values if necessary.

If you do not wish to save the target data definition as an entity independent of the message map, then continue with the mapping process and save the transaction map at the end of the Element Mapping process.

Target Definition Considerations

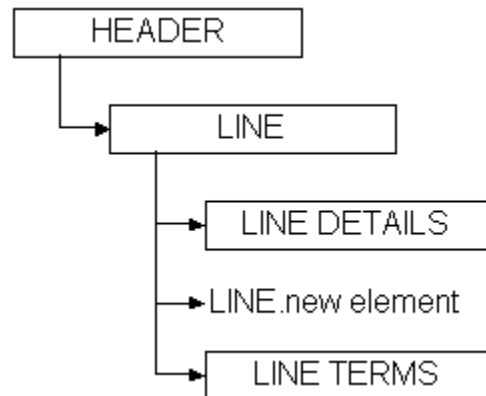
1. Refer to How to Map a Pass-Through Message, page 2-110, for guidelines related to

developing a pass-through transaction.

2. Refer to How to Map to an API, page 2-110 for guidelines for mapping an inbound message to an Application API (as opposed to mapping to Application Open Interface tables).
3. For DTD elements defined using "1" as the occurrence indicator, make sure you select one element from the choice list and delete the unused elements. The parser will validate that only one element is used.
4. Delete optional DTD data types and elements that are not used and therefore not mapped. If not deleted, these elements will appear as empty tags in the resulting message.
5. If you decide not to delete DTD data types and elements that are not used and not mapped, default the data type or element attribute to "OTHER". The parser requires the attribute even though the data type or element is optional.
6. Refer to How to Implement Attachments in XML Messages, page 2-115 for details on how to include attachments with your XML documents.
7. Discontinuous nodes:

A discontinuous node is a non-level node that follows a level and is a sibling of that level. It can be represented graphically as follows:

Graphical Representation of a Discontinuous Node



In this example, the levels LINE DETAILS and LINE TERMS are children of the level LINE, and are siblings of each other. Inserted between LINE DETAILS and LINE TERMS is a new element of the LINE node. The new element is a continuation of the LINE node that creates a break between LINE DETAILS and LINE TERMS.

The ideal placement of the new element is as the last element of the LINE node before the LINE DETAILS node. However, some standards bodies do not have the flexibility to

restructure an existing DTD, or may not wish to for backwards compatibility reasons. Regardless of the reason, the condition exists, and XML Gateway supports it.

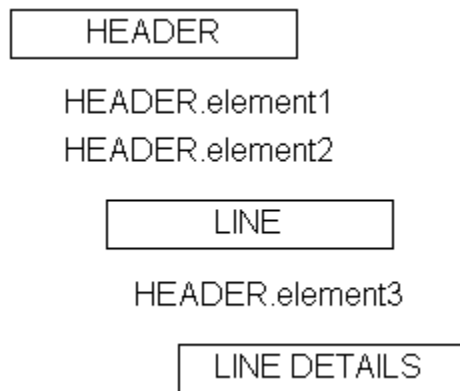
For data definitions based on the Oracle data model, use the Add Child or Add Sibling button in the Message Designer, Transaction Map window to define a discontinuous node. Any new node introduced on the source and distributed across multiple target levels (expanded) or consolidated into a single target level (collapsed) will be grouped with the parent node and mapped according to the target definition. The rules against level cross-over still apply.

For data definitions based on a DTD, use the Transaction Map window, Item Type column to explicitly identify the data levels. This exercise may create a discontinuous node, which is not a problem unless you define the invalid scenario described below. The only user extensions supported for OAG DTDs are in the USERAREA. If you modify the DTD (using the Add Child or Add Sibling buttons) outside of the USERAREA, a parsing error will be triggered.

Message Designer will allow you to define a discontinuous node anywhere in the source or target definition. You will not know until runtime if the definition is valid or not. Therefore you should avoid introducing a discontinuous node for a node which also contains a child node.

The following graphic illustrates an invalid definition:

Graphical Representation of an Invalid Definition



In the example above, HEADER.element3 is a continuation of the HEADER node. HEADER.element3 is also defined as a child of the LINE node. HEADER.element3 has a child node called LINE DETAILS. This is an invalid definition.

Target Definition Tab

Transaction Map, Target Definition Tab

The screenshot shows the 'Transaction Map : INVOICE_OUTBOUND' window with the 'Target Definition' tab selected. The left pane shows a tree structure of the target definition:

- PROCESS_INVOICE_001
 - CNTRLAREA
 - DATAAREA
 - PROCESS_INVOICE
 - INVOICEHDR
 - AMOUNT
 - DATETIME
 - INVOICEID
 - DESCRIPTN
 - DOCTYPE
 - PAYMETHOD
 - REF
 - REMITTANCE
 - USERID
 - VOUCHER
 - USERAREA

The right pane, 'Target Elements', contains a table with 15 rows:

	Field	ItemType	Default	Data Type
1	PROCESS_INVOICE_...	Level		
2	CNTRLAREA	Element		VARCHAR
3	BSR	Element		VARCHAR
4	VERB	Element		VARCHAR
5	value	Element	PROCESS	VARCHAR
6	NOUN	Element		VARCHAR
7	value	Element	INVOICE	VARCHAR
8	REVISION	Element		VARCHAR
9	value	Element	001	VARCHAR
10	SENDER	Element		VARCHAR
11	LOGICALID	Element		VARCHAR
12	COMPONENT	Element		VARCHAR
13	TASK	Element		VARCHAR
14	REFERENCEID	Element		VARCHAR
15	CONFIRMATION	Element		VARCHAR

Field

Field identifies the name of the element, document, or root. The names are based on the Application Open Interface table column names or DTD element names.

The field name can be changed if necessary, however, consider what this change implies. Because the field is based on the database column name or a DTD element name, corresponding changes may be necessary in Oracle E-Business Suite or the DTD. The only changes allowed to a DTD are to the USERAREA. Refer to How to Extend DTDs, page 2-108 for details.

The first row is reserved for the Data Definition name (if target is based on Application Open Interface) or DTD root element name (if target is a DTD).

Additional field names are displayed when sibling or child elements are added using the Add Sibling or Add Child buttons.

Item Type

Item type identifies the field as either a Level or an Element. Level represents the parent in a parent-child relationship. Element represents the child in a parent-child relationship.

The Item Type of the first row is defaulted to Level. The default value cannot be changed.

If the target is based on Application Open Interface tables, the Item Type for the tables is defaulted to "Level". The Item Type for the columns is defaulted to "Element".

If the target is based on a DTD or a production XML message, the default Item Type is "Element". DTDs do not support levels and therefore the levels must be explicitly defined by setting the Item Type to "Level".

Any node that represents a collection of data that repeats (for example, PO lines or shipment lines) represents a level. The Item Type for the node must be set to "Level".

For the OAG standard, change the Item Type of the following to "Level" to support Level Mapping:

- Root
- CNTROLAREA
- DATAAREA

Make the same change for all other DTD data types identified as data levels during the map analysis process.

The Item Type for a new sibling or child element is defaulted to "Element". Change the Item Type setting where appropriate.

Default

Enter a default value for the field as appropriate. This value is used in the outbound message. The default value will be used in an inbound message if the incoming value is null.

If the target is a DTD, use the Default column to set the DTD attribute values. The values will be displayed with the corresponding attribute tags when the message is created.

If the default value is based on a condition, set the default using the Assign Variable Value action. See *Assignments: Assign Variable Value*, page 2-74.

If Item Type is Level, this column is disabled.

Data Type

Each field is defined with a data type. The data types supported by the XML Gateway Execution Engine are VARCHAR2, DATE, NUMBER, CHAR, and CLOB.

The CLOB data type is used to support large objects up to 4 GB in size. The CLOB is displayed between CDATA tags to escape parsing.

If the target is based on Application Open Interface tables, the data type is defaulted to the data type defined for the database column.

If the target is based on a DTD or a production XML message, the data type is defaulted to VARCHAR2.

Important: It is not necessary to change the DTD element's data type until XML schemas are supported by the XML standards bodies which will make data types more meaningful.

The Data Type for a new sibling or child element is defaulted to VARCHAR2. Change the data type if necessary.

If Item Type is Level, the Data Type column is disabled.

DB Column

DB Column is available only when the target is based on Application Open Interface tables.

A check mark indicates the column is defined in the Oracle E-Business Suite data model. This setting informs the XML Gateway Execution Engine whether to validate the element against the Oracle E-Business Suite data model or not.

If the Item Type is Level, the DB Column is disabled.

Node Type

Node Type is available only when the target is a DTD. The default is the DTD setting.

The valid values are Element or Attribute. Elements contain the business data. Attributes contain qualifiers for the business data to indicate the intended meaning of the data.

The Node Type for a new sibling or child element is defaulted to "Element". Change the Node Type setting if appropriate.

If Item Type is Level, the Node Type column is disabled.

Add Sibling (button)

The Add Sibling button allows you to add new fields required to complete the map. This feature is also used to create duplicate DTD nodes such as PARTNER.

The same can be done if the target is Application Open Interface tables.

When the target is based on a DTD or production XML message, and you are adding a sibling between two attributes, set the Node Type for the new field to "Attribute". The default Node Type of "Element" will cause a parser violation.

Important: You cannot add a sibling to the root node.

Refer to How to Extend DTDs, page 2-108 for details on how to extend a DTD. Included are the naming conventions that must be followed for

the XML Parser to recognize the DTD extensions.

Add Child (button)

The Add Child button adds child elements to an existing sibling or child. In addition, Add Child can be used to define an attribute for the root element if the source or target is a DTD.

This function can be used if the target is Application Open Interface tables or a DTD.

If Node Type is "Attribute", the Add Child button is disabled. You cannot define an attribute for an attribute.

Note: Refer to How to Extend DTDs, page 2-108 for details on how to extend a DTD. Included are the naming conventions that must be followed for the XML Parser to recognize the DTD extensions.

Delete (button)

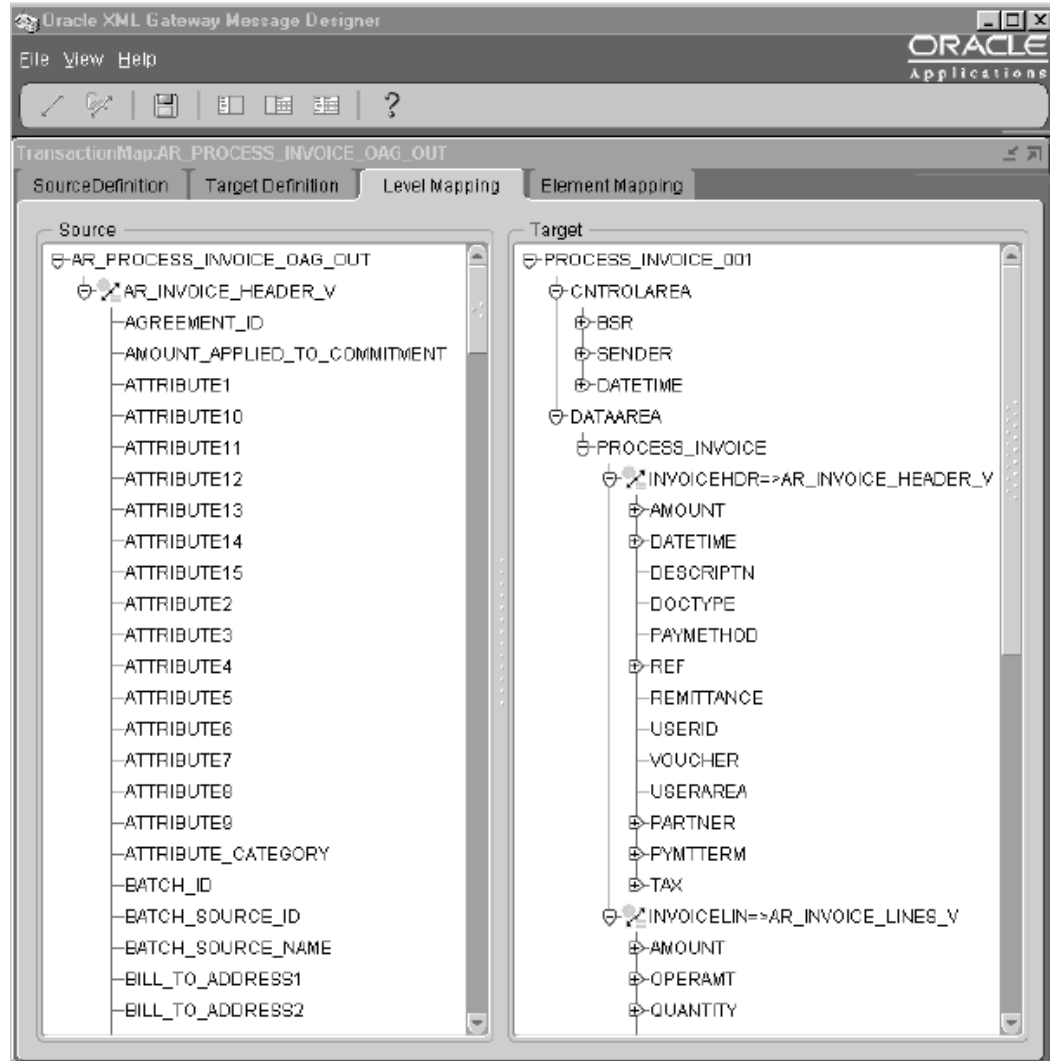
The Delete button deletes any sibling or child element that has not been mapped.

If a sibling or child has child elements associated with it, a warning is displayed before the delete occurs.

Important: If you are deleting any DTD extensions, be sure to remove the DTD extensions from the corresponding extension file created for the application or user. The extra information will not cause a parser violation, but it is a good practice to ensure the extension files match the message maps. Refer to How to Extend DTDs, page 2-108 for the details.

Transaction Map - Level Mapping Tab

Transaction Map, Level Mapping Tab



In the Level Mapping tab, the source definition is presented in the left window and the target definition in the right window. Use the Level Mapping tab to relate the source hierarchy to the target hierarchy. All entities defined as a level are displayed in bold.

Select the source level and drag it to the desired target level. The source level name and the mapped icon will appear to the right of the target level name indicating the level is mapped.

If this does not occur as described, you have not set the DTD entity Item Type to "Level". Return to the Source or Target Definition tab and set the Item Type accordingly, then resume with Level Mapping.

To unmap a mapped level, simply select the mapped level on the target window and drag it back to the source level.

Level Mapping Guidelines for OAG DTDs

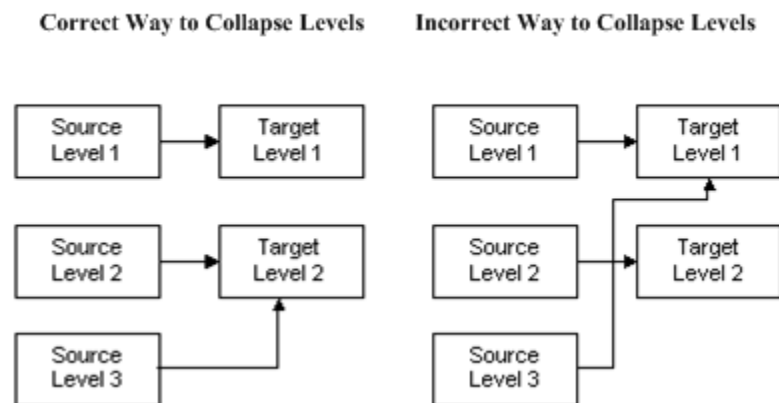
1. For the outbound messages using the OAG standard, map the ECX_OAG_CONTROLAREA_TP_V view (formerly ECX_OAG_CONTROLAREA_V) to the DTD CNTROLAREA data type. This step is not required for inbound messages because the content of the DTD CNTROLAREA is not stored in the Oracle E-Business Suite.

Note: The ECX_OAG_CONTROLAREA_TP_V view is an upgraded version of the ECX_OAG_CONTROLAREA_V view. Oracle XML Gateway supports both versions of the database view. For a detailed description of the differences, see the Note, page 2-34 , presented earlier in this chapter.

2. If you anticipate multiple documents in a message, map the database header view to the OAG document header data type. If you anticipate a single document in a message, map the database header view to the OAG DATAAREA data type.

Level Mapping Guidelines to Collapse Levels

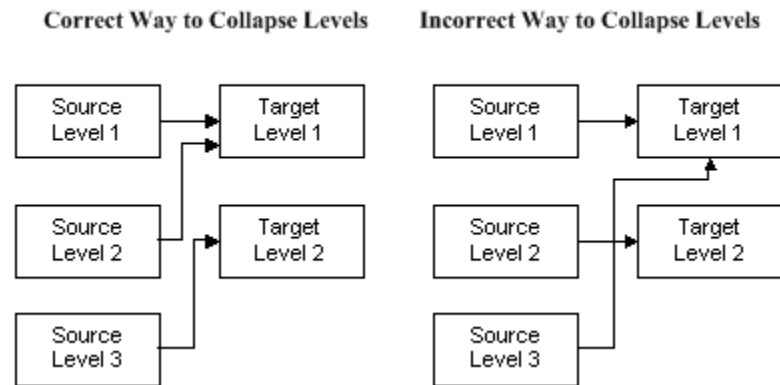
Multiple source levels can be mapped to the same target level. This is commonly referred to as collapsing levels. For example, if your source is 3 levels and your target is 2 levels you can collapse the levels as shown in the following figure:



In the correct example above, there are three source levels and two target levels. Source Level 1 is mapped to Target Level 1. Source Levels 2 and 3 are mapped to Target Level 2. The result of collapsing levels is that the data in Source Levels 2 and 3 are consolidated and mapped to Target Level 2. If there are two rows in Source Level 2 and three rows in Source Level 3, a total of six rows will be created in Target Level 2.

In the example showing the incorrect collapsing of levels, Source Level 3 is mapped over Target Level 2 to Target Level 1.

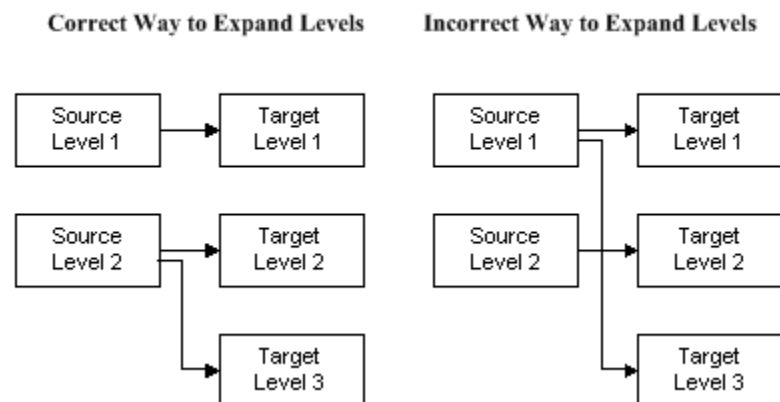
Another option shown below is to relate Source Levels 1 and 2 to Target Level 1 and relate Source Level 3 to Target Level 2. (Do **not** map Source Levels 1 and 3 to Target Level 1, crossing over Target Level 2.)



Whichever option you choose, consider what it means to promote lower level detail data to a higher level. The lower level data may need to be aggregated to be meaningful at the higher level.

Level Mapping Guidelines to Expand Levels

One source level can be mapped to multiple target levels. This is commonly referred to as expanding levels. For example, if your source is two levels and your target is three levels you can expand the levels as shown in the following figure:

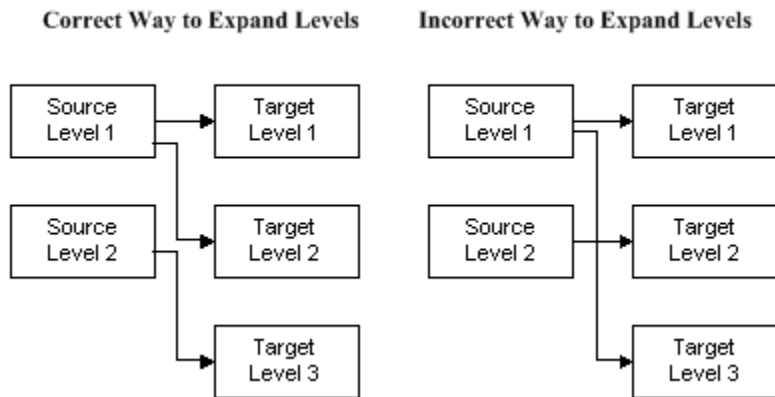


In the correct example above, Source Level 1 is mapped to Target Level 1 and Source Level 2 is mapped to Target Levels 2 and 3. The result of expanding levels is that the data in Source Level 2 is distributed and mapped to Target Levels 2 and 3. If there are two rows in Source Level 2, two rows will be created in Target Level 2 and Target Level

3.

In the example showing the incorrect expansion of levels, Source Level 1 is mapped to Target Level 1 and Target Level 3, crossing over Target Level 2.

Another option, shown below, is to distribute Source Level 1 to Target Levels 1 and 2 and map Source Level 2 to Target Level 3. (Do **not** map Source Level 1 to Target Levels 1 and 3, crossing over Target Level 2.)

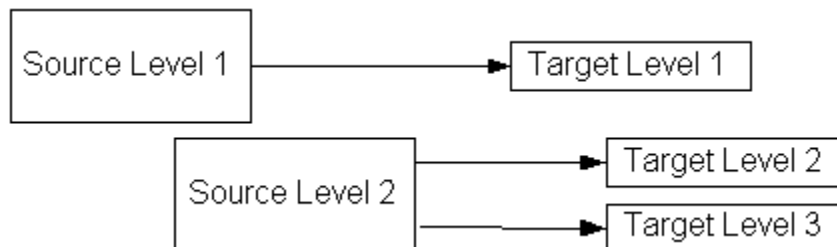


Whichever option you choose, consider what it means to demote data from a higher level to a lower level of detail. The higher level data may need to be deaggregated to be meaningful at the lower level.

Level Expansion for Discontinuous Nodes

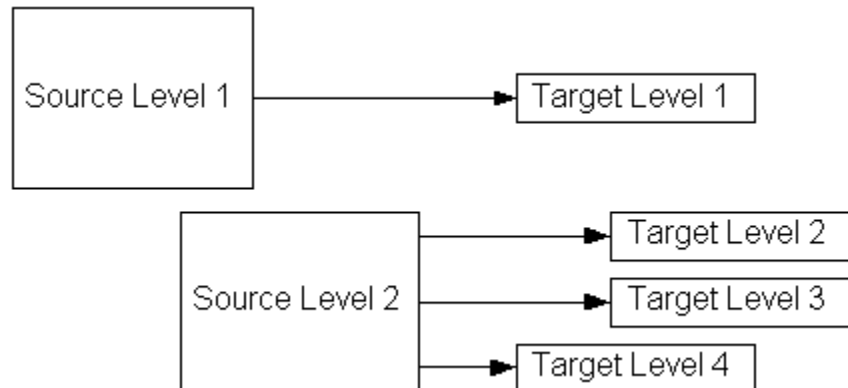
Level expansion is supported if the target expanded levels are all siblings of each other or if they are all children of the previous node.

Valid Level Expansion



In the example above, Target Level 2 and Target Level 3 are siblings to each other and children of Target Level 1.

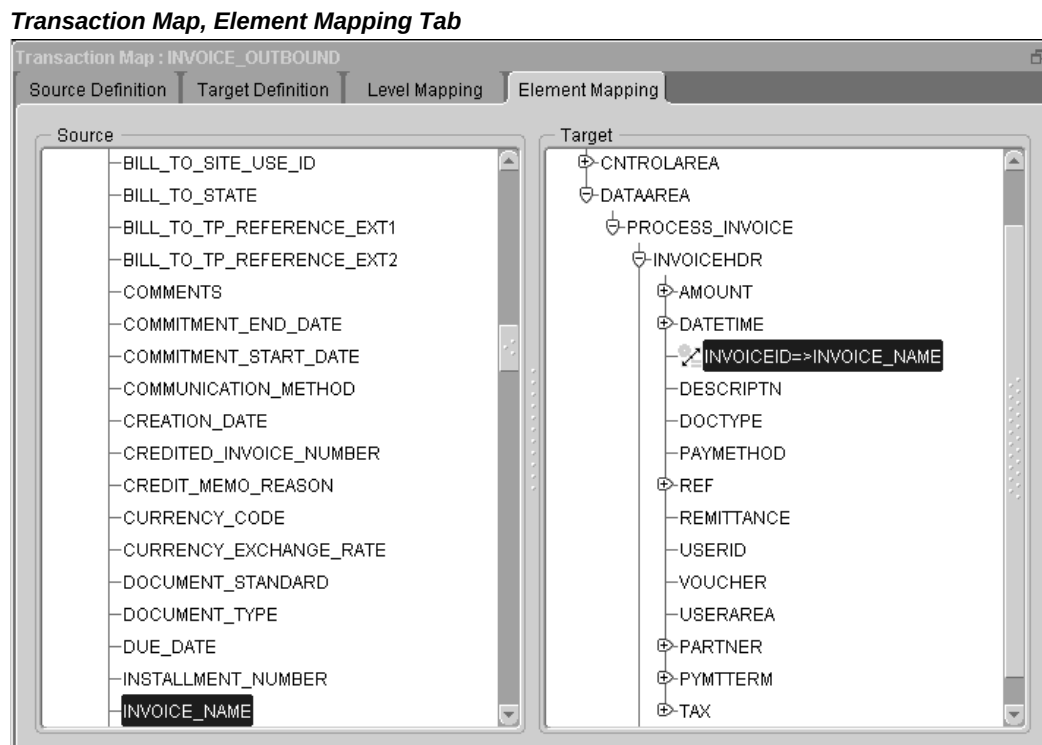
Invalid Level Expansion



In the example of invalid level expansion above, Target Level 2 and Target Level 3 are siblings to each other and children of Target Level 1. Target Level 4 is a sibling of Target Level 1, with no relationship to Target Levels 2 and 3.

See Discontinuous Nodes, page 2-44 for more information on discontinuous nodes.

Transaction Map - Element Mapping



If the Element Mapping tab is not available, it is an indication that you have not completed at least one level mapping in the Level Mapping process.

Once the Level Mapping process is complete, the source definition is presented on the left window and the target definition is presented on the right window. The source and target hierarchies defined in the Level Mapping tab are displayed in bold font. Use the Element Mapping tab to create the message map.

Select the source element and drag it to the target element. Pay special attention to any predefined node conditions (such as Conditional Node Mapping) to ensure that duplicate nodes are mapped to the correct target entities. The source element name and mapped icon will appear next to the target element name indicating the element is mapped.

To unmap a mapped element, select the mapped element on the target window and drag it back to the source element.

Element Mapping Guidelines

1. Do not map a lower level element to a higher level element. The lower level element values are not available until the header level elements are completely processed.

2. A higher level element can be mapped to a lower level element. However, you may need to manipulate the higher level element value to make it meaningful in the context of a detail level element.

For example, if the the header level element is invoice total and the line level element is invoice line total, the header invoice total must be distributed across the invoice lines based on the line quantity for it to be meaningful in the context of invoice lines.

3. One source element can be mapped to multiple target elements.
4. Review the source elements containing Conditional Node Mapping instructions. Map the source element to the target element associated with the condition.

For the outbound messages (where the source is the database and the target is a DTD) using the OAG standard, map the ECX_OAG_CONTROLAREA_TP_V view (formerly ECX_OAG_CONTROLAREA_V) columns to the DTD CNTROLAREA data type elements. This step is not required for inbound messages (where the source is a DTD and the target is the database) because the content of the DTD CNTROLAREA is not stored in the Oracle E-Business Suite.

Note: The ECX_OAG_CONTROLAREA_TP_V view is an upgraded version of the ECX_OAG_CONTROLAREA_V view. Oracle XML Gateway supports both versions of the database view. For a detailed description of the differences, see the Note, page 2-34 presented earlier in this chapter.

The view column names are similar to the DTD CNTROLAREA data type element names, so this is one-to-one mapping. Add the following required Actions to complete the map:

- Use the Convert to OAG DATETIME action to convert the Oracle date to the CNTROLAREA DATETIME element.
- Use the Append Where Clause action to bind the transaction type and transaction subtype to the ECX_OAG_CONTROLAREA_V view. Or if you are using the ECX_OAG_CONTROLAREA_TP_V view, use the Append Where Clause to bind the transaction type, transaction subtype, party ID, party site ID, and party type to the view.

Note: The ECX_OAG_CONTROLAREA_TP_V view is an upgraded version of the ECX_OAG_CONTROLAREA_V view. Oracle XML Gateway supports both versions of the database view. For a detailed description of the differences, see the Note, page 2-34, presented earlier in this chapter.

Use the Define Transactions form to define the transaction and transaction subtype

that represent the Oracle name for the message. Associated with the internal name for the message are the external type and subtype that represent the message name in the XML standard of choice.

For OAG, the external subtype corresponds to the BSR VERB and the external type corresponds to the BSR NOUN. The names entered on the Define Transactions form are stored in the database and accessed by the ECX_OAG_CONTROLAREA_TP_V view. The values are displayed in the Message Designer as default values for the BSR VERB and BSR NOUN elements.

Refer to the Define Transactions Form, page 3-11 for the details and observe the recommended naming conventions.

Element Mapping Icons

Message Designer uses an icon group to help you determine the status of a map at a glance. The components of the icon group are as follows:



Mapped Element icon.



Action Defined icon.



Code Conversion Enabled icon.

The components will be displayed as on or off (grayed out) within the group icon to indicate if the element is mapped, has an action defined, and/or is enabled for code conversion.

Element Mapping and Actions

As part of the element mapping process, Actions for data transformation and process control can be defined.

The following three sections are available for your reference. First time users should read all three sections. Experienced users should use the Map Action Editor section and reference the action type of interest.

- Transaction Map - Actions, page 2-65
 - Summary of XML Gateway Supported Actions, page C-1
 - Source or Target Action, page 2-66

- Action Levels, page 2-66
- XML Gateway Execution Engine processing sequence, page 2-66
- Map Action Editor - Overview, page 2-67
 - How to Invoke the Map Action Editor, page 2-67
 - Map Action Editor Components, page 2-68
- Map Action Editor - Available Actions, page 2-69

At the completion of the element mapping process, use the File > Save (Transaction Map) menu option or the Save icon to save the map onto the file system. The name of the map (.xgm) file should be the same as the map name for easy reference. Follow the map naming conventions as follows:

- Product mnemonic or user ID
- Transaction subtype as entered in the Define Transactions form
- XML standard and version used (such as OAG, Rosettanet, iFX)
- Outbound or inbound message

Given a map name of:

AR_INVO_OAG70_OUT

the map file name is:

AR_INVO_OAG70_OUT.xgm

Use the File > Properties menu option to change the default directory or map name if necessary.

The transaction map and the associated DTDs are now ready to load into the XML Gateway repository. The message maps will be used by the XML Gateway Execution Engine to create or to consume XML messages.

Refer to How to Load/Delete Message Maps and DTDs, page 2-111 for details on loading a message map created by the Message Designer and its associated DTDs into the XML Gateway repository.

Transaction Map - Actions

As part of the element mapping process, Actions for data transformation or process control can be defined.

Actions are similar to prebuilt functions in that they may be called to perform a specific activity.

Oracle XML Gateway supports actions for data transformation involving math functions, string manipulation, and data conversion between Oracle's data format and OAG's data format.

Oracle XML Gateway supports a set of predefined actions for process control. This includes user-defined procedure and function calls to extend the integration with the Oracle E-Business Suite. Other common process control actions allow you to inquire on the status of a transaction and to manage the process flow based on the status. For serious errors, the transaction can be aborted with error messages returned to the sender via an Oracle Workflow process.

The actions supported by Oracle XML Gateway are listed in Appendix C, page C-1.

Source or Target Action

Most Actions are defined on the target side of the Element Map with the exception of the Append Where Clause action type, which is defined on the source side of the Element Map if the source is based on database views or tables.

See Map Action Editor - Append Where Clause, page 2-76 for a detailed description of this action and how to define it.

Action Levels

An Action may be defined for any of the following entities:

- Element
An element is the smallest unit of a message. An action defined at the element level is applied to that element only.
- Document
A document is a collection of elements representing a business document. An action defined at the document level is applied to the document.
- Root
A root represents a collection of documents. An action defined at the root level is applied to all documents contained by the root.

Some actions are designed to be applied to the element only, while others are intended for the document only.

XML Gateway Execution Engine Processing Sequence

The processing sequence of a three-level document by the XML Gateway Execution Engine is described in the following table.

Refer to Pre Process, In Process, and Post Process Tabs, page 2-69, for a description of process stages.

Level	Stage
Root	Preprocess
Root	In-Process
Header	Preprocess
Header	In-Process
Header	Postprocess
Line	Preprocess
Line	In-Process
Line	Postprocess
Line Detail	Preprocess
Line Detail	In-Process
Line Detail	Postprocess
Root	Postprocess

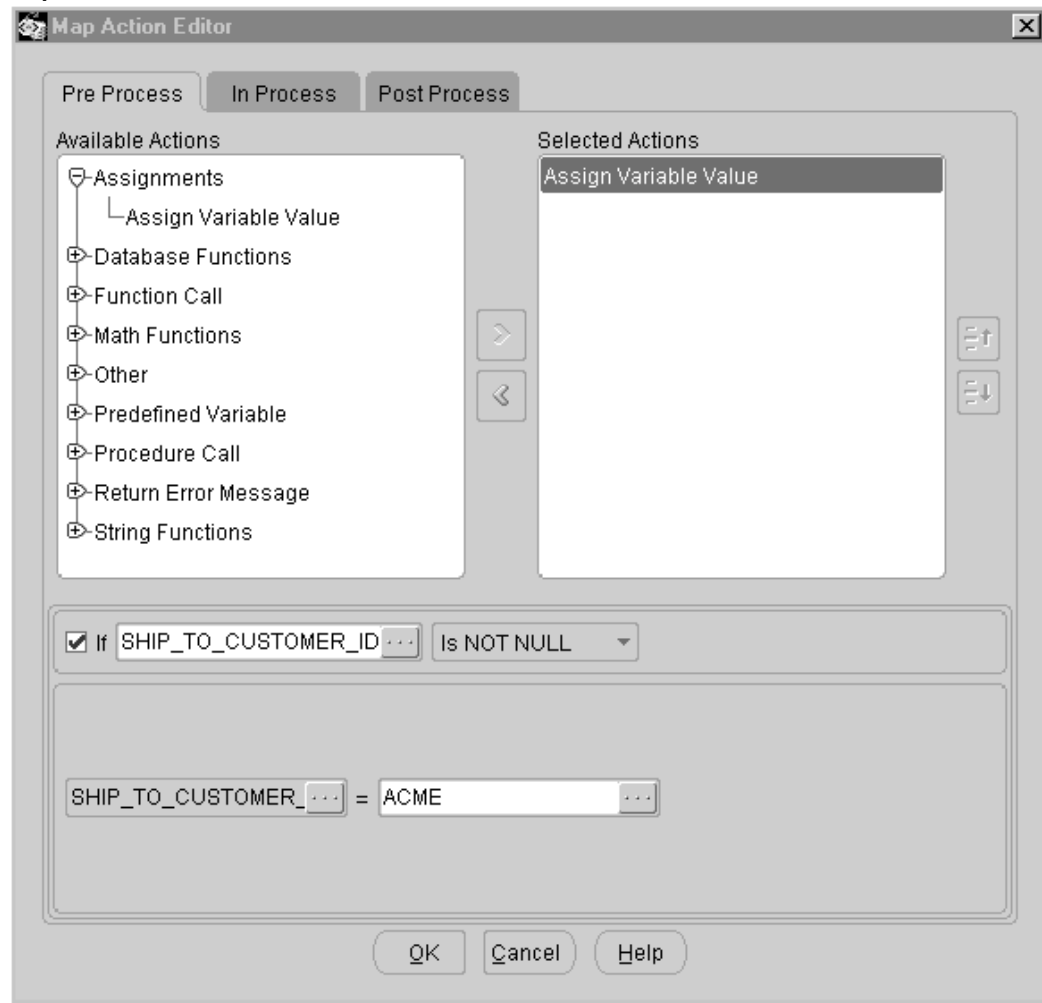
Given the processing sequence described above, header data is processed before line data and line data is processed before detail data. Any upper level data element that has a dependency on lower level data elements (for example, sum of invoice lines) must be processed using an API call at the upper level. Once the upper level data is processed, it cannot be updated, although it can always be accessed.

Map Action Editor

How to Invoke the Map Action Editor

In the Element Mapping tab, select the entity (element, document, root) and click the right mouse button to invoke the Map Action Editor pop-up window. The prompts presented by the Map Action Editor will vary based on the Action type selected.

Map Action Editor Window



Overview

The Map Action Editor has the following common components:

- Pre Process, In Process, Post Process Tabs
- Available Actions
- Selected Actions
- Optional Conditional Expression
- Result Variable
- Action Operands - Element Window

Pre Process, In Process and Post Process Tabs

An Action can be applied at any of the following stages of message creation or consumption:

- Pre Process

A preprocess action is executed before the message is created or consumed.

The Create Global Variable action is an example of a preprocess action. The variable must be defined before you can use it.

- In Process

An in-process action is executed during message creation or consumption.

The Math and String Functions are examples of in-process actions used to perform a computation or to manipulate a value.

- Post Process

A postprocess action is executed after the message is created or consumed.

The Insert into Database Table action is an example of a postprocess action. The row cannot be inserted into the database until all the data for the row has been processed.

Select the appropriate Map Action Editor Window Tab for the map entity (element, document, root) selected.

Available Actions

The available action categories for the selected process stage and selected map entity are displayed. Expand each category to view all the available action types.

The following table summarizes all action types sorted by Action Level and Action Stage. A "Y" indicates the action type is available for the Action Level and Action Stage combination. An "N" indicates the action type is not available for the Action Level and Action Stage combination.

Action Category & Description	Element In Process	Document Pre Process	Document In Process	Document Post Process	Root Pre Process	Root In & Post Process
Assignment: Assign variable value	Y	Y	Y	Y	N	Y

Action Category & Description	Element In Process	Document Pre Process	Document In Process	Document Post Process	Root Pre Process	Root In & Post Process
Assignment: Create global variable	N	N	N	N	Y	N
Database Function: Assign next sequence value	Y	Y	Y	Y	N	Y
Database Function: Append where clause	N	DB Source	N	N	N	N
Database Function: Insert into database table	N	N	N	Y	N	N
Derivations	Y	N	N	N	N	N
Function Call	Y	Y	Y	Y	N	Y
Math Functions	Y	Y	Y	Y	N	Y
OAG Standard Conversions	Y	N	N	N	N	N
Other: Exit Program	Y	Y	Y	Y	N	Y
Predefined Variable	Y	Y	Y	Y	N	Y
Procedure Call	Y	Y	Y	Y	N	Y

Action Category & Description	Element In Process	Document Pre Process	Document In Process	Document Post Process	Root Pre Process	Root In & Post Process
Return Error Message	Y	Y	Y	Y	N	Y
String Functions	Y	Y	Y	Y	N	Y
XSLT Transformation	N	N	N	N	N	Y(Post)

Selected Actions

To move an action type from Available Actions to Selected Actions, select the desired action type and click the shuttle button.

To deselect a selected action type, select the desired action type and click the shuttle button to move the selected action type from Selected Actions back to Available Actions.

Up and Down Arrows for the Selected Actions Window

A given entity (element, document, root) may have one or more Actions defined for it. The XML Gateway Execution Engine processes the Actions in the sequence defined, from the beginning of the list proceeding downward to the last action type on the list. Use the up and down arrows within the Selected Actions Window to change the sequence of the Actions defined for a given entity. Use the left shuttle button to deselect a Selected Action if necessary.

Optional Condition Expression: If <Operand 1><Operator><Operand 2>

Each Action may be defined as a conditional action (except the Append Where Clause and Create Global Variable). The condition is expressed as two operands. An operand may be a variable (source, target, or global variable) whose value is determined at runtime, or a literal value.

For each condition operand, click on the (...) icon to the right of the operand field to invoke the Element window. The Element window allows you to select a variable or provide a literal value.

Refer to Action Operands - Element Window, page 2-72 for details regarding the Element window.

The operators supported by Oracle XML Gateway are listed in the following table:

Operator	Description
=	Equal
!=	Not Equal
>	Greater Than
<	Less Than
>=	Greater Than or Equal To
<=	Less Than or Equal To
Is NULL	Null
Is NOT NULL	Not Null

Enter the two operands and select the operator if a condition expression is required.

Compound conditions can be defined by comparing two operands and storing the result in a variable (source, target, or global). You then compare the value in the variable with the next condition operand.

Result Variable

Some Actions have result variables defined and some do not. Those that do not have a result variable defined will use one of the action operands as the result variable.

The result variable is the left-most field of the action operand. The only exception occurs in the Convert To actions where the result variable is the right-most field. Per user interface standards, the result variable field is greyed, but is enabled for data entry.

For element-level Actions, the selected source or target variable is displayed as the default result variable. You can change the variable name as necessary.

For document-level or root-level actions, the result variable is blank. Click on the (...) icon to invoke the Element window. Select the source, target, or global variable as the result variable. The value of the result variable is determined at runtime.

The result variable cannot be a literal value.

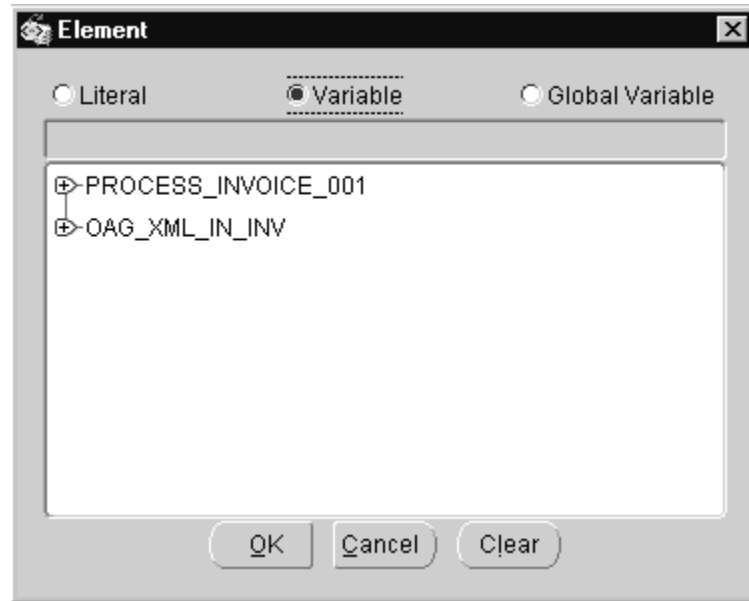
Refer to Action Operands - Element Window, page 2-72 for details regarding the Element window.

Action Operands - Element Window

An Action may or may not have operands associated with it.

For Actions that do have operands associated with them, the number of operands varies by action type. Click on the (...) icon to the right of the operand field to invoke the Element window.

Element Window



The Element window allows you to select a variable or provide a literal value. The Element window is presented in four parts as follows:

- Radio button for Literal, Variable, or Global Variable
- Operands displayed in gray indicate literal values are not allowed
- Field for literal value (available only when Literal radio button is selected)
- Window display for source and target variables

The default radio button setting is Variable. You can change the default by selecting a Literal (if available for the action type selected), or Global Variable (if defined).

The Literal radio button is enabled only if a literal value is applicable for the action type selected and the Element Window was not invoked from a Result Variable. Enter the literal value and click OK.

When the Variable radio button is enabled, you are presented with the source and target variables. Select the source or target variable name and click OK. The value for the selected variable is determined at runtime.

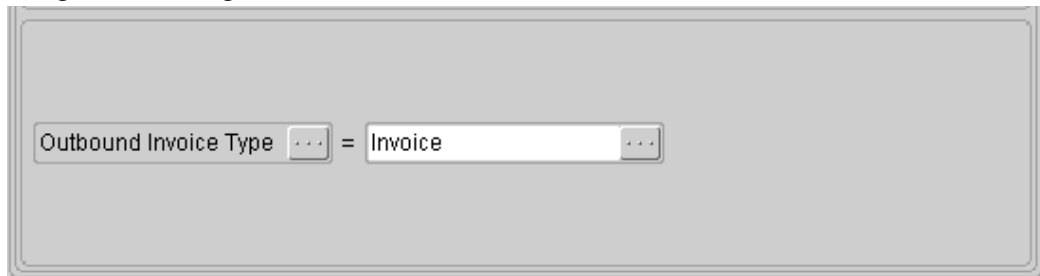
The Global Variable radio button is enabled only if global variables are defined for the message map. Select the Global Variable from the list of values and click OK. The value for the selected global variable is determined at runtime.

Map Action Editor - Assignments: Assign Variable Value

The Assign Variable Value action assigns a value to the result variable identified. The value may be based on another variable (source, target, or global variable) or a literal value.

If you are assigning a literal value as a default value to be applied universally (without a condition expression), you have the option to set the default value in the Transaction Map form using the Source or Target Definition tab.

Assignments: Assign Variable Value

The screenshot shows a window titled "Assignments: Assign Variable Value". Inside the window, there is a text input field containing "Outbound Invoice Type" followed by a small button with three dots "...". To the right of this is an equals sign "=", followed by another text input field containing "Invoice", which also has a small button with three dots "...".

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Result Variable

Select the source, target, or global variable to store the assigned value.

Action Operands

Select the variable (source, target, or global variable) or enter the literal value to assign to the result variable.

Map Action Editor - Assignments: Create Global Variable

The Create Global Variable action allows you to define a variable available to both the source and target. A default value may be assigned to the variable or the variable may get its value from the Assign Variable Value action.

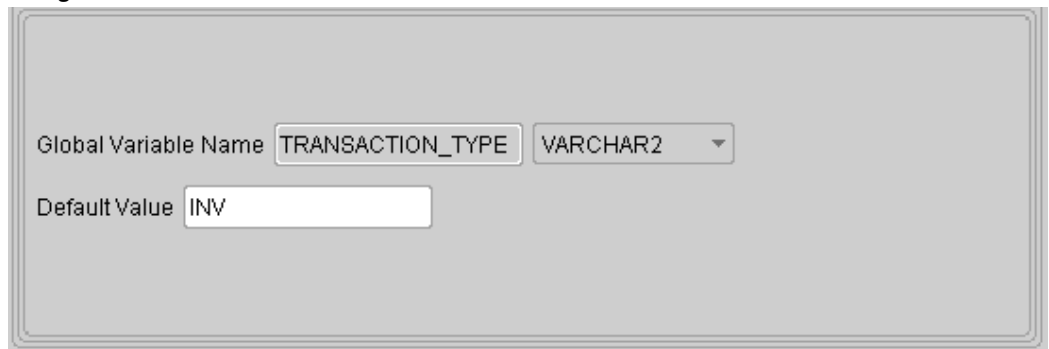
Once a global variable is defined, it is included in the Element window Global Variable Name list of values.

Note: The following are reserved names for global variables:

- TRANSACTION_TYPE
- TRANSACTION_SUBTYPE

- DOCUMENT_ID
- TP_SITE_ID (also known as PARTY_SITE_ID)
- TP_ID (also known as PARTY_ID)
- TP_TYPE (also known as PARTY_TYPE)
- PARAMETER1, PARAMETER2, PARAMETER3, PARAMETER4, and PARAMETER5

Assignments: Create Global Variable



Global Variable Name TRANSACTION_TYPE VARCHAR2 ▼

Default Value INV

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

The Optional Conditional Expression is not available for the Create Global Variable action because the action is a preprocess action and therefore data is not available yet.

Global Variable Name

Enter a global variable name and select the data type for the variable. The valid data types supported by the XML Gateway Execution Engine are VARCHAR2, NUMBER, DATE, and CHAR.

Important: Define a unique and meaningful name that does not contain a reserved word and is not a predefined variable. Once the variable name is defined and stored for the map, it cannot be changed.

However, if necessary, you can delete the Create Global Variable action containing the incorrect variable name and then add a new action containing the correct name.

Note: The CLOB data type defined to support large objects (up to 4GB) is supported by the XML Gateway execution engine, but not by the

Create Global Variable action.

Default Value

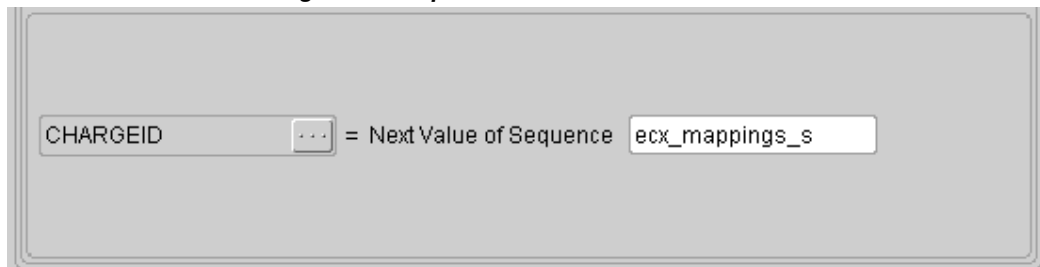
Enter a default value if applicable.

Map Action Editor - Database Functions: Assign Next Sequence Value

Oracle E-Business Suite defines database sequences to maintain counters for document numbers such as PO or invoice number. When inserting documents into the Oracle E-Business Suite Application Open Interface tables, the next available document number is required.

The Assign Next Sequence Value action retrieves the next available sequence number from the database sequence identified and assigns it to the result variable.

Database Functions: Assign Next Sequence Value



The screenshot shows a configuration window titled "Database Functions: Assign Next Sequence Value". Inside the window, there is a text input field containing "CHARGEID", followed by a small dropdown menu, an equals sign, the text "Next Value of Sequence", and another text input field containing "ecx_mappings_s".

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Result Variable

Select the source, target, or global variable to store the next value of the sequence identified.

Next Value of Sequence

Identify the database sequence name. The function will assign the next value from the sequence to the result variable.

Map Action Editor - Database Functions: Append Where Clause

The Append Where Clause action is used for outbound messages only. It is used to pass the document selection criteria to the database views used in the transaction map.

The Append Where Clause action is defined in the source at the document level as a preprocess activity. The bind variables and bind values are set up in advance of the

document being processed. The actual bind to the database views occurs as an in-process activity when the data becomes available. The default where clause of "where 1=1" is appended at runtime to dynamically construct the where clause based on the selection criteria provided.

Each Append Where Clause action accepts one set of a bind variable and a bind value. To pass multiple selection criteria, you must define multiple Append Where Clause actions.

The document selection criteria are identified in the event and event subscription.

If you are using the ECX_OAG_CONTROLAREA_V view in your map, you will need to bind to the TRANSACTION_TYPE and TRANSACTION_SUBTYPE columns for this view to execute properly.

If you are using the ECX_OAG_CONTROLAREA_TP_V view in your map, you will need to bind to the TRANSACTION_TYPE, TRANSACTION_SUBTYPE, PARTY_ID, PARTY_SITE_ID, and PARTY_TYPE columns for this view to execute properly.

Note: You must add the ECX_EVENT_MESSAGE item attribute to your Workflow item type to have access to the event details stored in the REFERENCE ID column of the ECX_OAG_CONTROLAREA_TP_V view.

The following table shows an example:

Where Clause	Bind Variable	Bind Value
and ECX_OAG_CONTROLAREA_TP_V. TRANSACTION_TYPE=:TTYPE	TYPE	TRANSACTION_TYPE
and ECX_OAG_CONTROLAREA_TP_V. TRANSACTION_SUBTYPE=:SUBTYPE	SUBTYPE	TRANSACTION_SUBTYPE
and ECX_OAG_CONTROLAREA_TP_V. PARTY_ID=:PARTY_ID	PARTY_ID	TP_ID
and ECX_OAG_CONTROLAREA_TP_V. PARTY_SITE_ID=:PARTY_SITE_ID	PARTY_SITE_ID	TP_SITE_ID
and ECX_OAG_CONTROLAREA_TP_PARTY_TYPE=: PARTY_TYPE	PARTY_TYPE	TP_TYPE
and <APPS_HEADER_V>.<document id> =: DOCID	DOCID	document_id

Note: For this method, use the Create Global Variable action to define variables to store the bind value. The global variables must be defined with the exact spelling of the parameters. In this example, the global variables are TRANSACTION_TYPE, TRANSACTION_SUBTYPE, TP_ID, TP_SITE_ID, TP_TYPE, and DOCUMENT_ID.

The following are reserved names for global variables:

- TRANSACTION_TYPE
- TRANSACTION_SUBTYPE
- DOCUMENT_ID
- TP_SITE_ID (also known as PARTY_SITE_ID)
- TP_ID (also known as PARTY_ID)
- TP_TYPE (also known as PARTY_TYPE)
- PARAMETER1, PARAMETER2, PARAMETER3, PARAMETER4, and PARAMETER5

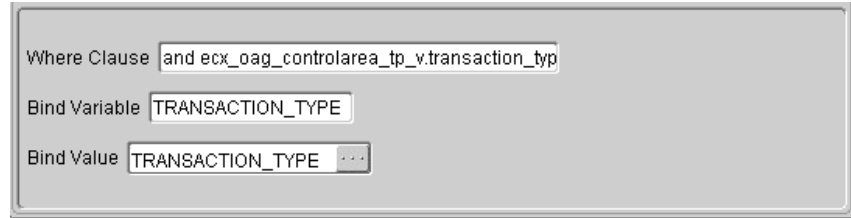
Other source variables not related to the event triggering process can be used as bind values to ensure that the correct document is selected from the Oracle E-Business Suite database.

If you know the key values, you can define them in your where clause. With this method, you do not have to use the Bind Variable and Bind Value. The following table shows an example:

Where Clause	Bind Variable	Bind Value
and ECX_OAG_CONTROLAREA_TP_V.TRANSACTION_TYPE='POO'	N/A	N/A
and ECX_OAG_CONTROLAREA_TP_V.TRANSACTION_SUBTYPE='POOB'	N/A	N/A
and ECX_PO_HEADER_V.PO_NUMBER='A754739'	N/A	N/A

Note: Literal strings must be bound by single quotes.

Database Functions: Append Where Clause



Where Clause

Bind Variable

Bind Value

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

The Optional Conditional Expression is not available for the Append Where Clause action because it is a preprocess action and therefore the data is not available yet.

Where Clause

Identify the where clause to bind the database views used for the outbound message. This where clause will be appended to the default where clause of "where 1=1".

If you have multiple selection criteria based on the same database view, define an Append Where Clause action for each of the criteria. Define all fields for the first Append Where Clause action (Where Clause, Bind Variable, and Bind Value). For the subsequent Append Where Clause actions, define only the Bind Variable and Bind Value (leaving the Where Clause blank).

Bind Variable

Identify the bind variable to be used by the where clause.

Bind Value

Identify the bind value to be assigned to the bind variable used by the where clause.

The bind value is a global variable whose value is determined at run time, or a literal value.

Map Action Editor - Database Functions: Insert into Database Table

The Insert into Database Table action is a postprocess action used for inbound messages to insert the data into Application Open Interface tables identified in the message map. This action is available only if the target is database.

This action must be executed once for each target level identified in the Transaction Map - Level Mapping tab. If the target has three levels (header, line, and line detail), then it must be executed three times.

You can identify a condition for the Insert into Database Table action. The action does

not require any operands.

The inserts are performed by the XML Gateway Execution Engine but are not committed to the database until the entire document is processed. This eliminates the possibility of inserting a partial document into the database.

Note: An alternate way to insert data is to use the Procedure Call action to execute an Application API. See How to Map to an API, page 2-110 for details.

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Map Action Editor - Derivations: Derive Address ID from Location Code

The Derive Address ID from Location Code action is used for inbound messages only. It uses the source location code and associated address type to derive the address and organization ID meaningful to the target Oracle E-Business Suite module.

The IDs are meaningful in the context of Oracle E-Business Suite only. The sender is not expected to know a valid address or organization ID.

If the parent table ID for the customer or supplier site is also required by the Application Open Interface, use the Derive Parent ID from Location Code, page 2-81 action to derive it.

Location code is commonly used to refer to the address site represented by the physical address. Using location codes eliminates the need to include the physical address in an XML message. For OAG messages, the location code is commonly found in the PARTNRID element of the PARTNER data type.

The Derive Address ID from Location Code action will access all trading partner locations across all organizations to derive the address ID. If the source location code is found in multiple organizations, the action will produce an error because it cannot uniquely identify an address ID.

Derivations: Derive Address ID from Location Code

The screenshot shows a configuration window for the 'Derive Address ID from Location Code' action. It contains four fields with dropdown menus:

- Derive Target Address ID: NAME
- Derive Target ORG ID: (empty)
- From Source Location Code: PARTNRID
- with the Address Type of: Customer

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Derive Target Address ID

Identify the target variable to store the address ID derived from the source location code. The value for the target variable is determined at runtime.

The target variable cannot be a global variable unless you are using it as a temporary variable to populate multiple application columns with the same value. The target variable cannot be a literal value.

Derive Target ORG ID

Identify the target variable to store the organization ID derived from the source location code. The value for the target variable is determined at runtime.

The target variable cannot be a global variable unless you are using it as a temporary variable to populate multiple application columns with the same value. The target variable cannot be a literal value.

From Source Location Code

Identify the source variable containing the location code. The source variable can be a global variable, whose value is determined at runtime, or a literal value.

With the Address Type of

Select the address type from the list of values. The valid address types are as follows:

- Customer
- Supplier
- Internal Location
- Bank Branch

This address type is used by the XML Gateway Execution Engine to determine the appropriate Oracle address data model to access to derive the address ID.

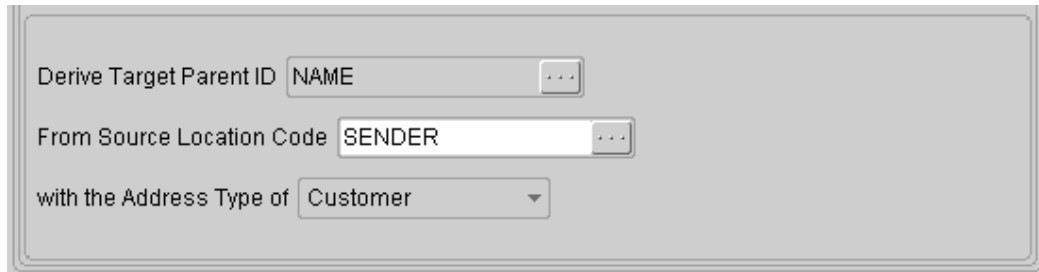
Map Action Editor - Derivations: Derive Parent ID from Location Code

The Derive Parent ID from Location Code action is used for inbound messages only. It derives the Parent ID associated with the site level source location code.

For example, given a SOLD-TO site in the XML message, this action will derive the parent customer associated with it.

Keep in mind that IDs are meaningful in the context of Oracle E-Business Suite only. The sender is not expected to know a valid parent ID.

Derivations: Derive Parent ID from Location Code



Derive Target Parent ID

From Source Location Code

with the Address Type of

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Derive Target Parent ID

Identify the target variable to store the parent ID derived from the site level source location code. The value for the target variable is determined at runtime.

The target variable cannot be a global variable unless you are using it as a temporary variable to populate multiple application columns with the same value. The target variable cannot be a literal value.

From Source Location Code

Identify the source variable containing the location code. The source variable can be a global variable, whose value is determined at runtime, or a literal value.

With the Address Type of

Select the address type from the list of values. The valid address types are as follows:

- Customer
- Supplier
- Internal Location
- Bank Branch

This address type is used by the XML Gateway Execution Engine to determine the appropriate Oracle address data model to access to derive the address ID.

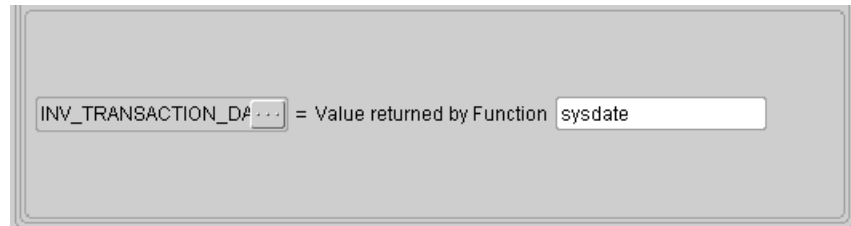
Map Action Editor - Function Call: Execute Function Call

The Execute Function action calls a system function (for example, FND_GLOBAL.USER_ID) or any application function to perform an activity and return the result to the result variable.

Function calls with parameters are not supported by the Execute Function Call action. See XML Gateway APIs, page F-1 for a list of special purpose functions.

Note: Procedure calls are not supported by the Execute Function Call action. See Map Action Editor - Procedure Call: Execute Procedure, page 2-100 for details on how to enable a procedure.

Function Call: Execute Function Call



See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Result Variable

Select the source, target, or global variable to store the function return value.

Value Returned by Function

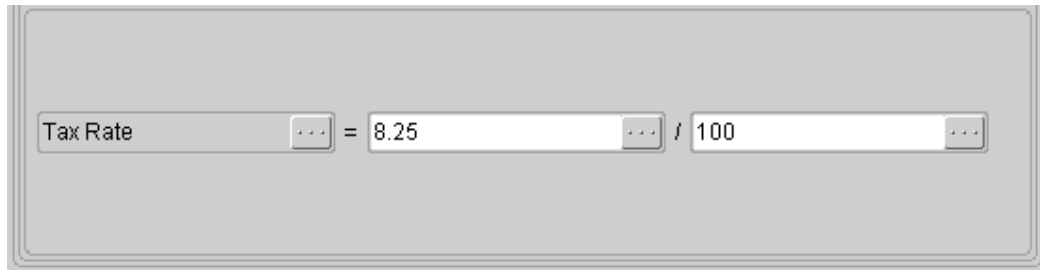
Identify the name of the Function. The value returned by the function will be assigned to the result variable.

Map Action Editor - Math Functions

Oracle XML Gateway supports the four basic math functions: addition, subtraction, multiplication, and division.

Each math function supports two operands. The Add, Subtract, Divide, and Multiply functions will each display the appropriate operators.

Math Functions

The screenshot shows a graphical user interface for defining a mathematical function. It features a sequence of input fields and operators. The first field contains the text 'Tax Rate', followed by a multiplication operator '*', then a field with the value '8.25', a division operator '/', and a final field with the value '100'. Each input field has a small square button with three dots to its right, likely for opening a selection menu.

If the mathematical expression requires more than two operands, compute the expression two operands at a time, store the result in a temporary variable (source, target, or global), and then join the values in the temporary variables.

Note: Use the Function Call or Procedure Call action to utilize database level functions for complex mathematical computations.

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Result Variable

Select the source, target, or global variable to store the result of the mathematical computation.

Action Operands

Identify the two operands and the math function desired.

Each operand may be a variable (source, target, or global variables if defined) whose value is determined at runtime, or a numeric literal value.

OAG Conversions

The availability of the OAG Conversion actions depends on the source and target. The following table summarizes when the actions are enabled:

Source	Target	OAG Actions
xml/dtd (OAG)	xml/dtd (OAG)	Both From and To actions are enabled.
xml/dtd (OAG)	database	Only From actions are enabled.

Source	Target	OAG Actions
database	xml/dtd (OAG)	Only To actions are enabled.
xml/dtd (OAG)	xml/dtd (cXML)	Only From actions are enabled.
xml/dtd (cXML)	xml/dtd (OAG)	Only To actions are enabled.
xml/dtd (cXML)	database	None of the OAG actions are enabled.
database	xml/dtd (cXML)	None of the OAG actions are enabled.

Map Action Editor - Convert to OAG DATETIME

The Convert To OAG DATETIME action converts the Oracle E-Business Suite representation for date to the OAG representation for date. This action is disabled if the target is database.

The OAG representation for date is as follows:

DATETIME (qualifier, type, index, YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, SUBSECOND, TIMEZONE)

The Convert to OAG DATETIME action converts the single column Oracle E-Business Suite date to the OAG DATETIME segment as follows:

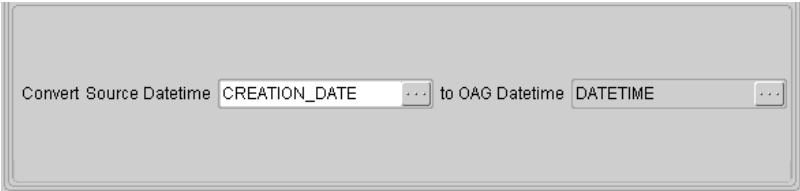
- The DATETIME attributes for qualifier, type, and index must be set as default values in the Target Definition tab of the Transaction Map window.
- The DATETIME elements for YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, and SUBSECOND are derived based on the Oracle E-Business Suite date value.
- The DATETIME element for TIMEZONE is determined as follows: The date and time data retrieved from the database, along with the value specified in the ECX: Server Time Zone profile option, are used to determine the Greenwich Mean Time (GMT) deviation. The deviation is used in the XML message generated by XML Gateway. No conversion is performed.

See XML Gateway Valid Time Zone Values, page E-1 for more information.

Note: The source and target elements are implicitly mapped via the Action. Additional element mapping will overwrite the original

Action instructions.

Convert to OAG DATETIME



Convert Source Datetime to OAG Datetime

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Convert Source Datetime

Identify the source date/time element to be converted into the OAG DATETIME format. The source can be a literal value defined using a numeric format (such as 20020808 for 08-Aug-2002). Literal dates entered using a character format (such as 08-Aug-2002) are not valid.

Target and variables do not apply although they are available in the Element window.

The converted value will be assigned to the target DATETIME element selected when the action was invoked.

Map Action Editor - Convert to OAG OPERAMT

The Convert To OAG OPERAMT action converts the Oracle E-Business Suite representation for operating amount to the OAG representation for operating amount.

The OAG representation for operating amount is as follows:

OPERAMT (qualifier, type, VALUE, NUMOFDEC, SIGN, CURRENCY, UOMVALUE, UOMNUMDEC, UOM)

The Convert to OAG OPERAMT action converts the single column Oracle E-Business Suite operating amount to the OAG OPERAMT segment as follows:

- The OPERAMT attributes for qualifier and type must be set as default values in the Target Definition tab of the Transaction Map window.
- The OPERAMT elements for VALUE, NUMOFDEC, SIGN, UOMVALUE, and UOMNUMDEC are derived based on the Oracle E-Business Suite operating amount value.
- The OPERAMT elements for CURRENCY and UOM are prompted for. If the appropriate Oracle E-Business Suite application columns are available, the values

from the columns can be mapped to the CURRENCY and UOM elements.

Note: The source and target elements are implicitly mapped via the Action. Additional element mapping will overwrite the original Action instructions.

Convert to OAG OPERAMT



Convert Source Operating Amount UNIT_STANDARD_PRICE to OAG Operating Amount OPERAMT

Source Currency USD

Source Unit of Measure EA

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Convert Source Operating Amount

Identify the source operating amount element to be converted into the OAG OPERAMT format. The source can be a literal value or based on a global variable. The literal or variable must be numeric.

Target variables do not apply although they are available in the Element window.

The converted value is assigned to the target OPERAMT element selected when the Action was invoked.

Source Currency

Identify the source currency code element if available or a literal value to be used by the Convert To OAG Action.

If an Oracle E-Business Suite application column is available, the value from the column can be mapped to the CURRENCY element.

If no Oracle E-Business Suite application column is available, the XML Gateway Execution Engine will leave the element blank.

Target and global variables do not apply although they are available in the Element window.

Source Unit of Measure

Identify the source unit of measure code element if available or a literal value to be used by the Convert To OAG Action.

If an Oracle E-Business Suite application column is available, the value from the column can be mapped to the UOM element.

If no Oracle E-Business Suite application column is available, the XML Gateway Execution Engine will default the unit of measure code to EACH as recommended by OAG.

Target and global variables do not apply although they are available in the Element window.

Map Action Editor - Convert to OAG QUANTITY

The Convert To OAG QUANTITY action converts the Oracle E-Business Suite representation for quantity to the OAG representation for quantity.

The OAG representation for quantity is as follows:

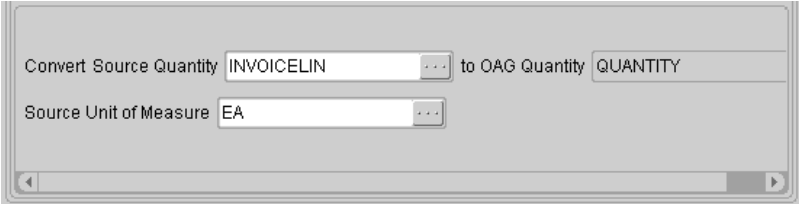
QUANTITY (qualifier, VALUE, NUMOFDEC, SIGN, UOM)

The Convert to OAG QUANTITY action converts the single column Oracle E-Business Suite quantity to the OAG QUANTITY segment as follows:

- The QUANTITY attribute for qualifier must be set as a default value in the Target Definition tab of the Transaction Map window.
- The QUANTITY elements for VALUE, NUMOFDEC, and SIGN are derived based on the Oracle E-Business Suite quantity value.
- The QUANTITY element for UOM is prompted for. If the appropriate Oracle E-Business Suite module column is available, the value from the column can be mapped to the UOM element.

Note: The source and target elements are implicitly mapped via the Action. Additional element mapping will overwrite the original Action instructions.

Convert to OAG QUANTITY



The screenshot shows a dialog box titled "Convert to OAG QUANTITY". It contains two rows of input fields. The first row is labeled "Convert Source Quantity" and has a text box containing "INVOICELIN" followed by a small button with three dots. To the right of this is the text "to OAG Quantity" followed by a text box containing "QUANTITY". The second row is labeled "Source Unit of Measure" and has a text box containing "EA" followed by a small button with three dots. At the bottom of the dialog box is a horizontal scrollbar.

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Convert Source Quantity

Identify the source quantity element to be converted into the OAG QUANTITY format.

The source can be a literal (numeric) value or based on a global variable.

Target variables do not apply although they are available in the Element window.

The converted value is assigned to the target QUANTITY element selected when the action was invoked.

Source Unit of Measure

Identify the source unit of measure code element if available, or a literal value to be used by the Convert To OAG Action.

If an Oracle E-Business Suite application column is available, the value from the column can be mapped to the UOM element.

If no Oracle E-Business Suite application column is available, the XML Gateway Execution Engine will leave the element blank.

Target and global variables do not apply although they are available in the Element window.

Map Action Editor - Convert to OAG AMOUNT

The Convert To OAG AMOUNT action converts the Oracle E-Business Suite representation for amount to the OAG representation for amount.

The OAG representation for amount is as follows:

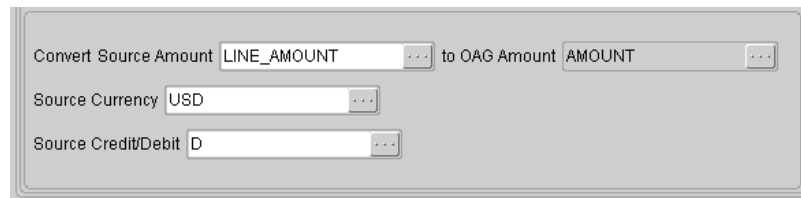
AMOUNT (qualifier, type, index, VALUE, NUMOFDEC, SIGN, CURRENCY, DRCR)

The Convert to OAG AMOUNT action converts the single column Oracle E-Business Suite amount to the OAG AMOUNT segment as follows:

- The AMOUNT attribute for qualifier, type, and index must be set as default values in the Target Definition tab of the Transaction Map window.
- The AMOUNT elements for VALUE, NUMOFDEC, and SIGN will be derived based on the Oracle E-Business Suite amount value.
- The AMOUNT element for CURRENCY and DRCR are prompted for. If the appropriate Oracle E-Business Suite columns are available, the values from the columns can be mapped to the CURRENCY and DRCR elements.

Note: The source and target elements are implicitly mapped via the Action. Additional element mapping will overwrite the original Action instructions.

Convert to OAG AMOUNT



Convert Source Amount ... to OAG Amount ...

Source Currency ...

Source Credit/Debit ...

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Convert Source Amount

Identify the source amount element to be converted into the OAG AMOUNT format. The source can be a literal (numeric) value or based on a global variable.

Target variables do not apply although they are available in the Element window.

The converted value is assigned to the target AMOUNT element selected when the action was invoked.

Source Currency

Identify the source currency code element if available or a literal value to be used by the Convert To OAG Action.

If an Oracle E-Business Suite application column is available, the value from the column can be mapped to the CURRENCY element.

If no Oracle E-Business Suite application column is available, the XML Gateway Execution Engine will leave the element blank.

Target and global variables do not apply although they are available in the Element window.

Source Credit/Debit

Identify the source credit/debit flag if available or a literal value to be used by the Convert To OAG Action.

If an Oracle E-Business Suite application column is available, the value from the column can be mapped to the DRCR element.

If no Oracle E-Business Suite application column is available, the XML Gateway Execution Engine will derive the setting based on the amount value.

Target and global variables do not apply although they are available in the Element window.

Map Action Editor - Convert from OAG DATETIME

The Convert From OAG DATETIME action converts the OAG representation for date to the Oracle E-Business Suite representation for date. This action is disabled if the source is database.

The OAG representation for date is as follows:

DATETIME (qualifier, type, index, YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, SUBSECOND, TIMEZONE)

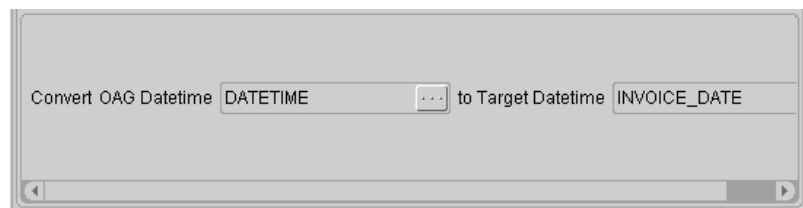
The Convert From OAG DATETIME action converts the OAG DATETIME segment to the single Oracle E-Business Suite date column as follows:

- The DATETIME qualifier attribute can be used to determine the appropriate Oracle E-Business Suite date column to use.
- The DATETIME attributes for type and index are not used by Oracle E-Business Suite.
- The DATETIME elements for YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, and SUBSECOND are used to construct the Oracle E-Business Suite date value.
- The DATETIME element for TIMEZONE is determined as follows: if the time zone of the incoming message is different from the time zone specified in the profile option ECX: Server Time Zone, the incoming date and time are converted.

See XML Gateway Valid Time Zone Values, page E-1 for more information.

Note: The source and target elements are implicitly mapped via the Action. Additional element mapping will overwrite the original Action instructions.

Convert from OAG DATETIME



See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Convert OAG Datetime

Identify the source DATETIME element to be converted into the Oracle E-Business Suite

format. The source can be a literal value.

Target variables do not apply although they are available in the Element window.

The converted value is assigned to the application (target) date element selected when the action was invoked.

Map Action Editor - Convert from OAG OPERAMT

The Convert From OAG OPERAMT action converts the OAG representation for operating amount to the Oracle E-Business Suite representation for operating amount.

The OAG representation for operating amount is as follows:

OPERAMT (qualifier, type, VALUE, NUMOFDEC, SIGN, CURRENCY, UOMVALUE, UOMNUMDEC, UOM)

The Convert From OAG OPERAMT action converts the OAG OPERAMT segment to the single Oracle E-Business Suite operating amount column as follows:

- The OPERAMT qualifier attribute can be used to determine the appropriate Oracle E-Business Suite application module operating amount column to use.
- The OPERAMT type attribute is not used by Oracle E-Business Suite.
- The OPERAMT elements for VALUE, NUMOFDEC, SIGN, UOMVALUE, and UOMNUMDEC are used to construct the Oracle E-Business Suite operating amount value.
- The OPERAMT elements for CURRENCY and UOM can be stored in Oracle E-Business Suite if the appropriate columns are available.

Note: The source and target elements are implicitly mapped via the Action. Additional element mapping will overwrite the original Action instructions.

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Convert OAG Operating Amount

Identify the source OPERAMT element to be converted into the Oracle E-Business Suite format. The source can be a literal value.

Target variables do not apply although they are available in the Element window.

The converted value is assigned to the application (target) operating amount element selected when the action was invoked.

Target Currency

The source OPERAMT value is represented as a collection of subcomponents. Identify the application (target) element to store the currency code. Literal values are not allowed.

If an application (target) element is identified, the Convert From OAG Action will move the source currency code to it.

If no application (target) element is identified, the source currency code will be ignored.

Target Unit of Measure

The source OPERAMT value is represented as a collection of subcomponents. Identify the application (target) element to store the unit of measure code. Literal values are not allowed.

If an application (target) element is identified, the Convert From OAG Action will move the source unit of measure code to it.

If no application (target) element is identified, the source unit of measure code will be ignored.

Map Action Editor - Convert from OAG QUANTITY

The Convert From OAG QUANTITY action converts the OAG representation for quantity to the Oracle E-Business Suite representation for quantity.

The OAG representation for quantity is as follows:

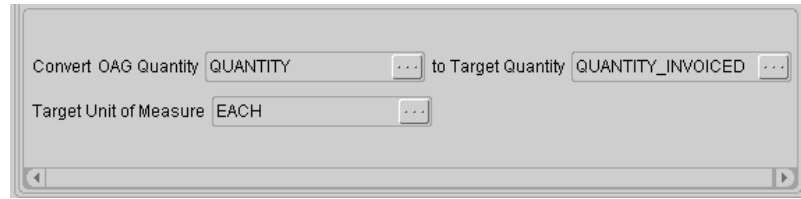
QUANTITY (qualifier, VALUE, NUMOFDEC, SIGN, UOM)

The Convert From OAG QUANTITY action converts the OAG QUANTITY segment to the single Oracle E-Business Suite quantity column as follows:

- The QUANTITY qualifier attribute can be used to determine the appropriate Oracle E-Business Suite quantity column to use.
- The QUANTITY elements for VALUE, NUMOFDEC, and SIGN are used to construct the Oracle E-Business Suite quantity value.
- The QUANTITY element for UOM may be stored in Oracle E-Business Suite if an appropriate column is available.

Note: The source and target elements are implicitly mapped via the Action. Additional element mapping will overwrite the original Action instructions.

Convert from OAG QUANTITY



Convert OAG Quantity ... to Target Quantity ...

Target Unit of Measure ...

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Convert OAG Quantity

Identify the source QUANTITY element to be converted into the Oracle E-Business Suite format.

Target variables do not apply although they are available in the Element window.

The converted value is assigned to the application (target) quantity element selected when the action was invoked.

Target Unit of Measure

The source QUANTITY value is represented as a collection of subcomponents. Identify the application (target) element to store the unit of measure code. Literal values are not allowed.

If an application (target) element is identified, the Convert From OAG Action will move the source unit of measure code to it.

If no application (target) element is identified, the source unit of measure code will be ignored.

Map Action Editor - Convert from OAG AMOUNT

The Convert From OAG AMOUNT action converts the OAG representation for amount to the Oracle E-Business Suite representation for amount.

The OAG representation for amount is as follows:

AMOUNT (qualifier, type, index, VALUE, NUMOFDEC, SIGN, CURRENCY, DRCCR)

The Convert From OAG AMOUNT action converts the OAG AMOUNT segment to the single Oracle E-Business Suite amount column as follows:

- The AMOUNT qualifier attribute can be used to determine the appropriate Oracle E-Business Suite amount column to use.

- The AMOUNT attributes for type and index are not used by Oracle E-Business Suite.
- The AMOUNT elements for VALUE, NUMOFDEC, and SIGN are used to construct the Oracle E-Business Suite amount value.
- The AMOUNT elements for CURRENCY and DRCR can be stored in Oracle E-Business Suite if appropriate columns are available.

Note: The source and target elements are implicitly mapped via the Action. Additional element mapping will overwrite the original Action instructions.

Convert from OAG AMOUNT

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Convert OAG Amount

Identify the source AMOUNT element to be converted into the Oracle E-Business Suite format. The source can be a literal value.

Target variables do not apply although they are available in the Element window.

The converted value is assigned to the application (target) amount element selected when the action was invoked.

Target Currency

The source AMOUNT value is represented as a collection of subcomponents. Identify the application (target) element to store the currency code. Literal values are not allowed.

If an application (target) element is identified, the Convert From OAG Action will move the source currency code to it.

If no application (target) element is identified, the source currency code will be ignored.

Target Credit/Debit

The source AMOUNT value is represented as a collection of subcomponents. Identify the application (target) element to store the credit/debit flag. Literal values are not allowed.

If an application (target) element is identified, the Convert From OAG Action will move the source credit/debit flag to it.

If no application (target) element is identified, the source credit/debit flag will be ignored.

Map Action Editor - Other: Exit Program

The Exit Program action can be executed based on the result of executing a Procedure call, Function call, or retrieving the value of a Predefined Variable.

The Exit Program action rolls back the current transaction and exits the XML Gateway Execution Engine.

The Exit Program action has no additional operands.

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Map Action Editor - Get Predefined Variable Value

The Get Predefined Variable Value action maintains predefined variables for the following:

- Inbound message envelope
- Code conversion status
- Transaction status

The values of these predefined variables are maintained by the XML Gateway Execution Engine and are available for inquiry and mapping to the Application Open Interface table column or Application API parameter.

You can control the process flow based on the values found in the predefined variables.

Get Predefined Variable Value

The screenshot shows a configuration window titled "Get Predefined Variable Value". Inside the window, there is a text field containing "REF", followed by an equals sign, a dropdown menu showing "Code Conversion Return Status for", and another text field containing "BILL_TO_ADDRESS1". Each of these three elements has a small square icon with three dots next to it, indicating they are clickable or expandable. The window has a standard scrollbar at the bottom.

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Result Variable

Select the source, target, or global variable to store the value of the predefined variable selected.

Predefined Variables

Select the predefined variable whose value you are interested in. The variables exposed by the XML Gateway Execution Engine are as follows:

Code Conversion Return Status for

Select the source column enabled for code conversion. The code conversion process will search the trading partner-specific code conversions and then the standard-specific list, followed by the universal list.

The status of the code conversion is assigned to the result variable. The return status codes are shown in the following table:

Return Status Code	Description
0	Code conversion was successful.

Return Status Code	Description
1	Code conversion process failed to find a match for the source column, the source value is copied to the target column. This is a warning only. It will not stop the XML Gateway Execution Engine. However, you may wish to take some action to control the process flow based on this status. The possible causes of this failure are: invalid code category passed to the code conversion API or code conversion cross-reference not found. Verify that the code category identified for the source column is valid (Refer to Seeded Code Categories, page B-1) and that a code conversion cross-reference is defined in the trading partner-specific list, the standard-specific list, or the universal list. Make the necessary corrections to prevent this error from occurring again.
2	Code conversion process encountered unexpected error, session is terminated. Check the process log file for the details, make the necessary correction, and reprocess the message.

Internal Control Number

The Internal Control Number is a system-generated number that uniquely identifies the XML message being processed. This number is useful for sending acknowledgments, for document audits, for document archival, and for troubleshooting.

The value for the Internal Control Number is assigned to the result variable identified for the action.

The Internal Control Number can be used to get envelope information. Refer to ECX_TRADING_PARTNER_PVT.getEnvelopeInformation API, page F-16 for more information.

Return Code

The Return Code is the error code associated with the error message. The error message may be returned to the trading partner contact or the XML Gateway system administrator using the Send Error Message action.

Multiple return codes can be concatenated in a single predefined variable. The value for the Return Code is assigned to the result variable.

The possible return codes are shown in the following table:

Return Code	Description
0	Success
1	Warning. The XML Gateway Execution Engine will not be stopped.
2	Failure. Check the process log file for the details, make the necessary corrections, and reprocess the message.

Return Message

The Return Message is the text associated with the return code. The error message may be returned to the trading partner contact or the XML Gateway system administrator using the Send Error Message action.

Multiple return messages can be concatenated in a single predefined variable. The value for the Return Message is assigned to the result variable.

Receiver Trading Partner ID

The Receiver Trading Partner ID is a unique identifier for the receiver of the XML message. The ID can be used to derive the trading partner name and other pertinent data.

Use the ECX_TRADING_PARTNER_PVT.get_receivers_tp_info procedure to derive the Party ID, Party Site ID, Organization ID, and Administrator e-mail Address associated with the Receiver Trading Partner ID. Refer to XML Gateway APIs, page F-1 for more information.

The Receiver Trading Partner ID is maintained for outbound messages and pass-through messages.

The value for the Receiver Trading Partner ID is assigned to the result variable.

Sender Trading Partner ID

The Sender Trading Partner ID is a unique identifier for the sender of the XML message. The ID may be used to derive the trading partner name and other pertinent data.

Use the ECX_TRADING_PARTNER_PVT.get_senders_tp_info procedure to derive the Party ID, Party Site ID, Organization ID, and Administrator e-mail Address associated with the Sender Trading Partner ID. Refer to XML Gateway APIs, page F-1 for more information.

The Sender Trading Partner ID is maintained for inbound messages and pass-through messages.

The value for the Sender Trading Partner ID is assigned to the result variable.

Organization ID

The Execution Engine derives the organization ID associated with the Sender or

Receiver Trading Partner ID. The combination of the organization ID and the Sender or Receiver Trading Partner ID uniquely identifies a trading partner in the Oracle E-Business Suite.

Map Action Editor - Procedure Call: Execute Procedure

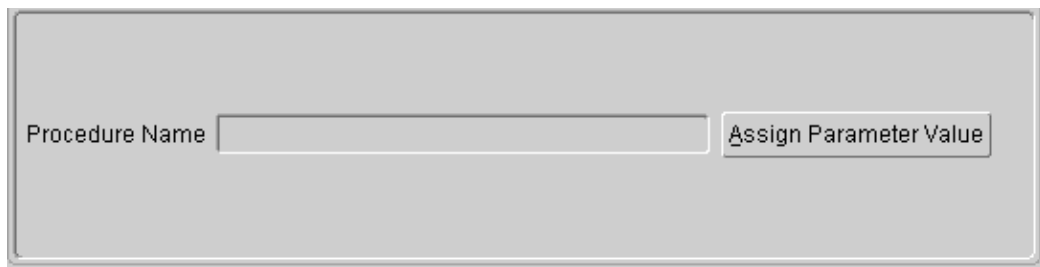
The Execute Procedure action calls a procedure to perform an activity. Some procedures have input (send) parameters as well as output (return) parameters, while others have no parameters at all.

The Execute Procedure action connects to the database to provide a list of available procedures for selection. Once you select the procedure, you will be prompted for the parameter values.

Function calls are not supported by the Execute Procedure action.

See Message Designer APIs, page F-4 for a list of special purpose procedures for use with the Message Designer.

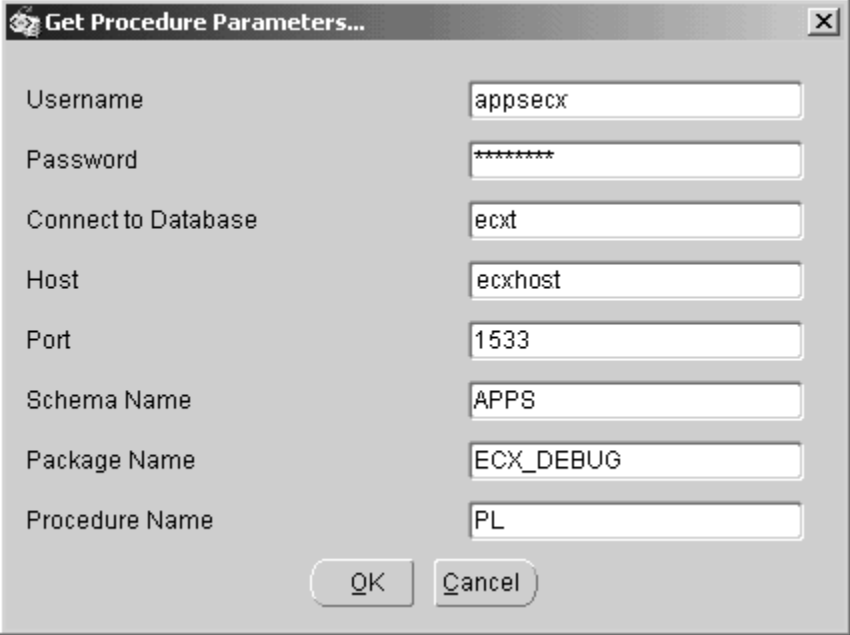
Procedure Call: Execute Procedure

The image shows a screenshot of a software dialog box titled "Procedure Call: Execute Procedure". The dialog box has a light gray background and a thin border. Inside, on the left, is the text "Procedure Name" followed by a rectangular text input field. To the right of the input field is a button with the text "Assign Parameter Value".

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Click the Assign Parameter Value button to invoke the Get Procedure Parameters window.

Get Procedure Parameters Window

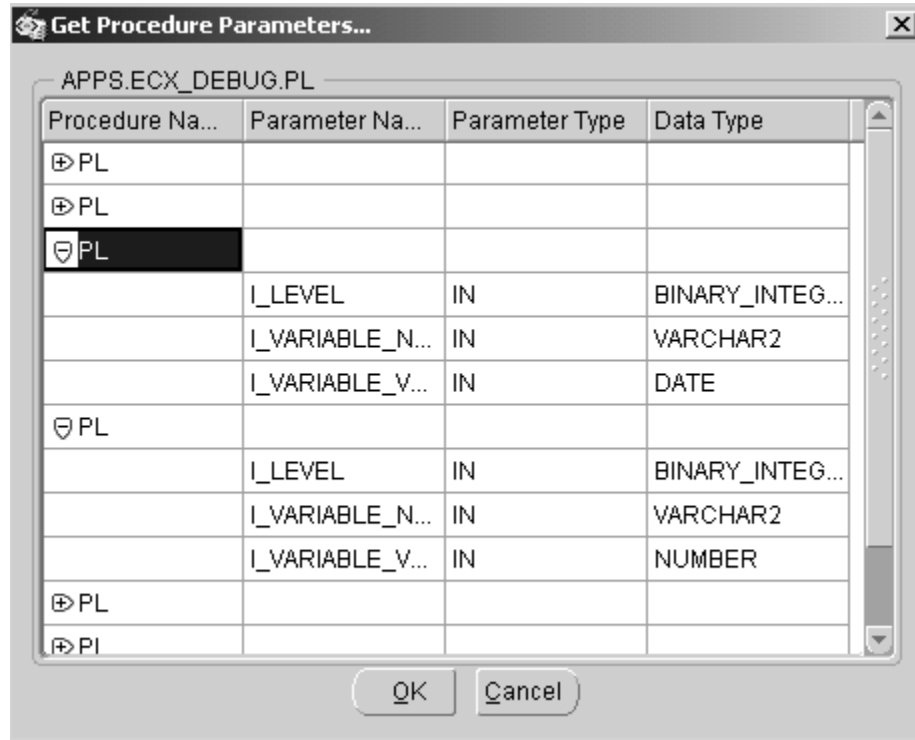
A screenshot of a Windows-style dialog box titled "Get Procedure Parameters...". The dialog box has a standard title bar with a close button (X). Inside, there are eight labeled text input fields arranged vertically. The labels are: Username, Password, Connect to Database, Host, Port, Schema Name, Package Name, and Procedure Name. The corresponding values entered in the fields are: appsecx, a masked password (seven asterisks), ecxt, ecxhost, 1533, APPS, ECX_DEBUG, and PL. At the bottom right of the dialog box, there are two buttons: "OK" and "Cancel".

Username	appsecx
Password	*****
Connect to Database	ecxt
Host	ecxhost
Port	1533
Schema Name	APPS
Package Name	ECX_DEBUG
Procedure Name	PL

This window requests database connection information. The default database access information is displayed from the Database tab, page 2-6 of the File > Properties menu. The defaults can be overwritten on this screen. Changes made on this screen are not copied back to the Properties file. Changes are used for the current session only.

Enter the Schema Name, Package Name, and Procedure Name and click OK.

Get Procedure Parameters Window Displayed with All Signatures for a Procedure



After connecting to the database, the window displays all signatures for the procedure. Expand all the signatures by clicking the "+". Select the procedure from the expanded list and click OK.

Get Procedure Parameters Window with a Selected Procedure Information

Variable Value	Parameter Name	Parameter Type	Data Type
3	I_LEVEL	IN	BINARY_INTEG...
confirmation_flag	I_VARIABLE_NAME	IN	VARCHAR2
CONFIRMATION	I_VARIABLE_VALUE	IN	DATE

Enter the parameter values in this window. A parameter value may be a literal value, from a source or target, or a global variable.

Regardless of how the parameters are defined in the procedure, a value is required (by the XML Gateway Execution Engine) for each parameter with a Parameter Type of IN unless a default value is provided by the procedure. The value can be based on a source variable, target variable, global variable (if defined), or a literal value. The value for the source, target, or global variable is determined at run time.

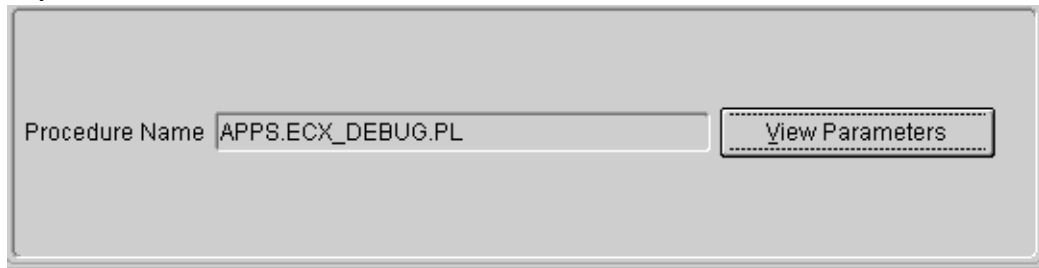
For procedures with return variables (Parameter Type of OUT), define temporary variables or use existing variables (source, target, or global) to store the return values. Status return values can be used to redirect the process flow based on severity. Return error codes and messages can be sent to the XML Gateway system administrator or Trading Partner contact using the Send Error Message action.

The XML Gateway Execution Engine supports parameters with a data type of VARCHAR2, DATE, NUMBER, CHAR, and CLOB. If the required procedure contains parameters of unsupported data types, the parameter values must be converted to a supported data type in order to utilize this action.

For parameters defined with default values, you can provide another variable or literal value if you wish to overwrite the default value.

Once you have entered all the required parameter values, click OK to return to the Map Action Editor window.

Map Action Editor with a Selected Procedure



The screenshot shows a window titled "Map Action Editor with a Selected Procedure". Inside the window, there is a label "Procedure Name" followed by a text input field containing the text "APPS.ECX_DEBUG.PL". To the right of the input field is a button labeled "View Parameters".

The procedure selected is displayed in the Procedure Name field. Click on the View Parameters button to review the parameter values entered. Verify the parameter values, make any necessary corrections, and click OK to return to the Map Action Editor window.

Map Action Editor - Return Error Message: Send Error Message

The Send Error Message action sends an error message to either the Trading Partner contact or the XML Gateway system administrator identified in the ECX_SYS_ADMIN_EMAIL system profile.

The Send Error Message Action is used for warnings that do not require the process to be terminated. For more serious errors requiring termination of the process, use the ECX_ACTIONS.set_error_exit_program API. See set_error_exit_program, page F-21 for more information about its usage.

Error messages can be obtained from the following sources:

- Return parameter of a procedure call
- Status of a function call

A Workflow notification containing the error message is sent to the party identified in the To prompt.

This action is provided to augment the standard error handling and notification process.

Return Error Message: Send Error Message



The screenshot shows a dialog box titled "Return Error Message: Send Error Message". Inside the dialog, there is a text input field containing the text "Send Error Message en..." and a dropdown menu to its right, which is currently set to "Trading Partner".

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Send Error Message

Identify the variable containing the error message to be sent. The error message can be represented as a literal.

When using this action in conjunction with the Procedure Call action, the variable may be a return parameter or a literal value.

When using this action in conjunction with the Function Call action, the variable may be the function return value or the value contained in the Return Message variable maintained by the XML Gateway Execution Engine.

You can concatenate several strings such as the value in Return Message with a literal value to form a complete message.

To

Identify the recipient of the error message. The valid options are the Trading Partner contact or the XML Gateway system administrator, identified in the ECX_SYS_ADMIN_EMAIL system profile.

Map Action Editor - String Functions: Perform Concatenation

The Perform Concatenation action concatenates the values in two operands. The operands can be based on variables whose values are determined at runtime, or based on literal values.

To concatenate more than two operands, start with the first two operands and store the result in a variable (source, target, or global). Then concatenate the value in the variable with the next operand.

The Perform Concatenation action can be applied to fields of any data type because the XML Gateway Execution Engine will convert the value to VARCHAR2 before performing the action.

String Functions: Perform Concatenation

The screenshot shows a configuration window for the 'String Functions: Perform Concatenation' action. It features a sequence of fields: a variable field containing 'AMOUNT', followed by an equals sign, another variable field containing 'AMOUNT', the text 'Concatenated with', a field containing a dollar sign '\$', and finally an empty variable field for the result.

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Result Variable

Select the source, target, or global variable to store the resulting concatenated string.

First Operand

Identify the first operand.

The operand can be a variable (source, target, or global variable if defined) whose value is determined at runtime, or a literal value.

Concatenated With

Identify the second operand to be concatenated with the first operand.

The operand can be a variable (source, target, or global variable if defined) whose value is determined at runtime, or a literal value.

The concatenated value is assigned to the result variable.

Map Action Editor - String Functions: Perform Substring

The Perform Substring action parses a given string from a start position and includes the characters within the length specified. The resulting substring is returned.

The Perform Substring action can be applied to fields of any data type because the XML Gateway Execution Engine will convert the value to VARCHAR2 before performing the action.

Refer to ECX_CONDITIONS Package APIs, page F-12 for additional substring functions.

String Functions: Perform Substring

NAME = element start position with length

See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Result Variable

Select the source, target, or global variable to store the resulting substring.

Element

Identify the operand requiring substring action.

The operand can be a variable (source, target, or global variable if defined) whose value is determined at runtime, or a literal value.

Start Position

Enter a start position greater than 0 but less than the maximum length of the Element.

With Length

Enter a length greater than 0 but less than or equal to the maximum length of the Element minus the start position.

The substring function will process the element identified from the start position up to the length specified and assign the resulting substring to the result variable.

Map Action Editor - XSLT Transformation

XML Gateway supports XSLT transformations for both inbound and outbound messages. An XSLT style sheet can be applied to an XML message before it is enqueued for delivery to a Trading Partner.

The file containing the XSLT style sheet must be stored in the directory defined by the ECX_UTL_XSLT_DIR system profile. The XML Parser applies the transformation as the last activity performed before an outbound message is sent, or as required for an inbound message is processed when different DTDs are used for the same business function. For example, an inbound message can be passed with the XSLT style sheet and transformed using ECX_STANDARD.perform_xslt_transformation API. The

transformed inbound message can then be passed for further processing by XML execution engine.

Refer to ECX_STANDARD.perform_xslt_transformation API, page F-1 for details regarding this API.

XSLT Transformation



See Map Action Editor - Overview, page 2-67 for details on the common components of the window.

Please note that if a new message map has action defined for custom XSLT transformations, then use XSL editor to create the XSLT style sheets (* .xsl). The xsl file can be staged in the file system or loaded into the database. See Loading and Deleting an XSLT Style Sheet, page 2-114.

For more information on deploying Oracle XML Gateway customizations, refer to the 'Component-Specific Steps for Application Tier Objects' section, *Deploying Customizations in Oracle E-Business Suite Release 12.2*, My Oracle Support Knowledge Document 1577661.1.

XSLT Style Sheet

The XSLT Stylesheet name can be a literal file name (such as ABC.xsl) or a Global Variable (such as PARAMETER6).

How to Extend DTDs

The XML Gateway Message Designer supports mapping to and from a DTD or an existing XML message. The use of an existing production XML message provides visibility to all the DTD extensions as well as duplicate nodes and elements necessary to support the actual business document. The use of a DTD is necessary if an existing XML message is not available. Using a DTD may require adding duplicate nodes, new segments, or elements to accommodate additional data.

When modifying an Oracle prebuilt message map or when creating a new message map that requires DTD extensions, follow the DTD extension guidelines outlined below. The Open Applications Group (OAG) has very strict guidelines regarding DTD extensions.

The only DTD extensions allowed are in the USERAREA. You can add elements in the appropriate OAG defined USERAREA or create new USERAREAs under the OAG

defined USERAREA.

Refer to the OAG guidelines section 2.11 titled "USERAREA Extensions" in the *Open Application Group's Integration Specification* (available at <http://www.openapplications.org>).

Use USERAREAs to support descriptive flexfields as well as other extensions.

The OAG guidelines are as follows:

Top Level Name	<vendor.transaction.context.USERAREA>
Element Name	<vendor.elementname>
Reference File Name	<vendor prefix>_<product>_<version>.dtd

Oracle's adaptation of the OAG guidelines is as follows:

Top Level Name for Descriptive Flexfield Extensions	<Oracle.transaction.name of flexfield. USERAREA> Example: Oracle.ARInvoice.ARLINES.USERAREA
Element Name	<Oracle.attributen> where <i>n</i> is the number of attributes defined for the table
Top Level Name for other Extensions	<Oracle.transaction.product_prefix. USERAREA> Example: Oracle.ARInvoice.AR. USERAREA
Element Name	<Oracle.elementname>
Reference File Name	Oracle_<application shortname>DTDExtensions_<version>.dtd Example: Oracle_ARDTDExtensions_001.dtd

XML Gateway provides a template to be used as the basis to create the application-specific DTD extensions. The template contains a definition for the generic flexfields named "attribute*" where * is the number of attributes defined for the table. The application-specific DTD extension files are source-controlled with the specific application.

The application-specific DTD extension file can be updated as follows:

- Top level USERAREA names representing the application table containing the descriptive flexfields

- Descriptive flexfield names
- Top level USERAREA names for other extensions
- Element names for other extensions

The updates must be added after the existing USERAREA tag.

Use Conditional Node Mapping and key on Top Level USERAREA name to ensure that you are mapping from and to the correct group of flexfields or application extensions.

Be sure to remove extension definitions from the reference file if you are removing them from the message map.

Update the message map to incorporate the new fields introduced as a DTD extension.

The updated message map and DTD extension files must be loaded in the XML

Gateway repository for access by the XML Gateway Execution Engine. Refer to How to Load/Delete Message Maps and DTDs, page 2-111 for the details.

How to Map a Pass-Through Message

Pass-through messages are messages that are received by Oracle Exchange for code conversion (if required) and then routed to a final destination. Oracle Exchange acts as a routing hub for these messages.

The map for a pass-through message is defined as follows:

Note: In a pass-through message, the same DTD is used for both the source and the target. If different DTDs are used the transaction is an XML-to-XML transformation, not a pass-through.

- Map the highest level only. The CNTROLAREA is the highest level in an OAG DTD.
- Load the message map into the XML Gateway repository for processing. Refer to How to Load/Delete Message Maps and DTDs, page 2-111 for details.
- Define the Exchange hub as the trading partner and enable the XML message as a pass-through transaction. See Trading Partner Setup, page 3-19.
- Define the necessary code conversions in Oracle XML Gateway or Oracle Exchange. See Code Conversion, page 3-38.

How to Map to an API

This section describes how to map to an API instead of an Application Open Interface Table. Data for an inbound XML message is copied to a set of staging tables known as

Application Open Interface tables. The data in these staging tables are validated using an Application Open Interface API containing the business rules. Valid data is moved to the Application base tables. Invalid data is marked for review.

In the case where the staging table and the validation rules are embedded in the same code (commonly referred to as the Application API), data for an inbound XML message is mapped to the parameters of the Application API.

To map directly to an Application API, the map must be defined as follows:

- The data definition type for the source and target must be XML.
- The source and target DTD and version must be the same.
- Map the source levels to the target levels.
- Define a target action for Execute Procedure referencing the Application API as the procedure name. The action is defined at the document level as a postprocess activity. Map the source variables to the API parameters. Include an action to check the API return status.

Apply this approach for each level of the document. If the document has three levels, this approach is applied three times.

- Load the message map and corresponding DTDs into the XML Gateway repository for processing. Refer to How to Load/Delete Message Maps and DTDs, page 2-111 to load the maps into the XML Gateway repository.
- Define the trading partner and enable the XML message.
- Define any necessary code conversions.

Message Designer does not support Application APIs consisting of multiple document levels. Message Designer is designed to process data one document level at a time.

Loading and Deleting Message Maps and DTDs

Loading/Deleting a Map

Oracle message maps are delivered and installed on the \$APPLTOP directory. They are automatically loaded into the XML Gateway repository using the LoadMap program. A map can be deleted using the DeleteMap program.

If you modified an Oracle prebuilt message map or created a new message map using the Message Designer, these maps are saved on your local file system. Perform the following steps to load the maps into the XML Gateway repository for use by the XML Gateway Execution Engine:

1. Move the <mymap>.xgm file from the local file system to the application tier. The

`apps.zip` file containing the maps that were installed on the \$APPLTOP is installed on the application tier.

2. Execute `java oracle.apps.ecx.loader.LoadMap <DB username> <DB password> <Hostname>:<Port>/<SID> <mymap.xgm>` to load the message map into the XML Gateway repository. LoadMap will replace existing maps with the same name.

Example: `java oracle.apps.ecx.loader.LoadMap User1 password host01.example.com:1521/sid01 <mymap.xgm>`

Note: The LoadMap process will load a map if the map version is compatible with the engine version. The map version is stored in the `<ECX_MAJOR_VERSION>` and the `<ECX_MINOR_VERSION>` tags of the map file (.xgm). The engine version is stored in `WF_RESOURCES`. Maps are compatible with the engine if the major version is the same and the minor version is the same or lower.

For more information on how to develop and deploy Oracle XML Gateway customizations, refer to the 'Component-Specific Steps for Application Tier Objects' section, *Deploying Customizations in Oracle E-Business Suite Release 12.2*, My Oracle Support Knowledge Document 1577661.1.

A map can be deleted using the DeleteMap program as follows:

1. Execute `java oracle.apps.ecx.loader.DeleteMap <DB username> <DB password> <Hostname>:<Port>/<SID> <mapcode>` to delete a message map from the XML Gateway repository. LoadMap will replace existing maps with the same name, but obsolete maps with a different name must be deleted manually.

`<mapcode>` is the map name entered in the Message Designer.

Example: `java oracle.apps.ecx.loader.DeleteMap User1 password host01.example.com:1521/sid01 <mapcode>`

Loading and Deleting a DTD

Each message map is associated with a set of DTDs: the main and its reference DTDs. Similar to message maps, the DTDs associated with the Oracle prebuilt message maps are delivered and installed on the \$APPLTOP. They are automatically loaded into the XML Gateway database using the LoadDTDToClob program. A DTD can be deleted using the DeleteDTDFromClob program.

To change a DTD referenced in an existing message map, use the File > Properties menu option of the Message Designer to make the necessary change. The updated message map must be loaded into the XML Gateway repository using LoadMap (refer to Loading/Deleting a Map, page 2-111). In addition, the new DTD used must be loaded into the XML Gateway database using LoadDTDToClob program as follows:

1. Move the `<mydtd>.dtd` file from the local file system to the application tier. The `apps.zip` file containing the DTDs that were installed on the \$APPLTOP is on the application tier.
2. Run `java oracle.apps.ecx.loader.LoadDTDToClob <DB username> <DB password> <Hostname>:<Port>:<SID> <mydtd.dtd> <RootElementName> <Location>` to load the DTD into the XML Gateway database. `LoadDTDToClob` will replace existing DTDs with the same name.

`<RootElementName>` is the XML Root Element entered in the Message Designer.

`<Location>` is the subdirectory name entered in the Specify XML File and Root Element window of the wizard.

A DTD can be deleted using the `DeleteDTDFromClob` program as follows:

Run `java oracle.apps.ecx.loader.DeleteDTDFromClob <DB username> <DB password> <Hostname>:<Port>:<SID> <mydtd.dtd> <RootElementName><Location>` to delete a DTD from the XML Gateway database. `LoadDTDToClob` will replace existing DTDs with the same name, but obsolete DTDs with a different name must be manually deleted.

`<RootElementName>` is the XML Root Element entered in the Message Designer.

`<Location>` is the subdirectory name entered in the Specify XML File and Root Element window of the wizard.

`LoadMap` and `LoadDTDToClob` are provided as two separate programs to support various combinations of changes, and help deploy your customized message maps and DTDs. To ensure that the maps and DTDs are synchronized, always execute `LoadMap` and `LoadDTDToClob` as a pair.

For more information on how to develop and deploy Oracle XML Gateway customizations, refer to the 'Component-Specific Steps for Application Tier Objects' section, *Deploying Customizations in Oracle E-Business Suite Release 12.2*, My Oracle Support Knowledge Document 1577661.1.

Downloading a Map

You can download a map from the ECX table to produce the corresponding XGM file. Use the `DownloadMap.java` utility to download the map to the log directory.

The usage is as follows:

```
java oracle.apps.ecx.loader.DownloadMap <DB username> <DB password> <Hostname>:<Port>:<SID> <MAP_CODE>
```

Note that there will be differences between the original map and the downloaded map.

Differences Between the Downloaded Map and the Original Map

The downloaded map and original map will differ as follows:

- If the original map has a code category that is not present in `ecx_xref_hdr`, the downloaded map will not show the code category.
- If the original map uses variables of data type other than `VARCHAR2`, `CHAR`, `NUMBER`, `CLOB`, or `DATE`, the download map will default it to `VARCHAR2`.
- If the original map was created with a Message Designer version other than 2.6.0.X, the DownloadMap utility will default it to 2.6.3.0.0.
- If the original map was created with a version of Message Designer that did not include the dbdrv hints, these hints will be present in the downloaded map.

Loading and Deleting an XSLT Style Sheet

XSLT style sheets are used to transform an XML message for rendering. The style sheets are created using an XSL editor. The `xsl` file can be staged in the file system or loaded into the database.

Note: The XML parser does not support nested style sheets using the `xsl:import` function. Define individual style sheets instead.

The `ECX:XSLT File Path` profile option identifies the file system directory where XSLT style sheets are staged. The engine checks the database first for the stylesheet, then if necessary, checks the file system.

Style sheets can be used by XML Gateway in one of two ways:

1. To be applied to an XML message generated or received by Oracle XML Gateway. This is accomplished by defining the XSLT Transformation action in the message map to reference a valid style sheet. Refer to XSLT Transformation, page 2-107 for details on how to define this action.
2. Passed with a well-formed and valid XML message to the `ECX_STANDARD.perform_xslt_transformation` API to process and return a transformed XML message for further processing by the calling environment. Refer to `ECX_STANDARD.perform_xslt_transformation` API, page F-1 for details regarding this API.

Because the engine will look for the style sheet in the database or the file system, you have the option of either loading or staging the stylesheet. To prevent accidental overwrite, a style sheet should reside in only one place.

Style sheets can be loaded into the database as follows:

1. Move the `<myxsl>.xsl` file from the local file system to the application tier.
2. Run `java oracle.apps.ecx.loader.LoadXSLTToClob <DB username> <DB password> <Hostname>:<Port>:<SID> <myxsl.xsl>`

`<Application_code> <Version>`

`<Application_code>` is the subdirectory where the XSLT style sheet is source-controlled (for example: `ar/xml/xslt`)

If `<Version>` is not specified, the file with the highest version that matches the file name, application code, and file type is loaded into the `ecx_files` table. If a matching style sheet is found in the `ecx_files` table, the row is updated. If the specified style sheet is not found in the `ecx_files` table, a new row is added to the table with version number 0.

Style sheets can be deleted from the database as follows:

```
Run java oracle.apps.ecx.loader.DeleteXSLTToClob <DB_username>
<DB_password> <Hostname>:<Port>:<SID> <myxsl.xml>
<Application_code> <Version>
```

`<Application Code>` is the name of the subdirectory where the XSLT style sheet is source-controlled (for example: `ar/xml/xslt`).

If version is not specified, the file with the highest version that matches the file name, application code, and file type is deleted from the `ecx_files` table. Run this program multiple times to delete multiple versions of a style sheet.

How to Implement Attachments in XML Messages

This topic consists of the following sections:

- Attachments and Oracle E-Business Suite, page 2-115
- Attachments and OAG Standard, page 2-116
- Attachments and Oracle Transport Agent, page 2-117
- Attachments and Outbound Documents, page 2-117
- Enable Attachments for Unit Test (Outbound), page 2-118
- Attachments and Inbound Documents, page 2-118
- Enable Attachments for Unit Test (Inbound), page 2-119

Attachments and Oracle E-Business Suite

The Oracle Foundation module offers the ability to define attachments. An attachment may be defined as short text, long text, long RAW, or BLOB data type. When the attachment is defined in FND, unique identifiers are provided so that the attachment may be correlated to the business document. Example uses of attachments are terms and conditions associated with a purchase order or images associated with catalog items.

Attachments and OAG Standard

With OAG version 7.X, the ATTCHREF data type is used to associate large objects with a business document. Attachments are associated to an XML message by FILENAME. The ATTCHREF data type is defined as follows:

FILENAME	(Required) The name of a file for reference purposes. For XML Gateway message maps, the CID is mapped to the FILENAME element.
CMPRSNTYPE	(Optional) Identifies the method used to compress or minimize the size, if a file is attached to the document. This enables the receiving application to process the file appropriately. For Oracle Foundation module, the compression type is determined when the attachment is deposited.
CMPRSNID	(Optional) Identifies the name of the compressed file. This enables the receiving application to find the file within the compressed package. For XML Gateway, the physical file is not passed. The CID identified in the message map is used by OTA to construct the message payload with its associated attachments.
DATETIME	(Optional) The creation date of the attachment.
DESCRIPTN	(Optional) A free-form description of the transaction or any portion of the transaction.
FILETYPE	(Optional) Identifies the application source or format of the data within the file. Examples are MS Excel, CSV, and AutoCAD.
NOTES1- NOTES99	(Optional)
QUANTITY	(Optional) File size of the attachment.
TITLE	(Optional) Describes the formal name of "title" of the item, person, or object.

Note: The URL option supported by OAG 7.3 is not supported by XML Gateway at this time.

The ATTCHREF data type is not required if the attachment is a text string. These attachments can be associated with an XML data type or element for text strings.

Textual attachments will be in-line with the business document. Large attachments of the BLOB data type will be associated off-line to the business document using the ATTCHREF data type as the reference.

Attachments and Oracle Transport Agent (OTA)

The OTA component of Oracle XML Gateway is used to deliver business documents containing large objects of type BLOB in a Multipurpose Internet Mail Extension (MIME) payload over HTTP or HTTPS to a Trading Partner. Similarly, inbound documents containing attachments may be sent by a Trading Partner's OTA servlet and received by the Oracle E-Business Suite user's OTA servlet.

Note: The support for large object attachments is limited to Business-to-Business document exchanges with Trading Partners delivered/received using OTA.

OTA is responsible for the following:

- Construction of the outbound XML message with a valid MIME message containing the message payload and associated attachments.
- Extraction of the message payload and associated attachments from the inbound MIME message.

Note: Oracle Transport Agent also supports MIME messages without attachments. Similarity to the message process with attachments, OTA uses the same approach to process the message, but without further processing attachments if they do not exist.

Attachments and Outbound Documents

For outbound documents, it is assumed that an attachment has been defined in the Oracle Foundation (FND) module and that it has been associated with a business document. A given business document can have any number of attachments defined at any level of the document.

To include an attachment in the generated XML message, the XML Gateway must be informed of the relationship between the attachment and the business document. This is accomplished by adding a Procedure Call action to the message map to call the register attachment API at every point in the business document where an attachment is identified.

The relationship data is maintained in the ECX_ATTACHMENT_MAPS table, which is used by OTA to construct the outbound XML message with a valid MIME message containing the message payload and associated attachments. The register_attachment API returns a unique content ID (referred to as CID) that is mapped to the OAG

ATTCHREF data type, FILENAME element, or other standards-specific equivalent.

See `register_attachment` API, page F-22 for more information.

Because the attachment, with its unique identifiers defined for it in FND, has a direct relationship with the map for the business document, any changes to the attachment (such as deletion) or to the attachment identifiers must be reflected in the message map to maintain data integrity.

Attachment content changes are reflected in the next usage of the attachment on a business document. The change will not be retroactively applied to previously generated business documents.

Enable Attachments for Unit Test for Outbound Documents

The following outlines the steps necessary to set up attachments for outbound documents:

- Define attachments in the Oracle Foundation module.
- Associate the attachment with the business document.
- Create/update the message map to call the `register_attachment` API for every off-line attachment associated with the business document.
- Map the off-line attachment (that is, BLOB data type) CID to the ATTCHREF data type, FILENAME element, or other standards-specific equivalent.
- Map the in-line attachment (that is, SHORT_TEXT, LONG_TEXT, or LONG_RAW data type) to the appropriate XML data type or element for text strings.
- Load the map and DTD to the XML Gateway repository.
- Define the Trading Partner, enable the transaction for the Trading Partner, and use protocol type OTA-ATCH, OTAHS-ATCH, HTTP-ATCH, or HTTPS-ATCH. The OTA-ATCH and OTAHS-ATCH protocol types are used when an OTA envelope is required. The HTTP-ATCH and HTTPS-ATCH protocol types are used when an OTA envelope is not required.

Attachments and Inbound Documents

For inbound documents, OTA will extract the business document from the multi-part message and enqueue it onto the ECX_INBOUND queue. It will also extract the associated attachments and deposit them into the FND module of the receiving instance. For each attachment deposited into the receiving instance, an entry is written to the ECX_ATTACHMENT_MAPS table to record the relationship between the attachment and the business document.

If the inbound business document contains a reference to an attachment via the OAG

ATTCHREF data type, FILENAME element, or other standards-specific equivalent, and the Oracle E-Business Suite module is interested in the attachment, the attachment content can be retrieved from the FND module of the receiving instance. This is accomplished by adding a Procedure Call action to the message map at every point of the business document where an attachment of interest is identified. There is no requirement to retrieve every attachment associated with an inbound business document unless the application module is interested in them.

Because the BLOB data type is handled at the database layer, the Oracle E-Business Suite module must define an API that internally calls the XML Gateway `retrieve_attachment` API. The BLOB attachment content is passed from the `retrieve_attachment` API `x_file_data` OUT parameter to the IN parameter of the application API. The application module is responsible for storing the attachment content in the appropriate column of the application table.

The receiving Oracle E-Business Suite module may change the FND attributes associated with the attachment originally deposited by OTA. This is accomplished by calling the `retrieve_attachment` API to retrieve the original attachment, and then calling the `reconfig_attachment` API to reset the FND attributes for the attachment with the application-specific identifiers.

See `retrieve_attachment`, page F-24 and `reconfig_attachment`, page F-25 for more information about these APIs.

Enable Attachments for Unit Test for Inbound Documents

The following outlines the steps necessary to set up attachments for inbound documents:

- Create an application-specific API which internally calls the `retrieve_attachment` API. This API is responsible for storing the retrieved attachment in the application column.
- Create/update the message map to check for the attachment reference (that is, OAG ATTCHREF data type, FILENAME element) and if the reference is present, call the application-specific API to retrieve each off-line attachment associated with the business document that the Oracle E-Business Suite module is interested in.
- Optionally create/update the message map to call `reconfig_attachment` API for previously retrieved attachments to reset the FND attributes.
- Load the map and DTD to the XML Gateway repository.
- Define the Trading Partner, enable the transaction for the Trading Partner, and use protocol type OTA-ATCH, OTAHS-ATCH, HTTP-ATCH, or HTTPS-ATCH. The OTA-ATCH and OTAHS-ATCH protocol types are used when an OTA envelope is required. The HTTP-ATCH and HTTPS-ATCH protocol types are used when an OTA envelope is not required.

XML Gateway Setup

This chapter covers the following topics:

- Setup Overview
- Define System Profile Options
- Assign XML Gateway Responsibility
- Define Database Directories
- Hub Definitions Form
- Define XML Standards Form
- Define Transactions Form
- Define Lookup Values
- Trading Partner Setup
- Code Conversion
- Trading Partner Code Conversion Form

Setup Overview

Implementation Checklist

There are ten setup steps for the XML Gateway as shown in the following table:

Step	Completed	Task
1		Define System Profile Options, page 3-3
2		Assign XML Gateway Responsibility, page 3-8

Step	Completed	Task
3		Define Database Directories, page 3-8 (performed by a DBA)
4		Define Hubs, page 3-8
5		Define XML Standards, page 3-10
6		Define Transactions, page 3-11
7		Define Lookup Values, page 3-17
8		Define Trading Partners, page 3-19
9		Set up Trading Partner Code Conversion, page 3-48
10		Define Standard Code Conversion, page 3-47

Note: Additional setup steps are required for Web services. See the Web Services chapter for details.

XML Gateway Forms

There are seven forms in the XML Gateway that you will use to complete the setup steps:

- System Profile Options, page 3-3
- Hub Definitions, page 3-8
- Define XML Standards, page 3-10
- Define Transactions, page 3-11
- Oracle XML Gateway Lookups, page 3-17
- Define Trading Partner Setup, page 3-19
- Define Code Conversion, page 3-47

Define System Profile Options

The XML Gateway uses profile options to define the following for your system:

- The directory path for XML messages and log files
- The directory path for XSLT style sheets
- The XML Gateway System Administrator's e-mail address
- The sender's information system
- The time zone for the database server
- The maximum size an outbound document may reach before validating whether parsing should be continued
- The maximum memory available for the database session running the Java portion of the XML transformation
- The option of either purging all XML Gateway data along with workflow data or only purging workflow data
- The continuation of parsing XML document if an outbound document reaches its maximum size
- The Trading Partner user security feature for inbound transactions
- The XML Gateway engine processing of inbound messages either in Java or in PL/SQL

To define these profile options, you must sign on to Oracle E-Business Suite as the System Administrator responsibility. Navigate to the System Profile Values form using the path (N) Profile > System.

For additional information on setting profile options and profile categories, see User Profiles and Profile Options in Oracle Application Object Library, *Oracle E-Business Suite Setup Guide*.

The following table lists the XML Gateway system profile options:

Profile Option	Description	Category	Required	Default Value
ECX: Log File Path	Log File Path where the XML messages and runtime log are stored	Deployment	YES	None

Profile Option	Description	Category	Required	Default Value
ECX: XSLT File Path	XSLT Path where XSLT style sheets are stored	Deployment	YES	None
ECX: System Administrator Email Address	XML Gateway System Administrator e-mail address	Administrator Setup	YES	None
ECX: OAG_LOGICALID	Identifier for Sender's Information System	Administrator Setup	NO	None
ECX: Server Time Zone	The time zone in which the database server is running. See Important, page 3-6 below.	Administrator Setup	YES	Null
ECX: Maximum XML Size	This profile option specifies the maximum size of an outbound XML document (in bytes). For more information on how this profile option works, see Note, page 3-7 below.	XML Processing	NO	2 MB
ECX: XML Validate Flag	This profile option specifies whether an outbound document should continue to be parsed by the engine after the maximum size defined in the 'ECX: Maximum XML Size' profile option has been reached. This profile option works together with the 'ECX: Maximum XML Size' profile option. For more information on how this profile option works and how to disable the XML parsing and validation, see Note, page 3-7 below.	XML Processing	NO	Y
ECX: Enable User Check for Trading Partner	This profile option controls whether an inbound transaction can be enabled based on the value set in the profile option and the association between a user and Trading Partner.	Security/Compatibility	NO	NO

Profile Option	Description	Category	Required	Default Value
ECX: Notification Timeout in Minutes	This profile option sets the timeout period for a notification that is waiting for a response. If the notification timeout value is exceeded, XML Gateway execution engine will automatically reprocess the erred transaction if it does not exceed the maximum retries limit.	XML Processing	NO	NO
ECX: Maximum Reprocess Attempts For Errored Inbound Transaction	This profile option governs the number of times that XML Gateway execution engine will automatically reprocess the erred transaction after the notification timeout period.	XML Processing	NO	NO
ECX: Purge ECX data with WF	This profile controls purging of XML Gateway data when Purge Obsolete Workflow Runtime Data concurrent program is submitted to purge XML Gateway transactions attached to workflow. • If the profile value is set to 'Y' (default), then all XML Gateway data along with workflow data will be purged. • If the profile value is set to 'N', then XML Gateway data will not be purged with workflow data, but only workflow data will be purged. See Purge Obsolete Workflow Runtime Data Concurrent Program, page 6-61.	Exportable	NO	Y

Profile Option	Description	Category	Required	Default Value
ECX: Java Memory Limit	This profile option allows you to increase the maximum memory (in megabytes) available for the database session running the Java portion of the XML transformation at the site level. Note: With the increase of maximum memory while processing XML messages, it not only helps avoid Java out of memory errors (Java heap space exception), but also helps smooth XSLT transformation and XML message processing.	XML Processing/Exportable	NO	1024 MB
ECX_IN_JAVA_PROCESSING	This profile option determines if inbound messages will be processed in Java or in PL/SQL. - If the profile value is set to Yes, inbound messages will be processed in Java. If the message was received from WF function activity, then set the return status as Notified. - If the profile value is set to No, inbound messages will be processed in PL/SQL.	XML Processing	YES	YES

Important: The valid values for ECX: Server Time Zone are listed in Appendix E, page E-1. If this profile option is not set, the time zone ID will default to Greenwich Mean Time (GMT).

Note: While parsing the outbound document, XML engine will check the following two profile options to determine if the outbound document will continue to be parsed:

- The 'ECX: Maximum XML Size' profile value to obtain the

maximum size that an outbound XML document should be parsed.

If the maximum size is not set, the default maximum value will be used.

- The 'ECX: XML Validate Flag' profile value only if the outbound document size is greater than the maximum size.

If the document is less than the maximum size, then the parsing will be continued regardless of the profile value set in the 'ECX: XML Validate Flag'.

If the document is greater than the maximum size and the 'ECX: XML Validate Flag' profile value is also set to 'Y', then the parsing will be continued.

For example, while parsing the outbound document, if an outbound XML document has size greater than the maximum 2MB, then the 'ECX: XML Validate Flag' profile value will be checked. If the validate flag is also set to 'Y', then the document will continue to be parsed. However, if the document reaches the maximum size, but the validate flag is set to 'N', then the parsing will not be continued.

Disabling XML Parsing and Validation: Because the 'ECX: XML Validate Flag' profile option works together with the 'ECX: Maximum XML Size' profile option, to disable XML parsing and validation for outbound messages, you not only need to set the 'ECX: XML Validate Flag' profile value to 'N', but also need to set the 'ECX: Maximum XML Size' profile to a value that is low enough so that your outbound document size can easily surpass it. This way, when the document size is greater than the maximum size you defined, and you have the 'ECX: XML Validate Flag' profile set to 'N', then the parsing and validation will be disabled. The XML engine will not parse the document and perform the XSL transformation.

Setting the 'ECX: XML Validate Flag' profile value to anything other than 'Y' will turn off parser validation for a document once the maximum size has been parsed. Turning off the validation avoids Out of Memory errors for large documents. If validation is turned off, the XSLT Transformation action cannot be performed. For more information on the XSLT Transformation action, see [perform_xslt_transformation](#), page F-1.

For additional information on setting profile options and profile categories, see *User Profiles and Profile Options in Oracle Application Object Library, Oracle E-Business Suite Setup Guide*.

Assign XML Gateway Responsibility

Use the System Administrator responsibility to assign the Oracle XML Gateway responsibility to a user to access the Oracle XML Gateway database and forms. Use standard procedures to assign the responsibility.

For additional information on defining responsibilities, see the *Oracle E-Business Suite Security Guide*.

Define Database Directories

Perform this step in cooperation with a Database Administrator (DBA).

Define Database Directories

To use Oracle XML Gateway, you must first define database directories where the XML message process log and XSLT style sheets will be stored. Oracle XML Gateway uses the UTL_FILE package to read and write to the server. UTL_FILE can only write to accessible directories.

See My Oracle Support Knowledge Document 2525754.1, *Using UTL_FILE_DIR or Database Directories for PL/SQL File I/O in Oracle E-Business Suite Releases 12.1 and 12.2*, for more information on defining database directories for PL/SQL file I/O for Oracle E-Business Suite.

In addition, the values defined in the Oracle XML Gateway profile options ECX_UTL_LOG_DIR File Path (ECX: Log File Path) and ECX_UTL_XSLT_DIR File Path (ECX: XSLT File Path) must each match a defined database directory for file I/O.

Refer to Define Profile System Values, page 3-3 for details.

Hub Definitions Form

Before defining a hub, you must first complete these setup steps:

1. Define System Profile Values, page 3-3
2. Define XML Gateway Responsibility, page 3-8
3. Define Database Directories, page 3-8 (performed by a DBA)

Navigate to the Hub Definitions form from the XML Gateway Responsibility by selecting Setup > Define Hubs.

A hub is an integration point within your network (either your intranet or the internet). Hubs are typically used to route documents to and from trading partners. Oracle Exchange is an example of a hub. You may also decide to create a private hub within your intranet through which all your ERP systems communicate.

The Hub Definitions form is used to define the hub and the authorized users conducting business via the hub. The hub users entered in this form will appear on the Trading Partner Setup form, page 3-19.

Name (Required)

Enter the Hub name.

Protocol Type (Required)

Protocol Type is the communication protocol associated with the hub, such as SMTP or HTTP. Select a value from the seeded list of values. The description for the protocol type is displayed.

Protocol Address

When protocol type is HTTP or HTTPS, protocol address is prompted.

Protocol Address is the complete URL (including service/servlet) where the Transport Agent, page 7-1 will attempt to post the XML Document.

If the Protocol type is SMTP, the protocol address is an e-mail address.

Hub Users**Username (Required)**

Enter the user name of the trading partner conducting business via the hub.

Password (Required)

Enter the password for this user. The encrypted password is stored in the database. The password is not echoed when it is entered. You will be prompted on the same field to confirm the password.

Hub Entity Code (Required)

Enter the hub entity code for this user.

Hub Entity Code has the same function as the Source Trading Partner Location Code, page 3-28 in the Trading Partner Setup form, page 3-19. It is the code found in the XML envelope to identify the source of the message.

If you are sending and receiving messages from a trading partner, you will have a mixture of your location codes and your trading partner's location codes depending on the direction of the message. For example, if you are sending messages out, the Source Trading Partner Location Code is your location code because you are the source of the XML message. If you are receiving messages, the Source Trading Partner Location Code is your trading partner's location code because they are the source of the XML message.

This code is examined for inbound messages during Trading Partner validation. When

placed on an outbound message, the recipient will validate it as a valid source location code.

The Hub Entity Code is placed in the PARTY ID field in the XML Gateway envelope, page 4-8.

Define XML Standards Form

Before defining XML standards, you must first complete these setup steps:

1. Define System Profile Values, page 3-3
2. Define XML Gateway Responsibility, page 3-8
3. Define Database Directories, page 3-8 (performed by a DBA)

Navigate to the Define XML Standards form from the XML Gateway Responsibility by selecting Setup > Define XML Standards.

This form defines standards bodies for XML messages, such as OAG.

The Standard Code entries made in this form will appear in a list of values for the Standard Code field in the following forms:

- Define Transactions form, page 3-11
- Trading Partner Code Conversion form, page 3-48 (accessed through the Trading Partner Setup form)
- Standard Code Conversion form, page 3-47

The Define Transactions form identifies the creating organization of the DTD, and hence, the XML message structure. The values entered in the Define XML Standards form will be the list of values for the Standard Code field in the Define Transactions form.

In the Trading Partner Code Conversion form and the Standard Code Conversion form, this value is used as part of the search key to access the code conversion tables.

Standard Code will also be the default standard code used in Standard Code Conversion table searches, when the Standard Code is assigned to the entries in the Define Transactions form.

Standard Code (Required)

This field is the name or code for the standard's body for the XML messages.

Only XML Standards are entered in this form with the exception of the value UNIVERSAL. The seeded value UNIVERSAL is needed for the Standard Code Conversion form to identify universally used codes, such as ISO codes.

Standard Type

Standard Type is any appropriate value that you choose.

Description

Enter a description of the standard.

Define Transactions Form

Navigate to the Define Transactions form from the XML Gateway Responsibility by selecting Setup > Define Transactions.

Use the Define Transactions form to define the transactions that will be used by the XML Gateway Execution Engine. You will then associate these transactions with a trading partner in the Trading Partner Setup form.

The Define Transactions form provides the following:

- A cross-reference between the external transaction identifiers and the internal Oracle transaction identifiers.
- Identification of the queue from which to retrieve inbound messages

Cross-Reference Transaction Identifiers

This form provides a cross-reference between internal Oracle transaction identifiers, represented by the Transaction Type and Transaction Subtype, and External Transaction Types and External Transaction Subtypes.

Note: The data element pair of External Transaction Type and External Transaction Subtype, and the data element pair of Transaction Type and Transaction Subtype are not a one-to-one cross-reference by similar data element names. It is the combination of each pair that is significant to identify the external representation of the XML message, or the internal representation to identify the transaction to the Oracle E-Business Suite.

For example: The following is an inbound message with a Party Type of "CUSTOMER": OAG PROCESS_INVOICE_003. The message has the External Transaction Type "PROCESS" and External Transaction Subtype "INVOICE." The inbound direction is determined by the XML Gateway.

The External Transaction Type "PROCESS" and External Transaction Subtype "INVOICE" will be matched to Transaction Type and Transaction Subtype equal to AP and INI as they are defined in the Define Transactions form. For this trading partner, the message map defines on which tables in Oracle Payables to place the inbound data.

Another example is an outbound message with a Party Type of "SUPPLIER".

Invoice transaction from Oracle Receivables has the Transaction Type "AR" and the Transaction Subtype "INO." The outbound direction is determined by the XML Gateway.

This Transaction Type and Transaction Subtype will be matched to the External Transaction Type "INVOICE" and External Transaction Subtype "PROCESS" as defined in the Define Transactions form for the specified trading partner. For this trading partner, the message map identifies what data to extract from Oracle Receivables.

The Party Type, Transaction Type, and Transaction Subtype are key codes used by Oracle E-Business Suite to integrate with the Workflow Business Event System (BES) to trigger message creation or message consumption.

Queues

Different queues can be defined for various transactions from the application, or for XML messages to be transmitted. The queue names are assigned to the transactions via this form.

See Message Queues, page 5-1.

Define Transactions Form Fields

Party Type (Required)

Party Type defines the type of trading partner, such as Supplier, Customer, Bank, or internal locations (such as warehouses). Select a value from the list of values.

Transaction Type (Required)

Transaction Type is the product short name for the base Oracle E-Business Suite application associated with the transaction, such as "AR" for Oracle Receivables. Refer to Transaction Type and Transaction Subtype Naming Conventions, page 3-14.

If you are using OAG standards, refer to Setting VERB and NOUN in OAG Standards, page 3-17.

Transaction Subtype (Required)

Transaction Subtype is a code for a particular transaction within the application specified by the Transaction Type. The last position of the code represents the direction of the transaction: "I" for inbound, "O" for outbound.

See Transaction Type and Transaction Subtype Naming Conventions, page 3-14.

The combination of the Transaction Type and the Transaction Subtype identifies an Oracle transaction with which to associate this message. This data will appear on the Trading Partner Setup form.

If you are using OAG standards, refer to Setting VERB and NOUN in OAG Standards,

Transaction Description

Enter a description for the transaction.

Transaction Owner

You can optionally enter a transaction owner's user name. A transaction owner is considered as an expert for a specified transaction type defined in this form.

When an error occurs during a transaction, in addition to the XML Gateway system administrator for system or process errors and Trading Partner contact for data errors, Oracle Workflow can send a notification to a transaction owner if it is specified here based on the transaction type of an erred inbound transaction. For example, if an inbound transaction error occurs in Purchasing transaction type, then the transaction owner of the Purchasing if defined here will receive a notification in addition to the XML Gateway system administrator or Trading Partner contact depending on the error type.

When notifications are sent to the XML Gateway system administrator and the transaction owner if specified, both of them can take actions in response to the error notifications if necessary. See: XML Gateway Error Processing Item Type, page 6-27.

Standard Code (Required)

The XML standard to be used for this transaction. The Standard Codes are set up in the Define XML Standards form, page 3-10. Choose the code from the list of values.

Direction (Required)

Direction indicates if the message is inbound or outbound. Select "IN" for inbound messages, or "OUT" for outbound messages from the list of values.

External Transaction Type (Required)

External Transaction Type is the primary external identifier for the XML message.

The combination of the External Transaction Type and the External Transaction Subtype should cross-reference this message to the Oracle internal transaction identified by the Transaction Type and the Transaction Subtype.

External Transaction Subtype (Required)

External Transaction Subtype is the secondary external identifier for the XML message.

The combination of the External Transaction Type and the External Transaction Subtype should cross-reference this message to the Oracle internal transaction identified by the Transaction Type and the Transaction Subtype.

Queue (Required for Inbound Messages)

A queue is a table in a database where transactions are staged for processing. Default

queues are defined during installation. Select a queue from the list of values.

The field is disabled for outbound messages.

See Message Queues, page 5-1.

Note: Only queues with a prefix of ECX will display in the list of values.

Transaction Type and Transaction Subtype Naming Conventions

Naming conventions for Transaction Type and Transaction Subtype are necessary to facilitate the reading of audit trails and for troubleshooting across applications.

The Transaction Type and Transaction Subtype codes should not focus on naming conventions based on the standard XML message that is implemented. A given transaction from an application may be mapped to several XML standards based on the trading partner agreements, likewise for inbound messages.

The following naming conventions are recommended:

Transaction Type

The Transaction Type is the product short name for the base Oracle E-Business Suite application. Examples of Oracle E-Business Suite application product short names are shown in the following table:

Transaction Type	Application
AP	Payables
AR	Receivables
OM	Order Management
PO	Purchasing
SS	Supplier Scheduling
RLM	Release Management
CUSTOM	(for user-defined messages)

Transaction Subtype

The Transaction Subtype is a code for a particular transaction within the application

specified by the Transaction Type. The last position of the subtype code represents the direction of the transaction: "T" for inbound, "O" for outbound.

The naming convention is XXXXI or XXXXO where XXXX is significant to the application and its function, such as invoicing to Receivables and Payables.

For custom transactions, add a prefix to distinguish custom transactions from the Oracle supplied transactions.

The table below lists common combinations of Transaction Type and Transaction Subtype:

Transaction Type (Application)	Transaction Subtype (Transaction)	Transaction Type's Processing Application	Transaction Type's Processing Application (Transaction Code plus Direction)
PO	POCO	Purchasing	Purchase Order Change, Outbound
PO	POO	Purchasing	Purchase Order, Outbound
PO	POAI	Purchasing	Purchase Order Acknowledgment, Inbound
OM	POI	Order Management	Purchase Order, Inbound
OM	POCI	Order Management	Purchase Order Change, Inbound
OM	POAO	Order Management	Purchase Order Acknowledgment, Outbound
OM	POCAO	Order Management	Purchase Order Change Acknowledgment, Outbound
AR	INO	Receivables	Invoices, Outbound
AP	INI	Payables	Invoices, Inbound

The actual transaction detail is therefore recognized by its unique combination of the Transaction Type and Transaction Subtype. The Transaction Subtype does not have to be unique across applications, but only within the application specified by the

Transaction Type.

Consequently, both the Purchasing and the Order Management application can use the same Transaction Subtype (for example, POI) because they may both import some type of purchase order. For example, Order Management will load orders from its customers; Purchasing may load from another purchasing application.

Additional Suffix on Transaction Subtype

If different database views are used to extract different types of transaction data, such as a blanket order versus a standard order, or different XML messages are needed for transactions such as a blanket order versus a standard order, then you can create different message maps using the Message Designer. You may also need to define different Transaction Subtypes to distinguish between such transactions in the Transaction setup.

When appropriate, a suffix can be attached to the base Transaction Subtype following the XXXXI or XXXXO naming convention to further distinguish a transaction in the XML Gateway. For queries in a form, it is recommended to keep the same base Transaction Subtype XXXXI or XXXXO, so the Transaction Subtypes will follow each other in queries.

The suffix naming convention is therefore XXXXI-yyyy and XXXXO-yyyy where yyyy is the suffix.

The following table illustrates the Transaction Subtype naming convention using a suffix:

Transaction Type (Application)	Transaction Subtype (Transaction)	Transaction Type's Processing Application	Transaction Type's Processing Application (Transaction Code plus Direction)
PO	POO-BLK	Purchasing	Blanket Purchase Order, Outbound
PO	POO-REL	Purchasing	Purchase Order Release, Outbound
PO	POO-STD	Purchasing	Standard Purchase Order, Outbound

For example in a Purchasing application, all types of outbound purchase orders can be initiated under the general subtype POO with qualifiers for the various types of purchase orders such as blanket, standard, and release. Their subtype codes can be recognized as POO-BLK, POO-STD, and POO-REL respectively by the process. Avoid using long suffixes such as -BLANKET, -STANDARD, and -RELEASE to keep the naming convention simple. This sample is for illustration only.

Setting VERB and NOUN in OAG Standards

The following topic discusses sources of data for the OAG CNTROLAREA.

The VERB and NOUN are key values required in the OAG CNTROLAREA data type. XML Gateway provides a database view that provides key data from the transaction table, while the attribute values are default values from the DTD used to create the outbound message map.

The purpose of the database view is to populate the VERB and NOUN elements illustrated below.

The Verb and Noun in OAG's CNTROLAREA

```
<NOUN value = "INVOICE">INVOICE </NOUN>
```

```
<VERB value = "PROCESS">PROCESS </VERB>
```

When the External Transaction Type and External Transaction Subtype are entered into the transaction table through the Define Transactions form, they can be mapped to the NOUN and VERB value fields respectively by using the database view ECX_OAG_CONTROLAREA_TP_V (formerly ECX_OAG_CONTROLAREA_V). See Transaction Map - Element Mapping, page 2-62.

Note: The ECX_OAG_CONTROLAREA_TP_V view is an upgraded version of the ECX_OAG_CONTROLAREA_V view. Oracle XML Gateway supports both versions of the database view. For a detailed description of the differences, see the Note, page 2-34, in the Message Designer chapter.

Important: Within the OAG CNTROLAREA, the sequence of fields is VERB then NOUN in the message. In the Define Transactions form, the sequence of fields is NOUN in the External Transaction Type, then VERB in the External Transaction Subtype. Be careful not to reverse the order of the data.

Define Lookup Values

To navigate to the Oracle XML Gateway Lookups form from the XML Gateway Responsibility select Setup > Define Lookup Values.

The Oracle XML Gateway Lookups form allows both entry and display of seeded data. This is a standard Oracle Application Object Library form.

Once a Type is selected in the upper section, its seeded values are displayed in the Lookup Details section of the form.

Type (Required)

Type is the key code for the element stored in the seeded table.

The XML Gateway seeded lookup types are listed in the following table:

Type	Description	Sample Values
COMM_METHOD	Communications Method	HTTP, HTTP-ATTCH, HTTP-OXTA, HTTP-WM, HTTPS, HTTPS-ATTCH, HTTPS-OMB, HTTPS-OXTA, HTTPS-WM, IAS, ITG03, SMTP, OTAH-ATCH, OTAHS-ATCH, NONE, SOAP, JMS
CONFIRMATION_CODE	Confirmation Code	0, 1, 2
DOCUMENT	Documents/Transactions	CBODI and CBODO are seeded by Oracle XML Gateway. CBODI is OAG's inbound confirmation. CBODO is OAG's outbound confirmation.
MESSAGE_STANDARD	XML Message Standard	OAG, ORCL, RN, UNIVERSAL, PESC
MESSAGE_TYPE	Message Type	XML, EDI, FF (for flat file)
PARTY_TYPE	Party Types	B (for bank), C (for customer), S (for supplier), I (for internal location), E for Exchange, CARRIER (for Carrier)
TRANSACTION_CODE	Transaction Code	CBOD for confirmation BOD is the XML Gateway seeded transaction code.

User Name (Required)

User name is defined by the user when data is entered.

Application (Required)

The text name for the application responsible for this Type.

Description

Description of the Type.

Access Level

The Access Level restricts changes that are possible to a lookup type. The possible levels are:

- System - No changes to the lookup codes are allowed.
- Extensible - New lookup codes can be added. However, you cannot modify seeded lookup codes.
- User - You can change any lookup code.

Lookup Details**Code (Required)**

The seeded code for the lookup.

Meaning (Required)

The meaning of the lookup code.

Description

Description of the lookup code.

Tag

Not used.

From Effective Date

The starting effective date for the lookup code.

To Effective Date

The expiration date for the lookup code. The ending date is optional.

Enabled Checkbox

Check the box if the Code is enabled. Remove the check to disable the Code.

Trading Partner Setup

You can set up and manage trading partner information in the following ways:

- Use the Define Trading Partner Setup form
See: Using the Define Trading Partner Setup Form, page 3-20.
- Use the Trading Partner Setup API
See: Managing Trading Partner Information Through the Trading Partner Setup API, page 3-22.

In Oracle XML Gateway, the term "Trading Partner" refers to an entity such as a customer, supplier, bank branch, or internal locations at a particular address with which you exchange messages. Since a given entity may have several locations, you must define one Trading Partner for each customer address, supplier site, or bank branch as required for processing transactions by the Oracle XML Gateway.

Using the Define Trading Partner Setup Form

Navigate to the Define Trading Partner Setup form from the XML Gateway Responsibility by selecting Setup > Define Trading Partners.

You can use the Trading Partner Setup form to:

- Set up trading partners for multiple operating units.
- Enable messages for the trading partner by identifying the internal and external transaction type and transaction subtype codes, and the XML standard associated with the message.
- Access the Trading Partner User Setup form.
- Access the Trading Partner Code Conversion form.
- Select a message map for the trading partner.
- Identify the communications protocol and address for a message. Optionally, the user can be selected from a hub.

For example, if JMS messages are used in the transactions, then you must select JMS as the Protocol Type and appropriate JMS queues in the Protocol Address fields.

Note that you can use the SMTP communication protocol in conjunction with the SMTP server that is configured for the Oracle Workflow Notification Mailer to send emails. To use the Oracle Workflow Notification Mailer for the SMTP protocol, the required configuration steps need to be performed. For information on the setup tasks, see Use Oracle Workflow Notification Mailer with SMTP Protocol, page 3-21.

This is the component that will enable a message to be processed through the XML Gateway engine. During message processing, Trading Partner data is used to:

- Link a particular address location in Oracle E-Business Suite to the Trading Partner definition in the Gateway.

- Provide a means of telling the Execution Engine which Trading Partner message map to use.
- Enable specific transactions for Trading Partners.
- Determine how to deliver the message.

This form defines the parameters for the Trading Partner setup. The setup includes the identification of the operating unit, trading partner site, the messages enabled for that site, and the delivery mechanism.

The Trading Partner Setup form requires an entry for each Transaction Type and Transaction Subtype associated with this trading partner.

Use Oracle Workflow Notification Mailer with SMTP Protocol

For sending email messages, Oracle XML Gateway allows you to use the SMTP communication protocol in conjunction with the SMTP server that is configured for the Oracle Workflow Notification Mailer. In order to use the Oracle Workflow Notification Mailer to send emails, you must perform the following steps:

1. Ensure that you add the following new property in the `$INST_TOP/appl/admin/oafm_wls.properties` file:
`OXTAMAILER=WFMAILER`
2. Configure the Oracle Workflow Notification Mailer for SMTP.
3. Additionally, to send the email messages using the Oracle Workflow Notification Mailer with OAuth 2.0, java mail 1.6.2 jar is required on the oafm server class path. If your Oracle E-Business Suite Technology Stack (TXK) Release Update Packs (RUPs) level is earlier than R12.TXK.C.Delta.16, perform the following steps to add the java mail 1.6.2 jar to the oafm class path:
 1. Navigate to `$FMW_HOME/modules`. Take a backup of `javax.mail_1.1.0.0_1-4-1.jar`.
 2. Rename `javax.mail_1.6.2.jar` to `javax.mail_1.1.0.0_1-4-1.jar`.
 3. Stop and restart the application tier services.

If both the above steps 1 and 2 about setting the property and configuring the Oracle Workflow Notification Mailer for SMTP are not performed, Oracle XML Gateway will fall back to the existing behavior for sending emails.

Note: This configuration impacts all the trading partners with the communication protocol set to SMTP.

Managing Trading Partner Information Through the Trading Partner Setup API

In addition to using the Define Trading Partner Setup form, you can manage one or more trading partner information through the use of the Trading Partner Setup API (ECX_TP_SETUP_API) or web service. This feature provides another option to create, update, delete, and retrieve trading partner information from Oracle XML Gateway.

This Trading Partner Setup API is published in Oracle Integration Repository where you can access the API and deploy it as a web service through the **Integrated SOA Gateway** responsibility.

Important: Before using the Trading Partner Setup API as a web service, you must perform the setup tasks as described in My Oracle Support Knowledge Document 1311068.1 to configure Oracle E-Business Suite Integrated SOA Gateway.

Oracle Integration Repository with Trading Partner Setup API Interface Page

Integration RepositoryAdministration

Integration Repository > PLSQL Interface : Trading Partner Setup

BrowseSearchPrintable Page

Log Configuration DisabledConfigure

Internal NameECX_TP_SETUP_APITypePL/SQLProductOracle XML GatewayStatusActiveBusiness EntityXML Gateway Trading PartnerScopePublicInterface SourceOracle

OverviewSOAP Web ServiceREST Web ServiceGrants

Full Description

This API lets you manage trading partner information in Oracle XML Gateway. It contains procedures to create, update, retrieve and delete trading partner details for one or more party sites in a party. Trading Partner refers to an entity such as a customer, supplier, bank or any organization at a particular location or address - with which you exchange business transactional documents through XML Gateway. A trading partner should be setup in Oracle XML Gateway for each required party site of a party in Oracle Trading Community Architecture. Setup includes trading partner specific transaction, code conversion details and user level security.

Source Information

Source Filepatch/115/sql/ECXTPSETUPS.plsSource Version120.0.12020000.5Source ProductECX

Procedures and Functions

Name	Internal Name	Status	Description
Create Trading Partner	CREATE_TRADING_PARTNER	Active	This procedure is used for the creation of TP-Setup corresponding the the party_site_id passed or for all the sites in party depending on the parameter passed.
Delete Trading Partner	DELETE_TRADING_PARTNER	Active	This procedure used for the modification of details part of TP-Setup corresponding to the party_site_id passed or for all the sites in party depending on the parameter passed.
Get Trading Partner	GET_TRADING_PARTNER	Active	This procedure used for the retrieval of TP-Setup details corresponding the the party_site_id passed or for all the sites in party depending on the parameter passed.
Update Trading Partner	UPDATE_TRADING_PARTNER	Active	This procedure is used for the modification of details part of TP-Setup corresponding to the party_site_id passed or for all the sites in party depending on the parameter passed.

Note: This Trading Partner Setup API of a PL/SQL interface type can be exposed as a SOAP-based or REST-based web service.

For more information on deploying the API as a SOAP or REST web service, see *Administering Native Integration Interfaces and Services* chapter, *Oracle E-Business Suite Integrated SOA Gateway Implementation Guide*.

For more information about this API, see *Trading Partner Setup API*, page F-29.

Trading Partners for Multiple Organizations

To support multiple organization functionality, Oracle XML Gateway allows you to set up Trading Partners for all operating units associated with your responsibility.

All Trading Partner related data, including Trading Partner Types, Trading Partner Names, and Trading Partner Sites, is associated with the specified operating unit. You can select an operating unit that is linked to your responsibility while defining a Trading Partner using the Trading Partner Setup form.

Multiple Organization Access Control

To secure data access, Oracle XML Gateway uses security profiles that are linked to your responsibility to control access to one or more operating units. The security profile concept allows system administrators to predefine the scope of access privileges as a security profile and then associate the security profile with responsibility for a user.

Multiple operating units are associated with a security profile and the security profile is in turn associated with a responsibility. Therefore, through the access control of security profiles, users can access data in multiple operating units without changing responsibility.

Security profiles are defined based on organization hierarchies. For example, a sales company consists of USA and UK operating units; each operating unit also contains sublevel operating units for various regions. These top or sub level operating units can be defined as security profiles based on the organization hierarchy. Sales managers, supervisors, and representatives are responsible for their designated sales units or regions. System administrators can then associate those predefined security profiles with appropriate sales people through their responsibilities. Therefore, sales supervisors can easily access sales data in different sales regions without changing their responsibilities.

See Trading Partner chapter, *Oracle e-Commerce Gateway Implementation Guide* for details.

Trading Partner Setup Header

The Trading Partner Setup Header includes the following fields:

Operating Unit (Required)

The Operating Unit field displays the default operating unit that is restricted by the security profile linked to your logon responsibility. You can change the default by selecting other operating unit from the list of values if you are associated with more

than one operating units.

Trading Partner Type is tied to the operating unit; that is, once the operating unit is selected, only the associated Trading Partner Type, such as Supplier, Customer, Bank, or internal location, will be displayed in the list of values.

Trading Partner Type (Required)

Trading Partner Type defines the type of trading partner, such as Supplier, Customer, Bank or internal location. Once the Trading Partner Type is selected from the list of values, the Trading Partner Names and Trading Partner Sites associated with the Trading Partner Type are displayed in the Trading Partner Name and Trading Partner Site lists of values below.

Trading Partner Name (Required)

Given the selection in the Trading Partner Type, the appropriate Trading Partner Names are displayed in the Trading Partner Name list of values. For example, if Partner Type is Customer, then customer names will be displayed. These Trading Partners are limited to those trading partners associated with the organization of your logon responsibility. Select the appropriate Trading Partner Name.

Trading Partner Site (Required)

Given the selection in the Trading Partner Name, the appropriate Trading Partner Sites are displayed in the list of values. Select the appropriate Trading Partner Site.

Company Admin Email (Required)

This is the e-mail address of the administration contact to receive e-mails regarding warnings and errors. These notifications may be initiated by Oracle Workflow or by an action defined in the message map using the Message Designer. Users should check the error log.

Code Conversion Button

Use the Code Conversion button to access the Trading Partner Code Conversion form. See:

- Code Conversion, page 3-38
- Trading Partner Code Conversion Form, page 3-48

Trading Partner Details

The combination of Party Type (Trading Partner Type), Trading Partner Name, Trading Partner Site, Transaction Type, and Transaction Subtype uniquely identify an outbound transaction.

The combination of External Transaction Type, External Transaction Subtype, Standard Code, and Source Trading Partner Location Code uniquely identify an inbound

transaction.

The Trading Partner Setup form includes the following detail for each message:

Transaction Type (Required)

Transaction Type is the standard product short code for the base Oracle E-Business Suite application. These values are defined in the Define Transactions form. The list of values will display the available combinations of Transaction Type, Transaction Subtype, Standard Code, External Transaction Type, External Transaction Subtype, and Direction. Select the desired combination.

These values are only used internally to the XML Gateway. Refer to the Define Transactions Form, page 3-11 for details.

Transaction SubType

Transaction subtype is a code for a particular transaction within the application specified by the Transaction Type. The last position represents the direction of the transaction: I for inbound, O for outbound.

The combination of Party Type (Trading Partner Type), Transaction Type, and Transaction Subtype should identify an Oracle transaction with which to associate this message. These values are defined in the Define Transactions form.

These values are only used internally to the XML Gateway. Refer to the Define Transactions Form, page 3-11 for details.

Standard Code

The Standard Code associated with the Transaction Type selected above is displayed.

Standard Codes are set up in the Define XML Standards form, page 3-10.

External Transaction Type

The External Transaction Type associated with the Transaction Type selected above is displayed.

The External Transaction Type is the primary external identifier for the XML message. These values are defined in the Define Transactions form, page 3-11 and they are found in the XML Gateway envelope, page 4-8.

The combination of the External Transaction Type and the External Transaction Subtype should identify this external message to the Oracle E-Business Suite.

External Transaction Subtype

The External Transaction Subtype associated with the Transaction Type selected above is displayed.

The External Transaction Subtype is the secondary identifier for the XML message. These values are defined in the Define Transactions form, page 3-11 and they are found in the XML Gateway envelope, page 4-8.

The combination of the External Transaction Type and the External Transaction Subtype should identify this external message to the Oracle E-Business Suite.

Direction

The Direction associated with the Transaction Type selected above is displayed.

This code identifies the direction of the transaction. The value IN identifies an inbound message, and the value OUT identifies an outbound message.

Map (Required)

(Message) Map is the name of the map created using Message Designer.

Select the appropriate map from the list of values.

The naming convention for message maps consists of the four components: Transaction Type, Transaction Subtype, Standard and Release, and Direction. For example: "PO_POO_OAG70_OUT" is the outbound Oracle Purchasing Purchase Order in the OAG standard, release 7.0. The direction code OUT makes the message direction more apparent in any list.

Refer to Message Designer, page 2-26 for more details on the naming convention.

If the desired map does not appear in the list of values, then the map has not been loaded into the XML Gateway database. Refer to the How to Load Message Maps and DTDs, page 2-111.

Connection/Hub (Required for outbound messages only)

A DIRECT connect and a hub are the methods by which the message can be communicated. The XML message can be sent directly to a trading partner, or sent to a trading partner via a hub. The hub will then communicate the message to the trading partner.

Select DIRECT to conduct business directly with a trading partner, or select a hub from the list of values. If a hub is selected, then select a trading partner from that hub.

See the Hub Definition form, page 3-8 to define hubs.

Requirements for the entries for Protocol Type, Username, Password, and Protocol Address depend on whether you select DIRECT or a hub, page 3-8.

If you select DIRECT, complete these fields as follows:

- **Protocol Type (Required)**

Each message enabled for a Trading Partner includes a protocol type (also known as communication method). The associated correlation ID is determined internally and is associated with the message enqueued onto the agent (queue). Listeners defined for the agent will dequeue messages based on the correlation ID defined for it. For example, Oracle Transport Agent will only dequeue messages with a correlation ID of OXTA.

Note: If you want to use the SMTP communication protocol in conjunction with the SMTP server that is configured for the Oracle Workflow Notification Mailer to send email messages, you must add the new property `OXTAMAILER=WFMAILER` in the `$INST_TOP/appl/admin/oafm_wls.properties` file, and configure the Oracle Workflow Notification Mailer for SMTP. If the property is not set correctly or if the Oracle Workflow Notification Mailer is not configured for SMTP, then Oracle XML Gateway will fall back to the existing behavior for sending emails. For more information, see *Use Oracle Workflow Notification Mailer with SMTP Protocol*, page 3-21.

The following table shows the Protocol Type, Correlation ID, and Message System.

Protocol Type	Correlation ID	Message System
NONE	N/A	Message disabled
HTTP-WM, HTTPS-WM	WebMethods	Third party system
HTTP, HTTP-OXTA, HTTPS, HTTPS-OXTA, SMTP	OXTA	Oracle Transport Agent without attachments
OTAH-ATCH, OTAHS-ATCH	OXTA	Oracle Transport Agent with attachment using standard OTA envelope
HTTP-ATCH, HTTPS-ATCH	OXTA	Oracle Transport Agent with attachment and no OTA envelope
ITG03	ITG03	Oracle iProcurement Connector
IAS	IAS	Oracle Integration Server
SOAP	N/A	Web Service Agent
JMS	N/A	JMS Provider

Select the protocol type from the list of values. This data is seeded by the XML Gateway.

Protocol type NONE will disable the outbound message for this trading partner.

- **Username**

Enter the destination Username used to log in to the receiving server for the server that is identified in the server address.

For protocol types HTTP and HTTPS, Username and Password are required fields.

- **Password**

Enter the Password for the destination Username. The password is not echoed when it is entered. You will be prompted on the same field to confirm the password.

Note that the password accepts special characters.

For protocol types HTTP and HTTPS, Username and Password are required fields.

- **Protocol Address**

Protocol Address is the complete URL (including service/servlet) where the Transport Agent, page 7-1 will attempt to post the XML Document.

For protocol type SMTP, the Protocol Address is an e-mail address, and is required.

For protocol type JMS, the Protocol Address is a JMS queue.

If you select a Hub, complete these fields as follows:

- **Protocol Type**

Protocol Type defaults from the Hub definition.

- **Username**

Select the Username from the list of values supplied by the Hub definition. If the Protocol Type is SMTP, the Username is not required. For all other Protocol Types, this field is required.

- **Password**

This field defaults to the password that is associated with the Username selected. The username and password combination is supplied from the Hub definition. If the Protocol Type is SMTP, this field is not required.

- **Protocol Address**

Protocol Address defaults from the Hub definition.

Source Trading Partner Location Code (Required)

Source Trading Partner Location Code is the code found in the XML Gateway envelope, page 4-8 to identify the source of the message. This is the code for the source trading

partner of the message.

If you are sending and receiving messages from a trading partner, you will have a mixture of your location codes and your trading partner's location codes depending on the direction of the message. For example, if you are sending messages out, the Source Trading Partner Location Code is your location code because you are the source of the XML message. If you are receiving messages, the Source Trading Partner Location Code is your trading partner's location code because they are the source of the XML message.

This code is examined for inbound messages for Trading Partner validation. When placed on an outbound message, the recipient will validate it as a valid source or sending location.

This field is placed in the PARTY SITE ID in the XML Gateway envelope, page 4-8.

See: XML Gateway Envelope, page 4-8.

Destination Trading Partner Location Code

There are two types of routing in the XML Gateway: Static Routing and Dynamic Routing. Dynamic Routing allows the message to be rerouted by the first recipient of the message to the final destination recipient. Refer to the Routing, page 3-29 field below for Static Routing.

Destination Trading Partner Location Code is the code for the final recipient of the XML message. This code is not needed by the XML Gateway creating this message, but needed by the hub or the first trading partner receiving the message to identify their final trading partner to receive the message. It is the hub's or the first trading partner's code for that final destination trading partner.

For outbound messages, the final intended recipient location code identified by the Destination Trading Partner Location Code will be placed in ATTRIBUTE3 in the XML Gateway envelope, page 4-8.

For inbound messages, this code is found in ATTRIBUTE3 in the XML Gateway envelope. Refer to Static and Dynamic Routing, page 3-31 for an illustration that explains how ATTRIBUTE3 is populated.

Document Confirmation

Document Confirmation is the indicator for the confirmation level that this Trading Partner would like to send or receive a confirmation.

0 (Default value) means Never send a confirmation

1 means Send a confirmation only if there are errors

2 means Always send a confirmation

It defines the condition under which a confirmation XML message is generated or received. Outbound messages receive inbound confirmations. Inbound messages generate outbound confirmations.

Routing

Use the Routing field to identify another trading partner to whom messages will be routed. You select a trading partner for outbound transactions from the list of values. The inbound message is forwarded as an outbound message to the trading partner identified in the Routing field.

The XML Gateway provides both Static Routing and Dynamic Routing. Routing is the address to route the outbound message to when using the Static Routing method. Refer to the Destination Trading Partner Location Code, page 3-29 field above for Dynamic Routing.

See: Static and Dynamic Routing, page 3-31 for more information.

Required Communications Data

Required Communications for Outbound Messages

Different data is required depending on whether DIRECT or a trading partner is selected from a hub.

If DIRECT is selected, the following data fields are entered by the user depending on the protocol type for outbound messages:

- Protocol Type (required)
- Protocol Address (required)
- User Name (depends on Protocol Type)
- Password (depends on Protocol Type)
- Source Trading Partner Location Code to identify the recipient of the message (required)

If the message is sent via a hub, select the hub, then select a Username from the presented list. The following data fields are retrieved for outbound messages:

- The following data fields are copied from the Username in the hub definition:
 - Protocol Type
 - Protocol Address
 - Hub Entity Code (acts like the Source Trading Partner Location Code for DIRECT communication)
 - Password (optional and depends on Protocol Type)

If the message will be rerouted to another trading partner by the hub using

Dynamic routing, then the following data is needed. It will be placed in ATTRIBUTE3 in the XML Gateway envelope, page 4-8:

- Destination Trading Partner Location Code

See: XML Gateway Envelope, page 4-8.

Required Communications Data for Inbound Messages

Inbound messages require the following:

- Source Trading Partner Location Code to identify the sender of the message

Static and Dynamic Routing

Messages can be passed through a middle trading party by using the static routing or dynamic routing feature. This method is also called a pass-through.

The following figure shows the routing flow of messages using the dynamic or static routing feature. The process starts with the review of the Trading Partner detail associated with the inbound message.

If data is found in the ATTRIBUTE3 field of the XML Gateway envelope, then the transaction is processed under the rules of Dynamic Routing. First the message is processed according to the inbound message map for the trading partner, then the message is rerouted to another trading partner who is the final recipient of the message. The trading partner detail for the final recipient is found in the Trading Partner Detail as an outbound message for the trading partner identified in the ATTRIBUTE3 field.

If there is no data in the ATTRIBUTE3 field of the XML Gateway envelope and there is data in the Routing field in the Trading Partner Detail for the inbound message, then the transaction is processed under the rules of Static Routing.

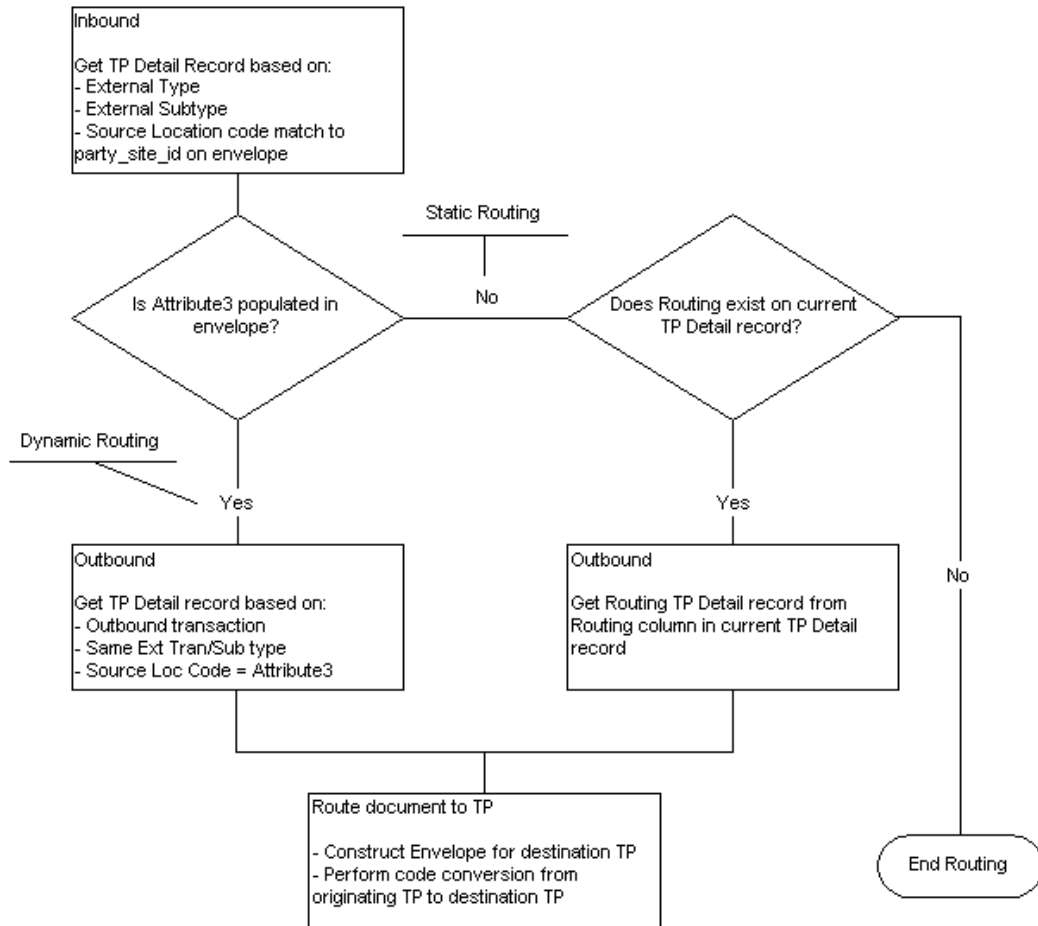
In Static Routing the message is processed according to the inbound message map for the trading partner, then the message is rerouted to another trading partner who is the final recipient of the message. The trading partner detail for the final recipient is found in the Trading Partner Detail as the trading partner identified in the Routing field for that inbound message.

If there is data in the ATTRIBUTE3 field of the XML Gateway envelope and there is data in the Routing field in the Trading Partner Detail for the inbound message, ATTRIBUTE3 will be used for the routing data.

If there is no data in the ATTRIBUTE3 field of the XML Gateway envelope and there is no data in the Routing field in the Trading Partner Detail for the inbound message, the message is not rerouted to another trading partner.

In both dynamic and static routing, data in the message may have code conversion applied at the trading partner level before constructing the outbound message for the final recipient. The retrieved code conversion values will be substituted into the XML message so the final trading partner receives the values that apply to them.

Dynamic and Static Routing Process Flow Diagram



To explain Static and Dynamic routing for pass through messages, the following three trading partners are defined:

- Party 1: Initiator of the Message
- Party 2: First Recipient of the Message (This may be a hub)
- Party 3: Final Destination of the Message

Party 1 will send the message to Party 2. Then Party 2 will reroute it to Party 3.

Static Routing

Static routing allows you to select another XML Gateway Trading Partner detail record from all your entries in the Trading Partner Setup, page 3-19 form. You can select a trading partner within the same trading partner or from any other displayed trading partner. Every time a message is received for that trading partner and that transaction, it will automatically be routed to the trading partner defined in the "Routing" column.

Static Routing occurs under the following conditions:

- The originator of the message (Party 1) does *not* include the final Destination Trading Partner Location Code in the message.
- For the given transaction and trading partner, the first recipient of the message (Party 2) has selected a trading partner in the Routing field to automatically forward that message to.

In order for this to happen, the following is necessary:

- Proper trading partner setup for the final recipient must be entered in the Trading Partner Setup in Party 2's environment.

Static Routing Illustration

The following illustration reviews the required trading partner setup for each party in the example.

Party 1: Initiator of the Message:

1. In Static Routing, the first initiator of the message does *not* indicate the final Destination Trading Partner Location Code in their setup. No data should appear in ATTRIBUTE3 of the XML Gateway envelope. This is illustrated in the following table:

Entry	Trading Partner	Direction	External Transaction Type	External Transaction Subtype	Protocol	Source TP Location Code	Routing	Destination TP Location Code (in ATTRIBUTE3)
(1)	Party 2	OUT	INVOICE	ADD	HTTP	Party 1	N/A	N/A

Party 2: First Recipient of the Message:

PARTY 2 must define the message for both the inbound trading partner and the outbound trading partner to whom the message will be rerouted.

(2) Party 2 received the message from Party 1. The message was an inbound invoice to Party 2. See entry (2) of the table below.

(3) Party 2 will reroute the message to Party 3, because the trading partner setup for Party 3 was stored under the Routing field for Party 2's trading partner setup for Party 1. The trading partner setup data for the trading partner in the Routing field is used to create the new message and the XML Gateway envelope. See entry (3) of the table below.

Entry	Trading Partner	Direction	External Transaction Type	External Transaction Subtype	Protocol	Source TP Location Code	Routing	Destination TP Location Code
(2)	Party 1	IN	INVOICE	ADD	N/A	Party 1	(points to Party 3 for the outbound message)	N/A
(3)	Party 3	OUT (will be set to out)	INVOICE (use the same External Transaction Type)	ADD (use the same External Transaction Subtype)	HTTP	Party 2	N/A	N/A

Dynamic Routing

The following method is dynamic because the first party originating the message indicates the Destination Trading Partner Location Code for the final destination trading partner to receive the message.

Dynamic Routing occurs under the following conditions:

- The originator of the message (Party 1) includes the final Destination Trading Partner Location Code in ATTRIBUTE3 in the XML Gateway envelope.
- The first recipient of the message (Party 2) can identify the trading partner code in ATTRIBUTE3 in their own Trading Partner setup.

In order for this to happen the following is necessary:

- Party 1's XML process must be set up to write the final trading partner to ATTRIBUTE3 of the XML Gateway envelope. The XML Gateway uses the Destination Trading Partner Location Code field in the Trading Partner Setup form.
- The proper trading partner setup must be entered in Party 2's process to act on ATTRIBUTE3. For the XML Gateway, this is in the Source Trading Partner Location Code field in the Trading Partner Setup form.

Dynamic Routing Illustration

The following illustration reviews the required trading partner setup for each party in the example.

Party 1: Initiator of the Message:

(1) In Dynamic Routing, the first initiator of the message (Party 1) passes a Destination Trading Partner Code in ATTRIBUTE 3 to the first message recipient (Party 2).

This code is defined by the first recipient of the message, so they can identify the trading partner to whom the message will be forwarded. The Destination Trading Partner Code will be copied to ATTRUBUTE3 in the XML Gateway envelope. See the table below:

Entry	Trading Partner	Direction	External Transaction Type	External Transaction Subtype	Protocol	Source TP Location Code	Routing	Destination TP Location Code (in ATTRIBUTE3)
(1)	Party 2	OUT	INVOICE	ADD	HTTP	Party 1	N/A	Party 3

Party 2: First Recipient of the Message:

This second party must define the message for both the inbound trading partner and the outbound trading partner.

(2) Party 2 received the message from Party 1. The message was an inbound invoice to Party 2. See entry (2) in the Trading Partner Setup for Party 2 table.

Party 2 will validate Party 1 as a trading partner for that inbound message. The data is shown in the following table

Direction	External Transaction Type	External Transaction Subtype
IN	INVOICE	ADD

(3) Party 2 will reroute the message to Party 3. The message becomes an outbound invoice from Party 2. See entry (3) below.

Party 2 will also perform a trading partner lookup for the trading partner to whom the message is to be forwarded. The direction is changed to OUT. External Transaction Type and External Transaction subtype are copied from the inbound message. The lookup involves the data shown in the following table:

Direction	External Transaction Type	External Transaction Subtype
OUT	INVOICE	ADD

The Trading Partner Setup for Party 2, is shown in the following table:

Entry	Trading Partner	Direction	External Transaction Type	External Transaction Subtype	Protocol	Source TP Location Code	Routing	Destination TP Location Code (in ATTRIBUTE3)
(2)	Party 1	IN	INVOICE	ADD	N/A	Party 1	N/A	N/A
(3)	Party 3	OUT (set to OUT)	INVOICE (use the same External Transaction Type)	ADD (Use the same External Transaction Subtype)	HTTP	Party 2	N/A	N/A

Party 3: Final Recipient of the Message:

(4) Party 3 received the message from Party 2. The message was an inbound invoice. The data is shown in the following table:

Entry	Trading Partner	Direction	External Transaction Type	External Transaction Subtype	Protocol	Source TP Location Code	Routing	Destination TP Location Code (in ATTRIBUTE3)
(4)	Party 3	IN	INVOICE	ADD	N/A	Party 2	N/A	N/A

Trading Partner User Security

To ensure that only an authorized user is allowed to perform inbound XML transactions on behalf of any Trading Partner, Oracle XML Gateway provides the Trading Partner user security feature through the association of authorized users with specific Trading Partners using the Trading Partner User Setup form.

Once users are defined or associated with specific Trading Partners, at run time during Web service invocation for XML Gateway services, authorization checks for user credentials are performed at the Trading Partner level as specified here.

For more information about XML Gateway Web services, see the Web Services Chapter, page 8-1.

Important: Each Trading Partner can have one or more authorized users associated with it, but each user can be authorized for only one Trading Partner.

What's the Impact?

Oracle Transaction Agent (OTA) and Web Services use internal APIs to validate the Trading Partner and authorize a user for a Trading Partner to perform XML transactions on behalf of a Trading Partner.

Existing Users will have to authorize an Oracle E-Business Suite user by associating a user with a Trading Partner using the Trading Partner User Setup form to perform XML transactions on behalf of a Trading Partner.

For backward compatibility, Oracle XML Gateway uses the 'ECX: Enable User Check for Trading Partner' profile option to turn the user security feature on or off.

- If the value is set to "No" (default) which turns the security OFF, the transaction will be enabled regardless of the association between the user and the Trading Partner.
- If you change the value from "No" to "Yes" which turns the user security ON, Oracle XML Gateway will validate the Trading Partner user against the Trading Partner user setup data to determine if the inbound transaction can be enabled.
 - If the user is linked to the Trading Partner, then the transaction is enabled for the Trading Partner.
 - If the user is not linked to the Trading Partner, then the transaction is not enabled. An error message will be shown in the transition monitor indicating that the user is not enabled in the XML Gateway Server. Please check your setup.

The Trading Partner User Setup Form

The Trading Partner User Setup form allows you to associate authorized users with a Trading Partner. The same user cannot be associated with more than one Trading Partner.

Navigate to this form by clicking the User Setup button on the Define Trading Partner Setup form (XML Gateway Responsibility > Setup > Define Trading Partners).

Trading Partner Header

The Trading Partner Type, Trading Partner Name, and Trading Partner Site values are entered in the Trading Partner Setup, page 3-19 form and displayed here as read-only fields. You can update Company Admin Email address as needed. The address of the administration contact is to receive e-mail notifications regarding warnings and errors.

Note: If the Trading Partner Header information is not saved in the

Trading Partner Setup form before selecting the User Setup button, an error message will occur.

User Information

User Name

Select valid Oracle E-Business Suite user names to be associated with the Trading Partner from the drop-down list. An error message appears if the selected user has been assigned to any other Trading Partner.

Note: To update a selected user name, you need to first delete the selected name and then add a new one selected from the drop-down list so that the new user is also a valid Oracle E-Business Suite user.

User Description

This field populates automatically once the User Name field is selected.

Code Conversion

The Oracle XML Gateway code conversion function provides a method to cross-reference the codes defined in Oracle E-Business Suite to codes used by trading partners, the XML standard, or other standard codes in the transactions.

For example, assume that the ABC Corporation transmits a purchase order to the XYZ Corporation. The XYZ Corporation processes the incoming data using its own Oracle values (for example, unit of measure, currency, or freight carriers), but XYZ is required to return ABC's codes. In this way, the trading partner that created the original transaction receives response transactions that use their own values.

The Code Conversion features include the following:

1. Define XML standard code conversion values defined by the XML standard. These codes are used by all trading partners.
2. Define universally used code conversion values that are defined by other standard organizations such as ISO, X12, and EDIFACT. These codes are used by all trading partners.
3. Define trading partner-specific code conversion values. The trading partner-specific code may override an XML standard or the other universally used code conversion values.

Code Conversion Setup Across XML Gateway Forms

In the Define XML Standards form, Standard Codes such as OAG are entered to represent a message standard. There is a special purpose Standard Code called "UNIVERSAL" that is described below. See: Define XML Standards Form, page 3-10.

Entries from the Define XML Standards form appear in a list of values in the Define Transactions form that defines transactions to the XML Gateway. In this form, one Standard Code is associated with each transaction listed under External Processes. See: Define Transactions Form, page 3-11.

The actual code conversion values are entered in the Standard Code Conversion form and the Trading Partner Code Conversion form. The code conversion form provides a one-to-one code conversion from the Oracle values to an external value. The external code may be the XML standard codes such as OAG's code, the universal code such as ISO codes, or trading partner-specific codes. These are discussed below.

Code Conversion Values Marked for the Trading Partner

The trading partner code conversion values are accessed through the Trading Partner Setup form. Refer to Trading Partner Setup, page 3-19 for details. Press the Code Conversion button to access the Trading Partner Code Conversion form from the Trading Partner Setup form.

The values associated with the trading partner's code values are the first set of codes to be examined during code conversion.

Before making entries in the trading partner specific table, first update the Standard Code Conversion values if necessary. This is important because the entries in the Standard Code Conversion tables are displayed and potentially overridden in the Trading Partner Code Conversion form.

The following table illustrates the linking of a Trading Partner's message maps to Code Conversion:

Trading Partner	Message Map	Code Conversion Values are created for the Trading Partner?	The Message Map has a Unit of Measurement (UOM) field?	Accessed Trading Partner Code Conversion Values when the Message is Processed
Acme Corp, Atlanta	PROCESS_PO	Yes, for Acme, Atlanta	Yes, So Category UOM is entered for Code Conversion on the field	For UOM Category Code, the Acme, Atlanta code conversion values are accessed for the message map PROCESS_PO

Trading Partner	Message Map	Code Conversion Values are created for the Trading Partner?	The Message Map has a Unit of Measurement (UOM) field?	Accessed Trading Partner Code Conversion Values when the Message is Processed
Acme Corp, Chicago	PROCESS_PO	Yes, for Acme, Chicago	Yes, So Category UOM is entered for Code Conversion on the field	For UOM Category Code, the Acme, Chicago code conversion values are accessed for the message map PROCESS_PO
Acme Corp, Atlanta	ADD_INVOICE	Yes, for Acme, Atlanta	Yes, So Category UOM is entered for Code Conversion on the field	For UOM Category Code, the Acme, Atlanta code conversion values are accessed for the message map ADD_INVOICE
Acme Corp, Chicago	ADD_INVOICE	Yes, for Acme, Chicago	Yes, So Category UOM is entered for Code Conversion on the field	For UOM Category Code, the Acme, Chicago code conversion values are accessed for the message map ADD_INVOICE

- Message Map is assigned to this trading partner site in the Trading Partner Setup form (under Trading Partner Details)
- Access Code Conversion through the Trading Partner Setup form
- Data field is assigned the Category Code when the message map is created in the Message Designer

Duplicate Entries Not Found

Since duplicate Oracle Values cannot be entered into the code conversion tables, the trading partner must always use the same "To Trading Partner Value" for all the transactions that it has enabled. The following table illustrates this rule:

Oracle Value	Description	From Trading Partner Value	To Trading Partner Value
Each	Each	EA	EA
Piece	Piece	PC	PC
Each	Each	PC	PC

- **Oracle Value** - Cannot key on multiple occurrences of the value "EACH"
- **From Trading Partner Value** - Cannot key on multiple occurrences of the value "PC"
- **To Trading Partner Value** - Cannot key on multiple occurrences of the value "PC"

Code Conversion Values Marked as a Standard Code

Given the Code Conversion setup discussed above, there is a standard code associated with each message. The standard code is the source organization for the code value, such as OAG for the given XML message to be processed.

The values associated with that standard organization's code values are the second set of codes to be examined during code conversion.

Code Conversion Values Marked UNIVERSAL

All other standards except the one that defined the XML message are marked collectively as UNIVERSAL. The XML Gateway does not store the other code conversion values under each standard such as the ISO codes or the X12 codes. It does not know which order the user wishes to access those code sets. For example, should the ISO codes be accessed before the X12 Codes, or the reverse? Hence, they are stored under a generic name.

The values associated with the UNIVERSAL code values are the third set of codes to be examined during code conversion.

The table below displays samples of ISO Country Codes that can be used across all trading partners and entered only once in the Standard Code Conversion forms.

Standard Code	Oracle Value	Description	From Trading Partner Value	To Trading Partner Value
UNIVERSAL	United States	United States	US	US

Standard Code	Oracle Value	Description	From Trading Partner Value	To Trading Partner Value
UNIVERSAL	United Kingdom	United Kingdom	UK	UK

If there is a conflict between two codes that must be entered as UNIVERSAL, it may be resolved by entering the conflicting codes under the Trading Partner code conversion for each trading partner needing that code.

Only the conflicting or duplicate code values need to be entered under each trading partner. The entries that do not cause conflicts can be entered under UNIVERSAL at the same time.

Code Categories

A Category Code is a label for a set of entries in the code conversion table. For example, CARRIER is the category code for code conversion values associated with the carrier.

During transaction processing, only the code conversion table entries with an assigned category code are accessed for the given data element.

See Seeded Code Categories, page B-1 for the seeded list.

Accessing the Code Conversion Values

Key Access

Outbound and inbound transactions use different keys in the code conversion tables to access the codes.

- For outbound transactions, "Oracle Value" and the Standard Code are keys to access this table to determine the "To Trading Partner Value" to write in the transaction.
- For inbound transactions, "From Trading Partner Value" and the Standard Code are keys to access the table to determine the "Oracle Value" to pass to Oracle E-Business Suite application tables.

Order of Table Search

The following table search order is performed for all transactions until a code conversion value table entry is found:

1. Access the Trading Partner code conversion table.
If the code is not found in this table, perform a second search.
2. Access the Standard code conversion table using the XML message's Standard

Code, such as OAG. This Standard Code is associated with the transaction in the transaction table as defined via the Define Transactions form.

If the code is not found in this table, perform a third search.

3. Access the Standard code conversion table using the Standard Code UNIVERSAL. The universal entries may represent ISO codes or other standards.
 - For inbound transactions: If the code is not found in the tables after the three searches described above, then the "From Trading Partner Value" is copied to the target field in the message map.
 - For outbound transactions: If the code is not found in the tables after the three searches described above, then the "Oracle Value" is copied to the target field in the message map.

Important: If a code conversion value is not found in the table, it is not an error. There may be cases where only select values for a data element need code conversion. To require all values to have a code conversion table entry may cause you to do an excessive number of code conversion entries that are not necessary.

The following table illustrates the order that the tables and Standard Code Values are searched:

Search Order	Code Conversion Form	STANDARD CODE	Purpose
1	Trading Partner Code Conversion	CUSTOM	Define the trading partner-specific code conversion values. This includes any overrides to any standard's or universal code conversion values that are displayed from the Standard Code Conversion form. The trading partner code conversion values are used by all transactions for that trading partner.
2	Standard Code Conversion	For example: OAG ROSETTANET (ROS)	Define standard code conversion values for a given XML standard that are used across all trading partners. These values can be overridden for a specific trading partner in the Trading Partner Code Conversion form. The Standard Code field is the default XML standard associated with the XML message.

Search Order	Code Conversion Form	STANDARD CODE	Purpose
3	Standard Code Conversion	UNIVERSAL	Define standard code conversion values that are used across all trading partners and not associated with an XML Standard. These values can be overridden for a specific trading partner in the Trading Partner Code Conversion form. Those entries accommodate universally used code lists such as ISO, X12, EDIFACT, that can be used by all XML Messages.

Outbound Transaction Code Conversion Table Access

For outbound transactions, the "Oracle Value" and the Standard Code in the code conversion tables are keys to retrieve the "To Trading Partner Value" to place it in the transaction. First the trading partner codes are searched. If an entry is not found, then the Standard code conversion table is searched for the standard codes and the universal codes.

If the code is not found in the tables after the three searches described above, then the Oracle Value is copied to the target field in the message map. Otherwise, the "To Trading Partner Value" will be copied to the target field in the message map.

The following table illustrates the code conversion concepts for outbound transactions using the category code UOM. The Outbound Search Key is comprised of the values for Conversion Table, Standard Code, and Oracle Value. The To Trading Partner Value is the data retrieved.

Conversion Table	Standard Code	Oracle Value	Description	From Trading Partner Value	To Trading Partner Value
Trading Partner or Standard	Appropriate Code for the first, second, third search	Each	Each	(ignore the value here - not relevant for Outbound Transactions)	EA (derive this code to place in the transaction)

The following table illustrates a sample outbound transaction code conversion for the category code UOM. The Outbound Search Key is comprised of the values for Conversion Table, Standard Code, and Oracle Value. The To Trading Partner Value is the data retrieved.

Conversion Table	Standard Code	Oracle Value	Description	From Trading Partner Value	To Trading Partner Value
Trading Partner		Box	Box	(ignore the value here)	BX
Trading Partner	CUSTOM	Each	Each	(ignore the value here)	EA
Standard	OAG	Box	Box	(ignore the value here)	BX
Standard	OAG	Each	Each	(ignore the value here)	EA
Standard	UNIVERSAL	Box	Box	(ignore the value here)	BX

Inbound Transaction Code Conversion Table Access

For inbound transactions, the "From Trading Partner Value" and the Standard Code in the code conversion tables are keys to retrieve the "Oracle Value." First the trading partner codes are searched. If an entry is not found, the Standard code conversion table is searched for the standard codes and then the universal codes.

If the code is not found in the tables after the three searches described above, then the "From Trading Partner Value" is copied to the target field in the message map. Otherwise, the "Oracle Value" will be copied to the target field in the message map.

The following table illustrates the code conversion concepts for inbound transactions using the category code UOM. The Inbound Search Key-1 is comprised of the values for Conversion Table and Standard Code. The Oracle Value is the retrieved data. The From Trading Partner Value is the Inbound Search Key-2.

Conversion Table (Inbound Search Key-1)	Standard Code (Inbound Search Key-1)	Oracle Value (Retrieve)	Description	From Trading Partner Value (Inbound Search Key-2)	To Trading Partner Value
Trading Partner or Standard	Appropriate Code for the first, second, third search	Each (Derive this code to place in the Oracle table)	Each	EA	(Ignore the value here - Not relevant for Inbound Transactions)

The following table illustrates a sample inbound transaction code conversion for the category code UOM. The Inbound Search Key-1 is comprised of the values for Conversion Table and Standard Code. The Oracle Value is the data retrieved. The From Trading Partner Value is the Inbound Search Key-2.

Conversion Table (Inbound Search Key-1)	Standard Code (Inbound Search Key-1)	Oracle Value (Retrieve)	Description	From Trading Partner Value (Inbound Search Key-2)	To Trading Partner Value
Trading Partner		Box	Box	BX	(Ignore the value here)
Trading Partner	CUSTOM	Each	Each	EA	(Ignore the value here)
Standard	OAG	Box	Box	BX	(Ignore the value here)
Standard	OAG	Each	Each	EA	(Ignore the value here)
Standard	UNIVERSAL	Box	Box	BX	(Ignore the value here)
Standard	UNIVERSAL	Each	Each	EA	(Ignore the value here)

One Internal Code to Multiple External Codes

You can use the Action Assign Variable Value in Message Designer to assign multiple external codes given the single Oracle internal code by using conditions on the internal code. You must update the message in Message Designer as new codes are needed.

For example, the single Oracle internal carrier code stored in the SHIP_VIA column may need to cross-reference to two external codes: the carrier and the transportation mode. For example,

If SHIP_VIA = 'TRUCK-LAND'

move 'TRUCK' to the CARRIER.

If SHIP_VIA = 'TRUCK-LAND'

move 'L' to the TRANSPORTATION_METHOD.

If SHIP_VIA = 'TRUCK-AIR',

move 'TRUCK' to the CARRIER.

If SHIP_VIA = 'TRUCK-AIR'
move 'A' to the TRANSPORTATION_METHOD.

Note: For Outbound Transactions:

If you have the SHIP_VIA source data written twice to the file, then code conversion could be performed to convert the two fields separately: Once to derive the carrier code and once to derive the transportation method.

Standard Code Conversion Form

Navigate to the Standard Code Conversion form from the XML Gateway Responsibility by selecting Setup > Define Code Conversion.

The Standard Code Conversion form displays a list of code values that are used by all trading partners unless the codes are overridden by trading partner specific code values. For a discussion of trading partner specific codes see Trading Partner Code Conversion Form, page 3-48.

To read more about Code Conversion features and concepts see Code Conversion, page 3-38.

Category Code

Select a category code to view its list of values in the Category Values portion of the form. A category code cannot be added or deleted.

Description

Description of the category code.

Category Values

Standard Code (Required)

A Standard Code identifies one of the following:

- The source organization for the code value, such as OAG, for the given XML message to be processed.
- The word UNIVERSAL, the seeded code to identify any other organizations such as ISO, X12, EDIFACT.

Note: Only XML standards and the single non-XML standard UNIVERSAL should be defined in the Define Transactions form. The process would not recognize standard codes such as ISO or EDIFACT though they can be entered into this form.

Select a standard code from the list of values.

Use the Define XML Standards form, page 3-10 to add new standard codes as needed.

Oracle Value (Required)

The Oracle Value is a code defined in the Oracle E-Business Suite, regardless if it is an inbound or outbound transaction. The Oracle Value is case-sensitive.

Description

Description of the Oracle Value.

From Trading Partner Value (Used with Inbound Transactions)

The "From Trading Partner Value" is a code in the message that represents data from the trading partner's perspective. This code is found in the inbound source transaction.

To Trading Partner Value (Used with Outbound Transactions)

The "To Trading Partner Value" is a code to be written in the outbound message. It represents data that the trading partner is expecting to receive. You cannot enter data in this field. This code value is always equal to and copied from the "From Trading Partner Value."

Standard Checkbox

The Standard checkbox is enabled for all entries made in the Standard Code Conversion form.

If a code conversion value table entry has the Standard checkbox checked, and the code conversion value is later overridden in the Trading Partner Code Conversion table, this checkbox is switched "off" (made blank) in the Trading Partner Code Conversion form.

The system controls setting this checkbox off and on.

Data Seeded Checkbox

Data can be seeded into the database by the XML Gateway. Seeded data is indicated by a check mark in this checkbox. If data is marked as seeded, it cannot be modified in the forms.

REVERT ALL Button

The Revert All Button is disabled in this form. It is used in the Trading Partner Code Conversion form to revert all codes back to the standard code value found in the standard Code Conversion table up to the last saving of the entries.

All fields except the Standard checkbox, Data Seeded checkbox, and the To Trading Partner Value can be changed.

REVERT Button

The Revert Button is disabled in this form. It is used in the Trading Partner Code Conversion form to revert the codes back to the standard code value found in the standard Code Conversion table up to the last saving of the entries.

Trading Partner Code Conversion Form

The Trading Partner Code Conversion form allows the entry and display of code values for a specific trading partner.

Navigate to this form by clicking the Code Conversion button on the Define Trading Partner Setup form (XML Gateway Responsibility > Setup > Define Trading Partners).

The Trading Partner Code Conversion form contains data from the following sources:

- All trading partner-specific code values entered in this form
- All codes entered in the Standard Code Conversion form. The Standard checkbox on the form indicates if the data was entered in the Standard Code Conversion form. The standard code conversion values can be changed.

To read more about Code Conversion features and concepts see Code Conversion, page 3-38.

Category Code

Select a category code to view its list of values in the Category Values portion of the form. A code category cannot be added or deleted.

Description

Description of the category code.

Category Values

The following fields are found in the Category Values region:

Standard Code

When adding Trading Partner-specific code conversion values, this field defaults to CUSTOM.

Oracle Value

The Oracle Value is a code defined in the Oracle E-Business Suite, regardless if it is an inbound or outbound transaction. The Oracle Value is case-sensitive.

Description

Description of the Oracle Value.

From Trading Partner Value (Used with Inbound Transactions)

The "From Trading Partner Value" is a code in the message that represents data from the trading partner's perspective. This code is found in the inbound source transaction.

To Trading Partner Value (Used with Outbound Transactions)

The "To Trading Partner Value" is a code to be written in the outbound message. It represents data that the trading partner is expecting to receive. You cannot enter data in this field. This code value is always equal to and copied from the "From Trading Partner

Value."

Standard Checkbox

The Standard checkbox is enabled for all entries made in the Standard Code Conversion form.

If a code conversion value table entry has the Standard checkbox checked, and the code conversion value is later overridden in the Trading Partner Code Conversion table, this checkbox is switched "off" (made blank) in the Trading Partner Code Conversion form.

The system controls setting this checkbox off and on.

Data Seeded Checkbox

Data can be seeded into the database by the XML Gateway. Seeded data is indicated by a check mark in this checkbox. If data is marked as seeded, it cannot be modified in the forms.

REVERT ALL Button

The Revert All Button in the Trading Partner Code Conversion form reverts all codes back to the standard code value found in the standard Code Conversion table up to the last saving of the entries.

All fields except the Standard checkbox, the Data Seeded checkbox, and the To Trading Partner Value can be changed.

REVERT Button

The Revert Button in the Trading Partner Code Conversion form reverts the codes back to the standard code value found in the standard Code Conversion table up to the last saving of the entries.

Note: The Revert Button applies to only the selected entry for this specific trading partner.

What Can Be Updated

If you are adding a Custom code, all fields except the Standard checkbox, and Data Seeded checkbox can be changed.

If the Standard Code is not "CUSTOM" then only the From Trading Partner Value and the To Trading Partner Value can be changed. The Standard checkbox will be enabled.

Execution Engine

This chapter covers the following topics:

- Execution Engine Overview
- Protocol Type
- XML Gateway Envelope
- Trading Partner Validation for Inbound Messages
- Trading Partner Validation for Outbound Messages
- Dynamic Embedding of XML Fragments
- How to Implement the OAG Confirmation Business Object Document

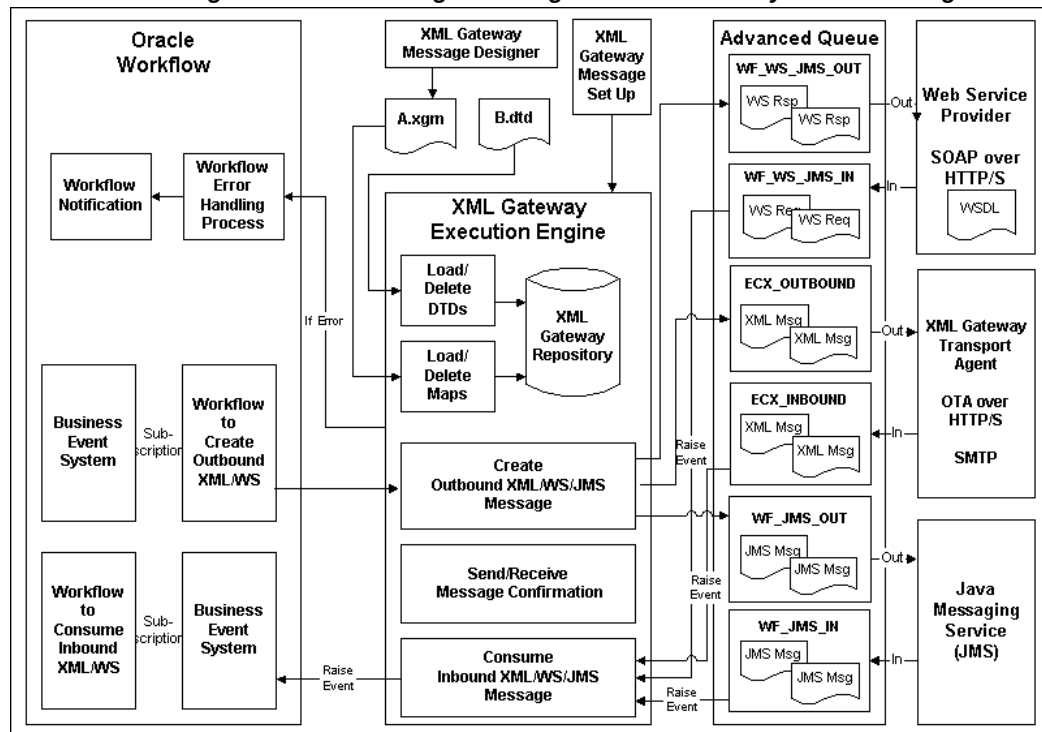
Execution Engine Overview

The XML Gateway consists of three components: the Message Designer, Setup, and the Execution Engine. It interfaces with the following Oracle products:

- Oracle Transport Agent for message delivery
- Oracle Advanced Queuing for message propagation, and queue management
- Oracle Workflow Business Event System to publish and subscribe to business events. Workflow also provides an active e-mail notification to report errors detected by the XML Gateway Execution Engine, Advanced Queuing (AQ), or the transport agent

The following diagram illustrates the process flow of XML messages through the XML Gateway Execution Engine using all the components mentioned above. Details of the XML Gateway Execution Engine are summarized below.

Process Flow Diagram of XML Messages Through the XML Gateway Execution Engine



The XML Gateway Execution Engine can process messages properly after the following:

- Message maps are created and loaded into the repository along with their associated DTDs.
- Trading Partners are defined.
- Code Conversions are defined.
- Transactions are defined.
- The value of ECX_IN_JAVA_PROCESSING profile option is retrieved for processing inbound messages.

The XML Gateway execution engine retrieves the profile value to determine if inbound messages will be executed in Java or in PL/SQL.

For more information about the ECX_IN_JAVA_PROCESSING profile option, see Define System Profile Options, page 3-3.

- Oracle Workflow Business Event System events are published by the Oracle E-Business Suite and subscriptions to those events are defined.
- Engine and listeners are started.

The XML Gateway listeners are actively polling for interested events. The Execution Engine will begin processing once Oracle Workflow Business Event System detects an outbound transaction to be processed, or that an inbound message has arrived on the queue.

The XML Gateway Execution Engine does the following during processing:

- **(Inbound Message) Dequeue Message from Inbound Queue**

- **Validate Message via XML Parser**

(Inbound Message) Uses the XML Parser to validate the inbound message to determine if it is well-formed and valid (based on a DTD stored in the DTD directory) before proceeding further.

(Outbound Message) Uses the XML Parser to validate the newly created message to ensure that it is well-formed and valid. A poorly formed or invalid message (based on a DTD stored in the DTD directory) will not be enqueued onto the Outbound Queue.

- **Validate Trading Partner or Hub**

If the inbound message is both well-formed and valid, the Execution Engine proceeds to validate that the Trading Partner and document are defined. If the Trading Partner is not defined or the document is not defined for the Trading Partner, the XML message will not be processed further.

- **Get Message Map from Repository**

If the message map associated with the Trading Partner is not available in the XML Gateway repository, the XML message will not be processed further.

(Inbound Messages) Maintain Maps Cache if Processing in Java

When the execution engine receives payload for processing inbound messages in Java, the engine will check if the map code associated with the trading partner and transaction was loaded.

- If the map code was already loaded, the engine checks if map was already loaded in cache and then uses already loaded metadata for further processing of the XML messages.
 - If the map code was not loaded, then load the map metadata from database and store into memory.
- **Execute the Message Map**
 - **(Outbound Messages) Gather Application Data**

If the Trading Partner is valid and the message map exists in the repository, the Execution Engine gathers the application data from the Oracle E-Business Suite

using the database views and columns identified in the message map.

- **(Inbound Messages) Maps Data**

If the Trading Partner is valid and the message map exists in the repository, the Execution Engine maps the data in the XML message to its target data fields in Oracle E-Business Suite tables and columns identified in the message map. These are often the Application Open Interface tables.

- **Apply Code Conversion**

Apply code conversion for source columns enabled for code conversion.

- **Apply Actions**

Apply actions where defined (may be document, element, or root level).

(Inbound Message) Action codes include inserting data into the Application Open Interface tables, then executing the Application Open Interface API to populate the base application tables.

- **(Outbound Messages) Create XML Message**

Create XML message using the message map and the application data as described above.

- **Dynamically Add XML Fragments**

XML Gateway allows the dynamic addition of XML Segments at runtime as a child, not a sibling, to any element defined on the Outbound XML Gateway Map.

- **Detect and Report Processing Errors**

Errors may be detected by the Oracle XML Gateway Execution Engine, Oracle Advanced Queuing, Oracle Workflow, or Oracle Transport Agent. Information regarding the error is enqueued onto the Error Queue. An e-mail notification is sent via Oracle Workflow to notify the trading partner regarding data errors, or the XML Gateway system administrator regarding system or process errors.

In addition, for system or process errors, a copy of the XML message is placed in the XML message directory for use in troubleshooting the reported error. For trading partner-related data errors, the trading partner can refer to the copy of the XML message.

- **Copy XML Message**

For system or process errors, a copy of the XML message is placed in the XML message directory for use in troubleshooting the reported error. For trading partner-related data errors, the trading partner can refer to their copy of the XML message.

- **(Outbound Messages) Enqueue Message to Outbound Queue**

Enqueue well-formed and valid message onto the Outbound Queue. Oracle Transport Agent will dequeue the message from the Outbound Queue and deliver it to the trading partner.

- **Create or Consume Confirmation Messages**

(Inbound Message) Receive confirmation message, if it is enabled for the Trading Partner and confirmation is requested on the outbound message, or a default is set up for the trading partner.

(Outbound Message) Create a confirmation message, if it is enabled for the Trading Partner and confirmation is requested on the inbound message, or a default is set up for the trading partner.

Protocol Type

Each message enabled for a Trading Partner includes a protocol type (also known as communication method) that identifies the message delivery method. The associated correlation ID is determined internally and is associated with the message enqueued onto the agent (queue). Listeners defined for the agent will dequeue messages based on the correlation ID defined for it. For example, Oracle Transport Agent will only dequeue messages with a correlation ID of OXTA.

The following table shows the Protocol Type, Correlation ID, and Message System.

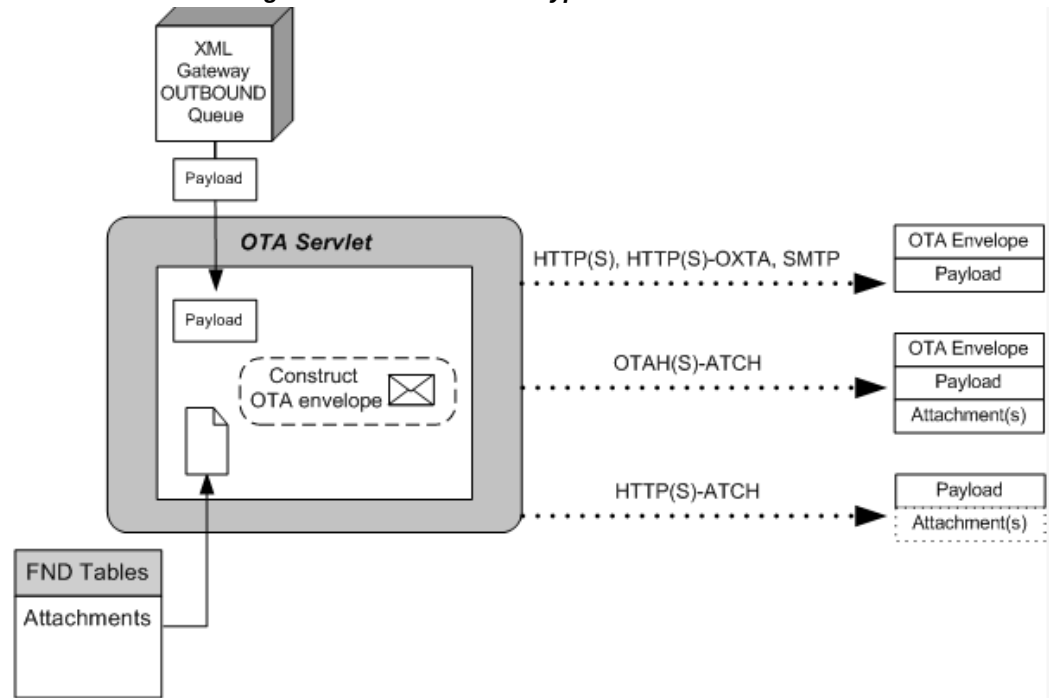
Protocol Type	Correlation ID	Message System
NONE	N/A	Message disabled
HTTP, HTTP-OXTA, HTTPS, HTTPS-OXTA, SMTP	OXTA	Oracle Transport Agent without attachments
OTAH-ATCH, OTAHS-ATCH	OXTA	Oracle Transport Agent with attachment using standard OTA envelope
HTTP-ATCH, HTTPS-ATCH	OXTA	Oracle Transport Agent with attachment and no OTA envelope
ITG03	ITG03	Oracle iProcurement Connector, must be enabled
IAS	IAS	Oracle Application Server, must be enabled

Protocol Type	Correlation ID	Message System
HTTP-WM, HTTPS-WM	WebMethods	Third party system available only to users of Oracle Exchange
SOAP	N/A	Web Service Agent
JMS	N/A	JMS Provider

The following graphics display the construction for each protocol type:

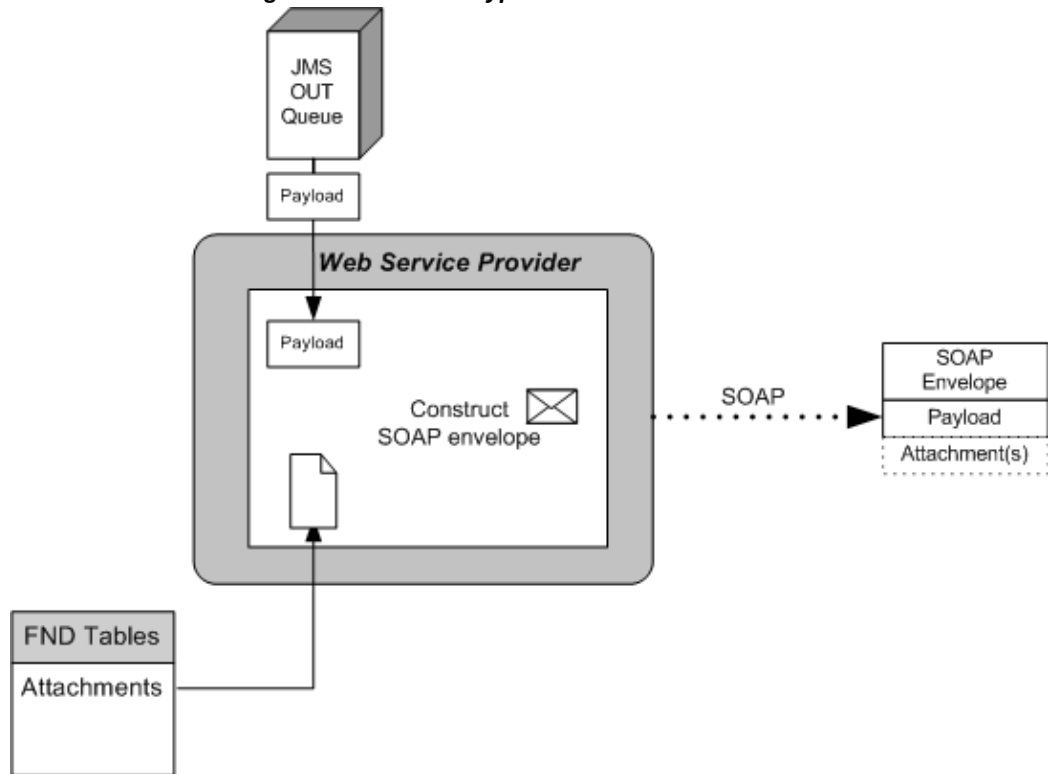
- For protocol types HTTP(S) and HTTP(S)-OXTA, the OTA servlet dequeues the message payload and constructs the OTA envelope.
- For protocol type OTAH(S)-ATCH, the OTA servlet dequeues the message payload, constructs the OTA envelope, and assembles the attachment(s).
- For protocol type HTTP(S)-ATCH, the OTA servlet dequeues the message payload and assembles any attachments, if present.

Construction Flow Diagram for Each Protocol Type



For protocol type SOAP, the Web Services provider dequeues the message payload, assembles any attachments, and constructs the SOAP envelope.

Construction Flow Diagram for Protocol Type SOAP



XML Gateway Envelope

In addition to the business document such as a purchase order or invoice in the XML Payload, a set of message attributes are transmitted. Collectively, these attributes are called the XML Gateway envelope.

This section discusses the XML Gateway envelope and its data in the validation process for inbound messages, or its source of data for its creation for outbound messages. Data entered into the Trading Partner Setup form is referred to as data in the trading partner table. Data entered into the Define Transactions form is referred to as data in the transaction table.

Most of the data elements are copied from the Trading Partner tables or the Transaction tables to the XML Gateway envelope.

Transaction direction is determined by the XML Gateway. The direction values used by the XML Gateway are the following:

- IN for inbound messages
- OUT for outbound messages

The XML Gateway envelope consists of the data presented in the following table:

Attribute	Contents	Sample Values	Data Source
MESSAGE_TYPE	Payload message format	XML	(Hard-coded)
MESSAGE_STANDARD	Message format standard	OAG	Transaction table
TRANSACTION_TYPE	External Transaction Type for that business document	INVOICE	Trading Partner table
TRANSACTION_SUBTYPE	External Transaction Subtype for that business document	PROCESS	Trading Partner table
DOCUMENT_NUMBER	Business document number such as invoice number	(Not Used)	N/A
PARTYID	(Not Used)	(Not Used)	Trading Partner table. This is not used by the XML Gateway.
SOURCE_TP_LOCATION_CODE	Source Trading Partner Location Code	ACME_CHICAGO	SOURCE TP LOCATION (if not recreated) or TARGET TP LOCATION CODE (if recreated) from Trading Partner table
PARTY_TYPE	(Not Used)	(Not Used)	(Not Used)
PROTOCOL_TYPE	Transmission Protocol	SMTP, HTTP, HTTP-WM	Trading Partner table
PROTOCOL_ADDRESS	Transmission Address	me@example.com, http://www.example.com:5555	Trading Partner table
USERNAME	User Name	User1	Trading Partner table if Connection is "DIRECT"; Hub Definition if "HUB"
PASSWORD	password	*****	Trading Partner table if Connection is "DIRECT"; Hub Definition if "HUB"

Attribute	Contents	Sample Values	Data Source
ATTRIBUTE1	(determined by application)	N/A	(determined by application)
ATTRIBUTE2	(determined by application)	N/A	(determined by application)
ATTRIBUTE3	Rerouting Location (used only if the message is rerouted)		TARGET TP LOCATION CODE from Trading Partner table
ATTRIBUTE4	(determined by application)	N/A	(determined by application)
ATTRIBUTE5	(determined by application)	N/A	(determined by application)
PAYLOAD	XML business document	(XML Message)	

MESSAGE_TYPE

Payload message format. This defaults to "XML".

MESSAGE_STANDARD

(Defaults to OAG.) Message format standard as displayed in the Define Transactions form and entered in the Define XML Standards form.

TRANSACTION_TYPE

External Transaction Type for the business document from the Trading Partner table.

TRANSACTION_SUBTYPE

External Transaction Subtype for the business document from the Trading Partner table.

DOCUMENT_NUMBER

The document identifier used to identify the transaction, such as a purchase order or invoice number. This field is not used by the XML Gateway, but it may be passed on inbound messages.

SOURCE_TP_LOCATION_CODE

Source Trading Partner Location Code if no data is found in the Destination Trading Partner Location Code in the Trading Partner table.

PROTOCOL_TYPE

Transmission Protocol as defined in the Trading Partner table.

PROTOCOL_ADDRESS

Transmission address as defined in the Trading Partner table.

USERNAME

USERNAME as defined in the Trading Partner table.

PASSWORD

The password associated with the USERNAME defined in the Trading Partner table.

ATTRIBUTE3

For outbound messages, this field has the value from the Destination Trading Partner Location Code in the Trading Partner table.

For inbound messages, the presence of this value generates another XML message that is sent to the trading partner identified in the Destination Trading Partner Location Code in the Trading Partner table. This value must be recognized by the hub to forward the XML message to the final recipient of the XML Message. Refer to XML Gateway Setup, page 3-1 for details.

PAYLOAD

The XML message.

Parameters defined by the Application

The following parameters may be defined by the base application:

- ATTRIBUTE1
- ATTRIBUTE2
- ATTRIBUTE4

- ATTRIBUTE5

Parameters Not Used

The following parameters are not used:

- PARTYID
- PARTYTYPE

Trading Partner Validation for Inbound Messages

Events Raised in Oracle Workflow Business Event System

Oracle Workflow Business Event System is a tool used to integrate with the Oracle E-Business Suite for event-based XML message creation and consumption.

Before XML Gateway processes the message:

Messages sent by the trading partner are staged onto a message queue. An Oracle Workflow listener dequeues the message from the message queue and raises an event to the XML Gateway to begin processing.

Refer to Message Queues, page 5-1 for details.

After XML Gateway processes the message:

When XML Gateway completes processing the data according to the instructions in the message map associated with the trading partner, an event is published by XML Gateway to inform the Oracle E-Business Suite that it has successfully processed the inbound message. Any Oracle E-Business Suite module interested in this event may register a subscription to continue with the transaction.

XML Gateway Processing

Inbound messages can take two paths:

- Standard XML Messages

If ATTRIBUTE3 in the XML Gateway envelope is NULL, the XML message is processed according to the message map.

- "Pass-Through" Messages

If ATTRIBUTE3 in the XML Gateway envelope has data, the inbound message will be recreated as an outbound message according to a message map for the trading partner identified in ATTRIBUTE3. This transaction is referred to as a "pass-through" message and defined as a message with "Dynamic Routing." Refer to Static and Dynamic Routing, page 3-31 for details.

Standard XML Messages

The following table summarizes the required data in the XML Gateway envelope to identify standard XML messages. How this data is validated for an inbound message is described below.

Attribute	Sample Values	Used in Trading Partner Lookup
MESSAGE_STANDARD	OAG	YES
TRANSACTION_TYPE	INVOICE	YES
TRANSACTION_SUBTYPE	PROCESS	YES
PARTY_SITE_ID	ACME_CHICAGO	YES
ATTRIBUTE3	NULL	NO
PAYLOAD	(XML MESSAGE)	N/A

Note: USERNAME and PASSWORD can be placed in the XML Gateway envelope by the system that created the XML message. The transport agent uses the USERNAME and PASSWORD for delivery purposes. These fields are not passed to the XML Gateway from the transport agent software.

"Pass-Through" XML Messages

XML Gateway can recreate, then route messages to another trading partner without loading the transaction into a base Oracle E-Business Suite application. This case is recognized by the presence of a trading partner code in ATTRIBUTE3 in the XML Gateway envelope in the inbound message.

If a trading partner location code is identified in ATTRIBUTE3 for an inbound message the following happens:

- A Trading Partner Setup table entry for an *outbound message* for this same TRANSACTION_TYPE and TRANSACTION_SUBTYPE is found for the Trading Partner identified in ATTRIBUTE3. The values in ATTRIBUTE3 must match an entry in the Destination Trading Partner Location Code defined in the Trading Partner Setup form.

- If the entity in ATTRIBUTE3 is found in the Trading Partner Setup tables, then another XML Gateway message is created according to the message map for that trading partner. This new XML message is then routed to the trading partner identified in the original ATTRIBUTE3.

The following table summarizes the required data in the XML Gateway envelope to identify these messages, and key data for the pass through XML message.

Attribute (Original XML Message)	Sample Inbound Message Values	Pass-Through Message Created (Find its corresponding Trading Partner Setup for an OUTBOUND message where the Trading Partner is identified in ATTRIBUTE3 of the original message)
MESSAGE_STANDARD	OAG	OAG
TRANSACTION_TYPE	INVOICE	INVOICE
TRANSACTION_SUBTYPE	PROCESS	PROCESS
PARTY_SITE_ID	ACME_CHICAGO	BETA-LONDON (from original ATTRIBUTE3)
ATTRIBUTE3 (the DESTINATION TRADING PARTNER LOCATION CODE in the Trading Partner Setup table. This becomes the PARTY_SITE_ID for the outbound message.)	BETA-LONDON	
PAYLOAD	(XML Message)	(XML Message)

Validation Against Data in the Trading Partner Setup Form

Inbound messages validate data only against data that was entered in the Trading Partner Setup form.

Source Trading Partner Location Code

Source Trading Partner Location Code is the code found in the XML envelope in PARTY_SITE_ID to identify the source of the message. This field must contain a value stored in Source Trading Partner Location Code in the Trading Partner setup table.

When a trading partner is entered into the Trading Partner Setup form, the Trading Partner Name and Trading Partner Site are selected from a list of values obtained from Oracle Receivables, Oracle Payables, or Oracle HR Locations. Associated with each Trading Partner site is a table ID in the base application. See Derive Address ID from Location Code action, page 2-80 and Get Predefined Variable Value action, page 2-96 if you wish to use this table ID in a message map.

The parameters listed in the table below compose a search key against the Trading Partner table. This key is used to determine if the Trading Partner is enabled for that transaction, then to retrieve the message map name to process the transaction.

Search Parameters	Description	Sample
PARTY_SITE_ID	Source Trading Partner Location Code	ACME_CHICAGO
TRANSACTION_TYPE	External Transaction Type for that business document	INVOICE
TRANSACTION_SUBTYPE	External Transaction Subtype for that business document	PROCESS
DIRECTION	Determined by the XML Gateway	IN

The following table shows data that is returned from a search against data that was entered in the Trading Partner Setup form:

Returned Data	Description	Sample
Trading Partner Site enabled for the message	This data is stored in the table given the trading partner that was selected. This data is not displayed.	12345 (a table ID)
Message Map to use for this inbound transaction for the given trading partner		OAG_IN_INVOICE

Note that transaction direction is determined by the XML Gateway. The direction values used by the XML Gateway are the following:

- IN for inbound messages
- OUT for outbound messages

Trading Partner Validation for Outbound Messages

Event Raised in Oracle Workflow Business Event System

Application business events for outbound messages are raised by the Oracle E-Business Suite module responsible for the transaction. The raise occurs at an application point of interest, such as when a purchase order is approved or an invoice is confirmed.

Event Subscription Defined in Oracle Workflow Business Event System

A corresponding event subscription is defined to consume the event. The event subscription may be based on the XML Gateway Rule Function or the Workflow Default Rule Function. Embedded in the XML Gateway Rule Function is a check to determine if the Trading Partner is defined and the message is enabled.

With the Workflow Default Rule Function, the Transaction Delivery Required function activity is used to make the same evaluation. The outbound document is generated and sent only if the Trading Partner is defined and the transaction is enabled. Which approach is used depends on what is known about the Trading Partner before the subscription is executed.

See: Integrating Oracle XML Gateway with Oracle Workflow Business Event System, page 6-1.

Dynamic Embedding of XML Fragments

XML Gateway can dynamically add XML segments at runtime as a child to any element defined on the Outbound XML Gateway Map during message generation.

To use this feature, users should create an In Process 'Execute Procedure' Action to execute procedure `ECX_ACTIONS.GET_XML_FRAGMENT` at the element under which the dynamic XML segment needs to be added. This procedure is available under APPS schema.

The API Signature

```
ECX_ACTIONS.get_xml_fragment  
(proc_name IN varchar2,  
 xml_fragment OUT NOCOPY varchar2);
```

- **Proc_name:** Name of the called procedure. This procedure should have the signature with the first IN parameter `Event` as `wf_event_t` and second OUT parameter `xml_fragment` as `varchar2`. In other words, this procedure takes `wf_event_t` as an input parameter and gives `xml_fragment` as an output.

For example,

```
my_pkg.my_procedure  
(event IN wf_event_t,  
 xml_fragment OUT NOCOPY varchar2);
```

The workflow event passed as an input parameter is packaged with all the global variables. Users need to use global variables in case context needs to be passed to the called procedure.

- `xml_fragment`: Name of the node at which the XML fragment will be returned.

How to Implement the OAG Confirmation Business Object Document

Purpose of the Confirmation Message

The purpose of the confirmation message is to communicate the status of a business document. In addition to providing general document status information, details regarding any errors detected may be included in the confirmation message.

The confirmation message is a general-purpose document that may be used to acknowledge any business document. It may be used in addition to but not as a substitute for specific purpose acknowledgements such as Purchase Order Acknowledgment or Purchase Order Change Acknowledgment documents.

Refer to www.openapplications.org for details regarding the Confirm Business Object Document (BoD) DTD.

Structure of the Confirmation Message

The confirmation message contains the following two parts:

- `CONFIRM_BOD/CONFIRM`

The `CONFIRM` data type is used to describe the business document it is acknowledging. Included are the document status, description of the document, and an identifier for the original business document.

- `CONFIRM_BOD/CONFIRMMSG`

The `CONFIRMMSG` data type is used to provide a detailed description of the document status. Included are the description and any reason codes. Each `CONFIRM` data type may have many `CONFIRMMSG` data types associated with it.

XML Gateway Seeded Confirmation Message Maps

Oracle XML Gateway provides seeded maps for both the inbound and outbound Confirm BoD message. The names of the seeded message maps are as follows:

- `ECX_CBODI_OAG72_IN_CONFIRM`
- `ECX_CBODO_OAG72_OUT_CONFIRM`

The XML Gateway seeded maps are incorporated into the Oracle E-Business Suite

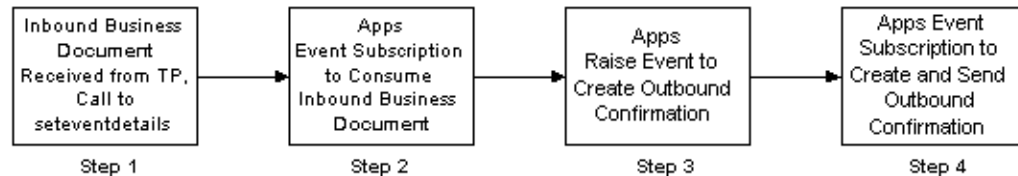
application business process where necessary.

E-Business Suite Seeded Events and Event Subscriptions

Usage of the Confirm BoD varies between application modules of the Oracle E-Business Suite. For application modules that have incorporated the Confirm BoD into their business processes, the process is modeled as follows:

Outbound Confirmation

Outbound Confirmation Process Flow Diagram



Oracle XML Gateway sends an outbound confirmation message in response to an inbound business document.

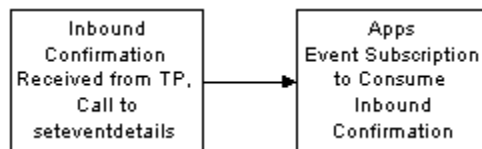
The inbound business document initiates the process (Step 1). As with all inbound messages, XML Gateway notifies the Oracle E-Business Suite application that it has processed an inbound business document.

The application event subscription defined for the inbound document is executed (Step 2) and is followed by a raise event (Step 3) to notify XML Gateway to create and send the outbound confirmation (Step 4).

Steps 2 and 3 are defined separately to allow for application business processes between the consumption of the inbound business document and the creation and delivery of the outbound confirmation message.

Inbound Confirmation

Inbound Confirmation Process Flow Diagram



Oracle XML Gateway will process an inbound confirmation message received from a the Trading Partner in response to an outbound business document. It will notify the Oracle E-Business Suite that it has processed an inbound confirmation message.

The application event subscription defined for the inbound confirmation message is

executed. The behavior of the event subscription will vary by application module and is dependent on whether the application module is interested in negative, positive, or both types of confirmations.

How to Implement or Disable a Seeded Confirmation Message

The application event subscriptions to create and send an outbound message or to consume an inbound message are delivered by the Oracle E-Business Suite application modules.

The seeded event name for the inbound or outbound Confirm BoD message is as follows:

ORACLE.APPS.<COMPONENT>.<TASK>.CONFIRM

COMPONENT is the internal transaction type entered on the Define Transactions form. It represents the product short name.

TASK is the internal transaction subtype entered on the Define Transactions form. It represents a description of the object.

An example of an event name for a confirmation event associated with an outbound purchase order is: ORACLE.APPS.PO.POO.CONFIRM.

The corresponding seeded event subscription is defined as enabled. Use the Oracle Workflow Administrator: Add Events/Event Group window to view or disable the seeded events and event subscriptions if you do not wish to implement the Confirm BoD message.

To implement the seeded event subscriptions, define the Trading Partner and enable the confirmation message as follows:

1. Define the Trading Partner using the XML Gateway Define Trading Partners form.
2. Enable the business document and set the Document Confirmation flag to one of the following:

1 = Send confirmation only if there are errors

2 = Always send confirmation

Important: Ensure that for an outbound business document you enable an inbound confirmation document (Transaction Subtype CBODI); and for an inbound business document you enable an outbound confirmation document (Transaction Subtype CBODO).

Note: Make sure that the value for Source Trading Partner Location Code is the same for the original business document and its corresponding confirmation document.

Refer to Trading Partner Setup, page 3-19 for more details on setting up the Trading Partner to implement the confirmation message.

Message Queues

This chapter covers the following topics:

- Queues

Queues

Queues are tables on a database that are managed by Oracle Advanced Queuing.

The XML Gateway uses queues specifically at two points in the process, as well as employing a general error queue. The first point is at the transport agent level between the transport agent module and the XML Gateway. The second point is at the transaction level between base Oracle E-Business Suite products or other source process, and the XML Gateway. Details about these queues are provided in the table below.

Queue Name	Description	Level and Position in Process
ECX_INBOUND	Inbound Message Queue: Holds all messages that enter the process through the Transport Agent, or are placed directly on the queue by an API.	Transport Agent Level Between the Transport Agent and the Inbound Transaction Queue
ECX_OUTBOUND	Outbound Message Queue: XML Gateway enqueues all outbound messages that it formatted on this queue.	Transport Agent Level Between the XML Gateway and the Transport Agent

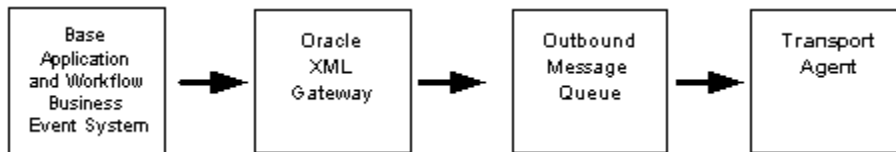
Queue Name	Description	Level and Position in Process
ECX_IN_OAG_Q	Inbound Transaction Queue: Holds inbound messages that originated from the ECX_INBOUND queue, then enqueued on this queue by Oracle Workflow. These messages will be processed by the XML Gateway Java or PL/SQL engine depending on the profile value set at the ECX_IN_JAVA_PROCESSING profile option.	Intermediate Level After the message is received through the Transport Agent (Transport Agent Level above), Oracle Workflow enqueues the message in this queue for the XML Gateway to process. If the profile option ECX_IN_JAVA_PROCESSING is set to 'Yes', inbound messages will be processed in Java. If the profile value is set to 'No', inbound messages will be continued in PL/SQL processing.
WF_ERROR	Workflow Error Queue: For errors detected by XML Gateway or WF BES. Notifications are sent to a Trading Partner contact or the System Administrator.	Transaction Level

Refer to the *Oracle Streams Advanced Queuing User's Guide* from Oracle Database Documentation Library for details on checking the status and contents of the queues. Also, refer to the XML Gateway B2B Transactions Using JMS Queues chapter in this book for details on JMS queues.

Outbound Queues

Outbound Message Queues

Components and Flow Diagram of Processing Outbound Messages



The outbound Message Queue is positioned between the XML Gateway and the

Transport Agent.

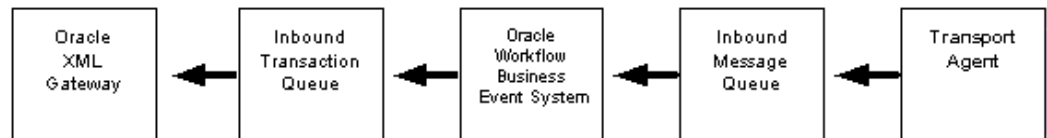
The XML Gateway creates XML messages, then enqueues them on this queue.

Next Step: The Transport Agent dequeues the message and delivers it to the Trading Partner.

Inbound Queues

Inbound Message Queue

Components and Flow Diagram of Processing Inbound Messages



Inbound messages are first queued on the Inbound Message Queue.

The inbound Message Queue is positioned between the Transport Agent and the Oracle Workflow Business Event System.

There are two methods for placing a message on the Inbound Message Queue:

- The Transport Agent enqueues the message.
- An API writes directly to the queue.

Refer to the *Oracle Streams Advanced Queuing User's Guide* for details. Also, see XML Gateway B2B Transactions Using JMS Queues chapter in this book for details on JMS queues.

The full message must be formatted according to the XML Gateway envelope message format described in the Execution Engine section. See XML Gateway Envelope, page 4-8 for details.

Next Step: Oracle Workflow Business Event System will copy the inbound messages to the proper inbound Transaction Queue. One queue is seeded, but you may define other queues to meet your business needs. See information below.

Inbound Transaction Queue

This is the second queue for an inbound message.

After the message is received through the inbound Message Queue, Oracle Workflow Business Event System enqueues the message to this queue for the XML Gateway to process.

Next Step: The XML Gateway engine dequeues the message and retrieves the ECX_IN_JAVA_PROCESSING profile value to determine if the inbound message

placed in the queue will be processed in Java or in PL/SQL. For more information about this profile option, see Define System Profile Options, page 3-3.

Error Queue

When an error is detected by Oracle XML Gateway or Oracle Workflow Business Event System the message is enqueued on to the error queue. A Workflow listener dequeues the error and sends a notification to the Trading Partner contact for data errors or to the System Administrator contact for system or process errors.

XML Gateway Message Format

XML messages that are enqueued or dequeued for processing by XML Gateway must conform to the `apps_ne.ecxmsg` data type.

The `apps_ne.ecxmsg` data type consists of the following:

Name	Type
MESSAGE_TYPE	VARCHAR2(80)
MESSAGE_STANDARD	VARCHAR2(80)
TRANSACTION_TYPE	VARCHAR2(80)
TRANSACTION_SUBTYPE	VARCHAR2(80)
DOCUMENT_NUMBER	VARCHAR2(80)
PARTYID	VARCHAR2(80)
SOURCE_TP_LOCATION_CODE	VARCHAR2(80)
PARTY_TYPE	VARCHAR2(80)
PROTOCOL_TYPE	VARCHAR2(200)
PROTOCOL_ADDRESS	VARCHAR2(200)
USERNAME	VARCHAR2(200)
PASSWORD	VARCHAR2(200)
ATTRIBUTE1	VARCHAR2(200)

Name	Type
ATTRIBUTE2	VARCHAR2(200)
ATTRIBUTE3	VARCHAR2(200)
ATTRIBUTE4	VARCHAR2(200)
ATTRIBUTE5	VARCHAR2(200)
PAYLOAD	CLOB

Refer to XML Gateway Envelope, page 4-8 for a description of each field.

Non-Oracle messaging systems can enqueue an XML message for the Oracle E-Business Suite to process. This is done by defining an object of the same data type, populating the object, then calling an Advanced Queuing enqueue API to enqueue the object. The Oracle E-Business Suite Transport Agent will receive the object and deposit it onto the ECX_OUTBOUND agent. From there, the standard inbound agent listener assumes control of the process.

Integrating Oracle XML Gateway with Oracle Workflow Business Event System

This chapter covers the following topics:

- Integrating Oracle XML Gateway with Oracle Workflow Business Event System
- Oracle Workflow Builder - Item Types
- XML Gateway Standard Item Type
- XML Gateway Error Processing Item Type
- Configure Oracle Prebuilt Inbound Messages
- Configure Oracle Prebuilt Outbound Messages
- Application to Application Integration
- Manage Workflow Processes
- Monitor Workflow Processes
- Development Guidelines for Custom Messages for B2B Integration
- Common Questions
- Message Delivery Status

Integrating Oracle XML Gateway with Oracle Workflow Business Event System

Overview

Oracle Workflow is a process management tool used to seamlessly integrate Oracle E-Business Suite business processes together. Introduced with Oracle Workflow 2.6 is the Business Event System to further support business process integration at the business event level.

Integral to the Business Event System is the ability to raise a business event to signal that something of interest has occurred in the Oracle E-Business Suite. Using Oracle Purchasing as an example, an event of interest would be the creation, change, confirmation, or deletion of a purchase order.

Complementary to the raise event is an event subscription defined to perform some activity when it detects an event of interest has occurred. An example of an event subscription in our Oracle Purchasing scenario is to send a message to the appropriate supplier when a new order is created, or when an existing order is changed, confirmed, or deleted.

An event subscription is controlled by a rule function. The result of the rule function determines whether the associated Workflow process will be executed. The actual activity performed by the event subscription is based on the Workflow process defined for it.

In the context of the integration between the Oracle Workflow Business Event System and Oracle XML Gateway, business events and their corresponding event subscriptions are seeded and deployed to the Oracle E-Business Suite. Once the Trading Partner is defined, the seeded business events and event subscriptions may be implemented as is, or they may be configured to perform specific activities relevant to the business requirements.

Application business events for outbound messages are raised by the Oracle E-Business Suite application module responsible for the message and processed by the event subscription defined for it. The seeded event subscription for the B2B integration may be based on the XML Gateway Rule Function or the Workflow Default Rule Function.

Embedded in the XML Gateway Rule Function is the Transaction Delivery Required Function used to determine if the Trading Partner is enabled for the transaction. The XML Gateway Rule Function can therefore be simply represented as follows:

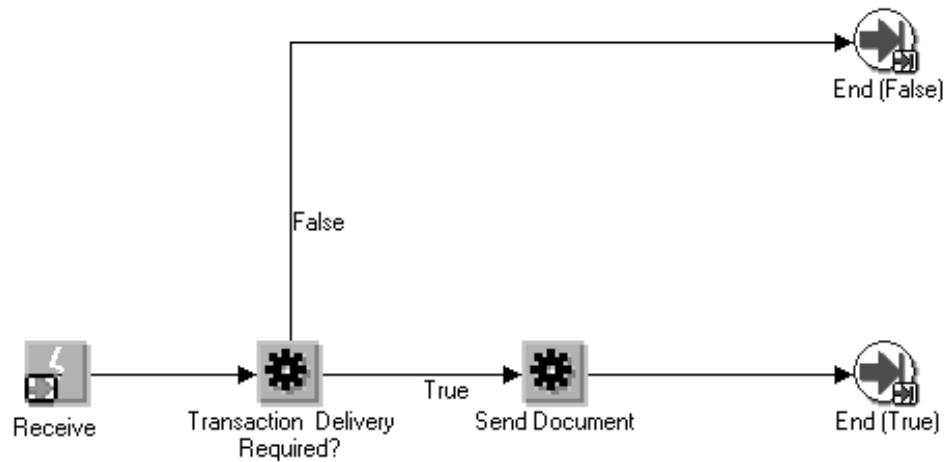
XML Gateway Rule Function Process Flow Diagram



The Receive Event is followed by the Send Document function activity which proceeds to End.

The Workflow Default Rule Function uses the Transaction Delivery Required function as a process activity to determine if the Trading Partner is enabled for the transaction. This process consists of the following steps: The Receive Event is followed by the Transaction Delivery Required function activity. If it returns TRUE, the Send Document function activity is executed. If FALSE, the process proceeds to End. This process is shown below:

Workflow Default Rule Function Process Flow Diagram



With either rule function, no message is sent if the Trading Partner is not enabled for the transaction.

For efficiency, the XML Gateway Rule Function is the recommended approach as it has already predetermined that the Trading Partner is enabled for the transaction before it proceeds with generating the message. However, the Workflow Default Rule function approach provides a visual audit trail of the Workflow process.

With either approach, Oracle XML Gateway gathers the data from the Oracle E-Business Suite, creates the XML message, and delivers it to the Trading Partner for Business to Business (B2B) integration or to another application module for Application to Application (A2A) integration.

Inbound messages received from the Trading Partner for Business to Business (B2B) integration are enqueued onto the inbound queue. Inbound messages received from another application module for Application to Application (A2A) integration are stored in the Event Message attribute (of the Consume XML Document function) to be accessed by the XML Gateway execution engine for processing.

An XML Gateway inbound rule function is executed in the B2B integration to validate the Trading Partner and to identify the transaction queue. If the Trading Partner is valid, the message is enqueued onto the transaction queue for processing by the XML Gateway execution engine.

The XML Gateway execution engine processes the data according to the instructions in the message map associated with the Trading Partner and message. If successful, the message map sets the Event Details to inform the Oracle E-Business Suite of the Event. It is the application of the map to the message that enables you to determine what the Event Details are. The event is then processed by an application event subscription interested in the inbound message.

Oracle Workflow Business Event System consists of three key components that are used

to integrate with the Oracle E-Business Suite to create and consume XML messages:

- The Oracle Workflow Builder contains the Oracle Workflow Standard Item Type and the Oracle XML Gateway Standard Item Type. The components of the standard item types can be used to define Workflow processes.
- Oracle Workflow Administrator used to register new events and event subscriptions as well as configure seeded event subscriptions.
- The Oracle Workflow processing engine used to execute the workflow processes.

Oracle Workflow Builder - Item Types

Delivered with XML Gateway are two standard item types:

- XML Gateway Standard Item Type
- XML Gateway Error Processing Item Type

An Item Type is a group of components of a particular category that share the same set of item attributes (also known as variables).

Both the XML Gateway Standard Item Type and XML Gateway Error Processing Item Type can be accessed using the Oracle Workflow Builder.

Components of an Item Type

Common to all item types are the following components:

- Attributes
- Process Activity
- Notification Activity
- Function Activity
- Event Activity
- Messages
- Lookup Types

A Workflow process may be made up of event activities, function activities, notification activities, or process activities. A message is the information sent in a notification activity. Lookup types represent a list of valid values. Attributes represent variables used to share data between activities in the Workflow process.

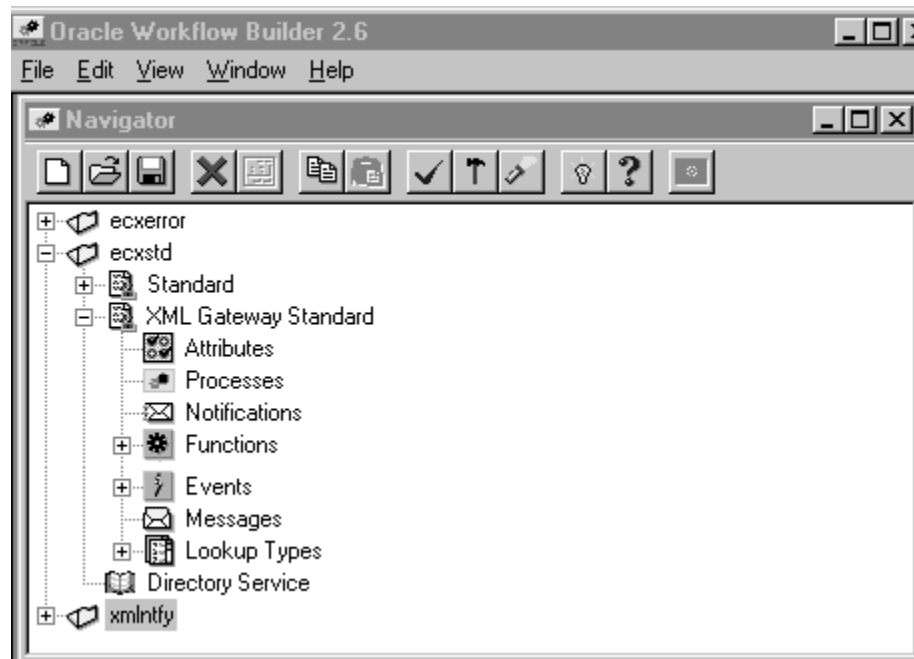
See the *Oracle Workflow User's Guide* for more information.

XML Gateway Standard Item Type

The XML Gateway Standard Item Type is a toolkit consisting of attributes, processes, functions, and event activities. Configure the seeded events and event subscriptions delivered by the Oracle E-Business Suite for prebuilt XML messages in support of B2B integration requirements.

In addition, the function activities can be used to define Workflow processes in support of A2A integration requirements.

Oracle Workflow Builder Window



See XML Gateway Standard Item Type, page 6-7 for details regarding the function and event activities.

XML Gateway Error Processing Item Type

The XML Gateway Error Processing Item Type contains error handling processes to manage errors detected by Oracle Workflow Business Event System or Oracle XML Gateway.

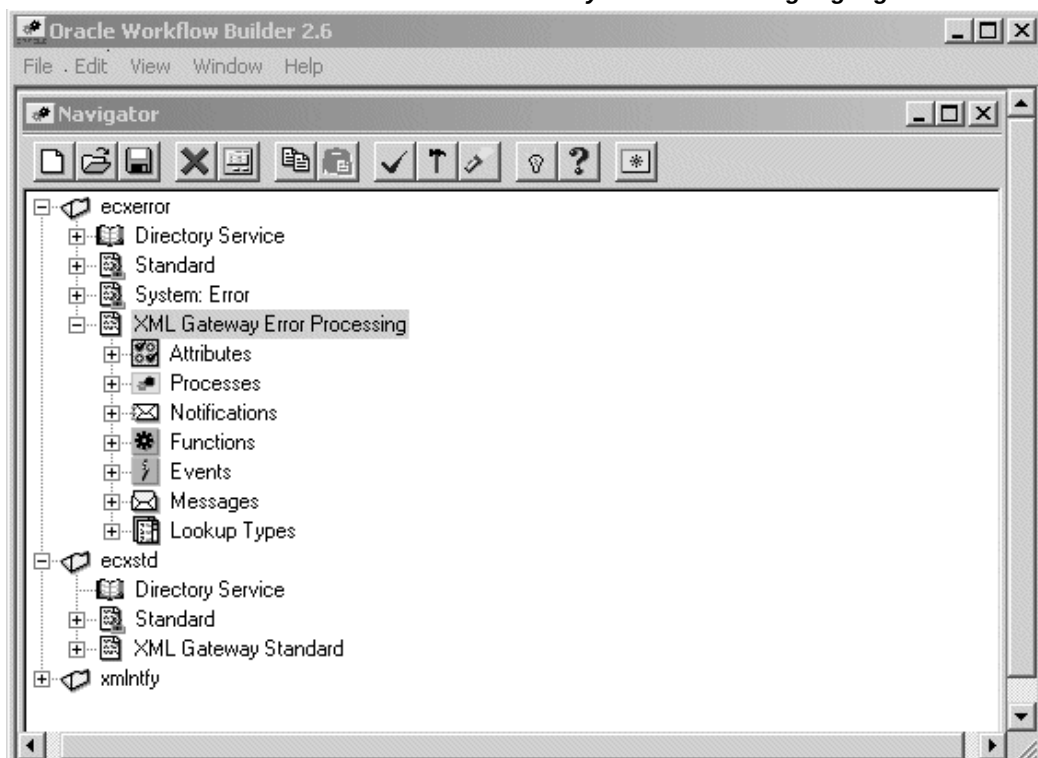
Oracle Workflow sends a notification to the Trading Partner contact for data errors or to the XML Gateway system administrator for system or process errors. For errors that require collaboration between the Trading Partner contact and the XML Gateway system administrator, a notification is sent to both parties to encourage discussion and to expedite problem solution.

In addition, Oracle Workflow sends a notification to an appropriate transaction owner

depending on the transaction type of an erred inbound transaction. If a transaction owner for a particular transaction type is identified, and the transaction error belongs to the transaction type with the owner defined, then the transaction owner will receive a notification in addition to the XML Gateway system administrator and Trading Partner contact depending on the error type. For example, if an inbound transaction error belongs to the Purchasing transaction type, then the Purchasing transaction owner also receives the notification if his or her user name is defined in the Define Transactions form.

A transaction owner can act on an error notification if needed. Once the notification has been responded on either reprocess or abort, any other subsequent response from the system administrator will be ignored and the notification should be closed.

Oracle Workflow Builder Window with XML Gateway Error Processing Highlighted



For more information on the error handling process, see XML Gateway Error Processing Item Type, page 6-27.

E-Business Suite Application Module-Specific Item Type

The seeded event subscription delivered for Oracle prebuilt messages is delivered in an Oracle E-Business Suite application module-specific item type. The seeded event subscription may be configured using the function activities defined in the XML Gateway Standard Item Type or customized to support the application-specific requirement.

See *Configure Oracle Prebuilt Inbound Messages*, page 6-52 or *Configure Oracle Prebuilt Outbound Messages*, page 6-54 for details on how to configure a seeded event subscription.

XML Gateway Standard Item Type

The XML Gateway Standard Item Type is a toolkit consisting of attributes, processes, functions, and event activities. These are provided to implementers to build Workflow processes to configure the seeded event subscriptions delivered by the Oracle E-Business Suite for prebuilt XML messages in support of B2B integration requirements.

In addition, the function activities can be used to define Workflow processes in support of A2A integration requirements.

The Notifications and Messages are contained in the Oracle E-Business Suite application module-specific item type and are defined using the XML Gateway Error Processing Item Type.

See *XML Gateway Error Processing Item Type*, page 6-27.

Attributes

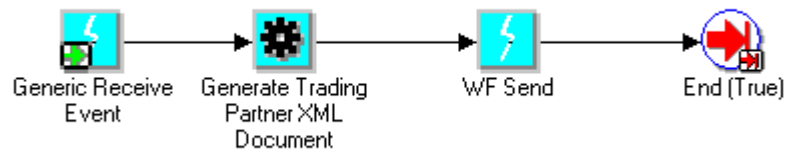
Attributes, also known as variables, are used to support the function, event, notification, and process activities defined for the XML Gateway Standard Item Type.

Processes

The XML Gateway Standard Item Type supports the following process:

- Create Trading Partner Message (for ECX internal use only)

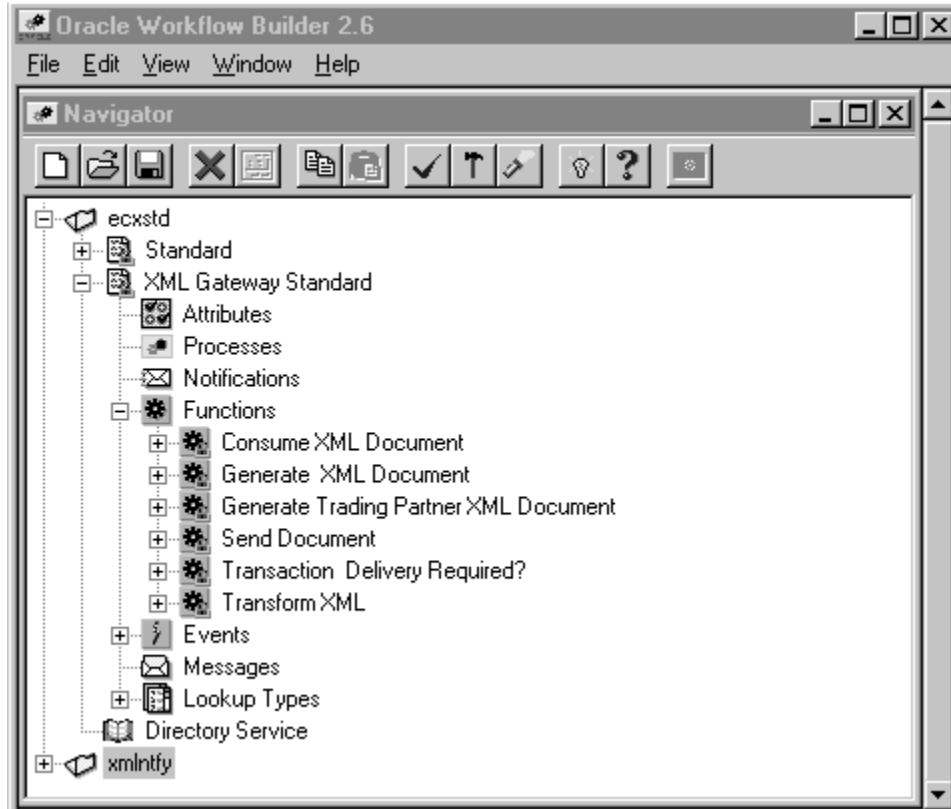
Create Trading Partner Message Process Diagram



The Create Trading Partner Message is a rule function available from the Workflow Event Subscription page.

Functions

Oracle Workflow Builder Window with Functions Node Expanded



The XML Gateway Standard Item Type includes five function activities as shown in the following table:

Function Display Name	A2A/B2B	Inbound / Outbound	Purpose
Consume XML Document	A2A	Inbound	Trigger XML Gateway to process inbound message on the Event Message attribute.
Generate XML Document	A2A	Outbound	Create XML message and store in Event Message attribute. Trading Partner verification is not required.
Generate Trading Partner XML Document	B2B	Outbound	Verify transaction is enabled for the Trading Partner, create XML message, and store in Event Message attribute.

Function Display Name	A2A/B2B	Inbound / Outbound	Purpose
Send Document	B2B	Outbound	Create and enqueue XML message onto outbound queue.
Transaction Delivery Required	B2B	Outbound	Verify transaction is enabled for the Trading Partner.
Transform XML	A2A/B2B	Inbound/Outbound	Enable XML to XML transformations

Application-to-Application (A2A) exchange of data is conducted within and outside your enterprise to share or synchronize data between information systems. There is no Trading Partner involved in an A2A integration. An example of an A2A exchange is to move data from the Oracle Payables module to Oracle General Ledger for GL consolidation.

Business-to-Business (B2B) exchange of data is conducted with a Trading Partner outside of your enterprise for which Trading Partner validation is required. Data will not be sent to or received from an invalid Trading Partner. An example of a B2B exchange is to send a purchase order to a supplier.

There is not a controlling Workflow process for inbound B2B exchanges. The XML Gateway execution engine subscribes directly to the inbound queue.

Functions will be referred to by their function display names. The internal function names are provided for reference in the following table:

Function Display Name	Internal Name
Consume XML Document	PROCESSXML
Generate XML Document	GETXML
Generate Trading Partner XML Document	GETTPXML
Send Document	DOCUMENT_SEND
Transaction Delivery Required	DELIVREQUIRED
Transform XML	XMLTOXML

The following is a description of each function supported by the XML Gateway Standard Item Type.

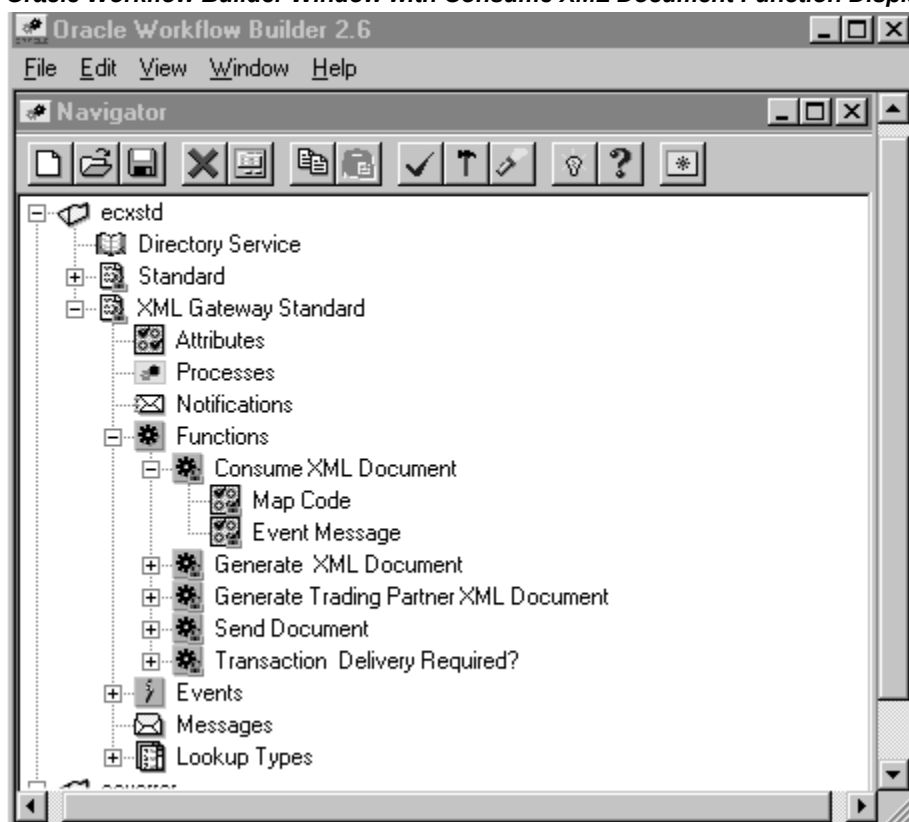
Consume XML Document (applies to A2A for inbound messages)

The Consume XML Document function triggers the XML Gateway Execution Engine to process the inbound message in the Event Message attribute. The XML Gateway execution engine will process the message according to the message map associated with the message, and will conclude by raising a business event to indicate that it has successfully processed the inbound message.

See Procedure Call: Execute Procedure, page 2-100 for details regarding the Procedure Call action to raise a business event.

The Consume XML Document function is for inbound messages in the A2A scenario only. The XML Gateway execution engine is automatically triggered in a B2B scenario when an inbound message arrives on the inbound queue.

Oracle Workflow Builder Window with Consume XML Document Function Displayed



The attributes for the Consume XML Document function are shown in the following table. The values for the attributes are provided by the business event, another function activity, or are set as constants.

Attribute Name	Attribute Description
Map Code	This is the Map Code defined in the Message Designer and associated to the Trading Partner that uniquely identifies the message map associated with the business document enabled for the Trading Partner.
Event Message	Event Message contains the event data as well as header properties for the event name, event key, and error data. The Event Message attribute is required to store the inbound message for processing by the XML Gateway execution engine.

Generate XML Document (applies to A2A for outbound messages)

The Generate XML Document function is used to retrieve data from the Oracle E-Business Suite. The data for the XML message retrieved from the Oracle E-Business Suite is stored in the Event Message attribute. The Event Message is then processed according to the subsequent Workflow instruction, which may be to send it to another Oracle E-Business Suite application module or to the WF_OUT agent.

The difference between the Generate XML Document function and the Generate Trading Partner XML Document function is that this function does not require Trading Partner validation since there is no Trading Partner involved in A2A integration.

The Generate XML Document function activity is provided for use in Workflow processes. For environments where a Workflow process does not exist, the ECX_STANDARD.generate function can be called from the Workflow Define Event window, Generate Function field. The details of the function are as follows:

ECX_STANDARD

PL/SQL Syntax

function generate

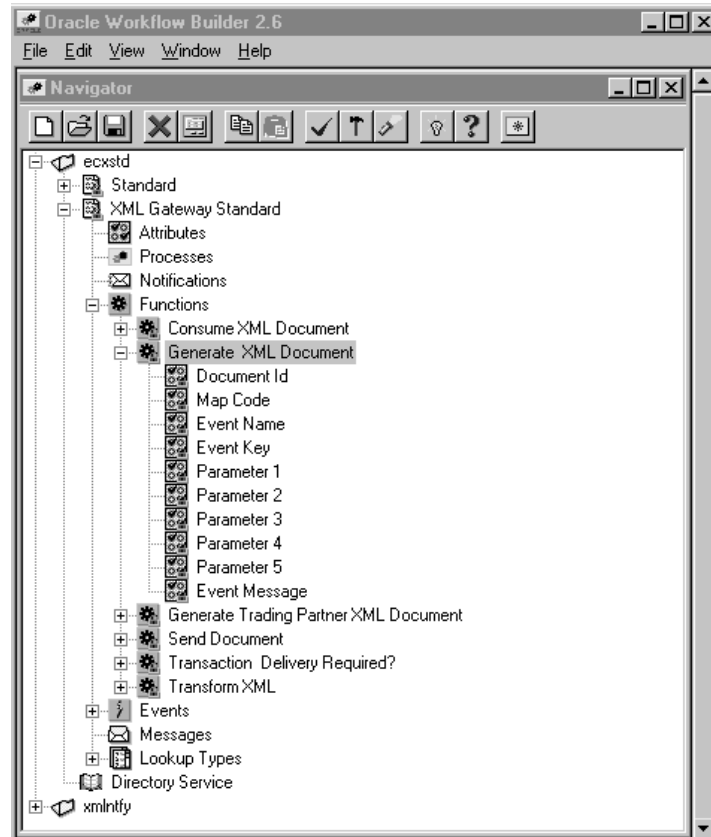
```
(p_event_name in varchar2,
 p_event_key in varchar2
 p_parameter_list in wf_parameter_list_t)
return CLOB;
```

Description

Arguments (input)

p_event_name	Unique identifier for the business event associated with the transaction.
p_event_key	Unique identifier for the business document from the Oracle E-Business Suite associated with the business event.
p_parameter_list	Parameter list containing the document selection criteria.

Oracle Workflow Builder Window with Generate XML Document Function Highlighted



The attributes for the Generate XML Document function are shown in the following table. The values for the attributes are provided by the business event, another function activity, or are set as constants.

Attribute Name	Attribute Description
Document ID	This is the unique identifier for the business document. It can be the document number or its associated database key, whichever is guaranteed to be unique for the transaction. The Document ID is optional for the Generate XML Document function unless it is used in the message map as a selection criterion.
Map Code	This is the Map Code defined in the Message Designer and associated to the Trading Partner that uniquely identifies the message map associated with the business document enabled for the Trading Partner. The Map Code is required for the Generate XML Document function.

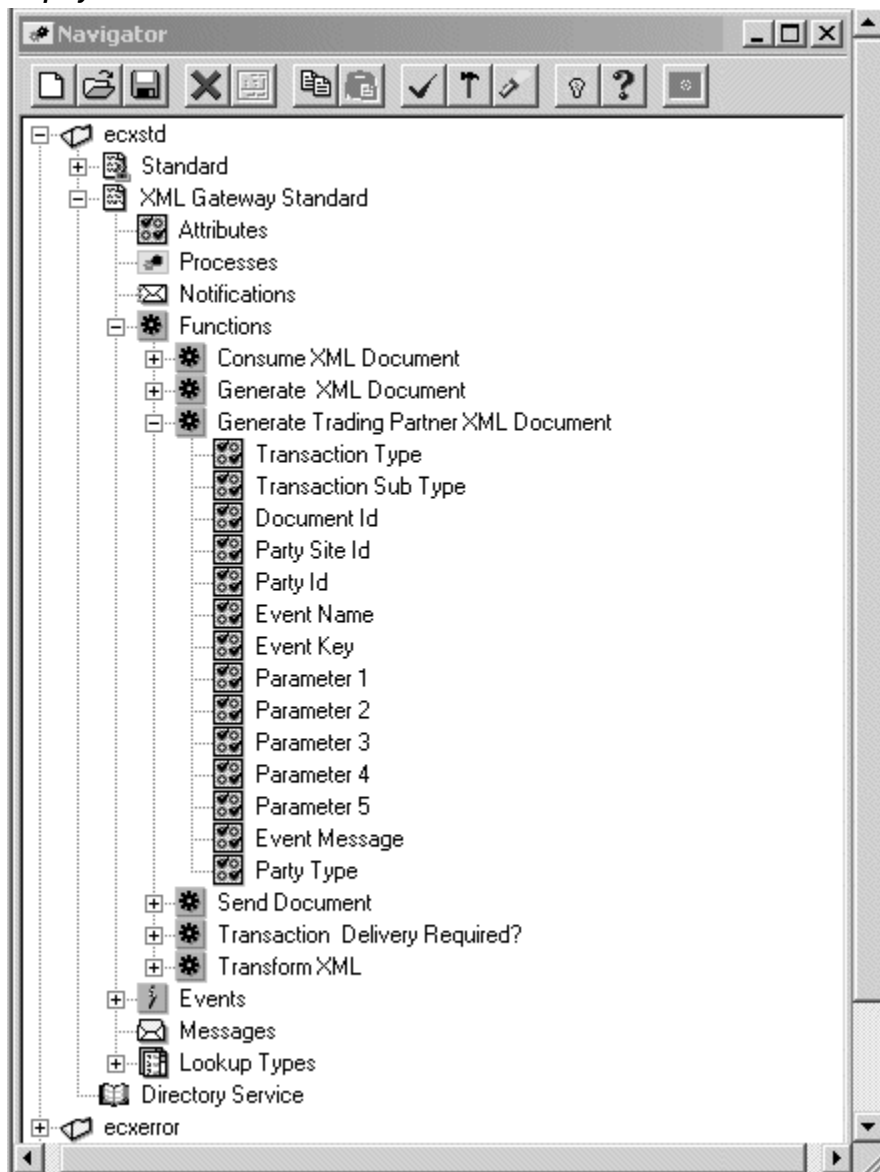
Attribute Name	Attribute Description
Event Name	This is a unique identifier for the business event. The naming convention is <code>oracle.apps.<product code>.<component>.<object>.<event></code> The Event Name is required for the Generate XML Document function to store the value returned.
Event Key	This is a unique identifier for an instance of an event. The combination of event name, event key, and event data fully describe what occurred in the event. The Event Key is required for the Generate XML Document function activity to store the value returned.
Parameter 1	Optional variable. This attribute is used if it is used in the message map.
Parameter 2	Optional variable. This attribute is used if it is used in the message map.
Parameter 3	Optional variable. This attribute is used if it is used in the message map.
Parameter 4	Optional variable. This attribute is used if it is used in the message map.
Parameter 5	Optional variable. This attribute is used if it is used in the message map.
Event Message	Event Message contains the event data as well as header properties for the event name, event key, and error data. The Event Message is required for the Generate XML Document function to store the value returned.

Generate Trading Partner XML Document (applies to B2B for outbound messages)

The Generate Trading Partner XML Document function is used to retrieve data from the Oracle E-Business Suite. The data for the XML message retrieved from the Oracle E-Business Suite is stored in the Event Message attribute. The Event Message is then processed according to the subsequent Workflow instruction, which may be to send it to another Oracle E-Business Suite application module or to the WF_OUT agent.

The difference between the Generate Trading Partner XML Document function and the Generate XML Document function is that this function is used in B2B integration for which a valid Trading Partner is required.

Oracle Workflow Builder Window with Generate Trading Partner XML Document Function Displayed



The attributes for the Generate Trading Partner XML Document function are shown in the following table. The values for the attributes are provided by the business event, another function activity, or are set as constants.

Attribute Name	Attribute Description
Transaction Type	This is the Transaction Type defined in the XML Gateway Define Transactions form. The Transaction Type is optional for the Generate Trading Partner XML Document function because it can be derived from the Map Code.
Transaction Subtype	This is the Transaction Subtype defined in the XML Gateway Define Transactions form. The Transaction Subtype is optional for the Generate Trading Partner XML Document function because it can be derived from the Map Code.
Document ID	This is the unique identifier for the business document. It can be the document number or its associated database key, whichever is guaranteed to be unique for the transaction. The Document ID is optional for the Generate XML Document function unless it is used in the message map as a selection criterion.
Party Site ID	This is the unique identifier for the Trading Partner site defined in Oracle E-Business Suite. The Party Site ID is required for the Generate Trading Partner XML Document function.
Party ID	This is the unique identifier for the Trading Partner defined in Oracle E-Business Suite. This field is optional.
Event Name	This is a unique identifier for the business event. The naming convention is <code>oracle.apps.<product code>.<component>.<object>.<event></code> . The Event Name is required for the Generate Trading Partner XML Document function to store the value returned.
Event Key	This is a unique identifier for an instance of an event. The combination of event name, event key, and event data fully describe what occurred in the event. The Event Key is required for the Generate Trading Partner XML Document function to store the value returned.
Parameter 1	Optional variable. This attribute is used if it is used in the message map.
Parameter 2	Optional variable. This attribute is used if it is used in the message map.
Parameter 3	Optional variable. This attribute is used if it is used in the message map.
Parameter 4	Optional variable. This attribute is used if it is used in the message map.
Parameter 5	Optional variable. This attribute is used if it is used in the message map.

Attribute Name	Attribute Description
Event Message	Event Message contains the event data as well as header properties for the event name, event key, and error data. The Event Message is required for the Generate Trading Partner XML Document function to store the value returned.
Party Type	The Party Type defined in the XML Gateway Trading Partner Setup window.

Send Document (applies to B2B for outbound messages)

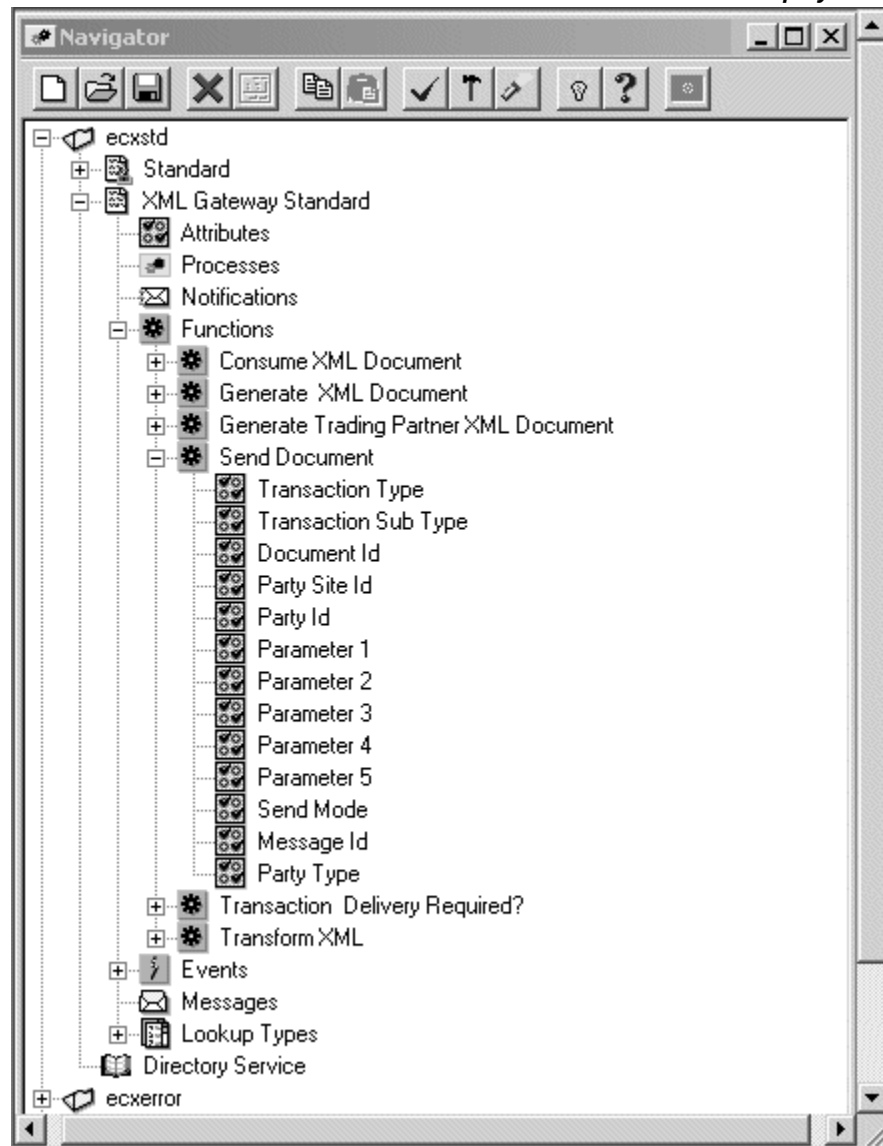
The Send Document function is used to trigger outbound message creation. The XML message may be created immediately or deferred depending on the setting of the Send Mode attribute. This function triggers XML Gateway to gather the data from the Oracle E-Business Suite, and to create and enqueue the XML message.

A Send Mode of "Deferred" defers the processing to the Workflow background engine.

A Send Mode of "Immediate" triggers the XML Gateway Execution Engine to create the XML message immediately after the data has been entered into the Oracle E-Business Suite. With this approach, a snapshot of the data is taken to avoid a race condition where the data may be changed after the time the event is raised and before the data is extracted.

The Send Document function may be modeled using the XML Gateway Rule Function or the Workflow Default Rule Function to ensure that XML messages are sent to Trading Partners enabled for the transaction.

Oracle Workflow Builder Window with Send Document Function Displayed



The attributes for the Send Document function are shown in the following table. The values for the attributes are provided by the business event, another function activity, or are set as constants.

Attribute Name	Attribute Description
Transaction Type	This is the Transaction Type defined in the XML Gateway Define Transactions form.

Attribute Name	Attribute Description
Transaction Subtype	This is the Transaction Subtype defined in the XML Gateway Define Transactions form.
Document ID	This is the unique identifier for the business document. It can be the document number or its associated database key, whichever is guaranteed to be unique for the transaction.
Party Site ID	This is the unique identifier for the Trading Partner site defined in Oracle E-Business Suite.
Party ID	This is the unique identifier for the Trading Partner defined in Oracle E-Business Suite. This field is optional.
Parameter 1	Optional variable. This attribute is used if it is used in the message map.
Parameter 2	Optional variable. This attribute is used if it is used in the message map.
Parameter 3	Optional variable. This attribute is used if it is used in the message map.
Parameter 4	Optional variable. This attribute is used if it is used in the message map.
Parameter 5	Optional variable. This attribute is used if it is used in the message map.
Send Mode	Select "Deferred" or "Immediate"
Message ID	This is a unique identifier provided by the XML Gateway execution engine for each outbound message.
Party Type	Party Type defined in the XML Gateway Trading Partner Setup window.

Transaction Delivery Required? (applies to B2B for outbound messages)

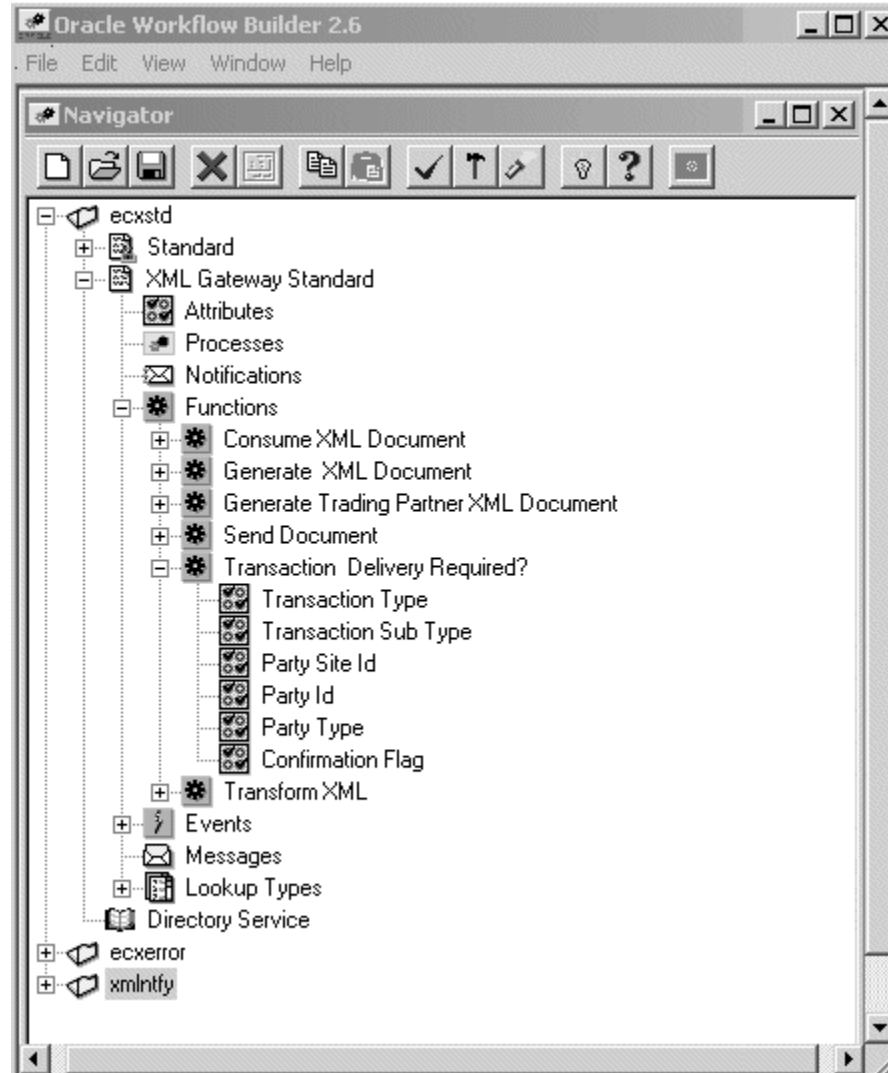
The Transaction Delivery Required function is used to determine if the Trading Partner is enabled for the transaction. It is used for B2B integration where Trading Partner validation is required. The function returns TRUE or FALSE.

The Transaction Delivery Required function is also embedded in the XML Gateway Rule Function. No Workflow process will be executed if the function returns FALSE.

The seeded event subscription delivered by Oracle E-Business Suite application modules can use the XML Gateway Rule Function or the Workflow Default Rule Function. The XML Gateway Rule Function is more efficient because it predetermines whether the Trading Partner is enabled for the transaction and executes the associated

Workflow process only if the result of the rule function is TRUE. It is therefore the recommended approach.

Oracle Workflow Builder Window with Transaction Delivery Required? Function Displayed



The attributes for the Transaction Delivery Required function are shown in the following table. The values for the attributes are provided by the business event, another function activity, or are set as constants.

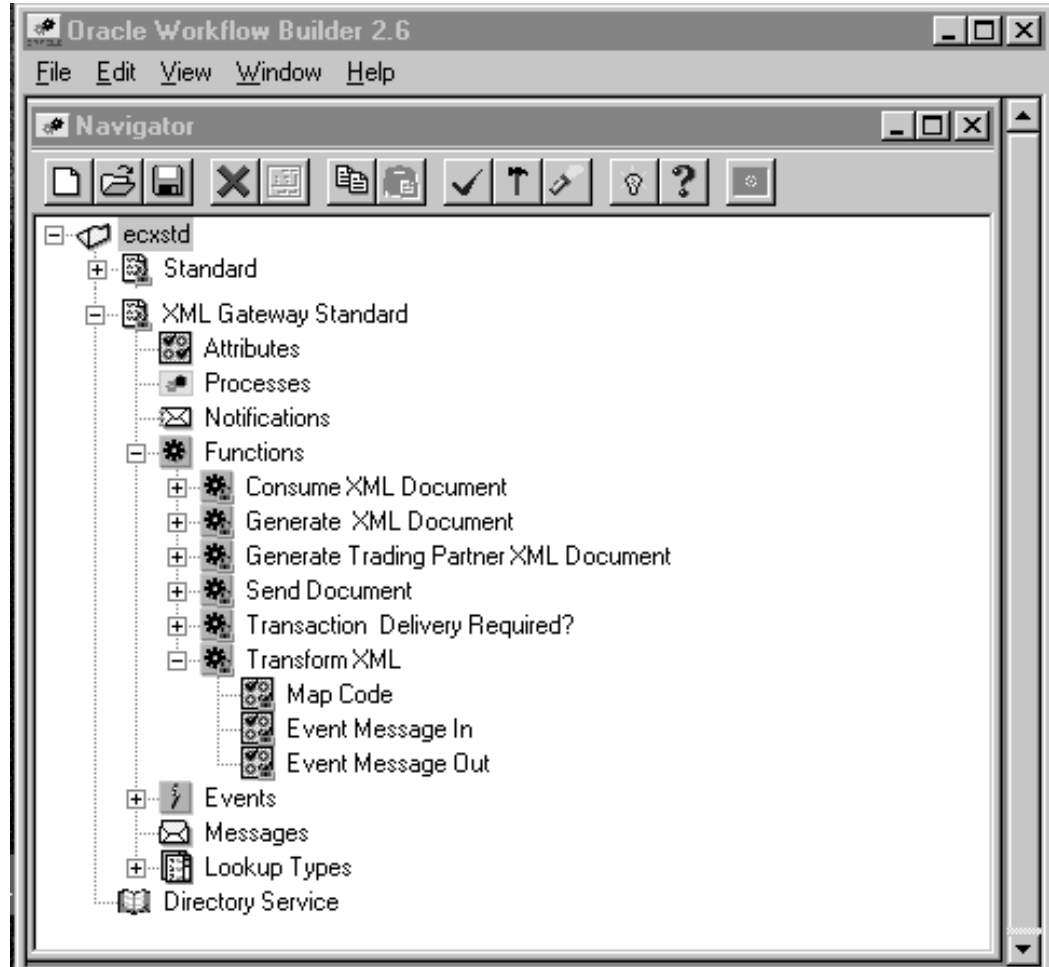
Attribute Name	Attribute Description
Transaction Type	This is the Transaction Type defined in the XML Gateway Define Transactions form.

Attribute Name	Attribute Description
Transaction Subtype	This is the Transaction Subtype defined in the XML Gateway Define Transactions form.
Party Site ID	This is the unique identifier for the Trading Partner site defined in Oracle E-Business Suite.
Party ID	This is the unique identifier for the Trading Partner defined in Oracle E-Business Suite.
Party Type	The Party Type defined in the XML Gateway Trading Partner Setup window.
Confirmation Flag	The Document Confirmation flag from the Trading Partner Setup window. Indicates whether a confirmation is sent or received. The Confirmation Flag has been added to the Transaction Delivery Required? (DELIVREQUIRED) function activity. Workflows for outbound business documents can now be designed to wait for the inbound confirmation if the ECX_CONFIRMATION_FLAG is set to 1 (send confirmation if error detected) or 2 (always send confirmation). The Confirmation Flag is an attribute of the Trading Partner that is defined using the Trading Partner Setup window. Use this Workflow design approach if you want to link the outbound business document with the inbound confirmation. This is optional.

Transform XML

The Transform XML function is used for XML to XML transformation.

Oracle Workflow Builder Window with Transform XML Function Displayed



The Transform XML function transforms a given Input XML document into another XML document. The input parameters to this activity are the Map Code and the Input XML Document. The output of the activity is the transformed XML document. The execution engine will execute the transformation according to the Map Definition.

The attributes for the Transform XML function are shown in the following table. The Event Message In attribute stores the input XML information and the Event Message Out attribute stores the transformed XML information.

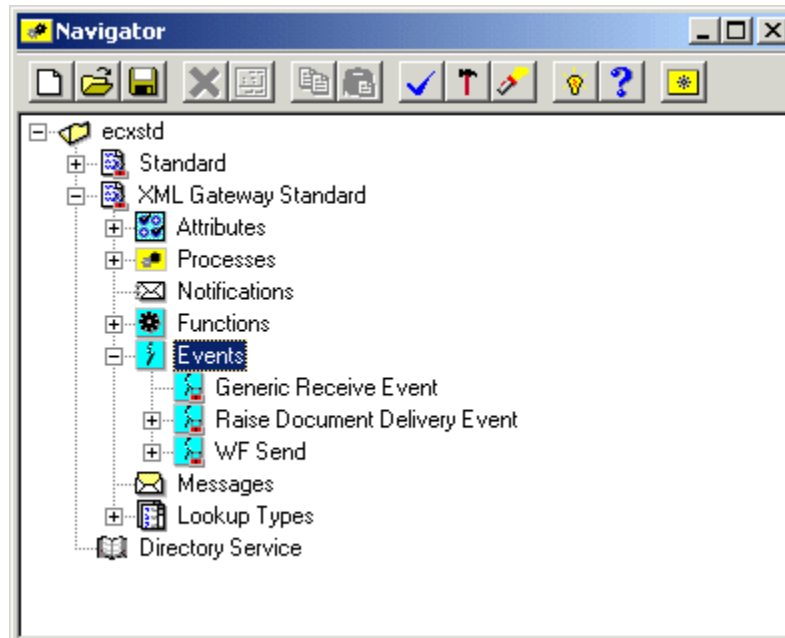
Attribute Name	Attribute Description
Map Code	The Map Code associated with the Transaction as defined in the Trading Partner Details form.

Attribute Name	Attribute Description
Event Message In	The Input XML message.
Event Message Out	The Transformed XML message.

Events

The XML Gateway Standard Item Type includes three Events to raise a business event from an existing Workflow process.

Oracle Workflow Builder Window with Events Highlighted



Generic Receive Event

The Generic Receive Event is used to support the Create Trading Partner Message process.

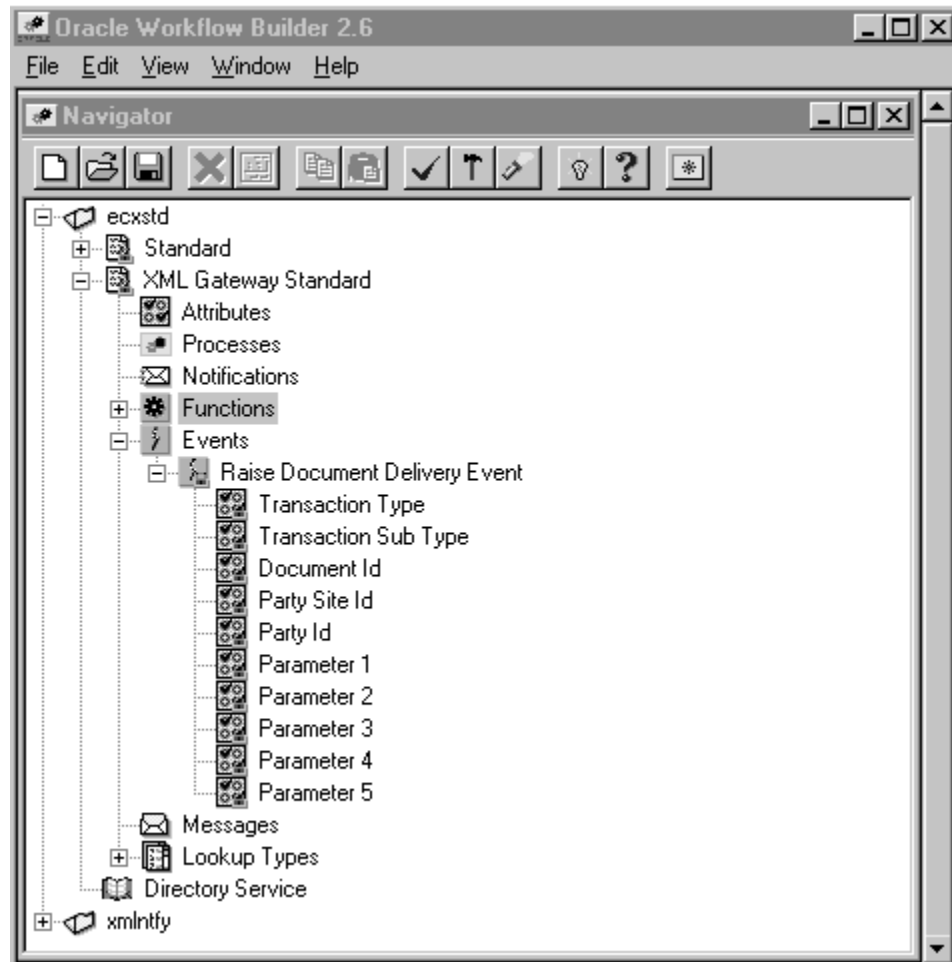
Raise Document Delivery Event

The Raise Document Delivery Event is used to raise a business event from an existing Workflow process. This allows you to seamlessly integrate your existing Workflow process with Oracle XML Gateway to create an outbound XML message.

An alternate approach is to raise the business event directly from PL/SQL code. Refer to

the Raise, *Oracle Workflow API Reference* for the details.

Oracle Workflow Builder Window with Raise Document Delivery Event Displayed



The attributes for the Raise Document Delivery Event are shown in the following table. The values for the attributes are provided by a function activity or are set as constants.

Attribute Name	Attribute Description
Transaction Type	This is the Transaction Type defined in the XML Gateway Define Transactions form.
Transaction Subtype	This is the Transaction Subtype defined in the XML Gateway Define Transactions form.

Attribute Name	Attribute Description
Document ID	This is the unique identifier for the business document. It can be the document number or its associated database key, whichever is guaranteed to be unique for the transaction.
Party Site ID	This is the unique identifier for the Trading Partner site defined in Oracle E-Business Suite.
Party ID	This is the unique identifier for the Trading Partner defined in Oracle E-Business Suite.
Parameter 1	Optional Variable. This attribute is used if it is used in the message map.
Parameter 2	Optional Variable. This attribute is used if it is used in the message map.
Parameter 3	Optional Variable. This attribute is used if it is used in the message map.
Parameter 4	Optional Variable. This attribute is used if it is used in the message map.
Parameter 5	Optional Variable. This attribute is used if it is used in the message map.

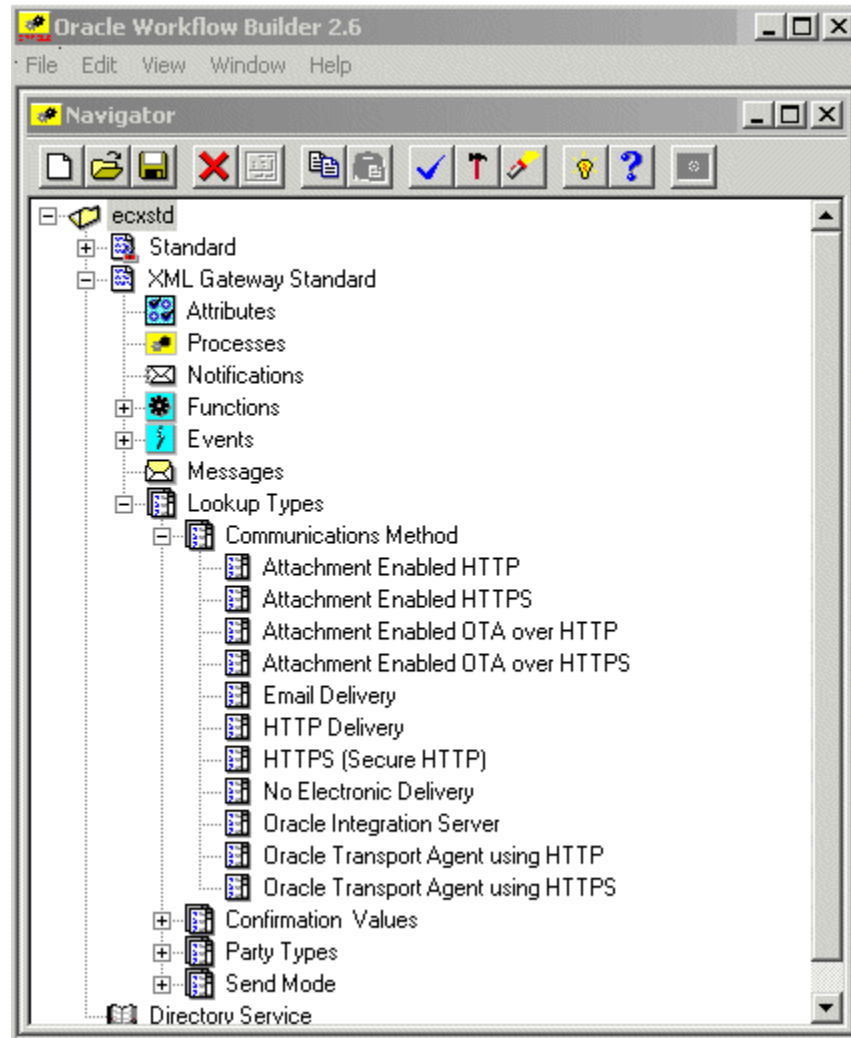
WF Send

The WF Send Event enqueues a message onto the ECX_OUTBOUND queue. This event is used in conjunction with Callback. See Message Delivery Status, page 6-66 for a complete description of this event and its attributes.

Lookup Types

The XML Gateway Standard Item Type supports the Send Mode Lookup Type used in the Send Document function to indicate whether the event subscription should be processed immediately or in deferred mode.

Oracle Workflow Builder Window with Lookup Types Displayed



Communications Method

- Attachment Enabled HTTP
- Attachment Enabled HTTPS
- Attachment Enabled OTA over HTTP
- Attachment Enabled OTA over HTTPS
- Email Delivery
- HTTP Delivery

- HTTPS (Secure HTTP)
- No Electronic Delivery
- Oracle Integration Server
- Oracle Transport Agent Using HTTP
- Oracle Transport Agent Using HTTPS

Confirmation Values

Confirmation Values are used in the Transaction Delivery Required function activity.

- No confirmation Business Object Document Delivery
- Send a confirmation Business Object Document Regardless
- Send back a confirmation Business Object Document only if an error has occurred

Party Types

Party Types are used in the Get Trading Partner Role and Send Document function activities.

- Bank
- Customer
- Carrier
- Exchange
- Internal
- Supplier
- XML Gateway Standalone

Send Mode

Send Mode is used in the Send Document function activity.

- Deferred

The Deferred mode informs the XML Gateway execution engine to defer the message creation to the Workflow background engine. This is also known as asynchronous processing.

- Immediate

The Immediate mode informs the XML Gateway execution engine to create and enqueue the XML message immediately after the data has been entered into the Oracle E-Business Suite. This is also known as synchronous processing.

Synchronous processing avoids the race condition where the data may be changed between the time the event is raised and before the data is extracted.

XML Gateway Error Processing Item Type

The XML Gateway Error Processing Item Type contains error handling processes to manage errors detected by Oracle Workflow Business Event System or Oracle XML Gateway.

Oracle Workflow sends a notification to the Trading Partner contact for data errors or to the XML Gateway system administrator for system or process errors. For errors that require collaboration between the Trading Partner contact and the XML Gateway system administrator, a notification is sent to both parties to encourage discussion and to expedite problem solution.

In addition, Oracle Workflow sends a notification to an appropriate transaction owner depending on the transaction type of an erred inbound transaction. If a transaction owner for a particular transaction type is identified, and the transaction error belongs to the transaction type with the owner defined, then the transaction owner will receive a notification in addition to the system administrator and Trading Partner contact depending on the error type. For example, if an inbound transaction error belongs to the Purchasing transaction type, then the Purchasing transaction owner also receives the notification if his or her user name is defined in the Define Transactions form.

A transaction owner can act on an error notification if needed. Once an open notification has been responded on either reprocess or abort, any other subsequent response from the XML Gateway system administrator will be ignored and the notification should be closed.

However, if the system administrator or transaction owner has not taken an explicit action such as reprocess or abort on a notification that was received earlier, after the notification timeout period, the XML Gateway execution engine will automatically reprocess the corresponding erred transaction until the number of maximum retries is exceeded. Oracle XML Gateway uses the following two profile options to manage the timeout feature:

- **ECX: Notification Timeout in Minutes (ECX_NOTIFY_TIMEOUT) Profile Option**

This profile option sets the timeout period measured in minutes for a notification that is waiting for a response. If the notification timeout value is exceeded, the XML Gateway execution engine will automatically reprocess the erred transaction if it does not exceed the maximum retries limit.

- **ECX: Maximum Reprocess Attempts For Errored Inbound Transaction (ECX_MAX_RETRY) Profile Option**

This profile option governs the number of times that the XML Gateway execution engine will automatically reprocess the erred transaction after the notification timeout period. When an error occurs during reprocessing, notifications will be sent to the XML Gateway system administrator and the transaction owner if identified. If the maximum retries limit is exceeded, the engine will not reprocess the transaction until the system administrator or the transaction owner explicitly opts for reprocessing it.

Note: The Workflow Retry feature is used to support outbound messages; it resumes from the point of failure. The ECX Reprocess Inbound function is used to support the reprocess feature for inbound messages; it resumes using a copy of the inbound message stored in the ECX_DOCLOGS table.

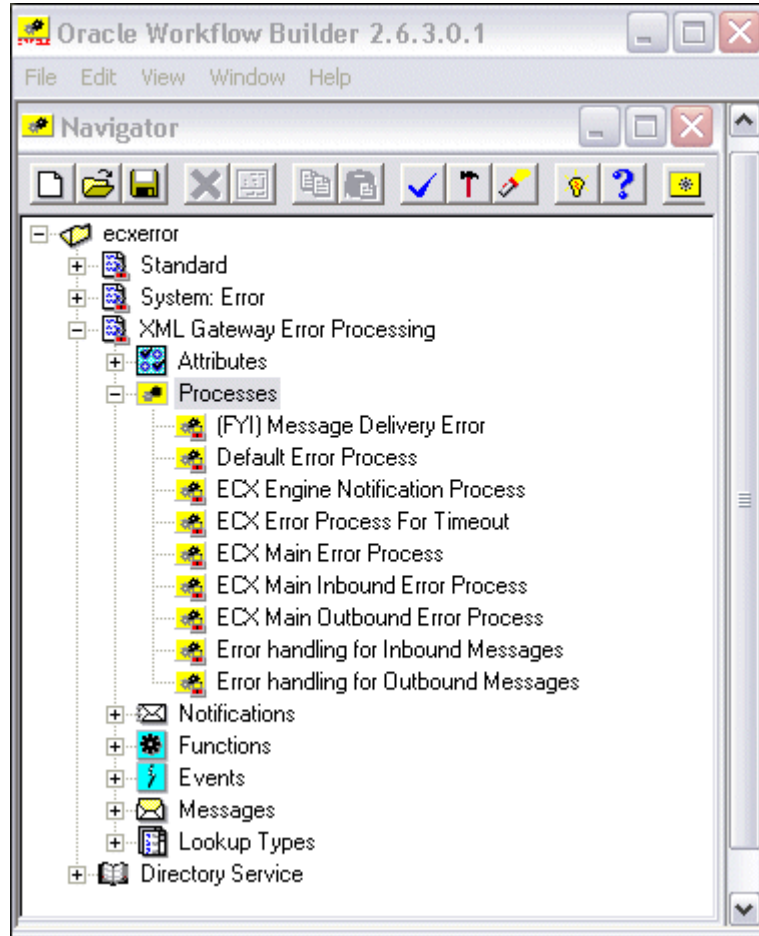
Attributes

Attributes, also known as variables, are used to support the function, event, notification, and process activities defined for the XML Gateway Error Processing Item Type.

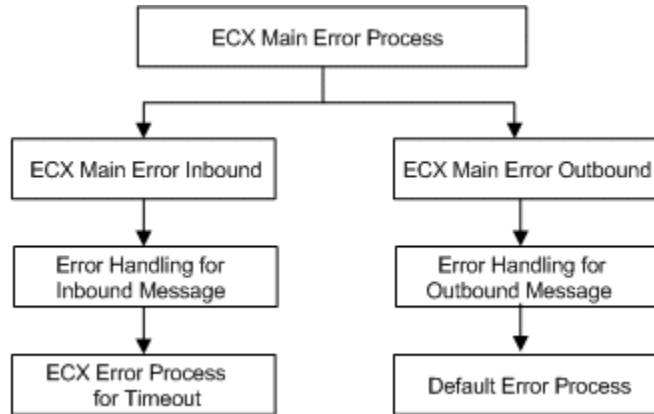
Processes

The XML Gateway Error Processing Item Type supports the following error handling processes:

- (FYI) Message Delivery Error
- Default Error Process
- ECX Engine Notification Process
- ECX Error Process for Timeout
- ECX Main Error Process
- ECX Main Inbound Error Process
- ECX Main Outbound Error Process
- Error handling for Inbound Messages
- Error handling for Outbound Messages



The relationship of the error handling processes is shown below. The ECX Main Error Process calls the ECX Main Error Inbound process for inbound message errors or the ECX Main Error Outbound process for outbound message errors. If inbound, the ECX Main Error Inbound process calls the Error Handling for Inbound Message process, which calls the ECX Error Process for Timeout. If outbound, the ECX Main Error Outbound process calls the Error Handling for Outbound Message process, which calls the Default Error process.

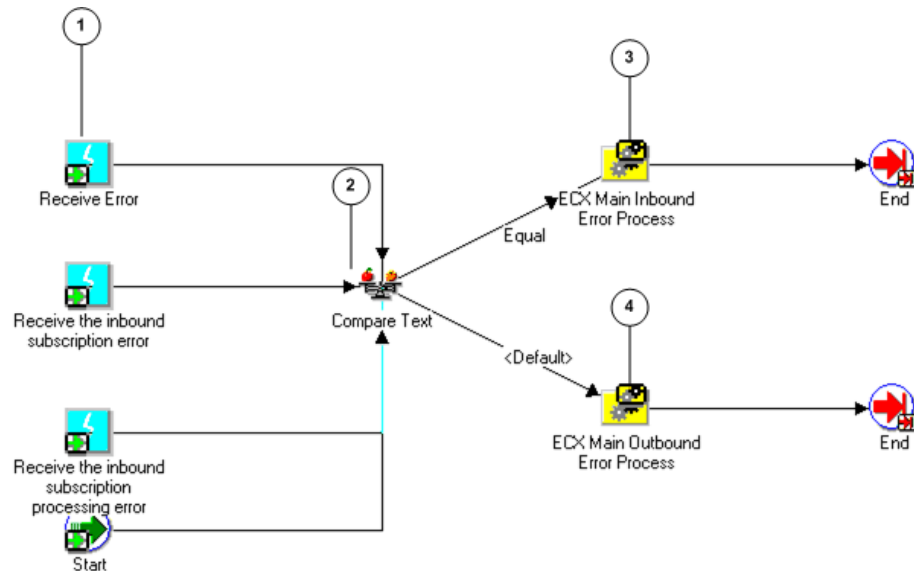


The following is a description of each Process (listed in their functional order above) supported by the XML Gateway Error Processing Item Type:

ECX Main Error Process

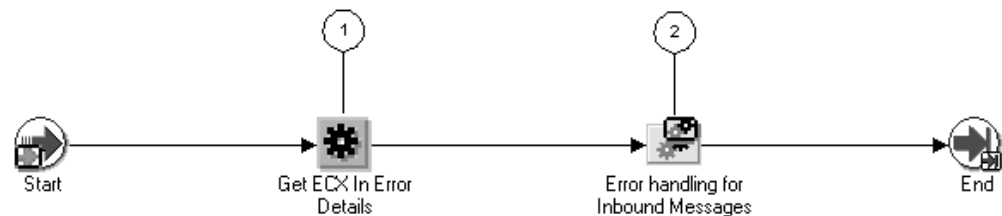
The ECX Main Error Process is triggered by the Receive Error Event (see illustration, node 1), the Receive Inbound Subscription Error event, or the Receive Inbound Subscription Processing Error event for errors detected by the XML Gateway execution engine; or by Workflow for errors involving the Workflow processes created using the XML Gateway Standard Item Type.

At node 2, the process determines whether the error relates to an inbound or outbound message. The ECX Main Error Process calls the ECX Main Inbound Error Process (node 3) for errors related to an inbound message. For errors related to an outbound message, it calls the ECX Main Outbound Error Process (node 4).



ECX Main Inbound Error Process

As shown in the illustration below, ECX Main Inbound Error Process gets the error information (node 1) from the XML Gateway execution engine or Workflow engine and calls the Error Handling for Inbound Messages process (node 2).



Error Handling for Inbound Messages

Errors detected by the XML Gateway execution engine or Workflow engine may trigger a notification. The notification can be sent to the XML Gateway system administrator, the Trading Partner contact, or both depending on the nature of the error, as well as an appropriate transaction owner if the information is identified.

Notifications can be e-mailed to the target recipient(s) if the Workflow mailer is enabled. The e-mail or user name information is defined in the following places:

- XML Gateway system administrator's e-mail address is defined in the ECX: System Administrator Email Address profile option.
- The Trading Partner contact e-mail address is defined in the Trading Partner form.

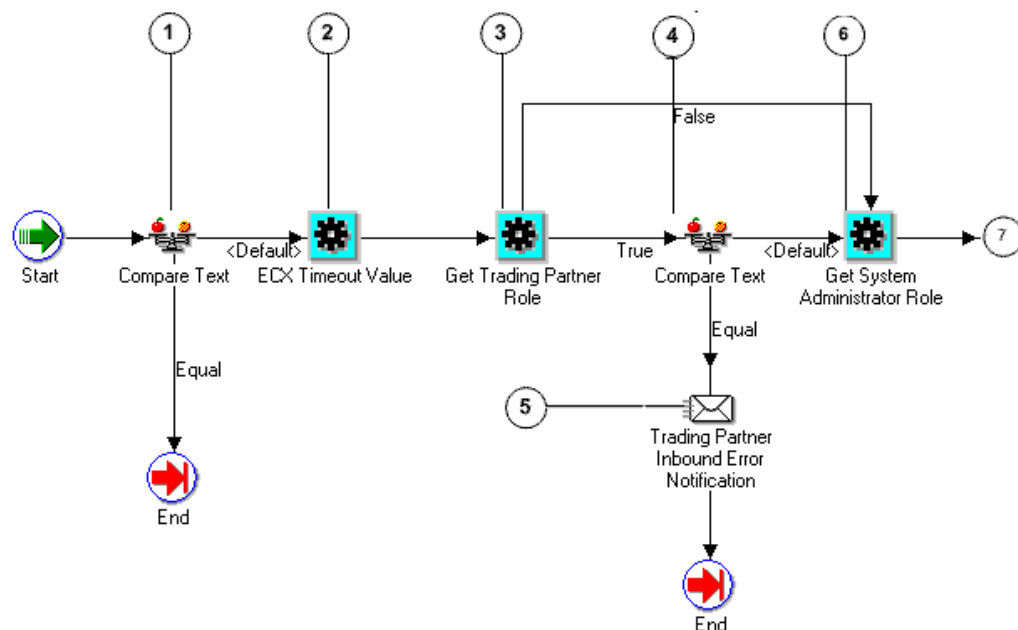
- The transaction owner's user name can be specified in the Define Transactions form.

The target recipient(s) of the notification are predefined for the error message and cannot be changed.

The Error Handling for Inbound Messages process checks the type of the error detected (shown in the following diagram, node 1). If the error type indicates that no notification is sent, the failed process ends, and the error process ends.

If the compare in node 1 indicates that a notification is to be sent, the process retrieves the timeout value measured in minutes from the ECX: Notification Timeout in Minutes profile option (node 2), and retrieves the trading partner contact information (Company Admin e-mail) from the trading partner tables (node 3). If the error type indicates that the target recipient is the Trading Partner contact (performed in node 4), a notification is sent to the Trading Partner (node 5), the failed process ends, and the error process ends.

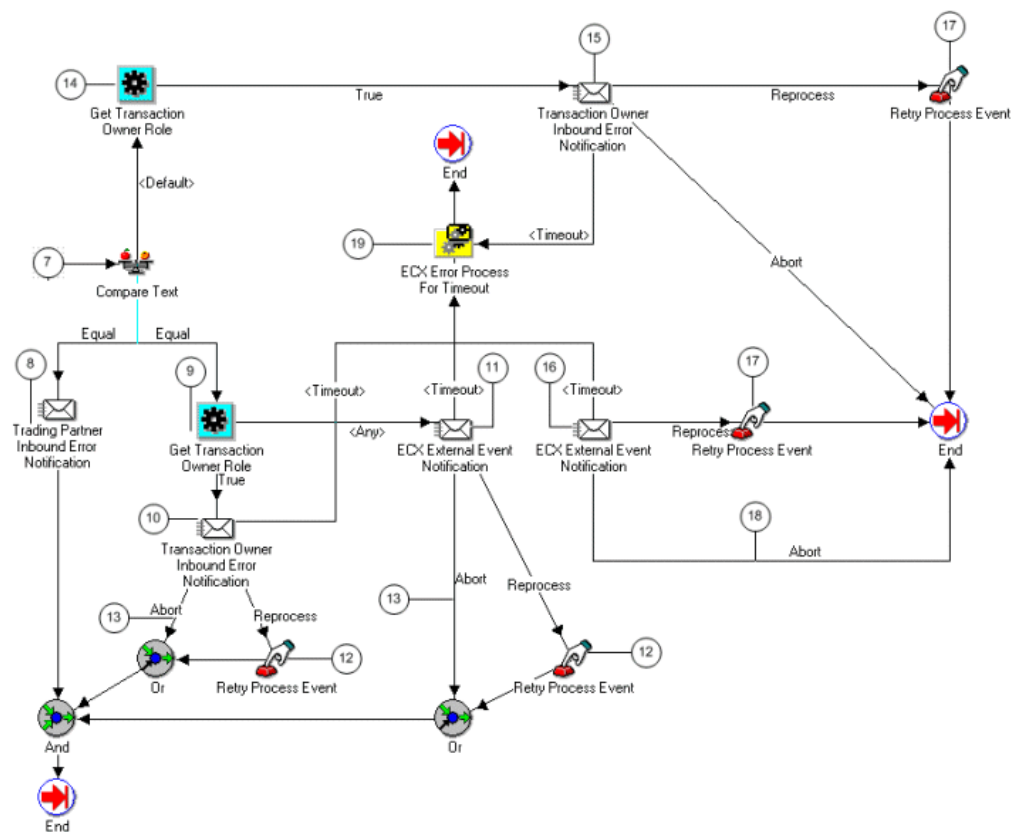
If the check in node three did not indicate that the target recipient is solely the Trading Partner contact, the process retrieves the System Administrator contact information and the transaction owner contact information if it is available (node 6) and checks the error type again (node 7).



If the target recipients are both the Trading Partner contact and the XML Gateway system administrator, then the process retrieves the transaction owner contact information if available from the Define Transactions form (node 9), and then three notifications are sent, one to each party (nodes 8, 10, and 11). The system administrator and the transaction owner both have the option of reprocessing or aborting the failed inbound process (node 12). Reprocessing resumes using a copy of the inbound message stored in the ECX_DOCLOGS table.

If the target recipient is solely the XML Gateway system administrator, then the process retrieves the transaction owner contact information if available from the Define Transactions form (node 14), and then two notifications are sent, one for the system administrator and the other for an appropriate transaction owner (node 15 and 16). Both of them have the option to reprocess (node 17) or abort (node 18) the failed inbound process.

If the system administrator or a transaction owner does not respond to the notification that is waiting for action, and the notification timeout period specified in the ECX: Notification Timeout in Minutes profile option is exceeded, the ECX Error Process for Timeout, page 6-33 will be called (node 19). This step enables XML Gateway execution engine to automatically reprocess the corresponding erred transaction if it does not exceed the maximum retries limit set by the ECX: Maximum Reprocess Attempts For Errored Inbound Transaction profile option.

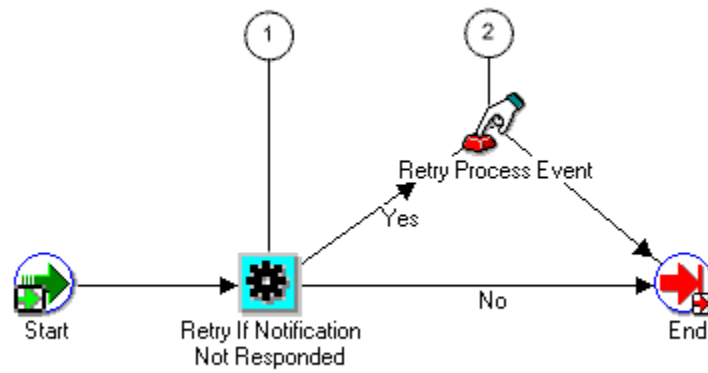


ECX Error Process for Timeout

During the error handling for inbound messages, the ECX Error Process for Timeout process is called by the Error Handling for Inbound Messages process, page 6-31 to have the XML Gateway execution engine automatically reprocess an erred inbound transaction when the XML Gateway system administrator or the transaction owner has not taken an explicit action on a notification that was sent earlier, and the timeout value for a notification waiting for a response is exceeded but the number of automatically

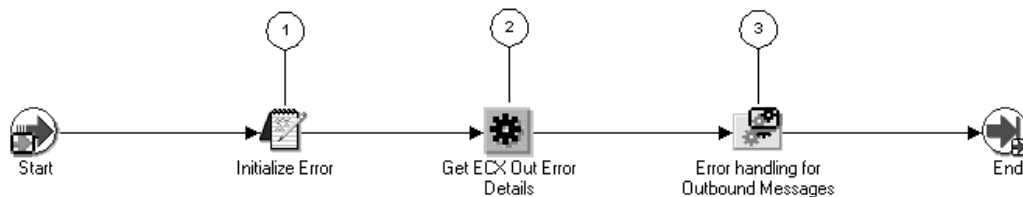
reprocess has not yet reached the maximum retries limit.

When a notification that was sent earlier has not yet responded explicitly (shown in the following diagram, node 1), the XML Gateway execution engine will automatically reprocess an erred transaction (node 2).



ECX Main Outbound Error Process

ECX Main Outbound Error Process initializes the error (shown below, node 1) and gets the error information (node 2) from the XML Gateway execution engine or Workflow engine and calls the Error Handling for Outbound Messages process (node 3).



Error Handling for Outbound Messages

Errors detected by the XML Gateway execution engine or Workflow engine may trigger a notification. The notification can be sent to the XML Gateway system administrator, the Trading Partner contact, or both depending on the nature of the error.

Notifications may be e-mailed to the target recipient(s) if the Workflow mailer is enabled. Their e-mail addresses are defined in the following places:

- XML Gateway system administrator's e-mail address is defined in the ECX: System Administrator Email Address profile option
- The Trading Partner contact e-mail address is defined in the Trading Partner form.

The target recipient(s) of the notification are predefined for the error message and

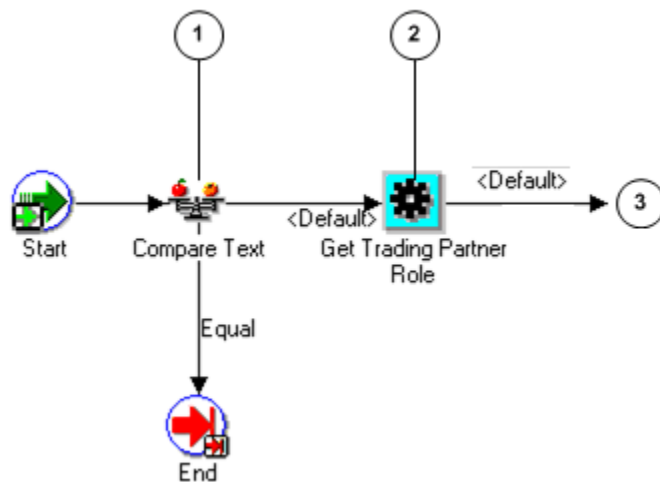
cannot be changed.

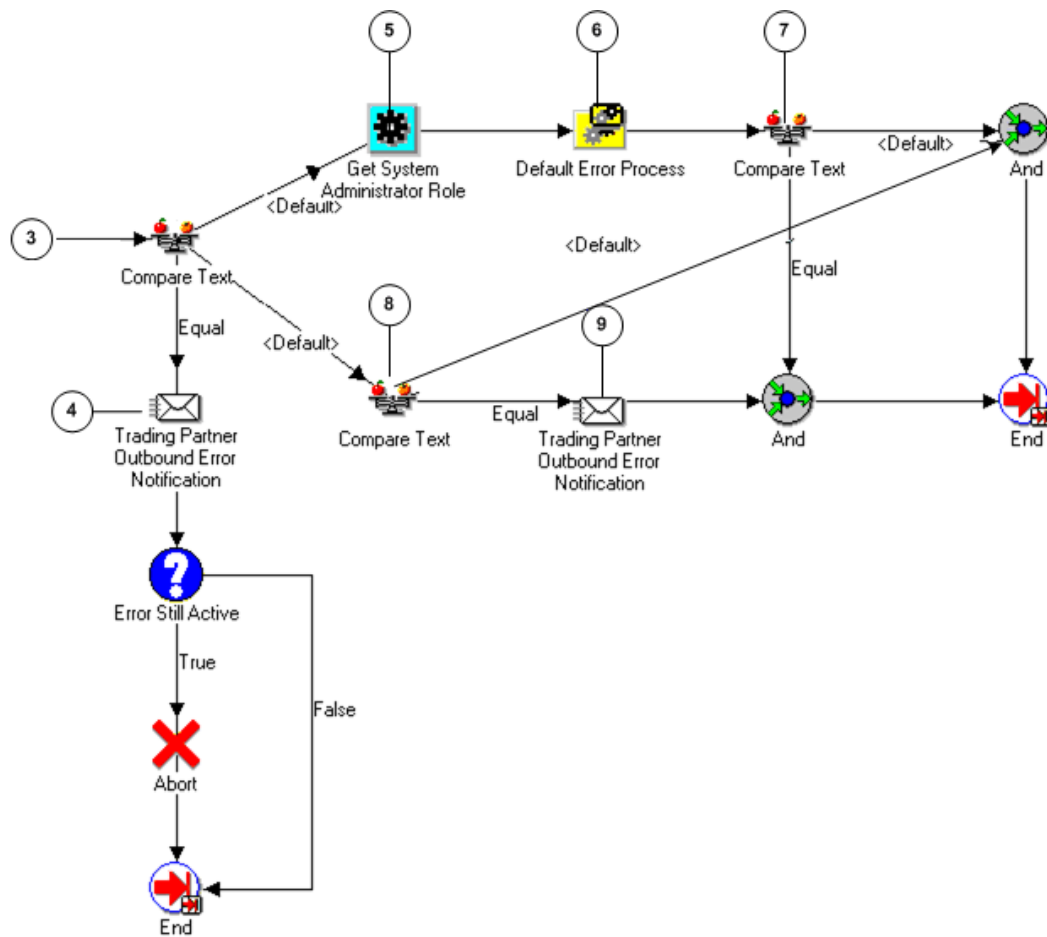
The Error Handling for Outbound Messages process checks the error type to determine if a notification is to be sent (shown below, node 1). If the type indicates that no notification is to be sent, the process ends.

If the type does not indicate that no notification is to be sent, the process retrieves the trading partner contact information (Company Admin e-mail) from the trading partner tables (node 2). The notification type is checked (node 3), and if the target recipient is solely the Trading Partner contact, a notification is sent to the Trading Partner (node 4) and the failed process is aborted.

If the target recipient is not solely the trading partner contact, the system administrator contact is retrieved (node 5) and the Default Error Process will be called (node 6). At node 8 the type is checked to determine if the notification is to be sent to both the trading partner and the system administrator.

If the target recipients are both the Trading Partner contact and the XML Gateway system administrator, then notification is also sent to the Trading Partner contact (node 9).

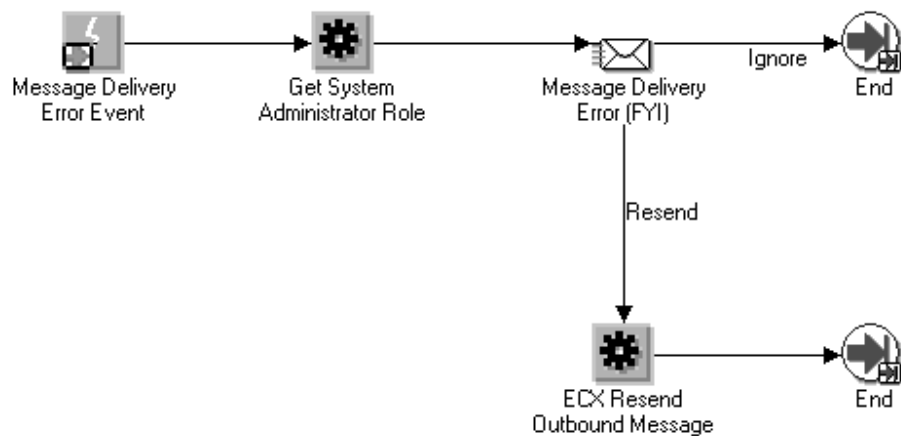




(FYI) Message Delivery Error Process

The (FYI) Message Delivery Process is used with Oracle Transport Agent (or other messaging systems) to report message delivery status back to the Workflow process that initiated message creation.

A corresponding event subscription (seeded in XML Gateway) for the oracle.apps.ecx.processing.message.callback event initiates the XML Gateway Standard Error Processing Item Type: FYI_MESSAGE_DELIVERY_ERROR error handling process. In the event of an error, a notification is sent to the System Administrator who has the option to "Resend" the message after correcting the error, or to "Ignore" the notification if no action is required.

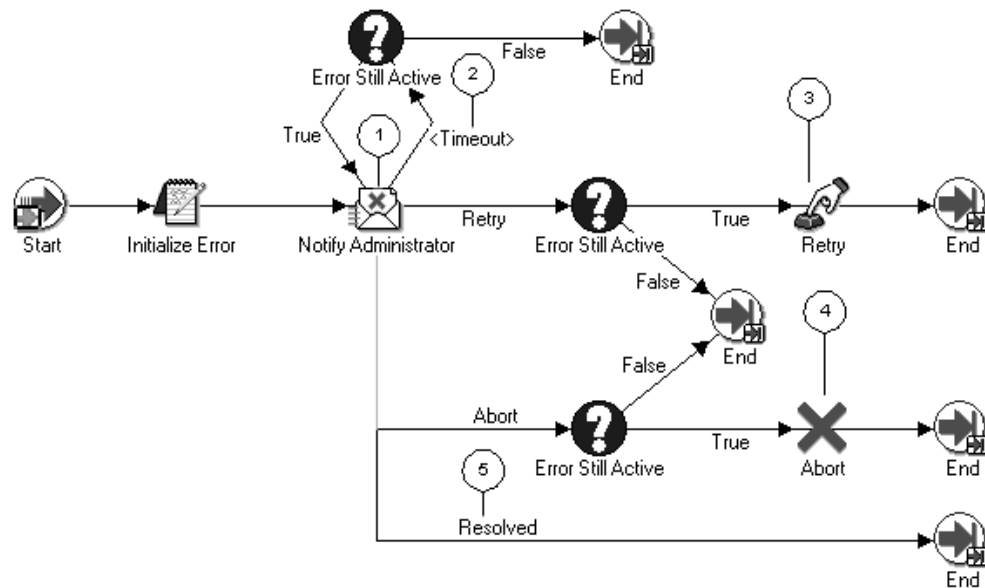


Default Error Process

The Default Error Process is called by the Error Handling for Outbound Messages process when the error message notification is for the system administrator.

The Workflow notification administrator delivers the notification (shown below, node 1) to the system administrator as well as proceeds on one of the following paths:

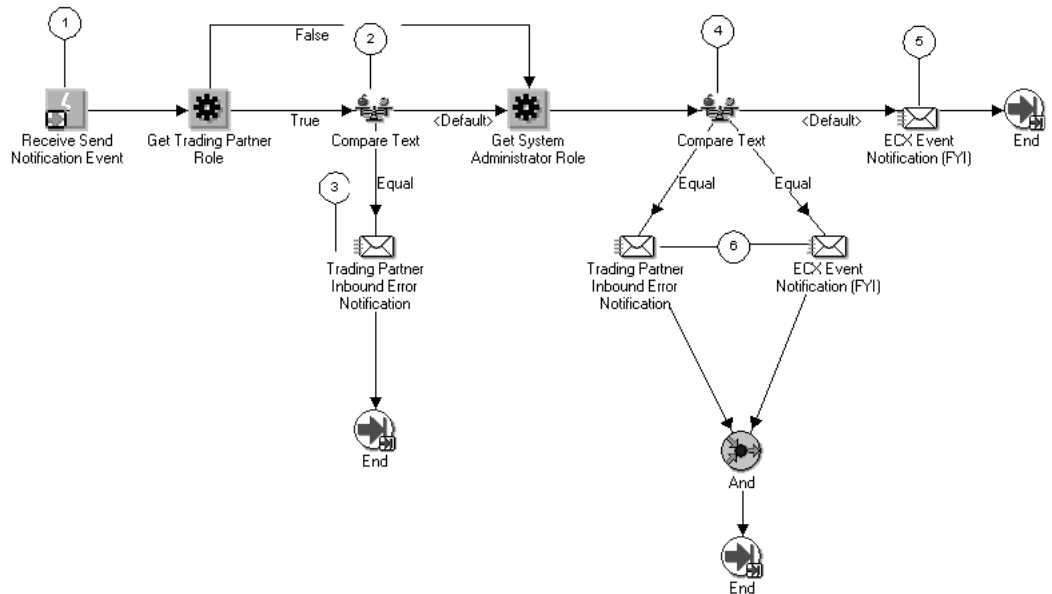
- Check for response from the system administrator and issue time-out if no response received (node 2).
- Execute retry function (node 3) if the system administrator wants to retry the failed outbound process. The retry process resumes from the point of failure. Continue retry function until error is resolved or failed process is aborted.
- Execute abort function (node 4) if the system administrator wants to abort the failed outbound process.
- Resolve error and end process (node 5)



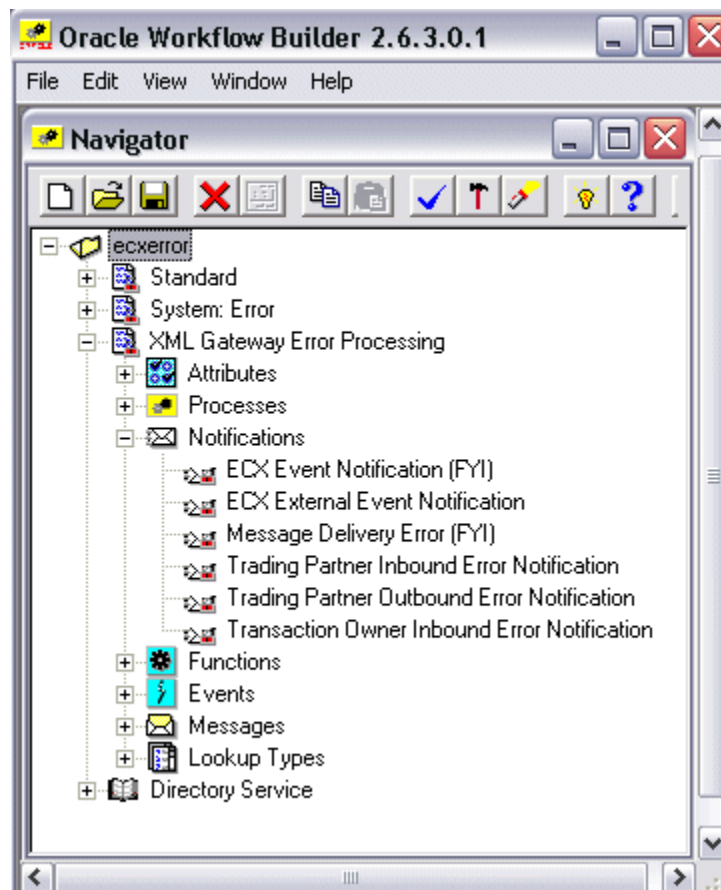
ECX Engine Notification Process

The Receive Send Notification Event (shown below, node 1) triggers the ECX Engine Notification Process.

The ECX Engine Notification Process checks the error type to determine the target recipient (node 2). If the target recipient is the Trading Partner contact, notification is sent (node 3). If the error type does not indicate that the Trading Partner contact is the sole target recipient, the process checks the error type (node 4) to determine if the notification should be sent to the XML Gateway system administrator (node 5), or both parties (node 6).



Notifications



The following is a description of each Notification supported by the XML Gateway Error Processing Item Type:

ECX Event Notification (FYI)

The ECX Event Notification [FYI] is used to send a notification to the system administrator.

This notification is used by the ECX Engine Notification Process.

ECX External Event Notification

The ECX External Event Notification is used to send a notification to the system administrator.

This notification is used by the Error Handling for Inbound Messages process.

Message Delivery Error (FYI)

The Callback feature allows messaging systems (including OTA) to report message delivery status back to the Workflow process that initiated message creation. If delivery fails, the `apps.ecx.processing.message.callback` event is raised. The corresponding event subscription for the `FYI_MESSAGE_DELIVERY_ERROR` process is executed and a Workflow notification is sent to the System Administrator (defined in the ECX: System Administrator Email Address profile) who has the option to retry/resend or about/ignore the process.

Trading Partner Inbound Error Notification

The Trading Partner Inbound Error Notification is used to send a notification to the Trading Partner contact.

This notification is used by the Error Handling for Inbound Message and ECX Engine Notification processes.

Trading Partner Outbound Error Notification

The Trading Partner Outbound Error Notification is used to send a notification to the Trading Partner contact.

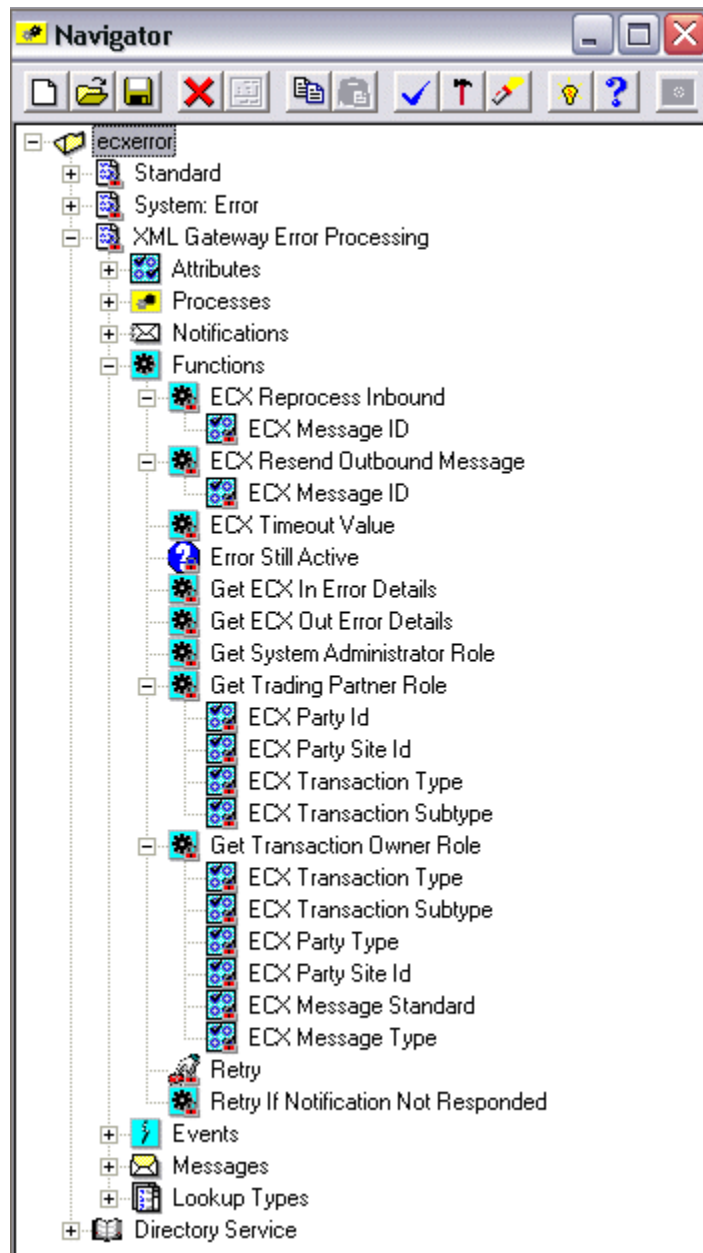
This notification is used by the Error Handling for Outbound Message process.

Transaction Owner Inbound Error Notification

The Transaction Owner Inbound Error Notification is used to send a notification to the transaction owner if the owner is specified in the Define Transactions form.

This notification is used by the Error Handling for Inbound Message process.

Functions



Several functions are provided in the XML Gateway Error Processing Item Type to support the error handling process.

ECX Reprocess Inbound

The ECX Reprocess Inbound function is used if the XML Gateway system administrator responded to the error notification by selecting the "reprocess" option. The reprocess function will rerun the inbound process using a copy of the inbound message stored in

the ECX_DOCLOGS table.

The "reprocess" function is not supported for external notifications to the Trading Partner.

The attribute for the ECX Reprocess Inbound function is:

ECX Message ID	This is a unique identifier provided by the XML Gateway execution engine for each outbound message.
-----------------------	---

ECX Resend Outbound Message

The ECX Resend Outbound function is provided to support Oracle Exchange only, it is not used in the XML Gateway Error Processing Item Type. The ECX Resend Outbound function resumes from a copy of the outbound message stored in the ECX_DOCLOGS table.

The Oracle Workflow "retry" function is used to rerun an outbound process from the point of failure. This is handled using the Default Error Process.

The attribute for the ECX Resend Outbound function is shown in the following table:

ECX Message ID	This is a unique identifier provided by the XML Gateway execution engine for each outbound message.
-----------------------	---

ECX Timeout Value

The ECX Timeout Value function is used to retrieve the timeout period measured in minutes stored in the ECX: Notification Timeout profile option.

If the XML Gateway system administrator or the transaction owner has not taken explicit action, such as reprocess or abort, on a notification that was received earlier, after the notification timeout period, the XML Gateway execution engine will automatically reprocess the erred transaction until the number of maximum retries is exceeded. The maximum retries value is stored in the ECX: Maximum Reprocess Attempts For Errored Inbound Transaction profile option.

Error Still Active

The Error Still Active function is used to check whether the error still existed for the outbound message after notifications have sent to appropriate parties, such as Trading Partner contact and XML Gateway system administrator.

Get ECX In Error Details

The Get ECX In Error Details function activity is used to get the details regarding an inbound error to prepare the e-mail notification. In addition, required information is passed to the Error Handling for Inbound Messages process activity.

Get ECX Out Error Details

The Get ECX Out Error Details function is used to get the details regarding an outbound error to prepare the notification. In addition, required information is passed to the Error Handling for Outbound Messages process.

Get System Administrator Role

The Get System Administrator Role function is used to retrieve the e-mail address for the system administrator stored in the ECX: System Administrator Email Address system profile.

The e-mail address retrieved from the system profile is returned by the function and stored in the ECX System Administrator Role (ECX_SA_ROLE) item attribute.

A notification is sent to the XML Gateway system administrator for system or process errors detected by Oracle XML Gateway or Oracle Workflow Business Event System.

Get Trading Partner Role

The Get Trading Partner Role function is used to determine the e-mail address for the Trading Partner contact that was provided when the Trading Partner was defined.

The function uses the attribute values passed to it to uniquely identify the Trading Partner. The e-mail address associated with the Trading Partner selected returned by the function is stored in the ECX Trading Partner Role (ECX_TP_ROLE) item attribute.

A notification is sent to the Trading Partner contact for data errors detected by Oracle XML Gateway.

The attributes for the Get Trading Partner Role function are shown in the following table:

Attribute Name	Attribute Description
ECX Party ID	This is the unique identifier for the Trading Partner defined in Oracle E-Business Suite. This field is optional.
ECX Party Site ID	This is the unique identifier for the Trading Partner site defined in Oracle E-Business Suite.
ECX Transaction Type	This is the Transaction Type defined in the XML Gateway Define Transactions form.
ECX Transaction Subtype	This is the Transaction Subtype defined in the XML Gateway Define Transactions form.

Get Transaction Owner Role

The Get Transaction Owner Role function is used to determine the e-mail address for the transaction owner if the owner's user name was specified in the Define Transactions form.

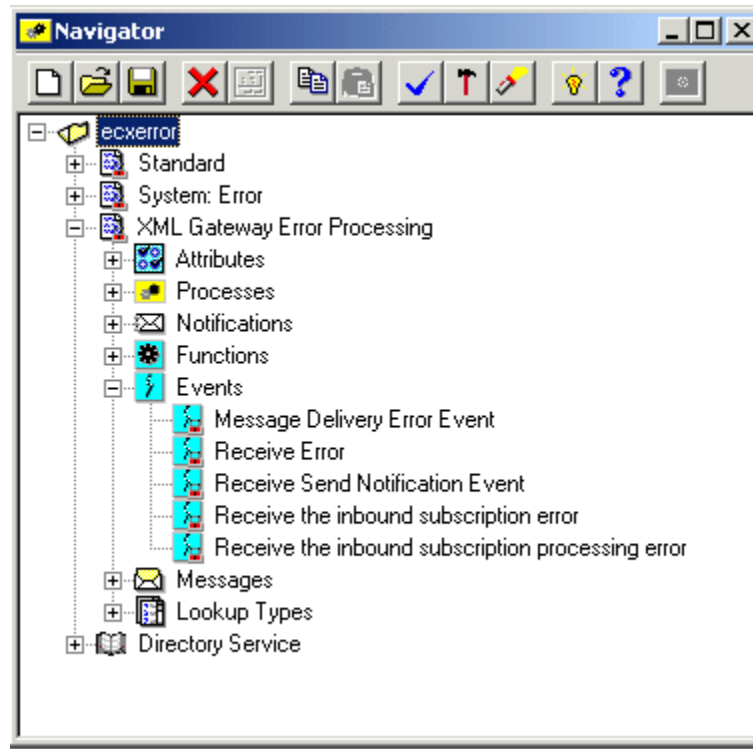
The function uses the attribute values passed to it to uniquely identify the transaction owner if the owner is defined. The e-mail address associated with the transaction owner's user name that was defined in the applications as the application login name is retrieved and returned by the function.

A notification is sent to an appropriate transaction owner depending on the erred transaction type for process errors detected by Oracle XML Gateway.

The Get Transaction Owner Role function uses the following attributes:

Attribute Name	Attribute Description
ECX Transaction Type	This is the Transaction Type defined in the XML Gateway Define Transactions form.
ECX Transaction Subtype	This is the Transaction Subtype defined in the XML Gateway Define Transactions form.
ECX Party Type	This is the party type for the Trading Partner defined in Oracle E-Business Suite.
ECX Party Site ID	This is the unique identifier for the Trading Partner site defined in Oracle E-Business Suite.
ECX Message Standard	This is the XML message format standard defined in the Define Transactions form.
ECX Message Type	This is the payload message format, which is XML.

Events



The XML Gateway Error Processing Item Type supports the following event activities:

- Message Delivery Error
- Receive Error
- Receive Send Notification Event
- Receive the inbound subscription error
- Receive the inbound subscription processing error

All of the XML Gateway seeded events and event subscriptions are seeded with a Customization Level of "L" for Limit, which means the event or event subscription may be enabled or disabled but may not be changed in any other way. Because the seeded events and event subscriptions play an integral role in the total solution, Oracle does not recommend that you disable any of them. If necessary, you may define additional events and event subscriptions to augment the seeded activities.

Message Delivery Error

The Message Delivery Error Event is used by the XML Gateway to indicate that an error has occurred. This event is used with OTA Callback.

The seeded event name is `oracle.apps.ecx.processing.message.callback`. This event triggers the (FYI) Message Delivery Error Process workflow.

Receive Error

The Receive Error event is used by XML Gateway to indicate that the execution engine has detected an error. Regardless of whether the error was detected by Oracle XML Gateway or Oracle Workflow Business Event System, the same ECX Main Error Process manages the error. See: ECX Main Error Process, page 6-30 for the details.

The seeded event name is `oracle.apps.ecx.processing.message.error`. This event triggers the ECX Main Error Process workflow.

Receive Send Notification Event

The Receive Send Notification Event is used by XML Gateway to indicate that the execution engine has identified a need to send a notification for errors related to an inbound process. The ECX Engine Notification Process manages the error. See: ECX Engine Notification Process, page 6-38 for the details.

The seeded event name is `oracle.apps.ecx.processing.notification.send`. This event triggers the ECX Engine Notification Process workflow.

Receive the inbound subscription error

The event `"oracle.apps.ecx.inbound.message.receive"` is raised by the `ECX_INBOUND` agent listener after it dequeues a message from the `ECX_INBOUND` queue. The "Receive Inbound Subscription Error" Event is used to indicate that there is an error when executing the subscription for the event `"oracle.apps.ecx.inbound.message.receive"`.

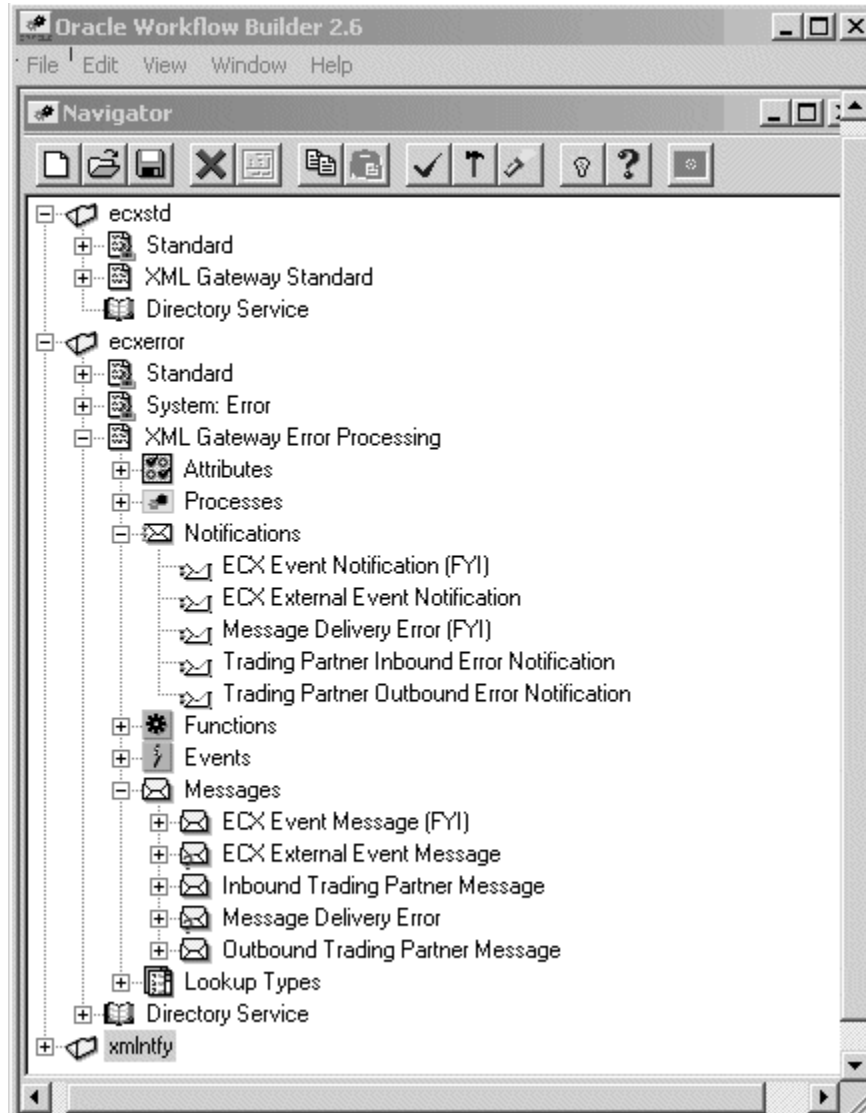
This subscription saves the xml message to `ecx_doclogs`, checks the trading partner setup for the given transaction, performs logging, and enqueues the message to the next queue for the actual map processing.

Receive the inbound subscription processing error

The event `"oracle.apps.ecx.inbound.message.process"` is raised by the `ECX_TRANSACTION` agent listener after it dequeues a message from the `ECX_IN_OAG_Q` queue. The "Receive Inbound Subscription Processing Error" is used to indicate that there is an error when processing the inbound xml document.

Messages

The XML Gateway Error Processing Item Type provides several message templates used by the various notification activities to send a notification to the XML Gateway system administrator or the Trading Partner contact based on the nature of the error.



Below is a description of each message template. The message template is displayed in the Body tab and Text Body subtab.

ECX Event Message (FYI)

The ECX Event Message [FYI] template is used by the ECX Event Notification. The template includes information regarding the transaction, the type of error detected, and the associated error message.

ECX External Event Message

The ECX External Event Message template is used by the ECX External Event Notification activity. The template includes information regarding the transaction, the type of error detected, and the associated error message.

Inbound Trading Partner Message

The Inbound Trading Partner Message template is used by the Trading Partner Inbound Error Notification activity. The template includes information regarding the transaction, the type of error detected, and the associated error message.

Message Delivery Error

The Message Delivery Error template is used by the XML Message Delivery Callback error process. The template includes information regarding the action to take in case of error.

Outbound Trading Partner Message

The Outbound Trading Partner Message template is used by the Trading Partner Outbound Error Notification activity. The template includes information regarding the transaction, the type of error detected, and the associated error message.

Message Template Attributes

Each message template has the attributes listed in the following table. The values for the attributes are provided by the Get ECX Out Error Details and Get ECX In Error Details function. The values are used to replace the message tokens in the message template.

For a detailed explanation of each error and possible corrective actions, see Manual Troubleshooting Steps, page I-15.

The following table describes all the possible attributes:

- Event Key is a unique identifier for an instance of an event. The combination of event name, event key, and event data fully describe what occurred in the event.

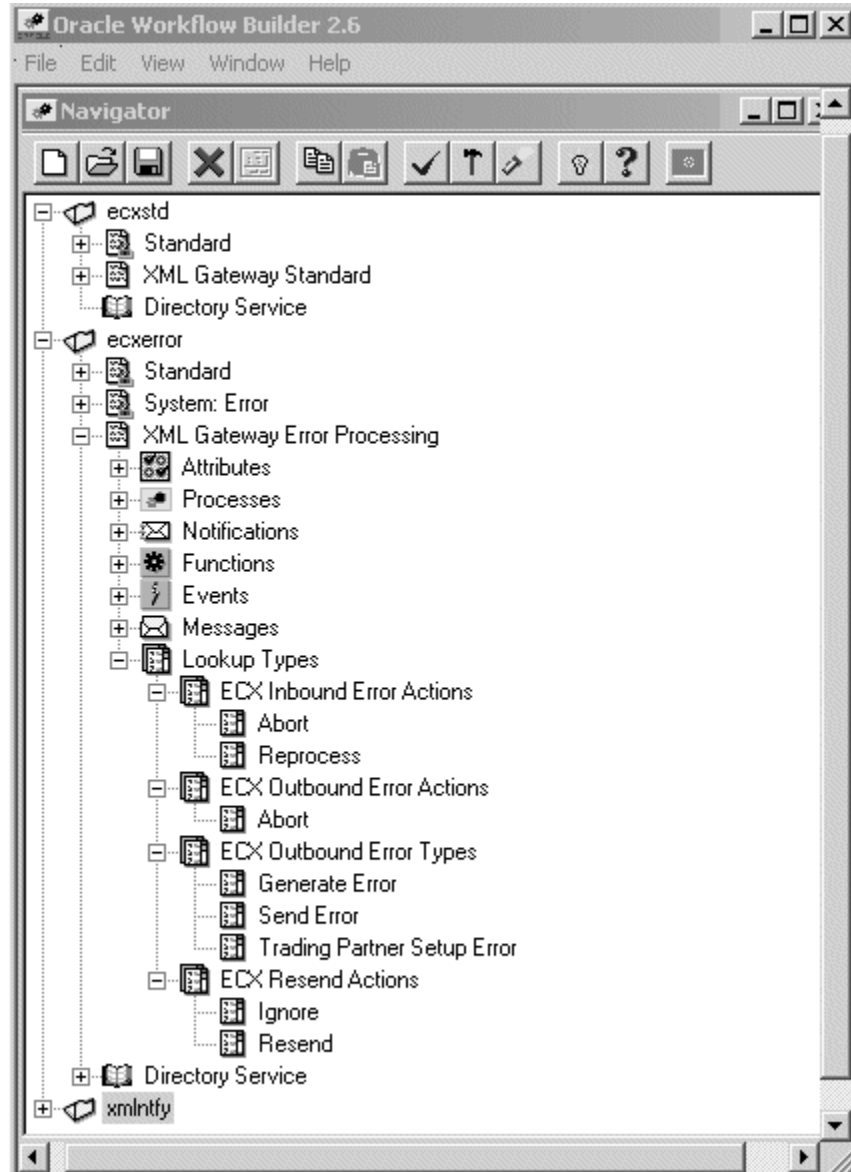
Attribute Name	Attribute Description
Event Key	Event Key is a unique identifier for an instance of an event. The combination of event name, event key, and event data fully describe what occurred in the event.
ECX Transaction Type	This is the Transaction Type defined in the XML Gateway Define Transactions form.
ECX Document ID	This is the unique identifier for the business document. It can be the document number or its associated database key, whichever is guaranteed to be unique for the transaction.
ECX Error Message	This is the error message text describing the error detected.

Attribute Name	Attribute Description
ECX Error Type	This is the error type code. The valid values are: • 10 = Success • 20 = Send error notification to Trading Partner contact • 25 = Send error notification to both Trading Partner contact and XML Gateway System Administrator • 30 = Send error notification to XML Gateway System Administrator
ECX Return Code	This is the return status code. The valid values are: • 0 = Success • 1 = Warning • 2 = Error
ECX Message Standard	Message format standard (such as OAG) as defined in the Define Transactions form.
ECX TP Header ID	Trading Partner name.
ECX Message Type	Message Type, which is XML.
ECX Logfile	If a log file is created, the name and location.
ECX Status	The status of the transaction.
ECX Time Stamp	The date the log file was created.
ECX Transaction Subtype	This is the Transaction Subtype defined in the XML Gateway Define Transactions form.
ECX Party ID	This is the unique identifier for the Trading Partner defined in Oracle E-Business Suite.
ECX Party Site ID	This is the unique identifier for the Trading Partner site defined in the XML Gateway Define Transactions form.

Attribute Name	Attribute Description
ECX Internal Control Number	The Internal Control Number is a system-generated number that uniquely identifies the message being processed.
Event Name	This is a unique identifier for the business event. The naming convention is oracle.apps.<product code>.<component>.<object>.<event> The Event Name is required for the Generate XML Document function to store the value returned.
ECX Party Admin Email	System Administrator as defined in the Trading Partner Setup window.
ECX Protocol Address	Protocol address used (for example, SMTP, HTTP, HTTPS).
ECX Attribute 1	Optional variable as defined in the message map.
ECX Attribute 2	Optional variable as defined in the message map.
ECX Attribute 3	Optional variable as defined in the message map.
ECX Attribute 4	Optional variable as defined in the message map.
ECX Attribute 5	Optional variable as defined in the message map.
ECX Trigger ID	Unique number given to each event being raised.
ECX Trading Partner Role	The e-mail address from the Trading Partner Setup window that is returned by the Get Trading Partner Role function.
ECX Protocol Type	Transmission protocol as defined in the Setup Trading Partner window.

Lookup Types

The XML Gateway Error Processing Item Type supports several lookup types for the error handling processes.



ECX Inbound Error Actions

The ECX Inbound Error Actions lookups are used by the ECX External Event Notification as a Result Type attribute. The valid values are as follows:

- Abort
Abort the failed inbound process.
- Reprocess
Reprocess the failed inbound process.

ECX Outbound Error Actions

The ECX Outbound Error Actions lookups is used by the Default Error Process. The valid value is as follows:

- Abort
Abort the failed outbound process

The retry function is supported by the Default Error Process.

ECX Outbound Error Types

The ECX Outbound Error Types lookups are used by the Get ECX Error Type function as a Result Type attribute. The valid values are as follows:

- Generate Error
The error was detected during the message creation process as opposed to the message delivery process.
- Send Error
The error was detected during the message delivery process as opposed to the message creation process.
- Trading Partner Setup Error
The error was based on incorrect Trading Partner setup. Correct the erroneous set up before proceeding further.

ECX Resend Actions

ECX Resend Actions are lookups used by the XML Gateway Error Process. The valid values are as follows:

- Ignore
- Resend

Configure Oracle Prebuilt Inbound Messages

Seeded Business Event and Corresponding Event Subscription

Inbound messages that are prebuilt and delivered by Oracle E-Business Suite application modules are delivered with the following:

- Message map containing an XML Gateway Procedure Call action as the last activity. This activity sets event details to indicate that XML Gateway has successfully

processed the inbound message.

- Seeded event subscription to consume the business event raised in the message map.

The event subscription is seeded as an enabled subscription. You can disable the event subscription if necessary.

An event subscription is required for inbound messages from Trading Partners (B2B) regardless of whether the application is interested in the inbound message. The seeded event subscription uses the Workflow Default Rule Function but may be configured to use a rule function and Workflow process relevant to the business requirement.

See the Message Designer, page 2-1 for instructions on how to create a new message map and how to modify an Oracle prebuilt message map.

Configuration Options for Seeded Event Subscription

Use the Workflow Administrator to configure the seeded event subscription for the prebuilt inbound message if necessary.

1. Register new event subscription

Use the Workflow Administrator Add Event Subscription window to register the new event subscription if you defined a new subscription instead of configuring the seeded subscription.

See Manage Workflow Processes, page 6-59 for instructions on how to register new event subscriptions.

2. Configure Seeded Subscription

Use Oracle Workflow Administrator Add Event Subscription window to configure the seeded event subscription to consume the inbound message if your application is interested in it. The subscription will trigger the Oracle E-Business Suite to perform some activity. The actual activity performed by the event subscription will be based on the Workflow process defined for it. Some examples are as follows:

- Integrate with existing Workflow process defined in the Oracle E-Business Suite
- Call an application API to perform a specific function

See: Manage Workflow Processes, page 6-59 for instructions on how to register new event subscriptions and configure a seeded event subscription.

Configure Oracle Prebuilt Outbound Messages

Seeded Business Event and Corresponding Event Subscription

Outbound messages that are prebuilt and delivered by Oracle E-Business Suite application modules are delivered with the following:

- Calls in the application module to raise a business event to indicate when something of interest has occurred. This includes event points indicating when a document was created, changed, confirmed, or deleted.
- Seeded event subscription using the XML Gateway or Workflow Default Rule Function to consume all the business events raised in the application module.

The event subscription is seeded as an enabled subscription. You can disable the event subscription if necessary.

The seeded event subscription consumes all the seeded events but may be configured to execute a specific Workflow process for the outbound events it is interested in.

See the Message Designer chapter, page 2-1 for instructions on how to create a new message map and how to modify an Oracle prebuilt message map.

Configuration Options for Seeded Event Subscription

Use the Workflow Administrator to configure the seeded event subscription for the prebuilt outbound message if necessary.

1. Register new event subscription

Use the Workflow Administrator Add Event Subscription window to register the new event subscription if you defined a new subscription instead of configuring the seeded subscription.

See Manage Workflow Processes, page 6-59 for instructions on how to register new event subscriptions.

2. Configure Seeded Event Subscription

Use Oracle Workflow Administrator Add Event Subscription window to configure the seeded event subscription to consume the application business event of interest. The subscription will trigger the Oracle E-Business Suite to perform some activity. The actual activities performed by the event subscription will be based on the Workflow process defined for it.

There are three subscription options for the Business-to-Business (B2B) integration scenario described below. The function activities outlined are supported by the XML Gateway Standard Item Type.

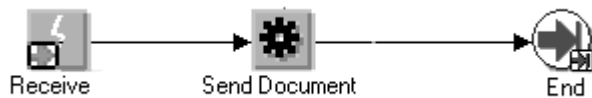
Send Document Function Using XML Gateway Rule Function

This option uses the XML Gateway Rule Function which has the Transaction Delivery Required function embedded in it to determine if the Trading Partner is enabled for the transaction before the message is generated.

The "Send Document" function can be configured to generate the message in synchronous or asynchronous mode by setting the Send Mode attribute to "Immediate" or "Deferred" respectively.

The following is an example of a Workflow process launched by the XML Gateway Rule Function. The Receive activity is followed by the Send Document function activity, which proceeds to End. The Transaction Delivery Required Function is not modeled in the Workflow process since it is already included in the rule function.

An Example of a Workflow Process Flow Launched by the XML Gateway Rule Function



Note: The first Workflow activity may be a Receive or a Start activity. Use Receive to continue an existing Workflow process. Use Start to begin a new Workflow process.

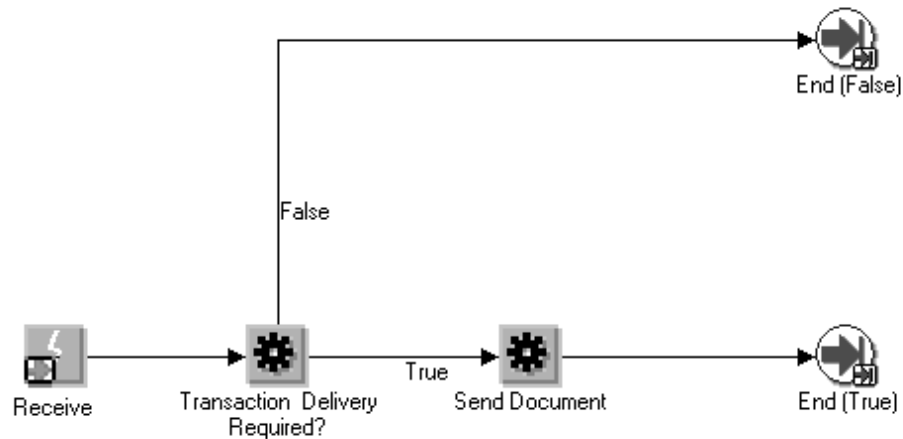
3. Send Document Function Using Workflow Default Rule Function

This option is used to first determine if the Trading Partner is enabled for the transaction. If the result of the Transaction Delivery Required function activity returns TRUE, the "Send Document" function activity is executed.

The "Send Document" function activity may be configured to send the message in synchronous or asynchronous mode by setting the Send Mode attribute to "Immediate" or "Deferred" respectively.

The following is an example of a Workflow process launched by the Workflow Default Rule Function. The Transaction Delivery Required Function is modeled in the Workflow process since it is not included in the Workflow Default Rule Function.

An Example of a Workflow Process Flow Launched by the Workflow Default Rule Function



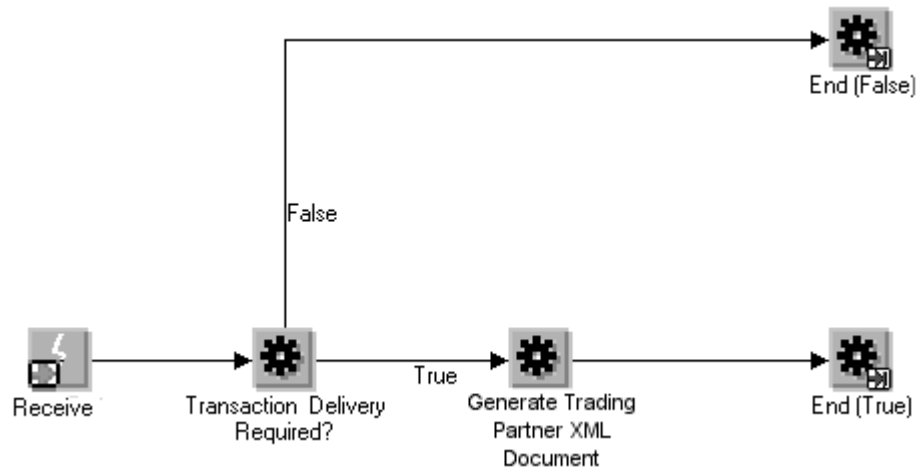
Note: The first Workflow step may be a Receive or a Start activity. Use Receive to continue an existing Workflow process. Use Start to begin a new Workflow process.

4. Generate Trading Partner XML Document

This option is used to gather and return the message data in the Event Message attribute without delivering it to the Trading Partner. The Event Message is then processed according to the subsequent Workflow instruction.

The following is an example of a Workflow process using the Generate Trading Partner XML Document function. A Receive activity is followed by the Transaction Delivery Required function activity. If it returns False, the process proceeds to End. If True, the Generate Trading Partner XML Document function activity is executed and proceeds to End. In a production Workflow process, the Generate Trading Partner XML Document function activity would be followed by an application-specific activity.

An Example of a Workflow Process Flow Using the Generate Trading Partner XML Document Function



Note: The first Workflow step may be a Receive or a Start activity. Use Receive to continue an existing Workflow process. Use Start to begin a new Workflow process.

Application to Application Integration

The XML Gateway Standard Item Type provides function activities to support A2A integration requirements. Listed below are the options for inbound and outbound transactions.

Inbound Option

Consume XML Document

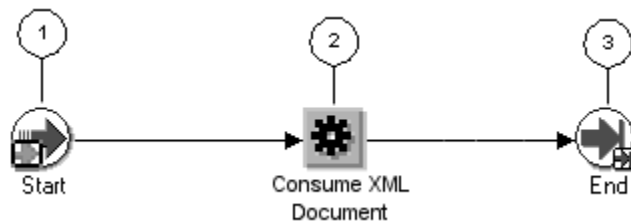
This option is used to interact with Oracle XML Gateway to insert data into the base application tables. Depending on how the message map was defined, the data may be inserted using an Application Open Interface API or an Application API. See: How to Map to an API, page 2-110.

The Consume XML Document function is used for A2A integration. It may be used in existing Workflow processes for which the Event Message attribute is used to pass the message data. This makes it unnecessary to raise a business event or define an event subscription as recommended in the B2B integration scenario.

The following is an example of a Workflow process using the Consume XML Document function:

This Workflow process shows the Consume XML Document (2) function activity

completing and followed by an End function (3). In a production Workflow process, the Consume XML Document function activity would be followed by an application-specific activity.



Note: The first Workflow step (1) may be a Receive or a Start activity. Use Receive to continue an existing Workflow process. Use Start to begin a new Workflow process.

Outbound Option

Generate XML Message

This option is used to generate the XML message and forward it to the subsequent Workflow process. The Generate XML Document function activity is used to gather the document data and return it in the Event Message attribute. The Event Message is then processed according to the Workflow instructions which may be to send it to another Oracle E-Business Suite application module.

The following is an example of a Workflow process using the Generate XML Document function:

An Example of a Workflow Process Flow Using the Generate XML Document Function



The Workflow process shows the Generate XML Document function activity completing and followed by an End function. In a production Workflow process, the Generate XML Document function activity would be followed by an application specific activity.

Note: The first Workflow step may be a Receive or a Start activity. Use Receive to continue an existing Workflow process. Use Start to begin a new Workflow process.

Manage Workflow Processes

Use the following Oracle Workflow Administrator windows to help manage your Workflow processes.

For more information about these windows, see *Oracle Workflow Developer's Guide* and *Oracle Workflow User's Guide*.

Register New Business Events and Event Subscription

This process is necessary for new events and subscriptions added as a result of creating a new XML message. The seeded business events and event subscriptions delivered by the Oracle E-Business Suite have already been registered. There is no need to register a seeded event subscription that was configured during implementation.

Use the Events window, *Oracle Workflow Developer's Guide* to add a new business event.

Use the Subscriptions window, *Oracle Workflow Developer's Guide* to add a new subscription to consume the new event.

Identify Seeded Item Types

Use the Developer Studio, *Oracle Workflow Developer's Guide* to view the seeded item types delivered by Oracle Workflow Business Event System, Oracle XML Gateway, and Oracle E-Business Suite modules.

Identify Seeded Business Events and Associated Event Subscriptions

Use the Events window, *Oracle Workflow Developer's Guide* to view the seeded business events delivered by Oracle Workflow Business Event System, Oracle XML Gateway, and Oracle E-Business Suite modules.

Select the business event and click on Edit Subscription to proceed to the Subscriptions window, *Oracle Workflow Developer's Guide* to view the associated subscriptions.

Configure or Delete Seeded Event Subscriptions

From the Subscriptions window, *Oracle Workflow Developer's Guide*, select the subscription and click Edit to configure the subscription. Click Delete to delete the subscription. You can modify the rule function or Workflow process defined for the subscription.

View and Respond to Error Notifications

Use the Worklist window, *Oracle Workflow User's Guide* to view and respond to notifications awaiting your attention.

Purge XML Gateway Transactions

To purge workflow or non-workflow related XML Gateway transaction data, Oracle XML Gateway allows you to use the following concurrent programs to purge XML Gateway transactions:

- Purge Obsolete ECX Data Concurrent Program, page 6-60
- Purge Obsolete Workflow Runtime Data Concurrent Program, page 6-61

Purge Obsolete ECX Data Concurrent Program

To purge non-workflow related XML Gateway transactions, Oracle XML Gateway lets you use the Purge Obsolete ECX Data concurrent program to exclusively purge XML Gateway transaction data based on transaction type, transaction subtype, and date range.

Note: Since Workflow purge is associated with item_type and item_key, XML Gateway logs do not have these parameters available all the time. In case of inbound transaction item_type and item_key parameters are not available, you can submit the Purge Obsolete ECX Data concurrent program to specifically purge XML Gateway transaction data.

Parameters and Purge Behavior

Parameters	Expected Behavior
Transaction Type	All data corresponding to that specific Transaction Type will be deleted.
Transaction Type Transaction Subtype	All data corresponding to that specific Transaction Type and Subtype will be deleted.
Transaction Type Transaction Subtype From Date	All data corresponding to that specific Transaction Type and Subtype will be deleted after the specified date.

Parameters	Expected Behavior
Transaction Type	All data corresponding to that specific Transaction Type and Subtype will be deleted within the specified date range.
Transaction Subtype	
From Date	
To Date	
From Date	All data after the specified date will be deleted regardless of Transaction Type and Subtype.
From Date	All data within the specified date range will be deleted regardless of Transaction Type and Subtype.
To Date	
No parameter is given	All data will be deleted regardless of Transaction Type, Subtype, and date range.

Use the Submit Requests form in Oracle E-Business Suite to submit these concurrent programs.

Note: Commit Cycle will be of 500 rows as it is done in Workflow Purge.

Purge Obsolete Workflow Runtime Data Concurrent Program

When purging workflow related XML Gateway transactions, use the Purge Obsolete Workflow Runtime Data concurrent program instead. This concurrent program is used in conjunction with the 'ECX: Purge ECX data with WF' profile option which lets you have an option to either purge XML Gateway data along with workflow data or purge workflow data only.

When the Purge Obsolete Workflow Runtime Data concurrent program is submitted, the 'ECX: Purge ECX data with WF' profile value will be retrieved to determine whether XML Gateway data will be purged together with workflow data or only workflow data will be purged.

- If the profile value is set to 'Y' (default), then all XML Gateway data along with workflow data will be purged.
- If the profile value is set to 'N', then XML Gateway data will not be purged with workflow data, but only workflow data will be purged.

For more information on 'ECX: Purge ECX data with WF' profile option, see Define System Profile Options, page 3-3.

Note: You can use Oracle Workflow Manager to submit and manage the Purge Obsolete Workflow Runtime Data concurrent program. See: *Purging Workflow Data, Oracle Workflow Administrator's Guide.*

Monitor Workflow Processes

Identified below are several Oracle Workflow Administrator windows to help monitor your Workflow processes:

Transaction-Level Trace

The trace function is performed via centralized logging through the Applications Logging Framework. Using the logging framework you can enable logging and set log levels using FND profile options. For information on logging, see the Logging chapter, *Oracle E-Business Suite Maintenance Guide.*

Monitor Transaction Status

Use the Event Queue Summary window to monitor the status of a transaction. Select the Agent Listener and click the View Detail icon. Once you enter the selection criteria to identify the transaction, the detail information including status and an option to view the event data in XML or text format, are presented.

Review XML Message Returned by Generate Functions

Modify the subscription for the Generate XML Document or Generate Trading Partner XML Document function and set the Action: Out Agent field to the WF_OUT. The XML data gathered by the function is directed to the WF_OUT agent.

For instructions on viewing the XML message directed to WF_OUT see: Monitor Transaction Status.

Start Agent Listeners

There is one Agent Listener per inbound agent. The agents necessary to support the integration between Oracle Workflow Business Event System and Oracle XML Gateway are shown in the following table:

Agent Name	Description
ECX_INBOUND	Agent for inbound messages originating from outside the enterprise for B2B integration.

Agent Name	Description
ECX_TRANSACTION	Agent for inbound messages originating from within the enterprise for A2A integration.
WF_OUT	Agent to stage outbound messages. ECX_OUTBOUND is the agent used to stage and deliver outbound XML messages to the Trading Partner.
WF_IN	Agent to receive inbound messages sent to the Oracle E-Business Suite.
WF_DEFERRED	Agent for outbound messages sent in deferred mode.
WF_ERROR	Agent for errors detected by Oracle Workflow Business Event System or Oracle XML Gateway.

The agents must be enabled and the seeded agent listener service components must be running.

Note: Additional agents are provided for Web services. See Web Services Setup, page 8-19.

Use the Workflow Administrator: Agents window to enable the required agents if necessary (the agents are enabled by default). The same window can be used to disable an agent.

Use Oracle Applications Manager to schedule and manage agent listener service components. For more information, refer to the Oracle Applications Manager online help.

Ensure that the seeded agent listener service components named Workflow Deferred Agent Listener and Workflow Error Agent Listener, provided by Oracle Workflow, are running for the WF_DEFERRED and WF_ERROR agents, respectively.

The agent listener service components ECX Inbound Agent Listener and ECX Transaction Agent Listener are seeded with a manual startup mode and therefore must be manually started. The startup mode can be changed to "Automatic" using the Oracle Applications Manager, if desired.

Place any user-defined agents in their own container.

Development Guidelines for Custom Messages for B2B Integration

If you are using Oracle XML Gateway to create new inbound and outbound messages, use the following guidelines to raise business events and define event subscriptions.

A prerequisite to this process is a message map created using the XML Gateway Message Designer.

See the Message Designer section, page 2-1 for instructions on how to create a new message map and how to modify an Oracle prebuilt message map.

Development Guidelines for Outbound Messages

Following are the guidelines for developing outbound messages:

1. Register the Business Event and Corresponding Event Subscription

Register the business event and corresponding event subscription defined in the Oracle E-Business Suite.

See: Manage Workflow Processes, page 6-59 for the instructions on registering business events and event subscriptions.

2. Raise a Business Event in the Application

Use the Workflow function to raise business events in your application at points indicating when a document has been created, changed, confirmed, or deleted. Business events may be raised in PL/SQL code, by an existing Workflow process using the Raise Document Delivery Event activity, or as the first activity of a new Workflow process.

It is important to raise an event at all possible event points even though you may not define a specific event subscription for it. This will allow you to define an event subscription for any event of interest without having to modify the application code to raise the event.

3. Define an Event Subscription

Add an event subscription for each business event of interest. Use the function and event activities provided in the XML Gateway Standard Item Type to define the Workflow process associated with the event subscription.

See: Configure Oracle Prebuilt Outbound Messages, page 6-54 for the Workflow process configuration options.

See: Manage Workflow Processes, page 6-59 for the instructions on adding an event subscription.

There is no need to add an event subscription for business events that you are not interested in. These may be added at a later point in time when the event becomes of interest to your implementation.

Development Guidelines for Inbound Messages

The guidelines for developing inbound messages are as follows:

1. Register the Business Event and Corresponding Event Subscription

Register the business event and corresponding event subscription defined in the Oracle E-Business Suite.

See: Manage Workflow Processes, page 6-59 for the instructions on registering business events and event subscriptions.

2. Set Event Details in the Message Map

Make sure your message map includes a post process Procedure Call action to call the Workflow function to set event details indicating XML Gateway has successfully processed the inbound message.

See: Procedure Call: Execute Procedure, page 2-100 for details regarding the Procedure Call action.

3. Define an Event Subscription

An event subscription is required for inbound messages from Trading Partners (B2B) regardless of whether the application is interested in the inbound message.

Assuming your application is interested in the inbound message, define an event subscription with a rule function and Workflow process relevant to the business requirement.

See: Configure Oracle Prebuilt Outbound Messages, page 6-54 for the Workflow process configuration options.

See: Manage Workflow Processes, page 6-59 for the instructions on addition a Workflow process.

There is no need to add a subscription for business events that you are not interested in. These can be added later if the business event becomes of interest to your implementation.

Common Questions

The following are common implementation questions:

1. **Is the combination of the Generate Trading Partner XML Document and Send Document function activity valid?**

No. This combination of function activities is invalid.

The Generate Trading Partner XML Document function returns the XML message in the Event Message attribute. The Send Document function gathers the data, creates the XML message, and enqueues it onto the outbound queue.

The result of this combination of function activities is that the Send Document function activity will complete successfully but the Generate Trading Partner XML Document function activity will have data in the Event Message attribute that is not sent anywhere unless something is defined in the Out Agent attribute (associated with the Event Subscription) which will enqueue the message.

2. Is the combination of the Generate XML Document and Send Document function activity valid?

Same as answer 1.

3. Does the Generate Trading Partner XML Document function activity require the Transaction Delivery Required function activity to determine if the Trading Partner is defined to receive a document?

Yes. The Generate Trading Partner XML Document function activity must be preceded by the Transaction Delivery Required function activity.

Message Delivery Status

XML Message Delivery Callback allows messaging systems (such as Oracle Transport Agent) to report message delivery status back to the Workflow process that initiated message creation. If delivery fails, a Workflow notification is sent to the System Administrator who has the option to retry/resend or abort/ignore the process.

Use of XML Message Delivery Callback

Prior to the introduction of this feature, the common Workflow process to send an outbound document to a Trading Partner was the Send Document function activity as follows:

An Example of a Workflow Process Flow Using the Send Document Function



The Send Document function activity consists of message creation and message enqueue onto the ECX_OUTBOUND queue. If XML message delivery fails, the entire Workflow is re-executed causing the message creation step to be performed again unnecessarily.

With message delivery callback, the message creation is separated from the message enqueue, so that only the message enqueue is re-executed if the message delivery fails. The new model consists of the Generate Trading Partner XML Document function activity and the Workflow Send as follows:

An Example of a Workflow Process Flow Using the Generate Trading Partner XML Function and Workflow Send



The Generate Trading Partner XML Document function activity is responsible for gathering the message data. The Workflow Send event activity is responsible for enqueueing the message onto the ECX_OUTBOUND queue.

The Oracle E-Business Suite application module delivers a Workflow with the event subscription in support of outbound documents that follows this model. Both models are valid. As with any seeded event subscription, you have the option to disable it. For information on how to disable a seeded event subscription, see: Manage Workflow Processes, page 6-59.

There are four ways that XML Message Delivery Callback can be used. They are summarized as follows:

Block Mode = Y

Block Mode is an attribute of the Workflow Send event activity. A Block Mode value of "Y" implies that your Workflow process will await the message delivery status before proceeding to the next activity in the Workflow process. If delivery fails, the Workflow Default Error Process is executed to send a notification to the System Administrator. The System Administrator has the option to "Retry" the process assuming the error was corrected, or "Ignore" the process if the error cannot be corrected.

- **User Event Defined**

The XML message generated by the XML Gateway is enqueued onto the ECX_OUTBOUND queue. The messaging system (such as OTA or webMethods) dequeues the message from the ECX_OUTBOUND queue and attempts to deliver the message to the Trading Partner. Any user-defined events (defined in the Workflow Send event) will be raised and corresponding event subscriptions will be executed.

The message delivery status is returned regardless of success or failure. If the delivery fails, the Workflow Default Error Process is executed, allowing the System Administrator to retry or abort the process.

- **No User Event Defined**

The XML message generated by the XML Gateway is enqueued onto the ECX_OUTBOUND queue. The messaging system (such as OTA or webMethods)

dequeues the message from the ECX_OUTBOUND queue and attempts to deliver the message to the Trading Partner. The delivery status is returned regardless of success or failure. If the delivery fails, the Workflow Default Error Process is executed, allowing the System Administrator to retry or abort the process.

Block Mode = N

A Block Mode value of "N" implies that your Workflow process will proceed to the next activity on the Workflow process without waiting for the message delivery status. If delivery fails, the FYI_MESSAGE_DELIVERY_ERROR process is executed to send a notification to the System Administrator.

The System Administrator has the option to "Resend" the message assuming the error was corrected or "Ignore" the notification if no action is required.

The (FYI) Message Delivery Error is a seeded process that is part of the XML Gateway Error Processing Item Type.

- User Event Defined

The XML message generated by the XML Gateway is enqueued onto the ECX_OUTBOUND queue. The messaging system (such as OTA or webMethods) dequeues the message from the ECX_OUTBOUND queue and attempts to deliver the message to the Trading Partner. Any user-defined events (defined in the Workflow Send event) will be raised and corresponding event subscriptions will be executed.

If the delivery fails, the apps.ecx.processing.message.callback event is raised. The corresponding event subscription for the FYI_MESSAGE_DELIVERY_ERROR process is executed allowing the System Administrator to resend or ignore the message notification.

- No User Event Defined

The XML message generated by the XML Gateway is enqueued onto the ECX_OUTBOUND queue. The messaging system (such as OTA or webMethods) dequeues the message from the ECX_OUTBOUND queue and attempts to deliver the message to the Trading Partner.

If the delivery fails, the apps.ecx.processing.message.callback event is raised. The corresponding event subscription for the FYI_MESSAGE_DELIVERY_ERROR process is executed allowing the System Administrator to resend or ignore the message notification.

For more details, see XML Gateway Error Processing Item Type, page 6-27.

How Other Messaging Systems Use XML Message Delivery Callback

Other messaging systems such as webMethods and iAS can use the XML Message Delivery Callback feature by calling the ECX_ERRORLOG.external_system API. The

API is used by all messaging systems (both Oracle and non-Oracle) to report message delivery status. The status information is written to the XML Gateway log tables to track and report message delivery data.

For details on the API see: `ECX_ERRORLOG.external_system`, page F-3.

Oracle Transport Agent

This chapter covers the following topics:

- Oracle Transport Agent Overview
- Certificate-Based Authentication Methods
- Setup Parameters
- Oracle Transport Agent Inbound Test Page
- Connecting to Non-OTA Servers
- Code Connection Samples
- Troubleshooting

Oracle Transport Agent Overview

Oracle Transport Agent (OTA) is a lightweight messaging platform for transmitting documents over HTTP and Secure HTTP (HTTPS). OTA implements a messaging protocol on top of the HTTP Application protocol.

The OTA server is a Java-based servlet that uses the OTA messaging protocol to support the following requirements:

- Guaranteed, exactly-once delivery of a message over HTTP(S)
- Complete audit and history tracking of messages sent and received
- Outbound e-mail delivery of messages (SMTP)
- Server certificate authentication (when using TLS mode)
- Mutual authentication
- Built-in application user authentication to Oracle E-Business Suite and Oracle Exchange

Oracle Transport Agent Protocol Stack

The OTA protocol stack is as follows:

OTA (Messaging Protocol)

HTTP (Application Protocol)

TLS (Encryption/Security Protocol - optional)

TCP/IP (Network Protocol)

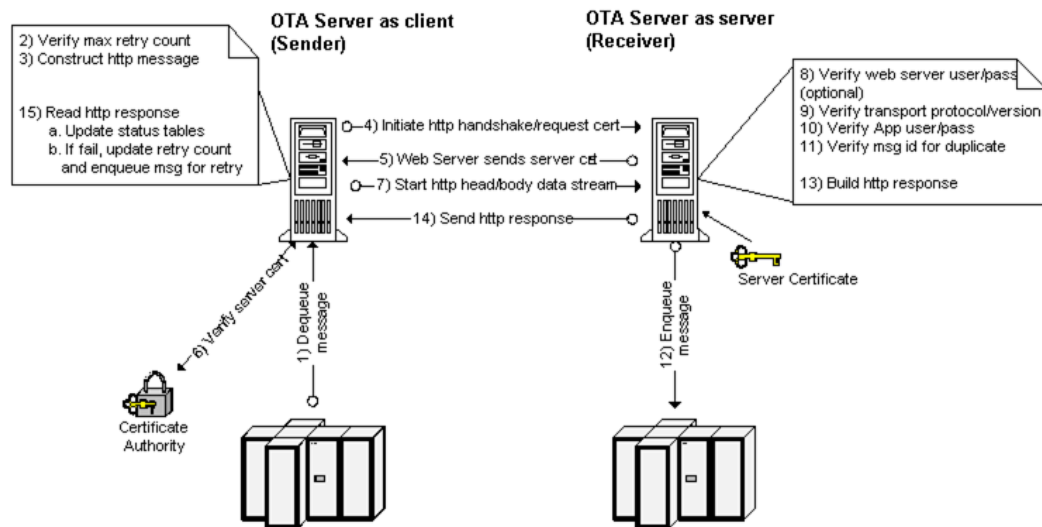
The OTA protocol defines the conversation semantics used by two Web servers running the OTA Servlet. The two OTA Servlets "talk" to each other to provide guaranteed, exactly-once delivery of the message.

OTA Message Propagation Flow

In this document, the Sender refers to the "Client" that sends a document or requests connection. The Receiver refers to the "Server" that receives the document or connection request.

The following diagram displays the message propagation flow of an OTA message with TLS enabled:

OTA Message Propagation Flow (with TLS Enabled)



1. The OTA client dequeues the message from the ECX_OUTBOUND queue.
2. The OTA client verifies the maximum retry count.
3. The OTA client constructs the HTTP message.

4. The client initiates the HTTP handshake and request for certification with the destination server.
5. The destination Web server sends its certification.
6. The client receives and verifies the server's certification.
7. If verified, the client starts the HTTP message head and body data stream.
8. The server optionally verifies the username and password.
9. The server verifies the transport protocol and version.
10. The server verifies the Oracle E-Business Suite username and password.
11. The server verifies that the message ID is not a duplicate.
12. The server enqueues the message for consumption.
13. The server builds the HTTP response message.
14. The server sends the HTTP response back to the client.
15. The client reads the response and updates the status tables. If the response indicates the message failed, the retry count is updated and the message is enqueued for retry.

Oracle Transport Agent Post Message

Two OTA servers communicate by sending and receiving a series of name/value pairs in the HTTP body of an HTTP POST/RESPONSE. Following is an example post from the sending OTA server (Note: the header authorization encryption follows the W3C standard):

```

HTTP Header
Http-Version: HTTP/1.1
Authorization: Digest username="myusername",
realm="testrealm@example.com",
nonce="dcd98b7102dd2f0e8b11d0f600bfb0c093",
uri="/dir/index.html",
qop=auth,
nc=00000001,
cnonce="0a4f113b",
response="6629fae49393a05397450978507c4ef1",
opaque="5ccc069c403ebaf9f0171e9517f40e41"
Content-length: 12345
Content-type: text/html
HTTP Body
TRANSPORT_PROTOCOL=OXTA
TRANSPORT_PROTOCOL_VERSION=1.0
REQUEST_TYPE=SEND
MESSAGE_ID=A1234567890ZZ0987654321
MESSAGE_TYPE=XML
MESSAGE_STANDARD=OAG
TRANSACTION_TYPE=PO
TRANSACTION_SUBTYPE=PROCESS
DOCUMENT_NUMBER=12345
PARTYID=9999
PARTY_SITE_ID=8888
PROTOCOL_TYPE=HTTPS-OXTA
PROTOCOL_ADDRESS=HTTPS://www.example.com/servlets/oracle.ecx.oxta.
transportAgentServer
USERNAME=myusername
PASSWORD=password
ATTRIBUTE1=
ATTRIBUTE2=
ATTRIBUTE3=
ATTRIBUTE4=
ATTRIBUTE5=
PAYLOAD=<xml ... .>

```

The HTTP Body contains the message envelope, message payload, and the following transport parameters:

- **TRANSPORT_PROTOCOL**

The Transport Protocol specifies the messaging protocol to the receiving servlet. The value will always be "OXTA" when sent by an OTA server. The receiving OTA server validates that the TRANSPORT_PROTOCOL is OTA, and then sends back the appropriate response in the HTTP response body.

- **TRANSPORT_PROTOCOL_VERSION**

The Transport Protocol Version specifies the version of the messaging protocol used to send the message

- **REQUEST_TYPE**

This parameter indicates the type of request sent from the OTA server. REQUEST_TYPE is not required on the receiving side. If no value is set, the default "SEND" type is used. Valid values are in the following list:

- **SEND** - (Default) The receiving OTA server treats this as a live, production message.

- AUTH - (Authorization test) The receiving OTA server treats this as a request to test the communication according to the OTA AUTH protocol. This request type requires only the following parameters:

Parameter	Sample Value
TRANSPORT_PROTOCOL	OXTA
TRANSPORT_PROTOCOL_VERSION	1.0
REQUEST_TYPE	AUTH
USERNAME	MYUSERNAME
PASSWORD	<i>password</i>

If the authorization passes, the response will contain an HTTP 200 response code and an OXTA 1000 status message. Refer to HTTP Status Codes, page 7-35 for a list of status codes.

- AUTH2 -(Authorization test, method 2) The receiving OTA server treats this as a request for a test according to the OTA AUTH2 test protocol. This request type requires only the following parameters:

Parameter	Sample Value
TRANSPORT_PROTOCOL	OXTA
TRANSPORT_PROTOCOL_VERSION	1.0
REQUEST_TYPE	AUTH2
USERNAME	MYUSERNAME
PASSWORD	<i>password</i>
PARTY_SITE_ID	123
TRANSACTION_TYPE	PO
TRANSACTION_SUBTYPE	PROCESS

- **EME - (Email me)** The receiving OTA server receives the message and places it on the outbound queue for delivery to the address specified in the `PROTOCOL_ADDRESS` parameter. This request type requires only the following parameters:

Parameter	Sample Value
<code>TRANSPORT_PROTOCOL</code>	OXTA
<code>TRANSPORT_PROTOCOL_VERSION</code>	1.0
<code>REQUEST_TYPE</code>	EME
<code>USERNAME</code>	MYUSERNAME
<code>PASSWORD</code>	<i>password</i>
<code>PROTOCOL_ADDRESS</code>	me@example.com
<code>PAYLOAD</code>	<xml....>

- **MESSAGE_ID**

This is the unique message identifier set by the sending OTA server. The receiving OTA server uses this identifier to determine if the message is a duplicate. The identifier is the Oracle AQ message ID from the outbound XML Gateway queue.

- **MESSAGE_TYPE**

This indicates the type of content in the payload. When used with XML Gateway, the value will always be "XML".

- **TRANSACTION_TYPE**

Used by XML Gateway to determine the type of document sent. This is a user-defined value entered in the External Transaction Type field in the Transactions form. Sample values include PO, Invoice, and Shipping. For more information see Define Transactions, page 3-11.

- **TRANSACTION_SUBTYPE**

Used by XML Gateway to determine the transaction subtype of the document sent. This is a user-defined value entered in the External Transaction Subtype field in the Transactions form. Sample values include Create or Change. For more information see Define Transactions, page 3-11.

- **DOCUMENT_NUMBER**

This is the primary identifier of the business document in the payload. Examples include purchase order number and invoice number. This parameter is used in the Transaction Monitor as an identifier when checking the status of a document.

- **PARTYID**

The sender/receiver negotiated identifier that identifies the receiver of the document (company level). This is a user-defined value that is entered in the External Source Location Code of the Trading Partner form. For more information see: Define Trading Partners, page 3-19.

- **PARTY_SITE_ID**

The sender/receiver negotiated identifier that identifies the receiver of the document (company site level). The value of this parameter is the SOURCE_TP_LOCATION_CODE from the Trading Partner form. For more information see: Define Trading Partners, page 3-19.

- **PROTOCOL_TYPE**

This is the application protocol to transmit the document. It also contains an identifier for the program to use to transmit the document over the protocol. The following table lists the valid values for PROTOCOL_TYPE. For more information, see Protocol Type, page 4-5.

PROTOCOL_TYPE	Description
HTTP	Straight HTTP post
HTTP-ATCH	Straight HTTP post with attachment
HTTPS	Straight HTTP post using TLS
HTTPS-ATCH	Straight HTTP post with attachment, using TLS
HTTP-OXTA	OTA Protocol over HTTP (guaranteed delivery)
HTTPS-OXTA	OTA Protocol over HTTPS (guaranteed delivery)
OTAH-ATCH	OTA Protocol over HTTP (guaranteed delivery) with attachment

PROTOCOL_TYPE	Description
OTAHS-ATCH	OTA Protocol over HTTPS (guaranteed delivery) with attachment
SMTP	Send document via SMTP (e-mail)

- **PROTOCOL_ADDRESS**

This is the fully-qualified address to which to transmit the document. The following table lists sample protocol addresses for the protocol types shown:

PROTOCOL_TYPE	Sample PROTOCOL_ADDRESS
HTTP	http://www.example.com:8080/servlets/mycustom
HTTP-OXTA	http://www.example.com/webservices/TransportAgentServer
HTTPS-OXTA	http://www.example.com/webservices/TransportAgentServer
SMTP	Me@example.com

- **USERNAME/PASSWORD**

This is the username/password to authenticate on the receiving server. The OTA server uses this username and validates it against valid Oracle E-Business Suite users or valid Exchange users (buyer/supplier login). The username/password values are also in the authorization section of the HTTP header for optional Web server-level authentication.

- **ATTRIBUTE1**

This contains the identifier of the system sending the message. This is a user-defined value entered in the ECX_OAG_LOGICALID profile option. See Define System Profile Options, page 3-3.

- **ATTRIBUTE2**

Not used.

- **ATTRIBUTE3**

This contains the identifier of the final destination for the document, when dynamic routing is implemented. This is a user-defined value entered in the Target Location Code field in the Trading Partner definition form. It is a routing mechanism to notify the receiver to route the document to another trading partner known to the receiver.

For more information about dynamic routing, see: Static and Dynamic Routing, page 3-31.

- **ATTRIBUTE4**

Not used.

- **ATTRIBUTE5**

Not used.

- **PAYLOAD**

Contains the XML document to be processed.

Oracle Transport Agent Response Message

The OTA server uses standard HTTP response codes to determine if the HTTP post (at the HTTP protocol level) was successful. If successful, OTA will examine the HTTP header to determine if the OTA message was successfully delivered. If the HTTP response does not equal 200, OTA assumes the post failed and will requeue the message for retry (until maximum retry has been reached).

The following is an example of an HTTP response:

HTTP Header

```
HTTP/1.1 200 OK
Server: Apache/1.2.0
Date: Fri, 15 Jun 2002 16:20:46 GMT
Content-Length: 567
STATUS_CODE: 1000
STATUS_DESCRIPTION: OK
MESSAGE_RECEIPT_ID: A9876543210987654321
Content-type: text/html
```

HTTP Body

```
<HTML><BODY> <TABLE>  <TR><TD>Status Code</TD><TD>1000</TD></TR>
<TR><TD>Status Description</TD><TD>Message Successfully
received</TD></TR>  <TR><TD>Message Receipt
ID</TD><TD>A9876543210987654321</TD> </TR> </TABLE> </BODY></HTML>
```

The HTTP Response body is created for information only and is not read by the sending OTA server. The sending OTA server uses the STATUS_CODE in the HTTP Response header to determine the success or failure.

If the Status Code value is 1000 (meaning success), the MESSAGE_RECEIPT_ID in the HTTP Response body contains a unique identifier generated by the receiving OTA server. This completes the cycle for the guaranteed delivery process.

If the Status Code is anything other than 1000, a failure has occurred. The

STATUS_DESCRIPTION field will contain a brief description of the error. Refer to HTTP Status Codes, page 7-35 for a list of status codes.

OTA and Attachments

For outbound documents, the OTA client uses information in the Message Map to fetch and bundle the attachment in its outbound package.

For inbound documents, OTA extracts the associated attachments from the inbound MIME message and deposits them into the FND module of the receiving instance.

Note: Oracle Transport Agent also supports MIME messages without attachments. Similarity to the message process with attachments, OTA uses the same approach to process the message, but without further processing attachments if they do not exist.

- For outbound documents, the OTA client constructs the XML message with a valid MIME message containing the message payload and places it in its outbound package.
- For inbound documents, OTA extracts the body of the message and then passes the message to the receiving instance.

For more information about how OTA handles attachments, see Attachments and Oracle Transport Agent (OTA), page 2-117.

Certificate-Based Authentication Methods

OTA operates according to the security layer implemented at your site. It supports server authentication as well as mutual authentication methods.

- **Server Authentication**

When a client connects to a Web server securely via HTTPS, the server sends back its server certificate to the client for verification. Once verified, the client sends the data, encrypted, to the server. Server authentication allows the client to identify the server.

See OTA Message Propagation Flow, page 7-2.

- **Mutual Authentication**

In mutual authentication, the server and the client authenticate each other. The server authenticates itself to the client by presenting server certificate. The server identifies the client by requesting its client certificate. A client certificate is a digital certificate that certifies the identity of the client. You may choose to implement mutual authentication. It is not mandatory. Mutual authentication is useful when you want to restrict access to Oracle XML Gateway transactions to trusted clients

only.

OTA enables client certificate authentication through a set of Java programs.

The TLS handshake begins when an TLS client connects to an TLS server. The TLS handshake protocol is used to authenticate the TLS server to a client and the client to the server. It also contains agreed upon encryption algorithm and keys.

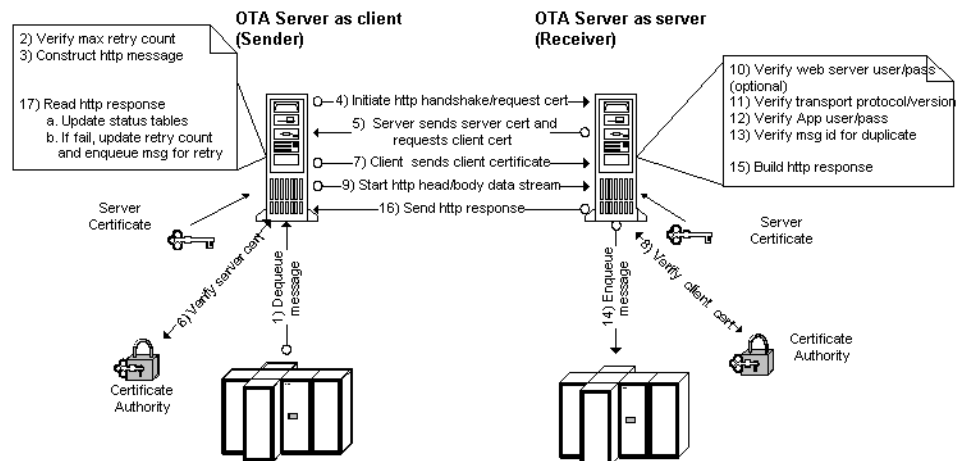
The authentication process consists of the following steps:

1. The client opens a connection and sends the ClientHello. This message contains the highest TLS version understood by the client, the session ID, and the list of compression methods supported.
2. The server responds. When the TLS server receives the ClientHello, it responds with either a handshake failure alert or a ServerHello message (that is, the TLS version used by the client, the session ID, and the compression method chosen by the server for this session).
3. The server sends the certificate along with the CA (who signed the certificate) certificate chain.
4. The client verifies the server certificate with the CA certificate in the TLS Certificate Chain File (list of CAs that the Client trusts).
5. The server sends a certificate request to the client.
6. The client sends its certificate along with the CA (who signed the certificate) certificate chain. If no certificate is available, the client sends a "no certificate" alert. It is up to the TLS server to decide what to do if a no certificate alert is received. The TLS server could continue the TLS transaction with an anonymous client or could terminate the connection by sending a data handshake failure alert.
7. The client sends a ClientKeyExchange. The client may send a key exchange message depending on the particular public key algorithm.
8. The server verifies the client certificate with the CA.
9. The client starts the HTTP message head and body data stream.
10. The server verifies the Oracle E-Business Suite user name and password.
11. The server verifies that the message ID is not a duplicate.
12. The server enqueues the message for consumption.
13. The server builds the HTTP response message.

14. The server sends the HTTP response back to the client.
15. The client reads the response and updates the status tables. If the response indicates the message failed, the retry count is updated and the message is enqueued for retry.

The following diagram illustrates the OTA message propagation flow for mutual authentication:

Mutual Authentication



The difference between this flow and the previous flow is the addition of the following two steps after the verification of the server certificate and before the start of the head/body stream by the client:

- After the client verifies the server certificate, it sends a client certificate.
- The client certificate is then verified by the server. If verified, the http head/body data stream is then started by the client.

Both the client and server certificates are issued by a Certification Authority (CA). Oracle E-Business Suite users can choose to support any CA they wish. Examples of CAs are Identris and Verisign.

Configuring XML Gateway for TLS Authentication

This section provides XML Gateway specific configurations for TLS authentication methods. The following topics are included in this section:

- Configuring TLS Authentication for Inbound Transactions, page 7-13
- Configuring TLS Authentication for Outbound Transactions, page 7-14

For information on configuring OTA client for TLS authentication using Oracle Wallet, see *Configuring OTA Client for TLS Authentication Using Oracle Wallet*, page G-1.

Configuring TLS Authentication for Inbound Transactions

In this document, OTA server refers to Oracle E-Business Suite with Oracle Transport Agent, which receives the message; client refers to the application that sends the message. That is, server receives inbound XML transactions, whereas client sends outbound XML transactions.

Configuring OTA Server for Server Authentication

Configuring TLS in Oracle E-Business Suite ensures that server authentication is enabled for Oracle XML Gateway inbound transactions through OTA.

For information on configuration details, refer to Section 5.2 Configure Inbound Connections, as described in My Oracle Support Knowledge Document 1367293.1, *Enabling TLS in Oracle E-Business Suite Release 12.2*.

Configuring OTA Server for Mutual Authentication (Optional)

If you want to restrict access to OTA server to only trusted clients, then you may opt for mutual authentication. Perform the following steps to enable mutual authentication on the OTA server:

1. Configure TLS in Oracle E-Business Suite, if this is not performed.

For information on configuration details, refer to Section 5.2 Configure Inbound Connections, as described in My Oracle Support Knowledge Document 1367293.1, *Enabling TLS in Oracle E-Business Suite Release 12.2*.

2. Import the trusted CA into the wallet.

Import the trusted root-CA certificate that signed your client certificate.

Refer to Step 5 in Section 5.2 Configure Inbound Connections, My Oracle Support Knowledge Document 1367293.1, *Enabling TLS in Oracle E-Business Suite Release 12.2*.

3. Enable client authentication by using the following steps:

1. Log in to Oracle Enterprise Manager Fusion Middleware Control Console (for example, `http://<hostname>:<domain>:<AdminServer Port>/em`).
2. Select "Web Tier Target" under the EBS Domain.
3. Select **Administration** and then **Advanced Configuration**.
4. Select the `ssl.conf` file for edit.
5. Copy and paste the following directives. (Ensure that `SSLVerifyClient` is set to

"require" (without quotes) only within the Location directive).

SSLVerifyClient optional

SSLWallet {s_web_ssl_directory}/Apache/cwallet.sso

<Location /webservices/TransportAgentServer>

SSLVerifyClient require

</Location>

The default location for the wallet is {s_web_ssl_directory}/Apache. Refer to the Application Context file for the exact location of the {s_web_ssl_directory} variable.

6. Click **Apply**.
7. Restart the Oracle HTTP Server instance by navigating to **Oracle HTTP Server**, then **Control**, and then **Restart**.
`$FMW_HOME/webtier/instances/<EBS-OHS-Instance>/bin/opmnctl startproc process-type=OHS`

Configuring TLS Authentication for Outbound Transactions

In this document, OTA client refers to Oracle E-Business Suite with Oracle Transport Agent, which sends the message; server refers to the application that receives the message. That is, server receives inbound XML transactions, whereas client sends outbound XML transactions.

Oracle recommends using Java KeyStore and TrustStore as the repository to store digital certificates for outbound transactions over HTTPS from Oracle E-Business Suite. This section describes the required steps to configure OTA client using Java KeyStore and TrustStore. If you plan to use Oracle Wallets to store digital certificates, refer to Configuring OTA Client for TLS Authentication Using Oracle Wallet, page G-1.

Configuring OTA Client for Server Authentication

Perform the following steps to configure OTA to send messages to the TLS-enabled server that is configured with server authentication only:

1. Import the server's trusted CA into the Java TrustStore by performing the following steps:
 1. Navigate to the \$OA_JRE_TOP/lib/security directory.
 2. Backup the existing cacerts file.
 3. Copy the rootCA.crt file to this directory.
 4. Execute the following command to ensure that cacerts has the write

permissions:

```
$ chmod u+w cacerts
```

5. Add rootCA.crt and intermediate certificates (intCA.crt) to the cacerts file:

```
$ keytool -importcert -keystore cacerts -alias rootCA -file rootCA.crt -v
```

```
$ keytool -importcert -keystore cacerts -alias intCA -file intCA.crt -v
```

Enter the keystore password when prompted.

If either one of the certificates already exists in cacerts, keytool will warn you and allow you to cancel the import. Cancel the import.

6. When you have completed the modifications to the cacerts file, reset the permissions:

```
$ chmod u-w cacerts
```

2. Update the following TLS related AutoConfig variables:

Parameter Name	AutoConfig Variable	Change Value To
javax.net.ssl.trustStore	s_ssl_truststore	\$OA_JRE_TOP/lib/security/cacerts
javax.net.ssl.trustStoreType	s_ssl_truststoretype	JKS
test.trustmanager.algorithm	s_ssl_trustmanageralgorithm	SunX509 If IBM AIX JRE is used, set the value of test.trustmanager.algorithm to "IbmX509" instead.

Note: The parameter names listed in the table are in the \$INST_TOP/appl/admin/oafm_wls.properties file.

3. Run AutoConfig in the application tier.

Refer to the *Oracle E-Business Suite Setup Guide Release 12.2* for information on changing AutoConfig variables and executing AutoConfig in the application tier.

As part of executing AutoConfig, you would be asked to restart the application tier services. Before you restart, ensure to perform step 4 mentioned below.

4. Manually add the following property to the `oafm_wls.properties` file:
`javax.net.ssl.trustStorePassword=<password to access cacerts>`
 This parameter is not configured through AutoConfig. Hence, you may have to set it manually every time you run AutoConfig.
5. Stop and restart the application tier services as part of the AutoConfig execution.

Configuring OTA Client for Mutual Authentication

Perform the following steps to configure OTA to send messages to the TLS-enabled server, that is configured with mutual authentication:

1. Follow steps described in Configuring OTA Client for Server Authentication, page 7-14 to import the server's trusted CA into the Java TrustStore.
2. If Oracle E-Business Suite is not already TLS-enabled, perform the following steps:
 1. Create Java Keystore by performing the following steps:
 1. Navigate to the web ssl directory as defined in the context file:
`$ grep s_web_ssl_directory $CONTEXT_FILE`
 2. Create a new directory with the name `j2ee`, and then change to this `j2ee` directory.
`$ mkdir j2ee`
`$ cd j2ee`
 3. Determine the values for the following parameters which will be used when you create the keystore for your instance:

Parameter Name	Value
server	Name of the server where you are creating the keystore
domain	The fully qualified domain of the server
password	In Release 12, the default password is <i>changeit</i> . Record the new password after you change it in step 5.
O	Name of your organization
L	Your city or locality

Parameter Name	Value
ST	Your state or province
C	Your two-letter country code

4. Create your keystore by entering the following command:

```
$ keytool -genkeypair -keyalg "RSA" -sigalg SHA256withRSA -
keystore oxtakey.jks -alias client -keysize 2048 -dname
"CN=server.domain, OU=JKS, O=0, ST=ST, C=C"
```

Enter the key password and keystore password when prompted.

You should now see the file `oxtakey.jks` in your directory.

2. Create your certificate request by performing the following steps:

1. Execute the following command to generate a certificate request signed by CA for the keypair:

```
$ keytool -certreq -keystore oxtakey.jks -alias client -sigalg
SHA256withRSA -file client.csr
```

2. Submit the `client.csr` file to your Certificate Authority.

3. When you receive your signed certificate, such as `client.crt`, copy it to the `<s_web_ssl_directory>/j2ee` directory.

4. Verify `client.crt` is PEM (base64) encoded certificate.

Please note that keytool does not like extra data to be present in the PEM file. If the file you received contains anything but the BEGIN and END lines with base64-encoded data between them, delete the extra data before importing the PEM file with keytool.

5. Use the following command to verify the textual and technical attributes of the certificate:

```
$ keytool -printcert -file client.crt
```

3. Execute the following command to import CA-signed certificate to Java Keystore:

```
$ keytool -import -keystore oxtakey.jks -alias client -
trustcacerts -file client.crt
```

Note: If CA's root certificate and intermediate certificates are

not in the `cacerts` file, ensure to import CA's root certificate (`rootCA1.crt`) and intermediate certificates (`intCA1.crt`) in the following order of sequence to Java Keystore, before importing CA-signed certificate:

```
$ keytool -importcert -keystore oxtakey.jks -alias rootCA1 -trustcacerts -file rootCA1.crt
```

```
$ keytool -importcert -keystore oxtakey.jks -alias intCA1 -trustcacerts -file intCA1.crt
```

4. Update the following TLS related parameters:

Parameter Name	AutoConfig Variable	Value
<code>javax.net.ssl.trustStore</code>	<code>s_ssl_truststore</code>	<code>\$OA_JRE_TOP/lib/security/cacerts</code>
<code>javax.net.ssl.trustStoreType</code>	<code>s_ssl_truststoretype</code>	<code>JKS</code>
<code>test.trustmanager.algorithm</code>	<code>s_ssl_trustmanageralgorithm</code>	<code>SunX509</code> If IBM AIX JRE is used, set the value of <code>test.trustmanager.algorithm</code> to "IbmX509" instead.
<code>javax.net.ssl.keyStore</code>	<code>s_ssl_keystore</code>	<code><s_web_ssl_directory>/j2ee/oxtakey.jks</code>
<code>javax.net.ssl.keyStoreType</code>	<code>s_ssl_keystoretype</code>	<code>JKS</code>
<code>test.keymanager.algorithm</code>	<code>s_ssl_keymanageralgorithm</code>	<code>SunX509</code> If IBM AIX JRE is used, set the value of <code>test.keymanager.algorithm</code> to "IbmX509" instead.

Note: The parameter names listed in the table are in the `$INST_TOP/appl/admin/oafm_wls.properties` file.

5. Run AutoConfig in the application tier.

Refer to the *Oracle E-Business Suite Setup Guide Release 12.2* for information on changing AutoConfig variables and executing AutoConfig in the application tier.

As part of executing AutoConfig, you would be asked to restart the application tier services. Before you restart, ensure to perform step 6 mentioned below.

6. Manually add the following property to the `oafm_wls.properties` file:
 - `javax.net.ssl.trustStorePassword=<password to access cacerts>`
 - `javax.net.ssl.keyStorePassword=<password to access keystore>`

These parameters are not configured through AutoConfig. Hence, you may have to set them manually every time you run AutoConfig.

7. Stop and restart the application tier services as part of the AutoConfig execution.
3. If Oracle E-Business Suite is already configured for TLS, the AutoConfig variables for SSL Keystore and Truststore would be configured by this step.

Setup Parameters

In Release 12.2, OTA runs in the `oafm` managed server and the system properties are read from the `oafm_wls.properties` file of the `oafm` managed server which is `$INST_TOP/appl/admin/oafm_wls.properties`.

Parameters Set Through AutoConfig

The following parameter is set through AutoConfig:

`applSysSchema=%s_applsys_user% : APPLSYS`

- `applSysSchema`: OTAPparameterName
- `s_applsys_user`: autoconfig-variable-name
- `APPLSYS`: default value of the autoconfig variable

This parameter indicates that the queues in `applsys` schema are used.

Proxy Parameters

The following parameters control whether a proxy is used for outgoing socket connections:

`OXTAOutUseProxy=%s_oxta_proxy% : false`

OXTAOutProxyHost=%s_oxta_proxyhost% : ""

OXTAOutProxyPort=%s_oxta_proxyport% : ""

TLS Parameters

In Release 12.2, OTA uses SSO wallet and uses JSSE APIs to operate the wallet with the following TLS parameters contained in the `oafm_wls.properties` file:

Note: There are two sets of variables; one is used for TrustMaterial and the other one is for KeyMaterial. The AutoConfig by default sets them to the same sets of values since both KeyMaterial and TrustMaterial are present in the default cwallet.

TLS Parameters

Parameter Name	AutoConfig Variable	Description	Default Specified by AutoConfig
javax.net.ssl.trustStore	s_ssl_truststore	Indicates the location of the trust material for the TrustManager.	\$INST_TOP/certs/Apache/cwallet.sso
javax.net.ssl.trustStoreType	s_ssl_truststoretype	Specifies the trustStore file type of trust material for the TrustManager.	SSO
test.trustmanager.algorithm	s_ssl_trustmanageralgorithm	Specifies the TrustManager instance corresponding to this algorithm.	OracleX509
javax.net.ssl.keyStore	s_ssl_keystore	Indicates the location of the key material for the KeyManager.	\$INST_TOP/certs/Apache/cwallet.sso
javax.net.ssl.keyStoreType	s_ssl_keystoretype	Specifies the keyStore file type of key material for the KeyManager.	SSO

Parameter Name	AutoConfig Variable	Description	Default Specified by AutoConfig
test.keymanager.algorithm	s_ssl_keymanageralgorithm	Specifies the KeyManager instance corresponding to this algorithm.	OracleX509

The default values for the parameters listed in the table may change, depending on the type of the truststore you use. For more information on these parameter values, see *Configuring TLS Authentication for Outbound Transactions*, page 7-14.

Parameters Controlling Throughout

These parameters set the number of database connections in your system for inbound and outbound requests. The optimum settings for these depend on the load in your system. Setting them too high could result in performance degradation. However, setting them too low could result in transactions having to wait for an excessive time before they are able to get a connection in or out. Both parameters default to 1.

OXTAInPoolSize=%s_oxtainpool_size% : 1

This parameter sets the number of database connections available for inbound requests.

OXTAOutThreads=%s_outbound_threads : 1%

This parameter sets the number of database connections available for outbound requests.

Load OXTA Servlet (s_load_oxta_servlet) Parameter

This parameter `s_load_oxta_servlet` is used to control loading of the Oracle Transport Agent servlets (both inbound and outbound) when managed server starts up.

- This parameter `s_load_oxta_servlet` is defaulted to '1' which will load both OTA inbound servlet (TransportAgentServer) and outbound servlet (TransportAgentMain) when managed server starts up.
- If this parameter `s_load_oxta_servlet` is set to '-2', then both OTA inbound and outbound servlets will not be loaded by default when managed server starts up.

In this case, OTA inbound servlet will be loaded and started automatically when it receives the first request, whereas the outbound servlet will never get loaded as it is not supposed to receive any requests (it only sends). To start outbound servlet, this parameter should be exclusively set to '1'.

Parameters Not Set Through AutoConfig

The following parameters must be manually added to the `oafm_wls.properties` file if you wish to change the default settings. The default settings are shown with each

parameter.

Time Parameters

The following parameters control the amount of time allowed for transactions to complete over OTA:

OXTAOutBaseTimeout=10

The time in seconds allowed by OTA to complete the entire process flow for an outbound request. That is, the time it takes to:

1. Open a connection.
2. Send data.
3. Receive a response.

Note that this base time is adjusted for higher payloads based on the OutLinearTimeout factor described below.

OXTAOutLinearTimeout=500

This parameter sets a factor that applies to the timeout based on the size of the payload.

The timeout for a given request is calculated based on the following formula:

$\text{Timeout} = \text{OXTABaseTimeOut} + (\text{OXTAOutLinearTimeout}/1000) * \text{payload size}/1024$

where the payload size is in bytes.

For example, for a payload of 1 MB, the timeout using the default parameter settings is:

$10 \text{ sec} + [(500/1000) * 1048576/1024] = 523 \text{ seconds}$

In other words, by default, the factor 500 adds 513 seconds for every MB of data in the payload.

OXTAOutMaxAttempts=5

The number of times to attempt to send an outbound request.

OXTAOutResendDelay=1800

The time in seconds to wait before retrying to send a transaction.

OXTAThreadSleepTime=60

The amount of time in seconds the primary thread sleeps between monitoring the Transport Handler threads.

OXTAMaxDbConnectAttempts=20

The maximum number of DB failures that can be tolerated before refraining from making any more connection attempts and going to long sleep for a time given by the parameter below.

OXTABackoffTime=3600

If the Transport Handler threads have died for OXTAMaxDbConnectAttempts number

of times due to DB failure, the primary thread refrains from further attempts and goes to sleep for a time period in seconds given by this parameter.

OXTAOutDequeueRate=120000

The time in milliseconds for an OTA client to dequeue the message from the ECX_OUTBOUND queue. To reduce the dequeue time for each transaction, change the default value 120000 to a smaller value, such as 30000 (30 seconds).

Payload Size

OXTAInMaxContent=1000000

The maximum payload size in bytes.

Debug Setting

FND logging is used for OTA, and the AFLOG parameters should be used for enabling the logging. They can be either set in the FND profile options or can be set specifically to the oafm managed server in which the OTA is running. If the latter option is used, set the following parameters in the oafm_wls.properties file:

AFLOG_ENABLED=true

AFLOG_LEVEL=<1 for Statement level, 6 for Unexpected level. OTA logs only at either of these 2 levels>

AFLOG_MODULE=ecx.oxta%

AFLOG_FILENAME=<filename on application tier>

Since OTA uses Anonymous AppsLog instance, logging does not go to database if this parameter is omitted.

The old parameter OXTALogDebugMsg=false/true is used only in the standalone mode.

Oracle Transport Agent Inbound Test Page

Oracle Transport Agent provides Oracle Transport Agent Inbound Test Page to test inbound XML message processing.

Oracle Transport Agent Inbound Test Page presents an HTML page that lets you provide inbound payload and other OTA parameters, as explained in the Oracle Transport Agent Inbound Test Page Parameters table at the end of this section. To access this test page, open a web browser and enter the following URL:

http://<EBS hostname>:<port>/webservices/ECXOTAInbound

When prompted, provide Oracle E-Business Suite username and password. Then, provide the required input XML payload and other input parameters used by OTA messaging protocol. After authenticating the user credentials, the XML message will be posted to OTA for further processing. OTA messaging protocol parameters used in this test page are described in the Oracle Transport Agent Inbound Test Page Parameters

table.

Note: Oracle Transport Agent Inbound Test Page is included for testing purposes and should not be used in a production environment. For production, send inbound requests to `http(s)://<EBS hostname>:<port>/webservices/TransportAgentServer`.

Important: As part of the January 2017 CPU update, the access to the Oracle Transport Agent Inbound Test Page is restricted to trusted nodes or servers. Therefore, to use this page to send an inbound XML message, you should add a fully-qualified node name or IP address as a trusted node in Oracle E-Business Suite. To extend access to additional hosts, perform the following tasks:

1. Add space-separated node names, fully qualified node names, or client machine IP addresses to the existing content of the `s_admin_ui_access_nodes` context variable.
2. Run AutoConfig.
3. Restart the Oracle HTTP Server.

For more information, refer to the Known Issues section in My Oracle Support Knowledge Document 2212223.1, *Oracle E-Business Suite Release 12 Critical Patch Update Knowledge Document (January 2017)* for details.

This test page will prompt you for the input parameters used by the OTA messaging protocol.

Oracle Transport Agent Inbound Test Page Parameters

Column Name	Description
TRANSPORT_PROTOCOL	Defaults to OXTA.
TRANSPORT_PROTOCOL_VERSION	Defaults to 1.0.
REQUEST_TYPE	Select "Send".
MESSAGE_ID	Enter the unique reference number that identifies the transaction.

Column Name	Description
MESSAGE_TYPE	Defaults to XML.
MESSAGE_STANDARD	Defaults to OAG.
TRANSACTION_TYPE	(Required) Enter the type of transaction, such as PO for Purchase Order Inbound.
TRANSACTION_SUBTYPE	(Required) Enter the subtype of the transaction.
DOCUMENT_NUMBER	Enter the document number.
PARTYID	(Optional) Enter the trading partner ID for the sender.
SOURCE_TP_LOCATION_CODE	(required) Enter the Source Trading Partner Location Code, page 3-28 (should match the Trading Partner Details column of the same name).
PROTOCOL_TYPE	Not required for inbound messages.
PROTOCOL_ADDRESS	Required only if REQUEST_TYPE is EME.
USERNAME	(Required) Enter a valid user name for the receiving system.
PASSWORD	(Required) Enter the password for the user name entered.
ATTRIBUTE1	Optional
ATTRIBUTE2	Optional
ATTRIBUTE3	Required for pass-through transactions only.
ATTRIBUTE4	Optional
ATTRIBUTE5	Optional
PAYLOAD	Enter the message payload.

Connecting to Non-OTA Servers

The OTA server (client) includes the capability to send documents to non-OTA servlets that do not employ the OTA messaging protocol. When sending a message, the OTA server initiates an HTTP post to transmit the document. The HTTP response from the receiving Web server indicates whether the receiver was an OTA server.

If the HTTP response does not contain the OTA protocol response body, the sending OTA server assumes the message was received by a non-OTA server. In this case, the standard HTTP response code (that is, 200 meaning "success") is used to determine the success or failure of the message. If the sending OTA server receives an HTTP-200, the message delivery is assumed to be successful. If the sending OTA server receives any other response, it is assumed the delivery failed.

Successful transmissions of documents to non-OTA servers are logged as successfully delivered to a non-OTA server. Delivering to a non-OTA server has the following disadvantages:

- No guaranteed delivery
- No guaranteed once only delivery
- No message tracking available to sender
- Message ownership completely with the servlet that received the message

Please note that you can set up the OTA server to use HTTP 1.1 for sending documents to a specific non-OTA server by adding a system property with the following syntax:

`HttpProtocolVersion-<servername>\:<serverport>="1.X"`

For example, add a system property `HttpProtocolVersion-www.example.com\:
8080="1.1"`.

If there is no server port specified in the URL for the application, the port in the system property will have to be set to '-1'. For example, `HttpProtocolVersion-www.
example.com\:-1="1.1"`.

Code Connection Samples

The following are code examples for remote Web servers to post a message to the Oracle E-Business Suite using either the HTTP or HTTPS protocol. Use this as a guide to create your own code.

The code to post a message to the Oracle E-Business Suite consists of three parts: the message envelope, the message payload, and the message response.

The message envelope attributes are as follows:

- MESSAGE_TYPE

- MESSAGE_STANDARD
- TRANSACTION_TYPE
- TRANSACTION_SUBTYPE
- DOCUMENT_NUMBER
- PARTYID
- PARTY_SITE_ID
- PARTY_TYPE
- PROTOCOL_TYPE
- USERNAME
- PASSWORD

Define these as name-value pairs in your code. The USERNAME and PASSWORD should be set to something meaningful for the receiving Web server.

The PAYLOAD corresponds to your business document represented in XML format.

The receiving Web server sends the message response back to the sending Web server. The message response includes Message Receipt ID, Status Code, and Status Description. A status code of 1000 implies the message was successfully posted.

OTA Connection over HTTP

```
import java.sql.*;
import java.net.*;
import java.io.*;
import java.util.*;
public class SendHTTP
{
    final String[] columnNames = new String[] { "MESSAGE_TYPE",
"MESSAGE_STANDARD", "TRANSACTION_TYPE", "TRANSACTION_SUBTYPE", "DOCUMENT_NUMBER", "PARTYID",
"PARTY_SITE_ID", "PARTY_TYPE",
"PROTOCOL_TYPE", "PROTOCOL_ADDRESS", "USERNAME", "PASSWORD",
"ATTRIBUTE1", "ATTRIBUTE2", "ATTRIBUTE3",
"ATTRIBUTE4", "ATTRIBUTE5", "PAYLOAD" };
    Object[] columnVals = new Object[18];
    int numCols = 0;
    String Encoding = "UTF-8";
    URL url = null;
    String username = null;
    String password = null;
    String target = null;
    String filename = null;
    String MessageID = "SEND" + System.currentTimeMillis();
    String VALUE_SEP = "=";
    String PAIR_SEP = "&";
    String NEW_LINE = "\n";
    public SendHTTP()
    {
        numCols = columnNames.length;
        columnVals = new Object[numCols];
    }
    public void createBasicMessage() throws Exception
    {
        logMessage("Creating the message");
        StringBuffer tmp = new StringBuffer("This is a test message sent
from the Sample java program.");
        columnVals[0] = "XML"; //MESSAGE_TYPE
        columnVals[1] = "OAG"; //MESSAGE_STANDARD
        columnVals[2] = "ECX"; //TRANSACTION_TYPE
        columnVals[3] = "CBODO"; //TRANSACTION_SUBTYPE
        columnVals[4] = "12"; //DOCUMENT_NUMBER
        columnVals[5] = "206"; //PARTYID
        columnVals[6] = "206"; //PARTY_SITE_ID
        columnVals[7] = "I"; //PARTY_TYPE
        columnVals[8] = "HTTP"; //PROTOCOL_TYPE
        columnVals[9] = target;
        columnVals[10] = username ; //USERNAME
        columnVals[11] = password; //PASSWORD
        columnVals[12] = null ; //ATTRIBUTE1
        columnVals[13] = null ; //ATTRIBUTE2
        columnVals[14] = null ; //ATTRIBUTE3
        columnVals[15] = null ; //ATTRIBUTE4
        columnVals[16] = null ; //ATTRIBUTE5
        columnVals[17] = getPayload();// tmp ;//Payload
    }
    public void sendMessage() throws Exception
    {
        int bytesRead;
        int total_bytes = 0;
        int bufSize = 1024;
        char[] buf = new char[bufSize];
        logMessage("Started Sending the message.");
        url = new URL((String)columnVals[9]);
```

```

int port = url.getPort();
    if(port == -1)
        port = 80;
    URL newurl = new URL(url.getProtocol(),url.getHost(),port, url.
getFile());
    logMessage("Connecting to " + newurl.getHost() + " port " + port);
    HttpURLConnection conn = (HttpURLConnection)newurl.
openConnection();
    conn.setDoOutput(true);
    conn.setDoInput(true);
    OutputStreamWriter wr = new OutputStreamWriter(conn.
getOutputStream());
    String httpReq = getRequestStr();
    logMessage("Sending to URL.");
    wr.write(httpReq);
    wr.flush();
    logMessage("Response Received :-");
    logMessage("Message Receipt ID = "+conn.getHeaderField
("MESSAGE_RECEIPT_ID"));
    logMessage("Status Code          = "+conn.getHeaderField
("STATUS_CODE"));
    logMessage("Status Description = "+conn.getHeaderField
("STATUS_DESCRIPTION"));
}
public String getRequestStr() throws Exception
{
    StringBuffer buf = new StringBuffer();
    StringBuffer httpBody = getHttpBody();
    long bodyLen = 0;
    bodyLen = httpBody.length();
    buf.append(httpBody);
    buf.append("\n");
    return buf.toString();
}
public StringBuffer getHttpHeader(long contentLength) throws Exception
{
    StringBuffer buf = new StringBuffer();
    buf.append("POST " + url.getFile() + " HTTP/1.0\r\n");
    buf.append("Host: " + url.getHost() + ":" + url.getPort() + "\n");
    String userpass = username+ ":" +password;
    buf.append("Content-type: application/x-www-form-urlencoded\n");
    buf.append("Content-length: " + contentLength + "\n");
    buf.append("\r\n");
    return buf;
}
public StringBuffer getHttpBody() throws Exception
{
    StringBuffer contentBuf = new StringBuffer();
    contentBuf.append(encode("TRANSPORT_PROTOCOL"));
    contentBuf.append(VALUE_SEP);
    contentBuf.append(encode("OXTA"));
    contentBuf.append(PAIR_SEP);
    contentBuf.append(encode("TRANSPORT_PROTOCOL_VERSION"));
    contentBuf.append(VALUE_SEP);
    contentBuf.append(encode("1.0"));
    contentBuf.append(PAIR_SEP);
    contentBuf.append(encode("REQUEST_TYPE"));
    contentBuf.append(VALUE_SEP);
    contentBuf.append(encode("SEND"));
    contentBuf.append(PAIR_SEP);
    contentBuf.append(encode("MESSAGE_ID"));
    contentBuf.append(VALUE_SEP);
    contentBuf.append(encode(MessageID));
    contentBuf.append(PAIR_SEP);
    for (int i = 0; i < 18; i++)
    {

```

```

        if(columnVals[i] != null)
        {
            contentBuf.append(encode(columnNames[i]));
            contentBuf.append(VALUE_SEP);
            if(columnVals[i] != null)
                contentBuf.append(encode((String)columnVals[i]));
            else
                contentBuf.append("");
            if(i != 17 )
                contentBuf.append(PAIR_SEP);
        }
    }
    return contentBuf;
}
public void logMessage(String msg)
{
    System.out.println(msg);
}
public String encode(String str)
{
    String str1 = null;
    try{
        str1 = URLEncoder.encode(str,Encoding);
    }catch(Exception e)
    {
        System.out.println("Unsupported Encoding format");
        System.exit(1);
    }
    return str1;
}
public String getPayload()
{
    StringBuffer tmp = null;
    try
    {
        BufferedReader in = new BufferedReader(new FileReader(filename));
        String s = null;
        tmp = new StringBuffer();
        while((s = in.readLine())!= null)
        {
            tmp.append(s + "\n");
        }
        in.close();
    }
    catch(Exception ex)
    {
        logMessage("Exception: in reading file");
        logMessage("Sending a string : 'This is a Test String' as
payload");
    }
    if(tmp == null || tmp.length() == 0)
        return "This is a Test String";
    else
        return tmp.toString();
}
public static void main(String args[])
{
    //USAGE Target username password;
    SendHTTP sp = new SendHTTP();
    try
    {
        if(args.length < 4)
        {
            sp.logMessage("Usage java sample <Target> <username> <password>
<filename>");
            System.exit(1);
        }
    }
}

```



```

    }
    else
    {
        sp.target = args[0];
        sp.username = args[1];
        sp.password = args[2];
        sp.filename = args[3];
        sp.createBasicMessage();
        sp.sendMessage();
        sp.logMessage("Program exiting ");
    }
}
catch(Exception e)
{
    sp.logMessage("Exception = "+e);
}
}
}

```

OTA Connection over HTTPS

```
import java.sql.*;
import java.net.*;
import java.io.*;
import java.util.*;
import javax.net.ssl.*;

public class SendHTTPS
{
    final String[] columnNames = new String[] { "MESSAGE_TYPE",
"MESSAGE_STANDARD",
    "TRANSACTION_TYPE", "TRANSACTION_SUBTYPE", "DOCUMENT_NUMBER",
"PARTYID", "PARTY_SITE_ID", "PARTY_TYPE",
    "PROTOCOL_TYPE", "PROTOCOL_ADDRESS", "USERNAME", "PASSWORD",
"ATTRIBUTE1", "ATTRIBUTE2", "ATTRIBUTE3",
    "ATTRIBUTE4", "ATTRIBUTE5", "PAYLOAD" };
    Object[] columnVals = new Object[18];
    int numCols = 0;
    String Encoding = "UTF-8";
    URL url = null;
    String username = null;
    String password = null;
    String target = null;
    String filename = null;
    String MessageID = "SEND" + System.currentTimeMillis();
    String VALUE_SEP = "=";
    String PAIR_SEP = "&";
    String NEW_LINE = "\n";

    public SendHTTPS()
    {
        numCols = columnNames.length;
        columnVals = new Object[numCols];
    }

    public void createBasicMessage() throws Exception
    {
        logMessage("Creating the message");
        StringBuffer tmp = new StringBuffer("This is a test message sent
from the Sample java program.");
        columnVals[0] = "XML"; //MESSAGE_TYPE
        columnVals[1] = "OAG"; //MESSAGE_STANDARD
        columnVals[2] = "ECX"; //TRANSACTION_TYPE
        columnVals[3] = "CBOD0"; //TRANSACTION_SUBTYPE
        columnVals[4] = "12"; //DOCUMENT_NUMBER
        columnVals[5] = "206"; //PARTYID
        columnVals[6] = "206"; //PARTY_SITE_ID
        columnVals[7] = "I"; //PARTY_TYPE
        columnVals[8] = "HTTPS"; //PROTOCOL_TYPE
        columnVals[9] = target;
        columnVals[10] = username ; //USERNAME
        columnVals[11] = password; //PASSWORD
        columnVals[12] = null ; //ATTRIBUTE1
        columnVals[13] = null ; //ATTRIBUTE2
        columnVals[14] = null ; //ATTRIBUTE3
        columnVals[15] = null ; //ATTRIBUTE4
        columnVals[16] = null ; //ATTRIBUTE5
        columnVals[17] = getPayload();// tmp ;//Payload
    }

    public void sendMessage() throws Exception
    {
        int bytesRead;
        int total_bytes = 0;
        int bufSize = 1024;
        char[] buf = new char[bufSize];
        String proxyHost = "www-proxy.us.example.com";
```

```

String proxyPort = "80";
Properties systemProperties = System.getProperties();
systemProperties.setProperty("https.proxyHost", proxyHost);
systemProperties.setProperty("https.proxyPort", proxyPort);
    logMessage("Started Sending the message.");
    url = new URL((String)columnVals[9]);
    int port = url.getPort();
    if(port == -1)
        port = 80;
    URL newurl = new URL(url.getProtocol(),url.getHost(),port, url.
getFile());
    logMessage("Connecting to " + newurl.getHost() + " port " + port);
    HttpsURLConnection conn = (HttpsURLConnection)newurl.
openConnection();
    conn.setDoOutput(true);
    conn.setDoInput(true);
    OutputStreamWriter wr = new OutputStreamWriter(conn.
getOutputStream());
    String httpReq = getRequestStr();
    logMessage("Sending to URL.");
    wr.write(httpReq);
    wr.flush();
    logMessage("Response Received :-");
    logMessage("Message Receipt ID = "+conn.getHeaderField
("MESSAGE_RECEIPT_ID"));
    logMessage("Status Code = "+conn.getHeaderField
("STATUS_CODE"));
    logMessage("Status Description = "+conn.getHeaderField
("STATUS_DESCRIPTION"));
}
public String getRequestStr() throws Exception
{
    StringBuffer buf = new StringBuffer();
    StringBuffer httpBody = getHttpBody();
    long bodyLen = 0;
    bodyLen = httpBody.length();
    buf.append(httpBody);
    buf.append("\n");
    return buf.toString();
}
public StringBuffer getHTTPHeader(long contentLength) throws Exception
{
    StringBuffer buf = new StringBuffer();
    buf.append("POST " + url.getFile() + " HTTP/1.0\r\n");
    buf.append("Host: " + url.getHost() + ":" + url.getPort() + "\n");
    String userpass = username+ ":" +password;
    buf.append("Content-type: application/x-www-form-urlencoded\n");
    buf.append("Content-length: " + contentLength + "\n");
    buf.append("\r\n");
    return buf;
}
public StringBuffer getHttpBody() throws Exception
{
    StringBuffer contentBuf = new StringBuffer();
    contentBuf.append(encode("TRANSPORT_PROTOCOL"));
    contentBuf.append(VALUE_SEP);
    contentBuf.append(encode("OXTA"));
    contentBuf.append(PAIR_SEP);
    contentBuf.append(encode("TRANSPORT_PROTOCOL_VERSION"));
    contentBuf.append(VALUE_SEP);
    contentBuf.append(encode("1.0"));
    contentBuf.append(PAIR_SEP);
    contentBuf.append(encode("REQUEST_TYPE"));
    contentBuf.append(VALUE_SEP);
    contentBuf.append(encode("SEND"));
    contentBuf.append(PAIR_SEP);

```

```

contentBuf.append(encode("MESSAGE_ID"));
contentBuf.append(VALUE_SEP);
contentBuf.append(encode(MessageID));
contentBuf.append(PAIR_SEP);
    for (int i = 0; i < 18; i++)
    {
        if(columnVals[i] != null)
        {
            contentBuf.append(encode(columnNames[i]));
            contentBuf.append(VALUE_SEP);
            if(columnVals[i] != null)
                contentBuf.append(encode((String)columnVals[i]));
            else
                contentBuf.append("");
            if(i != 17 )
                contentBuf.append(PAIR_SEP);
        }
    }
    return contentBuf;
}
public void logMessage(String msg)
{
    System.out.println(msg);
}
public String encode(String str)
{
    String str1 = null;
    try{
        str1 = URLEncoder.encode(str,Encoding);
    }catch(Exception e)
    {
        System.out.println("Unsupported Encoding format");
        System.exit(1);
    }
    return str1;
}
public String getPayload()
{
    StringBuffer tmp = null;
    try
    {
        BufferedReader in = new BufferedReader(new FileReader(filename));
        String s = null;
        tmp = new StringBuffer();
        while((s = in.readLine())!= null)
        {
            tmp.append(s + "\n");
        }
        in.close();
    }
    catch(Exception ex)
    {
        logMessage("Exception: in reading file");
        logMessage("Sending a string : 'This is a Test String' as
payload");
    }
    if(tmp == null || tmp.length() == 0)
        return "This is a Test String";
    else
        return tmp.toString();
}
public static void main(String args[])
{
    //USAGE Target username password;
    SendHTTPS sp = new SendHTTPS();
    try

```

```

{
    if(args.length < 4)
    {
        sp.logMessage("Usage java sample <Target> <username> <password>
<filename>");
        System.exit(1);
    }
    else
    {
        sp.target = args[0];
        sp.username = args[1];
        sp.password = args[2];
        sp.filename = args[3];
        sp.createBasicMessage();
        sp.sendMessage();
        sp.logMessage("Program exiting ");
    }
}
catch(Exception e)
{
    sp.logMessage("Exception = "+e);
}
}

```

Troubleshooting

For information on TLS issues, see Common TLS Issues, page I-47.

HTTP Status Codes

The following table lists the error response codes returned in the HTTP Response by the OTA server.

Status Code	Error Code	Description
1000	OK	Request handled successfully.
2000	ECX_OXTA_DB_UNAVAIL	Database unavailable. Cannot get connection to the database.
2001	ECX_OXTA_SERVER_ERR	Unexpected server-side error. Client should retry.
3000	ECX_OXTA_BAD_REQ	Missing TRANSPORT_PROTOCOL, TRANSPORT_PROTOCOL_VERSION, or MESSAGE_TYPE.
3001	ECX_OXTA_UNKNOWN_REQ	Invalid value for MESSAGE_TYPE.

Status Code	Error Code	Description
3002	ECX_OXTA_AUTH_MISSVAL	Incomplete credentials. Username or Password not available for request.
3003	ECX_OXTA_AUTH_FAILURE	Authentication failure. Invalid user password.
3004	ECX_OXTA_PROTCL_NOTSUPP	Oracle Transport Agent protocol version not supported by server.
3005	ECX_OXTA_PAYLOAD_NULL	Message has NULL payload.
3100	ECX_OXTA_SEND_MISSVAL	Required parameters for the SEND post are missing.
3101	ECX_OXTA_LEN_MISS	Request header content length attribute not set.
3102	ECX_OXTA_LEN_TOOLARGE	Size of content larger than specified in request header content length attribute.
3200	ECX_OXTA_AUTH2_MISSVAL	Parameter missing for AUTH2 request. Verify that the party_site_id and the transaction_type are valid.
3201	ECX_OXTA_AUTH2_FAILURE	AUTH2 request failed.
3300	ECX_OXTA_EME_MISSVAL	E-mail address or payload was not passed in the EME request.
3301	ECX_OXTA_EME_INVALID_EMAIL	Incorrect e-mail address format for EME request.
3302	ECX_OXTA_EME_SMTP_NOTSET	Mail server not set up in config.
4000	ECX_OXTA_UNKNOWN_PROTCL	Invalid protocol type for SEND request.
4001	ECX_OXTA_TIMEOUT	Time out for this transport request reached.
4002	ECX_OXTA_CLIENT_ERR	Unexpected client side exception.

Status Code	Error Code	Description
4003	ECX_OXTA_MAX_ATTEMPTS	Exceeded the max number of attempts for transport.
4100	ECX_OXTA_SMTP_NOTSET	Mail server not set up in config.
4101	ECX_OXTA_INVALID_EMAIL	Incorrect e-mail address format.
4102	ECX_OXTA_MAIL_ERR	Error when trying to send mail.
4103	ECX_OXTA_MAILJAR_NOT_EXISTS	SMTP not enabled. Ensure that mail.jar is present in your Java classpath.
4104	ECX_OXTA_ACTJAR_NOT_EXISTS	SMTP not enabled. Ensure that activation.jar is present in your Java classpath.
4200	ECX_OXTA_INVALID_URL	Incorrect URL format.
4201	ECX_OXTA_PROXY_FAILURE	Cannot open connection to proxy server.
4202	ECX_OXTA_CONNECT_FAILURE	Cannot connect to host:port.
4203	ECX_OXTA_UNKNOWN_RES	Response from the server not in a format understood by the client.
4300	ECX_OXTA_INVALID_CACERT	Failed to open the certificate file/failed to read certificate from the file.
4301	ECX_OXTA_SSLHANDSHAKE_FAILURE	Failed to perform handshake when getting TLS connection.
4302	ECX_OXTA_SSLVERICHAIN_FAILURE	Error when verifying chain certificate.
5000	ECX_OXTA_PROTOCOL_MISS	Protocol value missing from inbound message.
5001	ECX_OXTA_USERNAME_MISS	Username value missing from the inbound message.

Status Code	Error Code	Description
5002	ECX_OXTA_PASSWORD_MISS	Password value missing from the inbound message.

Web Services

Overview

Web services are Web-based applications that provide a standard means of interoperating between different software applications, running on a variety of platforms or frameworks. Web services have an interface described in a machine-processable format called WSDL (Web Services Description Language). Other systems interact with the Web service in a manner prescribed by its description using SOAP (Simple Object Access Protocol) messages, typically conveyed using HTTP with an XML serialization in conjunction with other Web-related standards.

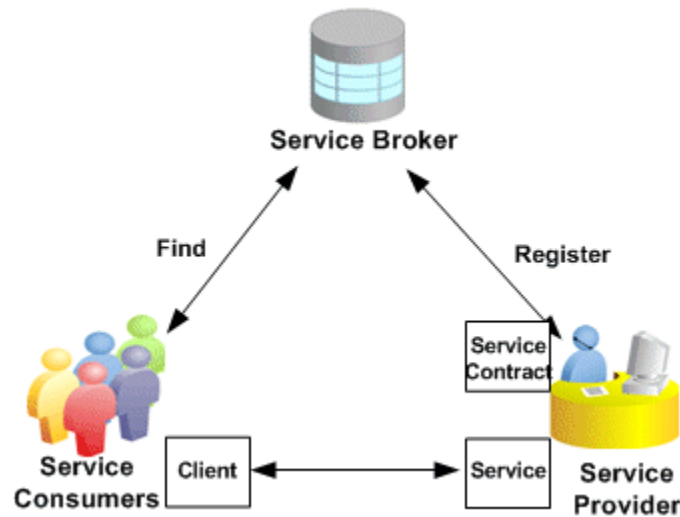
Service-Oriented Architecture for Enabling XML Gateway Web Services

By leveraging the concept of service-oriented architecture (SOA) in Oracle E-Business Suite, XML Gateway inbound messages can be service enabled.

Note: Outbound XML Gateway messages are first enqueued to the ECX_OUTBOUND queue. They are used through subscription model by first dequeuing the messages to retrieve outbound data and then invoking appropriate outbound XML Gateway maps to exchange messages with any trading partner that conforms to Oracle E-Business Suite Web service client. Hence, all the XML Gateway Web services discussed here are for inbound services.

The common service-oriented architecture used in enabling Web services is illustrated in the following diagram:

Service-Oriented Architecture



- Service Provider is the primary engine underlying the Web services. It acts as a bridge between Oracle E-Business Suite and Oracle SOA Suite to facilitate the service enablement for various types of Oracle E-Business Suite interfaces.

Note: In earlier Oracle E-Business Suite Releases, SOA Provider and Web Service Provider were used in enabling Oracle E-Business Suite Web services. In the Release 12.2, Service Provider is the engine for service enablement.

Service Provider is the engine that performs the actual service generation and deployment behind the scene.

- Service Consumer (Web service client) is the party that uses or consumes the services provided by the Service Provider.
- Service Broker (Service Registry) describes the service's location and contract to ensure service information is available to potential service consumers.

WSDL and SOAP are the communication vehicles facilitating the services. Part of the WSDL provided by a service provider is passed to the service consumer in specifying what the requests and responses are for the service provider. The service consumer sends SOAP messages as service requests to the service provider. The service provider then provides the expected SOAP responses back to the service consumer to complete the requests.

In Oracle XML Gateway, WSDL description is used to inform trading partners how to communicate with Oracle E-Business Suite. Inbound Web services must conform to this WSDL. Outbound message are first enqueued to ECX_OUTBOUND queue and the

Transport Agent dequeues the message and delivers it to any trading partner that conforms to Oracle E-Business Suite Web service client.

Any transaction supported by Oracle XML Gateway can be sent or received as a document style Web service using the Simple Object Access Protocol (SOAP).

Web Services Components and Features

Following are the components and features implemented for Web services in Oracle E-Business Suite.

Web Services Description Language (WSDL)

One of the major components for publishing a Web service is the WSDL. It enables your trading partner to call your Web service without any further details. Partners can use any third party Web service tools to call your service.

Typically, schema structure is defined as part of the WSDL. For strongly-typed XML Gateway services through Oracle E-Business Suite Integrated SOA Gateway's Service Provider, each service contains a complete definition of its input and output message content in XML schema included in the WSDL definition. XML Gateway knows the data structure based on the transaction type and the DTD defined in the map for the message enabled for the trading partner. For generic XML Gateway services through Web Service Provider, the schema structure type is defined as "anyType". This allows the Oracle E-Business Suite to accept XML documents containing any XML structure.

Note: Oracle recommends that you use strongly-typed XML Gateway Web services.

Oracle E-Business Suite publishes the WSDL to an URL for your customers to access.

Guaranteed Delivery

Delivery is guaranteed for messages exchanged between trading partners that both recognize unique message IDs. The Oracle E-Business Suite leverages the AQ message ID. This ID is guaranteed to be the same across any database system in the world.

The message IDs are maintained in a repository on the receiving side. Before receiving any message, it interrogates the repository for the ID. If the message has not been received, the Web service provider receives and commits. If the message has already been received, it ignores the message and sends a successful response.

The sending side reviews the response. If it is successful, it does not retry. If it is a failure, it retries based on two parameters: Number of attempts and time-to-live for a transaction. Once both are exhausted, the message is marked as undeliverable. If implemented, a sent to the Workflow.

Simple Object Access Protocol (SOAP)

SOAP is a protocol specification for exchanging structured information in the implementation of Web services in computer networks.

The current release of Oracle E-Business Suite supports document style Web services.

SOAP Servlet (Inbound)

The SOAP servlet receives inbound transactions. It is responsible for the following:

- Authentication

The system-level User ID and Password are validated against the Oracle E-Business Suite application user.

- Authorization

The trading partner information is validated against the trading partner tables.

- Message extraction

The message is enqueued.

Attachments

Attachments are supported for XML Gateway outbound messages.

Web Service Security

Security is the most critical feature that is designed to guard service content from unauthorized access. To ensure secure access and the execution of Web services, security can be enforced through subject authentication and authorization:

- To authenticate users on Oracle E-Business Suite Web service access, appropriate authentication types supported by Oracle E-Business Suite Integrated SOA Gateway must be selected from either one of the following mechanisms before service deployment.

To authenticate users who request Oracle E-Business Suite Web services, the SOAP requests must be authenticated through either one of the following authentication types supported by Oracle E-Business Suite Integrated SOA Gateway:

- Username Token: This authentication type provides username and password information in the security header for a Web service provider to use in authenticating the SOAP request.
- SAML Token (Sender Vouches): This authentication type is used for Web services relying on sending a username only through SAML Assertion.

The identified authentication information is embedded in the `wsse:security`

Web Security headers for service authentication.

- To authorize users on specific services or operations, the access permissions must be explicitly given to the users through security grants. Multiple organization access control (MOAC) security rule is also implemented for authorizing interface execution related to multiple organizations.

Additionally, for strongly-typed XML Gateway services through Oracle E-Business Suite Integrated SOA Gateway's Service Provider, all XML Gateway header parameters for inbound transactions are defined in `SOAHeader` part of a SOAP request. These parameters are used for XML Gateway inbound transactions to be completed successfully.

For generic XML Gateway services through Web Service Provider, all mandatory XML Gateway header parameters for inbound transactions are defined in `XMLGateway_Header` part of a SOAP request.

Understanding XML Gateway Web Services

Oracle XML Gateway provides a set of services that can be easily integrated with Oracle E-Business Suite to support XML messaging.

XML Gateway messages can be displayed through Oracle Integration Repository, an centralized repository that contains numerous interface endpoints exposed by applications within the Oracle E-Business Suite.

Based on the SOA architecture described earlier, XML messages resided on the Oracle Integration Repository (serves as a *service broker*) can be service enabled through a *service provider* and then in the form of WSDL description to inform trading partners how to communicate with Oracle E-Business Suite.

To better understand XML Gateway Web services, the following topics are discussed in this section:

- Enabling XML Gateway Services through Oracle E-Business Suite Integrated SOA Gateway, page 8-5
- Web Service Security, page 8-7
- Strongly-Typed XML Gateway Services, page 8-11
- Generic XML Gateway Services, page 8-15

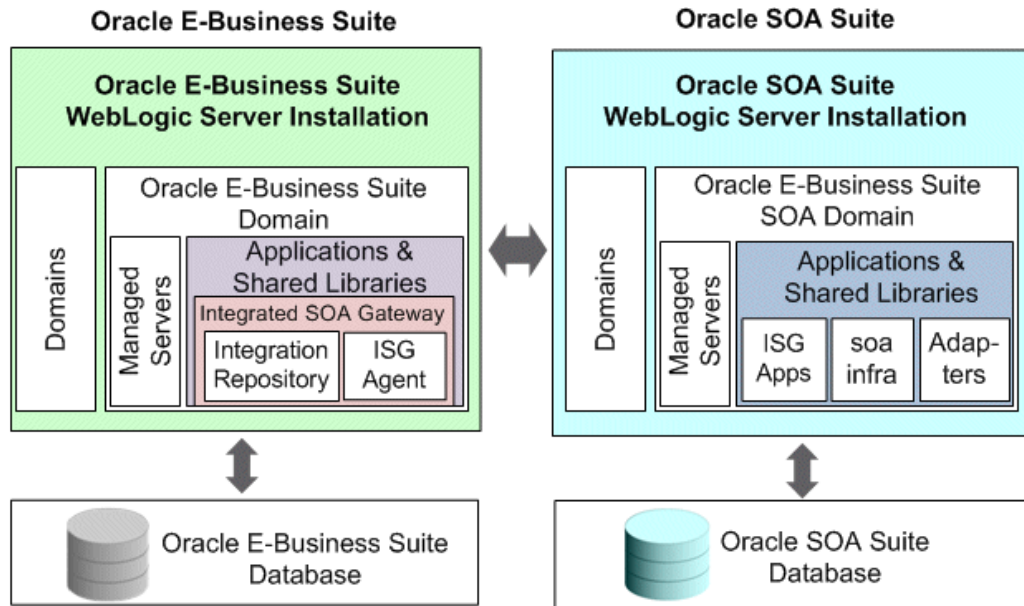
Enabling XML Gateway Services through Oracle E-Business Suite Integrated SOA Gateway

Oracle E-Business Suite Integrated SOA Gateway (ISG) leverages Oracle SOA Suite running on Oracle WebLogic Server to provide greater capabilities for exposing

integration interfaces within Oracle E-Business Suite as Web services.

Oracle E-Business Suite and Oracle SOA Suite

Oracle E-Business Suite is on Oracle WebLogic Server and is integrated with Oracle SOA Suite running on WebLogic Server.



With this integration, Oracle SOA Suite provides the most current SOA technology such as Web services standards support, protocols, and binding, while Oracle E-Business Suite provides the integration content (such as the XML Gateway interfaces), security model, and guidelines for best practices. This architecture not only provides best-in-class SOA infrastructure for Oracle E-Business Suite, but also facilitates the development and execution of complex business processes into highly flexible and reusable Web services.

XML Gateway messages are enabled through Service Provider, previously known as SOA Provider. It is the engine that performs the actual service generation and deployment behind the scene.

XML Gateway messages can be exposed as Web services in the following types:

- **Strongly-typed XML Gateway Services**

Strongly-typed XML Gateway service contains a complete definition of its input and output messages in XML Schema that is either included in the WSDL definition or referred to by that WSDL definition.

Strongly-typed or fine-grained services are enabled through Oracle E-Business Suite Integrated SOA Gateway's Service Provider in Oracle E-Business Suite release 12.2 and SOA Provider in Oracle E-Business Suite release 12.1.x.

XML Gateway header parameters are defined in `SOAHeader` part of a SOAP

request.

For more information about strongly-typed XML Gateway services, see *Strongly-Typed XML Gateway Services*, page 8-11.

- **Generic XML Gateway Services**

Compared to strongly-typed service, the generic structure or loosely-typed service definition in WSDL does not contain a schema that strongly defines any kind of message format that the service uses. Instead, a generic WSDL description is used.

This type of service is available prior to Oracle E-Business Suite release 12.1. Mandatory XML Gateway header parameters for inbound transactions are defined in `XMLGateway_Header` part of a SOAP request.

For more information about generic XML Gateway services, see *Generic XML Gateway Services*, page 8-15.

Web Service Security

Web service security (WS-Security) is a specification to enable applications to conduct secure SOAP message exchanges. It proposes a standard set of SOAP extensions that can be used when building secure Web services to implement message content integrity and confidentiality. It also provides support for multiple security tokens, the details of which are defined in the associated profile documents.

To secure Web service content and authenticate users who request XML Gateway services provided through Oracle E-Business Suite Integrated SOA Gateway, the following security mechanisms for inbound service requests are used:

- UsernameToken Based Security, page 8-7
- SAML Sender-Vouches Token Based Security, page 8-9

For more Web Service security information, see *Oracle E-Business Suite Integrated SOA Gateway Implementation Guide*.

UsernameToken Based Security

In the UsernameToken based security mechanism, the username/password is sent in the SOAP header for authentication. The username/password sent with the SOAP header is associated with the User created in Oracle E-Business Suite.

Username is a clear text; password is the most sensitive part of the UsernameToken profile. In this security model, the supported password type is plain text password (or PasswordText).

Note: The PasswordText password type is the password written in clear text. SOAP requests invoking the Web services should include

security header consisting of Username and plain text password. The password received as part of the SOAP request at runtime will be validated against the encrypted password stored in Oracle E-Business Suite. After validation, the plain text password from the SOAP request will be discarded.

At run time, SOAP request messages received through Oracle SOA Suite are passed on to a JAAS based login module for Oracle E-Business Suite for authentication based on the `wsse:security` Web Security headers. The username/password discussed here in `wsse:security` is associated with the User created in Oracle E-Business Suite.

A SOAP header should include the following `wsse:security` section:

```
<wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wsswssecurity-secext-1.0.xsd">
  <wsse:UsernameToken>
    <wsse:Username>username</wsse:Username>
    <wsse:Password>password</wsse:Password>
  </wsse:UsernameToken>
</wsse:Security>
```

Important: Authorization Check at Both the Trading Partner Level and WS-Security Level for XML Gateway Interfaces

In Oracle XML Gateway, each trading partner is configured with Oracle E-Business Suite users. Only these authorized users defined in the Trading Partner Setup form are allowed to perform XML transactions. External clients can pass such usernames in the `<USERNAME>` and `<PASSWORD>` elements defined within the `<ECX:SOAHeader>` element (or `<XMLGateway_Header>` element for generic XML Gateway services) in the SOAP body. These username parameters are validated by Oracle XML Gateway against the username defined in the trading partner setup before initiating a transaction.

Therefore, for XML Gateway interface type, the authorization check is performed at both the trading partner level, as well as on the username passed in the `wsse:security` header in the SOAP request.

For more information on how to set up trading partners, see Trading Partner Setup, page 3-19 and Trading Partner User Security, page 3-36.

A typical WS-Security header in the SOAP message from Oracle E-Business Suite can be as follows:


```

<xml version="1.0" encoding="UTF-8">
<env:Envelope xmlns:env="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <env:Header>

    <wsse:Security xmlns:wsse="http://docs.oasis-open.
org/wss/2004/01/oasis-200401-wsswssecurity-secext-1.0.xsd">
      <wsse:UsernameToken>
        <wsse:Username>username</wsse:Username>
        <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-
200401-wss-username-token-profile-1.0#PasswordText">password</wsse:
Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </env:Header>

  <env:Body>
    ...
  </env:Body>
</env:Envelope>

```

Note: The username/password information in XMLGateway_Header is the username/password associated with trading partner setup.

For more information about setting up a trading partner, see Trading Partner Setup, page 3-19.

SAML Sender-Vouches Token Based Security

To authenticate Web services relying on sending a username only through SAML assertion, SAML Token (Sender Vouches) based Web service security is supported by Oracle E-Business Suite Integrated SOA Gateway for XML Gateway services..

Security Assertion Markup Language (SAML) is an XML-based standard for exchanging authentication and authorization data between security domains, that is, between an identity provider and a service provider.

How to Authenticate Users through a Trusted Sender-Vouches SAML Token

A SAML token uses SAML assertions as security tokens. One type of SAML token is the sender-vouches SAML token. This token uses a method called a sender-vouches method to establish the correspondence between a SOAP message and the SAML assertions added to the SOAP message.

When a Web application invokes a service that uses SAML as its authentication mechanism, this SOAP request message containing or referencing SAML assertions is received and passed on to a JAAS based login module for Oracle E-Business Suite for authentication based on the `wsse:security` Web Security headers. As part of the validation and processing of the assertions, the receiver or the login module for Oracle E-Business Suite must establish the relationship between the subject, claims of the referenced SAML assertions, and the entity providing the evidence to satisfy the confirmation method defined for the statements.

In other words, in order to validate and authenticate a user who logs on to the

enterprise information system, a trusted sender-vouches SAML token security mechanism must be used to establish the correspondence between the SOAP message and the SAML assertions added to the SOAP message.

Note: Since everyone can send a SAML Token with valid conditions, the authentication framework only trusts certain SAML token sources and stores the public key of each of these sources in a common key store. This Public Key Infrastructure (PKI) based security provides more sophisticated trusted rules to authenticate Web services.

Please note that the following algorithms have been certified for SAML Token security in this release:

- Symmetric Encoding Algorithm:
<http://www.w3.org/2001/04/xmlenc#aes128-cbc>
- Key Encryption Algorithm:
<http://www.w3.org/2001/04/xmlenc#rsa-oaep-mgf1p>

Important: To ensure SAML Token security works properly, necessary setup steps need to be performed. For setup information on SAML Token security, see Setting Up SAML Token Security for Oracle E-Business Suite Integrated SOA Gateway Release 12.2, My Oracle Support Knowledge Document 1332262.1 for details.

To authenticate users, any entity that establishes a PKI trust with Oracle E-Business Suite Integrated SOA Gateway can send the SAML Assertion with a valid Username. A PKI trusted entity will send a SAML token profile with the username embedded with it and that must be digitally signed. The SAML Token policy attached to the Web service verifies attributes like "Issuer", "Conditions" etc. After the verification, the login module (`IsgSAMLLoginModule`) extracts the SAML principal (username in `NameIdentifier`) through a `NameCallback`. This is verified against LDAP for Single Sign-On (SSO) users or against Oracle E-Business Suite `FND_USER` for non-SSO users.

Please note that for strongly-typed XML Gateway services enabled through Oracle E-Business Suite Integrated SOA Gateway, it is mandatory that all users must be valid Oracle E-Business Suite users. If SSO mechanism is used, then the user in LDAP server for SSO should be in synchronous with Oracle E-Business Suite `FND_USER` table. Otherwise, the user authorization check will fail when looking up the application responsibilities for user authorization against entries in the `FND_USER` table. For more information on integrating Oracle E-Business Suite in an enterprise single sign-on environment, see the *Oracle E-Business Suite Security Guide*.

A sample sender-vouches SAML assertion for a non-SSO environment can be as follows:

```
<Assertion AssertionID="be7d9814c36381c27fefa89d8f27e126" IssueInstant="
2010-02-27T17:26:21.241Z" Issuer="www.oracle.com" MajorVersion="1"
MinorVersion="1" xmlns="urn:oasis:names:tc:SAML:1.0:assertion" xmlns:
samlp="urn:oasis:names:tc:SAML:1.0:protocol"><Conditions NotBefore="
2010-02-27T17:26:21.241Z" NotOnOrAfter="2011-02-27T17:26:21.241Z"/>
<AuthenticationStatement AuthenticationInstant="2010-02-27T17:26:21.241
Z" AuthenticationMethod="urn:oasis:names:tc:SAML:1.0:am:password">
<Subject>
<NameIdentifier Format="urn:oasis:names:tc:SAML:1.1:nameid-format:
unspecified" NameQualifier="notRelevant">SYSADMIN</NameIdentifier>
<SubjectConfirmation>
<ConfirmationMethod>urn:oasis:names:tc:SAML:1.0:cm:sender-
vouches</ConfirmationMethod>
</SubjectConfirmation>
</Subject>
</AuthenticationStatement>
</Assertion>
```

A sample sender-vouches SAML assertion for a SSO environment can be as follows:

```
<Assertion
IssueInstant="2010-02-27T17:26:21.241Z" Issuer="www.oracle.com"
MajorVersion="1" MinorVersion="1"
xmlns="urn:oasis:names:tc:SAML:1.0:assertion"
xmlns:samlp="urn:oasis:names:tc:SAML:1.0:protocol"><Conditions
NotBefore="2010-02-27T17:26:21.241Z"
NotOnOrAfter="2011-02-27T17:26:21.241Z"/>
<AuthenticationStatement
AuthenticationInstant="2010-02-27T17:26:21.241Z"
AuthenticationMethod="urn:oasis:names:tc:SAML:1.0:am:password">
<Subject>
<NameIdentifier Format="urn:oasis:names:tc:SAML:1.1:nameid-format:
unspecified"
NameQualifier="notRelevant">orclApplicationCommonName=PROD1,
cn=EBusiness,cn=Products,cn=OracleContext,dc=us,dc=oracle,
dc=com</NameIdentifier>
<SubjectConfirmation>
<ConfirmationMethod>urn:oasis:names:tc:SAML:1.0:cm:sender-
vouches</ConfirmationMethod>
</SubjectConfirmation>
</Subject>
</AuthenticationStatement>
</Assertion>
```

For information on how the sender-vouches SAML Token is used in SOAP security header to authenticate Web services, see *Managing Web Service Security, Oracle E-Business Suite Integrated SOA Gateway Implementation Guide*.

Strongly-typed XML Gateway Services

XML Gateway messages (along with other integration interfaces) within Oracle E-Business Suite are displayed through Oracle Integration Repository where they can be exposed to all users.

Note: Oracle Integration Repository is one of essential components within Oracle E-Business Suite Integrated SOA Gateway. Building on the principle of service-oriented architecture technology, Oracle E-Business Suite Integrated SOA Gateway provides the service enablement capability for strongly-typed XML Gateway services

through Service Provider.

Strongly-typed XML Gateway Service through Service Provider

Integration Repository

Integration Repository >
XML Gateway : Confirm BOD

BrowseSearchPrintable Page

Log Configuration Disabled

Internal Name XNB:CBODI
Type XML Gateway Map
Product Telecommunications Billing Integrator
Status Active
Business Entity [XML Gateway Confirmation Message](#)

Scope Public
Interface Source Oracle

Standard OAG 7.2 Confirm_BOD_004

Full Description
Confirmation BOD map annotation public description.

Web Service
Web Service Status **Deployed | Active** | [View WSDL](#)
Interaction Pattern **Synchronous**
* Authentication Type ☒ Username Token
☐ SAML Token (Sender Vouches)
[Show Generic XML Gateway Service](#)
Source Information

Methods

Name	Internal Name	Status	Description
CBODI	CBODI	Active	Process transaction.

TIP You can assign authorized users to perform XML Gateway inbound messages with a trading partner. Navigate to XML Gateway responsibility > Define Trading Partners > User Setup. Ensure to set profile option 'ECX: Enable User Check for Trading Partner' as 'Yes' to enable Trading Partner specific security feature. For more details, refer Oracle XML Gateway User's Guide.

Copyright (c) 1998, 2015, Oracle and/or its affiliates. All rights reserved. Privacy Statement

Web Service Region

The Web Service region is displayed in the interface details page for strongly-typed XML Gateway inbound maps, no matter if the service is generated or not.

This region contains the following fields:

- **Web Service Status:** This indicates whether the selected XML Gateway map has an associated service generated or not, or whether it is deployed.

If an XML Gateway service is available for the selected map, the Web Service Status field can be any values other than 'Not Generated'. It can be 'Generated' if is not deployed, or 'Deployed' with 'Active' state if it has been deployed to an Oracle SOA Suite managed server.

- **View WSDL link:** This link lets you view the WSDL description for the generated or deployed service.
- **Interaction Pattern:** This field appears only when the associated service is generated for this XML Gateway map.

Please note that XML Gateway interface can be service enabled only with synchronous operation pattern. If a selected XML Gateway interface is exposed as a Web service, the single method contained in the interface must be generated only with the support for synchronous pattern by default. Once the XML Gateway service is generated, 'Synchronous' is automatically displayed in the Interaction Pattern field by default.

- **Authentication Type:** This field displays the selected Authentication Type in read-only mode once the service has been deployed.
 - **Username Token:** This authentication type provides username and password information in the security header for a Web service provider to use in authenticating users who request XML Gateway services.
See: UsernameToken Based Security, page 8-7.
 - **SAML Token (Sender Vouches):** This authentication type is used for Web services relying on sending a username only through SAML Assertion.
See: SAML Sender-Vouches Token Based Security, page 8-9.

If the Web service has been generated successfully, the service can be deployed to Oracle SOA Suite for public access. For more information on the Web Service region for XML Gateway Web services, see the *Oracle E-Business Suite Integrated SOA Gateway User's Guide*.

Managing User Security for XML Gateway Web Services

User security is managed through the Trading Partner User Setup form where the administrator needs to associate users with a trading partner. Only these authorized users can perform XML Gateway inbound transactions with the trading partner. Specifically, the administrator needs to:

- Set the "ECX: Enable User Check for Trading Partner" profile option to "Yes" to enable the trading partner specific security feature
- Associate users with a trading partner in the Trading Partner User Setup form

For more information about setting the user security, see Trading Partner User Security, page 3-36.

For more information on how to access strongly-typed XML Gateway services, see Enabling and Accessing Strongly-typed XML Gateway Services through Oracle E-Business Suite Integrated SOA Gateway, page 8-25.

SOA Header for XML Gateway Messages through Service Provider

For strongly-typed XML Gateway services enabled through Service Provider, all XML Gateway header parameters for inbound transactions are defined in SOAHeader part of a SOAP request.

The following code snippet shows the SOAHeader part of a SOAP request for an XML Gateway inbound message through Service Provider:

```
<soapenv: Envelope xmlns:ecx="http://xmlns.oracle.com/apps/ecx/soapprovider/xmlgateway/ecx__cbodi/"
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:sys="http://xmlns.oracle.com/xdb/SYSTEM">
  <soapenv:Header>
    <wsse:Security soapenv:mustUnderstand="1"
      xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
      <wsse:UsernameToken wsu:Id="UsernameToken-10586449"
        xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd">
        <wsse:Username>SYSADMIN</wsse:Username>
        <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0#PasswordText">password</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
    <ecx:SOAHeader>
      <sys:ECXMSG>
        <MESSAGE_TYPE></MESSAGE_TYPE>
        <MESSAGE_STANDARD></MESSAGE_STANDARD>
        <TRANSACTION_TYPE></TRANSACTION_TYPE>
        <TRANSACTION_SUBTYPE></TRANSACTION_SUBTYPE>
        <DOCUMENT_NUMBER></DOCUMENT_NUMBER>
      <PARTYID></PARTYID>
      <PARTY_SITE_ID></PARTY_SITE_ID>
      <PARTY_TYPE></PARTY_TYPE>
      <PROTOCOL_TYPE></PROTOCOL_TYPE>
      <PROTOCOL_ADDRESS></PROTOCOL_ADDRESS>
      <USERNAME></USERNAME>
      <PASSWORD></PASSWORD>
      <ATTRIBUTE1></ATTRIBUTE1>
      <ATTRIBUTE2></ATTRIBUTE2>
      <ATTRIBUTE3></ATTRIBUTE3>
      <ATTRIBUTE4></ATTRIBUTE4>
      <ATTRIBUTE5></ATTRIBUTE5>
    </sys:ECXMSG>
  </ecx:SOAHeader>
</soapenv:Header>
```

For the description of each XML Gateway message parameter in the SOAHeader part of a SOAP request, see XML Gateway Envelope, page 4-8.

Please note that the Username and Password in <wsse:Security> is Oracle E-Business Suite username/password or the username/password created through the Users window in defining an application user.

The Username and Password in <ecx:SOAHeader> is the username and password associated with trading partner setup.

For more information about setting up a trading partner, see Trading Partner Setup, page 3-19.

Generic XML Gateway Services

In Oracle E-Business Suite release 12.0, XML Gateway maps were service enabled and deployed as generic XML Gateway services.

If your system is upgraded from a previous release and if you have been using generic XML Gateway services, these generic XML Gateway services can be displayed in the interface details page from Oracle Integration Repository by setting *FND: XML Gateway Map Generic Service* profile value to 'Yes'.

In the Web Service region of the interface details page, click the **Show Generic XML Gateway Service** or **Hide Generic XML Gateway Service** link to display or close the Generic XML Gateway Service subregion.

For information on setting profile options for XML Gateway services, see Setting Profile Options, page 8-21.

In addition to setting profile options, additional setup tasks for enabling generic XML Gateway services through Oracle E-Business Suite Integrated SOA Gateway need to be configured. For setup information, see *Installing Oracle E-Business Suite Integrated SOA Gateway, Release 12.2*, My Oracle Support Knowledge Document 1311068.1 for details.

Generic XML Gateway Service Subregion

▲ Web Service

Web Service Status **Deployed | Active** | [View WSDL](#)
Interaction Pattern **Synchronous**

* Authentication Type

☒ Username Token

☐ SAML Token (Sender Vouches)

▲ Hide Generic XML Gateway Service

Web Service Status **Not Deployed**

Interaction Pattern **Synchronous**
Authentication Type **Username Token**

The Generic XML Gateway Service subregion contains the following fields:

- **Web Service Status:** This field indicates the current state of the selected XML Gateway interface whether it has a generic service available, or if the service has been deployed or not.

If Oracle E-Business Suite Integrated SOA Gateway is not configured for generic XML services, the Web Service Status field is displayed as 'Not Deployed'.

- **View Generic WSDL:** Click the **View Generic WSDL** link to display the deployed generic WSDL URL for the selected XML Gateway interface.

The deployed generic WSDL URL has the following syntax:

`http://<SOA server host>:<SOA Suite managed server port>/soa-infra/services/default/XMLGatewayService!<version chosen while deploying>XMLGateway?WSDL`

- **<SOA Suite managed server port>:** It is the port of the server where SOA composite is deployed.
- **<version chosen while deploying>:** At the time of deployment, deployment version will be asked. Default version value is 1.0.

For example, `http://<SOA server host>:<SOA Suite managed server port>/soa-infra/services/default/XMLGatewayService!1.0/XMLGateway?WSDL`.

After the upgrade to Oracle E-Business Suite Release 12.2, the deployed WSDL URL has been changed from an earlier release. Therefore, you may have to replace it with the new WSDL URL and service location or address accordingly in Web service clients while invoking the generic XML Gateway service. To ensure backward compatibility, the previous service endpoint (services deployed in Oracle E-Business Suite) will continue to work at run time after the upgrade. New service endpoint from Oracle SOA Suite server will be shown in the interface detail page and should be used for client programs.

The updated WSDL URL is also populated in the *ISG: Generic Service WSDL URL for XMLG* profile option by default if setup tasks for generic XML Gateway services are configured properly.

- **Interaction Pattern:** 'Synchronous' is displayed by default in read-only mode.
- **Authentication Type:** 'Username Token' is displayed by default in read-only mode.

XMLGateway_Header for Generic XML Gateway Messages

For generic XML Gateway services, all mandatory XML Gateway header parameters for inbound transactions are defined in `XMLGateway_Header` part of a SOAP request.

The following code snippet shows the `XMLGateway_Header` part of a SOAP request for an XML Gateway inbound message:


```

<soap:Envelope>
  <soap:Header>
    ...
    <ns1:XMLGateway_Header
xmlns:ns1="http://xmlns.oracle.com/apps/fnd/XMLGateway
soapenv:mustUnderstand="0">
      <ns1:MESSAGE_TYPE></ns1:MESSAGE_TYPE>
      <ns1:MESSAGE_STANDARD></ns1:MESSAGE_STANDARD>
      <ns1:TRANSACTION_TYPE></ns1:TRANSACTION_TYPE>
      <ns1:TRANSACTION_SUBTYPE></ns1:TRANSACTION_SUBTYPE>
      <ns1:DOCUMENT_NUMBER></ns1:DOCUMENT_NUMBER>
      <ns1:PARTY_SITE_ID></ns1:PARTY_SITE_ID>
    </ns1:XMLGateway_Header>
  </soap:Header>
  ...
</soap:Envelope>

```

The following table describes the XML Gateway header information in XMLGateway_Header part of a SOAP request:

XMLGateway_Header Part of a SOAP Request

Parameter Name	Description
MESSAGE_TYPE	Payload message format. This defaults to XML. Oracle XML Gateway currently supports only XML.
MESSAGE_STANDARD	Message format standard as displayed in the Define Transactions form and entered in the Define XML Standards form. This defaults to OAG. The message standard entered for an inbound XML document must be the same as the message standard in the trading partner setup.
TRANSACTION_TYPE	External Transaction Type for the business document from the Trading Partner table. The transaction type for an inbound XML document must be the same as the transaction type defined in the Trading Partner form.
TRANSACTION_SUBTYPE	External Transaction Subtype for the business document from the Trading Partner table. The transaction subtype for an inbound XML document must be the same as the transaction subtype defined in the Trading Partner form.
DOCUMENT_NUMBER	The document identifier used to identify the transaction, such as a purchase order or invoice number. This parameter is not used by the XML Gateway, but it may be passed on inbound messages.

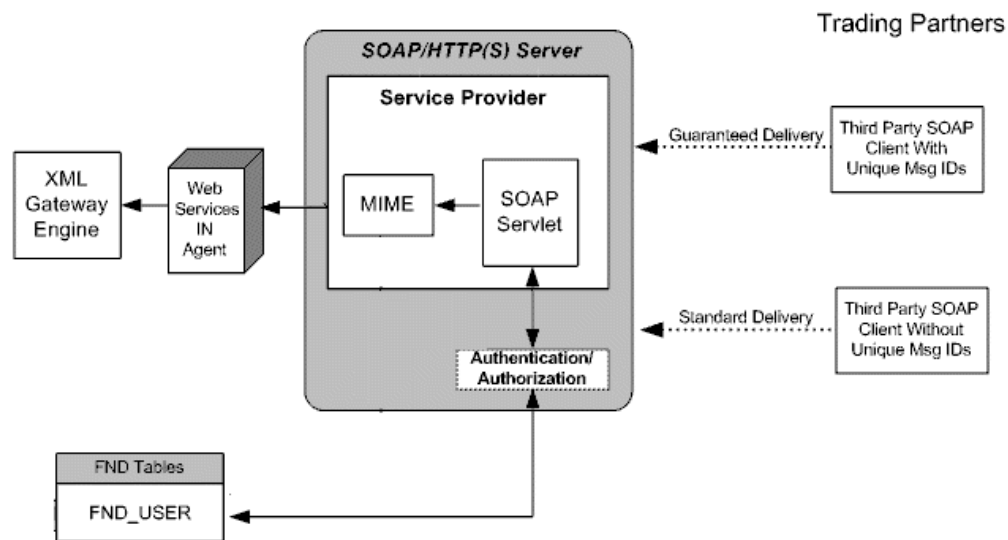
Parameter Name	Description
PARTY_SITE_ID	The party site identifier for an inbound XML document must be the same as the Source Trading Partner location defined in the Trading Partner form.

Process Flows

This section describes the process flow of an inbound Web service in Oracle E-Business Suite.

Inbound Process Flow

Requests are authorized against the Application Object Library user tables (fnd_user). After authorization, the body (XML data) and header (trading partner data) information is extracted from the request. XML messages are prepared and enqueued to the SOAP agent (Web Services IN Agent). The agent listeners pick up the message and push to XML Gateway engine for further processing.

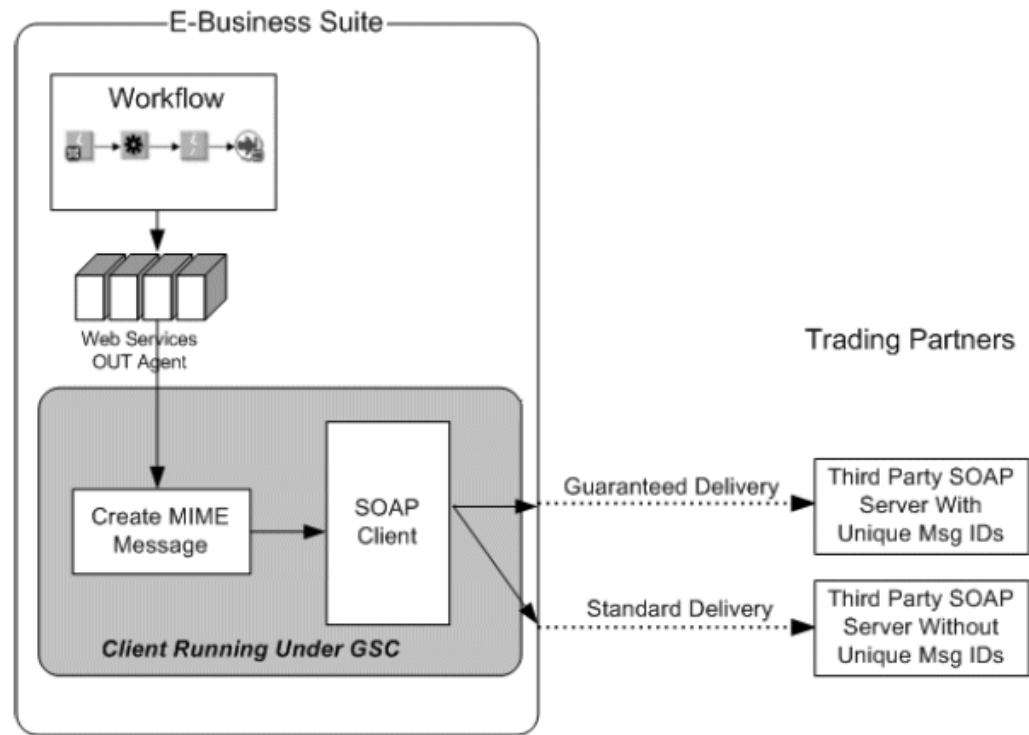


Outbound Process Flow

XML messages are created by Workflow processes and passed to the SOAP agent (Web Services OUT Agent). The messages are picked up by the SOAP Web service client running under the Web service container of the Generic Service Manager (GSM).

The client is responsible for the actual delivery of the message to the trading partner. The client examines the event or message from the queue and delivery. Messages are

formatted according to the required protocol. Attachments to the message are extracted and packaged in the MIME format. References to the attachments are passed throughout the message processing.



Setup Steps for Invoking XML Gateway Web Services

To enable XML Gateway services through Oracle E-Business Suite Integrated SOA Gateway, some setup tasks are required for both strongly-typed services and generic XML Gateway services. For detailed setup and configuration tasks, see *Installing Oracle E-Business Suite Integrated SOA Gateway, Release 12.2*, My Oracle Support Knowledge Document 1311068.1 for details.

Additionally, to enable Web service transactions, the following setup tasks are required to provide and consume Web services:

- Providing Services through XML Gateway Inbound Transactions, page 8-19
- Consuming Services through XML Gateway Outbound Transactions, page 8-20
- Setting Profile Options, page 8-21

Setup Tasks for Providing Services through XML Gateway Inbound Transactions

Perform the following setup tasks to provide services through XML Gateway inbound

transactions:

- **Enabling Agents and Starting Agent Listeners**

For providing services through XML Gateway inbound transactions, agent listeners running on ECX_INBOUND queue for strongly-typed XML Gateway services need to be running.

The following agents must be enabled and the seeded agent listener service components must be running:

Agent Name	Listener Name
WF_WS_JMS_IN	Web Services IN Agent for generic XML Gateway services

Use the Workflow Administrator: Agents window to enable the agents if necessary (the agents are enabled by default). The same window can be used to disable an agent.

Use Oracle Applications Manager to schedule and manage agent listener service components. For more information, refer to the Oracle Applications Manager online help.

The agent listener components are seeded with a manual startup mode and therefore must be manually started.

- **Setting profile option ECX_IN_JAVA_PROCESSING to Yes**

This enables message processing in Java. For more information about this profile option, see Setting Profile Options, page 8-21.

Setup Tasks for Consuming Services through XML Gateway Outbound Transactions

Perform the following setup tasks to consume services through XML Gateway outbound transactions:

- **Enabling Agents and Starting Agent Listeners**

The following agents must be enabled and the seeded agent listener service components must be running for XML Gateway outbound services

Agent Name	Listener Name
WF_WS_JMS_OUT	Web Services OUT Agent

Use the Workflow Administrator: Agents window to enable the agents if necessary

(the agents are enabled by default). The same window can be used to disable an agent.

Use Oracle Applications Manager to schedule and manage agent listener service components. For more information, refer to the Oracle Applications Manager online help.

The agent listener components are seeded with a manual startup mode and therefore must be manually started.

- **Defining Lookup Values**

You need to set up the correlation identifier in Oracle E-Business Suite. The correlation identifier enables you to label messages meant for a specific agent, in case there are multiple agents listening on the outbound queue. The agent listening for a particular correlation picks up the messages that match the correlation identifier for the agent.

In the Lookup form, search for COMM_METHOD in the Type field to see if it exists in the system. Add a new record to the COMM_METHOD type by entering BPEL for the Code field and the Meaning field. Enter description information and save the record.

Oracle XML Gateway puts the correlation of BPEL when enqueueing the message on the ECX_OUTBOUND queue. In addition to having the correlation identifier set up correctly, you need to ensure the trading partner that you want to use has the Protocol Type field set to BPEL.

See Oracle XML Gateway Lookups, page 3-17.

Setting Profile Options

To support XML Gateway Web services, the following profile options described in the table need to be set properly.

Profile Option	Description	Required	Default Value
FND: XML Gateway Map Generic Service	Use this profile option to display or hide the generic XML Gateway service information for the selected XML Gateway map. • If it is set to 'Yes', the Generic XML Gateway Service subregion is displayed within the Web Service region in the XML Gateway Map interface details page. • If it is set to 'No', the Generic XML Gateway Service subregion will not be displayed in the XML Gateway Map interface details page.	Yes	Yes Important: If you do not start from this release and you have been using generic XML Gateway Web service, set the profile option to 'Yes'. This allows the Generic XML Gateway Services subregion to be displayed within the Web Service region. Otherwise, subregion will not be shown and any invocations of generic XML Gateway Web services will return a fault message.

Profile Option	Description	Required	Default Value
ISG: Generic Service WSDL URL for XMLG	Once a generic XML Gateway Web service has been deployed, the deployed service WSDL URL should be populated as the profile value. The URL is also displayed in the 'Generic XML Gateway Service' subregion. If the profile value is not shown which means that the generic service has not yet been deployed, no WSDL URL is displayed in the subregion for the selected XML Gateway interface.	Yes	N/A

Profile Option	Description	Required	Default Value
ECX_IN_JAVA_PROCESSING	This profile option determines if inbound messages will be processed in Java or in PL/SQL. - If the profile value is set to Yes, inbound messages will be processed in Java. If the message was received from WF function activity, then set the return status as Notified. - If the profile value is set to No, inbound messages will be processed in PL/SQL.	Yes	Yes

Use the *FND: XML Gateway Map Generic Service* profile option to display or hide generic XML Gateway service information in the Generic XML Gateway Service subregion only if your system is upgraded from a previous release and you have been using generic XML Gateway Web services.

For information on setting profile options, see *User Profiles and Profile Options* in *Oracle Application Object Library, Oracle E-Business Suite Setup Guide*.

Implementation Steps

This section discusses how to enable and access strongly-typed XML Gateway services and how to invoke them from Oracle E-Business Suite. This section includes the following topics:

- Enabling and Accessing Strongly-typed XML Gateway Services through Oracle E-Business Suite Integrated SOA Gateway, page 8-25

- Accessing Generic XML Gateway Services, page 8-27
- Invoking XML Gateway Services from Oracle E-Business Suite, page 8-28

Enabling and Accessing Strongly-typed XML Gateway Services through Oracle E-Business Suite Integrated SOA Gateway

To enable strongly-typed XML Gateway services, you must log in to Oracle E-Business Suite as a user who has the Integration Administrator role.

Note: Oracle E-Business Suite Integrated SOA Gateway allows the following three user roles to access the Integration Repository user interface and perform different tasks. Each user role is associated with a specific responsibility by default to access the Integration Repository.

- Integration Analyst - Integration Repository responsibility
- Integration Developer - Integrated SOA Gateway responsibility
- Integration Administrator - Integrated SOA Gateway responsibility

Only users who have the Integration Administrator role can perform administrative tasks including generating and deploying Web services as well as creating security grants. Integration developers (or users who have the Integration Developer role) can generate Web services, but they cannot perform other administrative tasks. Integration analysts (or users who have the Integration Analyst role) can only search and view interfaces from the Integration Repository.

Select the **Integrated SOA Gateway** responsibility from the Navigator. Click the **Integration Repository** link to access the Integration Repository where you can find XML Gateway interfaces.

To enable a Web service for an XML Gateway interface, you must first locate the interface that you want to enable. This step can be achieved through the following ways:

- Navigate to the interface from the Integration Repository browser window
Select 'Interface Type' from the View By drop-down list. Select 'XML Gateway Map' folder to expand the navigation tree and see a list of the available XML Gateway interfaces.
- Locate the interface through a search
Click **Search** to open the main search window. Enter appropriate search criteria to locate a desired XML Gateway map.

Once you locate the desired interface from the tree or a search result, click the interface

name link to open the interface details page. The Web Service region appears allowing you to perform administrative tasks for the selected map.

Only users who have the Integration Administrator role or the Integration Developer role can find the **Generate** button in the interface details page for a selected XML Gateway map.

Generating Web Services with Support for Synchronous Interaction Pattern

Since XML Gateway interfaces can be service enabled only with the support for synchronous operation pattern, if a selected interface of XML Gateway interface type is exposed as a Web service, the service is generated synchronously by default. Therefore, once an XML Gateway service has been successfully generated, 'Synchronous' is displayed in the Interaction Pattern field along with the Web Service Status - 'Generated'.

Click the **View WSDL** link to view the WSDL description for the generated or deployed service.

Deploying Web Services with Authentication Types

To secure Web service content and authenticate Oracle E-Business Suite users at run time through Oracle SOA Suite WebLogic environments, prior to deploying a Web service, an integration administrator must first select one authentication type - Username Token or SAML Token (Sender Vouches).

Click **Deploy** to deploy the generated service from Oracle Integration Repository to an Oracle SOA Suite managed server with active deployment state. A deployed service with active state indicates that the service is ready to accept new SOAP requests.

Recording Deployed WSDL URL

To use and invoke the deployed Web services, an integration developer must record the deployed Web service WSDL URL for the inbound message map by clicking the View WSDL link. This opens a new window where you can record the deployed WSDL URL and use it to invoke a service.

For detailed instruction on how to generate and deploy a Web service, see:

- *Generating SOAP Web Services, Oracle E-Business Suite Integrated SOA Gateway Implementation Guide*
- *Deploying and Undeploying SOAP Web Services, Oracle E-Business Suite Integrated SOA Gateway Implementation Guide*

Use the following steps to enable and access strongly-typed XML Gateway services:

1. Log in to Oracle E-Business Suite as a user who has the Integration Administrator role.
2. Select the **Integrated SOA Gateway** responsibility.
Select the **Integration Repository** link to access the Integration Repository.

3. From the Integration Repository browser window, select 'Interface Type' from the View By drop-down list.
4. Select 'XML Gateway Map' folder to expand the navigation tree and see a list of the available XML Gateway interfaces.
5. Alternatively, locate an XML Gateway message directly by performing a search.
Click **Search** to open the main search window. Enter appropriate search criteria to locate a desired XML Gateway map.
6. Click a desired XML Gateway map link either from the navigation tree or from the search result. This opens the interface details page for the selected XML Gateway map.
7. Click **Generate** to generate the Web service with the support for synchronous interaction pattern for the selected XML Gateway map. Once the service has been successfully generated, the WSDL URL appears in the Web Service region.
8. Select one authentication type and click **Deploy** to deploy the generated service to Oracle SOA Suite.
9. Click the View WSDL link to open and view the deployed WSDL description.

Accessing Generic XML Gateway Services

If your system is upgraded from a previous release and you have been using generic XML Gateway services, you may find the Generic XML Gateway Service subregion available within the Web Service region in the interface details page if a selected XML Gateway map has the generic Web service deployed.

Click the **Show Generic XML Gateway Service** or **Hide Generic XML Gateway Service** link to display or close the Generic XML Gateway Service subregion.

Note: To support backward compatibility for generic XML Gateway services, a profile option *FND: XML Gateway Map Generic Service* is used allowing Generic XML Gateway Service subregion to be displayed in the interface details page.

For more information about setting profile options for generic XML Gateway services, see Setting Profile Options, page 8-21.

Click the **View Generic WSDL** link to display the deployed generic WSDL URL for the selected XML Gateway interface.

The deployed generic WSDL URL has the following syntax:

```
http://<SOA server host>:<SOA Suite managed server port>/soa-  
infra/services/default/XMLGatewayService!<version chosen while
```

deploying>XMLGateway?WSDL

- <SOA Suite managed server port>: It is the port of the server where SOA composite is deployed.
- <version chosen while deploying>: At the time of deployment, deployment version will be asked. Default version value is 1.0.

For example, `http://<SOA server host>:<SOA Suite managed server port>/soa-infra/services/default/XMLGatewayService!1.0/XMLGateway?WSDL`.

To locate a deployed XML Gateway service for an inbound transaction:

1. Log in to Oracle E-Business Suite as a user who has the Integration Administrator role or the Integration Developer role. Select the **Integrated SOA Gateway** responsibility from the Navigator.

If you log in to Oracle E-Business Suite as a user who has the Integration Analyst role, select the **Integration Repository** responsibility instead from the Navigator.

2. Select the **Integration Repository** link to access the Integration Repository.
3. From the Integration Repository browser window, select 'Interface Type' from the View By drop-down list.
4. Select 'XML Gateway Map' folder to expand the navigation tree and see a list of the available XML Gateway interfaces.
5. Alternatively, locate an XML Gateway message directly by performing a search.
Click **Search** to open the main search window. Enter appropriate search criteria to locate a desired XML Gateway map.
6. Click a desired XML Gateway map link either from the navigation tree or from the search result.

This opens the interface details page for the selected XML Gateway map.

7. If this message map has a generic XML Gateway service available, the generic service information can be displayed in the Generic XML Gateway Service subregion within the Web Service region of the interface details page.
Click the **Show Generic XML Gateway Service** or **Hide Generic XML Gateway Service** link to display or close the Generic XML Gateway Service subregion.

8. Click the **View Generic WSDL** link to view the deployed WSDL description.

Invoking XML Gateway Services from Oracle E-Business Suite

From Oracle E-Business Suite Release 12.1 onwards, XML Gateway messages within

Oracle E-Business Suite are displayed through Oracle Integration Repository where they can be exposed as Web services.

Before service invocation, XML Gateway interfaces must first be transformed into Web services, and then be deployed to Oracle SOA Suite for further service consumption.

- Only users who have the Integration Administrator role or the Integration Developer role can perform the generate action to transform a selected XML Gateway map into a Web service.
- Only users who have the Integration Administrator role can deploy the generated XML Gateway service to an Oracle SOA Suite managed server for further consumption.

For inbound transactions, locate and provide the deployed WSDL description to trading partners.

For information on enabling XML Gateway Web services and obtaining the WSDL URL information, see *Enabling and Accessing Strongly-typed XML Gateway Services* through Oracle E-Business Suite Integrated SOA Gateway, page 8-25.

For information on invoking XML Gateway Web services, see *Example of Using XML Gateway Services*, page 8-29.

Examples of Using XML Gateway Services

To enable bidirectional integration with Oracle E-Business Suite and consumers, XML Gateway services can be used through the following approaches:

- For an inbound XML Gateway Map interface, once a Web service of an inbound XML Gateway interface is deployed, the deployed service represented in WSDL can be used in creating a BPEL process to insert inbound data into Oracle E-Business Suite.
- For an outbound XML Gateway Map interface, since an outbound message is first enqueued to the ECX_OUTBOUND queue, it can be consumed through subscription model in Oracle E-Business Suite Integrated SOA Gateway. This can be done by first dequeuing the message to retrieve outbound data from Oracle E-Business Suite through a BPEL process. The retrieved data can then be passed to trading partners or consumers who subscribed to the message.

To better understand how to use a deployed Web service of an inbound XML Gateway interface as well as understand how the subscription model works for an outbound XML Gateway, the following topics are discussed in this chapter:

- Using XML Gateway Inbound Services, page 8-30
 - Using XML Gateway Inbound Services at Design Time, page 8-30

- Deploying and Testing the SOA Composite Application with BPEL Process at Run Time, page 8-48
- Using XML Gateway Outbound through Subscription Model, page 8-66
 - Using XML Gateway Outbound Services at Design Time, page 8-66
- Deploying and Testing the SOA Composite Application with BPEL Process at Run Time, page 8-84

For the examples described in the following sections, Oracle JDeveloper 11g (11.1.1.6.0) is used as a design-time tool to create the BPEL process and Oracle SOA Suite 11g (11.1.1.6.0) is used for the process deployment.

Note: While using Oracle JDeveloper with other Oracle Fusion Middleware components (such as Oracle SOA Suite), to enable SOA technologies, you need to manually download Oracle SOA Suite Composite Editor, a JDeveloper's extension for SOA technologies. For more information on installing additional Oracle Fusion Middleware design-time components, see the *Oracle Fusion Middleware Installation Guide for Oracle JDeveloper*.

Using XML Gateway Inbound Services

This section includes the following topics:

- Using XML Gateway Inbound Services at Design Time, page 8-30
- Deploying and Testing the SOA Composite with BPEL Process at Run Time, page 8-48

Using XML Gateway Inbound Services at Design Time

SOA Composite Application with BPEL Process Scenario

Use the XML Gateway Inbound Process PO XML Transaction as an example to explain the SOA Composite application with BPEL process creation. In this example, the XML Gateway inbound message map is exposed as a Web service through PROCESS_PO_007 inbound map. It allows sales order data including header and line items to be inserted into Order Management system while an associated purchase order is created.

When a purchase order is sent by a trading partner, the purchase order data is used as input to the BPEL process along with ECX Header properties such as MESSAGE_TYPE, MESSAGE_STANDARD, TRANSACTION_TYPE, TRANSACTION_SUBTYPE, PARTY_SITE_ID, and DOCUMENT_NUMBER. The BPEL process then pushes this purchase order in ECX_INBOUND queue. Agent Listeners running on ECX_INBOUND

would enable further processing by the Execution Engine. Oracle XML Gateway picks this XML message, does trading partner validation, and inserts order data to Order Management application.

If the BPEL process is successfully executed after deployment, you should get the same order information inserted into the Order Management table once a purchase order is created.

Prerequisites to Configure a SOA Composite Application with BPEL Process Using an XML Gateway Inbound Service

Before performing the design-time tasks for XML Gateway Inbound services, ensure the following tasks are in place:

- An integration administrator or an integration developer needs to generate a Web service first. The administrator will then deploy the generated service to an Oracle SOA Suite WebLogic managed server.
- An integration developer needs to locate and record the deployed WSDL URL for the inbound message map exposed as a Web service.
- XML Gateway header variables need to be populated for XML transaction.
- A trading partner should be available for receiving the XML documents.
- ECX Inbound Agent Listener and ECX Transaction Agent Listener must be running.

Deploying XML Gateway Inbound WSDL URL

Once a Web service for the selected XML Gateway inbound map has been successfully created, the integration administrator must deploy the service from Oracle Integration Repository to an Oracle SOA Suite WebLogic managed server.

For example, the following steps must be performed first before the integration developer creates a SOA composite application with BPEL process by using the deployed WSDL:

1. To generate a Web service, the integration administrator or the integration developer locates the interface definition first (such as an XML Gateway inbound interface `INBOUND:Process Purchase Order XML Transaction (ONT:POI)`) and then clicks **Generate** in the interface details page.

Once the service has been successfully generated, the Web Service Status field changed from 'Not Generated' to 'Generated' in the Web Service region.

For detailed instruction on how to generate a Web service, see *Enabling and Accessing Strongly-typed XML Gateway Services through Oracle E-Business Suite Integrated SOA Gateway*, page 8-25.

2. To deploy the generated Web service, the integration administrator selects one desired authentication type before clicking **Deploy**. The deployed XML Gateway

service in Oracle SOA Suite is an active service and is ready to accept new SOAP requests.

Once the service has been successfully deployed, the selected authentication type will be displayed along with 'Deployed' with 'Active' state in the Web Service Status field.

For more information on securing Web services with authentication types, see Web Service Security, page 8-7.

For information on how to deploy a Web service, see Deploying and Undeploying SOAP Web Services, *Oracle E-Business Suite Integrated SOA Gateway Implementation Guide*.

Searching and Recording WSDL URL

The integration developer needs to locate and record the deployed Web service WSDL URL for the inbound message map.

This WSDL information will be used later in creating a partner link for the inbound map exposed as a Web service during the BPEL process creation.

Populating XML Gateway Header Variables

Certain XML Gateway header variables in the BPEL PM need to be populated in order to provide context information for Oracle E-Business Suite. These variables are MESSAGE_TYPE, MESSAGE_STANDARD, TRANSACTION_TYPE, TRANSACTION_SUBTYPE, DOCUMENT_NUMBER and PARTY_SITE_ID. They are the mandatory header variables that need to be populated in order for the XML transaction to complete successfully.

Refer to Adding an Assign activity, page 8-44 for more information.

Setting Up a Trading Partner for Receiving XML Documents

Use the following steps to set up a trading partner:

1. Log in to Oracle E-Business Suite as a user who has the XML Gateway responsibility.
2. Click the **Define Trading Partners** link from the Navigator.
3. The Trading Partner Setup form is displayed. Search an existing trading partner by selecting **View > Query by Example > Enter** and enter the following information:
 - Trading Partner Type: Customer
 - Trading Partner Name: Example Inc.
 - Trading Partner Site: 401 Island Parkway Redwood Shores, CA 94065

Execute the query by selecting **View > Query by Example > Enter** to locate the trading partner along with existing transactions.

Trading Partner Setup Window

Transaction Type	Transaction SubType	Standard Code	External Transaction Type	External Transaction SubType	Direction	Map	Connection/Hub	Protocol Type
ONT	POA	OAG	PO	ACKNOWLEDG	OUT	ONT_3A4A_O	DIRECT	SMTP
ONT	CPO	OAG	PO	CANCEL	IN	ONT_3A9R_O		
ONT	POI	OAG	PO	PROCESS	IN	ONT_3A4R_O		
ONT	SSO	OAG	SALESORDER	SHOW	OUT	PROCESS_P	DIRECT	SMTP

Enter the following trading partner details:

- Transaction Type: ONT
- Transaction SubType: POI
- Standard Code: OAG
- External Transaction Type: PO
- External Transaction SubType: PROCESS
- Direction: IN
- Map: ONT_3A4R_OAG72_IN
- Source Trading Partner Location Code: Example-01

4. Save your entry.

Ensuring Agent Listeners Are Up and Running

Ensure listeners on the ECX_INBOUND, ECX_TRANSACTION queues are running. Use the following steps to configure these listeners in Oracle E-Business Suite:

1. Log in to Oracle E-Business Suite as a user who has the Workflow Administrator responsibility.
2. Click the **Workflow Administrator Web Applications** link from the Navigator.
3. Click the **Workflow Manager** link under Oracle Applications Manager.
4. Click the status icon next to **Agent Listeners**.
5. Configure and schedule the **ECX Inbound Agent Listener** and the **ECX Transaction Agent Listener**. Select the listeners, and select Start from the **Actions** box. Click **Go** if they are not up and running.

SOA Composite Application with BPEL Process Creation Flow

Based on the XML Gateway Inbound Process PO XML Transaction business scenario, the following design-time tasks are discussed in this chapter:

1. Create a New SOA Composite Application with BPEL Process, page 8-35
Use this step to create a new SOA Composite application with BPEL process called `XMLGatewayInbound.bpel` using an Synchronous BPEL Process template. This automatically creates two dummy activities - Receive and Reply - to receive input from a trading partner and to reply output of the BPEL process to the request application.
2. Create a Partner Link, page 8-38
Use this step to create a partner link to allow the inbound message to be inserted to the Oracle E-Business Suite.
3. Add a Partner Link for File Adapter, page 8-39
Use this step to add a partner link for File Adapter in order to pick up an XML file received from the trading partner to get the XML message.
4. Add Invoke Activities, page 8-43
Use this step to add two Invoke activities in order to:
 1. Get the XML message details that is received from the Receive activity.
 2. Enqueue the purchase order information to the ECX_INBOUND queue.
5. Add Assign Activities, page 8-44
Use this step to create two Assign activities in order to:
 1. Pass XML message obtained from the first Invoke activity to the second Invoke activity.

2. Pass ECX header variables to the second Invoke activity as input variables.

For general information on how to create SOA composite applications using BPEL process service component, see the *Oracle Fusion Middleware Developer's Guide for Oracle SOA Suite*.

Creating a New SOA Composite Application with BPEL Process

Use this step to create a new SOA composite application with BPEL process that will contain various BPEL process activities.

To create a new SOA composite application with BPEL process:

1. Open Oracle JDeveloper.
2. Click **New Application** in the Application Navigator.

The "Create SOA Application - Name your application" page is displayed.

The Create SOA Application - Name your application Page

Create SOA Application - Step 1 of 3

Name your application

Application Name
Project Name
Project SOA Settings

Application Name:
XMLGatewayInbound

Directory:
C:\Developer\mywork\XMLGatewayInbound Browse...

Application Package Prefix:

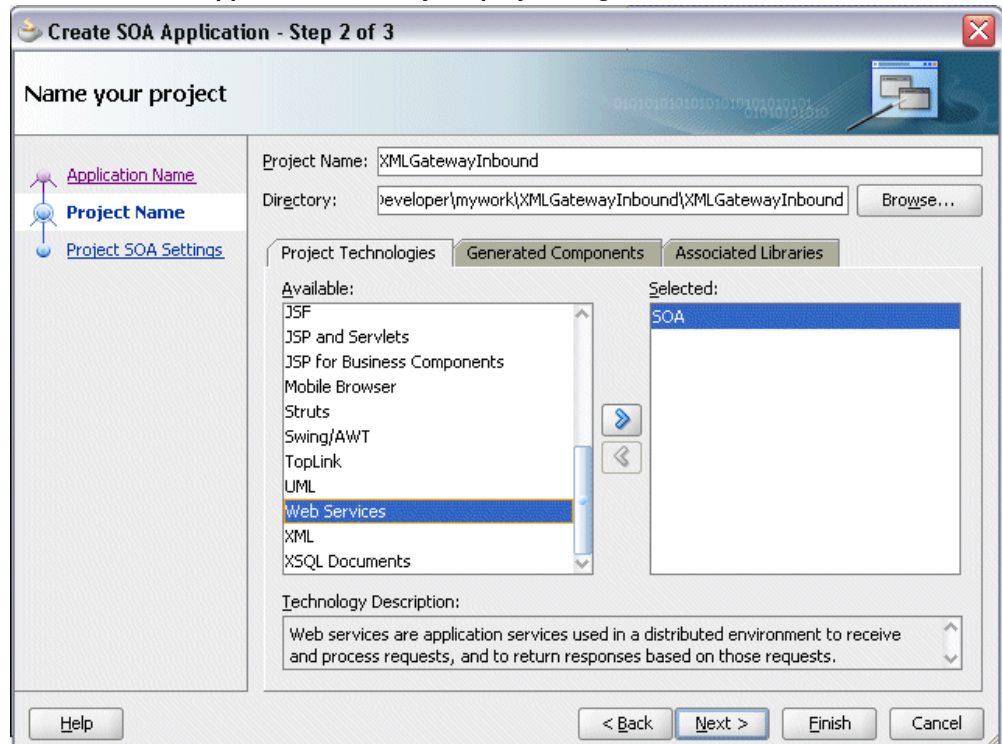
Application Template:

- Java Desktop Application (ADF)**
Creates a databound rich client application. The application consists of one project for the client (ADF Swing), and another project for the ADF Model (ADF Business Components).
- Java EE Web Application**
Creates a databound web application. The application consists of one project for the view and controller components (JSF), and another project for the data model (EJB session beans and JPA entities).
- SOA Application**
Creates a SOA (service-oriented architecture) application. The application consists of one SOA project for the SOA composite, components, and adapters.

Help < Back Next > Finish Cancel

3. Enter an appropriate name for the application in the Application Name field and select **SOA Application** from the Application Template list.
Click **Next**. The "Create SOA Application - Name your project" page is displayed.

The Create SOA Application - Name your project Page



4. Enter an appropriate name for the project in the Project Name field, for example, XMLGatewayInbound.
5. In the Project Technologies tab, select 'Web Services' and ensure that **SOA** is selected from the Available technology list to the Selected technology list.
Click **Next**. The "Create SOA Application - Configure SOA settings" page is displayed.
6. Select **Composite With BPEL Process** from the Composite Template list, and then click **Finish**. You have created a new application, and a SOA project. This automatically creates a SOA composite.
The Create BPEL Process page is displayed.
7. Leave the default **BPEL 1.1 Specification** selection unchanged. This creates a BPEL project that supports the BPEL 1.1 specification.
Enter an appropriate name for the BPEL process in the Name field, for example, XMLGInbound.
Select **Synchronous BPEL Process** in the Template field.
Select **required** from the Transaction drop-down list. Click **OK**.

A synchronous BPEL process is created with the Receive and Reply activities. The required source files including `bpel` and `wsdl`, using the name you specified (for example, `XMLGInbound.bpel` and `XMLGInbound.wsdl`) and `composite.xml` are also generated.

8. Navigate to SOA Content > Business Rules and double click `composite.xml` to view the composite diagram.

Double click on the `XMLGInbound` component to open the BPEL process.

Creating a Partner Link

Use this step to create a Partner Link called `ONT__POI` to insert sales order data to Oracle E-Business Suite.

To create a partner link to insert sales data to Oracle E-Business Suite:

1. In Oracle JDeveloper, place your mouse in the right side of the Partner Links area and right click to select **Create Partner Link...** from the pull-down menu.
Alternatively, you can drag and drop **Partner Link** from the **BPEL Constructs** list into the right Partner Link swim lane of the process diagram.

The Create Partner Link window appears.

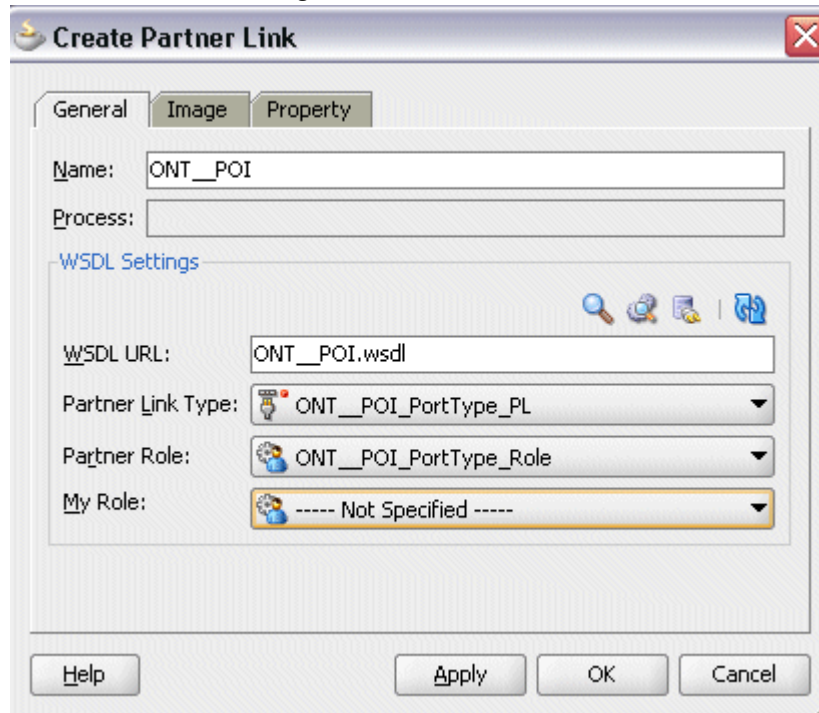
2. Copy the WSDL URL corresponding to the XML Gateway inbound map **INBOUND : Process Purchase Order XML Transaction (ONT:POI)** that you recorded earlier from the Integration Repository and paste it in the WSDL File field.

Press the **[Tab]** key.

3. A Partner Link Type message dialog box appears asking whether you want the system to create a new WSDL file that will by default create partner link types for you.

Click **Yes** to have the Partner Name value populated automatically. You can manually enter the partner link name as `ONT__POI`.

Create Partner Link Dialog

The image shows a 'Create Partner Link' dialog box with three tabs: 'General', 'Image', and 'Property'. The 'General' tab is active. It contains a 'Name' field with 'ONT__POI', an empty 'Process' field, and a 'WSDL Settings' section. The 'WSDL Settings' section includes a 'WSDL URL' field with 'ONT__POI.wsdl', a 'Partner Link Type' dropdown menu with 'ONT__POI_PortType_PL' selected, a 'Partner Role' dropdown menu with 'ONT__POI_PortType_Role' selected, and a 'My Role' dropdown menu with '----- Not Specified -----' selected. At the bottom are 'Help', 'Apply', 'OK', and 'Cancel' buttons.

Select the Partner Link Type and Partner Role fields from the drop-down lists.

Click **Apply** and **OK**.

The partner link is created with the required WSDL settings, and is represented in the BPEL project by a new icon in the border area of the process diagram.

Partner Link is added to the Partner Links section in the BPEL process diagram.

Adding a Partner Link for File Adapter

Use this step to configure a BPEL process by adding a partner link for File Adapter to get the XML Message.

To add the Partner Link for File Adapter to get the XML Message:

1. In Oracle JDeveloper, drag and drop the **File Adapter** service from the **BPEL Services** into the right Partner Link swim lane of the process diagram. The Adapter Configuration wizard welcome page appears.
2. Click **Next**. The Service Name dialog box appears.
3. Enter a name for the file adapter service, such as **GetXMLMsg**. You can add an optional description of the service.
4. Click **Next**. The Adapter Interface dialog box appears.

5. Select the **Define from operation and schema (specified later)** radio button and click **Next**. The Operation dialog box appears.

Adapter Configuration Wizard - Operation

Adapter Configuration Wizard - Step 4 of 8

Operation

The File Adapter supports four operations. There is a Read File operation that polls for incoming files in your local file system, a Write File operation that creates outgoing files, a Synchronous Read File operation that reads the current contents of a file, and a List Files operation that lists file names in specified locations. Specify the Operation type and Operation Name. Only one operation per Adapter Service may be defined using this wizard.

Operation Type:

- ☐ Read File
- ☐ Write File
- ☒ Synchronous Read File
- ☐ List Files

Operation Name:

[Help](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

6. Specify the operation type, for example **Synchronous Read File**. This automatically populates the **Operation Name** field.
Click **Next** to access the File Directories dialog box.

Adapter Configuration Wizard - File Directories

Adapter Configuration Wizard - Step 5 of 8

File Directories

Enter directory information for the incoming file of the Synchronous Read File operation.

Directory names are specified as: ☒ Physical Path ☐ Logical Name

Directory for Incoming Files (physical path):

☐ Archive processed files
Archive Directory for Processed Files (physical path):

☒ Delete files after successful retrieval

7. Select the **Physical Path** radio button and enter the input payload file directory information. For example, enter `/usr/tmp/`.

Note: To be able to locate the file from the physical directory that you specified here, you must first place the input payload file (such as `order_data_xm1g.xml`) to the specified directory.

Click **Next** to open the File Name dialog box.

8. Enter the name of the file for the synchronous read file operation. For example, enter `order_data_xm1g.xml`. Click **Next**. The Messages dialog box appears.

Adapter Configuration Wizard - Messages

The screenshot shows the 'Adapter Configuration Wizard - Step 7 of 8' window, specifically the 'Messages' tab. The window title bar includes a close button (X). The main content area has a header 'Messages' and a descriptive text: 'Define the message for the Synchronous Read File operation. Specify the Schema File Location and select the Schema Element that defines the messages in the incoming files. Use the Browse button to find an existing schema definition. If you check 'Schema is Opaque', then you do not need to specify a Schema.' Below this text is a section titled 'Message Schema' containing a checkbox labeled 'Native format translation is not required (Schema is Opaque)', a 'URL' text field with a browse button (magnifying glass icon), and a 'Schema Element' dropdown menu. At the bottom of the window are buttons for 'Help', '< Back', 'Next >', 'Finish', and 'Cancel'.

9. Select **Browse for schema file** in front of the URL field.

The Type Chooser window is displayed.

Click the **Import Schema Files** button on the top right corner of the Type Chooser window. This opens the Import Schema File pop-up window.

Enter the schema location for the service, such as http:

```
//<soa_suite_hostname>:<port>/soa-  
infra/services/default/<jndi_name>_XMLGATEWAY_ONT__POI/ONT__PO  
I_Service?XSD=xsd/PROCESS_PO_007.xsd
```

Schema location for your service can be found from the service WSDL URL (for example, http://<soa_suite_hostname>:<port>/soa-infra/services/default/<jndi_name>_XMLGATEWAY_ONT__POI/ONT__POI_Service?wsdl).

Select the **Copy to Project** checkbox and click **OK**.

10. The Localize Files window appears. Ensure the **Maintain original directory structure for imported files** checkbox is selected and click **OK**.

The Imported Schema folder is automatically added to the Type Chooser window.

11. Expand the imported schema folder and select `PROCESS_PO_007` from the `PROCESS_PO_007.xsd`. Click **OK**. The selected `PROCESS_PO_007.xsd` is displayed as URL and the `PROCESS_PO_007` element is selected as Schema Element.

Click **Next** and then **Finish**. The wizard generates the WSDL file corresponding to the partner link. The main Create Partner Link dialog box appears, specifying the new WSDL file `GetXMLMsg.wsdl`.

Click **Apply** and then **OK** to complete the configuration and create the partner link with the required WSDL settings for the File Adapter service.

The `GetXMLMsg` Partner Link appears in the BPEL process diagram.

Under applications window, navigate to file `GetXMLMsg_file.jca`. Set value of property "DeleteFile" to "false".

Adding Invoke Activities

This step is to configure three Invoke activities:

1. To get the XML message details that is received from the Receive activity by invoking the `GetXMLMsg` partner link in an XML file.
2. To enqueue the purchase order information to the `ECX_INBOUND` queue by invoking `ONT__POI` partner link in an XML file.

To add the first Invoke activity for a partner link to get XML message:

1. In Oracle JDeveloper, expand the **BPEL Constructs** from the Component Palette. Drag and drop the **Invoke** activity into the center swim lane of the process diagram, between the **receiveInput** and **replyOutput** activities.
2. Link the **Invoke** activity to the `GetXMLMsg` service. The Edit Invoke dialog box appears.
3. Enter a name for the **Invoke** activity (such as 'Invoke_Msg') and then click the **Create** icon next to the **Input Variable** field to create a new variable. The Create Variable dialog box appears.
4. Enter an input variable name. You can also accept the default name. Select **Global Variable** and click **OK**.
5. Click the **Create** icon next to the **Output Variable** field to create a new variable. The Create Variable dialog box appears.
6. Enter an input variable name. You can also accept the default name. Select **Global Variable** and click **OK**.
7. Click **Apply** and then **OK** in the Edit Invoke dialog box to finish configuring the

Invoke activity.

The first Invoke activity appears in the process diagram.

To add the second Invoke activity for a partner link to enqueue PO information:

1. In Oracle JDeveloper, expand the **BPEL Constructs** from the Component Palette. Drag and drop the second **Invoke** activity into the center swim lane of the process diagram, after the first **Invoke** activity and the **replayOutput** activity.
2. Link the **Invoke** activity to the ONT_POI service. The Edit Invoke dialog box appears.
3. Enter a name for the Invoke activity and then click the **Create** icon next to the **Input Variable** field to create a new variable. The Create Variable dialog box appears.
4. Enter an input variable name. You can also accept the default name. Select **Global Variable** and click **OK**.
5. Click **Apply** and then **OK** in the Edit Invoke dialog box to finish configuring the Invoke activity.
6. The process diagram appears.

Adding Assign Activities

This step is to configure two Assign activities:

1. To pass XML message as an input to the Invoke activity for enqueueing message.
2. To pass XML Gateway header variables as input variables to the Invoke activity in order to provide context information for Oracle E-Business Suite.

To add the first Assign activity to pass XML message as input to the Invoke activity:

1. In Oracle JDeveloper, expand the **BPEL Constructs** from the Component Palette. Drag and drop the **Assign** activity into the center swim lane of the process diagram between the two **Invoke** activities you just created earlier.
2. Double-click the **Assign** activity to access the Edit Assign dialog box.
3. Click the General tab to enter the name for the Assign activity, such as 'SetOrderXML'.
4. Select the Copy Rules tab and expand the source and target trees:
 - In the From navigation tree, navigate to **Variable > Process > Variables > Invoke_Msg_SynchRead_OutputVariable** and select **Process_PO_007**.
 - In the To navigation tree, navigate to **Variable > Process > Variables >**

Invoke_ONT_POI_InputVariable and select **body**.

Drag the source node (Process_PO_007) to connect to the target node (body) that you just identified. This creates a line that connects the source and target nodes. The copy rule is displayed in the From and To sections at the bottom of the Edit Assign dialog box.

5. Click **Apply** and then **OK** to complete the configuration of the Assign activity.

To add the second Assign activity to pass XML Gateway header variables to the Invoke activity:

1. In Oracle JDeveloper, expand the **BPEL Constructs** from the Component Palette. Drag and drop the **Assign** activity into the center swim lane of the process diagram between the first Assign and the second **Invoke** activities you just created earlier.

Add the second Assign activity by dragging and dropping the **Assign** activity into the center swim lane of the process diagram between the **SetOrderXML Assign** activity and the second **Invoke** activity.

2. Repeat Step 2 to Step 3 described in creating the first Assign activity to add the second Assign activity called 'SetECXHeader'.

3. Select the Copy Rules tab and expand the source and target trees:

- Click the Expression icon to invoke the Expression Builder dialog.

Enter 'XML' in the Expression box. Click **OK**. The Expression icon with the expression value ('XML') appears in the center of the Edit Assign dialog, between the From and To navigation tree nodes.

- In the To navigation tree, navigate to **Variable > Process > Variables > Invoke_ONT_POI_InputVariable > header > ns1:SOAHeader > ns4:ECXMSG** and select **MESSAGE_TYPE**.

The XPath field should contain your selected entry.

Drag the Expression icon to connect to the target node (MESSAGE_TYPE) that you just identified. This creates a line that connects the source and target nodes. The copy rule is displayed in the From and To sections at the bottom of the Edit Assign dialog box.

4. Use the same mechanism described in step 3 to assign values to the following parameters:

- MESSAGE_STANDARD: 'OAG'
- TRANSACTION_TYPE: 'PO'
- TRANSACTION_SUBTYPE: 'PROCESS'

- DOCUMENT_NUMBER: 'PO-4466-5'
- PARTY_SIDE_ID: 'Example-01'

5. Click **Apply** and **OK** to complete the configuration of the Assign activity.

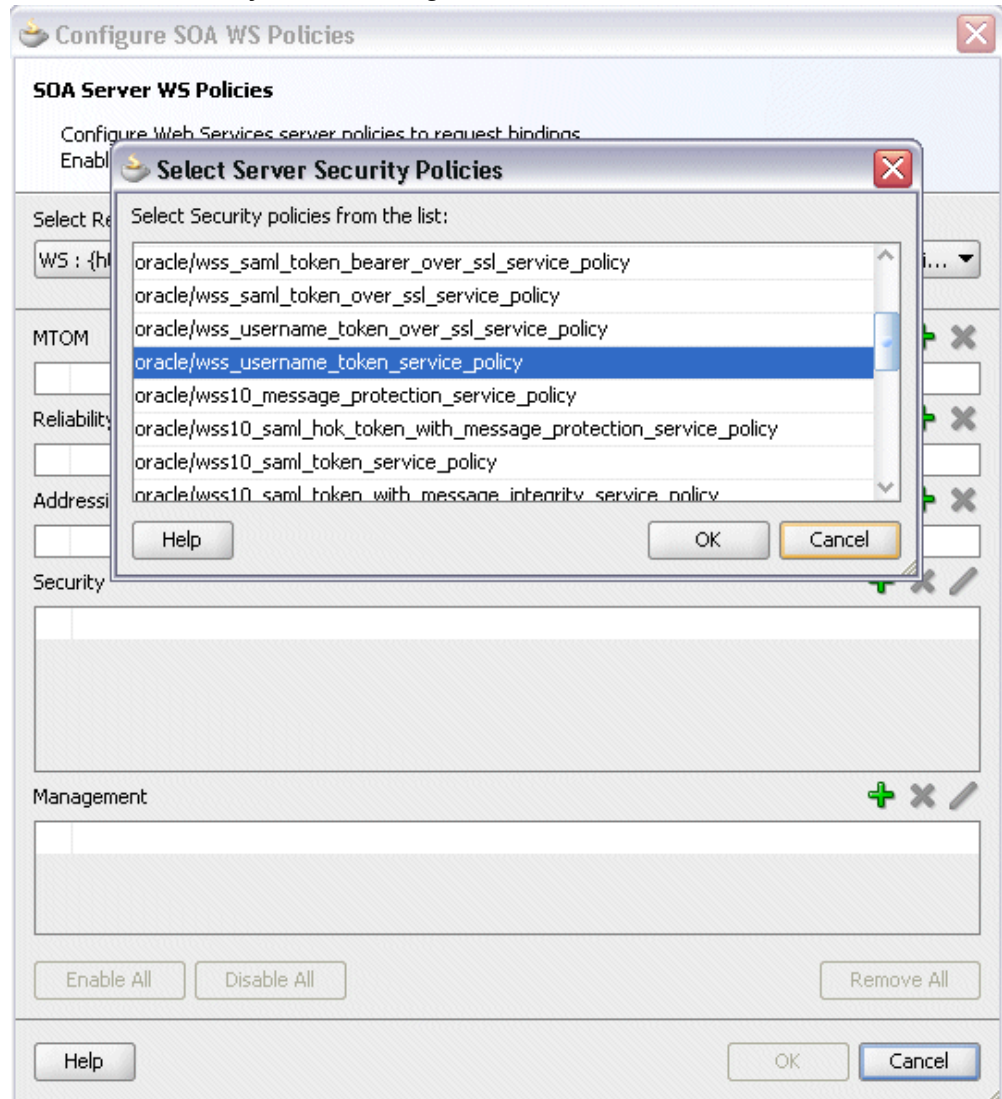
Configuring Web Service Policies

Use the following steps to add security policies at design time:

1. Navigate to SOA Content > Business Rules > composite.xml. Right click on the ONT__POI service and select "Configure WS Policies" from the drop-down list.
2. The Configure SOA WS Policies dialog appears.

In the Security section, click the **Add** icon (+). The Select Server Security Policies dialog appears.

Select Server Security Policies Dialog



Select 'oracle/wss_username_token_service_policy' and click **OK**.

The attached security policy is shown in the Security section.

A lock icon appears in the `ONT__POI` service of the `composite.xml` indicating that a security policy has been successfully attached.

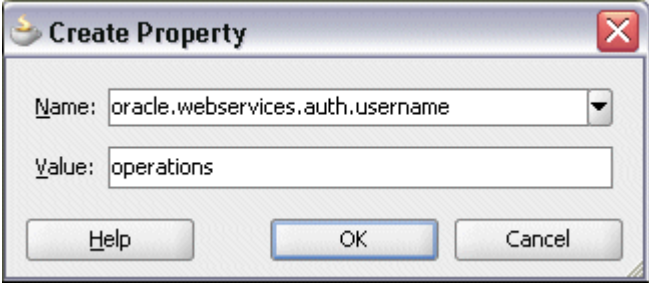
3. From the navigation menu, select **View > Property Inspector** to display the Property Inspector window for `ONT__POI` service component.

In the Properties section, click the **Add** icon (+) for binding properties. The Create Property dialog appears.

Enter 'oracle.webservices.auth.username' in the Name field and enter

'operations' as the value.

Create Property Dialog

A screenshot of the 'Create Property' dialog box. It has a title bar with a close button. Inside, there are two text fields: 'Name' and 'Value'. The 'Name' field contains 'oracle.webservices.auth.username' and has a dropdown arrow on the right. The 'Value' field contains 'operations'. At the bottom, there are three buttons: 'Help', 'OK', and 'Cancel'. The 'OK' button is highlighted with a blue border.

Click **OK**.

4. Use the same approach by clicking the **Add** icon (+) again in the Properties section for binding properties. Enter 'oracle.webservices.auth.password' in the Name field. Enter the associated password for user 'operations' in the Value field.

Click **OK**.

Both selected property names and values appear in the Properties section.

Click the Source tab of `composite.xml` and notice that the `oracle.webservices.auth.username` and `oracle.webservices.auth.password` property names and the associated values are added to the `ONT__POI` reference.

Property Information Shown in the composite.xml File

```
30 <reference name="ONT__POI" ui:wsdlLocation="ONT__POI.wsdl">
31 <interface.wsdl interface="http://xmlns.oracle.com/apps/cin/soapprovider/xmlgateway/ont__poi/#wsdl.interface(ON
32 <binding.ws port="http://xmlns.oracle.com/apps/cin/soapprovider/xmlgateway/ont__poi/#wsdl.endpoint(ONT__POI_Ser
33 location="ONT__POI.wsdl" soapVersion="1.1">
34 <wsp:PolicyReference URI="oracle/wss_username_token_client_policy"
35 orawsp:category="security" orawsp:status="enabled"/>
36 <property name="oracle.webservices.auth.username" type="xs:string"
37 many="false" override="may">operations</property>
38 <property name="oracle.webservices.auth.password" type="xs:string"
39 many="false" override="may">password</property>
40 </binding.ws>
41 </reference>
```

Deploying and Testing the SOA Composite with BPEL Process at Run Time

To invoke the synchronous XML Gateway inbound map (PROCESS_PO_007) service from the BPEL client contained in the SOA composite, the SOA composite needs to be deployed to the Oracle WebLogic managed server. This can be achieved using Oracle JDeveloper. Once the composite is deployed, it can be tested from the Oracle Enterprise Manager Fusion Middleware Control Console.

Prerequisites

Before deploying the SOA composite with BPEL process using Oracle JDeveloper, you must have established the connectivity between the design-time environment and the

run-time server. For information on how to configure the necessary server connection, see *Configuring Server Connection, Oracle E-Business Suite Integrated SOA Gateway Developer's Guide*.

Note: If a local instance of the WebLogic Server is used, start the WebLogic Server by selecting **Run > Start Server Instance** from Oracle JDeveloper. Once the WebLogic Admin Server "DefaultServer" instance is successfully started, the <Server started in Running mode> and DefaultServer started message in the Running:DefaultServer and Messages logs should appear.

To validate your BPEL process, perform the following run-time tasks:

1. Deploy the SOA Composite Application with BPEL Process, page 8-49
2. Test the SOA Composite Application with BPEL Process, page 8-52

Deploying the SOA Composite Application with BPEL Process

You must deploy the SOA composite application with BPEL process (`XMLGatewayInbound.bpel`) that you created earlier before you can run it.

Note: Before deploying the SOA composite application with BPEL Process for XML Gateway Inbound service, you should:

- Load the `order_data_xm1g.xml` file into the specified directory `'/usr/tmp/'` folder of SOA Suite server (or `D:\HOL` in case of SOA Server in Windows machine).
- Edit the input file `order_data_xm1g.xml` by entering values for <REFERENCEID> and <POID> such as `'order_xml_01'`.

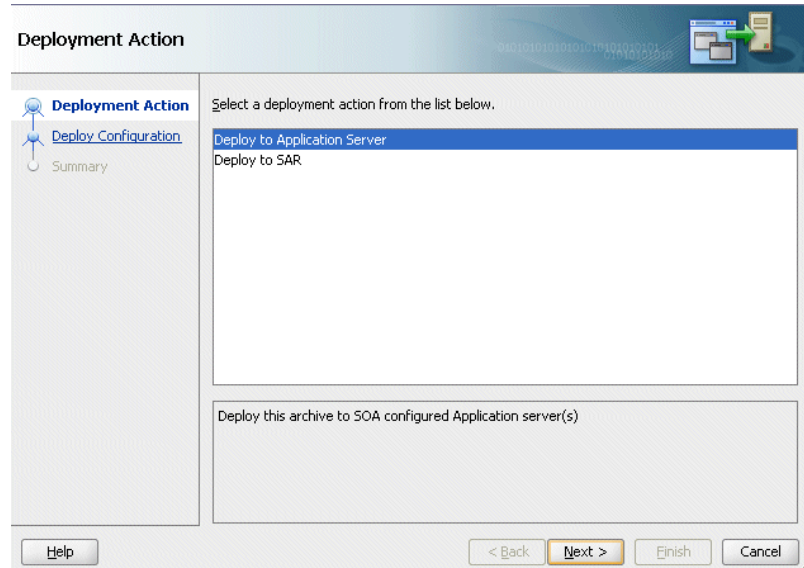
To deploy the SOA composite application with BPEL process:

1. In the Applications Navigator of Oracle JDeveloper, select the **XMLGInbound** project.
2. Right-click the project and select **Deploy > [project name] > [serverConnection]** from the menu.

For example, you can select **Deploy > XMLGInbound > SOAServer** to deploy the process if you have the connection appropriately.

Note: If this is the first time to set up the server connection, then the Deployment Action dialog appears. Select 'Deploy to Application Server' and click **Next**.

Deployment Action Dialog



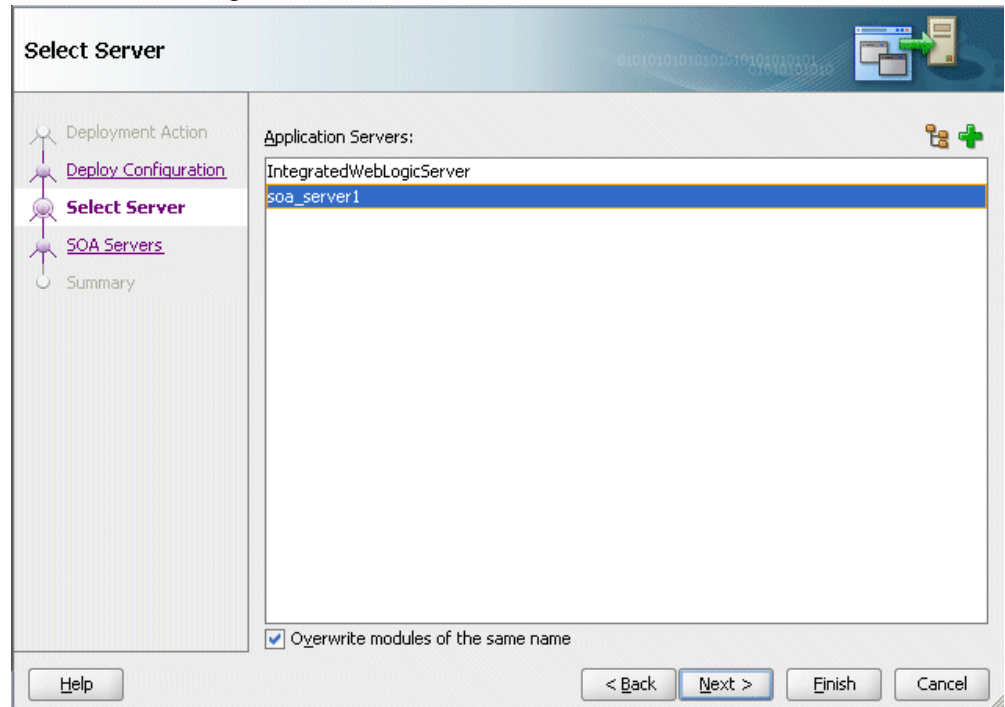
In the Deploy Configuration dialog, ensure the following information is selected before clicking **Next** to add a new application server:

- New Revision ID: 1.0
- Mark composite revision as default: Select this checkbox.
- Overwrite any existing composites with the same revision ID: Select this checkbox.

For information on how to create a new server connection for Oracle WebLogic Server, see *Configuring Server Connection, Oracle E-Business Suite Integrated SOA Gateway Developer's Guide*.

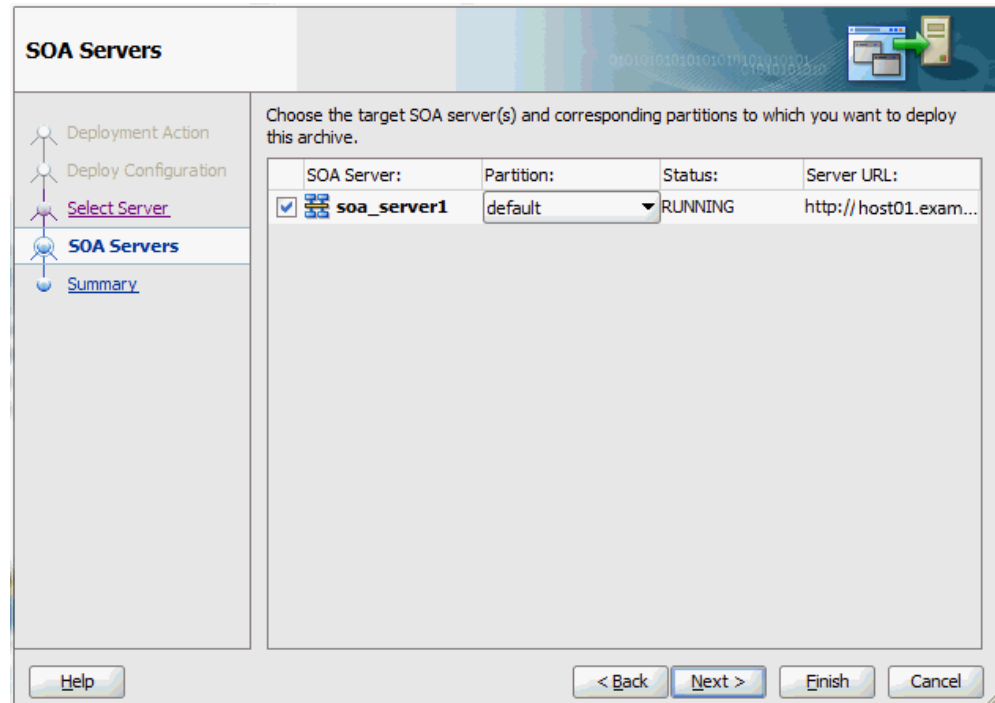
3. In the Select Server dialog, select 'soa-server1' that you have established the server connection earlier. Click **Next**.

Select Server Dialog



4. In the SOA Servers dialog, accept the default target SOA Server ('soa-server1') selection.

SOA Servers Dialog

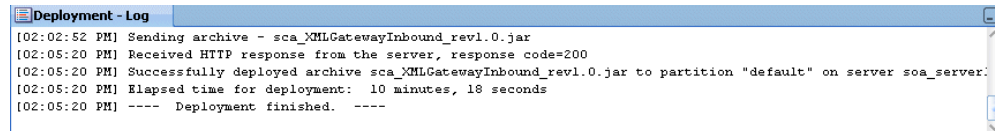


Click **Next** and **Finish**.

5. If you are deploying the composite for the first time from your Oracle JDeveloper session, the Authorization Request window appears. Enter username and password information specified during Oracle SOA Suite installation. Click **OK**.
6. Deployment processing starts. Monitor deployment process and check for successful compilation in the SOA - Log window.

Verify that the deployment is successful in the Deployment - Log window.

Deployment - Log Tab



Testing the SOA Composite Application with BPEL Process

Once the BPEL process contained in the SOA composite application has been successfully deployed, you can manage and monitor the process from Oracle Enterprise Manager Fusion Middleware Control Console. You can also test the process and the integration interface by manually initiating the process.

For more information about Oracle SOA Suite, see the *Oracle Fusion Middleware Administrator's Guide for Oracle SOA Suite*.

To test the SOA composite application with BPEL process:

1. Navigate to Oracle Enterprise Manager Fusion Middleware Control Console (<http://<hostname>:<port>/em>). The login page appears.
2. Enter the username and password information specified during the installation, and then click **Login** to log in to a farm. The composite (XMLGInbound) you deployed is displayed in the Applications Navigation tree.

You may need to select an appropriate target instance farm if there are multiple target Oracle Enterprise Manager Fusion Middleware Control Console farms.

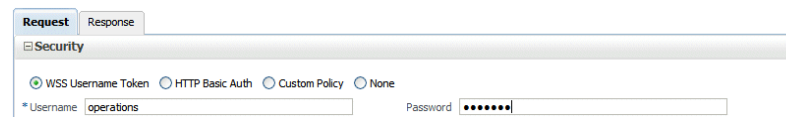
3. From the Farm navigation pane, expand the SOA >soa-infra node in the tree to navigate through the SOA Infrastructure home page and menu to access your deployed SOA composite applications running on soa-infra managed server.

Click the XMLGInbound [1.0] link.

4. Click the Policies tab and notice that the 'oracle/wss_username_token_service_policy' policy you attached to the ONT__POI service binding earlier at the design time is now displayed here.
5. In the XMLGInbound [1.0] home page, click **Test**.
6. The Test Web Service page for initiating an instance appears.

Note: If the WS-Security credentials are not entered at design time, you can enter the credentials at run time by selecting the WSS Username Token option in the Security section at the top of the Request tab. Enter 'operations' in the Username field and the associated password for user 'operations' in the Password field.

WS-Security Credentials in the Request Tab



The screenshot shows a web interface for testing a web service. At the top, there are two tabs: 'Request' and 'Response'. Below the tabs, there is a 'Security' section. Under 'Security', there are four radio buttons: 'WSS Username Token' (which is selected), 'HTTP Basic Auth', 'Custom Policy', and 'None'. Below these radio buttons, there are two input fields. The first field is labeled '* Username' and contains the text 'operations'. The second field is labeled 'Password' and contains a series of dots, indicating a masked password.

7. Enter the input string (such as 'test') required by the process and click **Test Web Service** to initiate the process.

The test results appear in the Response tab upon completion.

8. Click the Instances tab. The SOA composite application instance ID, name, conversation ID, most recent known state of each instance since the last data refresh

of the page are displayed.

9. Click your BPEL service component instance link (such as XMLGInbound) to display the Instances page where you can view execution details for the BPEL activities in the Audit Trail tab.

Click the Flow tab to check the BPEL process flow diagram. Click an activity of the process diagram to view the activity details and flow of the payload through the process.

Verifying Records in Oracle E-Business Suite

Once the BPEL process has been successfully initiated and completed, you can validate the transaction in Oracle E-Business Suite.

To Transmit the XML Payload Using Oracle Transport Agent (OTA)

Oracle Transport Agent is a lightweight messaging platform for transmitting documents over HTTP and Secure HTTP (HTTPS). Refer to Oracle Transport Agent, page 7-1 for details.

Perform the following steps to attach the payload for the inbound transaction:

1. Navigate to the Oracle Transport Agent Inbound Test Page (http://<EBS_hostname>:<port>/webservices/ECX0TAInbound).

Important: As part of the January 2017 CPU update, the access to the Oracle Transport Agent Inbound Test Page is restricted to trusted nodes or servers. Therefore, to use this page to send an inbound XML message, you should add a fully-qualified node name or IP address as a trusted node in Oracle E-Business Suite. To extend access to additional hosts, perform the following tasks:

1. Add space-separated node names, fully qualified node names, or client machine IP addresses to the existing content of the `s_admin_ui_access_nodes` context variable.
2. Run AutoConfig.
3. Restart the Oracle HTTP Server.

For more information, refer to the Known Issues section in My Oracle Support Knowledge Document 2212223.1, *Oracle E-Business Suite Release 12 Critical Patch Update Knowledge Document (January 2017)* for details.

Oracle Transport Agent Inbound Test Page

USERNAME	operations
PASSWORD	••••••
TRANSPORT_PROTOCOL	OXTA
TRANSPORT_PROTOCOL_VERSION	1.0
REQUEST_TYPE	SEND
MESSAGE_ID	
MESSAGE_TYPE	XML
MESSAGE_STANDARD	OAG
TRANSACTION_TYPE	PO
TRANSACTION_SUBTYPE	PROCESS
DOCUMENT_NUMBER	
PARTYID	1004
SOURCE_TP_LOCATION_CODE	Example-01
PROTOCOL_TYPE	<i>(not needed for Inbound)</i>
PROTOCOL_ADDRESS	<i>((needed only if REQUEST_TYPE=EME))</i>
ATTRIBUTE1	
ATTRIBUTE2	
ATTRIBUTE3	
ATTRIBUTE4	
ATTRIBUTE5	

2. Enter the following information:

- USERNAME: 'operations'
- PASSWORD: Enter the associated password for the user 'operations'.
- REQUEST_TYPE: SEND
- MESSAGE_STANDARD: OAG
- TRANSACTION_TYPE: PO
- TRANSACTION_SUBTYPE: PROCESS
- PARTYID: 1004
- SOURCE_TP_LOCATION_CODE: Example-01

3. Enter the payload information in the Payload field and click **Submit**.

For the payload information, see Sample Payload for Inbound Process Purchase Order XML Transaction, *Oracle E-Business Suite Integrated SOA Gateway Developer's Guide*.

Specify Payload Information

ATTRIBUTES

```
index="1"/></CONTACT>
  </PARTNER>
  <PARTNER><NAME
index="1"/><ONETIME/><PARTNRID/><PARTNRTYPE>DeliveryTo</PARTNRTYPE><PARTNRIDX/><USERAREA/><ADDRESS>
<ADDRLINE index="1"/><ADDRLINE index="2"/><ADDRTYPE/></ADDRESS><CONTACT><NAME
index="1"/><CONTACTTYPE/><EMAIL/><FAX index="1"/><TELEPHONE index="1"/></CONTACT>

</PARTNER><DROPSHIPDETAILS><DROPSHIPMENT/><DROPSHIPCUSTOMERNAME/><SHIPINSTR/><PACKINSTR/><SHIPMETHOD/><
CUSTOMERPONUM/><CUSTOMERLINENUM/><CUSTOMERSHIPNUM/><CUSTOMERDESC/></DROPSHIPDETAILS><CONSIGNEDINV>N
</CONSIGNEDINV><DISTPROJECT><REQUESTOR/><DISTNUM>1</DISTNUM><PROJECTNUM/><PROJECTTYPE/><TASKNUM/><Q
UANTITY
qualifier="ORDERED"><VALUE>12</VALUE><NUMOFDEC/><SIGN>+</SIGN><UOM>Ea</UOM></QUANTITY><CONVRATE/><D
ATETIME
qualifier="EXCHRATEDATE"><YEAR>2011</YEAR><MONTH>04</MONTH><DAY>05</DAY><HOUR>00</HOUR><MINUTE>00</
MINUTE><SECOND>00</SECOND><SUBSECOND>0000</SUBSECOND><TIMEZONE>+0000</TIMEZONE></DATETIME><DESTTYPE
>INVENTORY</DESTTYPE><DFFDISTRIBUTN><ATTRIBUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/><ATTRIBUTE5
/><ATTRIBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIBUTE10/><ATTRIBUTE11/><ATTRIBUTE12/><AT
TRIBUTE13/><ATTRIBUTE14/><ATTRIBUTE15/><ATTRIBUTE16/></DFFDISTRIBUTN></DISTPROJECT><AMOUNT
qualifier="TOTAL" type="T"><VALUE/><NUMOFDEC/><SIGN/><CURRENCY/><DRCR>C</DRCR></AMOUNT></USERAREA>
  </POLINESCHD>
</POORDERLIN>
  </PROCESS_PO>
</DATAAREA>
</PROCESS_PO_007>
```

Submit

The status code '1000' appears indicating that the request is successfully handled.

Successful Transmit the XML Payload

STATUS_CODE 1000

STATUS_DESCRIPTION OK

MESSAGE_RECEIPT_ID

To Validate the Transaction in Oracle Transaction Monitor:

You can validate it from the Transaction Monitor. The Transaction Monitor is a tool for monitoring the status of inbound and outbound transactions originating from and going into Oracle E-Business Suite that have been processed by the XML Gateway and delivered or received by the Oracle Transport Agent. It shows a complete history and audit trail of these documents.

1. Log in to Oracle E-Business Suite as a user who has the Workflow Administrator Web Applications responsibility.

Select the Transaction Monitor link to open the search window to search for the order.

2. Clear From Date and To Date fields and enter 'PO-4466-5' in the Document ID field.
3. Select Customer as the Party Type. Click **Go** to execute the search.

This retrieves XML inbound transaction 'PO-4466-5' in the Inbound Search Results region.

4. Confirm that the transaction 'PO-4466-5' has status 'SUCCESS'.

You can verify it by logging in to Oracle E-Business Suite as a user who has the Order Management Super User, Vision Operations (USA) responsibility. Select **Orders, Returns : Import Orders > Corrections**. The Find Orders window is displayed and select 'XML' from drop-down list in the Order Source field. The Corrections window is displayed with all the transactions with the XML order source.

Notice that 'PO_4466-5' is listed in the Orig_Sys_Document_Reference field.

Corrections Window

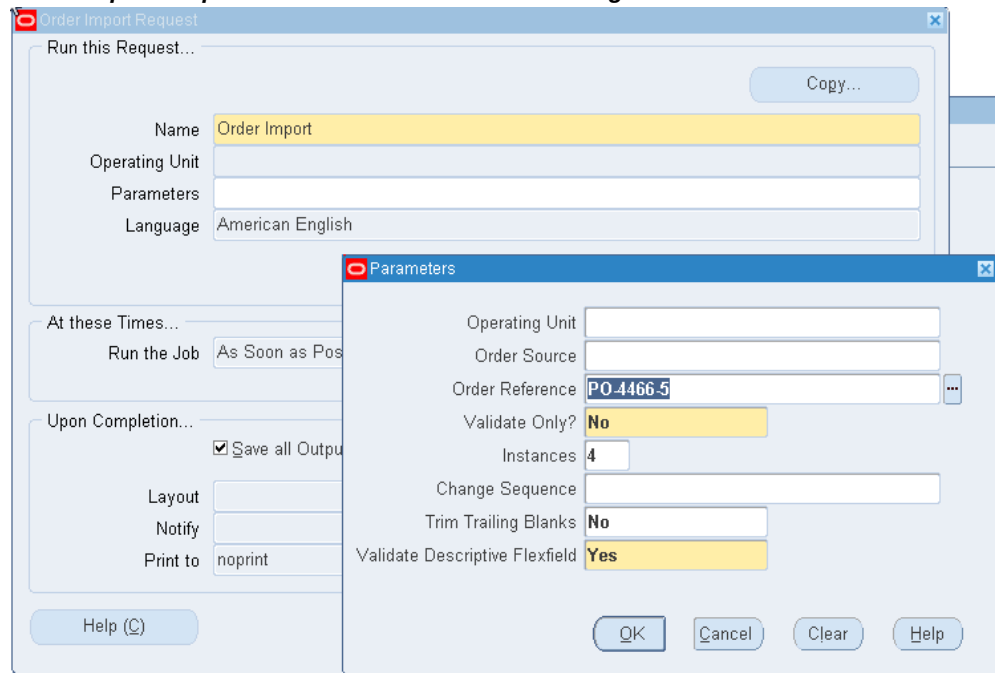
Order S	Orig Sys Docu	Change	Request ID	Operation Code	Order Number	Sold To Org ID
20	PO-4466-5	Change				
20	order_id_D1	Change				
20	order_id_D1	Change				

Payments Discounts Pricing Attributes Sales Credits
Actions Add Customers Errors Validate
Lines Import

To Import the Order to Oracle Order Management:

1. Log in to the Forms-based Oracle E-Business Suite as a user who has the Order Management Super User, Vision Operations (USA) responsibility.
2. Select **Orders, Returns : Import Orders > Order Import Request**. The Order Import Request window is displayed along with the Parameters dialog.
Select 'PO-4466-5' from the Order Reference drop-down list.

Order Import Request Window and Parameters Dialog



3. Click **OK** in the parameters dialog. The Order Import request name is populated automatically in the Import Request window.
4. Click **Submit** to submit the request. This displays a concurrent request number. Record the request number, but click **No** in the Decision dialog that you will not submit another request.
5. From the application menu, select **View > Requests** to open the Find Requests window.
6. Enter the request number you recorded earlier and click **Find**.

This would show the status of Order Import request. It should be success and the order should be created in Order Management application.

To Validate the Transaction in Oracle Order Management:

1. Log in to the Forms-based Oracle E-Business Suite as a user who has the Order Management, Super User responsibility.
2. Select **Order Returns > Sales Order**. The Sales Order Forms is displayed.
3. Search for an order by entering the order number in the Customer PO field (such as 'PO-4466-5'). The details of a newly created order appears.

Sales Orders Window

Sales Orders (Vision Operations) - 85182

Order Information Line Items

Default

Main Others

Customer		Order Number	
Customer Number		Order Type	Mixed
Customer PO	PO-4466-5	Date Ordered	04-SEP-2012 23:26:29
Customer Contact		Price List	Corporate
Ship To Location		Salesperson	
		Status	Entered
		Currency	USD
Bill To Location		Subtotal	26,156.40
		Tax	0.00
		Charges	150.00
		Total	26,306.40

[] ☐

Actions Related Items Configurator Availability Book Order

Prerequisite Files

Sample Payload: Inbound Purchase Order

The following information shows the sample payload in the `order_data.xmlg.xml` file:

```

<?xml version = '1.0' encoding = 'UTF-8' standalone = 'no'?>
<!-- Oracle eXtensible Markup Language Gateway Server -->
<!DOCTYPE PROCESS_PO_007 SYSTEM "003_process_po_007.dtd">
  <PROCESS_PO_007>
    <CNTRLAREA>
      <BSR>
        <VERB>PROCESS</VERB>
        <NOUN>PO</NOUN>
      <REVISION>007</REVISION>
    </BSR>
    <SENDER>
      <LOGICALID>ORACLE</LOGICALID>
      <COMPONENT>PURCHASING</COMPONENT>
      <TASK>POISSUE</TASK>
      <REFERENCEID>M04YD220.US.EXAMPLE.COM:oracle.apps.po.event.xmlpo:
32636-148970</REFERENCEID>
      <CONFIRMATION>0</CONFIRMATION>
      <LANGUAGE>us</LANGUAGE>
      <CODEPAGE>UTF8</CODEPAGE>
      <AUTHID>APPS</AUTHID>
    </SENDER>
    <DATETIME qualifier="CREATION" type="T" index="1">
      <YEAR>2011</YEAR>
      <MONTH>04</MONTH>
      <DAY>05</DAY>
      <HOUR>23</HOUR>
      <MINUTE>44</MINUTE>
      <SECOND>09</SECOND>
      <SUBSECOND>0000</SUBSECOND>
      <TIMEZONE>+0000</TIMEZONE>
    </DATETIME>
    <OPERAMT qualifier="EXTENDED" type="T">
      <VALUE>1329432</VALUE>
      <NUMOFDEC>2</NUMOFDEC>
      <SIGN>+</SIGN>
      <CURRENCY>USD</CURRENCY>
      <UOMVALUE>1</UOMVALUE>
      <UOMNUMDEC>0</UOMNUMDEC>
      <UOM/>
    </OPERAMT>
    <POID>PO-4466-5</POID>
    <POTYPE>STANDARD</POTYPE>
    <ACKREQUEST>N</ACKREQUEST>
    <CONTRACTB/>
    <CONTRACTS/>
    <DESCRIPTN/>
    <PORELEASE/>
    <USERAREA><DATETIME qualifier="ACTSTART" type="T" index="1"
><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
DATETIME><DATETIME qualifier="ACTEND" type="T" index="1"
><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
DATETIME><DATETIME><FOB><DESCRIPTN>Vendor's responsibility ceases upon
transfer to
carrier</DESCRIPTN><TERMD>Origin</TERMD></FOB><TANDC/><FTTERM><DESCRIP
TN>Buyer pays
freight</DESCRIPTN><TERMD>Due</TERMD></FTTERM><EXCHRATE/><DATETIME
qualifier="EXCHRATE"
><YEAR>2011</YEAR><MONTH>04</MONTH><DAY>05</DAY><HOUR>00</HOUR><MINUTE>0
0</MINUTE><SECOND>00</SECOND><SUBSECOND>0000</SUBSECOND><TIMEZONE>+0000<
/
TIMEZONE></DATETIME><DATETIME ualifier="APPREQ" type="T" index="1"
><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
DATETIME><CONFIRM>N</CONFIRM><SHIPPINGCONTROL/><DFFPOHEADER><ATTRIBUTE1/
><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/><ATTRIBUTE5/><ATTRIBUTE6/><ATTRI
BUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIBUTE10/><ATTRIBUTE11/><ATTRIBUTE1
2/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUTE15/><ATTRIBUTE16/></DFFPOHEADER
><PCARDHDR><MEMBERNAME/><PCARDNUM>0</PCARDNUM><DATETIME qualifier="

```

```

EXPIRATION"
><YEAR>2011</YEAR><MONTH>04</MONTH><DAY>05</DAY><HOUR>00</HOUR><MINUTE>0
0</MINUTE><SECOND>00</SECOND><SUBSECOND>0000</SUBSECOND><TIMEZONE>+0000<
/TIMEZONE></DATETIME><PCARDBRAND/></PCARDHDR></USERAREA>
<PARTNER>
  <NAME index="1">Example Inc.</NAME>
  <ONETIME>0</ONETIME>
  <PARTNRID><PARTNRID/>
  <PARTNRTYPE>Supplier</PARTNRTYPE>
  <CURRENCY>USD</CURRENCY>
  <DUNSNUMBER/>
  <PARTNRIDX>Example-01</PARTNRIDX>
  <TAXID/>
  <TERMID/>

<USERAREA><DFFVENDOR><ATTRIBUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/
><ATTRIBUTE5/><ATTRIBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRI
BUTE10/><ATTRIBUTE11/><ATTRIBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBU
TE15/><ATTRIBUTE16/></DFFVENDOR><CUSTOMERNUM/></USERAREA>
<ADDRESS>
  <ADDRLINE index="1"></ADDRLINE>
  <ADDRLINE index="2"/>
  <ADDRLINE index="3"/>
  <ADDRTYPE/>
  <CITY></CITY>
  <COUNTRY></COUNTRY>
  <COUNTY/>
  <DESCRIPTN></DESCRIPTN>
  <FAX index="1"/>
  <POSTALCODE></POSTALCODE>
  <REGION/>
  <STATEPROVN></STATEPROVN>
  <TAXJRSRCTN/>
  <TELEPHONE index="1"></TELEPHONE>
  <TELEPHONE index="2"/>
  <TELEPHONE index="3"/>
  <URL/>

<USERAREA><DFFVENDORSITE><ATTRIBUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBU
TE4/><ATTRIBUTE5/><ATTRIBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><A
TTRIBUTE10/><ATTRIBUTE11/><ATTRIBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATT
RIBUTE15/><ATTRIBUTE16/></DFFVENDORSITE></USERAREA>
</ADDRESS>
<CONTACT>
  <NAME index="1"/>
  <EMAIL/>
  <FAX index="1"></FAX>
  <TELEPHONE index="1"></TELEPHONE>
</CONTACT>
</PARTNER>
<PARTNER>
  <NAME index="1">Vision Operation</NAME>
  <ONETIME>0</ONETIME>
  <PARTNRID>204<PARTNRID/>
  <PARTNRTYPE>SoldTo</PARTNRTYPE>
  <CURRENCY>USD</CURRENCY>
  <DUNSNUMBER/>
  <PARTNRIDX>Example-01</PARTNRIDX>
  <PAYMETHOD/>
  <TAXID/>
  <TERMID/>
  <USERAREA/>
  <ADDRESS>
  </ADDRESS>
  <CONTACT>
    <NAME index="1">Lastname, Ms. Firstname</NAME>

```

```

<CONTACTTYPE/>
  <DESCRIPTN/>
  <EMAIL>username@example.com</EMAIL>
  <FAX index="1"/>
  <TELEPHONE index="1"/>
  <USERAREA/>
</CONTACT>
</PARTNER>
<PARTNER>
  <NAME index="1">Vision Operation</NAME>
  <ONETIME>0</ONETIME>
  <PARTNRID>204</PARTNRID>
  <PARTNRTYPE>BillTo</PARTNRTYPE>
  <CURRENCY/>
  <DUNSNUMBER/>
  <PARTNRIDX>Eample-01</PARTNRIDX>
  <PAYMETHOD/>
  <TERMID/>
  <USERAREA/>
  <ADDRESS>
  </ADDRESS>
</PARTNER>
<PARTNER>
  <NAME index="1">UPS</NAME>
  <ONETIME>0</ONETIME>
  <PARTNRID>204</PARTNRID>
  <PARTNRTYPE>Carrier</PARTNRTYPE>
  <PARTNRIDX>UPS</PARTNRIDX>
</PARTNER>
<POTERM>
  <DESCRIPTN>Scheduled for payment 30 days from the invoice date
(invoice terms date = system date, goods received date, invoice date or
invoice received date). Invoice terms date can default from supplier
header, site, PO, system default, etc.</DESCRIPTN>
  <TERMID>30 Net (terms date + 30)</TERMID>
  <DAYSNUM/>
  <QUANTITY qualifier="PERCENT">
    <VALUE/>
    <NUMOFDEC/>
    <SIGN/>
    <UOM/>
  </QUANTITY>
  <USERAREA/>
</POTERM>
</POORDERHDR>
<POORDERLIN>
  <QUANTITY qualifier="ORDERED">
    <VALUE>12</VALUE>
    <NUMOFDEC/>
    <SIGN>+</SIGN>
    <UOM>Ea</UOM>
  </QUANTITY>
  <OPERAMT qualifier="UNIT" type="T">
    <VALUE>110786</VALUE>
    <NUMOFDEC>2</NUMOFDEC>
    <SIGN>+</SIGN>
    <CURRENCY>USD</CURRENCY>
    <UOMVALUE>1</UOMVALUE>
    <UOMNUMDEC>0</UOMNUMDEC>
    <UOM>Ea</UOM>
  </OPERAMT>
  <POLINENUM>1</POLINENUM>
  <HAZRDMATL/>
  <ITEMRV></ITEMRV>
  <ITEMRVX/>
  <POLNSTATUS/>

```

```

<DESCRIPTN></DESCRIPTN>
  <ITEM>AS54888</ITEM>
  <ITEMX/>
  <USERAREA><REQUESTOR/><CATEGORYID>PRODUCTN.
  FINGOODS</CATEGORYID><CONTRACTNUM/><CONTRACTPONUM/><CONTRACTPOLINENUM/><
  VENDORQUOTENUM/><CONFIGID/><LISTPRICE>1107.
  86</LISTPRICE><MARKETPRICE>0</MARKETPRICE><PRICENOTTOEXCEED/><NEGPRICE>N
  </NEGPRICE><TAXABLE>N</TAXABLE><TXNREASONCODE/><TYPE1099>AVAN/N</TYPE109
  9><LINEORDERTYPE>Goods</LINEORDERTYPE><HAZRDUNNUM/><HAZRDUNDESC/><DFFLIN
  E><ATTRIBUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/><ATTRIBUTE5/><ATTR
  IBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIBUTE10/><ATTRIBUTE1
  1/><ATTRIBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUTE15/><ATTRIBUTE16/
  ></DFFLINE><DFFITEM><ATTRIBUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/>
  <ATTRIBUTE5/><ATTRIBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIB
  UTE10/><ATTRIBUTE11/><ATTRIBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUT
  E15/><ATTRIBUTE16/></DFFITEM><KFFITEM><ATTRIBUTE1>PRODUCTN</ATTRIBUTE1><
  ATTRIBUTE2>FINGOODS</ATTRIBUTE2><ATTRIBUTE3/><ATTRIBUTE4/><ATTRIBUTE5/><
  ATTRIBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIBUTE10/><ATTRIB
  UTE11/><ATTRIBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUTE15/><ATTRIBUT
  E16/><ATTRIBUTE17/><ATTRIBUTE18/><ATTRIBUTE19/><ATTRIBUTE20/></KFFITEM><
  GLOBALCONTRACT/><GLOBALCONTRACTLIN/><JOBTITLE/><AMOUNT qualifier="TOTAL"
  type="T"
  ><VALUE/><NUMOFDEC/><SIGN/><CURRENCY/><DRCR>C</DRCR></AMOUNT><CONTRACTOR
  FIRSTNAME/><CONTRACTORLASTNAME/><DATETIME qualifier="ACTSTART" type="T"
  index="1"
  ><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
  DATETIME><DATETIME qualifier="ACTEND" type="T" index="1"
  ><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
  DATETIME></USERAREA>
  <POLINESCHD>
    <DATETIME qualifier="NEEDELV" type="T" index="1">
      <YEAR>2011</YEAR>
      <MONTH>04</MONTH>
      <DAY>21</DAY>
      <HOUR>00</HOUR>
      <MINUTE>00</MINUTE>
      <SECOND>00</SECOND>
      <SUBSECOND>0000</SUBSECOND>
      <TIMEZONE>+0000</TIMEZONE>
    </DATETIME>
    <QUANTITY qualifier="ORDERED">
      <VALUE>12</VALUE>
      <NUMOFDEC/>
      <SIGN>+</SIGN>
      <UOM>Ea</UOM>
      </QUANTITY>
    <DESCRIPTN/>
    <PSCLINENUM>1</PSCLINENUM>
    <USERAREA><DATETIME qualifier="PROMSHIP"
    ><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
    DATETIME><DATETIME qualifier="APPROVAL"
    ><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
    DATETIME><OPERAMT qualifier="UNIT" type="T"
    ><VALUE>110786</VALUE><NUMOFDEC>2</NUMOFDEC><SIGN>+</SIGN><CURRENCY>USD<
    /CURRENCY><UOMVALUE>1</UOMVALUE><UOMNUMDEC>0</UOMNUMDEC><UOM>Ea</UOM></O
    PERAMT><PRICEOVRD/><TAXABLE>N</TAXABLE><TAXCODE/><PARTNER><NAME index="
    1">Vision
    Operations</NAME><ONETIME>0</ONETIME><PARTNRID>204</PARTNRID><PARTNRTYPE
    >ShipTo</PARTNRTYPE><CURRENCY>USD</CURRENCY><DUNSNUMBER/><PARTNRIDX>Exam
    ple-01</PARTNRIDX><PAYMETHOD/><TERMID/><USERAREA/>
    <ADDRESS></ADDRESS>
    <CONTACT><NAME index="1"/><CONTACTTYPE/><EMAIL/><FAX index="1"
    /><TELEPHONE index="1"/></CONTACT>
    </PARTNER>
    <PARTNER><NAME index="1"
    /><ONETIME/><PARTNRID/><PARTNRTYPE>DeliveryTo</PARTNRTYPE><PARTNRIDX/><U

```

```

SERAREA/><ADDRESS><ADDRLINE index="1"/><ADDRLINE index="2"
/><ADDRTYPE/></ADDRESS><CONTACT><NAME index="1"
/><CONTACTTYPE/><EMAIL/><FAX index="1"/><TELEPHONE index="1"/>
</CONTACT></PARTNER><DROPSHIPDETAILS><DROPSHIPMENT/><DROPSHIPCUSTNAME/><
SHIPINSTR/><PACKINSTR/><SHIPMETHOD/><CUSTOMERPONUM/><CUSTOMERLINENUM/><C
USTOMERSHIPNUM/><CUSTOMERDESC/></DROPSHIPDETAILS><CONSIGNEDINV>N</CONSIG
NEDINV><DISTPROJECT><REQUESTOR/><DISTNUM>1</DISTNUM><PROJECTNUM/><PROJEC
TTYPE/><TASKNUM/><QUANTITY qualifier="ORDERED"
><VALUE>12</VALUE><NUMOFDEC/><SIGN>+</SIGN><UOM>Ea</UOM></QUANTITY><CONV
RATE/><DATETIME qualifier="EXCHRATEDATE"
><YEAR>2011</YEAR><MONTH>04</MONTH><DAY>05</DAY><HOUR>00</HOUR><MINUTE>0
0</MINUTE><SECOND>00</SECOND><SUBSECOND>0000</SUBSECOND><TIMEZONE>+0000<
/TIMEZONE></DATETIME><DESTTYPE>INVENTORY</DESTTYPE><DFFDISTRIBUTN><ATTRI
BUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/><ATTRIBUTE5/><ATTRIBUTE6/>
<ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIBUTE10/><ATTRIBUTE11/><ATTR
IBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUTE15/><ATTRIBUTE16/></DFFDI
STRIBUTN></DISTPROJECT><AMOUNT qualifier="TOTAL" type="T"
><VALUE/><NUMOFDEC/><SIGN/><CURRENCY/><DRCR>C</DRCR></AMOUNT></USERAREA>

</POLINESCHD>
</POORDERLIN>
<POORDERLIN>
  <QUANTITY qualifier="ORDERED">
    <VALUE>13</VALUE>
    <NUMOFDEC/>
    <SIGN>+</SIGN>
    <UOM>Ea</UOM>
    </QUANTITY>
  <OPERAMT qualifier="UNIT" type="T">
    <VALUE>110786</VALUE>
    <NUMOFDEC>2</NUMOFDEC>
    <SIGN>+</SIGN>
    <CURRENCY>USD</CURRENCY>
    <UOMVALUE>1</UOMVALUE>
    <UOMNUMDEC>0</UOMNUMDEC>
    <UOM>Ea</UOM>
  </OPERAMT>
  <POLINENUM>2</POLINENUM>
  <HAZRDMATL/>
  <ITEMRV></ITEMRV>
  <ITEMRVX/>
  <POLNSTATUS/>
  <DESCRIPTN></DESCRIPTN>
  <ITEM>AS00021</ITEM>
  <ITEMX/>
  <USERAREA><REQUESTOR/><CATEGORYID>PRODUCTN.
  FINGOODS</CATEGORYID><CONTRACTNUM/><CONTRACTPONUM/><CONTRACTPOLINENUM/><
  VENDORQUOTENUM/><CONFIGID/><LISTPRICE>1107.
  86</LISTPRICE><MARKETPRICE>0</MARKETPRICE><PRICENOTTOEXCEED/><NEGPRICE>N
  </NEGPRICE><TAXABLE>N</TAXABLE><TXNREASONCODE/><TYPE1099>AVAN/N</TYPE109
  9><LINEORDERTYPE>Goods</LINEORDERTYPE><HAZRDUNNUM/><HAZRDUNDESC/><DFFLIN
  E><ATTRIBUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/><ATTRIBUTE5/><ATTR
  IBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIBUTE10/><ATTRIBUTE1
  1/><ATTRIBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUTE15/><ATTRIBUTE16/
  ></DFFLINE><DFFITEM><ATTRIBUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/>
  <ATTRIBUTE5/><ATTRIBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIB
  UTE10/><ATTRIBUTE11/><ATTRIBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUT
  E15/><ATTRIBUTE16/></DFFITEM><KFFITEM><ATTRIBUTE1>PRODUCTN</ATTRIBUTE1><
  ATTRIBUTE2>FINGOODS</ATTRIBUTE2><ATTRIBUTE3/><ATTRIBUTE4/><ATTRIBUTE5/><
  ATTRIBUTE6/><ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIBUTE10/><ATTRIB
  UTE11/><ATTRIBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUTE15/><ATTRIBUT
  E16/><ATTRIBUTE17/><ATTRIBUTE18/><ATTRIBUTE19/><ATTRIBUTE20/></KFFITEM><
  GLOBALCONTRACT/><GLOBALCONTRACTLIN/><JOBTITLE/><AMOUNT qualifier="TOTAL"
  type="T"
  ><VALUE/><NUMOFDEC/><SIGN/><CURRENCY/><DRCR>C</DRCR></AMOUNT><CONTRACTOR
  FIRSTNAME/><CONTRACTORLASTNAME/><DATETIME qualifier="ACTSTART" type="T"

```



```

index="1"
><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
DATETIME><DATETIME qualifier="ACTEND" type="T" index="1"
><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
DATETIME></USERAREA>
  <POLINESCHD>
    <DATETIME qualifier="NEEDELV" type="T" index="1">
      <YEAR>2011</YEAR>
      <MONTH>04</MONTH>
      <DAY>21</DAY>
      <HOUR>00</HOUR>
      <MINUTE>00</MINUTE>
      <SECOND>00</SECOND>
      <SUBSECOND>0000</SUBSECOND>
      <TIMEZONE>+0000</TIMEZONE>
    </DATETIME>
    <QUANTITY qualifier="ORDERED">
      <VALUE>12</VALUE>
      <NUMOFDEC/>
      <SIGN>+</SIGN>
      <UOM>Ea</UOM>
    </QUANTITY>
    <DESCRIPTN/>
    <PSCLINENUM>1</PSCLINENUM>
    <USERAREA><DATETIME qualifier="PROMSHIP"
><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
DATETIME><DATETIME qualifier="APPROVAL"
><YEAR/><MONTH/><DAY/><HOUR/><MINUTE/><SECOND/><SUBSECOND/><TIMEZONE/></
DATETIME><OPERAMT qualifier="UNIT" type="T"
><VALUE>110786</VALUE><NUMOFDEC>2</NUMOFDEC><SIGN>+</SIGN><CURRENCY>USD<
/CURRENCY><UOMVALUE>1</UOMVALUE><UOMNUMDEC>0</UOMNUMDEC><UOM>Ea</UOM></O
PERAMT><PRICEVRRD/><TAXABLE>N</TAXABLE><TAXCODE/><PARTNER><NAME index="
1">Vision
Operations</NAME><ONETIME>0</ONETIME><PARTNRID>204</PARTNRID><PARTNRTYPE
>ShipTo</PARTNRTYPE><CURRENCY>USD</CURRENCY><DUNSNUMBER/><PARTNRIDX>Exam
ple-01</PARTNRIDX><PAYMETHOD/><TERMD/><USERAREA/>
  <ADDRESS></ADDRESS>
  <CONTACT><NAME index="1"/><CONTACTTYPE/><EMAIL/><FAX index="1"
/><TELEPHONE index="1"/></CONTACT>
  </PARTNER>
  <PARTNER><NAME index="1"
/><ONETIME/><PARTNRID/><PARTNRTYPE>DeliveryTo</PARTNRTYPE><PARTNRIDX/><U
SERAREA/><ADDRESS><ADDRLINE index="1"/><ADDRLINE index="2"
/><ADDRTYPE/></ADDRESS><CONTACT><NAME index="1"
/><CONTACTTYPE/><EMAIL/><FAX index="1"/><TELEPHONE index="1"
/></CONTACT></PARTNER><DROPSHIPDETAILS><DROPSHIPMENT/><DROPSHIPCUSTNAME/
><SHIPINSTR/><PACKINSTR/><SHIPMETHOD/><CUSTOMERPONUM/><CUSTOMERLINENUM/>
<CUSTOMERSHIPNUM/><CUSTOMERDESC/></DROPSHIPDETAILS><CONSIGNEDINV>N</CONS
IGNEDINV><DISTPROJECT><REQUESTOR/><DISTNUM>1</DISTNUM><PROJECTNUM/><PROJ
ECTTYPE/><TASKNUM/><QUANTITY qualifier="ORDERED"
><VALUE>12</VALUE><NUMOFDEC/><SIGN>+</SIGN><UOM>Ea</UOM></QUANTITY><CONV
RATE/><DATETIME qualifier="EXCHRATEDATE"
><YEAR>2011</YEAR><MONTH>04</MONTH><DAY>05</DAY><HOUR>00</HOUR><MINUTE>0
0</MINUTE><SECOND>00</SECOND><SUBSECOND>0000</SUBSECOND><TIMEZONE>+0000<
/TIMEZONE></DATETIME><DESTTYPE>INVENTORY</DESTTYPE><DFFDISTRIBUTN><ATTRI
BUTE1/><ATTRIBUTE2/><ATTRIBUTE3/><ATTRIBUTE4/><ATTRIBUTE5/><ATTRIBUTE6/>
<ATTRIBUTE7/><ATTRIBUTE8/><ATTRIBUTE9/><ATTRIBUTE10/><ATTRIBUTE11/><ATTR
IBUTE12/><ATTRIBUTE13/><ATTRIBUTE14/><ATTRIBUTE15/><ATTRIBUTE16/></DFFDI
STRIBUTN></DISTPROJECT><AMOUNT qualifier="TOTAL" type="T"
><VALUE/><NUMOFDEC/><SIGN/><CURRENCY/><DRCR>C</DRCR></AMOUNT></USERAREA>

  </POLINESCHD>
</POORDERLIN>
</PROCESS_PO>
</DATAAREA>
</PROCESS_PO_007>

```

Using XML Gateway Outbound through Subscription Model

This section includes the following topics:

- Using XML Gateway Outbound Services at Design Time, page 8-66
- Deploying and Testing the SOA Composite Application with BPEL Process at Run Time, page 8-84

Using XML Gateway Outbound Services at Design Time

For an outbound XML Gateway Map interface, since an outbound message is first enqueued to the ECX_OUTBOUND queue, Oracle BPEL PM listens to ECX_OUTBOUND queue for the message with the same correlation Id.

Oracle E-Business Suite Integrated SOA Gateway supports it through subscription model by dequeuing the message in the ECX_OUTBOUND queue to retrieve outbound data from Oracle E-Business Suite. The retrieved data can be passed to trading partners or consumers who subscribed to the message.

SOA Composite Application with BPEL Process Scenario

Take XML Gateway outbound interface 'PO acknowledgement XML Transaction' as an example. The XML Gateway outbound interface is exposed as a Web service through ECX_CB0D0_0AG72_OUT outbound map.

When a purchase order is created and approved, on approval of the purchase order, a workflow will be triggered which creates the Purchase Order Acknowledgement flow and sends out the PO Acknowledgement as an XML file. The workflow delivers the Confirm BOD as the PO Acknowledgement to ECX_OUTBOUND queue for delivery to the other system.

The correlation Id for this message is set to 'BPEL' and the Oracle BPEL PM listens to ECX_OUTBOUND queue for the message with the correlation Id = 'BPEL'. 'Confirm BOD' as the PO Acknowledgement is written as an output XML file using File Adapter.

When the BPEL process has been successfully executed after deployment, the same order book reference ID (Customer PO) from the output XML file should be obtained once a purchase order is approved.

Prerequisites to Create a SOA Composite Application with BPEL Process Using XML Gateway Outbound Messaging

You need to set up the correlation identifier in Oracle E-Business Suite. The correlation identifier enables you to label messages meant for a specific agent, in case there are multiple agents listening on the outbound queue. The agent listening for a particular correlation picks up the messages that match the correlation identifier for the agent.

To set up the correlation identifier:

1. Log in to Oracle E-Business Suite as a user who has the XML Gateway

responsibility.

2. Click the **XML Gateway** link from the Navigator.
3. Click the **Define Lookup Values** link under XML Gateway.

XML Gateway Lookups Window

Code	Meaning	Description	Tag	Effective Dates	Enabled	
				From	To	
BPEL	BPEL	BPEL process		07-DEC-2006		☒
HTTP	HTTP	HTTP Delivery		28-SEP-2000		☒
HTTP-ATCH	Attachment Enabled H	Attachment Enabled HT		12-DEC-2002		☒
HTTP-OXTA	Oracle Transport Agen	OXTA over HTTP		29-MAY-2000		☒
HTTP-OXTAAI	Attachment Enabled C	Attachment Enabled OX		25-OCT-2002	12-DEC-2002	☐
HTTP-WM	WebMethods using HT	HTTP with WebMethods		05-OCT-2000		☒
HTTPS	HTTPS (Secure HTTP)	HTTPS Delivery		28-SEP-2000		☒
HTTPS-ATCH	Attachment Enabled H	Attachment Enabled HT		12-DEC-2002		☒
HTTPS-OMB	Oracle Message Broke	OMB using HTTPS		01-DEC-2000		☐
HTTPS-OXTA	Oracle Transport Agen	OXTA over HTTPS		29-MAY-2000		☒

4. Search for **COMM_METHOD** in the **Type** field to see if it exists in the system.
5. Add a new record to the **COMM_METHOD** type by entering **BPEL** for the **Code** field and the **Meaning** field. Enter description information and save the record.

Oracle XML Gateway puts the correlation of BPEL when enqueueing the message on the **ECX_OUTBOUND** queue.

In addition to having the correlation identifier set up correctly, ensure the trading partner that you want to use has the Protocol Type field set to **BPEL**. For information on how to set up the trading partner with desired transactions, see *Manually Testing the SOA Composite Application with BPEL Process*, page 8-84.

SOA Composite Application with BPEL Process Creation Flow

Based on the PO acknowledgement XML Transaction scenario, the following design-time tasks are discussed in this chapter:

1. Create a New SOA Composite Application with BPEL Process, page 8-68

Use this step to create a new SOA composite application with BPEL process called `XMLGOutbound.bpel`.

2. Create a Partner Link for AQ Adapter, page 8-69
Use this step to dequeue the event details from the ECX_OUTBOUND queue.
3. Add a Receive Activity, page 8-76
Use the Receive activity to take PO acknowledgement details as an input to the Assign activity.
4. Add a Partner Link for File Adapter, page 8-78
This is to write PO acknowledgement details in an XML file as an output file.
5. Add an Invoke Activity, page 8-82
This is to write PO acknowledgement information to an XML file through invoking the partner link for File Adapter.
6. Add an Assign Activity, page 8-83
Use the Assign activity to take the output from the Receive activity and to provide input to the Invoke activity.

For general information and how to create SOA composite applications using BPEL process service component, see the *Oracle Fusion Middleware Developer's Guide for Oracle SOA Suite* for details.

Creating a New SOA Composite Application with BPEL Process

Use this step to create a new SOA composite application that will contain various BPEL process activities.

To create a new SOA composite application with BPEL process:

1. Open Oracle JDeveloper.
2. Click **New Application** in the Application Navigator. The "Create SOA Application - Name your application" page is displayed.
3. Enter an appropriate name for the application in the Application Name field and select **SOA Application** from the Application Template list.
Click **Next**. The "Create SOA Application - Name your project" page is displayed.
4. Enter an appropriate name for the project in the Project Name field. For example, XMLGatewayOutbound.
5. In the Project Technologies tab, select 'Web Services' and ensure that **SOA** is selected from the Available technology list to the Selected technology list.
Click **Next**. The "Create SOA Application - Configure SOA settings" page is displayed.

6. Select **Composite With BPEL Process** from the Composite Template list, and then click **Finish**. You have created a new application, and a SOA project. This automatically creates a SOA composite.

The Create BPEL Process page is displayed.

7. Leave the default **BPEL 1.1 Specification** selection unchanged. This creates a BPEL project that supports the BPEL 1.1 specification.

Enter an appropriate name for the BPEL process in the Name field. For example, XMLGOutbound.

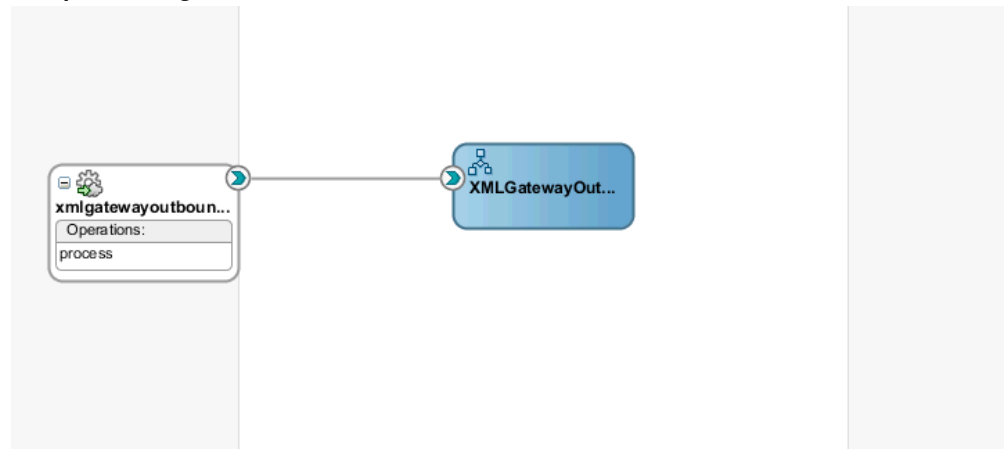
Select **Synchronous BPEL Process** in the Template field.

Select **required** from the Transaction drop-down list. Click **OK**.

A synchronous BPEL process is created with the Receive and Reply activities. The required source files including `bpel` and `wsdl`, using the name you specified (for example, `XMLGOutbound.bpel` and `XMLGOutbound.wsdl`) and `composite.xml` are also generated.

8. Navigate to SOA Content > Business Rules and double click `composite.xml` to view the composite diagram.

Composite Diagram



Double click on the XMLGOutbound component to open the BPEL process.

Creating a Partner Link for AQ Adapter

Use this step to create a Partner Link called `GetAck` for AQ Adapter to dequeue the XML Gateway outbound message (for example, `ECX_CBOD0_OAG72_OUT`) in the `ECX_OUTBOUND` queue.

To create a partner link for AQ Adapter:

1. In Oracle JDeveloper, drag and drop the **AQ Adapter** service from the **BPEL Services** list into the right Partner Link swim lane of the process diagram. The Adapter Configuration wizard welcome page appears.
2. Click **Next**. The Service Name dialog box appears.
3. Enter a service name in the Service Name dialog box, for example **GetAck**.

Adapter Configuration Wizard - Service Name

Adapter Configuration Wizard - Step 2 of 7

Service Name

Enter a Service Name.

Service Type: AQ Adapter

Service Name:

4. Click **Next**. The Service Connection dialog box appears.
5. You can use an existing database connection by selecting a database connection from the **Connection** list or define a new database connection by clicking **New** to open the Create Database Connection Wizard.

Note: You need to connect to the database where Oracle E-Business Suite is running.

To create a new database connection:

1. Click **New** to open the Create Database Connection Wizard. Click **Next** and

enter an unique connection name and then select a connection type, such as Oracle (JDBC), for the database connection. Click **Next**.

2. Enter appropriate username and password information to authenticate the database connection in the Authentication dialog box. Click **Next**.
3. Specify the following information in the Connection dialog box:
 - Driver: Thin
 - Host Name: Enter the host name for the database connection. For example, `myhost01.example.com`.
 - JDBC Port: Enter JDBC port number (such as 1521) for the database connection.
 - SID: Specify an unique SID value for the database connection.
4. Click **Next** to test your database connection.

The status message "Success!" indicates a valid connection.
5. Click **Next** to return to the Service Connection dialog box providing a summary of the database connection.
6. The JNDI (Java Naming and Directory Interface) name corresponding to the database connection you specified appears automatically in the **JNDI Name** field of the Service Connection dialog box. Alternatively, you can enter a different JNDI name.
7. Click **Next** to open Adapter Interface dialog box.

Select the **Define from operation and schema (specified later)** radio button and click **Next**. The Operation dialog box appears.
8. Select the **Dequeue** radio button in the Operation Type field. 'Dequeue' is also populated in the Operation Name field.

Adapter Configuration Wizard - Operation

The screenshot shows a window titled "Adapter Configuration Wizard - Step 5 of 7". The main heading is "Operation". Below it, a paragraph explains: "The AQ Adapter supports three operations. There is a Dequeue operation that polls for incoming messages from a queue, an Enqueue operation that puts outgoing messages on a queue, and a Enqueue/Dequeue operation that puts outgoing messages on a queue and expects response messages on a queue. Specify the Operation type and Operation Name. Only one operation per Adapter Service may be defined using this wizard." Below this text, there are three radio buttons for "Operation Type": "Dequeue" (selected), "Enqueue", and "Enqueue/Dequeue". Below the radio buttons is a text field labeled "Operation Name:" with the value "Dequeue" entered. At the bottom of the window, there are four buttons: "Help", "< Back", "Next >", and "Finish". The "Next >" button is highlighted with a blue border.

Operation

The AQ Adapter supports three operations. There is a Dequeue operation that polls for incoming messages from a queue, an Enqueue operation that puts outgoing messages on a queue, and a Enqueue/Dequeue operation that puts outgoing messages on a queue and expects response messages on a queue. Specify the Operation type and Operation Name. Only one operation per Adapter Service may be defined using this wizard.

Operation Type: ☒ Dequeue
☐ Enqueue
☐ Enqueue/Dequeue

Operation Name: Dequeue

Help < Back Next > Finish Cancel

9. Click **Next** to open the Queue Name dialog box.

Select 'APPLSYS' as the Database Schema field. Enter 'ECX_OUTBOUND' as the Queue Name field.

AQ Adapter Configuration Wizard - Queue Name

Queue Name

Specify the database schema and the queue to be used for the service. Use the Browse button to find the queue in the specified schema.

Queue Information

Database Schema: APPLSYS

Queue Name: ECX_OUTBOUND

Buttons: Help, < Back, Next >, Finish, Cancel

10. Click **Next** to open the Queue Parameters dialog box.

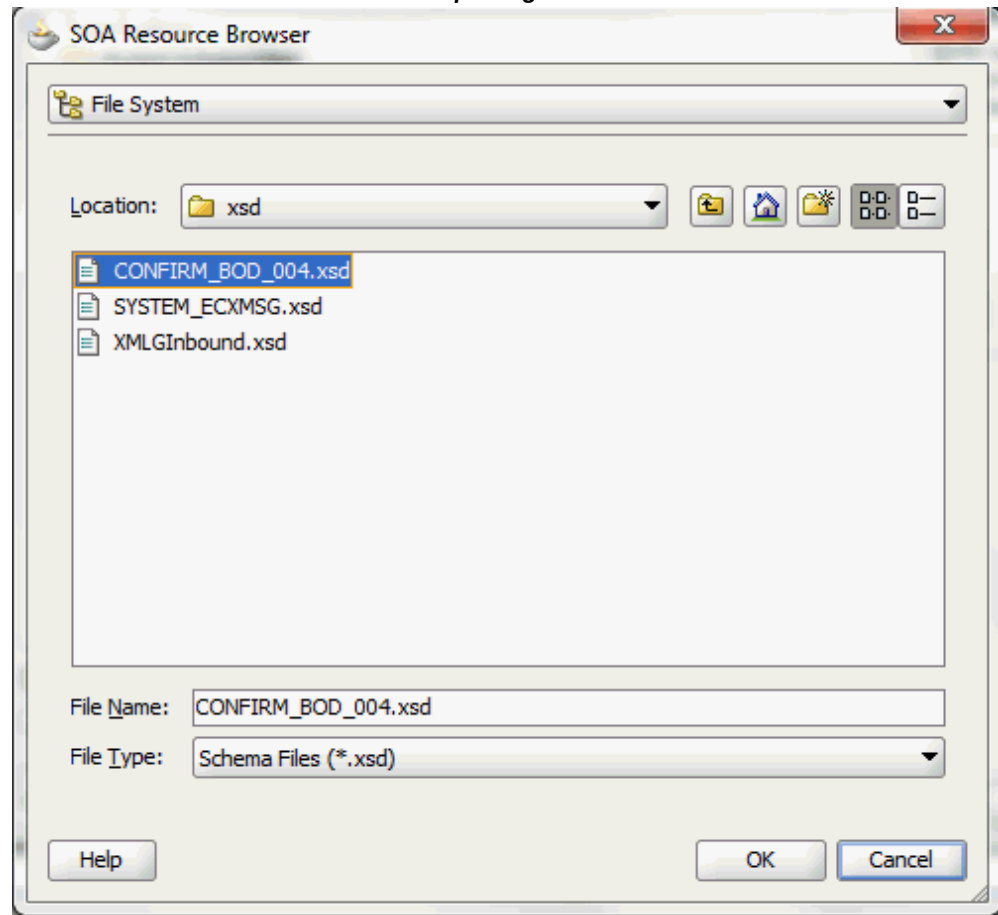
Enter a unique customer name, such as 'Example Inc.', in the Correction Id field.
Click **Next**. The Messages dialog box opens.

11. Click **Browse for schema file** to open the Type Chooser window.

Click **Import Schema Files** on the top right corner of the Type Chooser window.
This opens the Import Schema File pop-up window.

Click the **Browse Resources...** icon to display the SOA Resource Browser window.
Select the xsd folder as the location and CONFIRM_BOD_004.xsd file from the folder.

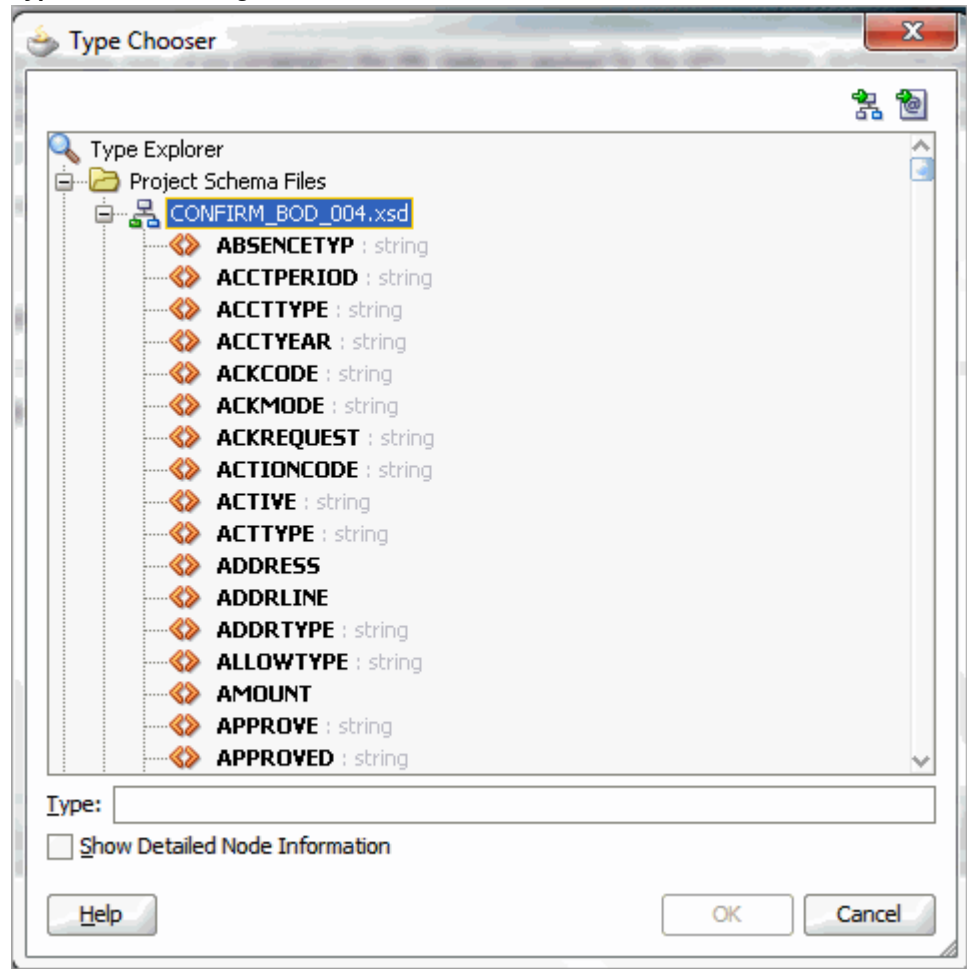
SOA Resource Browser Window for Importing a Desired Schema



Click **OK**. The Localize Files dialog box opens. Click **OK**.

In the Type Chooser window, the selected `CONFIRM_BOD_004.xsd` file is displayed. Scroll down to select `CONFIRM_BOD_004` from `CONFIRM_BOD_004.xsd`. Click **OK**.

Type Chooser Dialog



The Messages dialog box is automatically displayed with the selected schema location and schema element.

AQ Adapter Configuration Wizard - Messages

Messages

Define the message that will be contained in the XML Gateway payload for the APPLSYS.ECX_OUTBOUND queue. Specify the Schema File location and select the Schema Element that defines the message. Use the Browse button to find an existing schema definition. If you check Schema is Opaque, then you do not need to specify a Schema.

Message Schema

☐ Native format translation is not required (Schema is Opaque)

URL:

Schema Element:

☐ Enable Streaming

12. Click **Next** to proceed to the Finish dialog box to confirm that you have finished defining the AQ Adapter for the GetAck service.
13. Click **Finish**. The wizard generates the WSDL file corresponding to the GetAck service.

Click **Apply** and then **OK** to complete the partner link configuration. The partner link is created with the required WSDL settings, and is represented in the BPEL project by a new icon in the border area of the process diagram.

Adding a Receive Activity

This step is to configure a Receive activity to receive XML data from the partner link GetAck that you configured for the AQ adapter.

The XML data received from the Receive activity is used as an input variable to the Assign activity that will be created in the next step.

To add a Receive activity:

1. In Oracle JDeveloper, expand the **BPEL Constructs** from the Component Palette.

Drag and drop the **Receive** activity into the center swim lane of the process diagram.

2. Link the **Receive** activity to the **GetAck** partner link. The Receive activity will take event data from the partner link. The Edit Receive dialog box appears.

3. Enter a name for the Receive activity.

Click the **Create** icon next to the **Variable** field to create a new variable. The Create Variable dialog box appears.

4. Select **Global Variable**, and then enter a name for the variable. You can accept the default name. Click **OK** to return to the Edit Receive dialog box.

Edit Receive Dialog

The screenshot shows the 'Edit Receive' dialog box with the 'General' tab selected. The 'Name' field is set to 'Receive'. The 'Conversation ID' field is empty. The 'Create Instance' checkbox is unchecked. The 'Interaction Type' is set to 'Partner Link'. The 'My Role Web Service Interface' section shows 'Partner Link' as 'GetAck' and 'Operation' as 'Dequeue'. The 'Variable' field is set to 'Receive_Dequeue_InputVariable'. The dialog has buttons for 'Help', 'Apply', 'OK', and 'Cancel' at the bottom.

5. Click **Apply** and **OK** to finish configuring the Receive activity.

The Receive activity appears in the BPEL process diagram.

Adding a Partner Link for File Adapter

Use this step to configure a partner link by writing the purchase order acknowledgement to an XML file.

To add a Partner Link for File Adapter:

1. In Oracle JDeveloper, drag and drop the **File Adapter** service from the **BPEL Services** list into the right Partner Link swim lane of the process diagram. The Adapter Configuration wizard appears.
2. Click **Next**. The Service Name dialog box appears.
3. Enter a name for the File Adapter service, such as **WriteAck**.
4. Click **Next** and the Adapter Interface dialog box appears.
Select the **Define from operation and schema (specified later)** radio button and click **Next**. The Operation dialog box appears.
5. Specify the operation type, for example **Write File**. This automatically populates the **Operation Name** field.

Adapter Configuration Wizard - Operation

Operation

The File Adapter supports four operations. There is a Read File operation that polls for incoming files in your local file system, a Write File operation that creates outgoing files, a Synchronous Read File operation that reads the current contents of a file, and a List Files operation that lists file names in specified locations. Specify the Operation type and Operation Name. Only one operation per Adapter Service may be defined using this wizard.

Operation Type: ☐ Read File
☒ Write File
☐ Synchronous Read File
☐ List Files

Operation Name:

Help < Back Next > Finish Cancel

6. Click **Next** to access the File Configuration dialog box.

File Adapter Configuration Wizard - File Configuration

FILE Adapter Configuration Wizard - Step 5 of 7

File Configuration

Specify the parameters for the Write File operation.

Directory specified as ☐ Physical Path ☒ Logical Name

Directory for Outgoing Files (logical name):
outputDir

File Naming Convention (po_%SEQ%.txt): PO_%yyMMddHHmmss%.xml

☐ Append to existing file

Write to output file when any of these conditions are met

☒ Number of Messages Equals: 1

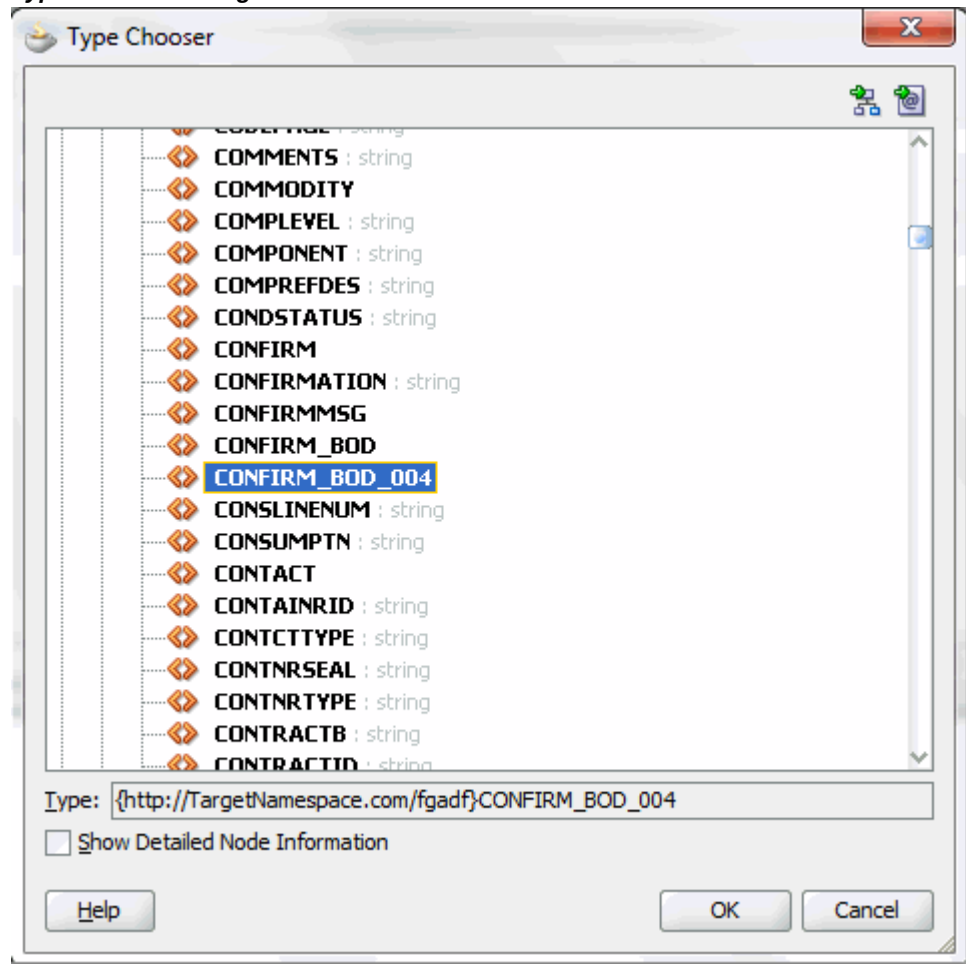
☐ Elapsed Time Exceeds: 1 minutes

☐ File Size Exceeds: 1000 kilobytes

Help < Back Next > Finish Cancel

7. For the Directory specified as field, select **Logical Path**. Enter directory path in the Directory for Outgoing Files field, and specify a naming convention for the output file such as PO_%yyMMddHHmmss%.xml.
8. Confirm the default write condition: Number of Messages Equals 1. Click **Next**. The Messages dialog box appears.
9. Select the **Browse** checkbox to locate the schema location and schema element.
The Type Chooser dialog box appears. Expand the **Project Schema Files > CONFIRM_BOD_004.xsd** and select CONFIRM_BOD_004.

Type Chooser Dialog



Click **OK** in the Type Chooser window to populate the selected schema location and element in the Messages dialog box.

File Adapter Configuration Wizard - Messages

Messages

Define the message for the Write File operation. Specify the Schema File Location and select the Schema Element that defines the messages in the outgoing files. Use the Browse button to find an existing schema definition. If you check 'Schema is Opaque', then you do not need to specify a Schema.

Message Schema

☐ Native format translation is not required (Schema is Opaque)

URL

Schema Element

10. Click **Next** and then **Finish**. The wizard generates the WSDL file corresponding to the partner link. The main Create Partner Link dialog box appears, specifying the new WSDL file `WriteAck.wsdl`.

Click **Apply** and then **OK** to complete the configuration and create the partner link with the required WSDL settings for the File Adapter Service.

The `WriteAck` Partner Link appears in the BPEL process diagram.

Adding an Invoke Activity

This step is to configure an **Invoke** activity to send the purchase order acknowledgement that is received from the **Receive** activity to the `WriteAck` partner link in an XML file.

To add an Invoke activity:

1. In Oracle JDeveloper, expand the **BPEL Constructs** from the Component Palette. Drag and drop the **Invoke** activity into the center swim lane of the process diagram, after the **Receive** activity.

2. Link the **Invoke** activity to the **WriteAck** service. The **Invoke** activity will send event data to the partner link. The Edit Invoke dialog box appears.
3. Enter a name for the **Invoke** activity, and then click the **Create** icon next to the **Input Variable** field to create a new variable. The Create Variable dialog box appears.
4. Select **Global Variable**, and then enter a name for the variable. You can also accept the default name. Click **OK**.

Click **Apply** and then **OK** in the Edit Invoke dialog box to finish configuring the **Invoke** activity.

The **Invoke** activity appears in the process diagram.

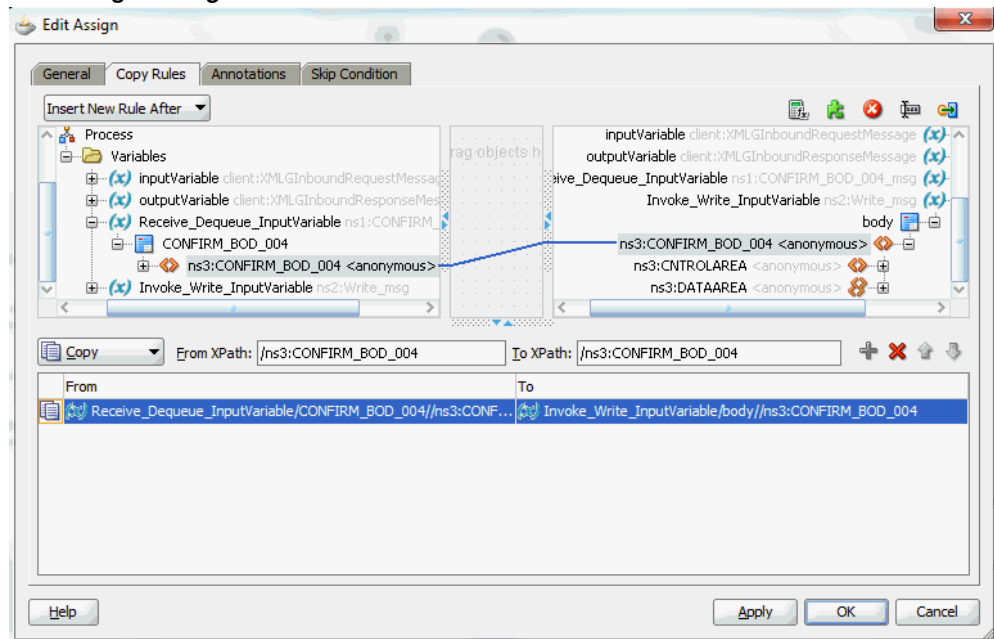
Adding an Assign Activity

Use this step to pass the purchase order acknowledgement details from the **Receive** activity to the **Invoke** activity.

To add an Assign activity:

1. In Oracle JDeveloper, expand the **BPEL Constructs** from the Component Palette. Drag and drop the **Assign** activity from the Component Palette into the center swim lane of the process diagram, between the **Receive** activity and the **Invoke** activity.
2. Double-click the **Assign** activity to access the Edit Assign dialog box.
3. Select the Copy Rules tab and expand the source and target trees:
 - In the From navigation tree, navigate to **Variable > Process > Variables > Receive_DEQUEUE_InputVariable** and select **CONFIRM_BOD_004**. The XPath field contains your selected entry.
 - In the To navigation tree, navigate to **Variable > Process > Variables > Invoke_Write_InputVariable > body** and select **CONFIRM_BOD_004**. The XPath field contains your selected entry.

Edit Assign Dialog



Drag the source node (CONFIRM_BOD_004) to connect to the target node (CONFIRM_BOD_004) that you just identified. This creates a line that connects the source and target nodes. The copy rule is displayed in the From and To sections at the bottom of the Edit Assign dialog box.

4. Click **Apply** and then **OK** in the Edit Assign dialog box to complete the configuration of the **Assign** activity.

Deploying and Testing the SOA Composite Application with BPEL Process at Run Time

After creating a SOA composite application with BPEL process for XML Gateway outbound message map, you need to deploy it to the Oracle WebLogic managed server. This can be achieved using Oracle JDeveloper. Once the composite is deployed, it can be tested from the Oracle Enterprise Manager Fusion Middleware Control Console.

Prerequisites

Before deploying the SOA composite with BPEL process using Oracle JDeveloper, you must have established the connectivity between the design-time environment and the run-time server. For information on how to configure the necessary server connection, see *Configuring Server Connection, Oracle E-Business Suite Integrated SOA Gateway Developer's Guide*.

Note: If a local instance of the WebLogic Server is used, start the WebLogic Server by selecting **Run > Start Server Instance** from Oracle

JDeveloper. Once the WebLogic Admin Server "DefaultServer" instance is successfully started, the <Server started in Running mode> and DefaultServer started message in the Running:DefaultServer and Messages logs should appear.

Perform the following run-time tasks:

1. Deploy the SOA Composite with BPEL Process, page 8-85
2. Manually Test the SOA Composite with BPEL Process, page 8-87

Deploying the SOA Composite with BPEL Process

You must deploy the SOA composite application with BPEL Process (XMLGOutbound.bpel) that you created earlier before you can run it.

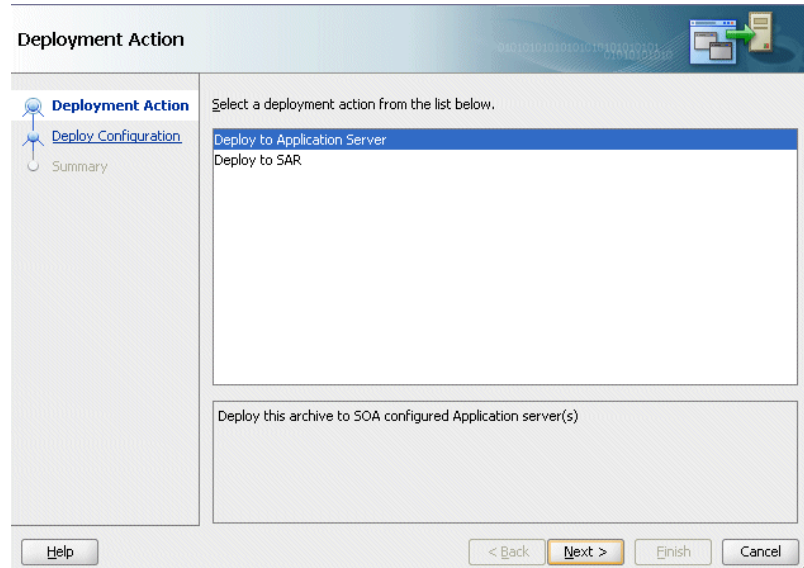
To deploy the SOA composite application with BPEL process:

1. In the Applications Navigator of Oracle JDeveloper, select the **XMLGOutbound** project.
2. Right-click the project and select **Deploy > [project name] > [serverConnection]** from the menu.

For example, you can select **Deploy > XMLGInbound > SOAServer** to deploy the process if you have the connection appropriately.

Note: If this is the first time to set up the server connection, then the Deployment Action dialog appears. Select 'Deploy to Application Server' and click **Next**.

Deployment Action Dialog



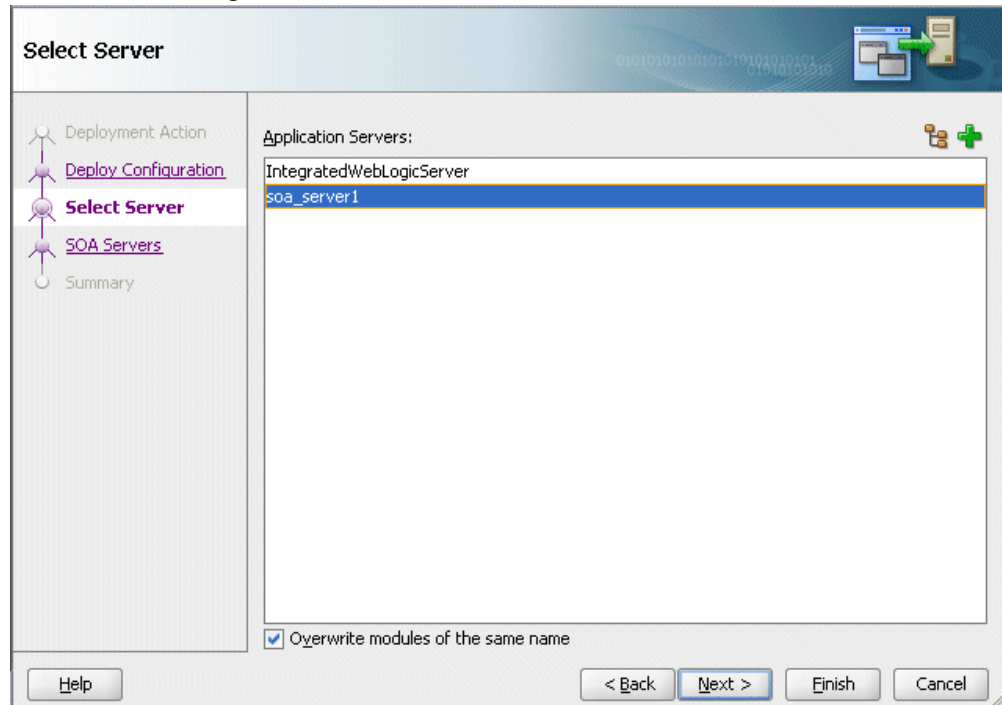
In the Deploy Configuration dialog, ensure the following information is selected before clicking **Next** to add a new application server:

- New Revision ID: 1.0
- Mark composite revision as default: Select this checkbox.
- Overwrite any existing composites with the same revision ID: Select this checkbox.

For information on how to create a new server connection for Oracle WebLogic Server, see *Configuring Server Connection, Oracle E-Business Suite Integrated SOA Gateway Developer's Guide*.

3. In the Select Server dialog, select 'soa-server1' that you have established the server connection earlier. Click **Next**.

Select Server Dialog



4. In the SOA Servers dialog, accept the default target SOA Server ('soa-server1') selection.

Click **Next** and **Finish**.

5. If you are deploying the composite for the first time from your Oracle JDeveloper session, the Authorization Request window appears. Enter username and password information specified during Oracle SOA Suite installation. Click **OK**.
6. Deployment processing starts. Monitor deployment process and check for successful compilation in the SOA - Log window.

Verify that the deployment is successful in the Deployment - Log window.

Manually Testing the SOA Composite Application with BPEL Process

Once the BPEL process contained in the SOA composite application has been successfully deployed, you can manage and monitor the process from Oracle Enterprise Manager Fusion Middleware Control Console. You can log on to Oracle E-Business Suite to manually create and book the order as well as generate the order acknowledgement by submitting a Workflow Background Process concurrent request.

Log on to the Oracle Enterprise Manager Fusion Middleware Control Console to validate the BPEL process which writes purchase order acknowledgement in an output

directory after receiving from the XML Gateway ECX_OUTBOUND queue.

To manually test the SOA composite application with BPEL process:

1. Log in to Oracle E-Business Suite as a user who has the XML Gateway responsibility.

This is to ensure that the XML Gateway trading partner is set up correctly so that a purchase order can have a valid customer that has been defined.

2. Select **Define Trading Partner** from the navigation menu to access the Trading Partner Setup window.
3. Enter the header values on the Trading Partner Setup form as follows:
 - Trading Partner Type: Customer
 - Trading Partner Name: For example, Example Inc.
 - Trading Partner Site: Enter a trading partner site information, for example, 401 Island Parkway Redwood Shores, CA 94065.
 - Company Admin Email: Enter a valid email address.
4. Enter the following trading partner details:
 - Transaction Type: ECX
 - Transaction SubType: CBODO
 - Standard Code: OAG
 - External Transaction Type: BOD
 - External Transaction SubType: CONFIRM
 - Direction: Out
 - Map: ECX_CBODO_OAG72_OUT
 - Connection / Hub: DIRECT
 - Protocol Type: BPEL
 - Username: 'operations'
 - Password: Enter the associated password for the user 'operations' twice.
 - Protocol Address: 'http://us.example.com'

Protocol Address is the complete URL where the XML document can be posted.

- Source Trading Partner Location Code: Example-01

Trading Partner Setup Form

Transaction Type	Transaction SubType	Standard Code	External Transaction Type	External Transaction SubType	Direction	Map	Connection/Hub	Protocol Type
ONT	POA	OAG	PO	ACKNOWLEDG	OUT	ONT_3A4A_O	DIRECT	SMTP
ONT	CPO	OAG	PO	CANCEL	IN	ONT_3A9R_O		
ONT	POI	OAG	PO	PROCESS	IN	ONT_3A4R_O		
ONT	SSO	OAG	SALESORDER	SHOW	OUT	PROCESS_P	DIRECT	SMTP
ECX	CBODO	OAG	BOD	CONFIRM	OUT	ECX_CBODO	DIRECT	BPEL

5. Save the trading partner details.

To successfully generated PO Acknowledgement, perform the following setup tasks in Order Management and then manually book the order:

1. Switch responsibility back to Order Management Super User, Vision Operations (USA) and select **Customer > Standard** from the navigation menu to open the Enter Customer form.
2. Search on the 'Example Inc.' in the Name field and click **Go**.
3. Select the Business World with the following information from the search results:
 - Name: Example Inc.
 - Registry ID: 1004
 - Address: 401 Island Parkway Redwood Shores, CA 94065

4. Select the account with the following information:
 - Account Number: 1004
 - Account Description: Example Inc.
 - Status: Active
5. Click the **Details** icon to open the Update Account: 1608 page.
6. Locate the address with 401 Island Parkway Redwood Shores, CA 94065 and click the **Details** icon to open the Site page of your selected account.
7. In the Site Details tab, the Account Site Details region, enter 'Example-01' in the EDI Location field.
8. In the Business Purposes tab, create a new row with the following values:
 - Purpose: Sold To
 - Check on the 'Primary' Checkbox

Click **Apply**.

Use the following steps to generate acknowledgement for the order that you have created for the XML Gateway Inbound service.

1. With the same Order Management Super User, Vision Operations (USA) responsibility, select **Order Returns > Sales Order** to open the Sales Order form.
2. Retrieve the order that you have created earlier in the XML Gateway inbound service by entering the order ID 'PO-4466-5' in the Customer PO field.
3. Click **Book Order** to book the order.
Notice that the Status field is now changed to 'Booked'.

Sales Orders Form

Sales Orders (Vision Operations) - 85182

Order Information Line Items

Default

Main Others

Customer		Order Number	
Customer Number		Order Type	Mixed
Customer PO	PO-4466-5	Date Ordered	04-SEP-2012 23:26:29
Customer Contact		Price List	Corporate
Ship To Location		Salesperson	
		Status	Entered
		Currency	USD
Bill To Location		Subtotal	26,156.40
		Tax	0.00
		Charges	150.00
		Total	26,306.40

[]

Actions Related Items Configurator Availability Book Order

4. Switch to the System Administrator responsibility and select **Request > Run**.
5. Select **Single Request** and click **OK**.
6. Enter the following information in the Submit Request form:

Submit Request Form and Parameters Dialog

Submit Request

Run this Request...

Copy...

Name: Workflow Background Process

Operating Unit:

Parameters:

Language: American English

Language Settings... Debug Options

At these Times... Run the Job As Soon As Possible

Upon Completion... ☒ Save all

Layout: Notify: Print to: noprint

Help (C)

Parameters

Item Type: OM Send Acknowledgment

Minimum Threshold:

Maximum Threshold:

Process Deferred: Yes

Process Timeout: No

Process Stuck: No

OK Cancel Clear Help

- Name: Workflow Background Process
- Enter the following parameters:
 - Item Type: OM Send Acknowledgment
 - Process Deferred: Yes
 - Process Timeout: No
 - Process Stuck: No
- Click **OK**.

7. Click **Submit** to submit the 'send acknowledgement' request.

8. View your request by entering the request ID to ensure its status is 'Success'.

Validation Using Oracle Enterprise Manager Fusion Middleware Control Console

Log on to Oracle Enterprise Manager Fusion Middleware Control Console (<http://<servername>:<portname>/em>) to confirm that the XMLGOutbound process has been deployed. This process is continuously polling the ECX_OUTBOUND queue for purchase order acknowledgement.

From the Farm navigation pane, expand the SOA > soa-infra node in the tree to navigate through the SOA Infrastructure home page and menu to access your deployed

SOA composite applications running on `soa-infra` managed server. Click the XMLGOutbound [1.0] link.

Click **Test**. The Test Web Service page for initiating an instance appears. Enter the input string required by the process and click **Test Web Service** to initiate the process. The test results appear in the Response tab upon completion. Click your BPEL service component instance link (such as XMLGOutbound) to display the Instances page where you can view execution details for the BPEL activities in the Audit Trail tab.

Click the Flow tab to check the BPEL process flow diagram. Click an activity of the process diagram to view the activity details and flow of the payload through the process. For example, click **Receive** activity. Click the **view xml document** link to open the received XML file. Note the Reference ID such as Customer PO.

Go to the directory you specified for the write operation, for example `outputDir` - logical location (typically `c:\temp`) where the File Adapter has placed the file after writing the PO Acknowledgement in an XML file (such as 'PO_060719175318.xml').

Open the 'PO_060719175318.xml' file and search for ORIGREF element. Its value should be 'PO-4466-5' (the order booked) for which the acknowledgement is generated.

XML Gateway B2B Transactions Using JMS Queues

This chapter covers the following topics:

- Java Messaging Service (JMS) Overview
- Oracle XML Gateway and B2B Transactions Integration Points
- Oracle XML Gateway and JMS Integration
- Creating Custom JMS Queues
- Steps to Create Custom JMS Queues for Outbound B2B Transactions
- Steps to Create Custom JMS Queues for Inbound B2B Transactions

Java Messaging Service (JMS) Overview

Java Messaging Service (JMS) is a messaging standard defined by Sun Microsystems, Oracle, and other vendors. JMS is a set of interfaces and associated semantics that define how a JMS client accesses the facilities of an enterprise-messaging product.

Oracle Java Messaging Service provides a Java API for Oracle Advanced Queuing (AQ) based on the JMS standard. Oracle JMS supports the standard JMS interfaces and has extensions to support the AQ administrative operations and other AQ features that are not a part of the standard.

JMS Terms and Definitions

JMS Text Message

JMS Text Message can represent a purchase order or an invoice. It contains the following components:

- Header: All messages support the same set of header properties which contain values used by both clients and providers to identify and route messages.

- **Properties:** In addition to the standard header properties, you can add optional header properties to a message. Properties can be standard properties, provider-specific, or application-specific properties.
- **Body:** This is the message payload. JMS defines various types of message payloads, as well as a type that can store JMS messages of any or all JMS-specified message types.

Oracle supports JMS Text messages using `aq$jms_text_message` or `aq$jms_message` types. Users must create the queue using one of these types as the payload.

JMS Topic

There are sample queues created for inbound and outbound JMS messages. These two seeded queues are `WF_JMS_IN` and `WF_JMS_OUT` to receive inbound and outbound JMS messages.

Note: In addition to these two seeded queues, Oracle XML Gateway allows JMS providers to create customized JMS queues for Business-to-Business transactions.

For example, when a message is sent to a Trading Partner using JMS messages, Oracle XML Gateway puts the messages in the `WF_JMS_OUT` topic.

JMS Client

A JMS client or provider is a Java-based client. A JMS client publishes a JMS Text message to a JMS topic. The Oracle Workflow Business Event System then processes this message from JMS queues.

Oracle XML Gateway and B2B Transactions Integration Points

To provide complete support for Business-to-Business transactions, Oracle XML Gateway provides the various points of Transport protocols for Trading Partner based message consumption and message generation. These various protocol integration points are:

- Oracle Transport Agent (OTA) Using the Seeded `ECX_INBOUND` and `ECX_OUTBOUND` Queues, page 9-3
- Web Services Using Seeded `WF_WS_JMS_IN` and `WF_WS_JMS_OUT` Queues, page 9-3
- JMS Client Application Using `WF_JMS_IN`, `WF_JMS_OUT`, or Any JMS Queues, page 9-3

Oracle Transport Agent Protocol

XML Gateway can receive and send XML messages between Trading Partners and Oracle E-Business Suite through OTA protocol using ECX_INBOUND and ECX_OUTBOUND queues.

Inbound Message Queue (ECX_INBOUND) holds all inbound messages that enter the process through the Transport Agent or placed directly by an API.

For outbound messages, XML Gateway creates XML messages and then enqueues them on this Outbound Message Queue (ECX_OUTBOUND).

See: Queues, page 5-1, Message Queues chapter for details.

Web Services Over SOAP Protocol

XML Gateway can receive and send XML messages as a document type Web service using the Simple Object Accessed Protocol (SOAP).

All inbound messages are prepared as designed event before being enqueued to the SOAP agent, WF_WS_JMS_IN queue. The Web Service IN Agent listeners pick up the message for further processing.

Outbound XML messages are created by Workflow processes and passed to the SOAP agent, WF_WS_JMS_OUT queue. The messages are picked up by the SOAP client who is responsible for the actual delivery of the message to the Trading Partner.

See: Process Flows, page 8-18, Web Service chapter for inbound and outbound process flow details.

JMS Client Application

By leveraging the Oracle Workflow Business Event System, Oracle XML Gateway allows receiving and sending JMS-based messages for business-to-business transactions between the Oracle E-Business Suite and Trading Partners using WF_JMS_IN, WF_JMS_OUT, or any JMS queues registered with Business Event System. See: Steps to Create Custom JMS Queues for Outbound B2B Transactions, page 9-7, and Steps to Create Custom JMS Queues for Inbound B2B Transactions, page 9-14.

How XML Gateway can be integrated with any JMS queues and the header properties for inbound and outbound transactions are explained in details in the next section.

Oracle XML Gateway and JMS Integration

Integration Features

To have seamless JMS integration and complete transaction monitoring and logging support, Oracle XML Gateway uses internal APIs to provide the following integration features:

- For inbound transactions, ability to provide complete Trading Partner user and transaction validation as well as authorization support so that XML Gateway only processes those messages that are valid and have the appropriate authorization.
- For outbound transactions, ability to provide a mechanism to verify at the runtime that the Trading Partner setup is complete and store the JMS queues as part of the setup and so that the generated messages can be sent to the desired JMS queue.

JMS Inbound and Outbound Messages

JMS Outbound Messages

After an application raises a business event for JMS queues, the event dispatcher catches the event, executes the event subscription to that event, enqueues the message and then places it to the default WF_JMS_OUT outbound agent or any customized outbound JMS queue registered with the Business Event System with the following JMS message header properties:

- ECX_MESSAGE_TYPE
- ECX_MESSAGE_STANDARD
- ECX_TRANSACTION_TYPE
- ECX_TRANSACTION_SUBTYPE
- ECX_PARTY_SITE_ID
- ECX_MSGID

A JMS provider then consumes the message placed in the JMS queue.

JMS Inbound Messages

A JMS provider publishes XML messages to an inbound JMS agent (WF_JMS_IN or any JMS queue) registered with the Business Event System. The User to Trading Partner and Trading Partner to Transaction validation and authorization will then be processed so that Oracle XML Gateway can process the valid message for the inbound transaction.

The following properties must be set in the JMS message header to enable Oracle XML Gateway to process the inbound message:

- ECX_MESSAGE_TYPE
- ECX_MESSAGE_STANDARD
- ECX_TRANSACTION_TYPE

- ECX_TRANSACTION_SUBTYPE
- ECX_PARTY_SITE_ID
- BES_EVENT_NAME=oracle.apps.ecx.jms.receive;
- BES_EVENT_KEY
- ECX_USERNAME (Optional)

Note: The ECX_USERNAME is used for authorization if the ECX: Enable User Check for Trading Partner profile option is enabled. The JMS payload must contain the transaction payload.

Impacts on the Trading Partner Setup and Seed Data

Impacts on Trading Partner Setup

To send a message to a Trading Partner using JMS messages, the appropriate protocol type must be set first in the Trading Partner Setup form, page 3-19:

1. Select JMS as the Protocol Type field for transactions enabled at the site level.
2. Select a JMS agent registered with the Business Event System (BES) for the Protocol Address field.

The agent can be WF_JMS_OUT or an agent you created and registered with the BES.

New Business Events and Event Subscriptions

New Business Events

Oracle XML Gateway uses the following seeded event name for inbound transactions through custom JMS queue:

- Event Name: oracle.apps.ecx.jms.receive
- Event Description: Event for JMS queues
- Status: Enabled
- Owner Name: Oracle Workflow

Event Subscriptions for JMS Queues

Business event for JMS queues, **oracle.apps.ecx.jms.receive**, has the following seeded subscriptions:

Event Subscriptions for JMS Queues

Descriptio n	Phase	Status	Source Type	Rule Function	WF Process Type	WF Process Name
Rule function to perform the User to Trading Partner and Trading Partner to Transaction validation and authorization	10	Enabled	External	ECX_RULE . TPPreProcessing	N/A	N/A
Rule function to perform the inbound processing	20	Enabled	External	ECX_RULE . receiveTP Message	N/A	N/A
Rule function for error handling	10	Enabled	Error	WF_RULE. error_rule	ECXMAIN	ECXERRO R

Creating Custom JMS Queues

Oracle XML Gateway provides you with seeded JMS queues (WF_JMS_IN and WF_JMS_OUT) and a mechanism allowing you to create custom JMS queues for your B2B transaction needs.

- Steps to Create Custom JMS Queues for Outbound B2B Transactions, page 9-7
- Steps to Create Custom JMS Queues for Inbound B2B Transactions, page 9-14

Steps to Create Custom JMS Queues for Outbound B2B Transactions

Performing Outbound B2B Transactions Using JMS Queues:

1. Create a JMS Topic for Outbound Messages

Create a JMS Topic for outbound processing by creating queues with payload type `aq$jms_text_message`.

Sample Code: (Create JMS topic, multiple consumers and grant the required permissions)

Following code creates a sample JMS queue `CUSTOM_IMS_OUT` in the `APPLSYS` schema.

```

/* Create Queue Table for CUSTOM_IMS_OUT Topic */
declare
    queue_table_exists exception;
    pragma EXCEPTION_INIT(queue_table_exists, -24001);
begin
    dbms_output.put_line('=====');
    dbms_output.put_line('Creating Queue Table for CUSTOM_JMS_OUT
Topic ');
    dbms_output.put_line('=====');
begin
    dbms_aqadm.create_queue_table
    (
        queue_table          => 'CUSTOM_JMS_OUT',
        queue_payload_type   => 'SYS.AQ$_JMS_TEXT_MESSAGE',
        sort_list            => 'PRIORITY,ENQ_TIME',
        multiple_consumers   => TRUE,
        comment              => 'Custom JMS Topic',
        compatible           => '8.1'
    );
exception
    when queue_table_exists then
        null;
    when others then
        dbms_output.put_line('Oracle Server Error = '||to_char
(sqlcode));
        dbms_output.put_line('Oracle Server Message = '||sqlerrm);
        raise_application_error(-20000, 'Oracle Error Mkr2= '
||to_char(sqlcode)||' -
'||sqlerrm);
    end;
end;
/
commit;

/* Create Topic for CUSTOM_JMS_OUT Topic */
DECLARE
    queue_exists exception;
    pragma EXCEPTION_INIT(queue_exists, -24006);
BEGIN
    dbms_output.put_line('=====');
    dbms_output.put_line('Creating CUSTOM_JMS_OUT Topic');
    dbms_output.put_line('=====');
begin
    dbms_aqadm.create_queue
    (
        queue_name          => 'CUSTOM_JMS_OUT',
        queue_table         => 'CUSTOM_JMS_OUT',
        comment             => 'Custom JMS Topic'
    );
exception
    when queue_exists then
        null;
    when others then
        dbms_output.put_line('Oracle Server Error = '||to_char
(sqlcode));
        dbms_output.put_line('Oracle Server Message = '||sqlerrm);
        raise_application_error(-20000, 'Oracle Error Mkr5= '
||to_char(sqlcode)||' -
'||sqlerrm);
    end;
END;
/
commit;

/* Start Topic */
declare

```

```

begin
    dbms_output.put_line('=====');
    dbms_output.put_line('Starting CUSTOM_JMS_OUT Queue ');
    dbms_output.put_line('=====');
    dbms_aqadm.start_queue(queue_name => 'CUSTOM_JMS_OUT');
    exception
        when others then
            dbms_output.put_line('Oracle Server Error = '||to_char
(sqlcode));
            dbms_output.put_line('Oracle Server Message = '||sqlerrm);
            raise_application_error(-20000, 'Oracle Error Mkr9= '
||to_char(sqlcode)||' -
'||sqlerrm);
end;
/
commit;

/* Creates Subscribers for Multiconsumer Queues */
REM
*****
REM  sqlplus apps/pwd@dbinst @custommoutqsubc.sql <wf schema> <wf
schema pwd>
REM
*****
    SET VERIFY OFF
    WHENEVER SQLERROR EXIT FAILURE ROLLBACK;
    connect &&1/&&2
    WHENEVER SQLERROR CONTINUE;
    declare
        lagent sys.aq$_agent;
        subscriber_exist exception;
        pragma EXCEPTION_INIT(subscriber_exist, -24034);
    begin
        lagent := sys.aq$_agent('CUSTOM_JMS_OUT',null,0);
        dbms_aqadm.add_subscriber(queue_name =>'&&1..CUSTOM_JMS_OUT',
subscriber=>lagent,
rule=>'1=1');
    exception
        when subscriber_exist then
            dbms_aqadm.alter_subscriber(queue_name =>'&&1..CUSTOM_JMS_OUT',
subscriber=>lagent,
rule=>'1=1');
        null; -- ignore if we already added this subscriber.
    end;
/
commit;
exit;

/* Create the necessary grants */

REM
*****
REM |          APPS username
REM |          APPS password
REM |          AOL  username
REM |          AOL  password
REM
*****
    WHENEVER SQLERROR EXIT FAILURE ROLLBACK;
    WHENEVER SQLERROR CONTINUE;
    WHENEVER OSERROR  EXIT FAILURE ROLLBACK;
    connect &3/&4
    REM create grants in APPLSYS
    grant all on custom_jms_out to &&1 with grant option;
    execute dbms_aqadm.grant_queue_privilege(privilege =>'ALL',

```

```

queue_name =>'&&3'||'.CUSTOM_JMS_OUT',grantee      =>'&&1',
grant_option=>TRUE);
  REM create synonyms in APPS
  WHENEVER SQLERROR CONTINUE;
  connect &1/&2
  create synonym custom_jms_out for &3..custom_jms_out;
  commit;
  exit;
/

```

2. Configure the Agent and Queue Handler for JMS Queues

Configure the agents and queue handler for the CUSTOM JMS topic through the Workflow Business Event Manager. Oracle Workflow provides a queue handler named wf_event_ojmstext_qh that provides enqueue and dequeue procedures for JMS topics.

The sample CUSTOM_JMS_OUT queue has the following properties:

Property	Value
Name	CUSTOM_JMS_OUT
Display Name	CUSTOM_JMS_OUT
Protocol	SQLNET
Address	<SCHEMA>.CUSTOM_JMS_OUT@<LOCAL DATABASE>
System	<LOCAL SYSTEM>
Queue Handler	WF_EVENT_OJMSTEXT_QH
Direction	OUT
Status	Enabled

3. Set Up JMS Protocol for Trading Partners in the Trading Partner Setup form, page 3-19

Use the following guidelines for custom JMS queues:

Trading Partner Setup Guidelines for Custom JMS Queues

Property	Value
Protocol_Type	JMS
Connection_Type	DIRECT
JMS_Queue_Name	JMS queue

4. Create a JMS Client and Subscribe to the Messages

Create a JMS Client that will subscribe to the JMS queue and consume the messages. All the message parameters will be available as part of the JMS message. The generated XML will be available as the payload in the JMS message.

Sample Code: (JMS Subscriber)

```
/**
 * This is a sample java file which uses Oracle JMS - Java
 * Messaging Service
 * API to retrieve text Message.
 *
 * This demo does the following:
 * -- Setup Connection
 * -- Get Session from Connection
 * -- Get topic from session
 * -- Create a Durable Subscriber for this topic
 * -- Receive the Message for subscriber
 *
 * The following instructions describe how to compile and execute
 * this sample on the client machine.
 *
 * System requirements:
 * =====
 * 1) The client machine should have JDK 1.1.x or JDK1.2 or higher
 * installed
 * 2) The following jar/zip files should be in the CLASSPATH on
 * the client
 * machine.
 * For JDK1.2.x
 * classes12.zip
 * aqapi.jar
 * jmscommon.jar
 * For JDK1.1.x
 * classes111.zip
 * aqapi11.jar
 * jmscommon.jar
 * Set up CLASSPATH, PATH, LD_LIBRARY_PATH based on JDK version and
 * platform.
 * Compilation and Running:
 * =====
 * 3) If you already have the jars in step 2) in classpath
 * javac JMSSubscriber.java
 * 4) java JMSSubscriber
 */
import javax.jms.JMSEException;
import javax.jms.Session;
import javax.jms.TextMessage;
import javax.jms.Topic;
import javax.jms.TopicSession;
import javax.jms.TopicSubscriber;
import oracle.jms.AQjmsConnection;
import oracle.jms.AQjmsFactory;
import oracle.jms.AQjmsSession;
import oracle.jms.AQjmsTopicConnectionFactory;
import oracle.jms.AQjmsTextMessage;
public class JMSSubscriber {
    /**
     * TopicConnectionFactory object used to create topic connections
     */
    AQjmsTopicConnectionFactory topicConnectionFactory = null;
    Topic topic = null;
    /** JMS Connection object */
    AQjmsConnection topicConnection = null;
    /**
     * TopicSession object for Subscribers
     */
    TopicSession subTopicSession = null;
    /**
     * Default Constructor
     */
}
```

```

public JMSSubscriber() {
    try {
        /* Get Connection Factory for topic with specified TNS and
        driver */
        topicConnectionFactory = (AQjmsTopicConnectionFactory)
        AQjmsFactory
        .getTopicConnectionFactory
        ("ap601sdb",
        "atgwfddev", 4135, "thin");

        /* Create topicConnection with username and password */
        topicConnection = (AQjmsConnection) topicConnectionFactory
        .createTopicConnection("apps", "password");

        /* Create subscriber TopicSession */
        subTopicSession = topicConnection.createTopicSession(false,
        Session.AUTO_ACKNOWLEDGE);
        /* Get topic object from schema and queue */
        topic = ((AQjmsSession) subTopicSession).getTopic("appls",
        "custom_jms_in");
        /*
        * Start Connection. It is in Stopped mode by default
        */
        topicConnection.start();
    }
    catch (JMSEException ex) {
        System.out.println("Exception = " + ex.toString());
    }
}
/**
 * Method to close topicConnection
 */
@throws JMSEException
*/
public void close() throws JMSEException {
    topicConnection.close();
}
/**
 * Method to recieve messages from topic
 */
public void recieveMessages() {
    try {
        TopicSubscriber topicSubscriber = subTopicSession
        .createDurableSubscriber(topic,
        "Subscriber1");
        boolean done = false;
        System.out.println("Dequeue Messages for Subscriber ");
        AQjmsTextMessage msg = null;
        while (!done) {
            msg = (AQjmsTextMessage) topicSubscriber.receiveNowait();
            if (msg == null) {
                done = true;
            }
            else {
                System.out.println("Recieved Message with ID : " +
                msg.getJMSMessageID());
                System.out.println("Message Properties -");
                System.out.println("ECX_MESSAGE_TYPE : " +
                msg.getStringProperty
                ("ECX_MESSAGE_TYPE"));
                System.out.println("ECX_MESSAGE_STANDARD : " +
                msg.getStringProperty
                ("ECX_MESSAGE_STANDARD"));
                System.out.println("ECX_TRANSACTION_TYPE : " +
                msg.getStringProperty

```

```

("ECX_TRANSACTION_TYPE"));
        System.out.println("ECX_TRANSACTION_SUBTYPE : " +
            msg.getStringProperty
("ECX_TRANSACTION_SUBTYPE"));
        System.out.println("ECX_PARTY_SITE_ID : " +
            msg.getStringProperty
("ECX_PARTY_SITE_ID"));
        System.out.println("BES_EVENT_NAME : " +
            msg.getStringProperty
("BES_EVENT_NAME"));
        System.out.println("BES_EVENT_KEY : " +
            msg.getStringProperty
("BES_EVENT_KEY"));
        System.out.println("Message Text -");
        System.out.println(msg.getText());
    }
}
catch (Exception e) {
    e.printStackTrace();
}
}
public static void main(String[] args) {
    JMSSubscriber jMSSubscriber = new JMSSubscriber();
    jMSSubscriber.receiveMessages();
    try {
        /* Close topicConnection */
        jMSSubscriber.close();
    }
    catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Related Topics

Steps to Create Custom JMS Queues for Inbound B2B Transactions, page 9-14

Java Messaging Service (JMS) Overview, page 9-1

Oracle XML Gateway and B2B Transactions Integration Points, page 9-2

Steps to Create Custom JMS Queues for Inbound B2B Transactions

Performing Inbound B2B Transactions Using JMS Queues:

1. Create a JMS Topic for Inbound Messages

Create a JMS Topic for inbound processing by creating queues with payload type `aq$jms_text_message`.

See: Step 1, Performing Outbound B2B Transactions Using JMS Queues, page 9-7 for details. Replace the `CUSTOM_JMS_OUT` queue with `CUSTOM_JMS_IN` to create a JMS queue for inbound messages.

2. Configure the Agent and Queue Handler for JMS Queues

See: Step 2, Performing Outbound B2B Transactions Using JMS Queues, page 9-10 for details. Replace the CUSTOM_JMS_OUT queue with CUSTOM_JMS_IN and use direction as IN.

3. Create a JMS Client and Publish Messages to the Custom JMS Queue

Create a JMS Client that will publish messages to the JMS queue. When publishing the messages, there are some XML Gateway specific parameters that should be available as part of the JMS message:

Parameters Required in the Custom JMS Message

Name	Value
ECX_TRANSACTION_TYPE	External Transaction Type
ECX_TRANSACTION_SUBTYPE	External Transaction Subtype
ECX_MESSAGE_STANDARD	Message Standard
ECX_MESSAGE_Type	Message Type
ECX_PARTY_SITE_ID	Source TP Location Code
BES_EVENT_NAME	oracle.apps.ecx.jms.receive
BES_EVENT_KEY	Unique key for every run
ECX_USERNAME (Optional)	Valid Oracle E-Business Suite Users

Note: The XML Payload should be set as the payload in the JMS message. Any other optional parameters specific to the processing should also be provided as part of these name value pairs.

Sample Code: (JMS Publisher)

```
/**
 * This is a sample java file which uses Oracle JMS - Java
 * Messaging Service
 * API to publish text Message.
 *
 * This does the following:
 * -- Setup Connection
 * -- Get Session from Connection
 * -- Setup topic
 * -- Create a publisher for this topic
 * -- Publish Message to the Topic
 *
 * The following instructions describe how to compile and execute
 * this sample on the client machine.
 *
 * System requirements:
 * =====
 * 1) The client machine should have JDK 1.1.x or JDK1.2 or higher
installed
 * 2) The following jar/zip files should be in the CLASSPATH on
the client
 * machine.
 * For JDK1.2.x
 * classes12.zip
 * aqapi.jar
 * jmscommon.jar
 * For JDK1.1.x
 * classes111.zip
 * aqapi11.jar
 * jmscommon.jar
 * Set up CLASSPATH, PATH, LD_LIBRARY_PATH based on JDK version
and platform.
 * Compilation and Running:
 * =====
 * 3) If you already have the jars in step 2) in classpath
 * javac JMSPublisher.java
 *
 * 4) java JMSPublisher
 */
import javax.jms.JMSException;
import javax.jms.Session;
import javax.jms.TextMessage;
import javax.jms.Topic;
import javax.jms.TopicPublisher;
import javax.jms.TopicSession;
import oracle.jms.AQjmsConnection;
import oracle.jms.AQjmsFactory;
import oracle.jms.AQjmsSession;
import oracle.jms.AQjmsTextMessage;
import oracle.jms.AQjmsTopicConnectionFactory;

public class JMSPublisher {
    /**
     * TopicConnectionFactory object used to create topic connections
     */
    AQjmsTopicConnectionFactory topicConnectionFactory = null;
    Topic topic = null;
    /** JMS Connection object */
    AQjmsConnection topicConnection = null;
    /**
     * TopicSession object for Publishers.
     */
    static AQjmsSession pubTopicSession = null;
```

```

TopicPublisher topicPublisher = null;
/**
 * Default Constructor
 */
public JMSPublisher() {
    try {
        /* Get Connection Factory for topic with specified TNS and
driver*/
        topicConnectionFactory = (AQjmsTopicConnectionFactory)
AQjmsFactory
                                .getTopicConnectionFactory
("ap601sdb",
                                "atgwfddev", 4135, "thin");
        /* Create topicConnection with username and password*/
        topicConnection = (AQjmsConnection) topicConnectionFactory
                                .createTopicConnection("apps", "password");
        /* Create publisher TopicSession*/
        pubTopicSession = (AQjmsSession) topicConnection
                                .createTopicSession(false, Session.
AUTO_ACKNOWLEDGE);
        /* Get topic object from schema and queue*/
        topic = ((AQjmsSession) pubTopicSession).getTopic("applsys",
                                "custom_jms_in");
        /* Create Publisher*/
        topicPublisher = pubTopicSession.createPublisher(topic);
        /* Start Connection.
        * It is in Stopped mode by default
        */
        topicConnection.start();
    }
    catch (JMSEException ex) {
        System.out.println("Exception = " + ex.toString());
    }
}

/**
 * Method to close topicConnection
 * @throws JMSEException
 */
public void close() throws JMSEException {
    topicConnection.close();
}

/**
 * Method to publish message to topic
 * @return msgid MessageId of the published Message
 */
public String publishMessage() {
    String msgid = null;
    AQjmsTextMessage msg = null;
    try {
        /*
        * Create Text Message
        */
        msg = (AQjmsTextMessage) pubTopicSession.createTextMessage();
        /*
        * Set its properties
        */
        msg.setStringProperty("ECX_MESSAGE_TYPE", "XML");
        msg.setStringProperty("ECX_MESSAGE_STANDARD", "OAG");
        msg.setStringProperty("ECX_TRANSACTION_TYPE", "INVOICE");
        msg.setStringProperty("ECX_TRANSACTION_SUBTYPE", "PROCESS");
        msg.setStringProperty("ECX_PARTY_SITE_ID", "Business World");
        msg.setStringProperty("BES_EVENT_NAME", "oracle.apps.ecx.jms.
receive");
        msg.setStringProperty("BES_EVENT_KEY", "n6");
    }
}

```

```

/*
 * Set Payload.Here its XML text
 */
String payload = new String(
"<ECX_MAPS><ECX_MAPPINGS><MAP_ID>1</MAP_ID><MAP_CODE>hehe</MAP_CODE>
</ECX_MAPPINGS></ECX_MAPS>");
msg.setText(payload);
/*
 * Publish Message to topic
 */
topicPublisher.publish(topic, msg);

/* retrieve message id of posted message*/
msgid = msg.getJMSMessageID();
}
catch (JMSEException e) {
    e.printStackTrace();
}

return msgid;
}
public static void main(String[] args) {
    JMSPublisher jMSPublisher = new JMSPublisher();
    /* Post a message and get message id of posted message*/
    String msgid = jMSPublisher.publishMessage();
    /* Message Id is of Format - ID:<id>
     * Example ID:ASFG43543JH5K435H5K4
     * So retrieve correct id
     */
    msgid = msgid.substring(3, msgid.length() - 1);

    System.out.println(" Message ID = " + msgid);

    try {
        /* Close topicConnection */
        jMSPublisher.close();
    }
    catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Related Topics

[Steps to Create Custom JMS Queues for Outbound B2B Transactions, page 9-7](#)

[Java Messaging Service \(JMS\) Overview, page 9-1](#)

[Oracle XML Gateway and B2B Transactions Integration Points, page 9-2](#)

Collaboration History

This chapter covers the following topics:

- Understanding Collaboration History
- Using the Collaboration History
- Using the Collaboration History Window
- Using Collaboration History Simple Search
- Using Collaboration History Advanced Search
- Viewing the Collaboration History Search Results
- Using the Collaboration Events Window
- Viewing the Collaboration Event Messages
- Viewing the Collaboration Event XML Payload
- Using the Collaboration Event Definitions Window
- Creating a Collaboration Event Definition
- Updating a Collaboration Event Definition
- Using the Collaboration Final Event Definitions Window
- Creating a Collaboration Final Event Definition
- Updating a Collaboration Final Event Definition
- Using Collaboration Properties
- Creating a Collaboration Property
- Updating a Collaboration Property
- Using the Collaboration Display Setup
- Creating a Collaboration Display Label Header
- Updating the Collaboration Display Header and Details
- Using the Notification Definition Window

- Creating a Notification Definition
- Updating a Notification Definition
- Using the Notification Action Definitions Window
- Creating a Notification Action Header
- Updating a Notification Action Header
- Creating and Updating Notification Action Details
- Viewing Notification Action Definition Details
- Viewing Action Definition Details

Understanding Collaboration History

To easily track business transactions raised within Oracle E-Business Suite and messages exchanged with trading partners, Oracle XML Gateway uses the Collaboration History feature to record these business processes or events such as purchase order, change order. When an order is cancelled or created, relevant actions can be triggered such as notify the document owner or trading partner, or raise business events for the acknowledgement or exception handling. All of these business processes are tracked and recorded using the Collaboration History.

In order to capture business transactions or collaborations and also take actions in acknowledging or handling the business events, specifically Collaboration History comprises the following two components:

- Collaboration History, page 10-2
- Notification Processing, page 10-4

Collaboration History

Collaboration History records all business transactions and messages exchanged with trading partners.

Collaboration History allows you to:

- Record all key processing events of a unique collaboration or Partner Interface Process (PIP) from the initial event or message to the final message, including any exceptions issued by the trading partner or the business-to-business gateway.
- Search history by business document number, date, status, document owner, or trading partner.
- Reinitiate a collaboration.
- View the XML payloads associated with a particular collaboration.

- Use workflow events to create, update, and add details of collaboration history when implementing a collaboration.

Collaborations

A collaboration is an association where an enterprise that uses Oracle E-Business Suite exchanges messages with a trading partner. The trading partners use either an Oracle E-Business Suite application or the Internet for exchange of messages. Collaborations represent business processes such as purchase order, change order, inventory report, and item master. Collaborations usually require more than one interaction among the trading partners involved, with some involving existing business workflows.

Collaboration Events

For any collaboration, there can be many collaboration events. A collaboration event represents a system component that acts on information in a message or XML payload. Oracle XML Gateway generating an Open Applications Group (OAG) XML message and an Oracle E-Business Suite application accepting a change electronically are both collaboration events.

A collaboration event is supported by a XML payload or some collaboration event details. Some events have associated XML like an OAG CONFIRM_BOD or PROCESS_PO message.

For example, the Order Collaboration events are recorded as follows:

- After a purchase order (PO) is approved and the collaboration is created or initiated, a new unique collaboration ID is assigned.
- When the Oracle XML Gateway receives the PO Approval event and generates the OAG PROCESS_PO message. Once the message is available in the outbound Advanced Queue (AQ), a collaboration event is recorded.
- When the business-to-business gateway picks up the message from the buy side Oracle E-Business Suite AQ, transforms it to a RosettaNet PO request, and sends it to the trading partner gateway through the Internet. The trading partner gateway returns an acknowledgement. This acknowledgement signal is transformed into an OAG CONFIRM_BOD by the business-to-business gateway and placed on the Oracle E-Business Suite AQ. During the inbound processing of this BOD by the Oracle XML Gateway, a collaboration event is recorded.
- When the trading partner returns the PO confirmation request, the business-to-business gateway transforms it into an OAG ACKNOWLEDGE_PO BOD, and lists it on the buy side Oracle E-Business Suite AQ. During the inbound processing of this message by the Oracle XML Gateway, a collaboration event is recorded.
- When the buy side Oracle E-Business Suite recognizes the ACKNOWLEDGE_PO BOD and generates an outbound Confirm_BOD. During the inbound processing of the Acknowledge_PO BOD by the Oracle XML Gateway and after the

Confirm_BOD is generated, a collaboration event is recorded.

- Use the resend feature of Collaboration History to reinitiate any eligible failed collaboration. An update event is raised to update the Collaboration History.
- Whenever there is an exception in the business-to-business gateway of the host or that of the trading partner. An OAG CONFIRM_BOD is listed on the host Oracle E-Business Suite AQ by the Oracle XML Gateway and a collaboration event is recorded.
- Whenever there is an exception in the Oracle XML Gateway.

Collaboration Event Details

The collaboration event details contain information received from a trading partner or other target applications. The information can also include error messages and exceptions.

Notification Processing

When an order is cancelled, Collaboration History uses the Notification Processing feature to provide generic and consistent methods to capture and take actions in response to the collaboration event.

Notification Processing allows you to:

- Capture notification messages from sources outside the applications as business-to-business gateways or trading partners.
- Capture exceptions generated by the Oracle XML Gateway.
- Record notifications in Collaboration History.
- Perform actions based on the notification settings as Notify System Administrator, Notify Trading Partner, Call a predefined API, Raise a predefined Business Event, and Start Workflow.
- Provide predefined notification actions for all supported collaborations.
- Permit customization of preseeded notifications to fit your business requirements.
- Establish notification actions for new collaborations defined in Collaboration History.

Concepts and Terminology

Notification Processing is structured as notification types, notifications, and notification actions. Additionally, objects from the Collaboration History, such as applications and collaborations, are referenced and used to define portions of Notification Processing.

Notifications are raised to represent the success, exception, or error at predefined points within a collaboration. A notification is classified based on the source, that is, application, XML Gateway, or the business-to-business gateway. A notification action defines the action taken at a predefined point within a collaboration, based upon the notification raised.

The relationship between Collaboration History and Notification Processing is explained. Under the Notification Processing Subsystem, notification is defined for a notification source. Under Collaboration History Subsystem, collaboration is defined under applications. Notification actions are defined for a combination of collaboration and notification.

A notification is uniquely identified by a notification code and a notification source, which describes the location of the event within a given collaboration. An example of a notification source is the XML Gateway or the business-to-business gateway. A notification code is the actual error, warning, or status message passed from the notification source. Additionally, a description is specified for display purposes where notifications are referenced elsewhere.

Characteristics of a Notification Action

A notification action has the following characteristics:

- A collaboration which is an occasion where messages are communicated between an enterprise that uses the Oracle E-Business Suite and a trading partner who uses the Internet or two Oracle E-Business Suite applications. Collaborations represent business processes such as an order, change order, forecast, or work-in-process action.
- An application that owns the collaboration. For example, Oracle Purchasing or Oracle Order Management.
- A notification source which describes the software component generating the notification.
- A notification to describe the success, error, or warning.

All preseeded notification actions that handle error conditions are configured with the default action of notifying the administrator.

Additional business rules, such as initiating a workflow to place an inbound purchase order on hold when a trading partner identification error is encountered in a given collaboration, are specified using Notification Processing.

Example Flows

The notification processing engine handles the notifications from different sources such as the business-to-business gateway, Oracle XML Gateway, and Oracle E-Business Suite applications.

Business-to-Business Gateway

The business-to-business gateway generates a notice to Oracle E-Business Suite if it identifies an issue with the collaboration, such as Time to perform Exceeded (Notification Code: IP-05). The business-to-business gateway forms an OAG CONFIRM BOD and places it on the Oracle XML Gateway AQ. The Oracle XML Gateway processes the OAG CONFIRM BOD and passes it to the notification processing engine through an event subscription. The engine records an entry in the Collaboration History. Based on the notification code in the Confirm BOD and the setup data, the engine runs the indicated actions.

XML Gateway

The Oracle XML Gateway generates the XML messages outbound from Oracle E-Business Suite and uses the inbound messages to invoke the appropriate Oracle E-Business Suite APIs to affect a transaction. While processing these messages, if an error occurs, then the Oracle XML Gateway stops the process and raises a business event. The notification code is passed as a parameter of the event.

Oracle E-Business Suite Applications

For the inbound messages, APIs are called from the Oracle XML Gateway Message map. If the API returns an exception, then the application raises a business event. For example, the Order Import API raises an exception such as Invalid Sales Order Status (Notification Code: SO_IN03). This notification code is passed as a parameter of the event. The notification processing engine subscribes this event. The engine records an entry in the Collaboration History. Based on the notification code and the setup data, the engine runs the indicated actions.

Using the Collaboration History

Collaboration History uses the following windows to provide its features:

- Collaboration History Window, page 10-7
- Collaboration Events Window, page 10-10
- Collaboration Event Definitions Window, page 10-12
- Collaboration Final Event Definitions Window, page 10-15
- Collaboration Properties Window, page 10-18
- Collaboration Display Setup Window, page 10-20
- Notification Definition Window, page 10-23
- Notification Action Definitions Window, page 10-26

Using the Collaboration History Window

Use the Collaboration History window to view or monitor different business transactions and messages consumed by Oracle XML Gateway and exchanged between trading partners and Oracle E-Business Suite.

To view collaboration history, page 10-9, use the following ways to define your search:

- Using a Simple Search, page 10-7
- Using an Advanced Search, page 10-8

To set up collaboration display information, see Using the Collaboration Display Setup Window, page 10-20 and Updating the Collaboration Display Header and Details, page 10-22 for details.

Using Collaboration History Simple Search

The Collaboration History simple search enables you to search collaborations based on application, document number, collaboration type, and collaboration standard. Enter the search criteria and initiate a query to view the collaborations.

Prerequisites

Approve a purchase order and initiate an order collaboration.

To find collaborations using the simple search feature:

1. Navigate to the Collaboration History window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor > Collaboration History.
2. To narrow the search in the Search region:
 - Select Collaboration Standard such as OAG, Rosettanet, UCCnet or you can leave it blank.
 - Select an appropriate Application.
 - Select Collaboration Type as one of the many collaboration types used. This includes values such as Change Order, Cancel Order, Change Planning, Change Order Notification, Forecast.
 - Enter Document Number such as a PO number or a customer order number. You can perform a search to find a value.

3. Click **Go** to view the search results. See: Viewing the Collaboration History Search Results, page 10-9.

Using Collaboration History Advanced Search

Use the advanced search to search collaborations by a range of documents for a specific collaboration type. Enter the search criteria and initiate a query to view the collaborations.

Please note that you may enter additional search criteria through the advanced search if you have the Collaboration Standard, Application, and Collaboration Type fields in the simple search region specified and these values are corresponded with the fields you defined as the Display Labels Header for the Collaboration Display Setup window if it is set up properly.

For example, you enter "OAG" as collaboration standard, "Purchasing" as application, and "Order" as collaboration type in the simple search region. When clicking **Show More Search Options**, you can find additional search fields populated if you have the Collaboration Display Setup window defined properly for the collaborations with the same collaboration standard "OAG" and collaboration type "Order" in Oracle Purchasing application.

Note: You do not need to specify all the three fields mentioned here in the simple search region for additional search fields if the Collaboration Standard and Application, for example, are the only two fields that you defined in the Collaboration Display Setup window. In this case, you only need to enter "OAG" as collaboration standard and "Purchasing" as the application without specifying the collaboration type, such as "Order", and you can still see the additional search fields displayed in the advanced search region.

See Using the Collaboration Display Setup Window, page 10-20 and Updating the Collaboration Display Header and Details, page 10-22 for details.

Prerequisites

- Approve a purchase order and initiate an order collaboration.

To find collaborations using the advanced search feature:

1. Navigate to the Collaboration History window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor > Collaboration History.
2. Click the **Show More Search Option** link to see more search fields displayed in the Collaboration History window in addition to the Simple Search Criteria, page 10-7,

collaboration standard, type, application, and document number

3. To narrow the search with additional criteria:

- Select Organization name for your search.
- Enter the Initiate Date From and Initiate Date To fields as the range to search for transactions that are initiated, terminated, or completed.
- Enter Collaboration as a specific collaboration instance by entering the number. You can perform a search to find a desired value.
- Enter Trading Partner for your search. You can perform a search to find a desired value.
- Select Collaboration Status as Completed Normal, Completed Error, Initiated, Resent, Started, or Terminated.
- Select Disposition from the drop-down list with the values such as Accepted, Failed, Pending, Reviewed, Synchronized, or Rejected.

The Disposition indicates the outcome of a collaboration, which can be rejected by a trading partner even though the collaboration is completed normally.

4. Click **Go** to view the search results. See: Viewing the Collaboration History Search Results, page 10-9.

Viewing the Collaboration History Search Results

After a simple or an advanced search, collaborations that match the search criteria are displayed in the Collaboration History search results region in the Collaboration History windows. Each line in the results region is for one collaboration. For each collaboration, you can the following information displays: document number or other display name (such as "PO") specified in the Collaboration Display Setup window, collaboration, collaboration type, collaboration standard, application, collaboration date, trading partner, resend count, collaboration status, and disposition.

Note: In addition to Document Number, you can also rename the following three more parameters in the Collaboration Display Setup window for your applications: Document Owner, Message Initiator, and Partner Document Number. Also, you may see additional fields displayed in the search result region if you have the collaboration display information defined properly in the Collaboration Display Setup window and have those additional fields selected in the Display Enabled For Results Table checkbox. See Using the Collaboration Display Setup Window, page 10-20 and Updating the Collaboration

Display Header and Details, page 10-22 for details.

From the Collaboration History window, you can perform the following tasks:

- View the collaboration search results
- Reinitiate collaborations that resulted in errors by selecting one or more collaborations in the enabled **Select** checkbox and then click **Reinitiate**.
 - When reinitiating a collaboration, you will only resend the initiating message to the trading partner, not all the events in the collaboration. This rule should apply for both inbound and outbound messages on the Collaboration History window.
 - Collaborations whether they can be resent depend on their associated message types. If the **Select** checkbox is not enabled, then you cannot resend the collaboration. For more information on message types, see Creating a Collaboration Property, page 10-19.
- View the details of a collaboration by clicking the collaboration number link you want to see to access the Collaboration Events window where displays all events for the selected collaboration. See: Using the Collaboration Events Window, page 10-10.

Using the Collaboration Events Window

Use the Collaboration Events window to view read-only information for the collaboration details, such as collaboration number, status, date, type, document number, create and revision date.

Note: You may see additional fields displayed here if you have the collaboration display information defined properly in the Collaboration Display Setup window and have those additional fields selected in the Display Enabled For Events Screen checkbox. See Using the Collaboration Display Setup Window, page 10-20 and Updating the Collaboration Display Header and Details, page 10-22 for details.

In addition, all events for the collaboration are also listed in a table, including collaboration point, reference number, document type, direction, status, and messages when the trading partner returns a confirmation or rejection. You can also drill down to view event messages and payload information if it is available.

Prerequisites

- Search for a collaboration.
- Enable Message in the Oracle XML Gateway for a collaboration point.
- Enable Payload in the Oracle XML Gateway for a collaboration point.

To view collaboration events messages and payload information:

1. Navigate to the Collaboration Events window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor > Collaboration History. Click the collaboration number link that you want to see from the Collaboration History search results.
2. Click the enabled **Message** link for a Collaboration Point to view the message details. See: Viewing the Collaboration Event Messages, page 10-11.
3. Click the enabled **Payload** for a Collaboration Point to view the payload details. See: Viewing the Collaboration Event XML Payload, page 10-11.

Viewing the Collaboration Event Messages

By clicking the enabled Message link in the Collaboration Events window, you can view the message details, including document number, reference 1, reference 2, reference 3, reference 4, reference 5, and message text. The reference fields store additional information about the event. The value of the reference fields are PIP dependent.

For example, for a 3A4, if an acknowledgement has five lines out of which two are rejected and three are accepted, then all five lines display in the event details; reference 1 is line number, reference 2 is item, reference 3 is accepted or rejected, and so on.

The message text contains the actual message, which explains the reason for accepting or rejecting a collaboration event handled by Oracle XML Gateway.

Viewing the Collaboration Event XML Payload

The Collaboration Event Payload window displays payload details, such as document number, collaboration number, collaboration point, type, event message, XML payload details.

Click **Printable Page** to print the collaboration event payload information.

Using the Collaboration Event Definitions Window

Use the Collaboration Event Definitions window to query existing events. After performing a search, all events match with your search criteria are displayed in the results region.

In addition, you can also create a new event, update and delete an existing event. All collaboration events associated with the XML payload are set up using the Collaboration Event Definitions window.

Prerequisites

- Verify if you have the Workflow Administrator Web Applications responsibility to access the Collaboration Monitor: Setup privilege.
- Define the necessary lookups.

To search for a collaboration event definition:

1. Navigate to the Collaboration Event Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Event Definition.
2. To narrow the search:
 - Select Transaction Type as the type of transaction in use. This list of values is preseeded or you can define the transaction types in the Oracle XML Gateway.
 - Select Transaction Subtype as the subtype of a transaction in use. This list of values is preseeded or you can define the transaction subtypes in the Oracle XML Gateway.
 - Select Document Direction as In or Out.
 - Select the collaboration Application such as Oracle Purchasing, iProcurement Connector, Oracle Inventory.
 - Select Document Type as one of the many documents used in the business transactions. This list includes values such as Acknowledge PO, Change PO, Item Master.
 - Select Collaboration Type as one of the many collaboration types used. This list includes values such as Cancel Order, Change Order Notification, Forecast.
3. Click **Go** to execute the search.

To create a collaboration event definition:

1. Navigate to the Collaboration Event Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Event Definition.
2. Click **Create Definition** to create a new collaboration event in the Create Collaboration Event Definition window. See: Creating a Collaboration Event Definition, page 10-13.

To update or delete a collaboration event definition:

1. Navigate to the Collaboration Event Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Event Definition.
2. Search the collaboration event definition by entering search criteria.
3. Click the enabled **Update** for the collaboration event definition. The Update Collaboration Event Definition window appears. See: Updating a Collaboration Event Definition, page 10-14.
4. Click **Delete** to delete a collaboration event definition. You are prompted to confirm the deletion.

Creating a Collaboration Event Definition

Use the Create Collaboration Event Definition window to create a new collaboration event definition.

Prerequisites

- Verify if you have the Workflow Administrator Web Applications responsibility to access the Collaboration Monitor: Setup privilege.
- Define the necessary lookups.
- Enter the new collaboration type in the relevant lookup for a new collaboration.
- Enter the new application in the relevant lookup for a new application.
- Enter the new document type in the relevant lookup for a new document type.

To create a collaboration event definition:

1. Navigate to the Create Collaboration Event Definition window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Event Definition > the Create Definition button.
2. Select the values for the following fields:
 - Select Transaction Type as the type of transaction for your new event definition. This list of values is preseeded or you can define the transaction types in the Oracle XML Gateway.
 - Select Transaction Subtype as the subtype of a transaction. This list of values is preseeded or you can define the transaction subtypes in the Oracle XML Gateway.
 - Select Document Direction as In or Out.
 - Select the collaboration Application for your new event definition.
 - Select Document Type to be used in the business transactions.
 - Select Collaboration Type for your new event definition from the list of values such as Cancel Order, Change Order Notification, Forecast.
3. Click **Apply** to create a new collaboration event definition.

If the combination of values provided for the Transaction Type, Transaction Subtype, and Document Direction combination is not defined in the Oracle XML Gateway Transactions Setup, then an error message displays.

Updating a Collaboration Event Definition

Use the Update Collaboration Event Definition window to update an existing collaboration event definition.

Prerequisites

- Verify if you have the Workflow Administrator Web Applications responsibility to access the Collaboration Monitor: Setup privilege.
- Define the necessary lookups.

To update a collaboration event definition:

1. Navigate to the Collaboration Event Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Event Definition.
2. Search the collaboration event definition by entering search criteria in the fields.
3. Click the enabled **Update** for a Transaction Type in the search results area.
4. Update the collaboration event by selecting appropriate values for the Transaction Type, Transaction Subtype, Document Direction, Document Type, Application, and Collaboration Type fields.
5. Click **Apply** to commit the modifications.

If the combination of values provided for the Transaction Type, Transaction Subtype, Document Direction, Application, Document Type, and Collaboration Type combination is not defined in Oracle XML Gateway Transactions Setup, then an error message displays.

See Setting Up a New Collaboration chapter for details.

Using the Collaboration Final Event Definitions Window

The final collaboration event is generally an occurrence of an inbound or outbound message in the Oracle XML Gateway. It is set up in Collaboration History to record the final status of the overall collaboration.

Use the Collaboration Final Event Definitions window to query, update, and delete existing final events as well as create new final events.

To search a collaboration final event definition:

1. Navigate to the Collaboration Final Event Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Final Event Definition.
2. To narrow the search:
 - Select an appropriate value for the Application.
 - Select Collaboration Type for your search, such as Change Order, Cancel Order.
 - Select Document Type for the documents used in the business transactions, such as Acknowledge PO, Change PO, Item Master.

- Select Document Direction as In or Out.
3. Click **Go** to perform a search.

To create a collaboration final event definition:

1. Navigate to the Collaboration Final Event Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Final Event Definition.
2. Click **Create Definition** to create a new final event in the Create Collaboration Final Event Definition window.

See: Creating a Collaboration Final Event Definition, page 10-16.

To update or delete a collaboration final event definition:

1. Navigate to the Collaboration Final Event Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Final Event Definition.
2. Perform a search by entering search criteria.
3. Click the enabled **Update** for an Application in the search results area. The Update Collaboration Final Event Definition window appears.

See: Updating a Collaboration Final Event Definition, page 10-17.

4. Click **Delete** for the associated collaboration final event definition. You are prompted to confirm the delete action.

Creating a Collaboration Final Event Definition

Use the Create Collaboration Final Event Definition window to create a new collaboration final event definition.

To create a collaboration final event definition:

1. Navigate to the Create Collaboration Final Event Definition window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Final Event Definition > the Create Definition button.
2. Select the values for the following fields:
 - Select a value for the collaboration Application.

- Select Collaboration Type from the drop-down list.
- Select Document Type for your new final event definition.
- Select Document Direction as In or Out.
- Select Collaboration Standard from the drop-down list.
- Enter Message Count information for your new final event definition.

The Message Count field represents the total numbers of messages with the same application, document type, collaboration type, collaboration standard, and document direction.

3. Click **Apply** to create a new collaboration final event definition.

If the combination of values provided for the Application, Collaboration Type, Document Type, and Document Direction combination is not defined in Oracle XML Gateway Transactions Setup, then an error message displays.

Updating a Collaboration Final Event Definition

After performing a search, you can update an existing final event definition by clicking the Update button associated with the event you want to update. The Update Collaboration Final Event Definition window appears.

Prerequisites

- Set up and activate the Collaboration Event Definition.

To update a collaboration final event definition:

1. Navigate to the Update Collaboration Final Event Definition window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Final Event Definition > the Update button.
2. Update your final event definition in the Application, Collaboration Type, Document Type, Document Direction, Collaboration Standard, and Message Count fields.
3. Click **Apply** to commit the modifications made to the selected collaboration final event definition.

If the combination of values provided for the Application, Document Type, Collaboration Type, and Document Direction combination is not defined in Oracle XML Gateway Transactions Setup, then an error message displays.

Using Collaboration Properties

Use the Collaboration Properties window to associate collaboration types and application names with appropriate message types. These message types indicate the different nature of the collaborations. Based on the message types, collaborations displayed in the Collaboration History window can be resent in case they fail.

You can query existing collaboration properties, create new collaboration properties, and update or delete existing ones in the Collaboration Properties window.

To search a collaboration property:

1. Navigate to the Collaboration Property window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Property.
2. To narrow the search:
 - Select Application such as Oracle Purchasing, Oracle Inventory.
 - Select Collaboration Type such as Cancel Order, Forecast for your collaboration.
 - Select Message Type such as Notify, Distribute, Sync, and Request.
3. Click **Go** to execute the search.

To create a collaboration property:

1. Navigate to the Collaboration Property window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Property.
2. Click **Create Collaboration Property** to open the Create Collaboration Property window. See: Creating a Collaboration Property, page 10-19.

To update or delete a collaboration property:

1. Navigate to the Collaboration Property window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Property.
2. Perform a search by entering search criteria in the Collaboration Property window.
3. Click the enabled **Update** for a collaboration property in the search results area. The Update Collaboration Property window appears.

See: Updating a Collaboration Property, page 10-19.

4. Click the enabled **Delete** for a collaboration property in the search results area. You are prompted to confirm the delete action.

Creating a Collaboration Property

Use the Create Collaboration Property window to associate collaboration types and application names with appropriate message types. These message types indicate the different nature of the collaborations. Based on the message types, collaborations displayed in the Collaboration History window can be resent in case they fail.

For example, a Notify message type is associated with Oracle Purchasing with Ship Notice as the collaboration type, then the Reinitiate action should be available in the Collaboration History window for the Purchasing Ship Notice type of collaborations with statuses of Started, Complete Normal, and Completed Error. If it is a Request message type, then the Reinitiate action should be available only when the collaboration status is Started or Completed Error. For more information on reinitiating collaborations, see Viewing the Collaboration History Search Results, page 10-9.

To create a collaboration property:

1. Navigate to the Create Collaboration Property window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Property > the Create Collaboration Property button.

2. Enter the following fields for your Collaboration Property:

- Select a value for the collaboration Application.
- Select Collaboration Type from the drop-down list.
- Select Message Type for your new collaboration property.
- Select Application Linking Function.

The Application Linking Function field is used to link the Collaboration History feature to product specific windows by specifying the function along with the full URL information. This field appears as document number for a collaboration event listed in the Collaboration History window.

3. Click **Apply** to create a new collaboration property.

Updating a Collaboration Property

After performing a search in the Collaboration Property window, you can update an

existing collaboration property by clicking the Update button for the property that you want to update. The Update Collaboration Property window appears.

To update a collaboration property:

1. Navigate to the Update Collaboration Property window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Property > the Update button.
2. Update the property fields for Application, Collaboration Type, Message Type, or Application Linking Function fields.
3. Click **Apply** to update the selected collaboration property.

Using the Collaboration Display Setup

Use the Collaboration Display Setup window to set up additional fields to be displayed in the Collaboration History window for the advanced search and search results screens, as well as in the Collaboration Events window.

For example, a display setup is created for events with Collaboration Standard UCCNet, Oracle Engineering Online application, and Collaboration Type UCCNet Worklist Query, then when you search for collaboration events with the same collaboration standard, type, and application using the Collaboration History window, you should find additional fields if you specified properly in the setup window for these particular events appear in the advanced search and search results screens. If you click the event name link from the search results screen, then you should also see additional fields displayed in the Collaboration Events window if those additional fields are selected properly in the display setup window. Notification receivers will also receive the notification with the event information in line with the display setup. See Updating the Collaboration Display Header and Details, page 10-22 for details.

In addition, you can query existing collaboration display setups, update the header, update the details, view details, and delete existing setups.

To search a collaboration display setup:

1. Navigate to the Collaboration Display Setup window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Display Setup.
2. To narrow the search in the Search region:
 - Select Collaboration Standard such as OAG, Rosettanet, or UCCnet.
 - Select Application such as Oracle Purchasing, Oracle Inventory.

- Select Collaboration Type such as Cancel Order, Forecast for the display setup.
3. Click **Go** to execute the search. The search results are displayed in the tabular format.

To create a display label header:

1. Navigate to the Collaboration Display Setup window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Display Setup.
2. Click **Create Display Labels Header** to open the Create Display Labels Header window. See: Creating a Collaboration Display Label Header, page 10-21.

To update the collaboration display header, update the details, or delete the setup:

1. Navigate to the Collaboration Display Setup window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Display Setup.
2. Perform a search by entering search criteria in the Search region. The search results are displayed in the tabular format.
3. To update the header, click enabled **Update Header** to open the Update Display Labels Header window.

To update the details, click enabled **Update Details** to open the Update Display Labels Details window.

See: Updating the Collaboration Display Label Header and Details, page 10-22.

To delete an existing setup, click enabled **Delete** and confirm the action.

Creating a Collaboration Display Label Header

Use the Create Collaboration Display Labels Header window to create new display header information, including collaboration standard, application, and collaboration type.

To create a collaboration display label header:

1. Navigate to the Create Display Labels Header window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Display Setup > the Create Display Labels Header button.

2. Enter the following information for your new display header:
 - Select Collaboration Standard such as OAG, Rosettanet, or UCCnet.
Select Application such as Oracle Purchasing, Oracle Inventory.
 - Select Collaboration Type such as Cancel Order, Forecast for the display setup.
3. Click **Apply** to confirm the creation.

Updating the Collaboration Display Header and Details

You can update existing collaboration display header and details from the search results in the Collaboration Display Setup window.

The display header contains collaboration standard, type, and application information; the collaboration details contain the internal column names and associated display labels appropriate for calling applications. Each column name can have in the following attributes:

- **Search Enabled:** This checkbox indicates that the associated column name can be added to the Collaboration History advanced search screen to search for the events with the same display header information.

See: Using an Advanced Search, page 10-8.

- **Displayed Enabled for Events Screen:** This checkbox indicates that the associated column name can be displayed in the Collaboration Event window where you can view the read-only information for an event.

See: Using the Collaboration Events Window, page 10-10.

- **Display Enabled for Results Table:** This checkbox indicates that the associated column name can be added to the Collaboration History search results screen for the events retrieved with the same display header information. It also indicates that notification receivers will receive notifications with the event information containing this column.

See: Viewing the Collaboration History Search Results, page 10-9.

Collaboration History allows you to change the following display names appropriate for your application needs:

- Document Number
- Document Owner
- Message Initiator
- Partner Document Number

Additionally, you can add more columns or delete an entry of a column name if you do not want it to be displayed as the collaboration details.

To update the display header:

1. Navigate to the Update Display Labels Header window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Display Setup > the Update Display Labels Header button.
2. Update the Collaboration Standard, Application, and Collaboration Type fields as needed.
3. Click **Apply** to confirm the changes.

To update the display details:

1. Navigate to the Update Display Labels Details window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Collaboration Display Setup > the Update Display Labels Details button.
2. Update the following fields as needed:
 - Select CLN Column from the drop-down list, such as Attribute 1, Date Attribute 1, Document Number.
 - Enter Display Label for the CLN Column name you just entered.
 - Select the following checkboxes for each of your Column name if it is necessary:
 - Search Enabled
 - Display Enabled for Events Screen
 - Display Enabled for Results Table
3. Optionally click **Add Another Row** to add more entries for your display details.
4. Click **Delete** to remove an entry of a record if it is necessary.
5. Click **Apply** to confirm the changes.

Using the Notification Definition Window

The Notification Definition window enables you to search a notification based on source, notification code, and notification description. Also, it enables you to create a new notification definition and update or delete an existing notification definition.

Prerequisites

- Define the application and collaboration type lookups.

To search a notification definition:

1. Navigate to the Notification Definition window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Definition.
2. To narrow the search:
 - Select Source as the software component generating the notification. This list includes values such as Oracle Applications, B2B Server, iProcurement Connector (ITG), XML Gateway.
 - Enter Notification Code as the actual error, warning, or status message passed from the notification source.
 - Enter Notification as the description of the notification. A notification description is specified for display purposes where notifications are referenced elsewhere.
3. Click **Go**. The window displays the results of the search.

To create a notification definition:

1. Navigate to the Notification Definition window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Definition.
2. Click **Create Definition** to open the Create Notification Definition window.
See: Creating a Notification Definition, page 10-25.

To update or delete a notification definition:

1. Navigate to the Notification Definition window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Definition.
2. Search for a notification definition.
3. Click **Update** for a Source in the search results region to open the Update Notification Definition window. See: Updating a Notification Definition, page 10-

4. Click **Delete** for the associated notification definition. You are prompted to confirm the delete action.

Creating a Notification Definition

Use the Create Notification Definition window to create a new notification definition.

Prerequisites

- Define the application and collaboration type lookups.

To create a notification definition:

1. Navigate to the Create Notification Definition window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Definition > the Create Definition button.
2. Enter the values for the following fields:
 - Select Source as the software component generating the notification. This list includes values such as Oracle Applications, B2B Server, iProcurement Connector, XML Gateway.
 - Enter Notification Code as the actual error, warning, or status message passed from the notification source. The Notification Code values are preseeded or you can define them in the Oracle XML Gateway.
 - Enter Notification as the description of the notification. A notification description is specified for display purposes where notifications are referenced elsewhere.
3. Click **Apply** to create a new notification definition.

Updating a Notification Definition

After a search in the Notification Definition window, you can update an existing notification definition retrieved from the search. This update includes modifying its source, notification code, or notification description.

Prerequisites

- Define the application and collaboration type lookups.

To update a notification definition:

1. Navigate to the Update Notification Definition window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Definition > the Update button.
2. Update the values for the Source, Notification Code, or Notification fields.
The Notification field is the actual description of the notification to be displayed.
3. Click **Apply** to commit the changes.

Using the Notification Action Definitions Window

When notifications are raised at predefined points, such as cancel orders or change orders, within a collaboration, notification actions are taken places and routed to appropriate people, such as notify administrators or document owners, based on your action definitions specified in the Notification Action Definitions window.

The notification action definition contains two parts:

- Notification Action Header: The action header defines the combination of an application that owns the collaboration, collaboration type, source, and notification description.
- Notification Action Details: The action details specify the information of how the action will be taken. For example, when a notification is triggered, then the document owner will be notified or relevant business events will be raised.

You can use the Notification Action Definitions window to search, create, update, or delete notification actions, as well as to view and update notification action details.

Prerequisites

- Define the notification code lookup.
- Identify the user to send the notification.

To search a notification action definition:

1. Navigate to the Notification Action Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Action Definition.
2. To narrow the search:

- Select the collaboration Application where a collaboration occurs.
- Select Collaboration Type such as Cancel Order, Change Order Notification, Forecast.
- Select Source as the software component generating the notification. This list includes values such as Oracle Applications, B2B Server, iProcurement Connector, XML Gateway.
- Enter Notification as the description of the notification.

A notification description is specified for display purposes where notifications are referenced elsewhere. You can search a notification from the Search and Select: Notification window, which displays a list of values.

3. Click **Go**. The window displays the results of the search.

To create a notification action header:

1. Navigate to the Notification Action Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Action Definition.
2. Click **Create Definition** to open the Create Notification Action Header window.
See: Creating a Notification Action Header, page 10-28.

To update a action header or action details, view action details, or delete a notification:

1. Navigate to the Notification Action Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Action Definition.
2. Search for a notification.
3. You can perform the following tasks from the search results region:
 - *To update a notification action header*, click the enabled **Update** for a notification to open the Update Notification Action Header window.
See: Updating a Notification Action Header, page 10-29.
 - *To create and update notification action details*, click the enabled **Update Action Details** for a notification to open the Notification Action Definition window.
See: Creating and Updating Notification Action Details, page 10-29.
 - *To view notification action details*, click the enabled notification name link in the

search results region. The Notification Action Definition window appears.

See: Viewing Notification Action Details, page 10-31.

- *To delete a notification*, click **Delete** for an associated notification. You are prompted to confirm the delete action.

Creating a Notification Action Header

Use the Create Notification Action Header window to create a notification action header. The notification action header represents the transaction details for which notifications are defined in the notification action details section.

Prerequisites

- Define the notification code lookup.
- Identify the user to send the notification.

To create a notification action definition header:

1. Navigate to the Create Notification Action Header window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Action Definition > the Create Definition button.
2. Enter the values for the following fields:
 - Select the collaboration Application from the drop-down list.
 - Select Collaboration Type as one of the many collaboration types used such as Cancel Order, Change Order Notification.
 - Select Source as the software component generating the notification. This list includes values such as Oracle Applications, B2B Server, iProcurement Connector, XML Gateway.
 - Enter Notification as the description of the notification. A notification description is specified for display purposes where notifications are referenced elsewhere. You can search for a notification from the Search and Select: Notification window, which displays a list of values.
3. Click **Apply** to create a new notification action definition header.

Updating a Notification Action Header

After a search from the Notification Action Definitions window, you can update an existing notification action header from the search results by modifying the Application, Collaboration Type, Source, and Notification fields.

Prerequisites

- Define the notification code lookup.
- Identify the user to send the notification.

To update a notification action definition header:

1. Navigate to the Notification Action Definitions window from the Workflow Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Action Definition.
2. Click the enabled **Update** for a notification in the search results region to open to the Update Notification Action Header window.
3. Select the values for the following fields:
 - Select the collaboration Application as Oracle Purchasing, Oracle Inventory, Oracle Order Management.
 - Select Collaboration Type as one of the many collaboration types used. This list includes values such as Cancel Order.
 - Select Source as the software component generating the notification such as Oracle Applications, B2B Server, iProcurement Connector, XML Gateway.
 - Enter Notification as the description of the notification. A notification description is specified for display purposes where notifications are referenced elsewhere.
4. Click **Apply** to update the notification action definition header.

Creating and Updating Notification Action Details

Once a notification action has been created, you can search for the notification action that needs to be modified and then update the action details by clicking enabled **Update Action Details** from the search results region. This opens the Notification Action Definition window where you can perform the following tasks:

- **View a list of existing notification action information** including action header and action details

The "Notify Administrator" notification action is the default action for each notification.

- **Create new actions for a given notification**

You can select one of the following values from the Create Action drop-down list for your action:

- Call API
- Notify Document Owner
- Notify Trading Partner
- Notify Administrator
- Raise Business Event
- Start Workflow

You need to select Active checkbox to enable this new notify action. See: Setting Up Customized Collaborations, page 11-21 for technical requirements and information of each type of action.

- **Update existing actions**

You can activate or deactivate a notification action and update action details based on your selected action name.

For example, if the action is "Notify Administrator", then the Update Notify Administrator Action Definition window appears. In addition to activate or deactivate the action, you can also update the Notification User/Roles with valid users or roles in Workflow Directory. See *Oracle Workflow Administrator's Guide* for details.

- **Delete existing actions**

Prerequisites

- Define the notification code lookup.
- Identify the user to send the notification.

To create or update notification action details:

1. Navigate to the Notification Action Definitions window from the Workflow

Administrator Web Applications responsibility: Collaboration Monitor: Setup > Notification Action Definition.

2. Search for a notification action definition.
3. Click the enabled **Update Action Details** from the search results region to open the Notification Action Definition window.
4. *To create a new action*, select an action name from the Create Action drop-down list and click **Go**.

In addition to select the Active checkbox to enable the new notify action, enter appropriate information based on the selected action name:

- **Call API:**
Enter event name as the Procedure Name of the API. Select Online or Concurrent as the Mode of Execution field.
 - **Raise Business Event:**
Enter the event name as the fully qualified Name. Enter parameter 1 to parameter 14.
 - **Start Workflow:**
Enter appropriate information for the Workflow Item Type field and Workflow Process.
 - **Notify Administrator:**
Enter Notification User/Roles up to 15 roles. The user/role must be a valid user or role in Workflow Directory.
 - **Notify Trading Partner or Document Owner:**
Need to specify the Active checkbox to enable the notification.
5. *To update the action*, click the enabled **Update** for an action listed in tabular format. The associated update action definition window appears with various fields based on your selected action name. For example, if the action "Start Workflow" needs to be updated, then you may update the Workflow Item Type and Workflow Process fields.
 6. *To delete the action*, click the enabled **Delete** for an action. You are prompted to confirm the deletion.

Viewing Notification Action Definition Details

After a search in the Notification Action Definitions window, you can click the enabled

Notification Name link in the search results region to view the notification action information. This includes the notification header containing application, collaboration type, source, and notification description, as well as the details of actions for the notification. The action details include the action name such as "Notify Document Owner", active flag, parameter 1, parameter 2, parameter 3, parameter 4, and parameter 5.

Click the enabled **Action** to view the corresponding action details.

See: Viewing Action Definition Details, page 10-32.

Viewing Action Definition Details

After clicking a specific notification Action Name link from the Notification Action Details page, you can view the associated action name details. For example, if the enabled Action is Notify Administrator, then click to view the Notify Administrator Action Definition details, including action name, active status and notification role information.

Setting Up Collaborations

This chapter covers the following topics:

- Setting Up New Collaborations
- Setting Up Notification Processing for New Collaborations
- Setting Up Customized Collaborations

Setting Up New Collaborations

This section describes the setup tasks for collaboration tracking using the Collaboration History feature. It is intended for Oracle E-Business Suite application product teams, or other skilled implementers interested in implementing history tracking for a new collaboration. Using this section, you can design the tracking mechanism of a collaboration and then implement it.

To implement collaboration tracking:

1. Designing the Collaboration, page 11-2
2. Creating Message Maps, page 11-3
3. Setting Up FND Lookup Codes and Messages, page 11-3
4. Setting Up Collaboration Events and Collaboration Final Events, page 11-5
5. Adding the Workflow Events, page 11-5
6. Implementing Notification Processing for New Collaborations, page 11-17
7. Setting Up Trading Partners and Confirmation Messages, page 11-18
8. Defining Profile Options, page 11-18

9. Troubleshooting and Debugging Collaboration History, page 11-18

Use the Collaboration Setup Worksheet provided in Appendix H to design the collaboration.

Designing the Collaboration

When designing the collaboration, determine the information needed by the end user to evaluate the status of collaboration. When defining a collaboration tracking, consider the existing workflows, business events, and messages generated and used. Determine the messages to exchange and the events that trigger them. Also, determine whether there is information, such as errors or exceptions in the event, which is recorded to benefit the end user.

Review the Collaboration Detail window to identify appropriate data to capture.

Give a name to the collaboration and identify Oracle E-Business Suite application that owns it. This information displays in the Collaboration Type and Application fields in Collaboration History. Determine this information and enter it in the Collaboration Design section of the Collaboration Setup Worksheet in Appendix H.

An Exercise on Designing the Collaboration

A useful exercise in this evaluation is to draw a picture of the message flows between the application and the trading partner or other target system. Identify the payloads, the events that occur in the application before or after the payload is generated, and the points at which exceptions or errors are returned. Determine the XML message payloads to generate and use in the collaboration. This is determined during the high level design phase of a project.

In the Message Payloads section of the Collaboration Setup Worksheet, record the name of the payload that the collaboration generates and uses, including any inbound confirmation business object document whose receipt or contents are recorded in Collaboration History. Record whether the message is inbound or outbound from Oracle E-Business Suite. Record those that have already been mapped in Oracle XML Gateway and those that have not.

The next activity is to determine the events in the collaboration. Define all the events that are recorded for reference by the end user. These can reflect the business process activities as well as the XML message payload generation and usage. Following are a few examples:

- Purchase Order is Approved.
- Customer Order is Booked.
- PROCESS_PO OAG message is generated by Oracle XML Gateway.
- PROCESS_PO OAG message is used by Oracle XML Gateway.

- ACKNOWLEDGE_PO message is used by Oracle XML Gateway.
- The acknowledgment is recorded in Oracle Purchasing.
- The business-to-business gateway sends the message to the trading partner.

An event can occur in Oracle E-Business Suite applications, Oracle XML Gateway, or in another system such as a B2B gateway. If one occurs in another system, then that system sends an OAG CONFIRM_004 BOD with the correct information.

Record the events in the sequence they occur and give each one a number and an event message, use the Events section of the Collaboration Setup Worksheet. A default message indicating the creation of a collaboration displays in the Event Message column for the first event and cannot be changed. In the FND Message ID column, the message identification number is recorded.

In the Event Location (Point) column, the options are Oracle Applications, Oracle XML Gateway, and the B2B gateway. If the event occurs in a workflow, then note it in the Workflow column. In the Message Payloads column, note the payload number identified earlier. If an event occurs prior to the creation of a payload, but is related because it is either the triggering event or a subsequent event that occurs before the message is generated, then refer to it in this event definition.

For example, Purchase Order Approval occurs prior to the creation of the PROCESS_PO OAG message in the XML Gateway. Process_PO is referred to in the Purchase Order Approval event definition.

See Adding the Workflow Events, page 11-5 for information on the last column, CLN Event.

Creating Message Maps

At the end of this step, all the required XML Gateway Message Maps are created and the transaction type and subtype for all messages used are identified.

For each message, whether newly mapped in Oracle XML Gateway or existing, record the internal transaction type and subtype in the appropriate columns in the Oracle XML Gateway column of the Collaboration Setup Worksheet.

See Setting Up Collaboration Events and Collaboration Final Events, page 11-5 for more information.

Setting Up FND Lookup Codes and Messages

After this step, the setup of all the FND Lookup codes and messages to enable your collaboration are completed.

Adding the FND Lookup Codes

To enable Collaboration Tracking, add entries in the FND Lookup tables. The FND

Lookup Types that require additions include:

- Application ID
- Collaboration Type
- Document Type

For each FND Lookup Type, the FND Lookup Type name is presented along with the information gathered in the previous steps. Use this information to create the new Lookup Code. Also, for each Lookup Type, examples that currently exist are listed.

Application ID

The name of this FND Lookup Type is CLN_APPLICATION_ID. This information is identified as the owner of the collaboration. Enter the identified application only if it does not already exist. It is used as the list of values for the column APPLICATION_ID in the A table.

Examples are Oracle Order Management and Oracle Purchasing.

Collaboration Type

The name of this FND Lookup Type is CLN_COLLABORATION_TYPE. This information is identified as Collaboration Name. If one exists with the same name, then it can be reused. It is used as the list of values for the column COLLABORATION_TYPE in the Collaboration History Header table.

Examples are Order for RosettaNet PIP3A4, Change Order for RosettaNet PIP3A7, Cancel Order for RosettaNet PIP3A8, Change Order Notification for RosettaNet PIP3A6, Planning, Forecast, Change Planning, and Change Forecast.

Collaboration Document Type

The name of this FND Lookup Type is CLN_COLLABORATION_DOC_TYPE. This is identified as Message Payloads. You can reuse a FND Lookup type name. This is the list of values for the column MESSAGE_TYPE in the Collaboration History Detail table.

Examples are Process PO, Acknowledge PO, Confirm BOD, and Change PO.

Adding the FND Messages

For all the event messages, create FND messages and record the number in the FND Message ID column in the Events section of the Collaboration Setup Worksheet. This information is used when raising the events.

Refer to Adding the Workflow Events, page 11-5 for information.

Listing messages in the FND ensures their translation. These messages display in the Message column of the Collaboration History Details window.

Setting Up Collaboration Events and Collaboration Final Events

When this step is complete, Collaboration History recognizes all the events and final events defined.

Setting Up Collaboration Events

The activities in this step are key to raising the events. All collaboration events associated with the XML payload are set up using the Collaboration Event Definition window and existing events can be queried and new ones created.

See: Using the Collaboration Event Definitions Window, page 10-12.

Setting Up Collaboration Final Events

The collaboration final event is set up in Collaboration History so that it can record a final status of the overall collaboration. The final event is generally the occurrence of an inbound or outbound message in Oracle XML Gateway.

From the identified events and sequences identified, use the Collaboration Final Event Definition windows to record the final event.

See: Using the Collaboration Final Event Definitions Window, page 10-15.

Adding the Workflow Events

In this step, place the Collaboration History events in the locations that correspond to the defined events and event sequence. These events must be raised in the existing business workflows and the ECX workflows defined when the Oracle XML Gateway maps are built.

Following are the workflow events provided by Collaboration History:

- oracle.apps.cln.ch.collaboration.create
- oracle.apps.cln.ch.collaboration.update
- oracle.apps.cln.ch.collaboration.addmessage

In addition to the events recorded during the raising of the create, update, and add message events, Collaboration History also subscribes to all Oracle XML Gateway error events.

Raising Events

This section reviews where to place the events. It provides the instructions and guidance as to where they are raised. The individual collaborations have special requirements that do not fit the guidelines, but are to be satisfied.

Initial Event

The initial event can occur in any of the following situations:

- When an inbound XML message arrives at the inbound Advanced Queue (AQ) and is processed by Oracle XML Gateway.
- During an existing business workflow where there is an event activity, which signals the beginning of the collaboration.
- When an outbound XML message is generated by Oracle XML Gateway and sent to the trading partner or third party application.

An XML message's arrival on the inbound AQ initiates an event. When this occurs, since Collaboration History subscribes to Oracle XML Gateway's inbound message receive event, it is automatically recorded. In this case, the implementer does not have task.

If it is determined that the initiating event is in an existing business workflow, then raise the Collaboration History create event, `oracle.apps.cln.ch.collaboration.create`, at the appropriate location in the workflow. Determining the appropriate location requires knowledge of the business workflow and what the business end user is expecting to see when searching Collaboration History for their collaboration.

In the above example, the CLN Create Collaboration event is raised in the PO Approval Workflow prior to the raising of the Raise XML PO event. The PO Approval Workflow is a business workflow delivered with Oracle Purchasing.

An initiating event can occur when an outbound XML message is generated by Oracle XML Gateway. When the map between the Oracle tables and the XML message is built, an ECX Receive Workflow Process is defined. This process contains both generate and send activities. The Collaboration History Create event, `oracle.apps.cln.ch.collaboration.create`, is raised before the send activity.

Subsequent Events

All events after the initial event are called subsequent events. To record these subsequent events, a Collaboration History Update event, `oracle.apps.cln.ch.collaboration.update`, is raised.

As with the initial event, the update event can be raised in a business workflow or in the ECX Receive Workflow Process. Follow the instructions given in the initial event section, replacing the Create event for the Update.

Adding a Message Event

The last Collaboration History event, `oracle.apps.cln.ch.collaboration.addmessage`, records the additional information about the event that the end user may find useful. The exception messages or line level errors received from the trading partner are examples of the type of information that this event can be used to record. In both cases, there is no place to store the information that is easily accessible to a end user.

The add message events enable the recording of up to five values and there can be as many rows of information that are useful. In the values, store the information used to

easily focus the source of the error or exception. For a purchase order, it can be a line number and for a forecast, it can be an item number, at a location, for a date, or period of time.

Use of this event is implemented by raising it at the line level of the post process for the inbound XML message's map. When used, the Collaboration History Update event is raised at the header level. This is typically used for an acknowledgement type XML messages from a trading partner.

The information captured when this event is raised displays on the Collaboration Event Details window.

Working with the Collaboration History Events

This section describes how to raise the Collaboration History events. It discusses the parameters required to make your collaboration more useful.

The events supplied for Collaboration History can be viewed in the Workflow Builder by loading the Collaboration History Standard Events item type. Using the drag-and-drop function, Collaboration History activities can be inserted where appropriate.

Note that some implementers or product teams have their own naming conventions for events. If this is the case, then Collaboration History can be used.

First, define your own create, update, and add message events. Then, include all the parameters needed by the Collaboration History events.

Then, in their subscriptions, call the appropriate Collaboration History Application Programming Interface (API):

Create - CLN_CH_EVENT_SUBSCRIPTION_PKG.CREATE_EVENT_SUB

Update - CLN_CH_EVENT_SUBSCRIPTION_PKG.UPDATE_EVENT_SUB

Add message - CLN_CH_EVENT_SUBSCRIPTION_PKG.ADDMESSAGE_EVENT_SUB

Working with Key Parameters

Regardless of the event raised, one of the following sets of parameters is published to identify the event and linked to other events for the collaboration:

- XML_INTERNAL_CONTROL_NUMBER

Or

- XMLG_MESSAGE_ID

Or, one or more of the following parameters:

- XML_INTERNAL_TXN_TYPE
- XMLG_INTERNAL_TRANSACTION_SUBTYPE

- DOCUMENT_DIRECTION
- XMLG_DOCUMENT_ID
- TRADING_PARTNER_TYPE
- TRADING_PARTNER_ID
- TRADING_PARTNER_SITE

Or

- APPLICATION_ID

And one or more of the following:

- UNIQUEID1-5

When an event is raised, where there is an associated Oracle XML Gateway message, the XML Gateway Message ID is known. Additionally, for inbound messages, Internal Control Number is known.

The group of parameters is used if it is prior to an XML message being generated by Oracle XML Gateway. In most cases, these parameters are used if it is the initiating event from a business workflow.

The last group of parameters are APPLICATION_ID and UNIQUEID1-5. The combination of values used in each of the UNIQUE_ID parameters uniquely identify a collaboration. This is the case if the collaboration has no XML message payloads associated or they occur later in the sequence of events. If UNIQUEID1-5 parameters are chosen to identify a collaboration, then each time an event is raised in the collaboration flow, the same values in the same parameters are published. If an XML message payload is involved, then these same values are published with Internal Control Number or XML Message_ID of the XML message payloads.

In these cases, for Create Collaboration Event pass the Collaboration Type, Document Type, and Document Direction parameters along with Application ID and Unique ID1-5.

For Update Collaboration Event, pass the Document Type and Document Direction parameters.

Event: oracle.apps.cln.ch.collaboration.create

This event creates a new collaboration and records the initial event of the collaboration.

In addition to the key parameters, other required parameters are Document Number, Document Release Number, Document Revision Number, and Organization ID.

Document Number is the business document number associated with the collaboration. The business document number is typically the number that the business user recognizes, such as the purchase or customer order, or the forecast number. This value

is used to query Collaboration History. It is not an internal key.

Document Release Number is required if the business document is a purchase order release.

Document Revision Number is required if the purchase order or other business document has a revision number associated.

Organization ID is always required because business document numbering in Oracle typically does not cross organizations. Organization ID helps to identify the exact business document.

The remaining values are all optional. However, when determining whether to publish these parameters, evaluate whether using them makes searching Collaboration History easier for the end user. These are:

- Partner Document Number - The business document number from the trading partner. For a purchase order, it can be the customer order number. This displays on the Collaboration Detail window.
- Document Creation Date - This is the date that the business document is originally created. This displays on the Collaboration Detail window and used to search the history.
- Document Revision Date - This is the date that the business document was revisioned. This displays on the Collaboration Detail window and used to search the history.
- Document Owner - This is the owner of the document. This is the buyer, planner, or order taker. It contains the internal ID of the employees record.

Event Parameters

This table explains the key parameters:

Parameter	Internal Name	Description
Internal Control Number	XMLG_INTERNAL_CONTR OL_NUMBER	XML Gateway generated unique number for inbound messages (INTERNAL_CONTROL_NU MBER).
XML Gateway Message ID	XMLG_MESSAGE_ID	XML Gateway message ID, which is unique across messages.

Parameter	Internal Name	Description
XML Gateway Internal Transaction Type	XMLG_INTERNAL_TXN_TY PE	XML Gateway internal transaction type for the message to be created.
XML Gateway Internal Transaction SubType	XMLG_ INTERNAL_TXN_SUBTYPE	XML Gateway internal transaction subtype for the message to be created.
Document Direction	DOCUMENT_DIRECTION	The direction the document is going (In or Out of the application) for the message to be created; probably - Out.
XML Gateway Document ID	XMLG_DOCUMENT_ID	Internal key of the primary business document such as the PO_header, SO_header internal key.
Trading Partner Type	TRADING_PARTNER_TYPE	Trading partner type, as defined in the XML Gateway.
Trading Partner ID	TRADING_PARTNER_ID	Trading partner ID, as defined in the XML Gateway.
Trading Partner Site	TRADING_PARTNER_SITE	Trading partner site for the trading partner, as defined in the XML Gateway.
UNIQUEID1-5	UNIQUE_ID1 to UNIQUE_ID5	Five fields that can contain information that uniquely identify the collaboration.
Document Number	DOCUMENT_NO	Business document number that the end user is familiar with.
Organization ID	ORG_ID	Oracle Organization ID for the organization that the business document is in.

Parameter	Internal Name	Description
Document Release Number	RELEASE_NO	If the business document is a Blanket Purchase Order Release, then this contains the purchase order release number.
Document Revision Number	DOCUMENT_REVISION_NO	If this is a document revision, then this contains the document revision number.
Partner Document Number	PARTNER_DOCUMENT_NO	The trading partner document number; for an inbound order, it contains the purchase order number.
Document Creation Date	DOCUMENT_CREATION_DATE	Date on which the Primary Business document was created; is not the date when the collaboration event was created.
Document Revision Date	DOCUMENT_REVISION_DATE	Date on which the Primary Business document was Revisioned; is not the date on which the collaboration event was created.
Doc Owner	DOCUMENT_OWNER	Internal ID of the employee that owns the document; refers to a buyer, planner, or order taker.
Application ID	APPLICATION_ID	Application ID
Collaboration Type	COLLABORATION_TYPE	Collaboration Type. It is based on the lookup type CLN_COLLABORATION_TYPE.
Document Type	DOCUMENT_TYPE	Document Type. It is based on the lookup type CLN_COLLABORATION_DOCUMENT_TYPE.

Parameter	Internal Name	Description
Initiation Date	COLLABORATION_INITIATION_DATE	Collaboration Initiation Date
Reference ID	REFERENCE_ID	Reference ID, which uniquely identifies the collaboration.

Event: oracle.apps.cln.ch.collaboration.update

This workflow event is raised to add new collaboration events to an existing collaboration.

Besides the key parameters, other required parameters are Document Number, Document Release Number, Document Revision Number, Organization ID, Doc_Status, and MSG_Text.

Document Number, Document Release Number, Document Revision Number, and Organization ID have the same description as in the Create event.

Doc_Status is the status of this event or document being recorded. Its value is either Success or Error. MSG_Text is the FND message ID column.

The remaining values are all optional. However, when determining whether to use these parameters, evaluate whether using them makes searching Collaboration History easier for the end user. These are:

- Disposition - This is the disposition of the collaboration. It can be accepted or rejected by the trading partner. The point at which this is discovered varies, but is often when an XML message is returned from the trading partner.
- Partner Document Number - The business document number from the trading partner. For a PO, it is the customer order number. This displays on the Collaboration Detail window.
- Document Creation Date - This is the date the business document was originally created. It displays on the Collaboration Detail window and can be used to search the history.
- Document Revision Date - This is the date the business document was revisioned. It displays on the Collaboration Detail window and can be used to search the history.
- Document Owner - This is the owner of the document. It is the buyer, planner, or order taker. It contains the internal ID of the employees record.

Event Parameters

This table explains the key parameters:

Parameter	Internal Name	Description
Internal Control Number	XMLG_INTERNAL_CONTR OL_NUMBER	XML Gateway generated unique number for inbound messages (INTERNAL_CONTROL_NU MBER).
XML Gateway Message ID	XMLG_MESSAGE_ID	XML Gateway message ID, which is unique across messages.
XML Gateway Internal Transaction Type	XMLG_INTERNAL_TXN_TY PE	XML Gateway internal transaction type for the message to be created.
XML Gateway Internal Transaction SubType	XMLG_ INTERNAL_TXN_SUBTYPE	XML Gateway internal transaction subtype for the message to be created.
Document Direction	DOCUMENT_DIRECTION	The direction the document is going (In or Out of the application) for the message to be created; probably - Out.
XML Gateway Document ID	XMLG_DOCUMENT_ID	Internal key of the primary business document such as the PO_Header, SO_Header internal key.
Trading Partner Type	TRADING_PARTNER_TYPE	Trading partner type, as defined in the XML Gateway.
Trading Partner ID	TRADING_PARTNER_ID	Trading partner ID, as defined in the XML Gateway.
Trading Partner Site	TRADING_PARTNER_SITE	Trading partner site, as defined in the XML Gateway.
UNIQUEID1-5	UNIQUE_ID1 to UNIQUE_ID5	Information that uniquely identify the collaboration.

Parameter	Internal Name	Description
Document Number	DOCUMENT_NO	Business document number that the end user is familiar with.
Organization ID	ORG_ID	Oracle Organization ID for the organization that the business document is in.
Document Release Number	RELEASE_NO	If the business document is a Blanket Purchase Order Release, then this contains the purchase order release.
Document Revision Number	DOCUMENT_REVISION_NO	If this is a document revision, then this contains the document revision number.
Partner Document Number	PARTNER_DOCUMENT_NO	Trading partner document number; for an inbound order, it contains the PO number.
msg_text	MESSAGE_TEXT	This will contain the FND Message ID; alternatively, a message literal can be entered, but not recommended.
Org_ref	ORIGINATOR_REFERENCE	Reference number used to uniquely identify the message; for example, the XML gateway unique message ID or a unique ID from the business-to-business gateway in use; the primary purpose is troubleshooting.
Doc_status	DOCUMENT_STATUS	Status of the event; is either SUCCESS or ERROR.

Parameter	Internal Name	Description
Disposition	DISPOSITION	Disposition of the collaboration; indicates if the collaboration is accepted or rejected by the trading partner or target system; set on the event which occurs when response is received that indicates whether it is accepted (mention why it is accepted, but the next line is rejected).
Document Creation Date	DOCUMENT_CREATION_DATE	Date on which the Primary Business document was created; not the date that the collaboration event was created.
Document Revision Date	DOCUMENT_REVISION_DATE	Date on which the Primary Business document was Revisioned; not the date that the collaboration event was created.
Doc Owner	DOCUMENT_OWNER	Internal ID of the employee that owns the document; refers to a buyer, planner, or order taker.
Application ID	APPLICATION_ID	Application ID
Document Type	DOCUMENT_TYPE	Document Type. It is based on the lookup type CLN_COLLABORATION_DOC_TYPE.
Reference ID	REFERENCE_ID	Reference ID, which uniquely identifies the collaboration.

Event: oracle.apps.cln.ch.collaboration.addmessage

This workflow event is raised to add collaboration event details for an existing collaboration event. This information can be text messages or other data useful to the business user in understanding the state of the collaboration. The rejection messages or

information on changes data can be placed here.

In addition to the key parameters, Collaboration Detail ID can also be used.

The reference IDs can be used at the discretion of the implementers to improve the understanding of an end user. They can be line numbers, item numbers, and organization IDs.

Event Parameters

This table explains the key parameters:

Parameter	Internal Name	Description
Collaboration Detail ID	COLLABORATION_DETAIL_ID	Collaboration Detail ID
Internal Control Number	XMLG_INTERNAL_CONTR OL_NUMBER	XML Gateway generated unique number for inbound messages (INTERNAL_CONTROL_NU MBER).
XML Gateway Message ID	XMLG_MESSAGE_ID	XML Gateway message ID, which is unique across messages.
XML Gateway Internal Transaction Type	XMLG_INTERNAL_TXN_TY PE	XML Gateway internal transaction type for the message to be created.
XML Gateway Internal Transaction SubType	XMLG_ INTERNAL_TXN_SUBTYPE	XML Gateway internal transaction subtype for the message to be created.
Document Direction	DOCUMENT_DIRECTION	The direction the document is going (In or Out of the application) for the message to be created; probably - Out.
XML Gateway Document ID	XMLG_DOCUMENT_ID	Internal key of the primary business document such as the PO_header, SO_header internal key.
Trading Partner Type	TRADING_PARTNER_TYPE	Trading partner type, as defined in the XML Gateway.

Parameter	Internal Name	Description
Trading Partner ID	TRADING_PARTNER_ID	Trading partner ID, as defined in the XML Gateway.
Trading Partner Site	TRADING_PARTNER_SITE	Trading partner site for the trading partner as defined in the XML Gateway.
UNIQUEID1-5	UNIQUE_ID1 to UNIQUE_ID5	Information that uniquely identify the collaboration.
REFERENCE_ID 1-5	REFERENCE_ID1 to REFERENCE_ID5	Information to identify the message with a location on the business document, such as a line for a purchase order, or the item or location or date for a plan.
Detail Message	DETAIL_MESSAGE	FND Message ID or alternatively a message literal; in most cases, it is the returned message from the trading partner.
XML Gateway Message ID	XMLG_MESSAGE_ID	XML Gateway Message ID, which is unique across messages.
Application ID	APPLICATION_ID	Application ID
Reference ID	REFERENCE_ID	Reference ID, which uniquely identifies the collaboration.
UniqueId1-5	UNIQUE_ID1 to UNIQUE_ID5	Fields that can contain information that uniquely identifies the collaboration.

Implementing Notification Processing for New Collaborations

Refer to Setting Up Notification Processing for New Collaborations, page 11-20 to set up notifications and notification actions details.

Setting Up Trading Partners and Confirmation Messages

The collaborations are recorded only when a trading partner is set up to take part in the collaboration. Also, the OAG CONFIRM_BOD_004 inbound and outbound confirmation messages need to be set up in the Trading Partner Setup window.

Refer to Trading Partner Setup, page 3-19 for details.

Setting Profile Options

To have Collaboration History work properly, the profile option CLN_ADMINISTRATOR profile option (profile category Deployment) needs to be set correctly by specifying an appropriate user role or username in the profile value in order to receive notification message when unexpected errors occur.

Notify Administrator is the default notification action for a notification.

Troubleshooting and Debugging Collaboration History

This section contains hints for debugging a collaboration. First, set up the system to record the debugging messages. The first two sections describe how to set up the profile option associated with debugging and how to read the debug file. Following are the hints on how to debug the workflow and Oracle E-Business Suite.

CLN: Debug Level (Debug Profile Category)

This profile option informs the system at what level of detail to capture the log messages in the debug log. Valid values are:

- Statement - Low level logging message giving maximum details.
- Procedure - Logging message called upon entry and exit from a routine. An example is entering routine fdlllov().
- Event - High level logging message. An example of this is a user initiated Abort, beginning establishment of application security session.
- Exception - Internal routine returns a failure code or exception. However, the error does not indicate a problem at the user level.

Examples: Profile ABC not found; Networking routine XYZ could not connect; Retrying; File not found (in a low-level file routine); Database error (in a low-level database routine like afupi).

- Error - An error message to the user.

Examples: Entered a duplicate value for field XYZ. Invalid application username or password at the signon window. Function not available.

- Unexpected - An unexpected situation occurred which indicates or causes instabilities in the runtime behavior. System Administrator will take appropriate action on it.

Examples: Out of memory, Required file not found, Data integrity error, Network integrity error, Internal error, Fatal database error.

Profile Level: Site Level

Default Value: Unexpected

CLN: Debug Log Directory (Debug Profile Category)

This profile option informs the system where to place the log file. Values are a valid directory.

Profile Level: Site Level

Default Value: <None>

Reading the Debug File

- The CLN debug files are named as:
cln-<Date>-<running sequence number>.dbg
Example: cln-15-apr-2002-000015.dbg
- Each line and debug message's format:
<Date><Time>: <Message>
- An API entry can be:
ENTERING <package name>.<procedure/function name>.
Immediately after this, Name-Value pairs of all the parameters display.
- The success and exception messages are found in the log file.
- If the debug level is set to Statement, then extreme care is taken to ensure all of the information is logged.
- The message sequence indicates the flow.
- The exit of an API may appear as:
EXITING <package name>.<procedure/function name>
- Most of the messages printed in the log are low level messages and may contain technical information.

Example:

```
25-mar-2003 01:51:50:2: ENTERING CLN_XMLG_EVENT_SUB_F
25-mar-2003 01:51:50:1: Message ID:B9AEEABDB3F68F0AE0301490C3C43581
25-mar-2003 01:51:50:1: Transaction Type:PO
25-mar-2003 01:51:50:1: Transaction Subtype:CANCEL
25-mar-2003 01:51:50:1: Direction:IN
25-mar-2003 01:51:50:2: EXITING CLN_XMLG_EVENT_SUB_F
```

Debugging Workflow

The Oracle Workflow debug files write into the workflow log tables. The Workflow provides user-friendly windows through which log details are viewed. For example, the events raised and parameters passed to them can be found in the Agent Activity in the Workflow Manager window of Oracle Applications Manager. Similarly, the workflow process log details can be found in the Status Monitor in the Workflow Administrator responsibility.

Refer to the *Oracle Workflow User's Guide* for more details.

Debugging Oracle E-Business Suite

Each Oracle E-Business Suite application has its own profile option to set the debug level.

Refer to the respective user's guides for more information.

Setting Up Notification Processing for New Collaborations

After determining the messages you plan to receive in the new collaboration, define any new notifications and then define the notification actions to perform when these notifications are raised for the new collaboration.

For detailed notification processing setup steps, see:

- Using the Notification Definition Window, page 10-23
- Creating a New Notification, page 10-25
- Creating and Updating Notification Action Details, page 10-29
- Using the Notification Action Definitions Window, page 10-26

You do not have to define new notifications for the B2B gateway and Oracle XML Gateway, as the existing notifications for these notification sources are common to all collaborations. However, you will need additional notifications for the applications involved.

You can use the preconfigured notifications for existing collaborations as examples for creating new notifications.

Refer to Setting Up Customized Collaborations, page 11-21 for a full list of preconfigured notifications.

Setting Up Customized Collaborations

This section explains the types of notification actions triggered upon receiving a notification message. Notification Processing enables significant customization using standard Oracle frameworks such as Workflow, PL/SQL, and Business Events. You can make changes to reflect any business rule. However, it is recommended that a skilled implementer who is familiar with these technologies be responsible for the changes.

Notify Administrator

Notify Administrator is the default notification action supplied with all preconfigured notifications.

You can specify up to 15 User/Roles in the Notification Role 1 - 15 fields. Notification is sent to all Users/Roles. If no User/Role is specified, then the administrator specified in the profile option: CLN_ADMINISTRATOR receives the notification.

E-Mail Format

```
E-Mail Subject: Notification from ...
E-Mail Content:
Details of the Notification:
Notification : < Notification >Notification Desc : < Notification Desc
>
Message : < Notification Message >
Details of the Business Document:
Collaboration Type : < Collaboration Type >
Application : < Application >
Organization ID : < Organization ID >
Document Number : < Document Number >
Revision Number : < Revision Number >
Release Number : < Release Number >

Please check the following collaboration in collaboration history for
further details:
Collaboration ID : < Collaboration ID >
```

Notify Trading Partner

The Notify Trading Partner notification action sends an e-mail to the specified trading partner.

E-Mail Format

E-Mail Subject: Notification from Trading Partner
E-Mail Content:
Notification occurred for the following collaboration at your trading partner site
Details of the Notification:
Notification : < Notification >
Notification Desc : < Notification Desc >
Message : < Notification Message >
Details of the Business Document:
Collaboration Type : < Collaboration Type >
Application : < Application >
Organization ID : < Organization ID >
Document Number : < Document Number >
Revision Number : < Revision Number >
Release Number : < Release Number >

Raise Business Event

The Raise Business Event notification action enables you to trigger an event to which other processes can subscribe using Oracle Business Event System.

Event name: Specify the fully qualified event name.

Note that the user interface (UI) does not validate the availability of the event. Register the specified event using the standard self-service window for registering event, which is a part of Oracle E-Business Suite.

Parameters: Up to 14 parameters - Constant values - to be passed to the business event; can be specified in PARAMETER1 to PARAMETER14.

This event receives a parameter list (wf_parameter_list_t) that contains:

- Standard parameters - Details about the notification, such as notification code, description, and collaboration ID.

All the user specified parameters are appended to the parameter list, with PARAMETER1 to PARAMETER14 named as ATTRIBUTE1 to ATTRIBUTE14, respectively.

Example

If IP_08 (Message Correlation Error) results in the B2B gateway, then update the status of the transaction accordingly. Additionally, there is a business requirement to update information within one of the applications involved.

To implement this, define a business event. A function that is subscribed to this business event can update the transaction. This event can have any number of subscriptions to trigger further action within one or more applications. Using the Notification Action Definition window, define an action of type Raise Business Event, with the event name specified. Any notification arriving with the notification code for which this action is defined (here IP_08), has the business event raised by notification processor with all the parameter values defined, which invokes all the subscriptions

and their associated processing.

Note that the business events are generated if information is sent to all the subscribers interested in capturing the details and performing actions accordingly.

Start Workflow

The Start Workflow notification action initiate a predefined workflow using Oracle Workflow System.

Workflow Item Type: Specify the Workflow Item Type.

Workflow Process: Specify the Workflow Process name.

Define the workflow before defining the notification action, as the user interface does not validate the availability of the workflow.

Workflow will have the following attributes defined:

- APPLICATION_ID
- COLLABORATION_ID
- COLLABORATION_TYPE
- REFERENCE_ID
- TRADING_PARTNER_ID
- HEADER_DESCRIPTION
- NOTIFICATION_DESCRIPTION
- NOTIFICATION_CODE
- STATUS

These attributes specify the notification details and are set while starting this workflow.

Example

According to the business rules, hold a transaction when an IP_07 (TPA Identification error) results in Oracle XML Gateway.

Define a workflow that applies the hold. The internal name of the workflow item type is TXN_HOLD and the internal name of the process is APPLY_HOLD. Then, using the Notification Action Definition window, define an action of type Start Workflow with the workflow item type as TXN_HOLD, and the workflow process as APPLY_HOLD. Any notification arriving with the notification code IP_07 (TPA Identification error) from XML Gateway triggers the workflow to start with the attribute values defined.

Call API

The Call API notification action invokes a particular API either using online or concurrent mode of execution.

Procedure Name: Specify the qualified PL/SQL procedure name.

Mode of Execution: The options available are:

- Online - API is processed synchronously.
- Concurrent - API is submitted to the Concurrent Manager and processed asynchronously.

Create the specified PL/SQL procedure and validate it with the specified signature before defining the notification action, as the UI does not validate the PL/SQL procedure.

Procedure Signature: The procedure must have wf_parameter_list_t as the only parameter. Parameter type is IN.

This parameter contains details about the notification. For example, notification code, description, and collaboration ID.

For example, business rules call for updates to the security access violation tables when an IP_10 (Security Error) results in business-to-business gateway.

You can define a PL/SQL API that updates security access violation tables. If the PL/SQL is named `COLLABORATION_ACCESS.SECURITY_VIOLATED`, then in the Notification Action Definition window define an action of type Call API with the procedure name as `COLLABORATION_ACCESS.SECURITY_VIOLATED`. Any notification arriving with the notification code IP_10 (Security Error) from the B2B gateway triggers the procedure.

Call API Parameters

This table describes the Call API parameters:

Parameter Name	Description	Parameter Value
Parameter1	Application ID	Application ID to which this notification is intended.
Parameter2	Collaboration ID	Collaboration ID that can uniquely identify the collaboration for which the notification has come.

Parameter Name	Description	Parameter Value
Parameter3	Collaboration Type	Collaboration type of the notification; for example, Order and Change Order.
Parameter4	Reference ID	Application reference ID of the collaboration.
Parameter5	Trading Partner ID	Trading partner ID of the notification message.
Parameter6	Header Description	Description of the notification.
Parameter7	Notification Description	Comma separated list of description for each notification code; value retrieved from CONFIRM.CONFIRMMSG. DESCRIPTN tag of the notification message.
Parameter8	Notification Code	Comma separated list of notification codes; value retrieved from CONFIRMMSG.REASONCODE tag of the notification message.
Parameter9	Status	Status: 00 for Success and 99 for Error; value retrieved from CONFIRM.STATUSLVL of the notification message.

Map Analysis Guidelines

Map Analysis Overview

A map analysis is required to determine how to map data between the Oracle E-Business Suite data model and the required message. For outbound messages, this requires a comparison between the Oracle E-Business Suite database views and the Document Type Definition (DTD) or a production XML message.

The result of the map analysis is used as input to the XML Gateway Message Designer. It is imperative that a complete map analysis is done before attempting to create the message map. The map analysis must resolve all data gaps, identify derivation rules, identify default values, identify data transformation and process controls via Actions, and ensure that all required target fields are mapped.

- Map Analysis Guidelines for Outbound Messages, page A-1
- Map Analysis Guidelines for Inbound Messages, page A-9

Map Analysis Guidelines for Outbound Messages

The steps to performing a map analysis for an outbound message are as follows:

- Compare database views (source) to DTD (target)
- Identify application business event trigger
- Add XML Gateway required Actions to define selection criteria
- Identify source and target document levels

Map Analysis Guidelines for Outbound Messages Checklist

Use the following table as a checklist to track your progress when analyzing an

outbound message. Details of each step are provided in the following sections. Use the "Completed" column to check off the step as it is completed.

Step	Completed	Description
1		CREATE WORKSHEET: Create a worksheet to identify all data.
2		SET UP SOURCE: Create database views or get database views/table definitions (source) for the required message from the database schema.
3		SET UP TARGET: Populate the target column of the worksheet with the DTD or production XML message in the expanded format.
3A		Identify the required DTD (target) elements.
3B		For DTD elements with " " as the occurrence indicator, select one element from the list. Make a note of the elements not selected because these must be removed from the message map to prevent a parser violation.
4		RELATE SOURCE ELEMENT TO TARGET ELEMENT: For each DTD (target) element, identify the database view (source) column to map from.
4A		Identify where code conversion is needed.
4B		Identify Derivation Rules
4C		Identify Defaulting Rules
4D		Identify DTD Attribute Settings
5		CONVERT FROM ORACLE FORMAT TO OAG FORMAT: For each OAG DTD (target) DATETIME, AMOUNT, OPERAMT, or QUANTITY element, identify the database view/table (source) column to map from. Other XML standards may have similar elements.
5A		Identify and map each DATETIME element.
5B		Identify and map each AMOUNT element.
5C		Identify and map each OPERAMT element.
5D		Identify and map each QUANTITY element.

Step	Completed	Description
6		ADD SIBLING FOR DUPLICATE NODES/ELEMENTS: Add Sibling nodes and elements if needed.
7		ADD CHILD TO EXTEND DTD DEFINITION FOR FLEXFIELDS: If there is a database view or table column that you wish to map from but no DTD element is defined to support it, you can use the OAG's USERAREA. Map data to the USERAREA (or corresponding area if using other XML standards).
8		RESOLVE DATA GAPS: Any DTD (target) elements that have not been mapped are data gaps.
9		IDENTIFY APPLICATION BUSINESS EVENT TRIGGER: Identify the application business events to trigger message creation.
10		IDENTIFY REQUIRED ACTIONS: All outbound messages require the following Actions to define the selection criteria. Make a notation on the worksheet to include them. Use the Append Where Clause action to bind to transaction type, transaction subtype, party ID, and party site ID to ECX_OAG_CONTROLAREA_TP_V (formerly ECX_OAG_CONTROLAREA_V see Note, page 2-34). If using ECX_OAG_CONTROLAREA_V, use the Append Where Clause action to bind to transaction type and transaction subtype. Use the Append Where Clause action to bind document ID to header view to select the document
11		IDENTIFY SOURCE AND TARGET DOCUMENT LEVELS

Compare Database Views (Source) to DTD (Target)

Step 1 Create a Worksheet

Create a worksheet with the following columns: Target, Required (Y/N), Source, Code Category, Derivation Rule, Default Value, and Action, as shown in the following table:

Target	Required (Y/N)	Source	Code Category	Derivation Rule	Default Value	Action

Step 2 Set Up Source

Create database views or get database view and table definitions (source) for the required message from the database schema. Include ECX_OAG_CONTROLAREA_V or ECX_OAG_CONTROLAREA_TP_V (see Note, page 2-34) view to map to the CONTROLAREA segment of the DTD.

Step 3 Set Up Target

Populate the target column of the worksheet with the DTD or production XML message in the expanded format.

Note: You may use third party software to expand the DTD or use a production XML message.

Step 3A Identify Required Target Elements

Identify the required DTD (target) elements. An element is required if there is no special symbol next to the element name.

The XML occurrence indicators are:

"+" implies one or more

"*" implies zero or more

"?" implies zero or 1

"|" implies either one or the other

"," implies all listed child elements must be used in the order shown

Step 3B For Elements with "|" as the occurrence indicator

For DTD elements with "|" as the occurrence indicator, select one element from the list. Make a note of the elements not selected in order to remove them from the message map. They must be removed from the message map to prevent a parser violation.

Step 4 Relate Source Element to Target Element

For each DTD (target) element, identify the database view (source) column to map from.

Note: Use the *Open Application Group's Integration Specification* (available at <http://www.openapplications.org>) Appendix C, Field Identifier Descriptions and Appendix D, Segment Descriptions, to get detailed descriptions of each element identified on the DTD.

Step 4A

If the DTD (target) element can be sourced from a database view (source) column, determine if the database view column value requires code conversion.

- If code conversion is required, identify the code category from the list in Appendix B, page B-1.
- Identify the from or to value to determine if the required values are available in the database. If the values are not available, they must be added.

Step 4B Identify Derivation Rule

If the DTD (target) element cannot be sourced from a database view (source) column, determine if the value can be derived. If yes, identify the derivation rule. The derivation rule can be, for example, a combination of several view columns, a procedure call, or a function call. Make a notation that an XML Gateway Action may be required (for example, to get a value from another column, or to combine the values of several columns).

Refer to Appendix C, page C-1. for a list of XML Gateway supported Actions.

Determine if the derived value requires code conversion. If code conversion is required, identify the code category from the list in Appendix B, page B-1.

Important: Identify the from or to value to determine if the required values are available in the database. If the values are not available, they must be added.

Step 4C Identify Defaulting Rule

If the DTD (target) element cannot be sourced from a database view (source) column or derived, determine if the value can be defaulted. If yes, identify the default value. The default value can be a literal or based on a value from another database view column. Make a notation that an XML Gateway Action may be required.

The default value can be set directly in the map using the Message Designer or as an XML Gateway Action if the default value is based on a condition.

Refer to Transaction Map - Actions, page 2-65 for a list of XML Gateway supported Actions.

Determine if the default value requires code conversion. If code conversion is required, identify the code category from the list in Appendix B, page B-1.

Important: Identify the from or to value to determine if the required values are available in the database. If the values are not available, they must be added.

Step 4D Identify DTD Attribute Setting

For each DTD (target) element containing attributes (lower case tags), determine the appropriate setting by checking the usage of the database view or table column and then identifying a valid setting for the attribute by reviewing the options in OAG's Appendix D that lists Segment Descriptions for the DTD element. The appropriate attribute setting is set directly in the target definition as a default value using the Message Designer.

For example, if a view column is used for creation date, the DATETIME "qualifier" attribute value of "CREATION" will be set as the default value using the Message Designer.

Step 5 Convert from Oracle Format to OAG Format

For each DTD (target) DATETIME, AMOUNT, OPERAMT, or QUANTITY element, identify the database view or table (source) column to map from.

Oracle E-Business Suite represents date, amount, operating amount, and quantity in a single database column. OAG represents these as a collection of data. Oracle XML Gateway provides an Action to convert the Oracle representation to the OAG representation. Make a notation that this column requires an Action for "Convert to OAG" that will be defined using the Message Designer.

Step 5A

- For the DATETIME element, identify the database view or table (source) column for the date.
- Follow Step 4D to set the "qualifier," "type," and "index" attributes
- All other OAG fields are set by the "Convert to OAG" Action based on the date value.
- TIMEZONE is defaulted to Greenwich Mean Time (GMT).

Note: OAG DATETIME is made up of the following attributes and elements: qualifier, type, index, YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, SUBSECOND, TIMEZONE.

Step 5B

- For the AMOUNT element, identify database view or table (source) columns for currency code and debit/credit flag if available. The XML Gateway "Convert to OAG" Action can be defined to use the view or table column values or default to OAG recommended values if the view or table columns are not available.

- Follow Step 4D to set the "qualifier," "type," and "index" attributes.
- The OAG fields for NUMOFDEC and SIGN are set by the "Convert to OAG" Action based on the amount value.

Note: OAG AMOUNT is made up of the following attributes and elements: qualifier, type, index, VALUE, NUMOFDEC, SIGN, CURRENCY, DRCR.

Step 5C

- For the OPERAMT fields, identify database view or table (source) columns for currency and unit of measure code if available. The XML Gateway "Convert to OAG" Action can be defined to use the view or table column values or default to OAG recommended values if the view or table columns are not available.
- Follow Step 4D to set the "qualifier" and "type" attributes.
- The OAG fields for NUMOFDEC, SIGN, UOMVALUE, and UOMNUMDEC are set by the "Convert to OAG" Action based on the operating amount value.

Note: OAG OPERAMT is made up of the following attributes and elements: qualifier, type, VALUE, NUMOFDEC, SIGN, CURRENCY, UOMVALUE, UOMNUMDEC, UOM.

Step 5D

For the QUANTITY element, identify database view or table (source) column for the unit of measure code if available. The XML Gateway "Convert to OAG" Action can be defined to use the view or table column value or default to the OAG recommended value if the view or table column is not available.

Follow Step 4D to set the "qualifier" attribute.

The OAG fields for NUMOFDEC and SIGN are set by the "Convert to OAG" Action based on the quantity value.

Note: OAG QUANTITY is made up of the following attributes and elements: qualifier, VALUE, NUMOFDEC, SIGN, UOM.

Step 6 Add Sibling for Duplicate Nodes/Elements

A DTD defines a single occurrence of a node or element. If you have multiple occurrences from the database view or table (source) to map from, make a notation on your worksheet showing the duplicate node or element and then map the element and

attributes of the duplicate node or element. Make a notation to ADD SIBLING using the Message Designer.

An example is the DTD element for PARTNER used to map SHIP-TO, BILL-TO, and REMIT-TO from the Oracle E-Business Suite. You will need to add two PARTNER sibling nodes (same hierarchy level) to map BILL-TO and REMIT-TO.

Step 7 Add Child to Extend DTD Definition for Flexfields

If there is a database view or table column you wish to map from but no DTD element is defined to support it, you can use the USERAREA. You can add a USERAREA as a sibling (same hierarchy level) of another USERAREA or extend an existing USERAREA by adding a child (next level of hierarchy) element and then mapping the element. Make a notation to ADD CHILD using the Message Designer.

This approach is necessary to support all the flexfields included in the database views and tables. Since we do not know which flexfields are implemented at a user's site, all flexfields must be mapped.

Refer to How to Extend DTDs, page 2-108 for details.

Note: The only updates allowed for a DTD are to the USERAREA. Any other changes will invalidate the DTD.

Step 8 Resolve Data Gaps

At this point, you have identified database view and table (source) columns for as many DTD (target) elements as possible. Any required DTD (target) element that has not been mapped represents a data gap.

Consider resolving the data gap using an application flexfield that is included in the database view or table. If this is a valid option, you must implement the designated flexfield by defining and populating it in the Oracle E-Business Suite. Consider carefully which flexfield to implement as you do not want to overwrite a flexfield implemented by the user.

Consider resolving the data gap using an application column not included in the database view. If this is a valid option, you must modify the database view to include the application column as opposed to referencing the table and column directly. Load the modified database view into the database schema. Remember to map the field.

If neither of the above considerations resolves the data gap, then you must add functionality to your application module, create new database views, load the new views into the database schema, and then create the message map.

Step 9 Identify Application Event Trigger

Identify the application business events to trigger message creation. The common trigger points are when the document is created, confirmed, updated, or deleted.

Use the Oracle Workflow Business Event System to register the business event to define the corresponding event subscription to create and send the message.

See: Integrating Oracle XML Gateway with Oracle Workflow Business Event System, page 6-1 for details on how to register and subscribe to application business events.

Step 10 Identify Required Actions

All outbound messages require the following Actions to identify the selection criteria. Make a notation on your worksheet to include them.

- If you are using ECX_OAG_CONTROLAREA_TP_V use the Append Where Clause action to bind transaction type, transaction subtype, party ID, and party site ID

If you are using ECX_OAG_CONTROLAREA_V use the Append Where Clause action to bind transaction type and transaction subtype

Note: The ECX_OAG_CONTROLAREA_TP_V view is an upgraded version of the ECX_OAG_CONTROLAREA_V view. Oracle XML Gateway supports both versions of the database view. The more information see the Note, page 2-34.

- Use the Append Where Clause action to bind document ID to the header view to select the document

Step 11 Identify Source and Target Document Levels

See: Identifying Source and Target Document Levels, page A-17.

Map Analysis Guidelines for Inbound Messages

The map analysis process for inbound messages consists of the following steps:

- Compare DTD (source) to Application Open Interface tables (target)
- Add XML Gateway required Actions
- Add XML Gateway optional Actions
- Identify source and target document levels

Map Analysis Guidelines for Inbound Messages Checklist

Use the following table as a checklist to track your progress when analyzing an inbound message. Details of each step are provided in the following sections. Use the "Completed" column to enter completion data or to check off the step when completed.

Step	Completed	Description
1		CREATE WORKSHEET: Create a worksheet to identify all data.
2		SET UP SOURCE: Get the DTD (source) for the required message or use a production XML message.
3		SET UP TARGET: Populate the target column of the worksheet with the Application Open Interface table definitions or Application API parameter list.
3A		Identify the required Application Open Interface (target) table columns or Application API parameters.
4		RELATE SOURCE ELEMENT TO TARGET ELEMENT: For each Application Open Interface (target) table column, identify the DTD (source) element to map from.
4A		Identify where code conversion is needed.
4B		Identify Derivation Rules
4C		Identify Defaulting Rules
5		CONVERT FROM OAG FORMAT TO ORACLE FORMAT: For each OAG DTD (source) DATETIME, AMOUNT, OPERAMT, or QUANTITY element, identify the Application Open Interface (target) column or Application API parameter to map to. Other XML standards may have similar elements.
5A		Identify and map each DATETIME element.
5B		Identify and map each AMOUNT element.
5C		Identify and map each OPERAMT element.
5D		Identify and map each QUANTITY element.
6		ADD SIBLING FOR DUPLICATE NODES/ELEMENTS: Add Siblings nodes/elements if needed.
7		REVIEW DTD USERAREA: Review all OAG's USERAREA definitions and determine whether they should be mapped to the Application Open Interface table column or Application API parameter. Other XML standards may have similar elements.

production XML message.

Step 3 Set Up Target

Populate the target column of the worksheet with the Application Open Interface table definitions or Application API parameter list.

Note: You may use third party software to expand the DTD or use a production XML message.

Step 3A Identify Required Target Elements

Identify the required Application Open Interface (target) table columns or Application API parameter list as follows:

- NOT NULL columns defined in the data model
- Application Open Interface API or Application API code to enforce required columns that are not defined as NOT NULL
- Open Interface or Application API documentation (for the product) identifying the required columns and associated derivation and defaulting rules.

Step 4 Relate Source Element to Target Element

For each Application Open Interface (target) table column, identify the DTD (source) element to map from.

Note: Use the *Open Applications Group Integration Specification* Appendix C (Field Identifier Descriptions) and D (Segment Descriptions) to get detailed descriptions of each element identified on the DTD.

If the target column is for an internal ID, you must resolve this using a derivation or defaulting rule as outlined in steps 4B and 4C below. The sender cannot send an ID as they do not know what the valid internal IDs are. XML Gateway provides derivation actions to derive address and organization ID.

DTD attributes (lowercase tags) are not stored in the Oracle E-Business Suite, and therefore are not mapped. However, the value of the attribute can be used to determine the exact Application Open Interface table column or Application API parameter to map to.

Step 4A

If the Application Open Interface (target) table column can be sourced from a DTD (source) element, determine if the DTD (source) element value requires code conversion

so the resulting value is meaningful to the Oracle E-Business Suite.

If code conversion is required, identify the code category from the list in Appendix B, page B-1.

Identify the from/to value to determine if the required values are available in the database you are using. If the values are not available, they must be added.

Step 4B Identify Derivation Rule

If the Application Open Interface (target) table column cannot be sourced from a DTD (source) element, determine if it can be derived. Refer to your product's Application Open Interface documentation for any predefined derivation rules. If a derivation rule is defined, make sure the incoming message provides the data to support the derivation rule because the Application Open Interface API will use the data to derive the value for the target column. Make a notation of the derivation rule on your worksheet so that you can verify this during unit testing.

If no derivation rule is defined, you can define one. The derivation rule can be based on several DTD elements or some default values. Make a notation of the derivation rule on your worksheet. Make a notation that an Action may be required.

Refer to Transaction Map - Actions, page 2-65 for a list of XML Gateway supported Actions.

Determine if the derivation rule (element or literal) requires code conversion for the trading partner. If code conversion is required, identify the code category from the list in Appendix B, page B-1.

Identify the from/to value to determine if the required values are available in the database. If the values are not available, they must be added.

Step 4C Identify Defaulting Rule

If the Application Open Interface (target) table column cannot be sourced from a DTD (source) element or derived, determine if it can be defaulted. Refer to your product's Application Open Interface documentation for any predefined defaulting rules. If a defaulting rule is defined, make sure the incoming message provides the data to support the defaulting rule. The Application Open Interface API will use the data as the default value for the target column. Make a notation of the defaulting rule on your worksheet so that you can verify this during unit testing.

If no defaulting rule is defined, you can define one. The default value may be set directly in the target definition using the Message Designer or as an XML Gateway Action if the default value is based on a condition. See Appendix C, page C-1 - XML Gateway Supported Actions.

Determine if the defaulting rule (element or literal) requires code conversion for the trading partner. If code conversion is required, identify the code category from the list in Appendix B, page B-1.

Identify the from/to value to determine if the required values are available in the

database. If the values are not available, they must be added.

Step 5 Convert from OAG Format to Oracle Format

For each DTD (source) DATETIME, AMOUNT, OPERAMT, or QUANTITY element, identify the Application Open Interface (target) column or Application API parameter to map to.

Oracle E-Business Suite represents date, amount, operating amount, and quantity in a single database column. OAG represents these as a collection of data. Oracle XML Gateway provides an Action to convert the OAG representation to the Oracle representation. Make a notation that this column requires an Action for "Convert from OAG" that will be defined using the Message Designer.

Step 5A

For the DATETIME element, identify the Application Open Interface (target) table column for date.

Values for attributes "qualifier," "type," and "index" are not mapped because the meaning is implied in the Application Open Interface table column for date.

Note: OAG DATETIME is made up of the following attributes and elements: qualifier, type, index, YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, SUBSECOND, TIMEZONE.

Step 5B

For the AMOUNT element, identify the Application Open Interface (target) table columns for currency code and debit/credit flag if available. The XML Gateway "Convert from OAG" Action can be defined to store the OAG currency code and debit/credit flag in the columns identified, or ignored if the columns are not available.

Values for attributes "qualifier," "type," and "index" are not mapped because the meaning is implied in the Application Open Interface table column.

Values for NUMOFDEC and SIGN are implied in the Oracle representation of the amount value. The values are not mapped unless your application supports it. If so, you will need to map them using the Message Designer.

Note: OAG AMOUNT is made up of the following elements and attributes: qualifier, type, index, VALUE, NUMOFDEC, SIGN, CURRENCY, DRCR.

Step 5C

For the OPERAMT element, identify the Application Open Interface (target) table columns for the currency code and unit of measure code if available. The XML Gateway

"Convert from OAG" Action can be defined to store the OAG currency code and unit of measure code in the columns identified, or ignored if the columns are not available.

Values for attributes "qualifier" and "type" are not mapped because the meaning is implied in the Application Open Interface table column.

Values for NUMOFDEC, SIGN, UOMVALUE, and UOMNUMDEC are implied in the Oracle representation of the operating amount value. The values are not mapped unless your application supports it. If so, you will need to map them using the Message Designer.

Note: OAG OPERAMT is made up of the following attributes and elements: qualifier, type, VALUE, NUMOFDEC, SIGN, CURRENCY, UOMVALUE, UOMNUMDEC, UOM.

Step 5D

For the QUANTITY element, identify the Application Open Interface (target) table column for the unit of measure code if available. The XML Gateway "Convert from OAG" Action can be defined to store the OAG unit of measure code in the column identified, or ignored if the column is not available.

The value for the "qualifier" attribute is not mapped because the meaning is implied in the Application Open Interface table column.

Values for NUMOFDEC and SIGN are implied in the Oracle representation of the quantity value. The values are not mapped unless your application supports it. If so, you will need to map them using the Message Designer.

Note: OAG QUANTITY is made up of the following attributes and elements: qualifier, VALUE, NUMOFDEC, SIGN, UOM.

Step 6 Add Sibling for Duplicate Nodes/Elements

A DTD defines a single occurrence of an element, but additional occurrences can be added during message implementation to accommodate the business data. Map the duplicate node to the appropriate Application Open Interface table columns.

An example is the DTD element for PARTNER used to identify the trading party. If the Application Open Interface requires a SHIP-TO, BILL-TO, and REMIT-TO, then two PARTNER sibling nodes (same hierarchy level) were added to the original DTD to accommodate the BILL-TO and REMIT-TO. Map these entities to the appropriate Application Open Interface table columns.

Step 7 Review DTD USERAREA

Consider that the original DTD may have been extended to support additional data required by the message. Review all USERAREA definitions and determine whether

they should be mapped to the Application Open Interface table column or to an application API parameter.

For outbound messages, all application flexfields were mapped to USERAREA fields. If the outbound message is being processed as an inbound message to another Oracle E-Business Suite application module, the values may be meaningful. If so, they must be mapped to Application Open Interface table columns or Application API parameters.

Step 8 Resolve Data Gaps

At this point, you have identified DTD (source) elements for as many Application Open Interface (target) table columns or Application API parameters as possible. Any required Application Open Interface (target) table column or Application API parameter that has not been mapped represents a data gap.

- If the required column is an internal ID, you must resolve this using a derivation or defaulting rule as described in Steps 4B and 4C, or use the Derive Address ID or Derive Parent ID actions.
- Consider resolving the data gaps by extending the DTD to include the required data. You can extend the DTD by adding sibling (same hierarchy level) or child (next level hierarchy) USERAREA elements or by adding new elements to an existing USERAREA. Refer to How to Extend DTDs, page 2-108 for information on how to define these.
- Consider creating new Application Open Interface tables and associated API. If this is a valid option, create and load the new table definitions into the database schema and create the message map.

Note: The only updates allowed for a DTD are to the USERAREA. Any other changes will invalidate the DTD.

Step 9 Optional Actions

For Application products that use Application Open Interface tables to stage incoming data and utilize an Application Open Interface API to validate the staged data:

- Use the Insert to Database Table action to insert data into Application Open Interface tables.
- Use the Procedure Call action to execute Application Open Interface API.

For Application products that use application APIs that combine the staging tables and the validation API:

- Use the Procedure Call action and map source (DTD) data to the Application API parameter list. Refer to How to Map to an API, page 2-110 for additional instructions.

Step 10 Required Action

Add root-level, post-process action to raise business event indicating inbound process is complete.

Add Procedure Call action for ECX_STANDARD.setEventDetails. See setEventDetails, page F-6 in the API Appendix.

Step 11 Identify Source and Target Document Levels

See Identify Source and Target Document Levels, page A-17.

Identifying Source and Target Document Levels

After completing the map analysis, identify the document levels for both the source and target data.

A document level represents a collection of data that repeats. For the Application Open Interface tables or database views, each table represents a document level. A level is the parent in a parent-child relationship.

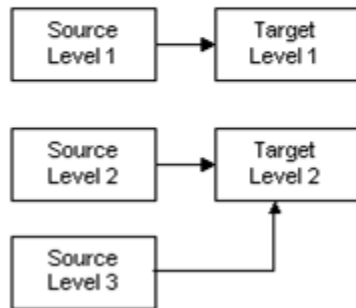
DTDs do not use document levels because the levels are implied based on usage. Refer to the tree diagram in the OAG definitions to get a sense of how the data is grouped. The tree diagram will also give you an idea of where additional occurrences of a datatype may be required.

Once you have identified the document levels for the source and target, proceed to relate the source data structure to the target data structure. This task is straightforward if the number of data levels in the source and target are identical, but can be difficult if the numbers are different. If the number of levels of the source is greater than the target number, you must collapse levels. If the number of levels of the source is less than the target number, you must expand levels.

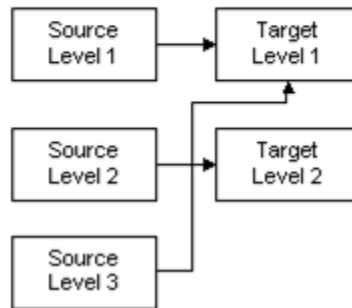
Collapsing Levels

Collapsing levels is the mapping of multiple source levels to the same target level. For example, if your source is 3 levels and your target is 2 levels you can collapse the levels as shown in the following figure:

Correct Way to Collapse Levels



Incorrect Way to Collapse Levels

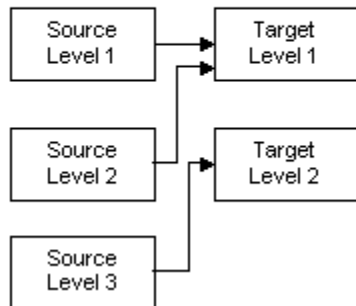


In the correct example above, the result of collapsing levels is that the data in Source Levels 2 and 3 are consolidated and mapped to Target Level 2. If there are two rows in Source Level 2 and three rows in Source Level 3, a total of six rows will be created in Target Level 2.

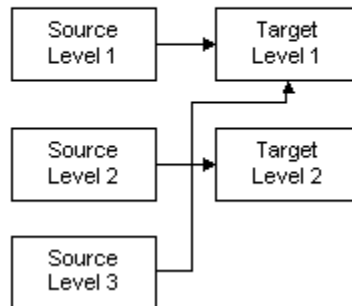
The incorrect example shows the collapsing of Source Levels 1 and 3 to Target Level 1, causing Source Level 3 to cross over Target Level 2.

Another option is to relate Source Levels 1 and 2 to Target Level 1 and relate Source Level 3 to Target Level 2, as shown in the correct example below. (Do **not** map Source Level 3 to Target Level 1, crossing over Target Level 2.)

Correct Way to Collapse Levels



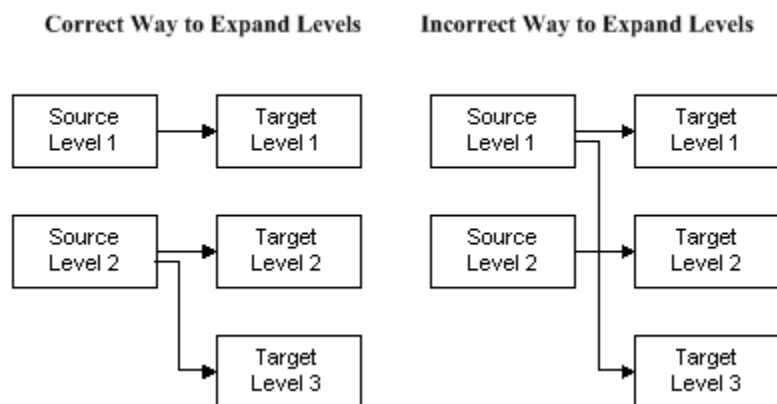
Incorrect Way to Collapse Levels



Whichever option you choose, consider what it means to promote lower level detail data to a higher level. The source data may need to be aggregated to be meaningful at the higher level.

Expanding Levels

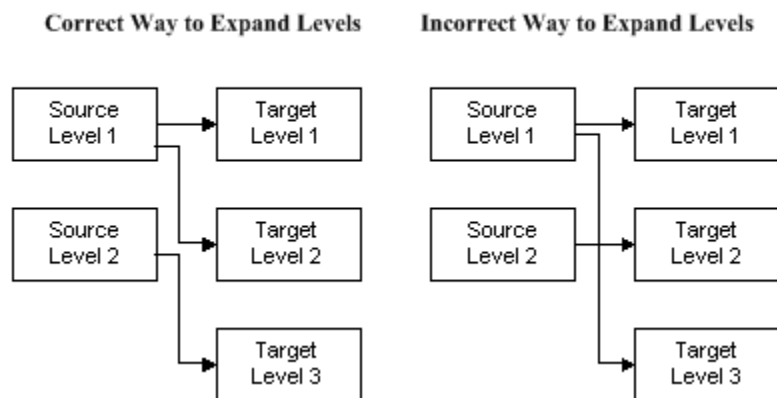
Expanding levels is the mapping of one source level to multiple target levels. For example, if your source is 2 levels and your target is 3 levels you can expand the levels as shown in the following figure:



The result of expanding levels, as shown in the correct example above, is that the data in Source Level 2 is distributed and mapped to Target Levels 2 and 3. If there are two rows in Source Level 2, two rows will be created in Target Level 2 and Target Level 3.

Do not expand Source Levels across Target Levels, as shown in the incorrect example above. Source Level 1 is incorrectly expanded to Target Levels 1 and 3, crossing over Target Level 2.

Another option, as shown in the correct example below, is to distribute Source Level 1 to Target Levels 1 and 2 and map Source Level 2 to Target level 3. (Do **not** map Source Level 1 to Target Levels 1 and 3, crossing over Target level 2, as shown in the incorrect example.)

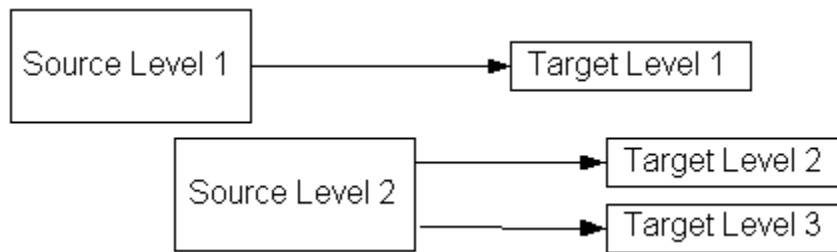


Whichever option you choose, consider what it means to demote data from a higher level to a lower level of detail. The source data may need to be deaggregated to be meaningful at the lower level.

Level Expansion for Discontinuous Nodes

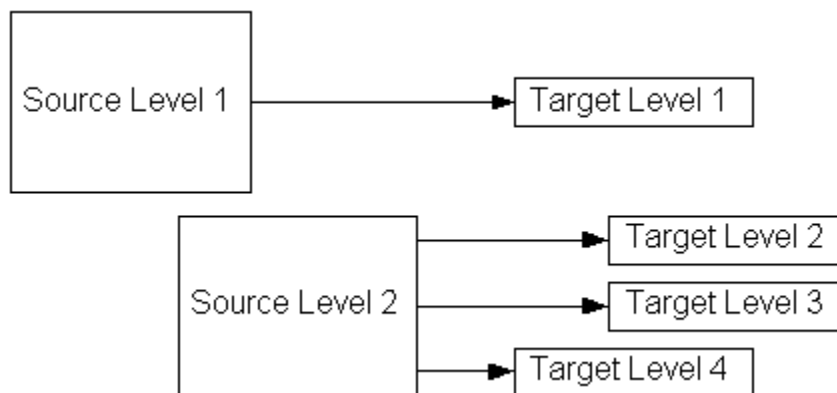
Level expansion is supported if the target expanded levels are all siblings of each other or if they are all children of the previous node. The following diagram shows an

example of correct expansion of levels.



In the example above, Target Level 2 and Target Level 3 are siblings to each other and children of Target Level 1.

The following diagram shows an example of invalid level expansion:



In the example of invalid level expansion above, Target Level 2 and Target Level 3 are siblings to each other and children of Target Level 1. Target Level 4 is a sibling of Target Level 1, with no relationship to Target Levels 2 and 3.

See *Discontinuous Nodes*, page 2-44 for more information on discontinuous nodes.

Another way to accommodate the data structure mismatch is to collapse or expand the database views or Application Open Interface tables and use the modified database views or application open interface table definitions to create the message map.

The resulting hierarchy mapping is defined using the Message Designer Level Mapping Tab.

Recommending DTD Additions or Changes to OAG

Oracle is a member of the Open Application Group's standards committee chartered with developing industry-independent XML standards. This group meets periodically to review recommendations and is responsible for publishing standards changes and additions.

During the map analysis process, you may discover that the needed fields are not represented in a DTD or that a needed DTD is not supported by OAG.

Whether you are recommending the creation of a brand new Business Object Document (BOD) and Document Type Definition (DTD) or additions or changes to an existing DTD, please present your recommendation in the OAG format and be prepared to identify the business case and justification. If you are making a recommendation to add or change an existing DTD, identify the DTD and version you are using. Please make sure your proposed DTD or DTD changes supports both the Application-to-Application (A2A) and Business-to-Business (B2B) scenarios.

If you would like Oracle to submit the request to OAG for you, enter an enhancement request. We will make the proposals to the OAG committee and report back to you. If you want to present your case directly to OAG, please let us know and we will include you in the meeting.

Special Considerations for Custom Messages

Once a document is extracted from the Oracle E-Business Suite, consider updating the application tables to indicate the document has been extracted. This may require adding columns to the appropriate tables. This may be important to the Oracle E-Business Suite for document status reporting or to manage the supply and demand data.

Updating the Oracle E-Business Suite application tables was necessary in the EDI world to prevent a document from being extracted more than once. In the XML Gateway model, documents are extracted based on an event trigger in the Oracle E-Business Suite (as the documents are created) as opposed to batch processing used in the EDI model.

For inbound messages, consider adding calls to application procedures (as an Action) to link the Application Open Interface API process to the downstream process.

Seeded Code Categories

XML Gateway Seeded Code Categories

The XML Gateway seeded code categories and their descriptions are shown in the following table:

Category Code	Description
ACTION_CODE	Action Code (such as Add/Change/Delete)
AC_HANDL	Allowance and Charge Handling Code
AC_SPEC_CHARGES	Allowance and Charge Special Charges
AC_SPEC_SERVICES	Allowance and Charge Special Services
AETC_REASON	AETC Reason Code
AETC_RESPONSIBILITY	AETC Responsibility Code
AGREEMENT	Agreement Code
ALLOW_CHARGES_INDICATOR	Allowance or Charge Indicator Code
ALLOW_METHOD_HANDLING	Allowance or Charge Method of Handling Code
BARCODE	Bar Code
BUYER_ITEM	Buyer Item

Category Code	Description
CARRIER_CODE	Freight Carrier Code
CHARGE_ALLOW_QUALIFIER	Allowance or Charge Qualifier Code
CHARGE_TYPE	Charge Type Code
COMMODITY_COD_QUALIFIER	Commodity Code Qualifier Code
COMPANY_CODE	Company Code
CONTAINER_CODE	Container Code
CONTAINER_TYPE	Container Type Code
COUNTRY	Country Code
CURRENCY	Currency
CURRENCY_CODE	Currency Code
CURRENCY_CONV_TYPE	Currency Conversion Type Code
CUSTOMER_BANK_ACCOUNT	Customer Bank Account Code
CUSTOMER_BANK_ACCOUNT_TYPE	Customer Bank Account Type Code
CUSTOMER_BANK_BRANCH	Customer Bank Branch Code
CUSTOMER_BANK_BRANCH_ID	Customer Bank Branch ID
CUSTOMER_BANK_BRANCH_TYPE	Customer Bank Branch Type Code
CUSTOMER_BANK_LOCATION	Customer Bank Location Code
CUSTOMER_BANK_ORG	Customer Bank Org Code
CUSTOMER_BANK_SITE	Customer Bank Site Code
CUSTOMER_CODE	Customer Code

Category Code	Description
CUSTOMER_ITEM	Customer Item
CUSTOMER_LOCATION_CODE	Customer Location Code
CUSTOMER_NAME	Customer Name
CUSTOMER_NUMBER	Customer Number
CUSTOMER_ORG	Customer Org Code
CUSTOMER_SITE	Customer Site Code
CUSTOMER_SITE_ORG	Customer Site Organization Code
DATE_TIME_QUALIFIER	Date/Time Qualifier Code
DATE_TYPE	Date Type Code
DESTINATION_CLASS	Destination Class Code
DETAIL_SUBTYPE	Detail Subtype Code
DETAIL_TYPE	Detail Type Code
DOC_HANDLING_CODE	Document Handling Code
DOC_PURPOSE_CODE	Document Purpose Code
DOC_STATUS	Document Status Code
DOC_SUBTYPE	Document Subtype
DOC_TYPE	Document Type
EAN	European Article Number (EAN)
EQUIPMENT_CODE	Equipment Code
EQUIPMENT_DESC	Equipment Description Code

Category Code	Description
EXCESS_TRANS_REASON_CODE	Excess Transportation Reason Code
EXCESS_TRANS_RESPONSIBILITY	Excess Transportation Responsibility Code
EXPORT_DETAIL	Export Detail Code
EXPORT_HEADER	Export Header Code
FOB	Freight Code
FOB_PAYMENT	FOB Payment Code
FOB_POINT	FOB Point Code
FREIGHT_TERMS	Freight Terms Code
HAZ_CERTIFICATION	Hazardous Certification Code
HAZ_CLASS	Hazard Class Code
HAZ_MATERIAL	Hazardous Material Code
HAZ_MATL_CODE_QUAL	Hazardous Material Code Qualifier
HAZ_MATL_SHIPMENT_INFO_QUAL	Hazardous Shipment Info Qualifier
INTL_DOMESTIC_CODE	International/Domestic Code
INT_LOC_LOCATION_CODE	Internal Location Code
INT_LOC_ORG	Internal Location Org Code
INT_LOC_SITE	Internal Location Site Code
INVENTORY_ITEM	Inventory Item
INVOICE_TYPE	Invoice Type Code
ISBN_CODE	International Standard Book Number Code

Category Code	Description
ITEM_CATEGORY	Item Category Code
ITEM_QUALIFIER	Item Qualifier Code
ITEM_TYPE	Item Type Code
LABEL_NAME	Label Name Code
LADING_DESC_QUALIFIER	Lading Description Qualifier
METHOD_OF_PAYMENT	Method of Payment Code
NETWORKS	Network Code
PACKAGING_CODE	Packaging Code
PACKAGING_FROM	Packaging From Code
PACK_CODE	Pack Code
PAYMENT_FORMAT	Payment Format Code
PAYMENT_METHOD	Payment Method Code
PAYMENT_TERMS	Payment Terms Code
PAYMENT_TYPE	Payment Type Code
PRICE_BASIS	Price Basis Code
PRICE_IDENTIFIER	Price Identifier Code
PROVINCE	Province Code
PURCHASE_ORDER_TYPE	Purchase Order Type Code
QUALIFIER	Qualifier Code
QUANTITY_TYPE	Quantity Type Code

Category Code	Description
RATING_CATEGORY	Rating Category Code
RECORD_TYPE	Record Type Code
REFERENCE_CODE	Reference Code
REFERENCE_ID	Reference ID
REGION1	Region 1 Code
REGION2	Region 2 Code
REGION3	Region 3 Code
SALES_ORDER_TYPE	Sales Order Type Code
SCHEDULE_PURPOSE	Schedule Purpose Code
SCHEDULE_TYPE	Schedule Type Code
SEGMENT_DESC	Segment Description Code
SERVICE_LEVEL	Service Level Code
SHIPMENT_METHOD_PAYMENT	Shipment Method of Payment Code
SHIPMENT_PRIORITY	Shipment Priority Code
SHIPMENT_QUALIFIER	Shipment Qualifier Code
SHIPMENT_STATUS	Shipment Status Code
SHIP_DELIV_PATTERN	Shipment Delivery Pattern Code
SITE_QUALIFIER	Site Qualifier Code
SPECIAL_CHARGE_ALLOWANCE	Special Charge or Allowance Code
SPECIAL_HANDLING	Special Handling Code

Category Code	Description
SPECIAL_SERVICES	Special Services Code
STATE	State Code
SUPPLIER_BANK_ACCOUNT	Supplier Bank Account Code
SUPPLIER_BANK_ACCOUNT_TYPE	Supplier Bank Account Type Code
SUPPLIER_BANK_BRANCH	Supplier Bank Branch Code
SUPPLIER_BANK_BRANCH_ID	Supplier Bank Branch ID Code
SUPPLIER_BANK_BRANCH_TYPE	Supplier Bank Branch Type Code
SUPPLIER_BANK_LOCATION	Supplier Bank Location Code
SUPPLIER_BANK_ORG	Supplier Bank Org Code
SUPPLIER_BANK_SITE	Supplier Bank Site Code
SUPPLIER_CODE	Supplier Code
SUPPLIER_ITEM	Supplier Item Code
SUPPLIER_LOCATION_CODE	Supplier Location Code (such as EDI Location Code)
SUPPLIER_NAME	Supplier Name Code
SUPPLIER_NUMBER	Supplier Number Code
SUPPLIER_ORG	Supplier Organization Code
SUPPLIER_SCHEDULING_DESCRIPTOR	Supplier Scheduling Descriptor Code
SUPPLIER_SITE	Supplier Site Code
SUPPLIER_SITE_ORG	Supplier Site Organization Code
TAX_CODE	Tax Code

Category Code	Description
TAX_EXEMPT	Tax Exempt Code
TAX_EXEMPT_REASON	Tax Exempt Reason Code
TAX_JURISDICTION	Tax Jurisdiction Code
TAX_NAME	Tax Name Code
TERMS	Terms Code
TERMS_BASIS_DATE_CODE	Terms Basis Date Code
TERMS DUE DATE QUALIFIER	Terms Due Date Qualifier
TERMS_TYPE	Terms Type Code
TERRITORY	Territory Code
TRADING_PARTNER	Trading Partner Code
TRANSPORTATION_TERMS	Transportation Terms Code
TRANSPORTATION_TERMS_QUALIFIER	Transportation Terms Qualifier Code
UNIT_PRICE_BASIS	Basis of Unit Price Code
UOM	Unit of Measure Code
UPC	Uniform Product Code (UPC) Code
VESSEL_REQUIREMENT	Vessel Requirement Code
WAREHOUSE_CODE	Warehouse Code
WAREHOUSE_LOCATION_CODE	Warehouse Location Code
WAREHOUSE_NAME	Warehouse Name Code
WAREHOUSE_NUMBER	Warehouse Number Code

Category Code	Description
WAREHOUSE_ORG	Warehouse Organization Code
WAREHOUSE_SITE	Warehouse Site Code
WAREHOUSE_SITE_ORG	Warehouse Site Organization
WAYBILL REQUEST	Waybill Request Code

Supported Actions

XML Gateway Supported Actions

The following table lists the XML Gateway supported Actions. For the map analysis process, identify the required Action. For detailed descriptions of each Action and instructions on how to define an Action see Transaction Map -Actions, page 2-65.

Action Category	Action Description
Assignments	Assign variable value
	Create global value
Database Functions	Assign next sequence value
	Append where clause
	Insert into database table
Derivations	Derive Address ID from Location Code
	Derive Parent ID from Location Code
Function Call	Execute function and assign function return value
Math Functions	Add
	Divide
	Multiply
	Subtract

Action Category	Action Description
OAG Standard Conversions	Convert Oracle date to OAG date format
	Convert Oracle operating amount to OAG operating amount format
	Convert Oracle quantity to OAG quantity format
	Convert Oracle amount to OAG amount format
	Convert OAG date to Oracle date format
	Convert OAG operating amount to Oracle operating amount
	Convert OAG quantity to Oracle quantity format
	Convert OAG amount to Oracle amount format
Other	Exit program
Predefined Variable	Get predefined variable value:
	Code Conversion Return Status for (a specific element)
	Internal Control Number
	Return Code
	Return Message
	Receiver Trading Partner ID
	Sender Trading Partner ID
Procedure Call	Execute procedure with send and return parameters
	Send error message to trading partner or XML Gateway system administrator contact
String Functions	Perform Concatenate
	Perform Substring

Action Category	Action Description
XSLT Transformation	Execute procedure to perform XSLT

Each ACTION may be based on a condition. A condition consists of two operands stated as a literal value or a variable. If a condition is defined, the ACTION is executed if the condition result is true. The condition operators supported by XML Gateway are listed in the following table:

Operator	Description
null	Null
not null	Not Null
=	Equal
!=	Not Equal
<	Less Than
>	Greater Than
<=	Less Than or Equal To
>=	Greater Than or Equal To

Note: Do not attempt to perform date comparisons unless your dates are stated as Julian dates. Use database functions to compare dates instead.

Naming Conventions

XML Gateway Naming Conventions Summary

There are several areas where naming conventions are defined to facilitate recognition of data. They are:

Message Designer

- Message Map Names
See Specify a Map Name, page 2-26.
- Data Definition Names
See Select/Create a Source/Target Data Definition, page 2-28.
- XGM File Names
See Transaction Map - Element Mapping, page 2-62.

Setup Forms

- Transaction Type and Transaction Subtype
See Define Transactions Form, page 3-11.
- External Transaction Type and External Transaction Subtype
See Define Transactions Form, page 3-11.

Processing

- Business Events
See Integrating Oracle XML Gateway with Oracle Workflow Business Event System, page 6-1.

Timezone Values

This appendix covers the following topics:

- XML Gateway Valid Time Zone Values

XML Gateway Valid Time Zone Values

Listed in the tables below are the valid values for the Oracle E-Business Suite profile option **ECX: Server Time Zone**. One of the following values must be entered exactly, or the time zone will default to Greenwich Mean Time (GMT).

The values are organized by geographical region. Select the region/city that corresponds to the time zone in which your database server is running.

For outbound transactions, the date and time data retrieved from the database, along with the time zone specified in the profile option, are used to determine the GMT deviation. The deviation is used in the XML message generated by the XML Gateway. No conversion is performed.

For inbound transactions, if the time zone of the incoming message is different from the time zone specified in the profile option, the incoming date and time will be converted.

The time zone values are categorized as follows:

- Africa, page E-2
- America, page E-4
- Antarctica, page E-8
- Asia, page E-8
- Atlantic, page E-12
- Australia, page E-12

- Europe, page E-13
- Indian, page E-15
- Pacific, page E-15

Time Zone Values: Africa

Time Zone Values: Africa

Africa/Abidjan

Africa/Accra

Africa/Addis_Ababa

Africa/Algiers

Africa/Asmera

Africa/Bangui

Africa/Banjul

Africa/Bissau

Africa/Blantyre

Africa/Bujumbura

Africa/Cairo

Africa/Casablanca

Africa/Conakry

Africa/Dakar

Africa/Dar_es_Salaam

Africa/Djibouti

Time Zone Values: Africa

Africa/Douala

Africa/Freetown

Africa/Gaborone

Africa/Harare

Africa/Johannesburg

Africa/Kampala

Africa/Khartoum

Africa/Kigali

Africa/Kinshasa

Africa/Lagos

Africa/Libreville

Africa/Lome

Africa/Luanda

Africa/Lubumbashi

Africa/Lusaka

Africa/Malabo

Africa/Maputo

Africa/Maseru

Africa/Mbabane

Africa/Mogadishu

Time Zone Values: Africa

Africa/Monrovia

Africa/Nairobi

Africa/Ndjamena

Africa/Niamey

Africa/Nouakchott

Africa/Ouagadougou

Africa/Porto-Novo

Africa/Sao_Tome

Africa/Timbuktu

Africa/Tripoli

Africa/Tunis

Africa/Windhoek

Time Zone Values: America

Time Zone Values: America

America/Adak

America/Anchorage

America/Anguilla

America/Antigua

America/Aruba

Time Zone Values: America

America/Asuncion

America/Barbados

America/Belize

America/Bogota

America/Buenos_Aires

America/Caracas

America/Cayenne

America/Cayman

America/Chicago

America/Costa_Rica

America/Cuiaba

America/Curacao

America/Dawson_Creek

America/Denver

America/Dominica

America/Edmonton

America/El_Salvador

America/Fortaleza

America/Godthab

America/Grand_Turk

Time Zone Values: America

America/Grenada

America/Guadeloupe

America/Guatemala

America/Guayaquil

America/Guyana

America/Halifax

America/Havana

America/Indianapolis

America/Jamaica

America/La_Paz

America/Lima

America/Los_Angeles

America/Managua

America/Manaus

America/Martinique

America/Mazatlan

America/Mexico_City

America/Miquelon

America/Montevideo

America/Montreal

Time Zone Values: America

America/Montserrat

America/Nassau

America/New_York

America/Noronha

America/Panama

America/Paramaribo

America/Phoenix

America/Port_of_Spain

America/Port-au-Prince

America/Porto_Acre

America/Puerto_Rico

America/Regina

America/Santiago

America/Santo_Domingo

America/Sao_Paulo

America/Scoresbysund

America/St_Johns

America/St_Kitts

America/St_Lucia

America/St_Thomas

Time Zone Values: America

America/St_Vincent

America/Tegucigalpa

America/Thule

America/Tijuana

America/Tortola

America/Vancouver

America/Winnipeg

Time Zone Values: Antarctica

Time Zone Values: Antarctica

Antarctica/Casey

Antarctica/DumontDUrville

Antarctica/Mawson

Antarctica/McMurdo

Antarctica/Palmer

Time Zone Values: Asia

Time Zone Values: Asia

Asia/Aden

Asia/Alma-Ata

Time Zone Values: Asia

Asia/Amman

Asia/Anadyr

Asia/Aqtau

Asia/Aqtobe

Asia/Ashkhabad

Asia/Baghdad

Asia/Bahrain

Asia/Baku

Asia/Bangkok

Asia/Beirut

Asia/Bishkek

Asia/Brunei

Asia/Calcutta

Asia/Colombo

Asia/Dacca

Asia/Damascus

Asia/Dubai

Asia/Dushanbe

Asia/Hong_Kong

Asia/Irkutsk

Time Zone Values: Asia

Asia/Ishigaki

Asia/Jakarta

Asia/Jayapura

Asia/Jerusalem

Asia/Kabul

Asia/Kamchatka

Asia/Karachi

Asia/Katmandu

Asia/Krasnoyarsk

Asia/Kuala_Lumpur

Asia/Kuwait

Asia/Macao

Asia/Magadan

Asia/Manila

Asia/Muscat

Asia/Nicosia

Asia/Novosibirsk

Asia/Phnom_Penh

Asia/Pyongyang

Asia/Qatar

Time Zone Values: Asia

Asia/Rangoon

Asia/Riyadh

Asia/Saigon

Asia/Seoul

Asia/Shanghai

Asia/Singapore

Asia/Taipei

Asia/Tashkent

Asia/Tbilisi

Asia/Tehran

Asia/Thimbu

Asia/Tokyo

Asia/Ujung_Pandang

Asia/Ulan_Bator

Asia/Vientiane

Asia/Vladivostok

Asia/Yakutsk

Asia/Yekaterinburg

Asia/Yerevan

Time Zone Values: Atlantic

Time Zone Values: Atlantic

Atlantic/Azores

Atlantic/Bermuda

Atlantic/Canary

Atlantic/Cape_Verde

Atlantic/Faeroe

Atlantic/Jan_Mayen

Atlantic/Reykjavik

Atlantic/South_Georgia

Atlantic/St_Helena

Atlantic/Stanley

Time Zone Values: Australia

Time Zone Values: Australia

Australia/Adelaide

Australia/Brisbane

Australia/Darwin

Australia/Lord_Howe

Australia/Perth

Australia/Sydney

Time Zone Values: Europe

Time Zone Values: Europe

Europe/Amsterdam

Europe/Andorra

Europe/Athens

Europe/Belgrade

Europe/Berlin

Europe/Brussels

Europe/Bucharest

Europe/Budapest

Europe/Chisinau

Europe/Copenhagen

Europe/Dublin

Europe/Gibraltar

Europe/Helsinki

Europe/Istanbul

Europe/Kaliningrad

Europe/Kiev

Europe/Lisbon

Europe/London

Europe/Luxembourg

Time Zone Values: Europe

Europe/Madrid

Europe/Malta

Europe/Minsk

Europe/Monaco

Europe/Moscow

Europe/Oslo

Europe/Paris

Europe/Prague

Europe/Riga

Europe/Rome

Europe/Samara

Europe/Simferopol

Europe/Sofia

Europe/Stockholm

Europe/Tallinn

Europe/Tirane

Europe/Vaduz

Europe/Vienna

Europe/Vilnius

Europe/Warsaw

Time Zone Values: Europe

Europe/Zurich

Time Zone Values: Indian

Time Zone Values: Indian

Indian/Antananarivo

Indian/Chagos

Indian/Christmas

Indian/Cocos

Indian/Comoro

Indian/Kerguelen

Indian/Mahe

Indian/Maldives

Indian/Mauritius

Indian/Mayotte

Indian/Reunion

Time Zone Values: Pacific

Time Zone Values: Pacific

Pacific/Apia

Pacific/Auckland

Time Zone Values: Pacific

Pacific/Chatham

Pacific/Easter

Pacific/Efate

Pacific/Enderbury

Pacific/Fakaofu

Pacific/Fiji

Pacific/Funafuti

Pacific/Galapagos

Pacific/Gambier

Pacific/Guadalcanal

Pacific/Guam

Pacific/Honolulu

Pacific/Kiritimati

Pacific/Kosrae

Pacific/Majuro

Pacific/Marquesas

Pacific/Nauru

Pacific/Niue

Pacific/Norfolk

Pacific/Noumea

Time Zone Values: Pacific

Pacific/Pago_Pago

Pacific/Palau

Pacific/Pitcairn

Pacific/Ponape

Pacific/Port_Moresby

Pacific/Rarotonga

Pacific/Saipan

Pacific/Tahiti

Pacific/Tarawa

Pacific/Tongatapu

Pacific/Truk

Pacific/Wake

Pacific/Wallis

Oracle XML Gateway APIs

The Oracle XML Gateway provides the following APIs:

- Execution engine-level APIs, page F-1
- Message Designer APIs, page F-4
- Trading Partner Setup API, page F-29

Execution Engine APIs

The following APIs are for use with the execution engine:

Package: ECX_STANDARD, page F-1

perform_xslt_transformation, page F-1

Package: ECX_ERRORLOG, page F-3

external_system, page F-3

APIs Defined in ECX_STANDARD

perform_xslt_transformation

PL/SQL Syntax

```
procedure perform_xslt_transformation
```

```
(i_xml_file in out clob,
 i_xslt_file_name in varchar2,
 i_xslt_file_ver in number,
 i_xslt_application_code in varchar2,
 i_dtd_file_name in varchar2 default null,
 i_dtd_root_element in varchar2 default null,
 i_dtd_version in varchar2 default null,
 i_retcode out pls_integer,
 i_retmsg out varchar2);
```

Description

Used to apply a style sheet to an XML message and return the transformed XML messaged for further processing by the calling environment.

The DTD file name, version, and root element are required input parameters for this API, therefore the associated DTDs must be loaded into the XML Gateway repository. Refer to Loading and Deleting a DTD, page 2-112 for details on loading a DTD.

This API is independent of the Message Designer: XSLT Transformation action. This API is not intended for use in a message map.

Note: The profile option ECX_XML_VALIDATE_FLAG must be set to "Y" for this action to be performed. For more information on this profile option, see Define System Profile Options, page 3-3.

Arguments (input)

<code>i_xml_file</code>	The XML message to be transformed.
<code>i_xslt_file_name</code>	The XSLT style sheet file to be used for the transformation.
<code>i_xslt_file_ver</code>	The version of the XSLT style sheet file. The highest version with the same file name and application code is used if no version number is provided.
<code>i_xslt_application_code</code>	The name of the subdirectory where the XSLT style sheet is source-controlled (for example: ar/xml/xslt).
<code>i_dtd_file_name</code>	The name of the DTD used in the XML message.
<code>i_dtd_root_element</code>	The root element of the DTD used in the XML message.
<code>i_dtd_version</code>	Version of the DTD used in the XML message.

Arguments (output)

<code>i_xml_file</code>	The transformed XML message.
<code>i_retcode</code>	Return code for the procedure.

<code>i_retmsg</code>	Return message for the procedure.
-----------------------	-----------------------------------

APIs Defined in ECX_ERRORLOG

external_system

PL/SQL Syntax

```
procedure external_system
(
  i_outmsgid in raw,
  i_status in pls_integer,
  i_errmsg in varchar2 default null,
  i_timestamp in date,
  i_errparams in varchar2,
  o_ret_code out pls_integer,
  o_ret_msg out varchar2);
```

Description

Used by both Oracle and non-Oracle messaging systems to report delivery status. The status information is written to the XML Gateway log tables to track and report transaction delivery data.

Arguments (input)

<code>i_outmsgid</code>	Message ID maintained by the XML Gateway execution engine for the outbound message delivered by the messaging system.
<code>i_status</code>	Message delivery status as reported by the messaging system.
<code>i_errmsg</code>	Error messages reported by the messaging system.
<code>i_timestamp</code>	Time stamp from the messaging system indicating when it processed the outbound message created by XML Gateway.
<code>i_errparams</code>	Errors reported by messaging system in multilingual delivery.

Arguments (output)

<code>o_ret_code</code>	Return code for the procedure
<code>o_ret_msg</code>	Return message for the procedure

Message Designer APIs

XML Gateway has provided special purpose procedures and functions for use in the Message Designer to perform the following tasks:

1. Set event details to raise a business event for inbound transactions
2. Get event details using the event name maintained by the XML Gateway Execution Engine
3. Perform string manipulations
4. Get Trading Partner information using the sender's or receiver's Trading Partner ID maintained by the XML Gateway Execution Engine
5. Get message envelope data for inbound message
6. Get delivery data for outbound message
7. Get document logging information for a business document
8. Set message delivery status
9. Get System Administrator e-mail address from ECX: System Administrator Email Address system profile
10. Get sender logical ID from ECX_OAG_LOGICALID system profile
11. Set Error Exit

Use the Message Designer, Procedure Call or Function Call actions to initiate the APIs.

Important: With the exception of ECX_ERRORLOG.external_system, the procedures and functions in this appendix are meant for use in the Message Designer only and should not be used in any other context.

Most procedures include input and output arguments. You can map a source or target variable to the input arguments. The output arguments can be used in conjunction with the Send Error Message action for warnings or the ECX_ACTIONS.

set_error_exit_program API for serious errors to send a notification. The notification may be sent to the Trading Partner contact, the system administrator (identified in the ECX: System Administrator Email Address system profile), or both.

If the successful completion of an API is critical to the success of your transaction, you may wish to use the Exit Program action to terminate the transaction if the API fails to process completely.

The following APIs are described in this section:

Package: ECX_STANDARD, page F-1
setEventDetails, page F-6
getEventDetails, page F-16
getEventSystem, page F-8
getReferenceID, page F-9
Package: ECX_DOCUMENT, page F-9
get_delivery_attribs, page F-9
Package: ECX_CONDITIONS, page F-12
getLengthForString, page F-12
getPositionInString, page F-12
getSubString, page F-13
Package: ECX_TRADING_PARTNER_PVT, page F-14
get_receivers_tp_info, page F-14
get_senders_tp_info, page F-15
get_sysadmin_email, page F-15
getEnvelopeInformation, page F-16
getOAGLOGICALID, page F-18
Package: ECX_ERRORLOG, page F-18
getDoclogDetails, page F-18
external_system, page F-3
Package: ECX_ACTIONS, page F-21
set_error_exit_program, page F-21
Package: ECX_ATTACHMENT, page F-22
register_attachment, page F-22
retrieve_attachment, page F-24
reconfig_attachment, page F-25
Package: ECX_ENG_UTILS, page F-27
convert_to_cxml_date, page F-27
convert_to_cxml_datetime, page F-28
convert_from_cxml_datetime, page F-28

APIs Defined in ECX_STANDARD

setEventDetails

PL/SQL Syntax

```
procedure setEventDetails
(eventname in varchar2,
eventkey in varchar2,
parameter1 in varchar2,
parameter2 in varchar2,
parameter3 in varchar2,
parameter4 in varchar2,
parameter5 in varchar2,
parameter6 in varchar2,
parameter7 in varchar2,
parameter8 in varchar2,
parameter9 in varchar2,
parameter10 in varchar2,
retcode out pls_integer,
retmsg out varchar2);
```

Description

Sets event details to raise a business event for inbound transactions.

This is defined at the root level as a post-process action to indicate that an inbound message has been processed. Any event subscription defined in the Oracle E-Business Suite interested in this inbound message will proceed to consume it.

This procedure must be used for all inbound transactions. The argument values vary by transaction with the exception of the confirmation message where the argument values are specific. See How to Implement an OAG Confirmation BoD, page 4-17 to see how the event details are defined.

Arguments (input)

eventname	Unique identifier for the business event associated with the inbound message. The event name consists of the following components:
------------------	--

ORACLE.APPS.<COMPONENT>.<TASK>.<EVENT>

where:

COMPONENT is based on the internal transaction type entered using the Define Transactions window. It represents the product short code.

TASK is based on the internal transaction subtype entered using the Define Transactions window. It represents a description of the object.

EVENT is a literal that describes the business function of

the message.

Following is an example of an Event Name that identifies a confirmation event associated with an outbound purchase order ORACLE.APPS.PO.POO.CONFIRM

eventkey	Unique identifier for the business document from the Oracle E-Business Suite associated with the business event.
parameter1 through parameter10	User-defined parameters to pass data of interest to the event subscription defined in the Oracle E-Business Suite for the inbound business document.

Arguments (output)

retcode	Return code for the procedure.
retmsg	Return message for the procedure.

getEventDetails

PL/SQL Syntax

```
procedure getEventDetails
(eventname out varchar2,
eventkey out varchar2,
itemtype out varchar2,
itemkey out varchar2,
parentitemtype out varchar2,
parentitemkey out varchar2,
retcode out pls_integer,
retmsg out varchar2);
```

Description

Gets event details using the event name maintained by the XML Gateway Execution Engine.

Arguments (input)

None.

Arguments (output)

eventname	Event name passed internally to the procedure. It is a unique identifier for the business event associated with the transaction.
eventkey	Event key associated with the event name passed.
itemtype	Unique identifier for a group of objects that share the same

set of item attributes (also known as variables).

Item types are created using the Workflow Builder and are used by Oracle e-Business Suite application modules to group related functions.

itemkey	Unique identifier for an item in an item type
parentitemtype	Parent item type for the item type
parentitemkey	Parent item key for the item key
retcode	Return code for the procedure
retmsg	Return message for the procedure

getEventSystem

PL/SQL Syntax

```
(from_agent out varchar2,  
to_agent out varchar2,  
from_system out varchar2,  
to_system out varchar2,  
retcode out pls_integer,  
retmsg out varchar2);
```

Description

Gets event details related to the system and agent using the event name maintained by the XML Gateway Execution Engine.

The procedure is context-sensitive, so you will receive return values relevant to the context. The "from" parameters are for inbound transactions. The "to" parameters are for outbound transactions.

Arguments (input)

None

Arguments (output)

from_agent	Workflow agent (queue) the inbound message is dequeued from.
to_agent	Workflow agent (queue) the outbound message is enqueued to.
from_system	System processing the inbound message
to_system	System processing the outbound message

retcode	Return code for the procedure
retmsg	Return code for the message

getReferenceID

PL/SQL Syntax

```
function getReferenceID
return varchar2;
```

Description

Returns the value associated with the REFERENCEID element of the OAG CONTROLAREA. The field contains a concatenated value consisting of system name, event name, and event key delimited by ":". This function is used for message maps in which the OAG standard is not used.

For message maps created using the OAG standard, the ECX_OAG_CONTROLAREA_TP_V view is used to retrieve the reference_id identified by the business event. This value is used to map to the OAG CONTROLAREA, REFERENCEID element.

Arguments (input)

None.

Arguments (output)

None.

APIs Defined in ECX_DOCUMENT

get_delivery_attrbs

PL/SQL Syntax

```
procedure get_devlivery_attrbs
```

```
(transaction_type in varchar2,
 transaction_subtype in varchar2,
 party_id in varchar2,
 party_site_id in varchar2,
 party_type in/out varchar2,
 standard_type out varchar2,
 standard_code out varchar2,
 ext_type out varchar2,
 ext_subtype out varchar2,
 source_code out varchar2,
 destination_code out varchar2,
 destination_type out varchar2,
 destination_address out varchar2,
 username out varchar2,
 password out varchar2,
 map_code out varchar2,
 queue_name out varchar2,
 tp_header_id out pls_integer,
 retcode out pls_integer,
 retmsg out varchar2);
```

Description

Gets setup data using the internal transaction type, subtype, party id, party_type, and party site id. The data is required to process an outbound transaction.

Arguments (input)

transaction_type	Internal transaction type passed to the procedure. The internal transaction type is entered using the Define Transactions window.
transaction_subtype	Internal transaction subtype passed to the procedure. The internal transaction subtype is entered using the Define Transactions window.
party_id	Trading partner ID passed to the procedure. Party site ID will be used if Party ID is null. The trading partner ID is entered using the Define Trading Partners window.
party_site_id	Trading partner site ID passed to the procedure. The trading partner site ID is entered using the Define Trading Partners window.
party_type	Party type associated with the trading partner and trading partner site passed to the procedure. Party type is entered using the Define Trading Partners window.

Arguments (output)

party_type	Party type associated with the trading partner and trading partner site passed to the procedure. Party type is entered using the Define Trading Partners window.
-------------------	--

standard_type	The XML standard type associated with the transaction passed to the procedure. Standard type is entered using the Define XML Standards window.
standard_code	The standard code (for example, OAG) associated with the transaction passed to the procedure. Standard code is entered using the Define XML Standards window.
ext_type	External transaction type associated with the internal transaction type passed to the procedure. External transaction type is entered using the Define Transactions window.
ext_subtype	External transaction subtype associated with the internal transaction subtype passed to the procedure. External transaction subtype is entered using the Define Transactions window.
source_code	Source location code associated with the trading partner and trading partner site passed to the procedure. Source location code is entered using the Define Trading Partners window.
destination_code	Destination location code associated with the trading partner and trading partner site passed to the procedure. Destination location code is entered using the Define Trading Partners window.
destination_type	Destination location type associated with the trading partner and trading partner site passed to the procedure. Destination location type is entered using the Define Trading Partners window.
destination_address	Destination address associated with the trading partner and trading partner site passed to the procedure. Destination address is entered using the Define Trading Partners window.
username	Username associated with the trading partner and trading partner site passed to the procedure. Username is entered using the Define Trading Partners window.
password	Password associated with the username for the trading partner and trading partner site passed to the procedure. The password is returned in encrypted format. Password is entered using the Define Trading Partners window.

map_code	Message map associated with the trading partner and transaction passed to the procedure. The map code is entered using the Define Trading Partners window.
queue_name	Queue name associated with the transaction passed to the procedure. The queue name is entered using the Define Transactions window.
tp_header_id	System-generated identifier for the trading partner passed to the procedure, entered using the Define Trading Partners window
retcode	Return code for the procedure
retmsg	Return message for the procedure

APIs Defined in ECX_CONDITIONS

getLengthForString

PL/SQL Syntax

```
procedure getLengthForString
(i_string in varchar2);
i_length out pls_integer);
```

Description

Determines the length of the string passed to it. This procedure may be used in conjunction with the getPositionInString procedure or the Perform Substring action.

Arguments (input)

i_string	The input string.
-----------------	-------------------

Arguments (output)

i_length	The length of the input string.
-----------------	---------------------------------

getPositionInString

PL/SQL Syntax

```
procedure getPositionInString
```

```
(i_string in varchar2,
 i_search_string in varchar2,
 i_start_position in pls_integer, default null,
 i_occurrence in pls_integer, default null,
 i_position out pls_integer);
```

Description

Parses a concatenated string with delimiters into its individual components. The `i_search_string` parameter identifies the delimiter. The `i_occurrence` parameter identifies which occurrence of the delimiter to check for. The return value of the procedure is the position of the first character of the portion of the string you are interested in.

This procedure can be used in conjunction with the `getLengthForString` procedure or the Perform Substring action.

Arguments (input)

<code>i_string</code>	The input string.
<code>i_search_string</code>	The delimiter used in the concatenated string.
<code>i_start_position</code>	The character position to begin parsing from.
<code>i_occurrence</code>	The occurrence of the delimiter to search for.

Arguments (output)

<code>i_position</code>	The character position of the first character of the portion of the input string you are interested in.
-------------------------	---

getSubString

PL/SQL Syntax

```
procedure getSubString
(i_string in varchar2,
 i_start_position in pls_integer, default 0,
 i_length in pls_integer, default 0,
 i_substr out varchar2);
```

Description

Parses a string passed to it given the start position and the length of the substring.

This procedure is used when the length of the substring is maintained in a variable whereas the Perform Substring action is used if the length of the string is a literal value.

Arguments (input)

<code>i_string</code>	The input string.
-----------------------	-------------------

i_start_position	The character position to begin parsing from.
i_length	Length from start position of input string to include in resulting substring.

Arguments (output)

i_substr	The substring.
-----------------	----------------

APIs Defined in ECX_TRADING_PARTNER_PVT

get_receivers_tp_info

PL/SQL Syntax

```
procedure get_receivers_tp_info
(p_party_id out number,
 p_party_site_id out number,
 p_org_id out pls_integer,
 p_admin_email out varchar2,
 retcode out pls_integer,
 retmsg out varchar2);
```

Description

Gets Trading Partner data using the receiver's trading partner ID maintained by the XML Gateway Execution Engine. The Trading Partner data is entered using the Define Trading Partners window.

Arguments (input)

None

Arguments (output)

p_party_id	Trading Partner ID
p_party_site_id	Site associated with the Trading Partner.
p_org_id	Organization associated with the Trading Partner site.
p_admin_email	E-mail address associated with the Trading Partner contact.
retcode	Return code for the procedure
retmsg	Return message for the procedure

get_senders_tp_info

PL/SQL Syntax

```
procedure get_senders_tp_info
(p_party_id out number,
 p_party_site_id out number,
 p_org_id out pls_integer,
 p_admin_email out varchar2,
 retcode out pls_integer,
 retmsg out varchar2);
```

Description

Gets Trading Partner data using the sender's trading partner ID maintained by the XML Gateway Execution Engine. The Trading Partner data is entered using the Define Trading Partners window.

Arguments (input)

None

Arguments (output)

p_party_id	Trading Partner ID
p_party_site_id	Site associated with the Trading Partner.
p_org_id	Organization associated with the Trading Partner site.
p_admin_email	E-mail address associated with the Trading Partner contact.
retcode	Return code for the procedure
retmsg	Return message for the procedure

get_sysadmin_email

PL/SQL Syntax

```
procedure get_sysadmin_email
(email_address out varchar2,
 retcode out pls_integer,
 errmsg out varchar2);
```

Description

Gets the system administrator e-mail address defined for the ECX: System Administrator Email Address profile option.

This procedure is not required if your are using the Send Error Message action to send

notifications to the system administrator. The address for the system administrator is derived.

Arguments (input)

None

Arguments (output)

email_address	E-mail address identified in the ECX: System Administrator Email Address system profile.
retcode	Return code for the procedure
errmsg	Return message for the procedure

getEnvelopeInformation

PL/SQL Syntax

```
procedure getEnvelopeInformation
(i_internal_control_number in pls_integer,
 i_message_type out varchar2,
 i_message_standard out varchar2,
 i_transaction_type out varchar2,
 i_transaction_subtype out varchar2,
 i_document_number out varchar2,
 i_party_id out varchar2,
 i_party_site_id out varchar2,
 i_protocol_type out varchar2,
 i_protocol_address out varchar2,
 i_username out varchar2,
 i_password out varchar2,
 i_attribute1 out varchar2,
 i_attribute2 out varchar2,
 i_attribute3 out varchar2,
 i_attribute4 out varchar2,
 i_attribute5 out varchar2,
 retcode out pls_integer,
 retmsg out varchar2);
```

Description

Retrieves message envelope data using the internal control number maintained by the XML Gateway Execution Engine.

See XML Gateway Envelope, page 4-8 for information regarding the message envelope.

Arguments (input)

i_internal_control_number	Internal control number maintained by the XML Gateway execution engine associated with the inbound message.
----------------------------------	---

Arguments (output)

i_message_type	The message type is defaulted to "XML"
i_message_standard	The XML standard used for the business document received from the Trading Partner. The XML standard is entered using the Define XML Standards window and used in the Define Transactions window.
i_transaction_type	External transaction type associated with the business document received from the Trading Partner. The external transaction type is entered using the Define Transactions window and is used in the Define Trading Partners window.
i_transaction_subtype	External transaction subtype associated with the business document received from the Trading Partner. The external transaction subtype is entered using the Define Transactions window and is used in the Define Trading Partners window.
i_document_number	Unique identifier for the business document received from the Trading Partner. This field is not used by XML Gateway but is available for the Oracle E-Business Suite receiving application module.
i_party_id	Not Used.
i_party_site_id	The source Trading Partner Location Code entered using the Define Trading Partners window if no data is found in the Destination Trading Partner Location Code.
i_protocol_type	The transmission method entered using the Define Trading Partners window.
i_protocol_address	The address/URL associated with the transmission method. It is entered using the Define Trading Partners window.
i_username	The username entered using the Define Trading Partners window.
i_password	The password associated with the username. Entered using the Define Trading Partners window.
i_attribute1	User-defined field to pass data.

i_attribute2	User-defined field to pass data.
i_attribute3	Data in this field will trigger the creation of another XML message that is sent to the Trading Partner identified in the Destination Trading Partner Location Code field entered using the Define Trading Partners window.
i_attribute4	User-defined field to pass data.
i_attribute5	User-defined field to pass data.
retcode	Return code for the procedure
retmsg	Return message for the procedure

getOAGLOGICALID

PL/SQL Syntax

```
procedure getOAGLOGICALID
return varchar2;
```

Description

Gets the sender's logical ID defined for the ECX_OAG_LOGICALID system profile.

The value defined in the ECX_OAG_LOGICALID system profile is retrieved by the ECX_OAG_CONTROLAREA_TP_V view. The value is used to map to the OAG CONTROLAREA, LOGICALID element.

This function is not required if you are using the ECX_OAG_CONTROLAREA_TP_V view.

Arguments (input)

None

Arguments (output)

None

APIs Defined in ECX_ERRORLOG

getDoclogDetails

PL/SQL Syntax

```
procedure getDoclogDetails
```

```

(i_msgid in raw,
 i_message_type out varchar2,
 i_message_standard out varchar2,
 i_transaction_type out varchar2,
 i_transaction_subtype out varchar2,
 i_document_number out varchar2,
 i_party_id out varchar2,
 i_party_site_id out varchar2,
 i_protocol_type out varchar2,
 i_protocol_address out varchar2,
 i_username out varchar2,
 i_password out varchar2,
 i_attribute1 out varchar2,
 i_attribute2 out varchar2,
 i_attribute3 out varchar2,
 i_attribute4 out varchar2,
 i_attribute5 out varchar2,
 i_logfile out varchar2,
 i_internal_control_number out number,
 i_status out varchar2,
 i_time_stamp out date,
 i_direction out varchar2,
 o_retcode out pls_integer,
 o_retmsg out varchar2);

```

Description

Gets information about transactions processed by XML Gateway. An entry is written to the ECX_DOCLOGS table for each outbound message created by XML Gateway and each inbound message processed by XML Gateway.

Error recovery is performed using the stored copy of a message. For details regarding error recovery, see XML Gateway Error Processing Item Type, page 6-5.

Arguments (input)

i_msgid	Message identifier provided by the XML Gateway execution engine for each message processed.
----------------	---

Arguments (output)

i_message_type	The message type is defaulted to "XML"
i_message_standard	The XML standard associated with the business document as entered using the Define XML Standards window and used by the Define Trading Partners window.
i_transaction_type	External transaction type associated with the business document as entered using the Define Transactions window and used by the Define Trading Partners window.
i_transaction_subtype	External transaction subtype associated with the business document as entered using the Define Transactions window and used by the Define Trading Partners window.

i_document_number	Unique identifier from the Oracle E-Business Suite for the outbound business document. Unique identifier for an inbound business document received from the Trading Partner.
i_party_id	Trading Partner identifier associated with the business document as entered using the Define Trading Partners window.
i_party_site_id	Trading Partner site identifier associated with the business document as entered using the Define Trading Partners window.
i_protocol_type	Communication method associated with the business document as entered using the Define Trading Partners window.
i_protocol_address	Address/URL associated with the communication method as entered using the Define Trading Partners window.
i_username	The username associated with the Trading Partner as entered using the Define Trading Partners window.
i_password	The password associated with the username as entered using the Define Trading Partners window.
i_attribute1	User-defined field to pass data.
i_attribute2	User-defined field to pass data.
i_attribute3	The Destination Trading Partner Location Code as entered using the Define Trading Partners window.
i_attribute4	User-defined field to pass data.
i_attribute5	User-defined field to pass data.
i_logfile	Identifies the name of the log file created by the XML Gateway execution engine for the business document processed. The log files are written to the directory identified in the ECX: Log File Path system profile.
i_internal_control_number	The unique identifier as defined by the XML Gateway execution engine for each business document processed.
i_status	The status of the business document processed.

i_time_stamp	Time stamp from the XML Gateway execution engine representing when the outbound business document was created or when the inbound business document was processed.
i_direction	Identifies whether the business document was outbound from the Oracle E-Business Suite or inbound into the Oracle E-Business Suite.
o_retcode	Return code for the procedure.
o_retmsg	Return message for the procedure.

APIs Defined in ECX_ACTIONS

set_error_exit_program

PL/SQL Syntax

```
procedure set_error_exit_program
(i_err_type in pls_integer,
 i_err_code in pls_integer,
 i_err_msg in varchar2);
```

Description

Sets the error code and message so that a notification can be sent to either the Trading Partner contact, the System Administrator (identified in the ECX: System Administrator Email Address system profile), or both.

The Send Error Message action is used for warnings that do not require the process to be terminated. This procedure is used for serious errors where you intend to terminate the process.

For more details regarding the Send Error Message action, see Map Action Editor - Return Error Message: Send Error Message, page 2-104.

Arguments (input)

i_err_type	A code to identify the intended recipient of the error notification. The valid values are as follows: 10 = Reporting error, do not send notification 20 = Send notification to the Trading Partner contact containing the error code and message 25 = Send notification to both the Trading Partner and System Administrator contact containing the error code and message
-------------------	---

30 = Send notification to the System Administrator contact containing the error code and message

i_err_code	Code for error detected
i_err_msg	Message string for errors detected. Multiple messages may be concatenated into this variable and sent in the notification.

Arguments (output)

None.

APIs Defined in ECX_ATTACHMENT

register_attachment

PL/SQL Syntax

```
procedure register_attachment
(i_entity_name in varchar2,
 i_pk1_value in varchar2,
 i_pk2_value in varchar2,
 i_pk3_value in varchar2,
 i_pk4_value in varchar2,
 i_pk5_value in varchar2,
 i_file_id in number,
 i_data_type in number,
 x_cid out varchar2);
```

Description

Called by message maps for outbound documents to register the correlation of attachment(s) to an outbound business document.

This API assumes the attachment has been defined in the Oracle Foundation module. The required input to the API are values returned when the attachment was deposited in FND.

Arguments (input)

i_entity_name	Entity name from the FND_LOBS table used to compose the x_cid value.
i_pk1_value	Key value from the FND_LOBS table used to compose the x_cid value.
i_pk2_value	Key value from the FND_LOBS table used to compose the x_cid value.

i_pk3_value	Key value from the FND_LOBS table used to compose the x_cid value.
i_pk4_value	Key value from the FND_LOBS table used to compose the x_cid value.
i_pk5_value	Key value from the FND_LOBS table used to compose the x_cid value.
i_file_id	File identifier from the FND_LOBS table used to determine the file_name value from the FND_LOBS table. The file_name is used to compose the x_cid value.
i_data_type	Identifies the data type of the attachment file. The valid value is BLOB. Currently, only the ecx_attachment.embedded_lob_data_type value is supported (only

Arguments (output)

x_cid	A unique identifier for the attachment within a given outbound document provided by FND when the attachment was deposited. The value is constructed using the formulate_content_id API by concatenating i_entity_name, i_pk1_value, i_pk2_value, i_pk3_value, i_pk4_value, i_pk5_value (which are provided as input to this API), and then adding the file_name (based on the file_id) from the FND_LOBS table.
--------------	---

Note: register_attachment is an overloaded API. The second signature is documented below. The difference between the two APIs is that the first uses the key values from the FND_LOBS table to uniquely identify the attachment; and the second uses a user-defined identifier.

register_attachment

PL/SQL Syntax

```
procedure register_attachment
(i_cid in varchar2,
 i_file_id in number,
 i_data_type in number);
```

Arguments (input)

i_cid	A unique identifier for the attachment provided by the user when the attachment was defined in FND.
--------------	---

i_file_id	File identifier from the FND_LOBS table.
i_data_type	Identifies the data type of the attachment file. The valid value is BLOB. The list of valid values is maintained in <code>ecx_attachment.embedded_lob_data_type</code> API.

Arguments (output)

None.

retrieve_attachment

PL/SQL Syntax

```
procedure retrieve_attachment
(
  i_msgid in raw,
  x_cid in varchar2,
  x_file_name out varchar2,
  x_file_content_type out varchar2,
  x_file_data out nocopy blob,
  x_ora_charset out varchar2,
  x_file_format out varchar2);
```

Description

Called by message maps for inbound documents to retrieve an attachment deposited by Oracle Transport Agent (OTA) when the inbound document was received.

Not all attachments deposited by OTA are of interest to the receiving application. Only the attachments identified by the `i_msgid` and `x_cid` parameters are retrieved using this API.

Arguments (input)

i_msgid	The message ID associated with the attachment deposited by OTA into the FND repository.
x_cid	A unique identifier for the attachment provided in the inbound XML document. With OAG, this would be provided in the ATTCHREF data type, FILENAME element. The exact location of the unique identifier in the XML document will vary by standard.

Arguments (output)

x_file_name	Name of the attachment file from the FND_LOBS table.
x_file_content_type	Content type specified during the attachment uploading process. Information from the FND_LOBS table.

x_file_data	The uploaded attachment stored as a binary LOB from the FND_LOBS table.
x_ora_charset	Oracle character set from the FND_LOBS table.
x_file_format	File format ("text" or "binary") from the FND_LOBS table.

reconfig_attachment

PL/SQL Syntax

```
procedure reconfig_attachment
(i_msgid in raw,
 i_cid in varchar2,
 i_entity_name in varchar2,
 i_pk1_value in varchar2,
 i_pk2_value in varchar2,
 i_pk3_value in varchar2,
 i_pk4_value in varchar2,
 i_pk5_value in varchar2,
 i_program_app_id in number,
 i_program_id in number,
 i_request_id in number,
 x_document_id out number);
```

Description

Called by message maps for inbound documents to reset FND attributes for a previously retrieved attachment deposited by OTA when the inbound document was received.

Not all attachments retrieved using the retrieve_attachment API must be reconfigured. Use the reconfig_attachment API only if you want to update the FND attributes. Use the standard Oracle Foundation module to create a new copy of the attachment if necessary.

Arguments (input)

i_msgid	The message ID associated with the attachment deposited by OTA into the FND repository.
i_cid	A unique identifier for the attachment. This is the same value provided to the retrieve_attachment API that was based on the value in the inbound XML document.
i_entity_name	Entity name from the FND_ATTACHED_DOCUMENTS table.
i_pk1_value	Key value from the FND_ATTACHED_DOCUMENTS table.

i_pk2_value	Key value from the FND_ATTACHED_DOCUMENTS table.
i_pk3_value	Key value from the FND_ATTACHED_DOCUMENTS table.
i_pk4_value	Key value from the FND_ATTACHED_DOCUMENTS table.
i_pk5_value	Key value from the FND_ATTACHED_DOCUMENTS table.
i_program_app_id	Standard extended who column from FND_ATTACHED_DOCUMENTS table.
i_program_id	Standard extended who column from FND_ATTACHED_DOCUMENTS table.
i_request_id	Standard extended who column from FND_ATTACHED_DOCUMENTS table.

Arguments (output)

x_document_id	Unique identifier for the reconfigured attachment.
----------------------	--

formulate_content_id

PL/SQL Syntax

```
procedure formulate_content_id
(i_file_id in number,
 i_entity_name in varchar2,
 i_pk1_value in varchar2,
 i_pk2_value in varchar2,
 i_pk3_value in varchar2,
 i_pk4_value in varchar2,
 i_pk5_value in varchar2,
 x_cid out varchar2);
```

Description

Called by register_attachment API to determine the unique CID for an attachment deposited into FND. This API is exposed for use outside of the register_attachment API context.

Arguments (input)

i_file_id	File identifier from the FND_LOBS table used to determine the file_name value from the FND_LOBS table. The
------------------	--

	file_name is used to compose the x_cid value.
i_entity_name	Entity name from the FND_LOBS table used to compose the x_cid value.
i_pk1_value	Key value from the FND_LOBS table used to compose the x_cid value.
i_pk2_value	Key value from the FND_LOBS table used to compose the x_cid value.
i_pk3_value	Key value from the FND_LOBS table used to compose the x_cid value.
i_pk4_value	Key value from the FND_LOBS table used to compose the x_cid value.
i_pk5_value	Key value from the FND_LOBS table used to compose the x_cid value.
Arguments (output)	
x_cid	A unique identifier for the attachment based on the concatenation of i_entity_name, i_pk1_value, i_pk2_value, i_pk3_value, i_pk4_value, i_pk5_value (which are provided as input to this API), and then adding the file_name value (based on the file_id) from the FND_LOBS table.

APIs Defined in ECX_ENG_UTILS

convert_to_cxml_date

PL/SQL Syntax

```
procedure convert_to_cxml_date
(p_ora_date in date,
 x_cxml_date out varchar2);
```

Description

Converts Oracle date to cXML date in ISO 8601 format. The time element is not included.

If both date and time are required, use convert_to_cxml_datetime procedure.

Arguments (input)

<code>p_ora_date</code>	The Oracle date.
-------------------------	------------------

Arguments (output)

<code>x_cxml_date</code>	The cXML date converted from the Oracle date.
--------------------------	---

`convert_to_cxml_datetime`

PL/SQL Syntax

```
procedure convert_to_cxml_datetime
(p_ora_date in date,
 x_cxml_date out varchar2);
```

Description

Converts Oracle date and time to cXML date and time in ISO 8601 format.

If only the date is required, use `convert_to_cxml_date` procedure.

Important: Be sure to set the profile option ECX: Server Time Zone. If you do not set this profile option, the time zone defaults to GMT. See [Define System Profile Options](#), page 3-3.

Arguments (input)

<code>p_ora_date</code>	The Oracle date and time.
-------------------------	---------------------------

Arguments (output)

<code>x_cxml_date</code>	The cXML date and time converted from the Oracle date and time.
--------------------------	---

`convert_from_cxml_datetime`

PL/SQL Syntax

```
procedure convert_from_cxml_datetime
(p_cxml date in varchar2,
 x_ora_date out date);
```

Description

Converts cXML date and time to Oracle date and time.

Important: Be sure to set the profile option ECX: Server Time Zone. If you do not set this profile option, the time zone defaults to GMT. See Define System Profile Options, page 3-3.

Arguments (input)

<code>p_cxml_date</code>	The cXML date and time on an incoming document.
--------------------------	---

Arguments (output)

<code>x_ora_date</code>	The Oracle date and time converted from the cXML date and time.
-------------------------	---

Trading Partner Setup API

Oracle XML Gateway allows you to manage trading partner information through the use of the Trading Partner Setup API (ECX_TP_SETUP_API). This API contains the following procedures allowing you to create, update, retrieve, and delete trading partner details for one or more party sites in a party.

Note: You can also manage trading partner information through the Define Trading Partner Setup Form in Oracle XML Gateway. See: Using the Define Trading Partner Setup Form, page 3-20.

- CREATE_TRADING_PARTNER, page F-30
- DELETE_TRADING_PARTNER, page F-32
- UPDATE_TRADING_PARTNER, page F-33
- GET_TRADING_PARTNER, page F-35

Additional record types included in these procedures are explained in this appendix. See: Record Types, page F-36.

Procedure CREATE_TRADING_PARTNER

PL/SQL Syntax

```
procedure CREATE_TRADING_PARTNER(  
    x_return_status out number,  
    x_msg out varchar2,  
    x_return_status_list out t_return_status_type,  
    p_party_type in varchar2,  
    p_org_id in number,  
    p_org_name in varchar2,  
    p_party_id in number,  
    p_party_name in varchar2,  
    p_company_admin_email v  
    p_all_sites in varchar2 := 'N',  
    p_override_sites in varchar2 := 'N',  
    p_party_site_details in t_party_site_type,  
    p_tp_details in r_tp_details_record  
);
```

Description

Creates trading partner setup corresponding to the `party_site_id` passed or for all the sites in the party depending on the parameter passed.

First all the input data will be validated. If it is not valid, a warning message will be returned and no insertion will take place until the complete data passed is valid. For the creation of trading partner setup, at least one site should exist in the party.

Arguments (input)

<code>p_party_type</code>	Only one party type can be passed as an input data during the trading partner creation. Party type can be: • S (for supplier) • C (for customer) • B (for bank) • I (for internal location, such as warehouses) • CARRIER (for carrier)
<code>p_org_id</code>	Organization ID
<code>p_org_name</code>	Organization name Preference will be given to <code>p_org_id</code> passed as an in parameter.
<code>p_party_id</code>	Party ID

p_party_name	Party name Preference will be given to p_party_id passed as an in parameter.
p_company_admin_email	Email address
p_all_sites	It is a flag (Y, N) and the default value is set to N. If it is Y, trading partner setup will be created for all sites. Otherwise, it will be created for the specified site(s).
p_override_sites	It is a flag (Y, N) and the default value is set to N. If it is Y, the existing trading partner setup will be overwritten.
p_party_site_details	This list contains the party site details, transaction, code conversion, and user details. The information can be passed through the p_party_site_type record type. See: p_party_site_type, page F-36.
p_tp_detail	This list contains the transaction, code conversion, and user details. The information can be passed through the r_tp_details_record record type. See: r_tp_details_record, page F-38.

Arguments (output)

x_return_status	Return status of program execution
x_msg	Error message of program execution
x_return_status_list	List of arrays of all the Ids created for the trading partner setup. This information can be passed through the t_return_status_type record type. See: t_return_status_type, page F-38.

Procedure DELETE_TRADING_PARTNER

PL/SQL Syntax

```
procedure DELETE_TRADING_PARTNER(  
  x_return_status out pls_integer,  
  x_msg out varchar2,  
  x_return_status_list out t_return_status_type,  
  p_party_type in varchar2,  
  p_org_id in number,  
  p_org_name in varchar2,  
  p_party_id in number,  
  p_party_name in varchar2,  
  p_all_sites in varchar2 := 'N',  
  p_party_site_details in t_party_site_type,  
);
```

Description

Modifies the details part of the trading partner setup corresponding to the party_site_id passed or for all the sites in the party depending on the parameter passed.

First all the input data will be validated. If it is not valid, a warning message will be returned and no modification will take place until the complete data passed is valid. For the modification of trading partner setup, at least one site should exist in the party.

Arguments (input)

p_party_type	While trying to delete a trading partner, only one party type can be passed as an input data. Party type can be: • S (for supplier) • C (for customer) • B (for bank) • I (for internal location, such as warehouses) • CARRIER (for carrier)
p_org_id	Organization ID
p_org_name	Organization name Preference will be given to p_org_id passed as an in parameter.
p_party_id	Party ID
p_party_name	Party name

	Preference will be given to p_party_id passed as an in parameter.
p_all_sites	It is a flag (Y, N) and the default value is set to N. If it is Y, trading partner setup will be created for all sites. Otherwise, it will be created for the specified site(s).
p_party_site_details	This list contains the party site details, transaction, code conversion, and user details. The information can be passed through the p_party_site_type record type. See: p_party_site_type, page F-36.

Arguments (output)

x_return_status	Return status of program execution
x_msg	Error message of program execution
x_return_status_list	List of arrays of all the Ids created for the trading partner setup. This information can be passed through the t_return_status_type record type. See: t_return_status_type, page F-38.

Procedure UPDATE_TRADING_PARTNER

PL/SQL Syntax

```

procedure UPDATE_TRADING_PARTNER(
  x_return_status out number,
  x_msg out varchar2,
  x_return_status_list out t_return_status_type,
  p_party_type in varchar2,
  p_org_id in number,
  p_org_name in varchar2,
  p_party_id in number,
  p_party_name in varchar2,
  p_party_site_details in t_party_site_type,
);

```

Description

Modifies the details part of the trading partner setup corresponding to the **party_site_id** passed or for all the sites in the party depending on the parameter passed.

First all the input data will be validated. If it is not valid, a warning message will be returned and no modification will take place until the complete data passed is valid. For the modification of trading partner setup, at least one site should exist in the party.

Arguments (input)

p_party_type	Only one party type can be passed as an input data while updating the trading partner information. Party type can be: • S (for supplier) • C (for customer) • B (for bank) • I (for internal location, such as warehouses) • CARRIER (for carrier)
p_org_id	Organization ID
p_org_name	Organization name Preference will be given to p_org_id passed as an in parameter.
p_party_id	Party ID
p_party_name	Party name Preference will be given to p_party_id passed as an in parameter.
p_party_site_details	This list contains the party site details, transaction, code conversion, and user details. The information can be passed through the p_party_site_type record type. See: p_party_site_type, page F-36.

Arguments (output)

x_return_status	Return status of program execution
x_msg	Error message of program execution
x_return_status_list	List of arrays of all the Ids created for the trading partner setup. This information can be passed through the t_return_status_type record type. See: t_return_status_type, page F-38.

Procedure GET_TRADING_PARTNER

PL/SQL Syntax

```
procedure GET_TRADING_PARTNER(  
    x_return_status out number,  
    x_msg out varchar2,  
    x_return_status_list out t_return_status_type,  
    p_party_type in varchar2,  
    p_org_id in number,  
    p_org_name in varchar2,  
    p_party_id in number,  
    p_party_name in varchar2,  
    p_all_sites in varchar2 := 'N',  
    p_party_site_details in t_party_site_type,  
);
```

Description

Retrieves the trading partner setup details corresponding to the `party_site_id` passed or for all the sites in the party depending on the parameter passed. It returns transaction details, code conversion details, and user details corresponding to the sites provided.

First all the input data will be validated. If it is not valid, a warning message will be returned and no retrieval will take place until the complete data passed is valid. For the retrieval of trading partner setup, at least one site should have the trading partner setup information.

Arguments (input)

<code>p_party_type</code>	Only one party type can be returned while retrieving trading partner information. Party type can be: • S (for supplier) • C (for customer) • B (for bank) • I (for internal location, such as warehouses) • CARRIER (for carrier)
<code>p_org_id</code>	Organization ID
<code>p_org_name</code>	Organization name Preference will be given to <code>p_org_id</code> passed as an in parameter.
<code>p_party_id</code>	Party ID

p_party_name	Party name Preference will be given to P_PARTY_ID passed as an in parameter.
p_all_sites	It is a flag (Y, N) and the default value is set to N. If it is Y, trading partner setup will be created for all sites. Otherwise, it will be created for the specified site(s).
p_party_site_details	This list contains the party site details, transaction, code conversion, and user details. The information can be passed through the p_party_site_type record type. See: p_party_site_type, page F-36.

Arguments (output)

x_return_status	Return status of program execution
x_msg	Error message of program execution
x_return_status_list	List of arrays of all the Ids created for the trading partner setup. This information can be passed through the t_return_status_type record type. See: t_return_status_type, page F-38.

Record Types

This Trading Partner Setup API is a complex PL/SQL API, and each procedure contained in the API may include the following record types to return or pass the status of trading partner setup, or records of transaction details, code conversion, or user details.

- p_party_site_type, page F-36
- r_tp_details_record, page F-38
- t_return_status_type, page F-38
- t_transaction_type, page F-39
- t_xref_type, page F-42
- t_tp_user_data_type, page F-43

Record Type p_party_site_type

This p_party_site_type record type returns the party site details, transaction details,

code conversion, and user details.

PL/SQL Syntax

```
party_site_id number,  
site_address1 varchar2(100),  
site_address2 varchar2(100),  
site_address3 varchar2(100),  
site_address4 varchar2(100),  
site_city varchar2(100),  
site_state varchar2(100),  
site_postal_code varchar2(100),  
ece_tp_location_code varchar2(100),  
p_company_admin_email varchar2(30),  
tp_header_id number,  
transactions t_transaction_type, -- record type for transaction details  
code_conversion t_xref_type, -- record type for code conversion  
tp_user t_tp_user_data_type -- record type for user details
```

Arguments

party_site_id	Party site identifier
site_address1	Site address
site_address2	Site address
site_address3	Site address
site_address4	Site address
site_city	City information where the site is located.
site_state	State information where the site is located.
site_postal_code	Postal code information where the site is located.
ece_tp_location_code	Trading partner location code
p_company_admin_email	The email address of the administration contact associated with the trading partner
tp_header_id	Trading partner header identifier if the trading partner is created successfully.
transactions	Transaction details can be passed through the t_transaction_type record type. See: t_transaction_type, page F-39.
code_conversion	Code conversion can be passed through the t_xref_type record type. See: t_xref_type, page F-42.

tp_user	Code conversion can be passed through the t_tp_user_data_type record type. See: t_tp_user_data_type, page F-43.
----------------	---

Record Type r_tp_details_record

This r_tp_details_record record type returns the transaction and code conversion details.

PL/SQL Syntax

```
transactions t_transaction_type, --record type for transaction details
code_conversion t_xref_type      --record type for code conversion
```

Arguments

transactions	Transaction details can be passed through the t_transaction_type record type. See: t_transaction_type, page F-39.
code_conversion	Code conversion can be passed through the t_xref_type record type. See: t_xref_type, page F-42.

Record Type t_return_status_type

This t_return_status_type record type returns the status of trading partner setup, Id information associated with trading partner, party, code conversion, and transaction details, as well as error messages if occur during the process.

PL/SQL Syntax

```
org_id,
org_name,
party_type,
party_id number,
party_name ,
company_admin_email ,
party_site_id ,
site_address1 ,
x_return_status , -- Status code
x_msg           , -- error message /warning message
tp_header_id    , -- Header id if tp created successfully
tp_detail_id    , -- transaction details id
tp_ref_detail_id -- code conversion details id
```

Arguments

org_id	Organization ID
org_name	Organization name

party_type	Only one party type can be returned during the trading partner creation. Party type can be: • S (for supplier) • C (for customer) • B (for bank) • I (for internal location, such as warehouses) • CARRIER (for carrier)
party_id	Party ID
party_name	Party name Preference will be given to p_party_id passed as an in parameter.
company_admin_email	Email address
party_site_id	Party site identifier
site_address1	Site address
x_return_status	Status code
x_msg	An error message or a warning message
tp_header_id	Trading partner header ID if the trading partner is created successfully.
tp_detail_id	Transaction details ID
tp_ref_detail_id	Code conversion details ID

Record Type t_transaction_type

This t_transaction_type record type passes the record for trading partner transaction details.

PL/SQL Syntax

`r_transaction_record` -- Transaction details record.

```
transaction_type varchar2(100),
transaction_sub_type varchar2(100),
standard_code varchar2(30),
standard_type varchar2(10),
ext_transaction_type varchar2(100),
ext_transaction_sub_type varchar2(100),
direction varchar2(5),
map_code varchar2(100),
map_id number,
connection varchar2(100),
protocol_type varchar2(30),
username varchar2(100),
password varchar2(100),
protocol_address varchar2(200),
source_tp_loc_code varchar2(100),
dest_tp_loc_code varchar2(100),
document_confirmation NUMBER,
routing_id NUMBER,
update_password varchar2(1),
ext_process_id number,
transaction_id number,
--tp_header_id number,
tp_detail_id number,
action_type varchar2(20),
hub_user_id number,
hub_id number
```

Arguments

transaction_type	Transaction type is the standard product short code for the base Oracle E-Business Suite application. These values are defined in the Define Transactions form.
transaction_sub_type	Transaction subtype is a code for a particular transaction within the application specified by the Transaction Type.
standard_code	The standard code (for example, OAG) associated with the transaction passed to the procedure.
standard_type	The XML standard type associated with the transaction passed to the procedure.
ext_transaction_type	External transaction type associated with the internal transaction type passed to the procedure.
ext_transaction_sub_type	External transaction subtype associated with the internal transaction subtype passed to the procedure.
direction	The direction associated with the internal transaction type passed to the procedure. The value IN identifies an inbound message, and the value OUT identifies an outbound message.

map_code	Message map associated with the trading partner and transaction passed to the procedure.
map_id	Message map ID
connection	The connection method associated with the XML message sent to a trading partner.
protocol_type	The application protocol passed to the procedure to transmit the document.
username	Username passed to the procedure to be authenticated on the receiving server.
password	The password associated with the username passed for the receiving server.
protocol_address	The complete URL (including service/servlet) passed to the procedure to transmit the XML document.
source_tp_loc_code	The code passed to the XML Gateway envelope to identify the source trading partner of the message.
dest_tp_loc_code	The code passed to the procedure for the final recipient of the XML message.
document_confirmation	Document confirmation number passed to the procedure for the confirmation level if the associated trading partner would like to send or receive a confirmation.
routing_id	Routing ID passed to the procedure to identify another trading partner to whom messages will be routed.
update_password	The updated password passed to the procedure.
ext_process_id	External process ID passed to the procedure.
transaction_id	Transaction ID passed to the procedure.
tp_detail_id	Transaction details ID passed to the procedure.
action_type	The action type passed to the procedure.
hub_user_id	The user ID passed to the procedure for the trading partner conducting business via the hub.
hub_id	The hub ID passed to the procedure to route documents to and from trading partners.

Record Type `t_xref_type`

This `t_xref_type` record type passes the record for code conversion details.

PL/SQL Syntax

```
r_xref_record -- Code conversion details.  
  
p_xref_category_code varchar2(100),  
p_standard varchar2(100),  
p_oracle_value varchar2(100),  
p_from_tp_value varchar2(100),  
p_to_tp_value varchar2(100),  
p_standard_type varchar2(100),  
p_description varchar2(100),  
p_standard_id number,  
p_xref_category_id number,  
p_action varchar2(20) -- INSERT/ UPDATE/ DELETE  
);
```

Arguments

p_xref_category_code	The category code passed to the procedure to cross-reference the codes defined in Oracle E-Business Suite to codes used by trading partners, the XML standard, or other standard codes in the transactions.
p_standard	The standard code passed to associate with the message.
p_oracle_value	This is the code defined in Oracle E-Business Suite.
p_from_tp_value	This is the code in the message that represents data from the trading partner's perspective. Its value is passed in an inbound source transaction.
p_to_tp_value	This is the code passed for an outbound message. It represents data that the trading partner is expecting to receive.
p_standard_type	Standard type that is passed to this procedure.
p_description	Description of the XML standard that is passed to this procedure.
p_standard_id	Standard ID
p_xref_category_id	Category ID
p_action	INSERT, UPDATE, or DELETE action that is passed for this code conversion.

Record Type `t_tp_user_data_type`

This `t_tp_user_data_type` record type passes the record of user data details.

PL/SQL Syntax

```
r_tp_user_data_record -- User data record  
  
user_name varchar2(100),  
valid_user varchar2(100),  
p_action varchar2(20) -- INSERT/ UPDATE/ DELETE
```

Arguments

user_name	Oracle E-Business Suite user name to be associated with the trading partner.
valid_user	The valid user name associated with the trading partner.
p_action	INSERT, UPDATE, or DELETE action that is passed for the user data record.

Configuring OTA Client for TLS Authentication Using Oracle Wallet

Overview

This appendix describes the instructions on configuring OTA client for TLS authentication using Oracle Wallet. It includes the following topics:

- Configuring OTA Client for Server Authentication, page G-1
- Configuring OTA Client for Mutual Authentication, page G-2

Configuring OTA Client for Server Authentication

Perform the following steps to configure OTA client for server authentication:

1. Import the server's trusted CA certificate into the Wallet.

When the wallet is created, the most common CA certificates are included automatically. If required, import the trusted root-CA certificate that signed your server's certificate.

Refer to Step 5 in Section 5.2 Configure Inbound Connections, My Oracle Support Knowledge Document 1367293.1, *Enabling TLS in Oracle E-Business Suite Release 12.2*.

2. Configure the following TLS related AutoConfig variables:

Parameter Name	AutoConfig Variable	Default Value
javax.net.ssl.trustStore	s_ssl_truststore	<s_web_ssl_directory>/Apache/cwallet.sso

Parameter Name	AutoConfig Variable	Default Value
javax.net.ssl.trustStoreType	s_ssl_truststoretype	SSO
test.trustmanager.algorithm	s_ssl_trustmanageralgorithm	OracleX509

Note: The parameter names listed in the table are in the \$INST_TOP/appl/admin/oafm_wls.properties file.

Refer to the *Oracle E-Business Suite Setup Guide Release 12.2* for information on changing AutoConfig variables and executing AutoConfig in the application tier.

Ensure to stop and restart application tier services as part of AutoConfig execution.

Configuring OTA Client for Mutual Authentication

Perform the following steps to configure OTA client for mutual authentication using Oracle Wallet:

1. Follow steps described in Configuring OTA Client for Server Authentication, page G-1 to import the server's trusted CA into Oracle Wallet.
2. Configure Oracle E-Business Suite WebLogic Server for TLS.

For information on configuration details, refer to Section 5.2 Configure Inbound Connections, My Oracle Support Knowledge Document 1367293.1, *Enabling TLS in Oracle E-Business Suite Release 12.2*.

3. Configure the following TLS related parameters:

Parameter Name	AutoConfig Variable	Value
javax.net.ssl.trustStore	s_ssl_truststore	<s_web_ssl_directory>/Apache/cwallet.sso
javax.net.ssl.trustStoreType	s_ssl_truststoretype	SSO
test.trustmanager.algorithm	s_ssl_trustmanageralgorithm	OracleX509

Parameter Name	AutoConfig Variable	Value
javax.net.ssl.keyStore	s_ssl_keystore	<s_web_ssl_directory>/Apache/cwallet.sso
javax.net.ssl.keyStoreType	s_ssl_keystoretype	SSO
test.keymanager.algorithm	s_ssl_keymanageralgorithm	OracleX509

Note: The parameter names listed in the table are in the \$INST_TOP/appl/admin/oafm_wls.properties file.

Run AutoConfig in the application tier. Refer to the *Oracle E-Business Suite Setup Guide Release 12.2* for information on changing AutoConfig variables and executing AutoConfig in the application tier.

Ensure to stop and restart application tier services as part of AutoConfig execution.

Sizing and Performance Guidelines

Overview

This appendix contains sizing and performance tuning recommendations for Oracle XML Gateway. Performance sizing and performance recommendations are provided for Oracle Workflow listeners of Oracle E-Business Suite and the oafm managed server (memory parameters and java cache settings).

This appendix includes the following topics:

- Recommendations for Inbound Messages and Transactions
- Recommendations for Outbound Messages and Transactions

Please note that all the recommendations mentioned here are based on Performance Testing in internal lab environments. The actual numbers may have to be considered based on load in production instances.

Recommendations for Inbound Messages and Transactions

Recommendations for Improving the Performance

When multiple inbound transactions are submitted concurrently, you may see high elapsed time for a transaction. You can fine tune parameters controlling Oracle Workflow listeners to improve the performance of inbound transaction in such scenario.

The following recommendations are applicable when the approximate number of concurrent transactions is more than 1,000 with a payload size of 20KB or more. These changes increase CPU consumption on the server.

1. Increase the value of the parameter "Inbound Thread Count" from 1 to 5 for the following listeners:
 - ECX Inbound Agent Listener

- ECX Transaction Agent Listener
- Workflow Java Deferred Agent Listener

To do this, perform the following steps:

1. Log in to Oracle Applications Manager as a user who has the System Administrator responsibility.
 2. Navigate to OAM Setup from the navigation menu and select the **Site Map** link.
 3. Select the **Service Components** link under Workflow.
 4. Select the corresponding listener and click the **Edit** button.
 5. Click the **Next** button.
 6. Update the parameter value and click the **Finish** button.
 7. Restart the Agent Listener.
2. Decrease the value of the parameter "Processor Min Loop Sleep" from 120 seconds to 5 seconds for the following listeners:

Note: Ensure the parameter "Processor Min Loop Sleep" is set to the default value (5 seconds) for the Workflow Java Deferred Agent Listener.

- ECX Inbound Agent Listener
- ECX Transaction Agent Listener

To do this, perform the following steps:

1. Log in to Oracle Applications Manager as a user who has the System Administrator responsibility.
2. Navigate to OAM Setup from the navigation menu and select the **Site Map** link.
3. Select the **Service Components** link under Workflow.
4. Select the corresponding listener and click the **Edit** button.
5. Click the **Next** button.

6. Update the parameter value and click the **Finish** button.
7. Restart the Agent Listener.

Refer to the *Oracle Workflow Administrator's Guide* for more information.

Resolving an Out of Memory Exception

When multiple inbound transactions are submitted concurrently, an out of memory exception may occur at two different levels during the processing of inbound transactions. Initially an out of memory exception may occur at the oafm managed server level even if it has significant amount of default JVM size, such as 1 GB, which is sufficient to handle a payload size of up to 50MB. Consequently, an out of memory exception may occur at the Workflow Java Deferred Agent Listener level when the agent listener is contained within the Generic Service Component Container, which has sufficient default JVM size, such as 384MB, to handle a payload size of up to 50MB.

These recommendations are applicable when an out of memory exception is encountered with a payload size more than 50MB.

Note: The default JVM size of the oafm managed server may cause an out of memory exception for payloads larger than 50MB.

Recommendations

The following table shows the maximum JVM size required for the oafm managed server and Workflow Java Deferred Agent Listener for a particular size of payload with a single transaction to avoid an out of memory exception. The Workflow Java Deferred Agent Listener is contained within the Generic Service Component Container; therefore, we must increase its JVM size to increase the listener's JVM size.

Payload Size	OAFM Server JVM Size	Generic Service Component Container JVM Size	Default or Recommended Values?
Up to 50MB	"-Xmx1024"	"-Xmx384m"	default
51MB-150MB	"-Xmx2048m"	"-Xmx1536m"	recommended
200MB	"-Xmx3072m"	"-Xmx2048m"	recommended

Perform the following steps to change JVM sizes:

- For the oafm manager server:
 1. Log in to the WebLogic Server Administration Console.

2. Click **Servers** and then <oafm-server-name>.
 3. Click the "Server Start" tab.
 4. Click the **Lock & Edit** button on the left top corner.
 5. Change the JVM size in the "Arguments:" text box, from "-Xms512m -Xmx512m" to "-XmsNm -XmxNm".

Please note that "N" determines the size to be, for example, 2048, 3072, and so on.
 6. Click the **Save** button.
 7. Click **Release Configuration**.
 8. Restart the oafm managed server.
- For the Generic Service Component Container:
 1. Log in to Oracle Applications Manager as a user who has the System Administrator responsibility.
 2. Navigate to **Overview** and then **AutoConfig**.
Click **Edit Parameters** under Applications.
 3. Select the Environments tab.
 4. Expand "oa_environment:adovars".
 5. Change the value of parameter "AFJSMARG" and set the required size after "-server" option, for example, to set size to 3584m:
`-server -Xmx3584m -Doracle.apps.fnd.common.Pool.leak.mode=stderr:off -verbose:gc`

Recommendations for Outbound Messages and Transactions

Similar to inbound transactions, when multiple outbound transactions are submitted concurrently, the elapsed time of transactions escalates with an increase in number of transactions. To improve the performance of outbound transactions, fine tune the outbound parameter settings and the parameters controlling Oracle Workflow listeners.

Recommendations

The following recommendations are applicable when the approximate number of concurrent transactions is more than 1,000 with a payload size 100KB or more. These changes increase CPU consumption on the server.

1. Decrease the "OXTAOutDequeueRate" parameter time from 12000 msec (2 minutes) to 1000 msec (1 second)

For instructions on how to change this value, refer to My Oracle Support Knowledge Document 829449.1, *Preserve System Properties Customized For Your ECX (XML Gateway)*.

2. Increase the "OXTAOutThreads" parameter value from 1 to 2

For instructions on how to change this value, refer to My Oracle Support Knowledge Document 829449.1, *Preserve System Properties Customized For Your ECX (XML Gateway)*.

3. Decrease the value of parameter "Processor Min Loop Sleep" for Workflow Deferred Agent Listener from 120 seconds to 2 seconds

To do this, perform the following steps:

1. Log in to Oracle Applications Manager as a user who has the System Administrator responsibility.
2. Navigate to OAM Setup from the navigation menu and select the **Site Map** link.
3. Select the **Service Components** link under Workflow.
4. Select the corresponding listener and click the **Edit** button.
5. Click the **Next** button.
6. Update the parameter value and click the **Finish** button.
7. Restart the Agent Listener.

4. Increase the value of parameter "Inbound Thread Count" for the Workflow Deferred Agent Listener from 1 to 5 for a map size up to 100KB and from 1 to 8 for a map size greater than 100KB.

To do this, perform the following steps:

1. Log in to Oracle Applications Manager as a user who has the System Administrator responsibility.
2. Navigate to OAM Setup from the navigation menu and select the **Site Map** link.
3. Select the **Service Components** link under Workflow.
4. Select the corresponding listener and click the **Edit** button.

5. Click the **Next** button.
 6. Update the parameter value and click the **Finish** button.
 7. Restart the Agent Listener.
-
5. If the destination server is Oracle E-Business Suite, then it should have the optimal settings mentioned in the previous section for inbound transactions.

Troubleshooting

Troubleshooting Your XML Gateway Installation

This section presents the following troubleshooting topics:

- Automated Troubleshooting Script, page I-1
- Transaction Monitor, page I-8
- Manual Troubleshooting Steps, page I-15
- XML Gateway Version Validation, page I-44
- Common Client Authentication Implementation Issues, page I-47
- Oracle Diagnostic Tests, page I-47

Automated Troubleshooting Script

Use the `ecxver.sql` script to verify the health of your Oracle XML Gateway installation. It can be run at any time. Outlined below are instructions for executing the script and interpreting the output.

How to Run the Automated Test Script

Log in to your database applications account and execute the following:

```
SQL> @$ECX_TOP/patch/115/sql/ecxver.sql
```

The output of the script will appear on your screen.

Sample Output of the Automated Test Script

Component	OBJECT_NAME	LOCKED_MODE
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
OTA	ECX_OUTQUEUE	Normal
WEBMETHODS	ECX_OUTQUEUE	Normal
WEBMETHODS	ECX_OUTQUEUE	Normal
WEBMETHODS	ECX_OUTQUEUE	Normal
WEBMETHODS	ECX_OUTQUEUE	Normal
WEBMETHODS	ECX_OUTQUEUE	Normal

```

ECX_UTL_XSLT_DIR Profile   : /u01/R122_EBS/temp
ECX_OAG_LOGICALID Profile : ORACLE
ECX_SERVER_TIMEZONE Profile: (GMT -08:00) Pacific Time
ECX_SYS_ADMIN_EMAIL Profile: admin.email@example.com
ECX_XML_VALIDATE_FLAG Profile: Y
ECX_XML_MAXIMUM_SIZE Profile : 20000000
ECX_UTL_XSLT_DIR Directory Path   : /u01/R122_EBS/temp
Oracle XML Developer's Kit 12.1.0.2.0 - Production
Parser Version OK

```

```

-----
XML Gateway Status Summary
-----

```

XML Parser Version	OK
All ECX Objects Valid?	OK
All XML Parser Objects Valid?	OK
webMethods Running?	OK
OTA Running?	OK
Total Messages on Outbound Queue	6
OTA Msgs on Outbound Queue	4
webMethods Msgs on Outbound Queue	1
Messages on Inbound Queue	8

```

-----
End of Summary
-----

```

How to Interpret the Output Generated by the Automated Test Script

The following table explains each line of the example output shown above. Some of the output is informational only and does not require further action.

Output	Explanation/Troubleshooting Tip
Transport Software	The "LOCKED_MODE" value should be "Normal", but may appear as "DEADLOCK". Deadlocks are normal if they remain for a short period of time. If a deadlock persists for more than 10 minutes, there may be a serious problem. Note: webMethods is used in Oracle Exchange environments only. It is not used in ERP environments

Output	Explanation/Troubleshooting Tip
System Profiles: ECX_UTL_XSLT_DIR ECX_OAG_LOGICALID ECX_SERVER_TIMEZONE ECX_SYS_ADMIN_EMAIL ECX_UTL_LOG_DIR ECX_XML_MAXIMUM_SIZE ECX_XML_VALIDATE_FLAG	The value associated with each XML Gateway System Profile is displayed for review.
ERROR: Parser Version Wrong	The XML Parser version associated with the environment is displayed. XML Parser version 2.0.2.9 or later is required.
ERROR: Some Invalid ECX Objects	Informational message indicating some invalid ECX database objects were identified.
Log Profile/database directory for file I/O	If value displayed is anything other than "OK", verify the following: The physical directories associated with the ECX_UTL_XSLT_DIR and ECX_UTL_LOG_DIR system profiles must be defined database directory for file I/O. See: Define Database Directories, page 3-8.
XML Parser Version	If value displayed is "FAIL", upgrade XML Parser to version 2.0.2.9 or later. The XML Parser version mentioned here is from the java component. This value is used to find out only whether the previous version is installed in the environment. The versions of plxml parser (plsql component of XML Parser) and corresponding PL/SQL wrappers created by load.sql could not be determined. Therefore, please check for any invalid XML Parser objects and invalid ECX objects. Oracle XML Gateway will not function properly, if the installation of XML Parser is not correct. If you get "ORA-00932: inconsistent datatypes" error, then you need to reinstall XML Parser (java component and PL/SQL component) in the environment. Only XML Parser 2.0.2.9 is certified by Oracle E-Business Suite.
All ECX Objects Valid?	If value displayed is "NO", the invalid objects must be recompiled. Recompile the invalid objects or reapply the XML Gateway patch to resolve the invalid ECX database objects.
All XML Parser Objects Valid?	If value displayed is "NO", the XML Parser must be reinstalled. Reinstall the XML Parser to resolve the invalid XML Parser objects.

Output	Explanation/Troubleshooting Tip
webMethods Running?	If value displayed is anything other than "OK", restart webMethods. For an Oracle-hosted environment: contact Operations For self-hosted environments: contact webMethods Note: webMethods is used in Oracle Exchange environments only. It is not used in ERP environments.
OTA Running?	If value displayed is anything other than "OK", restart Oracle Transport Agent (OTA). Follow instructions in OTA patch to start the Apache web server. Ensure the required JSERV properties are set.
Total Messages on Outbound Queue	Number reported represents the number of messages on the outbound queue awaiting processing by OTA or webMethods. If number is unusually high, check the individual totals in the OTA and webMethods queues.
OTA Messages on Outbound Queue	Number reported represents the number of messages on the outbound queue awaiting processing by OTA. This is a subset of the total. If number is unusually high, ensure OTA is running.
webMethods Messages on Outbound Queue	Number reported represents the number of messages on the outbound queue awaiting processing by webMethods. This is a subset of the total. If number is unusually high, ensure webMethods is running.
Messages on Inbound Queue	Number of messages on inbound queue (informational). As long as the agent for the inbound queue is enabled, the agent listener is running, and OTA/webMethods is running, these messages will be processed. See Start Agent Listeners, page 6-62 for information on enabling agents and starting agent listeners.

Additional Information: There are some common mistakes while using webMethods in Oracle Exchange 6.2.2 environment:

- Shutdown webMethods first before shutting down the Oracle Exchange database and start webMethods after starting up the database. Otherwise messages will not be delivered because of stale database connections.
- Set NLS_LANG for webMethods to AMERICAN_AMERICA.UTF8.
- Values for dbUser, dbPass and dbinstance have not been updated in the exchange_config.prop file.
- Packages have been copied in the replicate/inbound directory but not loaded into webMethods. Login into the webMethods and

verify the version of packages (Exchange.zip 115.14 and OracleExchange.zip 115.14).

- If you see an error message like following on the TxnMonitor.jsp - 'Job Failed: [Tx] Expired transaction', it indicates that either URL provided by user is not correct or the customer's webMethods Server is down.
- Any 'Guranteed Delivery disabled for webMethods' error should be resolved by the standard instructions given by webMethods support to reset the guranteed delivery. The error might be due to wrong version of webMethods.

In Oracle Exchange 6.2.2 environment, you will not receive XML messages if transactions are not set up. However, your system administrator should get an email indicating this error.

Verifying the Installation

Before running the `patch/115/sql/ECXTEST.sql` script to test Oracle XML Gateway, you need to run the `patch/115/sql/ECXLGINI.sql` script which does the necessary initialization for fnd logging for that particular sql session first. This step is necessary even if you have Enabled FND Logging for ECX from configuration screens.

Execute the following SQL program as the application user in SQL*Plus to test Oracle XML Gateway:

```
SQL> $ECX_TOP/patch/115/sql/ECXTEST.sql
```

This creates an XML file and log file in the FND_LOG infrastructure (the `fnd_log_attachment` table).

Testing Oracle Transport Agent

The `ECX0TAPing.html` file included with the Oracle Transport Agent (OTA) allows you to test the OTA installation and configuration by sending a "ping" to it. This ping only tests whether the OTA server is running and validates the username/password.

To use the `ECX0TAPing.html`, open a web browser and enter the following URL:

```
http://<EBS_hostname>:<port>/OA_HTML/US/ECX0TAPing.htm
```

A web page will appear prompting you for the username and password. You must enter a valid applications username and password. Click the Ping button and you should see a status HTML page in your browser. If it is successful, the web page will look like the following:

```
STATUS_CODE 1000
```

```
STATUS_DESCRIPTION OK
```

MESSAGE_RECEIPT_ID

Note: If you received an error response (anything other than STATUS_CODE 1000), check the Apache error log file in the `Apache_top/Apache/logs` directory for more specific information on the error.

Oracle Transport Agent Inbound Test presents an HTML page that lets you provide inbound payload and other OTA parameters. To access this test page, open a web browser and enter the following URL:

`http://<EBS_hostname>:<port>/webservices/ECX0TAInbound`

Note: Oracle Transport Agent Inbound Test Page is included for testing purposes and should not be used in a production environment. OTA messaging protocol parameters used in this test page are described in the Oracle Transport Agent Inbound Test Page Parameters table.

Important: As part of the January 2017 CPU update, the access to Oracle Transport Agent Inbound Test page is restricted to trusted nodes or servers. Therefore, to use this page to send an inbound XML message, you should add a fully-qualified node name or IP address as a trusted node in Oracle E-Business Suite. To extend access to additional hosts, perform the following tasks:

1. Add space-separated node names, fully qualified node names, or client machine IP addresses to the existing content of the `s_admin_ui_access_nodes` context variable.
2. Run AutoConfig.
3. Restart the Oracle HTTP Server.

For more information, refer to the Known Issues section in My Oracle Support Knowledge Document 2212223.1, *Oracle E-Business Suite Release 12 Critical Patch Update Knowledge Document (January 2017)* for details.

A web page will appear prompting you for the input parameters used by the OTA messaging protocol. Refer to the Oracle Transport Agent chapter for information on OTA messaging protocol parameters.

Error: "java.lang.IllegalArgumentException: port out range:-1" in Apache error logs

When sending any document through Oracle Transport Agent without explicitly mentioning the port number (even for default ports 80 and 443), this error is reported in the Apache error log file. After this error, OTA retries to send the document indicated

by OXTAOutMaxAttempts parameter. Please specify the port number explicitly for the Outbound documents.

All the activities by Oracle Transport Agent is recorded in the ECX_OXTA_LOGMSG table. Please query the table to find more information about OTA errors.

Transaction Monitor

The Transaction Monitor is a tool for monitoring the status of inbound and outbound transactions originating from and going into the Oracle E-Business Suite that have been processed by the XML Gateway and delivered or received by the Oracle Transport Agent. The Transaction Monitor shows a complete history and audit trail of these documents. You can also use the Transaction Monitor to resend an outbound document, if necessary.

Navigate to the Transaction Monitor page using the Workflow Administrator Web (New) responsibility.

The Transaction Monitor provides the following:

- Flexible search criteria to support access to a specific document or group of documents
- Search results at the document header level with drill down by document ID
- Resend capability for outbound messages
- Viewing capability of the XML message content

Transaction Monitor Search Page

Use the Transaction Monitor Search page to enter search criteria for a specific inbound or outbound document, or group of documents.

The LOVs contain the valid search values and are defined under the Lookup Type FND_TX_MONITOR_STATUS. The search page allows you to search on the following criteria:

Inbound Messages

Select this radio button to search for inbound messages, and then select a Processing Status.

Processing Status

Select the processing status of the inbound message(s) for which you want to search. Valid values are All, Pending, Warning, Error, and Success.

Outbound Messages

Select this radio button to search for outbound messages, and then select the Generation Status, Delivery Status, and Retry Status.

Generation Status

Select the generation status of the outbound message(s) for which you want to search. Valid values are All, Pending, Warning, Error, and Success.

Note: A Generation Status of "Success" indicates that the message was generated and enqueued.

Delivery Status

Select the delivery status of the outbound message(s) for which you want to search. Valid values are All, Pending, Warning, Error, and Success.

Note: A Delivery Status of "Success" indicates that the message was delivered successfully.

Retry Status

Select the retry status of the outbound message(s) for which you want to search. Valid values are All, Pending, Warning, Error, and Success.

Transaction Type

The transaction type is the product short name for the base Oracle E-Business Suite application associated with the transaction. Select a value from the Search and Select window.

Transaction Subtype

The transaction subtype is a code relating to the transaction type. Select a value from the Search and Select window.

Source TP Location Code

The source TP location code is the source trading partner of the message. Select a value from the Search and Select window.

Trading Partner Name

Select the trading partner name from the Search and Select window.

Document ID

Enter the document ID provided by Oracle E-Business Suite.

Site Name

Select the site name associated with the Trading Partner from the Search and Select window.

Party Type

Select the party type from the LOV. To search for all party types, select the blank LOV option.

From Date

Select the from date using the calendar icon, or enter a date in the format dd-mm-yyyy:hh:mm:ss (Example: 23-10-2012).

To Date

Select the to date using the calendar icon, or enter a date in the format dd-mm-yyyy:hh:mm:ss (Example: 23-10-2012). The to date must be equal to or after the from date. If the to date is the same as the from date, you are specifying one 24-hour period.

Calling the Transaction Monitor Search Page from other applications

Note: The Transaction Monitor Search page is a callable URL from any environment. Call the page by using the following URL and parameters:

```
OA.jsp?  
OAFunc=TXMONITORRESULTSPG&addBreadCrumb=Y&Direction=  
<value_direction>&...(other parameter-value pairs)
```

<value_direction> should be IN or OUT

addBreadCrumb=Y is added to maintain the breadcrumbs trail

Additional parameters that can be used to call the Transaction Monitor are:

ProcessingStatus (with Direction=IN)

GenerationStatus (with Direction=OUT)

DeliveryStatus (with Direction=OUT)

RetryStatus (with Direction=OUT)

PartyType (lookup values CARRIER, S, I, B, E, C)

TransactionType
TxSubtype
LocCode
TPName (must be submitted with PartyType)
DocumentId
SiteName (must be submitted with PartyType)
From (in format DD-MON-YYYY HH:MI:SS)
To (in format DD-MON-YYYY HH:MI:SS)

Search Results Page

The Transaction Monitor will return the Inbound Search Results page or the Outbound Search Results page depending on your selection. The document ID drills down to a details page from which you can view the XML message. For outbound documents, the Search Results page allows you to resend a document.

Inbound Search Results Page

Document ID

The document ID of the document being processed, such as the PO number or the Invoice number. This field drills down to the Message Details page.

Trading Partner Name

The trading partner name. A value of "N/A" indicates that the XML Gateway Trading Partner Details have not been synchronized with the Workflow Directory Services. For more information, see: Setting up an Oracle Workflow Directory Service in the *Oracle Workflow Administrator's Guide*.

External Transaction Type

The external transaction type is the primary external identifier for the XML message (for example, OAG noun). It is associated with the internal transaction type and transaction subtype.

External Transaction Subtype

The external transaction subtype is a code relating to the transaction subtype (for example, OAG verb). It is the secondary identifier for the XML message.

Note: The combination of Transaction Type, Transaction Subtype, and the External Transaction Type and External Transaction Subtype

identifies an Oracle transaction with which to associate this message.

Processing Time Stamp

The date and time the document was processed.

Processing Status

The processing status of an inbound message. Valid values are Pending, Warning, Error, and Success.

Outbound Search Results Page

The Outbound Search Results page allows you to resend a document. Select the document(s) and click the Resend button. This action will not recreate the document, but resends the document from information stored in the `ecx_doclogs` table.

Note: The Select option is disabled for transactions that are A2A (because no message ID is created), and for B2B transactions for which generation failed.

Document ID

The document ID drills down to the message detail screen.

Trading Partner Name

The trading partner name. A value of N/A indicates that the XML Gateway Trading Partner Details have not been synchronized with Workflow Directory Services.

Internal Transaction Type

The transaction type is the product short name for the base Oracle E-Business Suite application associated with the transaction.

Internal Transaction Subtype

The transaction subtype is a code relating to the transaction type.

Generation Date

The date and time the message was generated.

Generation Status

Possible values of the generation status are: Pending, Warning, Error, and Success.

Note: A Generation Status of "Success" indicates that the message was generated and enqueued.

Delivery Date

The date and time the message was delivered.

Delivery Status

Possible values of delivery status are: Pending, Warning, Error, and Success.

Note: A Delivery Status of "Success" indicates that the message was delivered successfully.

Retry Date

The "retry" date of the message. The Retry Status column is updated if Resend was initiated from the search detail window.

Retry Status

Possible values are: Pending, Warning, Error, and Success.

Resend (button)

Use this button to resend selected documents from information stored in the ecx_doclogs table. This function will not recreate the document. You can resend a single document, or select multiple documents to resend.

Transaction Monitor Details Page

The details screen is displayed when you drill down on the Document ID field.

Inbound Message Details

The Inbound Message Details Screen displays information regarding the selected document. Use the View XML button to view the XML document.

Fields displayed that are not shown on the Inbound Search Results screen, page I-11 are:

Internal Control Number

The unique number generated by the execution engine.

Party ID

The Trading Partner identifier.

Party Type

The type of Party, such as Customer, Supplier, or Bank.

Site Name

The trading partner's site name.

Processing Message

A message regarding the status of the transaction. Refer to Manual Troubleshooting, page I-15 for information on how to interpret messages and resolve reported errors.

Processing Logfile

The name and location of the processing log file. Use it to trace the process flow and identify errors.

View XML (button)

Clicking this button displays the XML message in the View XML page, page I-15.

Outbound Message Details

Click on the Document ID to drill down to the Transaction Monitor Details page.

Fields displayed that are not shown on the Outbound Search Results screen, page I-12 are:

Party ID

The identifier of the Trading Partner you are doing business with.

Site Name

The site name associated with the Trading Partner.

URL Sent To

The URL the document was sent to.

Generation Status

Valid values are Success, Error, Pending, or Warning.

Generation Message

The message returned by the system regarding the status of the transaction. For information regarding the interpretation of this message, see Manual Troubleshooting Steps, page I-15.

Generation Logfile

The name and location of the logfile. If this is a failed transaction, see Manual

Troubleshooting Steps, page I-15.

Delivery Message

The message returned by the system regarding the status of the delivery.

Retry Message

The message returned by the system regarding the status of the retry message.

View XML (button)

Select this button to display the XML message in the View XML, page I-15 page. This button is enabled when the Generation Status is "Success".

View XML

This page displays the XML document.

Note: The information in the "View XML" screen sample shown below was taken from the ecx_doclogs table on the basis of the out message ID stored in the view.

Manual Troubleshooting Steps

If this is a new install or a new upgrade and you are experiencing errors, run the verification script included in the XML Gateway patch to verify that all the necessary components of the XML Gateway solution are installed correctly. Refer to Automated Troubleshooting Script, page I-1 for details on how to execute the troubleshooting script, ecxver.sql.

If the troubleshooting script did not reveal any install or upgrade errors, or if the error occurs in the processing stage, perform the following manual troubleshooting procedures.

There are three types of error messages generated by Oracle XML Gateway:

- Engine-level, page I-16 - error messages generated by the XML Gateway execution engine.
- API-level, page I-36 - error messages generated by APIs.
- HTTP Response Codes, page 7-35 - error response codes returned in the HTTP Response by the OTA server

XML Gateway Engine-Level Messages

The following is a list of the seeded error messages that may be included in a notification to the Trading Partner or System Administrator. The same error message may also appear in the process log file depending on the trace level activated. Along with each error message is an explanation of possible causes and recommended corrective actions.

The list is organized by functional area and error type for easy reference. The list contains the following columns:

- Error Type
- Error Code
- Message Code
- Message
- Corrective Action

The Error Type identifies the target recipient of a notification. The target recipient is predetermined and is based on which party is most able to resolve the error. In a few instances, a notification is sent to both the Trading Partner and System Administrator so that the two parties may collaborate to resolve the error. The valid Error Types are listed in the following table:

Error Type	Description
10	No notification
20	Notification sent to Trading Partner
25	Notification sent to both Trading Partner and System Administrator
30	Notification sent to System Administrator

The Error Code identifies the status of the process. The valid Error Codes are shown in the following table:

Error Code	Description
10	In Process

Error Code	Description
0	Success
1	Warning
2	Error

Because the focus of this section is on troubleshooting, Error Type 10 (No Notification) and Error Code 0 (Success) are excluded from the list. Error Code 10 (In Process) is included to support troubleshooting, although this group does not represent an error. The engine will abort and roll back any process that does not have a status of "Success."

The Message Code is a code related to the message text string. This is used for quick identification of an error.

There are several error messages that can originate from multiple sources. The Message Code and Message string may be the same, but the Error Type and Error Code will be different. Ensure not to regard these as duplicates.

There are several categories of corrective action identified in the troubleshooting guide. The following table summarizes the categories and lists the corresponding reference section in this book:

Corrective Action Category	Reference Section
Verify lookup data	XML Gateway Setup chapter: Define Lookup Values, page 3-17
Verify development setup data	XML Gateway Setup chapter: Define XML Standards, page 3-10 Define Transactions, page 3-11
Verify implementation setup data	XML Gateway Setup chapter: Define System Profile Options, page 3-3 Define Trading Partner, page 3-19 Define Code Conversion, page 3-48
Verify queues and agent listeners are enabled	Integrating XML Gateway with Workflow Business Event System chapter: Manage Workflow Processes, page 6-59 Monitor Workflow Processes, page 6-62

Corrective Action Category	Reference Section
Enable trace or configure event subscriptions	Integrating XML Gateway with Workflow Business Event System chapter: Manage Workflow Processes, page 6-59 Monitor Workflow Processes, page 6-62
Modify a message map: Action Definition	Message Designer chapter (see note following table): Reference the specific action under Transaction Map - Actions, page 2-65
Modify a message map: Root Element	Message Designer chapter (see note following table): File > Properties menu option, page 2-5
Load/Delete a map or DTD	Message Designer chapter (see note following table): How to Load/Delete Message Maps and DTDs, page 2-111
Verify Confirmation Message setup	Execution Engine chapter: How to Implement a Confirmation Message, page 4-17
Enter a bug	Work with Oracle World Wide Support to enter a bug. Include log file, message map, (.xgm file), and the associated DTD in the bug report.
Enter an enhancement request	Work with Oracle World Wide Support to enter an enhancement request. Include business justification and your specific scenario. Be as specific as possible.

Note: Updated maps must be loaded into the XML Gateway repository.

Changes in the relationship between a map (.xgm) and the corresponding DTD may require the map, the DTD, or both to be loaded into the XML Gateway repository.

UTL_FILE Errors:

Error Type	Error Code	Message Code	Message	Corrective Action
30	1	ECX_UTL_INVALID_OPERATION	UTL FILE Error: The file could not be opened or operated on as requested.	Verify that the file is read and write enabled. Use commands appropriate for your operating system to change protections if necessary.
30	1	ECX_UTL_INVALID_PATH	UTL FILE Error: File location or file name was invalid.	Verify that the directory identified by the profile option exists. Verify that the file name exists. Change the directory or file name if necessary.
30	1	ECX_UTL_WRITE_ERROR	UTL FILE Error: An operating system error occurred during the write operation.	Verify the setting for the directory identified by the profile option. Confirm that the database directory for file I/O has been properly defined. See: Define Database Directories, page 3-8.

Trading Partner Errors:

Error Type	Error Code	Error Message Code	Message	Corrective Action
25	1	ECX_NO_UNIQUE_TP_SETUP	Could not resolve a Unique/Destination Configuration for this Partner.	The inbound message envelope contained a Trading Partner ID that was not uniquely defined in XML Gateway. This occurs when the wrong ID was sent or the Trading Partner has not been defined in XML Gateway. Use the window to view/add the Trading Partner profile.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_ADDR_DERIVATION_ERR	Unable to derive internal address ID for address type: &p_address_type and location code: &p_location_code.	Cannot successfully execute Derive Address ID from Location Code action. This occurs when the address type or the location code is invalid. Check the section of the Message Designer chapter for a list of supported address types. Verify that the location code is valid.
30	1	ECX_DELIVERY_TP_NOT_SETUP	Unable to determine the trading partner setup for Transaction type: &p_transaction_type, Transaction subtype: &p_transaction_subtype, Party site id: &p_party_site_id, Party id: &p_party_id, and Party type: &p_party_type.	Cannot determine Trading Partner with token values for Party Type, Party ID, Party Site ID, Transaction Type, and Transaction Subtype. Use the window/API to view/update the Trading Partner setup data.
30	1	ECX_DESTINATION_ADDRESS_NULL	Protocol Address missing for party id: &p_party_site_id and location code: &p_source_code.	The Protocol Address is null for the given Trading Partner (Party Site ID) and Source Location Code. Use the window to view/update the Trading Partner's protocol address.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_DYN_ROUTING_NOT_ENABLED	&p_ext_type and &p_ext_subtype Not enabled for &p_party_ext_code in Oracle XML Gateway Server for Dynamic Routing.	The inbound transaction associated with the given external transaction type, transaction subtype, and source location code is not enabled for dynamic routing. Ensure that the "attribute3" variable in the message envelope contains a valid value. Use the window to view/update the Trading Partner Routing information representing the route to the Trading Partner.
30	1	ECX_INVALID_PARTY_TYPE	Party Type: &p_party_type is invalid.	Review seeded Party Type lookup and use a valid party type code. Enter an Enhancement Request for new Party Type if required.
30	1	ECX_INVALID_TP_HDR_ID	Tp_header_id: &p_tp_header_id is invalid.	Invalid Trading Partner ID. Use the retrieve/update APIs in ECXTPXFB to query/update a Trading Partner profile.
30	1	ECX_INVALID_TXN_PARAMS	Party Site ID or Transaction Type or Transaction Subtype cannot be NULL.	Use the window to view/update the Trading Partner profile and ensure that the Trading Partner Site, Transaction Type, and Transaction Subtype are valid.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_NO_EMAIL_ADDR	Unable to determine TP email address for party_type: &p_party_type, transaction_type: &p_transaction_type, transaction_subtype: &p_transaction_subtype, and party_site_id: &p_party_site_id.	Cannot determine Trading Partner e-mail address with given token values to send notification. Use the window to review/update the e-mail address in the Trading Partner setup.
30	1	ECX_NO_ENVELOPE	Unable to determine envelope information for internal control number: &p_icn.	Envelope information for the given internal control number was not found. The internal control number is a system-generated number that uniquely identifies the XML message being processed. Verify that the internal control number is valid.
30	1	ECX_NO_UNIQUE_TP_SETUP	Could not resolve a Unique/Destination Configuration for this Partner.	Cannot determine unique Trading Partner detail for outbound transaction with the information identified. Check parameters for Party Type, Party ID, Party Site ID, Transaction Type, and Transaction Subtype.
30	1	ECX_PARTY_ID_NOT_NULL	Party ID is a required parameter.	Pass valid Party ID, Party ID cannot be null.
30	1	ECX_PARTY_SITE_ID_NOT_NULL	Party Site ID is a required parameter.	Pass valid Party Site ID, Party Site ID cannot be null.
30	1	ECX_PARTY_TYPE_NOT_NULL	Party Type is a required parameter.	Review seeded Party Type lookup and pass valid Party Type. Party Type cannot be NULL.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_PARTY_TYPE_NOT_SET	Please provide party_type for the trading partner.	Party Type not available for XML Gateway to uniquely determine Trading Partner. Provide valid Party Type from seeded Party Type lookup.
30	2	ECX_PARTY_TYPE_NOT_SET	Please provide party_type for the trading partner.	Party type not available for XML Gateway to uniquely determine Trading Partner. Provide valid Party Type from seeded Party Type lookup.
30	1	ECX_RCVR_NOT_SETUP	Receiver TP setup not found for tp_header_id: &p_tp_header_id.	The Receiving Trading Partner was not found. Use the window to verify that the Trading Partner is defined.
30	2	ECX_RULE_INVALID_TP_SETUP	The Standard: &p_standard_code, Transaction Type: &p_transaction_type, Transaction Subtype: &p_transaction_subtype, and Location Code: &p_party_site_id is not enabled in the XML Gateway Server. Please check your Setup.	Either invalid transaction, standard code, or Trading Partner encountered. Use the Define Transactions window to view/update the transaction setup. Use the window/API to view/update the Trading Partner setup and the transactions enabled for the the Trading Partner.
30	1	ECX_SNDR_NOT_SETUP	Sender TP setup not found for tp_header_id: &p_tp_header_id.	The Sending Trading Partner was not found. Use the window to verify that the Trading Partner is defined.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_STATIC_ROUTING_NOT_ENABLED	&p_ext_type and &p_ext_subtype Not enabled for &p_party_ext_code in Oracle XML Gateway Server for Static Routing.	Could not determine routing information for given Trading Partner and transaction. Use the window to view/update the Trading Partner details for the Routing ID.
30	1	ECX_TRANSACTION_NOT_ENABLED	&p_ext_type and &p_ext_subtype Not enabled for &p_party_ext_code in Oracle XML Gateway Server.	Inbound transaction not enabled in XML Gateway. Check envelope parameter values for Message Standard, Transaction Type, Transaction Subtype, and Source Trading Partner Location. Either the envelope information contains an error or the Trading Partner has not been defined. Use the window to define the Trading Partner to process this inbound transaction.
30	1	ECX_TRANSACTION_NOT_FOUND	No Transaction defined for Transaction Type: &p_transaction_type, Transaction Subtype: &p_transaction_subtype, and Party Type: &p_party_type.	Given Party Type, Transaction Type, and Transaction Subtype not found. Check parameter values being passed.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	2	ECX_USER_TP_VALID	The Standard: &p_standard_code, Transaction Type: &p_transaction_type, Transaction SubType: &p_transaction_subtype, Location Code: &p_party_site_id and User Name: &p_user_name is not enabled in the XML Gateway Server. Please check your Setup.	Either invalid standard code, transaction, user name, or Trading Partner encountered. Use the Trading Partner User Setup form to associate a user with a specific Trading Partner. Use the Trading Partner Setup form to view or update the Trading Partner setup.
30	2	ECX_TP_USER_ASSIGNED	The user &user_name is already assigned to another Trading Partner.	Assign different users to the Trading Partner in the Trading Partner User Setup form.
30	2	ECX_TP_INSUFFICIENT_VAL	Please save the header record before selecting User Setup.	In the Trading Partner Setup form, save the Trading Partner header record including Trading Partner Type, Trading Partner Name, Trading Partner Site, and Company Admin Email first before clicking the User Setup button to have access to the Trading Partner User Setup form.

Note: Party Side ID would be used almost everywhere in Oracle XML Gateway. If you get an error like "Party Site ID not authorized", then make sure that Party Site ID is passed in the Message Envelope for Inbound transactions. Also, Party Site ID should be passed in ATTRIBUTE3 of Message envelope. Otherwise transactions will not work properly.

Hub Errors

Error Type	Error Code	Error Message Code	Message	Corrective Action
25	1	ECX_DELIVERY_HUB_NOT_SETUP	Unable to resolve the hub destination parameters for this trading partner hub.	Cannot determine the hub attributes (for inbound transaction) for this Trading Partner. Use the Define Hubs and windows to view/update hub attributes. Hub attributes are used if Trading Partner "Connection/Hub" attribute is set to "Hub."
30	1	ECX_DELIVERY_HUB_NOT_SETUP	Unable to resolve the hub destination parameters for this trading partner hub.	Cannot determine the hub attributes (for outbound transaction) for this Trading Partner. Use the Define Hubs and windows to view/update hub attributes. Hub attributes are used if Trading Partner "Connection/Hub" attribute is set to "Hub."

Code Conversion Errors

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_CODE_CONVERSION_DISABLED	Standard Code: &MESSAGE_STANDARD not Found. Code Conversion is disabled.	The execution engine is using an XML standard not defined in ECX_STANDARDS, so the code conversion cannot be performed. Use the Define XML Standards window to review the seeded XML standards. Enter an enhancement request for required XML standard and identify the transactions that require this new standard.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_TP_NOT_FOUND	Trading Partner not Found. Code Conversion is disabled.	A warning indicating that the Trading Partner was not found, so Trading Partner code conversion cannot be performed. Use the window to view/update the Trading Partner setup.

Enqueue/Dequeue Errors

Error Type	Error Code	Error Message Code	Message	Corrective Action
10	10	ECX_DEQUEUED_LOGGED	Message dequeued and logged	Message dequeued and logged.
30	2	ECX_MANY_PROCESSING_QUEUES	More than one row resulted while querying the queue name.	Unable to determine unique queue for inbound transaction. Use the Define Transactions window to view/update the queue definition for the inbound transaction being processed.
30	1	ECX_NO_PROCESSING_QUEUE	Unable to determine processing engine queue.	Unable to determine processing queue for inbound transaction. Use the Define Transactions window to view/update the queue definition for the inbound transaction being processed.
30	1	ECX_PROCESSING_ENQ_ERROR	Error enqueueing to processing engine: &p_queue_name	Unable to enqueue when processing inbound transaction. Ensure that the transaction queue and agent listener are enabled.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_REPROCESSING_ERROR	Error enqueueing to processing engine, while REPROCESSING: &p_out_queue	Unable to enqueue when reprocessing inbound transaction. Ensure that the transaction queue and agent listener are enabled.

Note: The enqueue_enabled and dequeue_enabled are YES for all ECX% queues in the DBA_QUEUES view. Make sure that you always get 'Y' for all the transaction queues.

Workflow Business Event System Errors

Error Type	Error Code	Error Message Code	Message	Corrective Action
10	10	ECX_PROCESSING_RULE	Processing rule	Processing rule
30	2	ECX_BUSINESS_EVT_NOT_SET	Business Event Not Set for the Transaction	Check that the message map for the inbound transaction concludes with root level, postprocess procedure call action to call the ECX_STANDARD.setEventDetails API. This API sets the business event to be raised to indicate an inbound transaction has been successfully processed. The corresponding application event subscription will consume this event.
30	2	ECX_IN_RULE_PROCESSING_ERROR	Error in processing inbound rule	Unexpected exception when processing inbound rule. Check the setup for event and agent listener.

Confirmation BoD Errors

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	2	ECX_CONFIRM_DOC_NOT_FOUND	Unable to determine the inbound document for which this confirmation is being sent.	Verify that the Workflow for the outbound confirmation includes the internal control number corresponding to the inbound business document. Refer to the Workflow Development Guidelines on how to implement the Confirmation BoD for the details.
30	2	ECX_CONFIRM_GENERATE_FAILED	Cannot generate confirmation for Confirmation Inbound. Loop Detected.	Cannot use a Confirmation message to acknowledge a Confirmation message.
30	2	ECX_CONFIRM_STATUS_NOT_FOUND	Cannot find confirmation status for the trading partner.	Could not determine the Confirmation flag setting for the Trading Partner. Use the window to view/update the Trading Partner setup and set the Confirmation flag if necessary.
30	2	ECX_CONFIRM_TP_NOT_SETUP	Unable to determine trading partner setup for the Confirmation. Transaction Type: &p_transaction_type, Transaction Subtype: &p_transaction_subtype , Location Code: &p_location_code.	Cannot determine Trading Partner to send the Confirmation. Use the window to view/update the Trading Partner setup for the given Transaction Type, Transaction Subtype, and Source Trading Partner Location Code.

Message Map and DTD Errors

Error Type	Error Code	Error Message Code	Message	Corrective Action
20	1	ECX_INCOMPLETE_OAG_DATE	Incomplete OAG Date: &p_datetime.	A warning indicating the incoming OAG date was incomplete. This could cause the conversion to the Oracle date to be incomplete.
20	1	ECX_USER_INVOKED_EXIT	Program Exit invoked by the User.	A warning indicating a user-programmed exit was executed.
30	1	ECX_CANNOT_CONVERT_TO_DATE	Cannot Convert to Date.	Usage of (or other data conversion APIs) in the message map triggered an error. Use Message Designer to verify that the map action/API is defined correctly. Make the necessary changes and reload the updated map.
30	2	ECX_CANNOT_CONVERT_TO_NUMBER	Cannot convert the value &p_value to number.	Invalid number used with math function. Verify that the variable/literal value is numeric. Make the necessary changes and reload the updated map.
30	1	ECX_CANNOT_CONVERT_TO_NUMBER	Cannot Convert to number.	Condition evaluation error, inform the System Administrator or log a bug.
30	1	ECX_COND_NOT_DEFINED	Condition type: &TYPE not defined.	Review map actions using Message Designer to ensure that the condition expressions are defined correctly with two operands and a valid operator. Make the necessary changes and reload the updated map.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	2	ECX_DATATYPE_CONV_FAILED	DataType conversion failed while inserting the level into the table.	Datatype mismatch between source and database datatype. Make the necessary changes and reload the updated map.
30	1	ECX_INVALID_ADDRESS_TYPE	Address type: &p_address_type is invalid.	Invalid address type. Check section of Message Designer chapter for list of supported address types.
30	1	ECX_INVALID_NUMBER	Binding value &pre_var_value for bind variable &variable_name is an invalid number.	Usage of the action in the message map contained an invalid number for the given Bind Value and Bind Variable combination. Use Message Designer to review the Source Definition of the map for all usage of the Append Where Clause, make the necessary changes, and reload the updated map.
30	1	ECX_MAPPINGS_NOT_FOUND	Mappings for Map ID: &MAP_ID not found.	Map details for given Map ID not found because incorrect mappings, such as level cross overs, were detected. Use Message Designer to review the message map. Make the necessary changes, and reload the updated map.
30	1	ECX_MATH_FUNC_NOT_NULL	Math function type cannot be null.	Usage of Math function is incomplete. Use Message Designer to review all math functions defined, make the necessary changes, and reload the updated map.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_ROOT_ELEMENT_NOT_FOUND	Root Element information not found for Map ID: &MAP_ID.	Root element information not found for given message map. Use the Message Designer to open the map and use the option to add the root element. Reload the updated map.
30	1	ECX_ROOT_INFO_NOT_FOUND	Map root element or parent node ID not found.	Root element or parent node ID not found for given message map. Use Message Designer to open the map, and use the option to add the root element or parent node ID. Reload the updated map.
30	1	ECX_SEED_DATA_NOT_FOUND	Seed Data is missing for Map ID: &MAP_ID.	Map data missing for given Map ID. Make sure you are operating in the desired database instance.
30	2	ECX_STACKVAR_NOT_FOUND	Stack Variable not found.	Error in processing global variables. Inform System Administrator or log a bug.
30	1	ECX_UNSUPPORTED_CONDITION_TYPE	Unsupported condition type: &TYPE.	Review map actions using Message Designer to ensure that the condition expressions are defined correctly with two operands and a valid operator. Make the necessary changes and reload the updated map.
30	2	ECX_UNSUPPORTED_DATATYPE	Unsupported Data Type.	The message map contained usage of unsupported data types. Use the Message Designer to review the message map to verify that only the VARCHAR2, NUMBER, DATE, CHAR, and CLOB data types are used. Make the necessary map changes and reload the updated map.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_UNSUPPORTEDDATE_COND	Unsupported Condition for Date.	Cannot use Condition Expression to compare dates. Use database functions instead to compare dates.
30	1	ECX_UNSUPPORTED_MATH_FUNC	Unsupported Math function.	XML Gateway supports the four basic math functions: add, subtract, multiply, and divide. Use database functions for other math functions.
30	1	ECX_UNSUPPORTED_NUMBER_COND	Unsupported Condition for Number.	Unsupported condition for numbers. Verify that the condition operator is relevant for numbers. Make the necessary map changes and reload the updated map.
30	1	ECX_UNSUPPORTED_STRING_COND	Unsupported Condition for Strings.	Unsupported condition for strings. Verify that condition operator is relevant for strings. Make the necessary map changes and reload the updated map.

Inbound Processing Errors

Error Type	Error Code	Error Message Code	Message	Corrective Action
10	10	ECX_REPROCESSING_MESSAGE	Reprocessing message.	Reprocessing message.
25	1	ECX_PROTOCOL_ADDRESS_NULL	Destination Address missing for party ID: &p_tp_detail_id.	Inbound message envelope did not contain a protocol address for the given Trading Partner. Use the window to view/update the Trading Partner's protocol address.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	2	ECX_MSG_EXISTS_IN_LOG	Message already in the Log &p_msgid.	Inbound message already exists; duplicate not allowed.
30	1	ECX_MSGID_NOT_FOUND	Msg Id: &p_msgid not Found.	Inbound message for given message ID not found. Make sure the message ID is valid.
30	1	ECX_UNSUPPORTED_STANDARD	Not a Supported Standard in the XML Gateway Server.	Inbound message envelope indicates message is for an XML standard not supported by Oracle E-Business Suite. Use the Define XML Standards window to view the list of supported standards. Enter an enhancement request for the required XML standard and identify the transactions that require this new standard.

General Processing Errors

Error Type	Error Code	Error Message Code	Message	Corrective Action
10	10	ECX_PROCESSING_MESSAGE	Processing message.	N/A
10	10	ECX_TRIGGER_OUTBOUND	Triggering outbound.	N/A

Error Type	Error Code	Error Message Code	Message	Corrective Action
25	1	ECX_PARSE_ERROR	Parse Error.	Parse error encountered because the generated or received XML message was poorly formed or invalid as per the DTD. The other possible reason for a parse error is that the DTD referenced in the map is not the same DTD that was passed. Make sure that the DTD referenced in the map is loaded into the XML Gateway repository.
30	1	ECX_DOCLOGS_NOT_EXISTS	No document log exists for message ID: &p_msgid.	ECX_DOCLOGS entry does not exist for the given message ID. Ensure that the message ID is valid.
30	1	ECX_DTD_NOT_FOUND	DTD not found for Map ID: &MAP_ID.	The DTD referenced in the message map was not found because the map referenced the wrong DTD or the DTD was not loaded into the XML Gateway repository. Use Message Designer to check the DTD referenced in the map, make the necessary changes, and reload the updated map. If necessary load the correct DTD into the XML Gateway repository.
30	2	ECX_ERROR_NOT_SET	Unable to set error information.	Inform System Administrator or log a bug.

Error Type	Error Code	Error Message Code	Message	Corrective Action
30	1	ECX_MAP_NOT_FOUND	Not a Supported Map &p_map_code in the XML Gateway Server.	Given map code associated with the outbound transaction enabled for a Trading Partner is not supported. Use the window to view/update the map code associated with the enabled outbound transaction. Make sure the map has been loaded in the XML Gateway repository.
30	2	ECX_MAP_NOT_FOUND	Not a Supported Map &p_map_code in the XML Gateway Server.	Given map code associated with the inbound transaction enabled for a Trading Partner is not supported. Use the window to view/update the map code associated with the enabled inbound transaction. Make sure the map has been loaded in the XML Gateway repository.
30	1	ECX_MSGID_NOT_FOUN D	Message ID cannot be NULL.	Pass valid value for Message ID parameter.
30	1	ECX_MSGID_NOT_NULL	Message ID cannot be NULL.	Pass valid value for Message ID parameter.

XML Gateway API-Level Messages

The following is a list of messages generated by the XML Gateway APIs. The list is organized by API Return Code for easy reference. The list contains the following columns:

- API Return Code
- Message Code
- Message

The API Return Code represents the status of the API. The calling program can interrogate the return code and provide an application-specific error handler to address the error detected. Because these messages are designed for communication between

application APIs, the necessary corrective actions are addressed during the development process. There is no action required, unless you are developing a custom transaction.

API Return Code	Description
0	Success
1	Warning
2	Unexpected Error
3	Null value detected for required parameter
4	Invalid parameter value
5	Cannot insert duplicate row
6	No data found for given parameter values
7	Duplicate row exists for given parameter values
8	Cannot delete due to referential integrity

Because the focus of this section is on troubleshooting, API Return Code of 0 (Success) is excluded from the following list. As of this printing, there are no seeded messages with API Return Code of 1 or 2.

API Return Code	Message Code	Message
3	ECX_CATEGORY_ID_NOT_NULL	Code category ID is a required parameter.
3	ECX_CODE_CATEGORY_NOT_NULL	Code category is a required parameter.
3	ECX_CONNECTION_TYPE_NOT_NULL	Connection type is a required parameter.
3	ECX_DATA_SEEDED_NOT_NULL	XRef data seeded is a required parameter.
3	ECX_DIRECTION_NOT_NULL	Direction is a required parameter.
3	ECX_EMAIL_ADDRESS_NOT_NULL	Email_address is a required parameter.

API Return Code	Message Code	Message
3	ECX_EXT_PROCESS_ID_NOT_NULL	External process ID is a required parameter.
3	ECX_HUB_USER_ID_NOT_NULL	Hub User ID is a required parameter.
3	ECX_LOCATION_NOT_NULL	Location Code is a required parameter.
3	ECX_MAP_CODE_NOT_NULL	Map Code is a required parameter.
3	ECX_PARTY_ID_NOT_NULL	Party ID is a required parameter.
3	ECX_PARTY_SITE_ID_NOT_NULL	Party Site ID is a required parameter.
3	ECX_PARTY_TYPE_NOT_NULL	Party Type is a required parameter.
3	ECX_PROTOCOL_TYPE_NOT_NULL	Protocol type is a required parameter.
3	ECX_PWD_NOT_NULL	For protocol type: &p_protocol_type, the password is a required parameter.
3	ECX_STANDARD_CODE_NOT_FOUND	Standard code: &p_standard_code not defined.
3	ECX_STANDARD_CODE_NULL	Standard Code is a required parameter.
3	ECX_STANDARD_NOT_NULL	Document standard is a required parameter.
3	ECX_TP_DTL_ID_NOT_NULL	Trading partner detail ID is a required parameter.
3	ECX_TP_HDR_ID_NOT_NULL	Trading partner header ID is a required parameter.
3	ECX_TRAN_SUBTYPE_NOT_NULL	Transaction Subtype is a required parameter.
3	ECX_TRANSACTION_SUBTYPE_NOT_NULL	Transaction type is a required parameter.
3	ECX_TRANSACTION_TYPE_NOT_NULL	Transaction type is a required parameter.

API Return Code	Message Code	Message
3	ECX_XREF_DTL_ID_NOT_NULL	XRef detail ID is a required parameter.
3	ECX_XREF_EXT_VAL_NOT_NULL	XRef external value is a required parameter.
3	ECX_XREF_INT_VAL_NOT_NULL	XRef internal value is a required parameter.
3	ECX_XREF_STANDARD_ID_NOT_NULL	XRef standard ID is a required parameter.
3	ECX_XREF_STD_VAL_NOT_NULL	XRef standard value is a required parameter.
3	ECX_USERNAME_NOT_NULL	For protocol type: &p_protocol_type, the username is a required parameter.
4	ECX_DATA_OWNER_INCONSISTENT	Data owner: &p_owner is inconsistent with data seeded flag: &p_data_seeded.
4	ECX_INVALID_CONF_CODE	Confirmation code: &p_confirmation is invalid.
4	ECX_INVALID_DIRECTION	Direction: &p_direction is invalid.
4	ECX_INVALID_EMAIL_ADDRESS	Email_address: &p_email_address is invalid.
4	ECX_INVALID_EXT_PROCESS_ID	External process ID: &p_ext_process_id is invalid.
4	ECX_INVALID_HUB_ID	Hub ID: &p_hub_id is invalid.
4	ECX_INVALID_HUB_USER_ID	Hub user ID: &p_hub_user_id is invalid.
4	ECX_INVALID_MAP_CODE	Map Code: &p_map_code is invalid.
4	ECX_INVALID_PARTY_ID	Party ID: &p_party_id is invalid.
4	ECX_INVALID_PARTY_SITE_ID	Party_site_id: &p_party_site_id is invalid.
4	ECX_INVALID_PARTY_TYPE	Party Type: &p_party_type is invalid.

API Return Code	Message Code	Message
4	ECX_INVALID_PROTOCOL_TYPE	Protocol type: &p_protocol_type is invalid.
4	ECX_INVALID_PWD	The password should not contain special characters.
4	ECX_INVALID_PWD_LEN	The length of the password should be greater than 5 characters.
4	ECX_INVALID_ROUTING_ID	Routing ID: &p_routing_id is invalid.
4	ECX_INVALID_TP_DETAIL_ID	Tp_detail_id: &p_tp_detail_id is invalid.
4	ECX_INVALID_TP_HDR_ID	Tp_header_id: &p_tp_header_id is invalid.
5	ECX_CODE_CATEGORY_EXISTS	Code category: &p_xref_category_code is already defined in the table.
5	ECX_DOCUMENT_STANDARD_EXISTS	Standard already exists with standard code &p_standard_code and standard type &p_standard_type.
5	ECX_DUPLICATE_TRANSACTIONS	Duplicate Rows found for Transaction Type: &p_transaction_type, Transaction Subtype: &p_transaction_subtype, and Party Type: &p_party_type.
5	ECX_TP_DTL_EXISTS	Trading Partner detail record for tp_header_id: &p_tp_header_id, ext_process_id: &p_ext_process_id is already defined in the table.
5	ECX_TP_DTL1_EXISTS	Trading Partner detail record for the combination of External Transaction Type: &p_ext_type, External Transaction Subtype: &p_ext_subtype, Document Standard ID: &p_standard_id, Source TP Location Code: &p_source_tp_location_code is already defined for the inbound transaction.

API Return Code	Message Code	Message
	ECX_TP_DTL2_EXISTS	Trading Partner detail already exists for this combination of tp_header_id: &p_tp_header_id, Transaction Type: &p_transaction_type, Transaction Subtype: &p_transaction_subtype for outbound transactions.
5	ECX_TP_HDR_EXISTS	Trading Partner for party_type: &p_party_type, party_id: &p_party_id, party_site_id: &p_party_site_id is already defined in the table.
5	ECX_XREF_DTL_ID_ROW_EXISTS	Duplicate code conversion record for header id: &p_tp_header_id, external value: &p_xref_ext_value, internal value: &p_xref_int_value, direction: &p_direction.
5	ECX_XREF_DTL_ROW_EXISTS	Duplicate code conversion record for code category: &p_xref_category_code, Standard code: &p_standard_code, external value: &p_xref_ext_value, internal value: &p_xref_int_value, direction: &p_direction.
5	ECX_XREF_STD_ID_ROW_EXISTS	Duplicate standard code conversion record for XRef standard ID: &p_xref_standard_id, Standard value: &p_xref_std_value, Internal value: &p_xref_int_value.
5	ECX_XREF_STD_ROW_EXISTS	Duplicate standard code conversion record for document standard: &p_standard, Standard value: &p_xref_std_value, Internal value: &p_xref_int_value.
6	ECX_CODE_CATEGORY_NOT_FOUND	Code category: &p_xref_category_code not defined.
6	ECX_HUB_NOT_EXISTS	No Hub definition exists for connection_type: &p_connection_type and protocol: &p_protocol_type.

API Return Code	Message Code	Message
6	ECX_NO_ROWS_DELETED	No rows deleted in table: &p_table, &p_param_name: &p_param_id is invalid.
6	ECX_NO_ROWS_UPDATED	No rows updated in table: &p_table, &p_param_name, &p_param_id is invalid.
6	ECX_NOT_FOUND	No rows fetched from table: &p_table and key: &p_key.
6	ECX_STANDARD_NOT_FOUND	Document standard: &p_standard not found.
6	ECX_STANDARD_ROW_NOT_FOUND	No standard found with standard code &p_standard_code and standard type &p_standard_type.
6	ECX_TP_DTL_NOT_FOUND	No trading partner detail records found for tp_header_id: &p_tp_header_id, ext_process_id: &p_ext_process_id.
6	ECX_TP_DTL1_NOT_FOUND	No trading partner detail records found for party_type: &p_party_type, party_id: &p_party_id, party_site_id: &p_party_site_id, transaction_type: &p_transaction_type, transaction_subtype: &p_transaction_subtype, standard code: &p_standard_code, direction.
6	ECX_TP_HDR_NOT_FOUND	No trading partner header records found for party type: &p_party_type, party_id: &p_party_id, party_site_id: &p_party_site_id.
6	ECX_TRANSACTION_NOT_FOUND	No Transaction defined for Transaction Type: &p_transaction_type, Transaction Subtype: &p_transaction_subtype, and Party Type: &p_party_type.

API Return Code	Message Code	Message
6	ECX_XREF_DTL_EXT_ID_NOT_FOUND	No code conversion detail records found for standard code: &p_standard_code, code category: &p_xref_category_code, tp header ID: &p_tp_header_id, external value: &p_xref_ext_value.
6	ECX_XREF_DTL_INT_ID_NOT_FOUND	No code conversion detail records found for standard code: &p_standard_code, code category: &p_xref_category_code, tp header ID: &p_tp_header_id, internal value: &p_xref_int_value.
6	ECX_XREF_DTL_NOT_FOUND	No code conversion detail records found for standard code: &p_standard_code, code category: &p_xref_category_code, direction: &p_direction, external_value: &p_xref_ext_value, internal value: &p_xref_int_value.
7	ECX_STANDARD_TOO_MANY_ROWS	More than one row exists for the unique key-standard code: &p_standard_code and standard type: &p_standard_type.
7	ECX_TOO_MANY_ROWS	More than one row fetched from table: &p_table and key: &p_key.
7	ECX_TP_DTL_TOO_MANY_ROWS	Number of records fetched is greater than one for tp_header_id: &p_tp_header_id, ext_process_id: &p_ext_process_id.
7	ECX_TP_DTL1_TOO_MANY_ROWS	Number of records fetched is greater than one for party_type: &p_party_type, party_id: &p_party_id, party_site_id: &p_party_site_id, transaction_type: &p_transaction_type, transaction_subtype: &p_transaction_subtype, standard_code: &p_standard_code, direction
7	ECX_TP_HDR_TOO_MANY_ROWS	Number of records fetched is greater than one for party_type: &p_party_type, party_id: &p_party_id, party_site_id: &p_party_site_id.

API Return Code	Message Code	Message
7	ECX_TRANS_TOO_MANY_ROWS	Too many rows in transactions for Transaction Type: &p_transaction_type, Transaction Subtype: &p_transaction_subtype, and Party Type: &p_party_type.
7	ECX_XREF_DTL_EXT_TOO_MANY_ROWS	Number of records fetched is greater than one for standard code: &p_standard_code, code category: &p_xref_category_code, tp header ID: &p_tp_header_id, external_value: &p_xref_ext_value.
7	ECX_XREF_DTL_INT_TOO_MANY_ROWS	Number of records fetched is greater than one for standard code: &p_standard_code, code category: &p_xref_category_code, tp header ID: &p_tp_header_id, internal value: &p_xref_int_value.
8	ECX_CODE_CATEGORY_REFERENCED	Unable to delete code category ID: &p_category_id because it is referenced in other tables.

XML Gateway Version Validation

Associated with each data definition or map file are the major and minor version numbers indicating the version of Message Designer used to create the map. The numbers are available in the <ECX_MAJOR_VERSION> and <ECX_MINOR_VERSION> tags. Message Designer will compare the map version number to the Message Designer version number (available in the Help >About Menu) and will open the map only if the major versions are the same and the minor versions are the same or lower.

In addition, the map version numbers will be compared to the XML Gateway installed version number (available in WF_RESOURCES.ECX_VERSION) when you attempt to load the maps into the XML Gateway repository. The maps will be loaded only if the major versions are the same and the minor versions are the same or lower.

Problem: Error while running LoadMap

Version Incompatible: XML Gateway 2.10 map is not supported by currently installed XML Gateway 2.6.

Sample error message:

```
Java oracle.apps.ecx.loader.LoadMap apps password $JDBC mymap.xgm
Database Connect String
(PORT=1543))(CONNECT_DATA=(SID=ecxcert)))
(DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=ap935sun
)UserName apps
Connecting...
connected.
Installed XML Gateway Version: 2.6.0.0
Started Processing of the Document
Processing Instructions for xmlGateway=> MAP
Map Code: ECX_P00 Using Old Map id: 19
Version Incompatible: XML Gateway 2.10 map is not supported by currently
installed XML Gateway 2.6.0.0
```

Solution: Upgrade installed XML Gateway

The following table shows the compatibility of other sample map and XML Gateway versions. The version numbers are available in the <ECX_MAJOR_VERSION> and <ECX_MINOR_VERSION> tags.

In the example version 2.6, 2 is the major version number, and 6 is the minor version number.

Installed XML Gateway Version	XGM/LOADER Version	Compatible/ Incompatible	Action
2.6.0.0	2.10.0.0	Incompatible	Upgrade XML Gateway
2.6.0.0	2.6.0.0	OK	None
2.10.0.0	2.6.0.0	OK	None
2.6.0.0	2.4.0.0	OK	None
2.6.0.0	1.1.0.0	Incompatible	Upgrade Map
2.6.0.0	3.1.0.0	Incompatible	Upgrade XML Gateway

Problem: Error while processing transaction (using debug level 3)

Sample error message:

```
Enter ECX_UTILS.INITIALIZE
i_map_id==>19
Enter ECX_UTILS.CHECK_VERSION
i_major_version==>2
i_minor_version==>10
i_eng_version==>2.6.0.0
i_eng_major_version==>2
i_eng_minor_version==>6
Exit ECX_UTILS.CHECK_VERSION
[ECE_PROGRAM_ERROR] PROGRESS_LEVEL=ecx_utils.INITIALIZE
[ECE_ERROR_MESSAGE] ERROR_MESSAGE=Version Incompatible: XML Gateway
version 2.10 is not supported by currently installed XML Gateway 2.6.0.0
Exit ecx_utils.INITIALIZE
Clean-up i_stack, l_node_stack and i_tmpxmlExit
ECX_OUTBOUND.PROCESS_OUTBOUND_DOCUMENTS
ECX_UTILS.INITIALIZE: Version Incompatible: XML Gateway version 2.10 is
not supported by currently installed XML Gateway 2.6.0.0

Exit ECX_OUTBOUND.GETXML
```

Solution: Upgrade installed XML Gateway

At runtime, the engine compares the version from the .xgm file to the installed XML Gateway version. The Loader and the Engine only support maps that have the same major version and same (or lower) minor version. XML Gateway will return an error message if the versions are incompatible.

The following table shows other sample error conditions while processing transaction:

Installed XML Gateway Version	XGM/LOADER Version	Compatible/ Incompatible	Action
2.6.0.0	2.10.0.0	Incompatible	Upgrade XML Gateway
2.6.0.0	2.6.0.0	OK	None
2.10.0.0	2.6.0.0	OK	None
2.6.0.0	2.4.0.0	OK	None
2.6.0.0	1.1.0.0	Incompatible	Upgrade Map
2.6.0.0	3.1.0.0	Incompatible	Upgrade XML Gateway

Common TLS Issues

The following are common errors encountered during TLS handshake:

`javax.net.ssl.SSLHandshakeException`

`javax.net.ssl.SSLHandshakeException: Untrusted Cert Chain`

```
at com.sun.net.ssl.internal.ssl.BaseSSLSocketImpl.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.SunJSSE_az.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.SunJSSE_az.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.SunJSSE_ax.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.j(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.startHandshake
(DashoA12275)
at oracle.apps.ecx.oxta.getSSLConnection(SSLConnection.java:132)
```

Caused by: `java.security.cert.CertificateException: Untrusted Cert Chain`

```
at oracle.security.pki.ssl.C15.checkClientTrusted(C15)
at oracle.security.pki.ssl.C15.checkServerTrusted(C15)
at oracle.security.pki.ssl.C02.checkServerTrusted(C02)
at com.sun.net.ssl.internal.ssl.JsseX509TrustManager.
checkServerTrusted(DashoA12275)
```

Description: This means that the client failed to authenticate the server.

Resolution: To rectify this error, add the top level CA certificate of the destination server as a trusted certificate in the wallet. This is assuming that the server presents its entire certificate chain during TLS handshake. If it is configured to present only the leaf level certificate, then all the CA certificates in the chain have to be added to the wallet as trusted certificates.

`javax.net.ssl.SSLHandshakeException`

`Received fatal alert: unknown_ca`

```
at com.sun.net.ssl.internal.ssl.BaseSSLSocketImpl.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.BaseSSLSocketImpl.b(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.b(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.a(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.j(DashoA12275)
at com.sun.net.ssl.internal.ssl.SSLSocketImpl.startHandshake
(DashoA12275)
at oracle.apps.ecx.oxta.getSSLConnection(SSLConnection.java:132)
```

Description: This means client authentication failed.

Resolution: To rectify this error, the user certificate in the wallet along with issuer CA certificates if there are any has to be sent over to the server and used in the configuration for client authentication.

Oracle Diagnostic Tests

Oracle Diagnostics provides a mechanism that makes Oracle E-Business Suite more

supportable and robust by providing predefined tests to check for key setup requirements. Oracle XML Gateway provides several tests through Oracle Diagnostics that you can use to check the setup of XML Gateway and review debugging information.

You can access Oracle Diagnostics through different user interfaces, including Oracle Applications Manager and other administrative consoles. For more information, see the *Oracle Diagnostics Framework User's Guide*.

The XML Gateway tests are available in Oracle Diagnostics under the Application Object Library application.

XML Gateway Tests

The following tests are available in the XML Gateway Tests Group.

XML Gateway Health Check

This diagnostic test validates XML Gateway metadata including XML Gateway Maps, DTDs, XSLs, procedures and so on which are already loaded into the database. It does not require any parameters. This test generates a report which contains the following sections:

- **DTD Validation Test**

This test determines if all DTDs loaded into the database are valid. If invalid DTDs are found, the test lists the FileName, DTD ID, RootElement and ProblemDetails for the invalid DTDs. To resolve, fix the DTD issues and reload the DTDs.

- **Map Compatibility Test**

This diagnostic verifies that the map version is compatible with the XML Gateway engine version. A map is compatible with the engine if the map major version numbers are the same and the map minor version is less than or equal to the engine minor version.

If a map fails the test, the MapID, MapCode, and VersionNumber are reported.

- **Procedure Validity Test**

This diagnostic verifies that all user-defined procedures used by supported maps exist and are valid.

If the test finds an invalid procedure, the following details are reported: ProcedureFullName, MapCode and ProblemDetails.

To resolve, review the procedure to ensure that it exists and is valid. Additionally, verify that the defined procedure reflects the latest version of the API.

- **XML Gateway Engine Test**

This diagnostic verifies the XML Gateway engine is working properly.

The TestingDirectOut and TestingDirectIn message maps are used in this test to analyze the following parameters:

- Outbound Return Code
- Outbound Error Message
- Outbound XML File
- Outbound Log File
- Inbound Return Code
- Inbound Error Message

If the test finds an error, the following details are returned: OutboundReturnCode, OutboundErrorMessage, OutboundLogFile, InboundReturnCode, InboundErrorMessage, and InboundLogFile.

To resolve this error, verify that all packages related to XML Gateway are valid.

- **XML Parser Status Test**

This test prints the XML Parser version and XML Parser installation status.

- **XSL Validation Test**

This diagnostic verifies that all style sheets referenced by supported maps exist and are valid. The test queries the database first and if necessary queries the file system directory identified by the ECX: XSLT File Path system profile.

If the test finds an invalid XSLT, it returns the following details: MapID, MapCode, XSLFileName, ProblemCause.

To resolve, fix the XSLT issues and reload the XSLT.

- **XSL Transform Test**

This diagnostic transforms a sample XML having special characters with a sample XSL using database XDK.

If the test finds an issue in XSL Transformation, then it will print the error message.

To resolve, verify the database XDK.

- **URL List Extraction Test**

This test outputs a list of URLs which are defined in the Protocol Address field in the Trading Partner Setup form for Trading Partners enabled for HTTP or HTTPS protocol type.

The output URL can be used as input to the Ping Destination Server Test for the URL validation.

Get Document Details

This diagnostic will display the document details as in the Transaction Monitor. However, it will also perform the root cause analysis if the Trading Partner Name or Site Name does not show up. It requires the following parameters:

- Document Id: It is from the Transaction Monitor.
- Direction: It will be 'IN' if the Document ID is for inbound transaction, else it will be 'OUT'.

Basic OTA Diagnostic Test

This diagnostic test checks OTA parameters and posts sample SMTP, HTTP, or HTTPS messages to outbound queue as well as monitors the OTA processing statuses of posted sample messages for given input time. It requires the following parameters:

- Target HTTP OTA URL: The HTTP URL to which OTA will send the sample message.
- Email: It's the administrator's e-mail address.
- Username/Password
- SMTP: Set to 'NO' to test.
If it is set to 'YES', then the administrator's e-mail needs to be entered.
- HTTP, HTTP-ATCH: Set to 'YES' to test.
If it is set to 'YES', then the Target HTTP OTA URL parameter needs to be entered.
- Time to Live: The time in seconds that the test will continue if a response has not been received.

Note: A reasonable setting for the Time to Live parameter depends on the load in your environment.

If the test returns a successful response, but the report shows that the message is either not picked up by OTA or the OTA processing is not completed, the Time to Live parameter was not set high enough for the test to complete. In this case you can either increase the Time to Live and resubmit the test, or you can manually check the message IDs in the Message Monitor (described below). If you choose to check manually, allow a reasonable lapse of time for the messages to be picked up by OTA.

Ping Destination Server

This test verifies that the target destination is available. It can verify HTTPS destination URL in addition to the HTTP destination verification.

The input parameters are:

- Target OTA URL: The URL of the server for OTA to test
- Username/Password

Message Monitor

Use this test to query the status of inbound or outbound messages in the OXTA Log Message Table. This test enables you to check messages that are not queryable through the Transaction Monitor. You can query using any combination of the following criteria:

- Begin Date
- MessageID
- Direction

You can use this test in conjunction with other diagnostic tests that create sample messages. Use the message ID from the sample message to monitor its success.

Collaboration Setup Worksheet

This appendix covers the following topics:

- Collaboration Setup Worksheet

Collaboration Setup Worksheet

Collaboration Design

- Collaboration Name
- Oracle E-Business Suite applications

Message Payloads

Name	In or Out	XML Gateway Internal Transaction Type	XML Gateway Internal Transaction Subtype
Message Received			

Events

Event No.	Event Message	FND Message ID	Event Location (Point)	Workflow	Message Payload	CLN Event
1	Collaboratio n Created					

Index

A

Acknowledge_PO, 10-3

ACKNOWLEDGE_PO BOD, 10-3

acknowledgment, 11-3

Actions, 2-65

- append where clause, 2-76

- assign next sequence value, 2-76

- assign variable value, 2-74

- convert from OAG amount, 2-94

- convert from OAG datetime, 2-91

- convert from OAG OPERAMT, 2-92

- convert from OAG quantity, 2-93

- convert to OAG amount, 2-89

- convert to OAG datetime, 2-85

- convert to OAG OPERAMT, 2-86

- convert to OAG quantity, 2-88

- create global variable, 2-74

- derive target address ID from location code, 2-80

- derive target parent ID from location code, 2-81

- execute function call, 2-82

- execute procedure, 2-100

- exit program, 2-96

- get predefined variable value, 2-96

- insert into database table, 2-79

- math functions, 2-83

- perform concatenation, 2-105

- perform substring, 2-106

- send error message, 2-104

- XSLT transformation, 2-107

Adding a Message Event

- oracle.apps.cln.ch.collaboration.addmessage, 11-6

Adding the FND Lookup Codes

- APPLICATION_ID, 11-4

- Application ID, 11-4, 11-4

- CLN_APPLICATION_ID, 11-4

- CLN_COLLABORATION_DOC_TYPE, 11-4

- CLN_COLLABORATION_TYPE, 11-4

- Collaboration Document Type, 11-4

- Collaboration History Detail, 11-4

- Collaboration Tracking, 11-3

- Collaboration Type, 11-4, 11-4

- Document Type, 11-4

- FND Lookup Types, 11-3

- MESSAGE_TYPE, 11-4

- Message Payloads, 11-4

Adding the FND Messages

- Collaboration History Details, 11-4

- Collaboration Setup Worksheet, 11-4

- FND Message ID, 11-4

- Message, 11-4

Adding the Workflow Events

- ECX workflows, 11-5

- oracle.apps.cln.ch.collaboration.addmessage, 11-5

- oracle.apps.cln.ch.collaboration.create, 11-5

- oracle.apps.cln.ch.collaboration.update, 11-5

- Raising Events, 11-5

Adding Workflow Events, 11-5

Add message, 11-7

Add Message Event, 11-6

Advanced Queue, 10-3, 11-6

- Agent Activity, 11-20
- Agent Listeners, 6-62
 - enabling and disabling, 6-62
- API, 10-6, 11-7
- API-level error messages, I-36
- API Return Codes, I-36
- APIs
 - ECX_ACTIONS.set_error_exit_program, F-21
 - ECX_ATTACHMENT.formulate_content_id, F-26
 - ECX_ATTACHMENT.register_attachment, F-22, F-24, F-25
 - ECX_CONDITIONS.getLengthForString, F-12
 - ECX_CONDITIONS.getPositionInString, F-12
 - ECX_CONDITIONS.getSubString, F-13
 - ECX_DOCUMENT.get_delivery_attris, F-9
 - ECX_ENG_UTILS.
 - convert_from_cxml_datetime, F-28
 - ECX_ENG_UTILS.convert_to_cxml_date, F-27
 - ECX_ENG_UTILS.convert_to_cxml_datetime, F-28
 - ECX_ERRORLOG.external_system, F-3
 - ECX_ERRORLOG.getLogDetails, F-18
 - ECX_STANDARD.generate, 6-11
 - ECX_STANDARD.getEventDetails, F-7
 - ECX_STANDARD.getEventSystem, F-8
 - ECX_STANDARD.getReferenceID, F-9
 - ECX_STANDARD.
 - perform_xslt_transformation, F-1
 - ECX_STANDARD.setEventDetails, F-6
 - ECX_TRADING_PARTNER_PVT APIs, F-14, F-15, F-15, F-16, F-18
 - how to map to, 2-110
- Application, 11-2
- Application Programming Interface, 11-7
- Application-to-Application
 - example, 6-9
- Application to Application integration
 - inbound option, 6-57
 - outbound option, 6-58
- approved, 11-2
- apps_ne.ecxmsg data type, 5-4
- AQ, 11-6
- Architecture, XML Gateway
 - diagram, 1-3
- Attachments, 2-115
 - OTA, 7-10

- Web services, 8-4
- ATTRIBUTE3
 - in dynamic routing, 3-34
- ATTRIBUTE3 Field, 3-31, 4-11
- Attributes
 - Item type, 6-28
 - Workflow Item Type, 6-7
- AutoConfig
 - client authentication parameters, 7-19

B

- B2B, 11-3, 11-3, 11-22
- booked, 11-2
- business events, 11-2
- Business events
 - identifying seeded, 6-59
 - registering new, 6-59
- Business Events, 11-21
- Business Event System
 - common questions, 6-65
- business rule, 11-21
- business-to-business, 11-3
- Business-to-Business
 - example, 6-9
- business-to-business gateway, 10-6
- Buttons
 - Message Designer, 2-4
 - Message Designer Wizards, 2-21
- buyer, 11-9

C

- Call API, 11-24
 - Call API Parameters, 11-24
 - COLLABORATION_ACCESS.
 - SECURITY_VIOLATED, 11-24
 - IP_10, 11-24
 - Mode of Execution, 11-24
 - Procedure Name, 11-24
 - Procedure Signature, 11-24
 - Security Error, 11-24
 - wf_parameter_list_t, 11-24
- Callback, 6-66, 6-66
 - how other messaging systems use, 6-68
- Cancel Order, 10-12, 10-14
- CBOD
 - purpose, 4-17

- change order, 10-3
- Change Order Notification, 10-12, 10-14
- Client authentication
 - AutoConfig parameters, 7-19
 - parameters not set through AutoConfig, 7-21
 - setup parameters, 7-19
- CLN
 - DEBUG LEVEL, 11-18
 - capture the log messages, 11-18
 - Error, 11-18
 - Event, 11-18
 - Exception, 11-18
 - fdllov(), 11-18
 - Procedure, 11-18
 - Statement, 11-18
 - Unexpected, 11-19
 - Debug Log Directory, 11-19
 - place the log file, 11-19
- CLN_APPLICATION_ID, 10-13
- CLN_COLLABORATION_TYPE, 10-13, 10-13, 10-13
- Code categories, 3-42
 - seeded, B-1
- Code Conversion, 3-38
 - inbound transaction table access, 3-45
 - one to multiple external codes, 3-46
 - outbound transaction table access, 3-44
 - Standard Code Conversion Form, 3-47
 - Trading Partner Code Conversion Form, 3-48
- Code Conversion Errors, I-26
- Code Conversion Values
 - key access, 3-42
 - order of table search, 3-42
- Collaboration Design, 11-2, J-1
 - Collaboration Name, J-1
 - Oracle E-Business Suite, J-1
- Collaboration Detail, 11-2, 11-9, 11-9
- Collaboration Display Setup, 10-20
- Collaboration Event Definitions, 10-12, 10-13
- Collaboration Event Details, 10-4, 11-7
- Collaboration Event Payload, 10-11
- Collaboration Events, 10-3
- Collaboration Final Event Definitions, 10-15, 10-15
- Collaboration History, 10-2
- Collaborations, 10-3
- Collaboration Setup Worksheet, 11-2, 11-2, 11-3,

- 11-3
- Collaboration Type, 11-2
- Conditional Node Mapping, 2-49
- Confirm_BOD, 10-3
- Confirmation BoD errors, I-28
- Confirmation Business Object Document, 4-17
 - disabling, 4-19
 - implementing, 4-19
 - seeded events and subscriptions, 4-18
 - seeded message maps, 4-17
 - structure, 4-17
- Consume XML Document Function, 6-10
- Correlation ID and Protocol Type, 3-26
- Create Collaboration Display Header, 10-21
- Create Collaboration Event Definition, 10-13
- Create Collaboration Final Event Definition, 10-16, 10-16
- Create Collaboration Property, 10-19
- Create Notification Action Header, 10-28
- Create Notification Definition, 10-25
- Creating a Collaboration Event Definition, 10-13
- Creating a Collaboration Final Event Definition, 10-16
- creating new notifications, 11-20
- Cross-Reference Transaction Identifiers, 3-11
- Customer Order, 11-2
- Custom messages
 - B2B integration, 6-64
- Custom Messages
 - considerations for, A-21

D

- Debugging Oracle E-Business Suite, 11-20
 - set the debug level, 11-20
- Debugging Workflow, 11-20
- Default Error Process, 6-37
- define any new notifications, 11-20
- Define Database Directories, 3-8
- Define Lookup Values, 3-17
- define new notifications for the B2B gateway, 11-20
- define the notification actions, 11-20
- Define Transactions Form, 3-11
- Define Transactions form fields, 3-12
- Define XML Gateway Responsibility, 3-8
- Define XML Standards Form, 3-10

- Delivery status, 6-66
- Deploying and Testing the BPEL Process at Run Time
 - deploy, 8-49
 - test, 8-52
- Designing the Collaboration, 11-2
- determine the events in the collaboration, 11-2
- Development guidelines
 - inbound messages, 6-65
 - outbound messages, 6-64
- Diagnostics, I-47
 - XML Gateway test group, I-48
- Discontinuous Nodes
 - source definition, 2-44
 - target definition, 2-51
- Document Confirmation, 3-29
- Document Levels
 - collapsing levels, A-17
 - expanding levels, A-18
 - identifying source and target, A-17
- Downloading maps, 2-113
- draw a picture of the message flow, 11-2
- DTDs
 - how to extend, 2-108
 - loading, 2-112
- Dynamic Routing, 3-34

E

- E-Business Suite Application Module-Specific Item Type, 6-6
- ECX_ACTIONS, F-21
- ECX_ATTACHMENT, F-22
- ECX_CONDITIONS, F-12
- ECX_DOCLOGS table, 6-32, 6-42, 6-42
- ECX_DOCUMENT, F-9
- ECX_ENG_UTILS, F-27
- ECX_ERRORLOG, F-3, F-18
- ECX_IN_OAG_Q, 5-2
- ECX_INBOUND queue, 5-1
- ECX_OUTBOUND Queue, 5-1
- ECX_STANDARD, F-6
- ECX_TRADING_PARTNER_PVT, F-14
- ECX: Enable User Check for Trading Partner profile option, 3-4
- ECX: Log File Path profile option, 3-3
- ECX: Maximum Retries, 3-5

- ECX: Maximum XML Size profile option, 3-4
- ECX: Notification Timeout in Minutes, 3-5
- ECX: OAG_LOGICAL_ID profile option, 3-4
- ECX: Server Time Zone profile option, 3-4
- ECX: System Administrator Email Address profile option, 3-4
- ECX: XML Validate Flag profile option, 3-4
- ECX: XSLT File Path profile option, 3-4
- ECX Engine Notification Process, 6-38
- ECX Event Message (FYI), 6-47
- ECX Event Notification (FYI), 6-40
- ECX External Event Message (FYI), 6-47
- ECX External Event Notification, 6-40
- ECX Main Error Process, 6-30
- ECX Main Inbound Error Process, 6-31
- ECX Main Outbound Error Process, 6-34
- ECXOTAINbound.html File, 7-23
- ECX Reprocess Inbound Function, 6-41
- ECX Resend Outbound Message Function, 6-42
- ECX Standard Processes, 6-7
- ECX Timeout Value Function, 6-42
- ecxver.sql script, I-1
- EDI Transactions
 - compared to XML messages, 1-10
- Element Mapping, 2-62
 - applying actions, 2-64
 - guidelines, 2-62
 - icons, 2-64
- Engine-Level Error Messages
 - error code, I-16
 - error type, I-16
- Enqueue/Dequeue Errors, I-27
- Enqueue a message with Non-Oracle messaging system, 5-5
- Envelope, 4-8
 - components of, 4-8
- Error Handling
 - for inbound messages, 6-31, 6-33
 - for outbound messages, 6-34
- Error Handling Processes, 6-28
 - relationship, 6-29
- Error Messages
 - engine-level, I-16
- Error notifications, 6-59
- Error Queue, 5-4
- Error Still Active Function, 6-42
- Event

- oracle.apps.cln.ch.collaboration.addmessage, 11-15
- oracle.apps.cln.ch.collaboration.create, 11-8
- oracle.apps.cln.ch.collaboration.update, 11-12
- Event Location, 11-3
- Event Message, 11-3
- events, 11-2
- Events, J-1
 - XML Gateway Error Processing Item Type, 6-45
 - Message Delivery Error, 6-45
 - Receive Error, 6-46
 - Receive Send Notification, 6-46
 - Receive the inbound subscription error, 6-46
 - Receive the inbound subscription processing error, 6-46
 - XML Gateway Standard Item Type, 6-22
 - Generic Receive Event, 6-22
 - Raise Document Delivery Event, 6-22
 - WF Send, 6-24
- Event subscriptions
 - configuring, 6-59
 - deleting, 6-59
- Execution engine
 - APIs, F-1
- Execution Engine
 - functions, 1-8
 - process flow, 4-1
 - required setup, 4-2
 - trading partner validation for inbound messages, 4-12
 - trading partner validation for outbound messages, 4-16
- Execution Engine Processing Sequence, 2-66

F

- FND Lookup, 11-3
- FND Message ID, 11-3
- Forecast, 10-12, 10-14
- Forms

- Define Lookup Values, 3-17
- Define Transactions, 3-11
- Define XML Standards, 3-10
- Hub Definitions, 3-8
- Standard Code Conversion, 3-47

- Trading Partner Code Conversion, 3-48
- Trading Partner Setup, 3-19
- Functions (Workflow)
 - XML Gateway Error Processing Item Type, 6-41
 - ECX Reprocess Inbound, 6-41
 - ECX Resend Outbound Message, 6-42
 - ECX Timeout Value, 6-42
 - Error Still Active, 6-42
 - Get ECX In Error Details, 6-42
 - Get ECX Out Error Details, 6-43
 - Get System Administrator Role, 6-43
 - Get Trading Partner Role, 6-43
 - Get Transaction Owner Role, 6-44
 - XML Gateway Standard Item Type, 6-8
 - Consume XML Document, 6-10
 - Generate Trading Partner XML Document, 6-13
 - Generate XML Document, 6-11
 - Send Document, 6-16
 - Transaction Delivery Required, 6-18
- FYI Delivery Error Process, 6-36

G

- General processing errors, I-34
- Generate Trading Partner XML Document, 6-13
- Generate XML Document Function, 6-11
- Generic Receive Event, 6-22
- Get ECX In Error Details Function, 6-42
- Get ECX Out Error Details Function, 6-43
- Get System Administrator Role Function, 6-43
- Get Trading Partner Role Function, 6-43
- Get Transaction Owner Role Function, 6-44

H

- HTTP status codes, 7-35
- Hub Definitions Form, 3-8
- Hub Entity Code, 3-9
- Hub Errors, I-25

I

- Implementation Checklist, 3-1
- Implementing Notification Processing for a New Collaboration, 11-17
- Inbound Message Queue, 5-1, 5-3

- inbound messages
 - required communications data, 3-31
- Inbound processing errors, I-33
- Inbound Trading Partner Message, 6-48
- Inbound Transaction Queue, 5-2, 5-3
- Internal Control Number, 11-8
- inventory report, 10-3
- IP_08, 11-22
- item master, 10-3
- Item Type
 - components of, 6-4
 - identifying seeded, 6-59
- Item Types
 - E-Business Suite Application Module-Specific Item Type, 6-6
 - XML Gateway Error Processing Item Type, 6-5
 - XML Gateway Standard Item Type, 6-5, 6-7

L

- Level Mapping Guidelines
 - collapsing levels, 2-58
 - discontinuous nodes, 2-60, A-19
 - expanding levels, 2-59
 - OAG DTDs, 2-58
- Level Mapping Tab, 2-57
- Lookup Types
 - XML Gateway Error Processing Item Type
 - ECX Inbound Error Actions, 6-51
 - ECX Outbound Error Actions, 6-52
 - ECX Outbound Error Types, 6-52
 - XML Gateway Standard Item Type
 - send mode, 6-26
- Lookup Values
 - defining, 3-17

M

- Map Action Editor, 2-67
 - action level and action stage combinations, 2-69
 - invoking, 2-67
- Map analysis
 - compare database views to DTD, A-11
 - supported actions, C-1
- Map analysis guidelines
 - comparing database views to DTD, A-3
 - inbound message checklist, A-9

- Map Analysis Guidelines
 - inbound messages, A-9
 - outbound messages, A-1
- Map Analysis Guidelines for outbound messages, A-1
- Map Analysis overview, A-1
- Maps
 - downloading, 2-113
- Menus
 - Message Designer
 - help menu, 2-3
- Menus
 - Message Designer, 2-2
- Menus, Message Designer
 - file menu, 2-2
 - Message Designer
 - File Menu, properties option, 2-5
 - view menu, 2-3
- Message Correlation Error, 11-22
- Message delivery callback, 6-68
 - how other messaging systems use, 6-68
- Message Delivery Callback, 6-66
 - block mode, 6-67
- Message Delivery Error (FYI), 6-40
- Message Delivery Error Event, 6-45
- Message Designer
 - APIs, F-4
- Message Envelope, 4-8
- Message Format, 5-4
- message identification number, 11-3
- Message map and DTD errors, I-29
- Message Maps
 - loading, 2-111
- Message Payloads, 11-2, 11-3, J-1
- messages, 11-2
 - development guidelines inbound, 6-65
 - inbound
 - required communications data, 3-31
 - outbound
 - required communications data, 3-30
- Messages
 - development guidelines for outbound, 6-64
 - XML Gateway Error Processing Item Type, 6-46
 - ECX Event Message (FYI), 6-47
 - ECX External Event Message (FYI), 6-47
 - Inbound Trading Partner Message, 6-48

- message template attributes, 6-48
- Outbound Trading Partner Message, 6-48
- Multiple Organizations, 3-23

N

- Naming conventions, D-1
- Naming Conventions
 - transaction type and transaction subtype, 3-14
- Non-OTA servers
 - connecting to, 7-26
- notification, 10-5
- Notification Action Definitions, 10-26, 10-29
- Notification Actions, 10-4
- Notification Code
 - IP-05, 10-6
 - SO_IN03, 10-6
- Notification Definition, 10-23
- Notifications, 10-4
 - XML Gateway Error Processing Item Type, 6-39
 - ECX Event Notification (FYI), 6-40
 - ECX External Event Notification, 6-40
 - Message Delivery Error (FYI), 6-40
 - Trading Partner Inbound Error Notification, 6-40
 - Trading Partner Outbound Error Notification, 6-40
 - Transaction Owner Inbound Error Notification, 6-40
- Notification Types, 10-4
- Notify Administrator, 11-21
 - CLN_ADMINISTRATOR, 11-21
 - default notification action, 11-21
 - E-Mail Format, 11-21
 - User/Roles, 11-21
- Notify Trading Partner, 11-21
 - E-Mail Format, 11-22
- Noun
 - setting in OAG standards, 3-17

O

- OAG, 1-1
 - recommending changes, A-20
- OAG conversions table, 2-84
- OAG message, 11-2

- oracle.apps.cln.ch.collaboration.addmessage
 - Collaboration Detail ID, 11-16
 - Event Parameters, 11-16
 - key parameters, 11-16
- oracle.apps.cln.ch.collaboration.create
 - Document Creation Date, 11-9
 - Document Number, 11-8
 - Document Owner, 11-9
 - Document Release Number, 11-8
 - Document Revision Date, 11-9
 - Document Revision Number, 11-8
 - Event Parameters, 11-9
 - forecast number, 11-8
 - internal key, 11-9
 - key parameters, 11-9
 - Organization ID, 11-8
 - Partner Document Number, 11-9
- oracle.apps.cln.ch.collaboration.update
 - Collaboration Detail, 11-12
 - Disposition, 11-12
 - Doc_Status, 11-12
 - Document Creation Date, 11-12
 - Document Number, 11-12
 - Document Owner, 11-12
 - Document Release Number, 11-12
 - Document Revision Date, 11-12
 - Document Revision Number, 11-12
 - Event Parameters, 11-12
 - FND message ID, 11-12
 - key parameters, 11-13
 - MSG_Text, 11-12
 - Organization ID, 11-12
 - Partner Document Number, 11-12
- Oracle Applications Manager, 11-20
- Oracle Business Event System, 11-22
- Oracle E-Business Suite AQ, 10-3
- Oracle Purchasing, 11-3
- Oracle Transport Agent
 - Certificate-Based Authentication Methods, 7-10
- Oracle Transport Agent (OTA), 7-1
- Oracle XML Gateway, 11-2
- Oracle XML Gateway Transactions Setup, 10-14, 10-15, 10-17, 10-17
- order taker, 11-9
- OTA
 - attachments, 2-117

- code connection samples, 7-26
 - connection over HTTP, 7-28
 - connection over HTTPS, 7-32
- troubleshooting, 7-35
- OTA message propagation flow, 7-2
- OTA Protocol, 7-2
- Outbound Message Queue, 5-1, 5-2
- outbound messages
 - required communications data, 3-30
- Outbound Trading Partner Message, 6-48
- outbound Web services
 - Design Time, 8-66
- Outbound Web services
 - Run Time, 8-85
 - test process, 8-87
- overview, 1-1
- owner of the document, 11-9

P

- parameters, 11-7
- Pass-Through Messages
 - execution engine processing of, 4-13
 - how to map, 2-110
- payloads, 11-2
- PL/SQL, 11-21
- planner, 11-9
- Post message
 - OTA, 7-3
- preconfigured notifications, 11-20
- predefined notification, 10-4
- preseeded notifications, 10-4
- PROCESS_PO, 11-2
- Process_PO message, 10-3
- Processes
 - XML Gateway Error Processing Item Type, 6-28
 - XML Gateway Standard Item Type, 6-7
- Profile Options, 3-3
 - ECX: Enable User Check for Trading Partner, 3-4
 - ECX: Log File Path, 3-3
 - ECX: Maximum XML Size, 3-4
 - ECX: OAG_LOGICALID, 3-4
 - ECX: Server Time Zone, 3-4
 - ECX: System Administrator Email Address, 3-4

- ECX: XML Validate Flag, 3-4
- ECX: XSLT File Path, 3-4
- Maximum Reprocess Attempts For Errored Inbound Transaction, 3-5
- Notification Timeout in Minutes, 3-5
- Properties Menu Option, 2-5
- Protocol Type
 - and correlation ID, 3-26
 - in Trading Partner setup, 3-26
- Protocol types, 4-5
- purchase order, 10-3, 10-3, 10-7, 10-8
- Purchase Order, 11-2
- Purchase Order Approval, 11-3
- purchase order release, 11-9
- Purging transaction, 6-60

Q

- Queues
 - definition, 5-1
 - error, 5-4
 - inbound, 5-3
 - inbound message, 5-3
 - inbound transaction, 5-3
 - outbound, 5-2
 - outbound message, 5-2
 - used by XML Gateway, 5-1

R

- Raise Business Event, 11-22
 - ATTRIBUTE1, 11-22
 - ATTRIBUTE14, 11-22
 - collaboration ID, 11-22
 - description, 11-22
 - notification code, 11-22
 - PARAMETER1, 11-22, 11-22
 - PARAMETER14, 11-22, 11-22
 - Standard parameters, 11-22
 - wf_parameter_list_t, 11-22
- Raise Document Delivery Event, 6-22
- Raising Events
 - CLN Create Collaboration event, 11-6
 - Collaboration History Update event, 11-6
 - ECX Receive Workflow Process, 11-6
 - Initial Event, 11-5
 - PO Approval Workflow, 11-6
 - Raise XML PO event, 11-6

- Subsequent Events, 11-6
- Reading the Debug File, 11-19
 - CLN debug files, 11-19
- Receive Error Event, 6-46
- Receive Send Notification Event, 6-46
- Receive the inbound subscription error event, 6-46
- Receive the inbound subscription processing error event, 6-46
- Response message
 - OTA, 7-9
- Retry, 6-42
- Routing, 3-29
 - dynamic, 3-34
 - static, 3-32
 - static and dynamic, 3-31
- Rule Function, Workflow Default, 6-2
- Rule Function, XML Gateway, 6-2

S

- Send Document Function, 6-16
- Setting Up a Trading Partner for the Collaboration, 11-18
- Setting Up Collaboration Events, 11-5
- Setting Up Collaboration Events and Final Events, 11-5
- Setting Up Collaborations, 11-xviii
- Setting Up Customized Collaborations, 11-21
- Setting Up Final Collaboration Event
 - Collaboration Final Event Definition, 11-5
- Setting Up FND Lookup Codes and Messages, 11-3
 - Adding the FND Lookup Codes, 11-3
 - Adding the FND Messages, 11-4
- Setting Up New Collaborations, 11-1
- Setting Up Notification Processing for New Collaborations, 11-20
- Set up the collaboration events
 - Collaboration Event Definition, 11-5
- Set up the collaboration events and final events
 - Collaboration History, 11-5
- set up the collaborations, J-xviii
- Sizing and Performance Guidelines
 - Inbound Messages and Transactions, H-1
 - Outbound Messages and Transactions, H-4
- SOAP, 8-4

- SOAP servlet, 8-4
- Source Definition Overview, 2-43
- Source Definition Tab, 2-45
- Standard Code Conversion Form, 3-47
- standards used by XML Gateway, 1-1
- Start Workflow, 11-23
 - APPLICATION_ID, 11-23
 - APPLY_HOLD, 11-23
 - attributes, 11-23
 - availability of the workflow, 11-23
 - COLLABORATION_ID, 11-23
 - COLLABORATION_TYPE, 11-23
 - defining the notification action, 11-23
 - HEADER_DESCRIPTION, 11-23
 - initiate a predefined workflow, 11-23
 - IP_07, 11-23
 - NOTIFICATION_CODE, 11-23
 - NOTIFICATION_DESCRIPTION, 11-23
 - REFERENCE_ID, 11-23
 - STATUS, 11-23
 - TPA Identification error, 11-23
 - TRADING_PARTNER_ID, 11-23
 - TXN_HOLD, 11-23
 - Workflow Item Type, 11-23
 - Workflow Process, 11-23
- Static and Dynamic Routing, 3-31
- Static Routing, 3-32
- Status Monitor, 11-20
- Stylesheets
 - loading and deleting, 2-114
- Supported actions
 - map analysis, C-1
- System Profile Options, 3-3

T

- Target Definition, 2-50
- Target Definition Tab, 2-53
- Time zones
 - set up, 3-4
 - values for profile option, E-1
- Toolbar
 - Message Designer, 2-4
- Trace, 6-62
- trading partner, 10-2, 10-4, 10-5, 11-3
- Trading Partner Code Conversion Form, 3-48
- Trading Partner Errors, I-19

- Trading Partner Inbound Error Notification, 6-40
- Trading Partner Outbound Error Notification, 6-40
- Trading Partner Setup
 - API, F-29
- Trading Partner Setup Form, 3-19
 - data validation, 4-14
- Trading Partner User Security, 3-36
- Trading Partner Validation
 - inbound messages, 4-12
 - outbound messages, 4-16
- Transaction Delivery Required Function, 6-18
- Transaction Monitor, I-8
 - calling from other applications, I-10
 - Details page, I-13
- Transaction Owner Inbound Error Notification, 6-40
- Transaction status
 - monitoring, 6-62
- transaction type, 11-3
- Troubleshooting
 - API-level messages, I-36
 - engine-level error messages, I-16
 - engine-level errors
 - code conversion errors, I-26
 - Confirmation BoD errors, I-28
 - enqueue/dequeue errors, I-27
 - general processing errors, I-34
 - hub errors, I-25
 - inbound processing errors, I-33
 - message map and DTD errors, I-29
 - trading partner errors, I-19
 - UTL_FILE errors, I-18
 - Workflow Business Event System errors, I-28
 - HTTP Status Codes, 7-35
 - manual troubleshooting steps, I-15
 - running the ecxver.sql script, I-1
 - TLS issues, I-47
 - transaction monitor, I-8
 - version validation, I-44
- Troubleshooting and Debugging Collaboration History, 11-18
 - debugging a collaboration, 11-18
 - debugging messages, 11-18
 - debug the workflow, 11-18

U

- Understanding Collaboration History, 10-2
- Understanding XML Gateway Services
 - Generic XML Gateway Services, 8-15
 - Strongly-Typed XML Gateway Services, 8-11
- unique collaboration, 10-2
- Update Collaboration Event Definition, 10-13
- Update Collaboration Final Event Definition, 10-16
- Update Collaboration Property, 10-18, 10-19
- Update Notification Action Header, 10-29
- Update Notification Definition, 10-25
- Updating a Collaboration Event Definition, 10-14
- Updating a Collaboration Final Event Definition, 10-17
- Using Collaboration History Advanced Search, 10-8
- Using Collaboration History Simple Search, 10-7
- Using JMS Queues
 - Custom Inbound JMS Queues, 9-14
 - Custom JMS Queues, 9-6
 - Custom Outbound JMS Queues, 9-7
 - Integration Points, 9-2
 - JMS Integration, 9-3
 - Overview, 9-1
- Using the Collaboration Event Definitions Window, 10-12
- Using the Collaboration Event Details Window, 10-10
- Using the Collaboration Final Event Definitions Window, 10-15
- Using XML Gateway Inbound
 - Adding Assign Activities, 8-44
 - Adding Invoke Activities, 8-43
 - Creating a New BPEL Project, 8-35
 - Creating a Partner Link, 8-38
- Using XML Gateway Inbound by SOA Provider
 - Adding Partner Links for File Adapter, 8-39
- Using XML Gateway outbound design task
 - Add an Assign Activity, 8-83
 - Add an Invoke Activity, 8-82
 - Add a Partner Link for File Adapter, 8-78
 - Adding a Receive Activity, 8-76
 - create a new BPEL project, 8-68
 - create a Partner Link for AQ Adapter, 8-69

UTL_FILE errors, I-18

V

Validation

- trading partner for inbound messages, 4-12
- trading partner for outbound messages, 4-16

Verb

- setting in OAG standards, 3-17

Version validation, I-44

Viewing Action Definition Details, 10-32

Viewing the Collaboration Event Message, 10-11

Viewing the Collaboration Event XML Payload, 10-11

W

Web services, 8-1

- attachments, 8-4
- components, 8-3
- Design Time, 8-30
- example, 8-29
- Implementation Steps, 8-24
- inbound example, 8-30
- outbound example, 8-66
- Prerequisite files, 8-59
- process flow, 8-18
 - inbound, 8-18
 - outbound, 8-18
- Run Time, 8-48
- setup steps, 8-19
- SOAP servlet, 8-4
- WSDL, 8-3
- XML Gateway Web services, 8-5

WF_ERROR Queue, 5-2

WF Send Event, 6-24

Wizards

- data definition creation, 2-18
- map creation, 2-19

Workflow, 11-3, 11-21

Workflow Administrator, 11-20

Workflow Builder

- components of an item type, 6-4
- E-Business Suite Application Module-Specific Item Type, 6-6
- item types, 6-4
- XML Gateway Error Processing Item Type, 6-5
- XML Gateway Standard Item Type, 6-5, 6-7

Workflow Business Event System

- overview of integration, 6-1

Workflow Business Event System errors, I-28

Workflow Default Rule Function

- and Trading Partner validation for outbound messages, 4-16

Workflow Error Queue, 5-2, 5-4

Workflow Manager, 11-20

Workflow Processes

- monitoring, 6-62

workflows, 11-2

Working with Key Parameters, 11-7

- APPLICATION_ID, 11-8
- Collaboration Type, 11-8
- Create Collaboration Event, 11-8
- DOCUMENT_DIRECTION, 11-8
- Document Direction, 11-8
- Document Type, 11-8
- Internal Control Number, 11-8
- TRADING_PARTNER_ID, 11-8
- TRADING_PARTNER_SITE, 11-8
- TRADING_PARTNER_TYPE, 11-8
- UNIQUEID1-5, 11-8
- Update Collaboration Event, 11-8
- XML_INTERNAL_CONTROL_NUMBER, 11-7
- XML_INTERNAL_TXN_TYPE, 11-7
- XMLG_DOCUMENT_ID, 11-8
- XMLG_INTERNAL_TRANSACTION_SUBTYPE, 11-7
- XMLG_MESSAGE_ID, 11-7
- XML Gateway Message ID, 11-8

Working with the Collaboration History Events, 11-7

- Add message, 11-7
- CLN_CH_EVENT_SUBSCRIPTION_PKG.ADDMESSAGE_EVENT_SUB, 11-7
- CLN_CH_EVENT_SUBSCRIPTION_PKG.CREATE_EVENT_SUB, 11-7
- CLN_CH_EVENT_SUBSCRIPTION_PKG.UPDATE_EVENT_SUB, 11-7
- Collaboration History Standard Events item type, 11-7
- Create, 11-7
- Update, 11-7
- Workflow Builder, 11-7

write into workflow log tables, 11-20

WSDL, 8-3

schema structure type, 8-3

X

XML Fragment, 4-16

XML Gateway, 10-4

XML Gateway Error Processing Item Type, 6-5, 6-27

events, 6-45

Message Delivery Error, 6-45

Receive Error, 6-46

Receive Send Notification, 6-46

Receive the inbound subscription error, 6-46

Receive the inbound subscription processing error, 6-46

functions, 6-41

ECX Reprocess Inbound, 6-41

ECX Resend Outbound Message, 6-42

ECX Timeout Value, 6-42

Error Still Active, 6-42

Get ECX In Error Details, 6-42

Get ECX Out Error Details, 6-43

Get System Administrator Role, 6-43

Get Trading Partner Role, 6-43

Get Transaction Owner Role, 6-44

lookup types, 6-50

ECX Inbound Error Actions, 6-51

ECX Outbound Error Actions, 6-52

ECX Outbound Error Types, 6-52

messages, 6-46

ECX Event Message (FYI), 6-47

ECX External Event Message (FYI), 6-47

Inbound Trading Partner Message, 6-48

Outbound Trading Partner Message, 6-48

message template attributes, 6-48

notifications, 6-39

ECX Event Notification (FYI), 6-40

ECX External Event Notification, 6-40

Message Delivery Error (FYI), 6-40

Trading Partner Inbound Error Notification, 6-40

Trading Partner Outbound Error Notification, 6-40

Transaction Owner Inbound Error

Notification, 6-40

processes

Default Error Process, 6-37

ECX Engine Notification Process, 6-38

ECX Error Process for Timeout, 6-33

ECX Main Error Process, 6-30

ECX Main Inbound Error Process, 6-31

ECX Main Outbound Error Process, 6-34

Error Handling for Inbound Messages, 6-31

Error Handling for Outbound Messages, 6-34

FYI Delivery Error Process, 6-36

XML Gateway Rule Function

and Trading Partner validation for outbound messages, 4-16

XML Gateway services

Enabling steps through Service Provider, 8-25

SOA Header for XML Gateway Messages through Service Provider, 8-14

SOAP Header for Generic XML Gateway Messages, 8-16

XML Gateway Standard Item Type, 6-5, 6-7

events, 6-22

Generic Receive Event, 6-22

Raise Document Delivery Event, 6-22

WF Send, 6-24

functions, 6-8

lookup types, 6-24

send mode, 6-26

XML Gateway Web services

Enabling XML Gateway Web services, 8-5

Web Service Security, 8-7

XML message

reviewing outbound, 6-62

XML Payloads, 10-2

XSLT stylesheets

loading and deleting, 2-114